

Hieght and distance -- Gemini

```

import React, { useState, useMemo, useEffect } from 'react';

// --- INLINE CUSTOM SVG ICONS FOR CRISP UI ---
const RocketIcon = () => (
  <svg className="w-6 h-6 text-indigo-500" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M13 10V3L4 14h7v7l9-11h-7z" />
  </svg>
);

const CompassIcon = () => (
  <svg className="w-5 h-5 text-indigo-400 mr-2" fill="none" viewBox="0 0 24
24" stroke="currentColor">
    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 19l9 2-9-18-9 18 9-2zm0 0v-8" />
  </svg>
);

const CheckIcon = () => (
  <svg className="w-5 h-5 text-green-500 mr-1.5" fill="none" viewBox="0 0
24 24" stroke="currentColor">
    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={3}
d="M5 13l4 4L19 7" />
  </svg>
);

const StarIcon = ({ className = "w-5 h-5 text-yellow-400" }) => (
  <svg className={className} fill="currentColor" viewBox="0 0 20 20">
    <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0
00.95.69h3.462c.969 0 1.371 1.24.588 1.811-2.8 2.034a1 1 0 00-.364
1.118l1.07 3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175
0l-2.8 2.034c-.784.57-1.838-.197-1.539-1.118l1.07-3.292a1 1 0
00-.364-1.118L2.98 8.72c-.783-.57-.38-1.81.588-1.81h3.461a1 1 0
00.951-.69l1.07-3.292z" />
  </svg>
);

```

```
);
```

```
const RefreshIcon = () => (  
  <svg className="w-4 h-4 mr-1" fill="none" viewBox="0 0 24 24"  
  stroke="currentColor">  
    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}  
d="M4 4v5h.582m15.356 2A8.001 8.001 0 1121.21 8H17" />  
  </svg>  
);
```

```
const QuestionIcon = () => (  
  <svg className="w-5 h-5 text-indigo-500 mr-2" fill="none" viewBox="0 0 24  
24" stroke="currentColor">  
    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}  
d="M8.228 9c.549-1.165 2.03-2 3.772-2 2.21 0 4 1.343 4 3 0 1.4-1.278  
2.575-3.006 2.907-.542.104-.994.54-.994 1.093m0 3h.01M21 12a9 9 0 11-18 0  
9 9 0 0118 0z" />  
  </svg>  
);
```

```
// --- CHALLENGES LIST ---
```

```
const CHALLENGES = [  
  {  
    id: 0,  
    title: "Perfect Balance (45° Angle)",  
    instruction: "Move Sam until the angle of elevation to the star beacon  
is exactly 45°.",  
    targetDistance: 10.0,  
    targetAngle: 45,  
    hint: "Think: When the angle of elevation is 45°, what is the  
relationship between the height of the lighthouse and Sam's distance?  
(Hint:  $\tan(45^\circ) = 1$ )",  
    rewardXP: 100,  
  },  
  {  
    id: 1,
```

```

    title: "The Golden Horizon (30° Angle)",
    instruction: "Walk away from the lighthouse until the angle of
elevation shrinks down to exactly 30°.",
    targetDistance: 17.3,
    targetAngle: 30,
    hint: "To get a smaller angle, Sam needs to stand much further away.
Stop right around 17.3 meters!",
    rewardXP: 150,
  },
  {
    id: 2,
    title: "Up Close and Steep (60° Angle)",
    instruction: "Run closer to the lighthouse until Sam has to look up at
a steep 60° angle.",
    targetDistance: 5.8,
    targetAngle: 60,
    hint: "To get a steep angle, Sam must look almost straight up. Find the
sweet spot near 5.8 meters!",
    rewardXP: 200,
  }
];

// --- QUIZ QUESTIONS ---
const QUIZ_QUESTIONS = [
  {
    id: 1,
    question: "What is the line called that connects Sam's eyes directly to
the glowing beacon on top of the lighthouse?",
    options: [
      "Horizontal Line",
      "Line of Sight",
      "Vertical Pole Line",
      "Shadow Path"
    ],
    answerIndex: 1,
  },

```

explanation: "The direct straight line connecting an observer's eyes to the target object of sight is called the Line of Sight."

},

{

id: 2,

question: "Where is the Angle of Elevation always measured from?",

options: [

"The vertical lighthouse tower",

"The clouds in the sky",

"The flat horizontal ground level",

"The top of Sam's head vertically"

],

answerIndex: 2,

explanation: "The angle of elevation is the angle between the horizontal ground line (observer's eye-level reference) and the line of sight pointing upwards."

},

{

id: 3,

question: "As Sam walks further AWAY from the lighthouse, what happens to the Angle of Elevation?",

options: [

"It gets bigger (steeper)",

"It gets smaller (flatter)",

"It stays exactly the same",

"It instantly turns to 0°"

],

answerIndex: 1,

explanation: "As distance increases, the slope of look becomes gentler, making the angle of elevation smaller."

},

{

id: 4,

question: "If the lighthouse height is 10m, and Sam stands exactly 10m away, what will the angle of elevation be?",

options: [

```

    "30°",
    "60°",
    "45°",
    "90°"
  ],
  answerIndex: 2,
  explanation: "Since  $\tan(\text{Angle}) = \text{Height} / \text{Distance}$ , and  $\text{Height} / \text{Distance} = 10/10 = 1$ , the angle whose tangent is 1 is exactly  $45^\circ$ . A perfect right isosceles triangle!"
},
{
  id: 5,
  question: "Sam moves from 5 meters away to 20 meters away. His neck is likely to:",
  options: [
    "Hurt more because he has to tilt his head higher up",
    "Feel much more comfortable because the angle of elevation decreases",
    "Hurt the same since distance does not affect looking comfort"
  ],
  answerIndex: 1,
  explanation: "At 20 meters, the angle of elevation is much smaller (flatter), meaning Sam doesn't need to strain his neck looking up!"
}
];

```

```

export default function App() {
  const [activeTab, setActiveTab] = useState('simulator'); // 'simulator', 'guide', 'quiz'
  const [distance, setDistance] = useState(14.0); // ranges from 2.0m to 20.0m
  const [scoreXP, setScoreXP] = useState(50);
  const [currentChallenge, setCurrentChallenge] = useState(0);
  const [completedChallenges, setCompletedChallenges] = useState([]);
  const [showConfetti, setShowConfetti] = useState(false);

```

```

    const [message, setMessage] = useState("Drag the slider to move Sam and
look at the star!");

// Quiz State
const [selectedAnswers, setSelectedAnswers] = useState({});
const [submittedQuestions, setSubmittedQuestions] = useState({});
const [quizScore, setQuizScore] = useState(0);

const LIGHTHOUSE_HEIGHT = 10; // 10 meters constant

// Mathematical Trigonometric Computations
const { angleDeg, lineOfSightLength } = useMemo(() => {
    const radians = Math.atan(LIGHTHOUSE_HEIGHT / distance);
    const degrees = radians * (180 / Math.PI);
    const hypotenuse = Math.sqrt(distance * distance + LIGHTHOUSE_HEIGHT *
LIGHTHOUSE_HEIGHT);
    return {
        angleDeg: Math.round(degrees),
        lineOfSightLength: hypotenuse.toFixed(1)
    };
}, [distance]);

// Scaled dimensions for beautiful adaptive SVG viewport
// Lighthouse base at x = 400, y = 260
const BASE_X = 400;
const BASE_Y = 260;
const VISUAL_SCALE = 15; // px per meter

const observerX = BASE_X - distance * VISUAL_SCALE;
const topY = BASE_Y - LIGHTHOUSE_HEIGHT * VISUAL_SCALE;

// Track slider & automatically validate challenges
useEffect(() => {
    const target = CHALLENGES[currentChallenge];
    const isClose = Math.abs(distance - target.targetDistance) <= 0.2; //
strict within 0.2m tolerance

```

```

    if (isClose && !completedChallenges.includes(currentChallenge)) {
      setCompletedChallenges(prev => [...prev, currentChallenge]);
      setScoreXP(prev => prev + target.rewardXP);
      setShowConfetti(true);
      setMessage(`🎉 Splendid! You solved "${target.title}" and gained
    ${target.rewardXP} XP!`);

      // Auto-clear confetti state after a brief moment
      setTimeout(() => {
        setShowConfetti(false);
      }, 4000);
    }
  }, [distance, currentChallenge]);

// Handle quiz choices
const handleSelectOption = (questionId, optionIndex) => {
  if (submittedQuestions[questionId]) return; // locked once submitted
  setSelectedAnswers(prev => ({ ...prev, [questionId]: optionIndex }));
};

const handleQuizSubmit = (q) => {
  if (selectedAnswers[q.id] === undefined || submittedQuestions[q.id])
    return;

  const isCorrect = selectedAnswers[q.id] === q.answerIndex;
  setSubmittedQuestions(prev => ({ ...prev, [q.id]: true }));

  if (isCorrect) {
    setQuizScore(prev => prev + 1);
    setScoreXP(prev => prev + 50);
  }
};

const handleNextChallenge = () => {
  setCurrentChallenge(prev => (prev + 1) % CHALLENGES.length);
};

```



```

    // Set slider distance close to make them solve it
    if (currentChallenge === 0) setDistance(15.0);
    if (currentChallenge === 1) setDistance(4.0);
    if (currentChallenge === 2) setDistance(10.0);
  };

  const handleResetChallenge = () => {
    setDistance(14.0);
    setCompletedChallenges([]);
    setScoreXP(50);
    setCurrentChallenge(0);
    setSelectedAnswers({});
    setSubmittedQuestions({});
    setQuizScore(0);
    setMessage("Simulator successfully rebooted! Let's explore heights &
distances.");
  };

  return (
    <div className="min-h-screen bg-slate-50 text-slate-800 font-sans
antialiased pb-12">
      {/* Top Header Panel */}
      <header className="bg-gradient-to-r from-indigo-700 via-indigo-800
to-slate-900 text-white shadow-xl px-4 py-4 md:px-8">
        <div className="max-w-6xl mx-auto flex flex-col sm:flex-row
justify-between items-center gap-4">
          <div className="flex items-center gap-3">
            <div className="p-2 bg-indigo-600/50 rounded-lg
backdrop-blur-md border border-indigo-400/20">
              <CompassIcon />
            </div>
            <div>
              <h1 className="text-xl md:text-2xl font-bold
tracking-tight">The Star Gazer</h1>
              <p className="text-xs text-indigo-200">Heights & Distances •
Class 8 NCERT Mathematics</p>

```

```

    </div>
  </div>

  <div className="flex items-center gap-4">
    { /* Gamified Score Badge */ }
    <div className="bg-amber-400 text-amber-950 font-bold px-4
py-1.5 rounded-full flex items-center gap-1.5 shadow-md">
      <StarIcon className="w-5 h-5 text-amber-900
animate-spin-slow" />
      <span>{scoreXP} XP</span>
    </div>

    <button
      onClick={handleResetChallenge}
      className="px-3 py-1.5 bg-slate-800 hover:bg-slate-700
text-xs text-slate-300 hover:text-white rounded-lg transition-all flex
items-center gap-1 border border-slate-700"
      title="Reset all simulator state"
    >
      <RefreshIcon /> Reset All
    </button>
  </div>
</div>
</header>

{ /* Main Container */ }
<main className="max-w-6xl mx-auto px-4 mt-6">

  { /* Navigation Tabs */ }
  <div className="flex bg-slate-200 p-1 rounded-xl mb-6 max-w-lg
mx-auto shadow-inner">
    <button
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 py-2.5 px-4 text-sm font-semibold rounded-lg
transition-all flex items-center justify-center gap-2 ${
        activeTab === 'simulator'

```

```

      ? 'bg-white text-indigo-700 shadow-md'
      : 'text-slate-600 hover:text-slate-900 hover:bg-white/50'
    }`}
  >
   Simulator & Tasks
</button>
<button
  onClick={() => setActiveTab('guide')}
  className={`flex-1 py-2.5 px-4 text-sm font-semibold rounded-lg
transition-all flex items-center justify-center gap-2 ${
    activeTab === 'guide'
      ? 'bg-white text-indigo-700 shadow-md'
      : 'text-slate-600 hover:text-slate-900 hover:bg-white/50'
    }`}
  >
   Interactive Guide
</button>
<button
  onClick={() => setActiveTab('quiz')}
  className={`flex-1 py-2.5 px-4 text-sm font-semibold rounded-lg
transition-all flex items-center justify-center gap-2 ${
    activeTab === 'quiz'
      ? 'bg-white text-indigo-700 shadow-md'
      : 'text-slate-600 hover:text-slate-900 hover:bg-white/50'
    }`}
  >
   Brain Quiz
</button>
</div>

{/* Tab Content: Simulator */}
{activeTab === 'simulator' && (
  <div className="grid grid-cols-1 lg:grid-cols-12 gap-6
items-start">

    {/* Visual Canvas (Column Span 8) */}

```

```

<div className="lg:col-span-8 bg-slate-900 rounded-3xl p-4
md:p-6 shadow-2xl border-4 border-slate-800 relative overflow-hidden flex
flex-col">

  {/* Starry Night Sky Background overlay */}
  <div className="absolute inset-0 bg-gradient-to-b
from-indigo-950 via-slate-900 to-indigo-900 opacity-90
pointer-events-none" />

  {/* Twinkling Stars effect in background */}
  <div className="absolute top-8 left-12 w-1.5 h-1.5 bg-white
rounded-full opacity-60 animate-ping" />
  <div className="absolute top-24 left-1/3 w-1 h-1 bg-white
rounded-full opacity-40" />
  <div className="absolute top-16 right-1/4 w-2 h-2
bg-yellow-200 rounded-full opacity-70 animate-pulse" />
  <div className="absolute top-36 right-12 w-1 h-1 bg-white
rounded-full opacity-50" />
  <div className="absolute top-48 left-1/4 w-1.5 h-1.5
bg-indigo-300 rounded-full opacity-80" />

  {/* Simulation Header stats overlay */}
  <div className="relative z-10 flex justify-between
items-center bg-slate-950/70 p-3 rounded-xl border border-white/10
backdrop-blur-md mb-4 text-white">
    <div>
      <span className="text-xs text-slate-400 block
font-semibold uppercase tracking-wider">Observer Distance</span>
      <span className="text-xl md:text-2xl font-bold
text-sky-400">{distance.toFixed(1)} <span className="text-sm font-normal
text-slate-300">meters</span></span>
    </div>
    <div className="text-center">
      <span className="text-xs text-slate-400 block
font-semibold uppercase tracking-wider">Angle of Elevation</span>

```

```

        <span className="text-xl md:text-2xl font-bold
text-amber-400 animate-pulse">{angleDeg}°</span>
    </div>
    <div className="text-right">
        <span className="text-xs text-slate-400 block
font-semibold uppercase tracking-wider">Line of Sight</span>
        <span className="text-xl md:text-2xl font-bold
text-pink-400">{lineOfSightLength} <span className="text-sm font-normal
text-slate-300">m</span></span></div>
    </div>

    { /* Main SVG Simulation Canvas */ }
    <div className="relative z-10 bg-slate-950/30 rounded-2xl p-1
md:p-4 flex justify-center items-center">
        <svg
            width="100%"
            height="290"
            viewBox="0 0 460 290"
            className="overflow-visible"
        >
            { /* Grid system helper line for Class 8 perspective */ }
            <line x1="10" y1={BASE_Y} x2="450" y2={BASE_Y}
stroke="#1e293b" strokeWidth="1" strokeDasharray="4,4" />

            { /* Dynamic Sea Ground / Sandy Shore bottom line */ }
            <rect x="10" y={BASE_Y} width="440" height="30" rx="4"
fill="url(#groundGradient)" />

            { /* Gradient Definitions */ }
            <defs>
                <linearGradient id="groundGradient" x1="0%" y1="0%"
x2="0%" y2="100%">
                    <stop offset="0%" stopColor="#334155" />
                    <stop offset="100%" stopColor="#0f172a" />
                </linearGradient>
            </defs>
        </svg>
    </div>

```

```

        <linearGradient id="beamGradient" x1="0%" y1="100%"
x2="100%" y2="0%">
            <stop offset="0%" stopColor="#f43f5e"
stopOpacity="0.8" />
            <stop offset="100%" stopColor="#eab308"
stopOpacity="0.2" />
        </linearGradient>
        <linearGradient id="lighthouseGrad" x1="0%" y1="0%"
x2="100%" y2="0%">
            <stop offset="0%" stopColor="#cbd5e1" />
            <stop offset="50%" stopColor="#94a3b8" />
            <stop offset="100%" stopColor="#475569" />
        </linearGradient>
    </defs>

    { /* The Lighthouse Tower */ }
    <g transform={`translate(${BASE_X - 15}, ${topY - 12})`}>
        { /* Tower base blocks */ }
        <rect x="5" y="112" width="20" height="12"
fill="#1e293b" rx="2" />
        { /* Main tapered structure */ }
        <polygon points="8,112 22,112 25,25 5,25"
fill="url(#lighthouseGrad)" />
        { /* Red warning spiral lines wrapping the tower */ }
        <path d="M 6.5,100 L 23.5,88" stroke="#ef4444"
strokeWidth="4.5" />
        <path d="M 7.5,70 L 22.5,58" stroke="#ef4444"
strokeWidth="4.5" />
        <path d="M 8.5,40 L 21.5,28" stroke="#ef4444"
strokeWidth="4.5" />

        { /* Balcony deck */ }
        <rect x="2" y="18" width="26" height="7" fill="#1e293b"
rx="1.5" />

        { /* Light Cabin Room */ }

```

```

        <rect x="7" y="4" width="16" height="14" fill="#fef08a"
opacity="0.9" rx="1" />
        { /* Cabin Columns */}
        <line x1="10" y1="4" x2="10" y2="18" stroke="#1e293b"
strokeWidth="1.5" />
        <line x1="20" y1="4" x2="20" y2="18" stroke="#1e293b"
strokeWidth="1.5" />

        { /* Roof Dome */}
        <path d="M 7,4 Q 15,-6 23,4 Z" fill="#b91c1c" />

        { /* Flashing Yellow Star Beacon atop the dome */}
        <g transform="translate(15, -12)">
            <circle cx="0" cy="0" r="16" fill="#fef08a"
className="animate-ping opacity-25" />
            <circle cx="0" cy="0" r="8" fill="#eab308" />
            <polygon points="0,-7 2,-2 7,-2 3,1 5,6 0,3 -5,6 -3,1
-7,-2 -2,-2" fill="#ffffff" />
        </g>
    </g>

    { /* Right-angled Triangle Guide Lines */}
    { /* Vertical (Lighthouse Height Indicator) */}
    <line
        x1={BASE_X}
        y1={BASE_Y}
        x2={BASE_X}
        y2={topY}
        stroke="#10b981"
        strokeWidth="2.5"
        strokeDasharray="4,4"
    />

    { /* Horizontal (Observer Distance Base) */}
    <line
        x1={observerX}

```

```

        y1={BASE_Y}
        x2={BASE_X}
        y2={BASE_Y}
        stroke="#38bdf8"
        strokeWidth="2.5"
    />

    {/* Right Angle Corner Square symbol (90 degrees marker)
    */}

    <path d={`M ${BASE_X - 10} ${BASE_Y} L ${BASE_X - 10}
    ${BASE_Y - 10} L ${BASE_X} ${BASE_Y - 10}`} fill="none" stroke="#10b981"
    strokeWidth="1.5" />

    {/* Hypotenuse - Star Laser Beam (Line of Sight) */}
    <line
        x1={observerX}
        y1={BASE_Y - 12}
        x2={BASE_X}
        y2={topY}
        stroke="url(#beamGradient)"
        strokeWidth="3"
        className="transition-all"
    />

    {/* Dynamic Angle Arc Overlay */}
    {distance > 3 && (
        <g>
            <path
                d={`M ${observerX + 24} ${BASE_Y} A 24 24 0 0 0
                ${observerX + (24 * Math.cos(angleDeg * Math.PI / 180))} ${BASE_Y - (24 *
                Math.sin(angleDeg * Math.PI / 180))}`}
                fill="none"
                stroke="#fbbf24"
                strokeWidth="2.5"
            />
            {/* Interactive Angle Tag above the arc */}

```



```

        <g transform={`translate(${observerX + 32}, ${BASE_Y
- 14}})`}>

            <rect x="-1" y="-12" width="34" height="17" rx="4"
fill="#f59e0b" className="shadow-lg" />

            <text x="16" y="1" fill="ffffff" fontSize="10"
fontWeight="bold" textAnchor="middle">{angleDeg}°</text>

        </g>
    </g>
})

{/* Cute Observer Avatar (Sam) */}
<g transform={`translate(${observerX - 10}, ${BASE_Y -
26}})`}>

    {/* Head / Helmet */}
    <circle cx="10" cy="8" r="7" fill="#38bdf8" />
    {/* Space helmet visor */}
    <path d="M 8 5 A 4 4 0 0 1 15 8 Z" fill="#1e293b" />
    {/* Face looking up (smiley inside visor) */}
    <circle cx="11" cy="7" r="1" fill="#38bdf8" />
    {/* Space Suit Body */}
    <rect x="4" y="15" width="12" height="11" rx="4"
fill="ffffff" />

    {/* Laser emitter device in hand */}
    <circle cx="14" cy="14" r="3" fill="#f43f5e" />
    <line x1="14" y1="14" x2="16" y2="12" stroke="ffffff"
strokeWidth="1.5" />

    {/* Footwear */}
    <rect x="3" y="25" width="5" height="2" fill="#475569"
rx="1" />

    <rect x="12" y="25" width="5" height="2" fill="#475569"
rx="1" />

</g>

{/* Height dimension label */}
<g transform={`translate(${BASE_X + 24}, ${BASE_Y -
(LIGHTHOUSE_HEIGHT * VISUAL_SCALE) / 2}})`}>

```

```

        <text x="0" y="0" fill="#10b981" fontSize="12"
fontWeight="bold" textAnchor="middle" transform="rotate(90)">Height = 10m
(Fixed)</text>

</g>

```

```

        { /* Ground base distance bracket dimension */ }
        <path d={`M ${observerX} ${BASE_Y + 12} L ${observerX}
${BASE_Y + 18} L ${BASE_X} ${BASE_Y + 18} L ${BASE_X} ${BASE_Y + 12}`}
fill="none" stroke="#38bdf8" strokeWidth="1.5" />
        <text x={(observerX + BASE_X) / 2} y={BASE_Y + 32}
fill="#38bdf8" fontSize="12" fontWeight="semibold" textAnchor="middle">
            Distance = {distance.toFixed(1)}m
        </text>
    </svg>
</div>

```

```

        { /* Dynamic feedback banner */ }
        <div className="relative z-10 mt-4 bg-slate-950/60
text-slate-200 py-2.5 px-4 rounded-xl text-center border border-white/5
text-sm">
            💡 <span className="font-semibold">{message}</span>
        </div>

```

```

        { /* Beautiful Slider Controls Area */ }
        <div className="relative z-10 mt-6 bg-slate-950/90 p-5
rounded-2xl border border-white/10 flex flex-col md:flex-row items-center
gap-6">
            <div className="w-full flex-1">
                <div className="flex justify-between items-center
mb-2.5">
                    <span className="text-sm font-bold text-slate-300 flex
items-center gap-1.5">
                        🏃 Walk Sam Along Shoreline:
                    </span>
                    <span className="text-xs bg-slate-800 text-indigo-400
px-2.5 py-0.5 rounded-full font-mono">

```

```

        Min: 2.0m | Max: 20.0m
      </span>
    </div>

    <input
      type="range"
      min="2.0"
      max="20.0"
      step="0.1"
      value={distance}
      onChange={ (e) =>
setDistance(parseFloat(e.target.value)) }
      className="w-full accent-indigo-500 bg-slate-800 h-2.5
rounded-lg cursor-ew-resize focus:outline-none focus:ring-2
focus:ring-indigo-500/50"
    />

    { /* Stepper helpers to make fine tuning easy */ }
    <div className="flex justify-between mt-2.5">
      <button
        onClick={ () => setDistance(prev => Math.max(2.0,
Number((prev - 0.1).toFixed(1)))) }
        className="px-2 py-1 bg-slate-800 hover:bg-slate-700
text-slate-300 rounded text-xs transition"
      >
        ◀ Fine Move Closer
      </button>
      <span className="text-xs text-slate-400 font-mono
italic">Adjust carefully to hit the exact target!</span>
      <button
        onClick={ () => setDistance(prev => Math.min(20.0,
Number((prev + 0.1).toFixed(1)))) }
        className="px-2 py-1 bg-slate-800 hover:bg-slate-700
text-slate-300 rounded text-xs transition"
      >
        Fine Move Away ▶
    </div>

```

```

        </button>
      </div>
    </div>
  </div>
</div>

{/* Guided Gamified Task Desk (Column Span 4) */}
<div className="lg:col-span-4 flex flex-col gap-6">

  {/* Gamified Challenge Panel */}
  <div className="bg-white rounded-3xl p-6 shadow-xl border
border-slate-200">
    <div className="flex items-center gap-2 mb-4">
      <div className="p-1.5 bg-indigo-50 rounded-lg">
        <RocketIcon />
      </div>
      <h3 className="text-lg font-bold text-slate-800">Mission
Headquarters</h3>
    </div>

    {/* Star-studded XP status */}
    <div className="flex justify-between items-center
bg-indigo-50/50 p-3 rounded-2xl border border-indigo-100 mb-4">
      <span className="text-xs text-indigo-900 font-semibold
uppercase tracking-wider">Missions Cleared</span>
      <span className="text-sm font-bold text-indigo-700">
        {completedChallenges.length} of {CHALLENGES.length}
      </span>
    </div>

    {/* Selected Mission Card */}
    <div className="bg-slate-50 rounded-2xl p-4 border
border-slate-200">
      <div className="flex justify-between items-start gap-1">
        <span className="text-xs font-bold text-indigo-600
bg-indigo-100/80 px-2.5 py-0.5 rounded-full uppercase">

```

```

        Current Task
    </span>
    <span className="text-xs font-bold text-amber-600 flex
items-center gap-0.5">
        <StarIcon className="w-4 h-4 text-amber-500" />
        +{CHALLENGES[currentChallenge].rewardXP} XP
    </span>
</div>

<h4 className="font-bold text-slate-800 mt-2.5
text-base">
    {CHALLENGES[currentChallenge].title}
</h4>
<p className="text-xs text-slate-600 mt-1.5
leading-relaxed">
    {CHALLENGES[currentChallenge].instruction}
</p>

    {/* Highlight dynamic metrics relative to goal */}
    <div className="mt-4 bg-white rounded-xl p-3 border
border-slate-200 space-y-1 text-xs">
        <div className="flex justify-between">
            <span className="text-slate-500">Your current
Distance:</span>
            <span className="font-semibold
text-slate-800">{distance.toFixed(1)}m</span>
        </div>
        <div className="flex justify-between">
            <span className="text-slate-500">Target
Distance:</span>
            <span className="font-semibold
text-slate-800">{CHALLENGES[currentChallenge].targetDistance}m</span>
        </div>
        <div className="flex justify-between">
            <span className="text-slate-500">Target Angle:</span>

```

```

        <span className="font-semibold text-indigo-600
font-mono font-bold">{CHALLENGES[currentChallenge].targetAngle}°</span>
    </div>
</div>

    { /* Target Success Celebration */ }
    { completedChallenges.includes(currentChallenge) ? (
        <div className="mt-4 bg-green-50 border
border-green-200 text-green-800 p-3 rounded-xl flex items-center gap-2">
            <span className="bg-green-100 p-1
rounded-full"><CheckIcon /></span>
            <div className="text-xs">
                <p className="font-bold">Challenge Completed!
🎉</p>
                <p className="text-slate-500">Excellent spatial
visualization.</p>
            </div>
        </div>
    ) : (
        <div className="mt-3 bg-blue-50/70 border
border-blue-100 text-slate-600 p-3 rounded-xl text-xs leading-relaxed">
            💡 <strong>Hint:</strong>
            {CHALLENGES[currentChallenge].hint}
        </div>
    ) }
</div>

    { /* Next Task Controls */ }
    <div className="mt-4 flex gap-2">
        <button
            onClick={handleNextChallenge}
            className="flex-1 py-3 px-4 bg-indigo-600
hover:bg-indigo-700 text-white font-bold rounded-xl shadow-lg
shadow-indigo-200 transition text-sm flex items-center justify-center
gap-2"
        >

```

```

        <span>Next Mission</span>
        <span className="text-xs opacity-80">▶</span>
    </button>
</div>
</div>

{/* Discovery Card summary */}
<div className="bg-slate-900 text-slate-300 rounded-3xl p-5 shadow-xl relative overflow-hidden">
    <div className="absolute top-0 right-0 w-24 h-24 bg-indigo-500/10 rounded-full blur-2xl" />
    <h3 className="font-bold text-white mb-2 text-sm uppercase tracking-wider text-indigo-400">🔍 My Math Logs</h3>
    <p className="text-xs leading-relaxed text-slate-300">
        Observe that as you drag Sam closer to the lighthouse (Distance decreases), his line of sight gets steeper and the Angle of Elevation <strong className="text-yellow-400">increases</strong>!
    </p>
    <div className="mt-3 text-xs bg-slate-950 p-2.5 rounded-lg border border-slate-800 font-mono">
        Distance ↓ &nbsp;  ⇒&nbsp;   Angle of Elevation ↑
    </div>
</div>
</div>
</div>
)}

{/* Tab Content: Educational Guide */}
{activeTab === 'guide' && (
    <div className="bg-white rounded-3xl p-6 md:p-8 shadow-xl border border-slate-200 max-w-4xl mx-auto">

        {/* Concept explanation block */}
        <div className="border-b border-slate-100 pb-6">
            <span className="text-xs font-extrabold text-indigo-600 bg-indigo-100 px-3 py-1 rounded-full uppercase tracking-wider">

```

```

        Concept Masterclass
    </span>
    <h2 className="text-2xl font-bold text-slate-900 mt-3">Angles
of Elevation Demystified</h2>
    <p className="text-slate-600 mt-2 text-sm leading-relaxed">
        Heights and Distances is a highly useful branch of
mathematics that helps us measure tall structures like mountains, flags,
pyramids, or skyscrapers without using physical tapes!
    </p>
</div>

{/* Core terminologies grid */}
<div className="grid grid-cols-1 md:grid-cols-3 gap-6 my-8">

    <div className="bg-slate-50 p-5 rounded-2xl border
border-slate-200">
        <span className="text-2xl mb-2 block">👁️</span>
        <h3 className="font-bold text-slate-800
text-base">Horizontal Line</h3>
        <p className="text-xs text-slate-600 mt-1 leading-relaxed">
            The flat reference line parallel to the ground that
starts from the observer's eyes. It acts as our base (0°).
        </p>
    </div>

    <div className="bg-slate-50 p-5 rounded-2xl border
border-slate-200">
        <span className="text-2xl mb-2 block">📏</span>
        <h3 className="font-bold text-slate-800 text-base">Line of
Sight</h3>
        <p className="text-xs text-slate-600 mt-1 leading-relaxed">
            The imaginary straight diagonal line drawn from the
observer's eyes directly to the point on the object we are observing.
        </p>
    </div>

```



```

        <div className="bg-slate-50 p-5 rounded-2xl border
border-slate-200">
            <span className="text-2xl mb-2 block">

```

```

        { /* Visual Formula Guide */ }
        <div className="bg-indigo-950 text-indigo-100 rounded-3xl p-6
shadow-md my-6">
            <h3 className="font-bold text-white text-lg mb-3">The Magic
Math Formula</h3>
            <p className="text-xs text-indigo-200 leading-relaxed mb-5">
                Since towers, poles, and lighthouses stand straight
perpendicular to the ground, they form a Right-Angled Triangle. We can
calculate distances using simple ratios!
            </p>

```

```

            <div className="flex flex-col md:flex-row justify-around
items-center bg-slate-900/60 p-5 rounded-2xl border border-indigo-900
text-sm gap-6">

```

```

                { /* Clean HTML/CSS Fraction */ }
                <div className="flex items-center gap-3 font-serif text-lg
text-white">
                    <span>tan(θ) =</span>
                    <div className="flex flex-col items-center">
                        <span className="border-b border-indigo-300 pb-1 px-3
text-center">Height of Lighthouse</span>
                        <span className="pt-1 px-3 text-center">Distance on
Ground</span>
                    </div>
                </div>

```

```

    </div>
  </div>

  <div className="hidden md:block h-12 w-px bg-slate-700" />

  <div className="space-y-2 text-xs text-indigo-200">
    <p>☀️ <strong className="text-yellow-400">If  $\theta = 45^\circ$ :\tan(45^\circ) = 1 \Rightarrow \text{Height} = \text{Distance}</p>
    <p>☀️ <strong className="text-yellow-400">If  $\theta = 30^\circ$ :\tan(30^\circ) = 1 / \sqrt{3} \approx 0.577</p>
    <p>☀️ <strong className="text-yellow-400">If  $\theta = 60^\circ$ :\tan(60^\circ) = \sqrt{3} \approx 1.732</p>
  </div>
</div>

  </div>
</div>

  { /* Real Life Examples Accordion */ }
  <div className="mt-8">
    <h3 className="font-bold text-slate-900 text-lg mb-4 flex items-center gap-1.5">
      🌍 Real-World Angle of Elevation Examples:
    </h3>
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4 text-xs text-slate-600">
      <div className="p-3.5 bg-slate-50 rounded-xl flex items-start gap-2.5">
        <span className="p-1 bg-yellow-100 rounded-lg">🪁</span>
        <div>
          <h4 className="font-bold text-slate-800">Flying a Kite</h4>
          <p className="mt-1">Looking up at a colorful kite flying high up in the breeze.</p>
        </div>
      </div>
      <div className="p-3.5 bg-slate-50 rounded-xl flex items-start gap-2.5">

```

```
<span className="p-1 bg-yellow-100 rounded-lg">✈️</span>
<div>
  <h4 className="font-bold text-slate-800">Airplane
Spotting</h4>
  <p className="mt-1">A radar spotter tracking an
airplane taking off from the runway.</p>
</div>
</div>
<div className="p-3.5 bg-slate-50 rounded-xl flex
items-start gap-2.5">
  <span className="p-1 bg-yellow-100 rounded-lg">🗼</span>
  <div>
    <h4 className="font-bold text-slate-800">Statue of
Liberty Tourism</h4>
    <p className="mt-1">Standing nearby at the base and
tilting your head up to see her torch.</p>
  </div>
</div>
<div className="p-3.5 bg-slate-50 rounded-xl flex
items-start gap-2.5">
  <span className="p-1 bg-yellow-100 rounded-lg">🏀</span>
  <div>
    <h4 className="font-bold text-slate-800">Basketball
Shot</h4>
    <p className="mt-1">An athlete aiming up at the hoop
from the free-throw line.</p>
  </div>
</div>
</div>

{/* DIY Clinometer Activity */}
<div className="mt-10 border-t border-slate-200 pt-8">
  <div className="bg-amber-50 rounded-3xl p-6 border-2
border-amber-200">
```

```

        <span className="text-xs font-extrabold text-amber-800
bg-amber-100 px-3 py-1 rounded-full uppercase tracking-wider">
            Hands-On DIY Experiment
        </span>
        <h3 className="text-xl font-bold text-slate-950 mt-3">Build
Your Own Angle-Sighter (Clinometer)!</h3>
        <p className="text-slate-800 text-sm mt-1">
            You can measure the exact angle of elevation of any real
object inside your home using simple items!
        </p>

        <div className="mt-5 grid grid-cols-1 md:grid-cols-2 gap-6
items-center">
            <div className="space-y-3.5 text-xs text-slate-700">
                <div>
                    <strong className="text-amber-900 block font-semibold
mb-0.5">🔧 Materials Needed:</strong>
                    <p>A plastic protractor, a straw, string (20cm), a
small metal key or nut (weight), and tape.</p>
                </div>
                <div>
                    <strong className="text-amber-900 block font-semibold
mb-0.5">👣 Action Steps:</strong>
                    <ol className="list-decimal list-inside space-y-1">
                        <li>Tape the straight straw along the flat base
edge of your protractor.</li>
                        <li>Tie the string to the center hole of the
straight edge.</li>
                        <li>Attach the weight (key/nut) so the string hangs
straight down like a pendulum.</li>
                        <li>Look through the straw at the top of a tree or
building wall.</li>
                        <li>Let a friend read the angle on the protractor
scales! Subtract from 90° to get your angle of elevation!</li>
                    </ol>
                </div>
            </div>

```

</div>

```
<div className="bg-white p-4 rounded-2xl border
border-amber-200/50 shadow-inner flex flex-col justify-center
items-center">

  {/* Minimal SVG depiction of DIY Protractor */}
  <svg width="180" height="120" viewBox="0 0 180 120"
className="overflow-visible">
    {/* Protractor Semi-circle scale */}
    <path d="M 10 30 A 80 80 0 0 0 170 30" fill="none"
stroke="#d97706" strokeWidth="4" />
    <line x1="10" y1="30" x2="170" y2="30"
stroke="#b45309" strokeWidth="4" />
    {/* Straw */}
    <rect x="0" y="24" width="180" height="4"
fill="#60a5fa" rx="1" />
    <text x="90" y="16" fill="#3b82f6" fontSize="8"
fontWeight="bold" textAnchor="middle">STRAW</text>
    {/* Plumb string hanging */}
    <g transform="translate(90, 30)">
      {/* Hanging line rotated to demonstrate looking at
angle */}
      <line x1="0" y1="0" x2="30" y2="70"
stroke="#475569" strokeWidth="2" strokeDasharray="2,2" />
      <circle cx="30" cy="70" r="8" fill="#d97706" />
      <text x="44" y="74" fill="#78350f" fontSize="8"
fontWeight="bold">Hanging Weight</text>
    </g>
  </svg>

  <span className="text-slate-500 text-[11px] font-mono
mt-3">Simple Home Protractor Clinometer</span>
</div>
</div>
</div>
</div>
</div>
```

```

    })

    { /* Tab Content: Brain Quiz */
    {activeTab === 'quiz' && (
      <div className="max-w-3xl mx-auto space-y-6">
        <div className="bg-white rounded-3xl p-6 shadow-xl border
border-slate-200 text-center">
          <h2 className="text-2xl font-bold text-slate-800">Heights &
Distances Masterclass Quiz</h2>
          <p className="text-slate-600 text-sm mt-1">
            Put your mathematical deduction skills to the test! Earn 50
XP for every correct answer.
          </p>

          <div className="mt-4 inline-flex items-center gap-1.5
bg-indigo-50 text-indigo-800 text-xs px-4 py-1.5 rounded-full font-bold">
            🎯 Correct Answers: {quizScore} / {QUIZ_QUESTIONS.length}
          </div>
        </div>

      { /* Questions List */
      {QUIZ_QUESTIONS.map((q, idx) => (
        <div key={q.id} className="bg-white rounded-3xl p-6 shadow-md
border border-slate-200/80 space-y-4">
          <div className="flex items-start gap-2">
            <span className="bg-indigo-100 text-indigo-800
font-extrabold text-xs px-2.5 py-1 rounded-lg">
              Q{idx + 1}
            </span>
            <h3 className="font-bold text-slate-800 text-sm
md:text-base leading-snug">
              {q.question}
            </h3>
          </div>

          { /* Option buttons */

```

```

<div className="grid grid-cols-1 md:grid-cols-2 gap-3 pl-0
md:pl-8">

  {q.options.map((opt, optIdx) => {
    const isSelected = selectedAnswers[q.id] === optIdx;
    const isSubmitted = submittedQuestions[q.id];
    const isCorrectOption = optIdx === q.answerIndex;

    let btnClass = "border-slate-200 hover:bg-slate-50
text-slate-700";

    if (isSelected) {
      btnClass = "border-indigo-500 bg-indigo-50/70
text-indigo-900";
    }

    if (isSubmitted) {
      if (isCorrectOption) {
        btnClass = "border-green-500 bg-green-50
text-green-950";
      } else if (isSelected) {
        btnClass = "border-red-400 bg-red-50 text-red-900";
      } else {
        btnClass = "border-slate-200 text-slate-400
opacity-65";
      }
    }
  })

  return (
    <button
      key={optIdx}
      disabled={isSubmitted}
      onClick={() => handleSelectOption(q.id, optIdx)}
      className={`w-full text-left py-3 px-4 rounded-2xl
border text-xs font-semibold transition-all flex justify-between
items-center ${btnClass}`}
    >

    <span>{opt}</span>

    {isSubmitted && isCorrectOption && (

```

```

        <span className="text-green-600 font-bold
font-mono">✓ Correct</span>
      )}
      {isSubmitted && isSelected && !isCorrectOption && (
        <span className="text-red-500 font-bold
font-mono">✗ Incorrect</span>
      )}
    </button>
  );
  })}
</div>

{/* Confirm submit trigger */}
{selectedAnswers[q.id] !== undefined &&
!submittedQuestions[q.id] && (
  <div className="pl-0 md:pl-8 flex justify-end">
    <button
      onClick={() => handleQuizSubmit(q)}
      className="py-1.5 px-4 bg-slate-800
hover:bg-slate-700 text-white font-bold text-xs rounded-xl shadow-md
transition-all"
    >
      Lock Answer & Verify
    </button>
  </div>
)}

{/* Explanation text box once answered */}
{submittedQuestions[q.id] && (
  <div className="pl-0 md:pl-8 mt-2">
    <div className="bg-slate-50 p-4 rounded-2xl border
border-slate-100 text-xs text-slate-600 leading-relaxed">
      💡 <strong>Explanation:</strong> {q.explanation}
    </div>
  </div>
)}

```



```

        </div>
    )})
</div>
})

</main>

{/* Embedded Soft Celebration Overlay Banner */}
{showConfetti && (
    <div className="fixed bottom-6 right-6 z-50 bg-gradient-to-r
from-green-500 to-indigo-600 text-white p-4 rounded-2xl shadow-2xl border
border-white/20 animate-bounce max-w-sm flex items-center gap-3">
        <div className="text-3xl">🌟</div>
        <div>
            <h4 className="font-bold text-sm">Perfect Positioning!</h4>
            <p className="text-xs text-indigo-100">You matched the
trigonometry target beautifully!</p>
        </div>
    </div>
    )}
</div>
);
}

```

Fractions3

```

import React, { useState, useEffect, useRef } from 'react';

// Custom lightweight sound synthesizer to avoid external assets and keep the app
// self-contained
const playSound = (type) => {
  try {
    const AudioContext = window.AudioContext || window.webkitAudioContext;
    if (!AudioContext) return;
    const ctx = new AudioContext();

    switch(type) {
      case 'eat': // Cute munch sound
        const osc1 = ctx.createOscillator();
        const gain1 = ctx.createGain();
        osc1.type = 'triangle';
        osc1.frequency.setValueAtTime(150, ctx.currentTime);
        osc1.frequency.exponentialRampToValueAtTime(80, ctx.currentTime + 0.15);
        gain1.gain.setValueAtTime(0.3, ctx.currentTime);
        gain1.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.15);
        osc1.connect(gain1);
        gain1.connect(ctx.destination);
        osc1.start();
        osc1.stop(ctx.currentTime + 0.15);

        // Second crunch sound
        setTimeout(() => {
          const osc2 = ctx.createOscillator();
          const gain2 = ctx.createGain();
          osc2.type = 'triangle';
          osc2.frequency.setValueAtTime(180, ctx.currentTime);
          osc2.frequency.exponentialRampToValueAtTime(90, ctx.currentTime + 0.15);
          gain2.gain.setValueAtTime(0.2, ctx.currentTime);
          gain2.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.15);
          osc2.connect(gain2);
          gain2.connect(ctx.destination);
          osc2.start();
          osc2.stop(ctx.currentTime + 0.15);
        }, 100);
        break;
      case 'win': // Cheerful success sound
        const now = ctx.currentTime;
        [261.63, 329.63, 392.00, 523.25].forEach((freq, index) => {
          const osc = ctx.createOscillator();
          const gain = ctx.createGain();

```

```

    osc.type = 'sine';
    osc.frequency.setValueAtTime(freq, now + index * 0.1);
    gain.gain.setValueAtTime(0.15, now + index * 0.1);
    gain.gain.exponentialRampToValueAtTime(0.01, now + index * 0.1 + 0.25);
    osc.connect(gain);
    gain.connect(ctx.destination);
    osc.start(now + index * 0.1);
    osc.stop(now + index * 0.1 + 0.25);
  });
  break;
case 'click': // Soft UI feedback click
  const oscClick = ctx.createOscillator();
  const gainClick = ctx.createGain();
  oscClick.type = 'sine';
  oscClick.frequency.setValueAtTime(600, ctx.currentTime);
  oscClick.frequency.exponentialRampToValueAtTime(300, ctx.currentTime + 0.05);
  gainClick.gain.setValueAtTime(0.05, ctx.currentTime);
  gainClick.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.05);
  oscClick.connect(gainClick);
  gainClick.connect(ctx.destination);
  oscClick.start();
  oscClick.stop(ctx.currentTime + 0.05);
  break;
case 'reset': // Whistle slide up
  const oscR = ctx.createOscillator();
  const gainR = ctx.createGain();
  oscR.type = 'sine';
  oscR.frequency.setValueAtTime(200, ctx.currentTime);
  oscR.frequency.exponentialRampToValueAtTime(450, ctx.currentTime + 0.3);
  gainR.gain.setValueAtTime(0.1, ctx.currentTime);
  gainR.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.3);
  oscR.connect(gainR);
  gainR.connect(ctx.destination);
  oscR.start();
  oscR.stop(ctx.currentTime + 0.3);
  break;
case 'fail': // Soft warning buzz
  const oscF = ctx.createOscillator();
  const gainF = ctx.createGain();
  oscF.type = 'sawtooth';
  oscF.frequency.setValueAtTime(130, ctx.currentTime);
  oscF.frequency.linearRampToValueAtTime(100, ctx.currentTime + 0.2);
  gainF.gain.setValueAtTime(0.1, ctx.currentTime);
  gainF.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.2);

```

```

    oscF.connect(gainF);
    gainF.connect(ctx.destination);
    oscF.start();
    oscF.stop(ctx.currentTime + 0.2);
    break;
  default:
    break;
}
} catch (e) {
  // Audio Context blocked or not supported
}
};

```

```

export default function App() {
  const [activeTab, setActiveTab] = useState('simulator'); // 'simulator' | 'quiz' | 'lessons'
  const [xp, setXp] = useState(100);

  const addXp = (amount) => {
    setXp(prev => prev + amount);
  };

  return (
    <div className="min-h-screen bg-slate-50 text-slate-800 font-sans antialiased
selection:bg-orange-200">
      {/* Upper Brand / Interactive Header */}
      <header className="bg-white border-b border-slate-200 sticky top-0 z-30 shadow-sm">
        <div className="max-w-6xl mx-auto px-4 py-3 flex flex-col sm:flex-row items-center
justify-between gap-3">
          <div className="flex items-center gap-3">
            <span className="text-3xl" role="img" aria-label="Master Chef Hat">👨‍🍳</span>
            <div>
              <h1 className="text-xl font-bold text-slate-900 tracking-tight flex items-center gap-2">
                The Master Chef's Sharing Lab
                <span className="text-xs bg-orange-100 text-orange-700 font-semibold px-2 py-0.5
rounded-full border border-orange-200">
                  NCERT Classes 3-5
                </span>
              </h1>
              <p className="text-xs text-slate-500">Visual Math Simulator & Interactive Fractions
Playground</p>
            </div>
          </div>
          <div className="flex items-center gap-4">

```

```

    { /* Live XP Counter */ }
    <div className="flex items-center gap-2 bg-amber-50 border border-amber-200 px-3.5
py-1.5 rounded-full shadow-inner animate-pulse">
      <span className="text-lg"> ✨ </span>
      <span className="text-sm font-bold text-amber-700">{xp} <span className="text-xs
font-medium">XP Points</span></span>
    </div>

```

```

    { /* Tab Swappers */ }
    <nav className="flex bg-slate-100 p-1 rounded-xl border border-slate-200">
      <button
        onClick={() => { setActiveTab('simulator'); playSound('click'); }}
        className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm font-semibold transition-all ${
          activeTab === 'simulator'
            ? 'bg-white text-orange-600 shadow-sm'
            : 'text-slate-600 hover:text-slate-900'
          }`}
      >
        🎮 Play Lab
      </button>
      <button
        onClick={() => { setActiveTab('quiz'); playSound('click'); }}
        className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm font-semibold transition-all ${
          activeTab === 'quiz'
            ? 'bg-white text-orange-600 shadow-sm'
            : 'text-slate-600 hover:text-slate-900'
          }`}
      >
        📝 Pizza Quiz
      </button>
      <button
        onClick={() => { setActiveTab('lessons'); playSound('click'); }}
        className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm font-semibold transition-all ${
          activeTab === 'lessons'
            ? 'bg-white text-orange-600 shadow-sm'
            : 'text-slate-600 hover:text-slate-900'
          }`}
      >
        📖 Fun Lesson
      </button>
    </nav>
  </div>
</div>
</header>

```

```

    { /* Main Container Wrapper */}
    <main className="max-w-6xl mx-auto px-4 py-6">
      {activeTab === 'simulator' && <SimulatorView addXp={addXp} />}
      {activeTab === 'quiz' && <QuizView addXp={addXp} />}
      {activeTab === 'lessons' && <LessonView />}
    </main>

    <footer className="bg-slate-900 text-slate-400 py-6 mt-12 border-t border-slate-800
text-center text-xs">
      <p>👑 Made with Love for Budding Mathematicians • Designed according to
CBSE/NCERT Math-Magic guidelines.</p>
      <p className="mt-1 text-slate-600">Tip: Click or tap objects to instantly share them, or
slide the dial to cut them differently!</p>
    </footer>
  </div>
);
}

// =====
// SIMULATOR COMPONENT (THE MASTER CHEF LAB)
// =====
function SimulatorView({ addXp }) {
  const [foodType, setFoodType] = useState('pizza'); // 'pizza' | 'chocolate' | 'cake'
  const [totalCuts, setTotalCuts] = useState(4); // Denominator (2 to 8)
  const [eatenList, setEatenList] = useState([]); // indices of slices given to the monster
  const [fairCuts, setFairCuts] = useState(true); // Is sharing fair (equal sizes)?
  const [tipMessage, setTipMessage] = useState("Let's start! Drag a piece, or tap one to feed
our friendly Monster!");
  const [monsterState, setMonsterState] = useState('idle'); // 'idle' | 'happy' | 'chewing' |
'disappointed'
  const [confetti, setConfetti] = useState([]);

  // Audio state
  const lastEatenCount = useRef(0);

  // Dragging states
  const [draggedIndex, setDraggedIndex] = useState(null);
  const [dragPosition, setDragPosition] = useState({ x: 0, y: 0 });
  const containerRef = useRef(null);
  const monsterRef = useRef(null);

  // Trigger feedback messages when fractions change
  useEffect(() => {

```

```

const eatenCount = eatenList.length;
if (eatenCount === 0) {
  setTipMessage("Set the number of cuts below, then share some pieces with the monster!");
} else if (eatenCount === totalCuts) {
  setTipMessage("🎉 Awesome! You shared all ${totalCuts}/${totalCuts} parts. That equals 1 Whole item!");
  triggerConfetti();
  playSound('win');
  setMonsterState('happy');
  addXp(15);
} else if (!fairCuts) {
  setTipMessage("⚠️ Notice: When cuts are unfair (unequal sizes), we CANNOT use standard mathematical fractions!");
  setMonsterState('disappointed');
} else {
  const remaining = totalCuts - eatenCount;
  // Fraction recognition
  const halfVal = totalCuts / 2;
  if (eatenCount === halfVal) {
    setTipMessage("💡 Fun Spot: ${eatenCount}/${totalCuts} is exactly Half (1/2) of the whole food!");
    addXp(10);
  } else {
    setTipMessage(`Yum! You fed the monster ${eatenCount} out of ${totalCuts} parts. There are ${remaining} parts left.`);
  }
}
lastEatenCount.current = eatenCount;
}, [eatenList, totalCuts, fairCuts]);

const triggerConfetti = () => {
  const items = Array.from({ length: 40 }).map((_, i) => ({
    id: i,
    x: Math.random() * 100,
    y: -20 - Math.random() * 50,
    size: Math.random() * 8 + 6,
    color: ['#FF6B6B', '#FFD93D', '#6BCB77', '#4D96FF', '#FF8E9E'][Math.floor(Math.random() * 5)],
    delay: Math.random() * 0.5,
    speed: Math.random() * 3 + 2
  }));
  setConfetti(items);
  setTimeout(() => setConfetti([]), 4000);
};

```



```
const handleReset = () => {  
  playSound('reset');  
  setEatenList([]);  
  setMonsterState('idle');  
  setFairCuts(true);  
};
```

// Switch Food

```
const selectFood = (food) => {  
  playSound('click');  
  setFoodType(food);  
  setEatenList([]);  
  // Set typical defaults  
  if (food === 'chocolate') {  
    setTotalCuts(6);  
  } else if (food === 'cake') {  
    setTotalCuts(3);  
  } else {  
    setTotalCuts(4);  
  }  
};
```

// Change slices count

```
const handleCutsChange = (e) => {  
  playSound('click');  
  setTotalCuts(parseInt(e.target.value));  
  setEatenList([]);  
};
```

// Tap-to-feed action

```
const handleSliceClick = (index) => {  
  if (!fairCuts) {  
    playSound('fail');  
    setTipMessage("⚠️ Can't eat! These cuts are unequal. Toggle 'Fair Cuts' to make sharing  
mathematically correct!");  
    setMonsterState('disappointed');  
    return;  
  }  
  if (eatenList.includes(index)) return; // already eaten  
  
  playSound('eat');  
  setMonsterState('chewing');  
  setEatenList(prev => [...prev, index]);
```

```
setTimeout(() => {
  setMonsterState('happy');
  setTimeout(() => setMonsterState('idle'), 1200);
}, 600);
};
```

// Pointer Drag-and-drop Events

```
const handlePointerDown = (index, e) => {
  if (!fairCuts) {
    playSound('fail');
    setMonsterState('disappointed');
    return;
  }
  if (eatenList.includes(index)) return;
```

```
  setDraggedIndex(index);
  const rect = e.currentTarget.getBoundingClientRect();
  // Center of touch or cursor offset relative to the piece
  setDragPosition({
    x: 0,
    y: 0
  });
  e.currentTarget.setPointerCapture(e.pointerId);
};
```

```
const handlePointerMove = (e) => {
  if (draggedIndex === null) return;
  setDragPosition(prev => ({
    x: prev.x + e.movementX,
    y: prev.y + e.movementY
  }));
};
```

```
const handlePointerUp = (e) => {
  if (draggedIndex === null) return;
  e.currentTarget.releasePointerCapture(e.pointerId);
```

// Collision Check with Monster

```
if (monsterRef.current) {
  const monsterRect = monsterRef.current.getBoundingClientRect();
  const pieceRect = e.currentTarget.getBoundingClientRect();
```

// Simple overlap calculation

```

const overlap = !(
  pieceRect.right < monsterRect.left ||
  pieceRect.left > monsterRect.right ||
  pieceRect.bottom < monsterRect.top ||
  pieceRect.top > monsterRect.bottom
);

// Or if moved far right (towards monster layout)
const movedFarRight = dragPosition.x > 120;

if (overlap || movedFarRight) {
  // Shared successfully!
  playSound('eat');
  setMonsterState('chewing');
  setEatenList(prev => [...prev, draggedIndex]);
  setTimeout(() => {
    setMonsterState('happy');
    setTimeout(() => setMonsterState('idle'), 1500);
  }, 800);
} else {
  // Snap back
  playSound('click');
}
}

setDraggedIndex(null);
setDragPosition({ x: 0, y: 0 });
};

return (
  <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 items-start relative">
    {/* Dynamic Confetti Overlays */}
    {confetti.map((c) => (
      <div
        key={c.id}
        className="absolute pointer-events-none rounded-full animate-bounce z-50 transition-all
duration-1000 ease-out"
        style={{
          left: `${c.x}%`,
          top: `${c.y + 400}px`,
          width: `${c.size}px`,
          height: `${c.size}px`,
          backgroundColor: c.color,
          opacity: 0.8,

```

```

        transform: `translateY(${Math.random() * 200}px) rotate(${Math.random() * 360}deg)`
      }}
    />
  )))

  { /* LEFT: INTERACTIVE CANVAS AREA (8 Cols) */ }
  <div className="lg:col-span-8 bg-white border border-slate-200 rounded-3xl p-6
shadow-sm flex flex-col justify-between min-h-[520px]">

    { /* Sub Header Selector Panel */ }
    <div className="flex flex-wrap items-center justify-between gap-4 border-b
border-slate-100 pb-4 mb-4">
      <div className="flex items-center gap-2">
        <span className="text-xs font-bold text-slate-500 uppercase tracking-widest">Select
Food:</span>
        <div className="flex gap-1.5">
          {[
            { id: 'pizza', name: 'Pizza', icon: '🍕', color: 'hover:bg-orange-50
hover:text-orange-600' },
            { id: 'chocolate', name: 'Chocolate', icon: '🍫', color: 'hover:bg-amber-50
hover:text-amber-700' },
            { id: 'cake', name: 'Cake', icon: '🍰', color: 'hover:bg-pink-50 hover:text-pink-600' }
          ].map((food) => (
            <button
              key={food.id}
              onClick={() => selectFood(food.id)}
              className={`flex items-center gap-1.5 px-3 py-1.5 rounded-xl text-sm font-semibold
transition-all border ${
                foodType === food.id
                  ? 'bg-slate-900 border-slate-900 text-white shadow-sm'
                  : 'bg-slate-50 border-slate-200 text-slate-600 ${food.color}`
              }`
            >
              <span className="text-base">{food.icon}</span>
              <span>{food.name}</span>
            </button>
          )]}
        </div>
      </div>

    { /* Cuts Equality Toggle */ }
    <div className="flex items-center gap-2">
      <span className="text-xs font-bold text-slate-500 uppercase tracking-widest">Cuts
Layout:</span>

```

```

<button
  onClick={() => {
    playSound('click');
    setFairCuts(!fairCuts);
    setEatenList([]);
  }}
  className={`flex items-center gap-2 px-3 py-1.5 rounded-xl text-xs font-bold
transition-all border ${
  fairCuts
    ? 'bg-emerald-50 border-emerald-200 text-emerald-700'
    : 'bg-red-50 border-red-200 text-red-700'
  }`}
>
  <div className={`w-2.5 h-2.5 rounded-full ${fairCuts ? 'bg-emerald-500 animate-ping' :
'bg-red-500'}} />
  {fairCuts ? "Fair Sharing (Equal)" : "Unfair (Unequal Cuts)"}
</button>
</div>
</div>

```

```

{/* INTERACTIVE SHARING ARENA */}
<div
  ref={containerRef}
  className="relative flex-1 grid grid-cols-1 md:grid-cols-2 gap-6 items-center
justify-center p-4 bg-amber-50/20 rounded-2xl border border-dashed border-amber-200
min-h-[300px] select-none"
>
  {/* Chef Plate with Interactive Pieces */}
  <div className="flex flex-col items-center justify-center relative">
    <p className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-2">🍕
Slices on Chef's Plate</p>

```

```

    {/* White Plate Circle Backdrop */}
    <div className="w-64 h-64 bg-slate-100 rounded-full border-4 border-slate-200
shadow-inner flex items-center justify-center relative">

```

```

    {/* Dynamic Food Renderer */}
    <div className="w-56 h-56 relative rounded-full overflow-visible">
      {foodType === 'pizza' && (
        <PizzaSVG
          totalCuts={totalCuts}
          eatenList={eatenList}
          fairCuts={fairCuts}
          draggedIndex={draggedIndex}

```

```

    dragPosition={dragPosition}
    onPointerDown={handlePointerDown}
    onPointerMove={handlePointerMove}
    onPointerUp={handlePointerUp}
    onSliceClick={handleSliceClick}
  />
)}

```

```

{foodType === 'chocolate' && (
  <ChocolateSVG
    totalCuts={totalCuts}
    eatenList={eatenList}
    fairCuts={fairCuts}
    draggedIndex={draggedIndex}
    dragPosition={dragPosition}
    onPointerDown={handlePointerDown}
    onPointerMove={handlePointerMove}
    onPointerUp={handlePointerUp}
    onSliceClick={handleSliceClick}
  />
)}

```

```

{foodType === 'cake' && (
  <CakeSVG
    totalCuts={totalCuts}
    eatenList={eatenList}
    fairCuts={fairCuts}
    draggedIndex={draggedIndex}
    dragPosition={dragPosition}
    onPointerDown={handlePointerDown}
    onPointerMove={handlePointerMove}
    onPointerUp={handlePointerUp}
    onSliceClick={handleSliceClick}
  />
)}
</div>

```

```

</div>
</div>

```

```

{/* Interactive Hungry Monster Target Area */}
<div ref={monsterRef} className="flex flex-col items-center justify-center">
  <p className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-2">🦋
    Feed Target Zone</p>

```

```

<div className={`w-64 h-64 rounded-3xl border-2 transition-all flex flex-col items-center
justify-center p-4 relative ${
  monsterState === 'chewing'
    ? 'bg-amber-100 border-amber-400 scale-105'
    : monsterState === 'happy'
    ? 'bg-emerald-50 border-emerald-400 scale-105 shadow-lg'
    : monsterState === 'disappointed'
    ? 'bg-rose-50 border-rose-300'
    : draggedIndex !== null
    ? 'bg-orange-50 border-orange-300 border-dashed animate-pulse'
    : 'bg-white border-slate-200 shadow-sm'
}`}>

```

```

    {/* Beautiful Vector Animated Monster Face */}
    <svg className="w-32 h-32" viewBox="0 0 100 100" fill="none"
xmlns="http://www.w3.org/2000/svg">
      {/* Monster Body */}
      <path d="M15,80 C15,40 25,15 50,15 C75,15 85,40 85,80 C85,90 75,95 50,95
C25,95 15,90 15,80 Z" fill="#6C63FF" />

```

```

    {/* Horns */}
    <path d="M25,25 Q15,10 22,5 Q35,15 30,28 Z" fill="#FF6584" />
    <path d="M75,25 Q85,10 78,5 Q65,15 70,28 Z" fill="#FF6584" />

```

```

    {/* Eyes */}
    {monsterState === 'happy' || monsterState === 'chewing' ? (
      <>
        {/* Happy squinting eyes */}
        <path d="M30,42 Q40,35 42,45" stroke="#FFF" strokeWidth="5"
strokeLinecap="round" />
        <path d="M70,42 Q60,35 58,45" stroke="#FFF" strokeWidth="5"
strokeLinecap="round" />
      </>
    ) : (
      <>
        {/* Round attentive eyes */}
        <circle cx="38" cy="42" r="10" fill="white" />
        <circle cx="62" cy="42" r="10" fill="white" />
        <circle cx="38" cy="42" r="5" fill="#1e293b" />
        <circle cx="62" cy="42" r="5" fill="#1e293b" />
      </>
    )}

```

```

    { /* Cheeks */
    <circle cx="22" cy="52" r="4" fill="#FF8E9E" opacity="0.6" />
    <circle cx="78" cy="52" r="4" fill="#FF8E9E" opacity="0.6" />

    { /* Dynamic Mouth */
    {monsterState === 'chewing' ? (
      <path d="M35,68 Q50,55 65,68 Q50,75 35,68 Z" fill="#1E293B"
className="animate-bounce" />
    ) : monsterState === 'happy' ? (
      <path d="M30,62 Q50,85 70,62" fill="none" stroke="#FFF" strokeWidth="6"
strokeLinecap="round" />
    ) : monsterState === 'disappointed' ? (
      <path d="M35,70 Q50,55 65,70" fill="none" stroke="#FF6584" strokeWidth="5"
strokeLinecap="round" />
    ) : draggedIndex !== null ? (
      /* Big hungry wide open mouth ready for food */
      <circle cx="50" cy="70" r="14" fill="#1E293B" stroke="#FF8E9E" strokeWidth="3" />
    ) : (
      /* Standard Smile */
      <path d="M35,65 Q50,78 65,65" fill="none" stroke="#1E293B" strokeWidth="5"
strokeLinecap="round" />
    )
    }}

    { /* Single Cute Tooth */
    {monsterState !== 'happy' && monsterState !== 'disappointed' && (
      <polygon points="46,58 54,58 50,65" fill="white" />
    )
    }}
  </svg>

  <div className="text-center mt-3">
    <span className="text-xs font-bold uppercase tracking-wide px-2.5 py-1 rounded-full
bg-slate-100 text-slate-700">
      {monsterState === 'chewing' && "Crunch, Munch... 😊"}
      {monsterState === 'happy' && "Awesome! More! 🥰"}
      {monsterState === 'disappointed' && "Hey! That's not a fair slice! 😞"}
      {monsterState === 'idle' && draggedIndex !== null && "Feed me! Open wide! 💋"}
      {monsterState === 'idle' && draggedIndex === null && "Fractions Monster 🦋"}
    </span>

    { /* Quick Belly Fraction Display */
    <p className="text-xs text-slate-500 mt-2 font-medium">
      Belly Status: {eatenList.length === 0 ? "Empty!" : `${eatenList.length}/${totalCuts}
eaten`}
    </p>
  }

```



```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* FEEDBACK & HINT MESSAGE BANNER */}
```

```
<div className="mt-4 p-4 rounded-xl bg-orange-50 border border-orange-100 flex  
items-center gap-3">
```

```
<span className="text-2xl" role="img" aria-label="Tip Bulb">💡 </span>
```

```
<p className="text-sm font-semibold text-slate-700">{tipMessage}</p>
```

```
</div>
```

```
{/* LOWER CONTROLS & SLIDER AREA */}
```

```
<div className="mt-6 border-t border-slate-100 pt-5 flex flex-col md:flex-row items-center  
gap-5 justify-between">
```

```
<div className="flex-1 w-full">
```

```
<div className="flex justify-between items-center mb-1">
```

```
<label htmlFor="cuts-slider" className="text-sm font-bold text-slate-700 flex  
items-center gap-1.5">
```

```


|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

 Total Cuts (Denominator):
```

```
<span className="text-lg text-orange-600 font-extrabold">{totalCuts}</span>
```

```
</label>
```

```
<span className="text-xs text-slate-400">Range: 2 to 8 cuts</span>
```

```
</div>
```

```
<input
```

```
id="cuts-slider"
```

```
type="range"
```

```
min="2"
```

```
max="8"
```

```
value={totalCuts}
```

```
onChange={handleCutsChange}
```

```
className="w-full h-2.5 bg-slate-200 rounded-lg appearance-none cursor-pointer  
accent-orange-500"
```

```
/>
```

```
</div>
```

```
<div className="flex items-center gap-2.5 w-full md:w-auto justify-end">
```

```
<button
```

```
onClick={handleReset}
```

```
className="flex-1 md:flex-none flex items-center justify-center gap-1.5 px-5 py-2.5  
rounded-xl border border-slate-200 hover:bg-slate-50 text-slate-700 text-sm font-bold  
transition-all"
```

```

>
   Reset Plate
</button>

<button
  onClick={() => {
    playSound('win');
    triggerConfetti();
    addXp(5);
  }}
  className="flex-1 md:flex-none flex items-center justify-center gap-1.5 px-5 py-2.5
rounded-xl bg-emerald-500 hover:bg-emerald-600 text-white text-sm font-bold shadow-md
hover:shadow-lg transition-all"
  >
     Yay Chef!
  </button>
</div>

</div>

</div>

{/* RIGHT: LIVE OBSERVATION PANEL (4 Cols) */}
<div className="lg:col-span-4 flex flex-col gap-6">

  {/* LIVE FRACTION READOUT BOARD */}
  <div className="bg-gradient-to-br from-slate-900 to-slate-800 text-white rounded-3xl p-6
shadow-xl border border-slate-700">
    <h2 className="text-xs uppercase tracking-widest text-slate-400 font-extrabold
mb-4"> Fractional Dashboard</h2>

    <div className="grid grid-cols-2 gap-4">

      {/* Left: Monster's eaten plate */}
      <div className="bg-slate-800/80 p-3 rounded-2xl text-center border border-slate-700">
        <p className="text-xs text-slate-400 font-bold mb-1">MONSTER ATE</p>

        <div className="flex flex-col items-center justify-center my-3">
          <span className="text-3xl font-extrabold
text-orange-400">{eatenList.length}</span>
          <div className="w-10 h-0.5 bg-slate-500 my-1.5" />
          <span className="text-xl font-bold text-slate-300">{totalCuts}</span>
        </div>
        <p className="text-[10px] text-slate-400 mt-1">

```

```

    {fairCuts ? "Equal Parts" : "Unequal Parts"}
  </p>
</div>

{ /* Right: Left on Plate */ }
<div className="bg-slate-800/80 p-3 rounded-2xl text-center border border-slate-700">
  <p className="text-xs text-slate-400 font-bold mb-1">ON PLATE</p>

  <div className="flex flex-col items-center justify-center my-3">
    <span className="text-3xl font-extrabold text-teal-400">{totalCuts -
eatenList.length}</span>
    <div className="w-10 h-0.5 bg-slate-500 my-1.5" />
    <span className="text-xl font-bold text-slate-300">{totalCuts}</span>
  </div>
  <p className="text-[10px] text-slate-400 mt-1">Remaining Slices</p>
</div>

</div>

{ /* Golden Definition Tip Card */ }
<div className="mt-5 bg-slate-800/50 rounded-xl p-3 border border-slate-700 text-xs
flex flex-col gap-2">
  <p className="text-orange-300 font-bold flex items-center gap-1">
     Let's Remember:
  </p>
  <p className="text-slate-300">
    The <strong className="text-white underline
decoration-orange-400">Numerator</strong> (Top) tells us how many pieces we chose to eat.
  </p>
  <p className="text-slate-300">
    The <strong className="text-white underline
decoration-teal-400">Denominator</strong> (Bottom) tells us the total equal cuts we started
with!
  </p>
  <div className="bg-amber-500/10 border border-amber-500/20 text-amber-200 p-2
rounded-lg mt-1 font-semibold">
    ★ Memory Trick: <strong className="text-amber-300">D</strong>enominator starts
with <strong className="text-amber-300">D</strong> for <strong
className="text-amber-300">Down</strong>!
  </div>
</div>
</div>

{ /* GUIDED EXPLORATION TASKS CARD */ }

```

```

<div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-sm">
  <h2 className="text-sm font-extrabold text-slate-900 flex items-center gap-2 mb-4">
    🌟 Chef's Special Challenges
  </h2>

  <div className="space-y-4">
    {[
      {
        id: 1,
        title: "Represent 1/2 (Half)",
        task: "Choose Pizza 🍕, cut into 4 slices. Tap exactly 2 slices to feed the monster.",
        isComplete: totalCuts === 4 && eatenList.length === 2 && foodType === 'pizza'
      },
      {
        id: 2,
        title: "Make a Whole Object",
        task: "Choose Chocolate 🍫, cut into 6 slices. Feed all 6 pieces so the monster gets
the whole bar!",
        isComplete: totalCuts === 6 && eatenList.length === 6 && foodType === 'chocolate'
      },
      {
        id: 3,
        title: "The Tiny Cake Slice",
        task: "Choose Cake 🍰. Change cuts to maximum (8 slices). Observe how tiny the
slices become compared to 2 cuts!",
        isComplete: totalCuts === 8 && foodType === 'cake'
      }
    ]}.map((challenge) => (
      <div
        key={challenge.id}
        className={`p-3.5 rounded-2xl border transition-all ${
          challenge.isComplete
            ? 'bg-emerald-50 border-emerald-200'
            : 'bg-slate-50 border-slate-100 hover:border-slate-200'
        }`}
      >
        <div className="flex items-center justify-between gap-2 mb-1">
          <h3 className="font-bold text-xs text-slate-800">{challenge.title}</h3>
          <span className={`text-[10px] font-bold px-2 py-0.5 rounded-full ${
            challenge.isComplete ? 'bg-emerald-200 text-emerald-800' : 'bg-slate-200
text-slate-500'
          }`}>
            {challenge.isComplete ? "✅ Complete! +20XP" : "⌚ Incomplete"}
          </span>
        </div>
      </div>
    ))
  </div>

```

```

        </div>
        <p className="text-xs text-slate-500 leading-relaxed">{challenge.task}</p>
    </div>
  )}
</div>
</div>

</div>
</div>
);
}

```

```

// =====
// PIZZA DRAWING COMPONENT (SVG)
// =====
function PizzaSVG({
  totalCuts, eatenList, fairCuts,
  draggedIndex, dragPosition,
  onPointerDown, onPointerMove, onPointerUp, onSliceClick
}) {
  const slices = [];
  const radius = 96; // Pizza size radius
  const cx = 112; // Canvas Center X (based on 224px container)
  const cy = 112; // Canvas Center Y

  for (let i = 0; i < totalCuts; i++) {
    const isEaten = eatenList.includes(i);
    const isBeingDragged = draggedIndex === i;

    // Angle calculation
    let angleStart, angleEnd;

    if (fairCuts) {
      // Perfect fractions
      angleStart = (i * 360) / totalCuts;
      angleEnd = ((i + 1) * 360) / totalCuts;
    } else {
      // Uneven segments representing unfair/unequal divisions
      const unevenWeights = {
        2: [45, 315],
        3: [30, 90, 240],
        4: [40, 60, 110, 150],
        5: [30, 45, 90, 85, 110],
        6: [20, 40, 80, 50, 90, 80],
      };
    }
  }
}

```

```
7: [20, 30, 60, 40, 90, 80, 40],
8: [20, 25, 35, 40, 60, 80, 50, 50]
};
```

```
const weights = unevenWeights[totalCuts] || Array(totalCuts).fill(360 / totalCuts);
const sumBefore = weights.slice(0, i).reduce((a, b) => a + b, 0);
const sumCurrent = sumBefore + weights[i];
```

```
angleStart = (sumBefore / weights.reduce((a, b) => a + b, 0)) * 360;
angleEnd = (sumCurrent / weights.reduce((a, b) => a + b, 0)) * 360;
}
```

```
// Convert to Radian
const radStart = (angleStart - 90) * Math.PI / 180;
const radEnd = (angleEnd - 90) * Math.PI / 180;

const x1 = cx + radius * Math.cos(radStart);
const y1 = cy + radius * Math.sin(radStart);
const x2 = cx + radius * Math.cos(radEnd);
const y2 = cy + radius * Math.sin(radEnd);

// Large arc flag for segments larger than 180 deg
const largeArcFlag = angleEnd - angleStart > 180 ? 1 : 0;
```

```
// SVG path string
const dPath = `M ${cx} ${cy} L ${x1} ${y1} A ${radius} ${radius} 0 ${largeArcFlag} 1 ${x2}
${y2} Z`;
```

```
// To create a slight visual gap between slices, push them slightly outward in their base
coordinate
```

```
const midAngle = (radStart + radEnd) / 2;
const gapOffset = 3;
const offset_x = gapOffset * Math.cos(midAngle);
const offset_y = gapOffset * Math.sin(midAngle);
```

```
// Dynamic styles based on drag and drop positions
let transformStyle = `translate(${offset_x}px, ${offset_y}px)`;
let zIndex = 10;
```

```
if (isBeingDragged) {
  transformStyle = `translate(${dragPosition.x + offset_x}px, ${dragPosition.y + offset_y}px)
scale(1.1)`;
  zIndex = 50;
}
```

```

slices.push(
  <g
    key={i}
    style={{
      transform: transformStyle,
      transition: isBeingDragged ? 'none' : 'transform 0.3s cubic-bezier(0.175, 0.885, 0.32,
1.275)',
      opacity: isEaten ? 0.08 : 1,
      pointerEvents: isEaten ? 'none' : 'auto',
      cursor: 'grab',
      zIndex: zIndex
    }}
    onPointerDown={(e) => onPointerDown(i, e)}
    onPointerMove={onPointerMove}
    onPointerUp={onPointerUp}
    onClick={() => onSliceClick(i)}
    className="group origin-center"
  >
    {/* Golden Brown Crust */}
    <path
      d={dPath}
      fill="#D6802E"
      stroke="#9F5513"
      strokeWidth="4"
      className="group-hover:stroke-amber-400"
    />
    {/* Yellow-Orange Cheese Layer */}
    <path
      d={dPath}
      fill="#FFC043"
      transform="scale(0.92)"
      className="origin-center"
    />

    {/* Pepperoni dots/toppings on pizza for real detail! */}
    <circle cx={cx + (radius * 0.5) * Math.cos(midAngle)} cy={cy + (radius * 0.5) *
Math.sin(midAngle)} r="7" fill="#E23E3E" opacity="0.9" />
    <circle cx={cx + (radius * 0.7) * Math.cos(midAngle - 0.15)} cy={cy + (radius * 0.7) *
Math.sin(midAngle - 0.15)} r="5" fill="#C11E1E" opacity="0.9" />

    {/* White Fraction Overlay label in center of slice */}
    {!isEaten && (
      <text

```

```

        x={cx + (radius * 0.6) * Math.cos(midAngle)}
        y={cy + (radius * 0.6) * Math.sin(midAngle) + 4}
        fill="#7C2D12"
        fontSize="10"
        fontWeight="bold"
        textAnchor="middle"
        className="select-none pointer-events-none drop-shadow-md bg-white rounded-full"
      >
        {fairCuts ? `1/${totalCuts}` : '?'}
      </text>
    )}
  </g>
);
}

return (
  <svg className="w-full h-full overflow-visible" viewBox="0 0 224 224">
    {slices}
  </svg>
);
}

// =====
// CHOCOLATE BAR COMPONENT (SVG)
// =====
function ChocolateSVG({
  totalCuts, eatenList, fairCuts,
  draggedIndex, dragPosition,
  onPointerDown, onPointerMove, onPointerUp, onSliceClick
}) {
  const blocks = [];
  const barWidth = 200;
  const barHeight = 120;

  // Starting Coordinates inside the 224x224 Canvas
  const startX = 12;
  const startY = 52;

  // Render horizontal columns representing chocolate divisions
  for (let i = 0; i < totalCuts; i++) {
    const isEaten = eatenList.includes(i);
    const isBeingDragged = draggedIndex === i;

    // Define individual width of each strip

```



```

let blockWidth = barWidth / totalCuts;
let blockX = startX + i * blockWidth;

if (!fairCuts) {
  // Make slices structurally unfair
  const unfairWidths = {
    2: [60, 140],
    3: [40, 50, 110],
    4: [30, 40, 40, 90],
    5: [20, 30, 40, 50, 60],
    6: [20, 20, 30, 40, 40, 50],
    7: [15, 20, 25, 30, 35, 40, 35],
    8: [15, 15, 20, 20, 30, 30, 35, 35]
  };

  const widths = unfairWidths[totalCuts] || Array(totalCuts).fill(barWidth / totalCuts);
  const sumBefore = widths.slice(0, i).reduce((a, b) => a + b, 0);
  blockX = startX + sumBefore;
  blockWidth = widths[i];
}

// Interactive translations
let transformStyle = "";
let zIndex = 10;

if (isBeingDragged) {
  transformStyle = `translate(${dragPosition.x}px, ${dragPosition.y}px) scale(1.1)`;
  zIndex = 50;
}

blocks.push(
  <g
    key={i}
    style={{
      transform: transformStyle,
      transition: isBeingDragged ? 'none' : 'transform 0.3s ease-out',
      opacity: isEaten ? 0.05 : 1,
      pointerEvents: isEaten ? 'none' : 'auto',
      cursor: 'grab',
      zIndex: zIndex
    }}
    onPointerDown={(e) => onPointerDown(i, e)}
    onPointerMove={onPointerMove}
    onPointerUp={onPointerUp}
  >

```

```

onClick={() => onSliceClick(i)}
className="group origin-center"
>
{/* Outer Chocolate Block Frame */}
<rect
  x={blockX + 2}
  y={startY}
  width={blockWidth - 4}
  height={barHeight}
  rx="6"
  fill="#5c3d2e"
  stroke="#41291c"
  strokeWidth="3"
  className="group-hover:stroke-amber-400"
/>

{/* Inner raised sweet layer */}
<rect
  x={blockX + 5}
  y={startY + 5}
  width={blockWidth - 10}
  height={barHeight - 10}
  rx="4"
  fill="#784f3d"
/>

{/* Fancy Brand Imprint Grid lines inside block */}
<rect
  x={blockX + 8}
  y={startY + 15}
  width={blockWidth - 16}
  height={barHeight - 30}
  rx="3"
  fill="#4a3024"
  opacity="0.5"
/>

{/* Value Label */}
{!isEaten && (
  <text
    x={blockX + blockWidth / 2}
    y={startY + barHeight / 2 + 5}
    fill="ffffff"
    fontSize="10"
  >

```

```

        fontWeight="bold"
        textAnchor="middle"
        className="select-none pointer-events-none font-mono opacity-80"
      >
        {fairCuts ? `1/${totalCuts}` : '?'}
      </text>
    )}
  </g>
);
}

return (
  <svg className="w-full h-full overflow-visible" viewBox="0 0 224 224">
    { /* Decorative wrapper tray for chocolate */ }
    <rect x="5" y="45" width="214" height="134" rx="10" fill="none" stroke="#D1D5DB"
strokeWidth="2" strokeDasharray="4 4" />
    {blocks}
  </svg>
);
}

// =====
// CAKE DRAWING COMPONENT (SVG)
// =====
function CakeSVG({
  totalCuts, eatenList, fairCuts,
  draggedIndex, dragPosition,
  onPointerDown, onPointerMove, onPointerUp, onSliceClick
}) {
  const slices = [];
  const radius = 94;
  const cx = 112;
  const cy = 112;

  for (let i = 0; i < totalCuts; i++) {
    const isEaten = eatenList.includes(i);
    const isBeingDragged = draggedIndex === i;

    // Angle calculations
    let angleStart, angleEnd;

    if (fairCuts) {
      angleStart = (i * 360) / totalCuts;
      angleEnd = ((i + 1) * 360) / totalCuts;
    }
  }
}

```

```

} else {
  const unevenWeights = {
    2: [60, 300],
    3: [45, 45, 270],
    4: [30, 90, 120, 120],
    5: [30, 30, 100, 100, 100],
    6: [20, 20, 80, 80, 80, 80],
    7: [20, 20, 70, 70, 70, 70, 40],
    8: [20, 20, 60, 60, 60, 60, 30, 50]
  };

  const weights = unevenWeights[totalCuts] || Array(totalCuts).fill(360 / totalCuts);
  const sumBefore = weights.slice(0, i).reduce((a, b) => a + b, 0);
  const sumCurrent = sumBefore + weights[i];

  angleStart = (sumBefore / weights.reduce((a, b) => a + b, 0)) * 360;
  angleEnd = (sumCurrent / weights.reduce((a, b) => a + b, 0)) * 360;
}

const radStart = (angleStart - 90) * Math.PI / 180;
const radEnd = (angleEnd - 90) * Math.PI / 180;

const x1 = cx + radius * Math.cos(radStart);
const y1 = cy + radius * Math.sin(radStart);
const x2 = cx + radius * Math.cos(radEnd);
const y2 = cy + radius * Math.sin(radEnd);

const largeArcFlag = angleEnd - angleStart > 180 ? 1 : 0;
const dPath = `M ${cx} ${cy} L ${x1} ${y1} A ${radius} ${radius} 0 ${largeArcFlag} 1 ${x2}
${y2} Z`;

const midAngle = (radStart + radEnd) / 2;
const gapOffset = 4;
const offset_x = gapOffset * Math.cos(midAngle);
const offset_y = gapOffset * Math.sin(midAngle);

let transformStyle = `translate(${offset_x}px, ${offset_y}px)`;
let zIndex = 10;

if (isBeingDragged) {
  transformStyle = `translate(${dragPosition.x + offset_x}px, ${dragPosition.y + offset_y}px)
scale(1.1)`;
  zIndex = 50;
}

```

```

slices.push(
  <g
    key={i}
    style={{
      transform: transformStyle,
      transition: isBeingDragged ? 'none' : 'transform 0.3s ease-out',
      opacity: isEaten ? 0.05 : 1,
      pointerEvents: isEaten ? 'none' : 'auto',
      cursor: 'grab',
      zIndex: zIndex
    }}
    onPointerDown={(e) => onPointerDown(i, e)}
    onPointerMove={onPointerMove}
    onPointerUp={onPointerUp}
    onClick={() => onSliceClick(i)}
    className="group origin-center"
  >
    {/* Sweet Pink Frosting Layer */}
    <path
      d={dPath}
      fill="#FF8E9E"
      stroke="#DB5165"
      strokeWidth="3.5"
      className="group-hover:stroke-amber-400"
    />

    {/* Top White Whipped Cream Swirl details */}
    <path
      d={dPath}
      fill="#FFEBF0"
      transform="scale(0.88)"
      className="origin-center"
    />

    {/* Little Colorful Sprinkles (Small colored circles at random polar offsets) */}
    <circle cx={cx + (radius * 0.4) * Math.cos(midAngle - 0.1)} cy={cy + (radius * 0.4) *
Math.sin(midAngle - 0.1)} r="3" fill="#6BCB77" />
    <circle cx={cx + (radius * 0.52) * Math.cos(midAngle + 0.1)} cy={cy + (radius * 0.52) *
Math.sin(midAngle + 0.1)} r="3.5" fill="#4D96FF" />
    <circle cx={cx + (radius * 0.65) * Math.cos(midAngle - 0.25)} cy={cy + (radius * 0.65) *
Math.sin(midAngle - 0.25)} r="3" fill="#FFD93D" />

    {/* Strawberry visual topping in the middle of each cake segment! */}

```

```

    <path
      d={`M ${cx + (radius * 0.3) * Math.cos(midAngle)} ${cy + (radius * 0.3) *
Math.sin(midAngle)} Q ${cx + (radius * 0.25) * Math.cos(midAngle)} ${cy + (radius * 0.25) *
Math.sin(midAngle)} ${cx + (radius * 0.35) * Math.cos(midAngle + 0.1)} ${cy + (radius * 0.35) *
Math.sin(midAngle + 0.1)} Z`}
      fill="#D62828"
      stroke="#A81515"
      strokeWidth="1"
    />

```

```

    {/* Fraction text inside cake slice */}
    {!isEaten && (
      <text
        x={cx + (radius * 0.68) * Math.cos(midAngle)}
        y={cy + (radius * 0.68) * Math.sin(midAngle) + 4}
        fill="#9C1931"
        fontSize="9"
        fontWeight="bold"
        textAnchor="middle"
        className="select-none pointer-events-none bg-white p-1 rounded font-sans"
      >
        {fairCuts ? `1/${totalCuts}` : '?'}
      </text>
    )}
  </g>
);
}

```

```

return (
  <svg className="w-full h-full overflow-visible" viewBox="0 0 224 224">
    {slices}
  </svg>
);
}

```

```

// =====
// QUIZ VIEW COMPONENT (10 NCERT QUESTIONS)
// =====

```

```

function QuizView({ addXp }) {
  const [currentQuestion, setCurrentQuestion] = useState(0);
  const [selectedOption, setSelectedOption] = useState(null);
  const [isSubmitted, setIsSubmitted] = useState(false);
  const [score, setScore] = useState(0);
  const [showResults, setShowResults] = useState(false);

```

```

const questions = [
  {
    question: "Rahul cuts a delicious pizza into 4 equal slices. He eats 3 of them. What fraction of the pizza is left on the table?",
    options: [
      { label: "A", text: "3/4" },
      { label: "B", text: "1/4", isCorrect: true },
      { label: "C", text: "4/3" },
      { label: "D", text: "1/3" }
    ],
    explanation: "Rahul started with 4 slices total (the Denominator). He ate 3 slices, so only 1 slice is left (4 - 3 = 1). That means 1 out of 4 is left, written as 1/4!"
  },
  {
    question: "In the fraction 2/5, what does the bottom number 5 represent?",
    options: [
      { label: "A", text: "How many pieces we have eaten." },
      { label: "B", text: "The total number of equal parts the whole is divided into.", isCorrect: true },
      { label: "C", text: "How many pieces are left on the plate." },
      { label: "D", text: "The size of the monster's mouth." }
    ],
    explanation: "The bottom number (Denominator) ALWAYS tells us the total equal parts we cut the whole food into."
  },
  {
    question: "If a rectangular chocolate bar is broken into 8 equal pieces, each individual block is called one-_____ of the whole bar.",
    options: [
      { label: "A", text: "Half" },
      { label: "B", text: "Quarter" },
      { label: "C", text: "Eighth", isCorrect: true },
      { label: "D", text: "Sixth" }
    ],
    explanation: "When an item is broken into 8 identical equal chunks, each chunk is mathematically called an 'eighth' (1/8)."
  },
  {
    question: "Simran wants to divide a watermelon slice. She cuts it into 1 giant piece and 2 tiny pieces. Can we write this as the mathematical fraction 1/3?",
    options: [
      { label: "A", text: "Yes, because there are 3 pieces total." },

```

```

    { label: "B", text: "No, because standard fractions require all parts to be exactly equal.",
isCorrect: true }
  ],
  explanation: "Fractions can only represent parts that are shared FAIRLY. If parts are unequal
in size, we cannot represent them as basic mathematical fractions!"
},
{
  question: "What happens to the actual physical size of each pizza slice as the Denominator
(bottom number) gets larger?",
  options: [
    { label: "A", text: "The slices get larger." },
    { label: "B", text: "The slices get smaller.", isCorrect: true },
    { label: "C", text: "The slices stay exactly the same size." }
  ],
  explanation: "Think of sharing! If you cut a pizza for 2 people, the slices are huge. But if you
cut it for 8 people, the cuts are more and the slices get much smaller!"
},
{
  question: "Which of the following fractions represents the exact same thing as '1 Whole
Object'?",
  options: [
    { label: "A", text: "1/4" },
    { label: "B", text: "0/4" },
    { label: "C", text: "4/4", isCorrect: true },
    { label: "D", text: "4/1" }
  ],
  explanation: "When you have 4 out of 4 parts (4/4), you have taken all the pieces of the
item, which means you have 1 Whole!"
},
{
  question: "A chocolate bar has 6 identical squares. Amit eats 2 squares in the morning and
3 squares in the evening. What total fraction of the bar has he eaten?",
  options: [
    { label: "A", text: "5/6", isCorrect: true },
    { label: "B", text: "2/6" },
    { label: "C", text: "3/6" },
    { label: "D", text: "1/6" }
  ],
  explanation: "Amit ate 2 parts and then 3 parts, so he ate  $2 + 3 = 5$  parts total out of 6.
This is represented as 5/6."
},
{
  question: "If you see the fraction  $3/8$  on a blackboard, what is the mathematical term for the
number 3?",

```



```

    options: [
      { label: "A", text: "Denominator" },
      { label: "B", text: "Numerator", isCorrect: true },
      { label: "C", text: "Vinculum" },
      { label: "D", text: "Whole number" }
    ],
    explanation: "The top number above the line is the Numerator. It tells us how many pieces are taken or shaded."
  },
  {
    question: "Your mother gives you 2/4 of a circular cake, and your father gives you another 2/4 of the same cake. How much cake do you have now?",
    options: [
      { label: "A", text: "4/8 of a cake" },
      { label: "B", text: "1 Whole cake", isCorrect: true },
      { label: "C", text: "2/4 of a cake" }
    ],
    explanation: "2/4 + 2/4 = 4/4. Since the numerator matches the denominator, you now have exactly 1 complete Whole cake!"
  },
  {
    question: "What is the special name for the horizontal line that separates the numerator from the denominator?",
    options: [
      { label: "A", text: "The Slice Line" },
      { label: "B", text: "The Vinculum", isCorrect: true },
      { label: "C", text: "The Denominator Bar" },
      { label: "D", text: "The Divider" }
    ],
    explanation: "In mathematics, the line segment separating the numerator and the denominator is called the 'vinculum'!"
  }
];

```

```

const handleOptionSelect = (index) => {
  if (isSubmitted) return;
  playSound('click');
  setSelectedOption(index);
};

```

```

const handleSubmit = () => {
  if (selectedOption === null) return;
  setIsSubmitted(true);
  const isCorrect = questions[currentQuestion].options[selectedOption].isCorrect;

```

```

    if (isCorrect) {
      playSound('win');
      setScore(prev => prev + 1);
      addXp(10);
    } else {
      playSound('fail');
    }
  };

  const handleNext = () => {
    setSelectedOption(null);
    setIsSubmitted(false);
    if (currentQuestion < questions.length - 1) {
      setCurrentQuestion(prev => prev + 1);
    } else {
      setShowResults(true);
    }
  };

  const resetQuiz = () => {
    playSound('reset');
    setCurrentQuestion(0);
    setSelectedOption(null);
    setIsSubmitted(false);
    setScore(0);
    setShowResults(false);
  };

  return (
    <div className="max-w-3xl mx-auto bg-white border border-slate-200 rounded-3xl p-6 shadow-sm">

      {/* Progress indicators */}
      <div className="flex items-center justify-between mb-6 border-b border-slate-100 pb-4">
        <span className="text-xs font-bold text-slate-400 uppercase tracking-wider">
          Practice Desk • Question {currentQuestion + 1} of {questions.length}
        </span>
        <div className="w-32 bg-slate-100 rounded-full h-2">
          <div
            className="bg-orange-500 h-2 rounded-full transition-all"
            style={{ width: `${((currentQuestion + (isSubmitted ? 1 : 0)) / questions.length) * 100}%`
          }}
        </div>
      </div>

```

```

    </div>
  </div>

  {!showResults ? (
    <div>
      {/* Question text */}
      <h2 className="text-lg font-bold text-slate-800 leading-snug mb-5">
        {questions[currentQuestion].question}
      </h2>

      {/* Graphical Support Diagram (Simulating NCERT graphics) */}
      <div className="my-6 p-4 bg-slate-50 border border-slate-200 rounded-2xl flex
justify-center items-center">
        {currentQuestion === 0 && (
          <div className="flex items-center gap-6">
            <div className="text-center">
              <p className="text-[10px] text-slate-400 font-bold mb-1">RAHUL'S CAKE (3/4
eaten)</p>
              <svg className="w-24 h-24" viewBox="0 0 100 100">
                <circle cx="50" cy="50" r="40" fill="#FFEBF0" stroke="#DB5165" strokeWidth="2"
/>

                {/* Eaten slices marked faint */}
                <path d="M 50 50 L 50 10 A 40 40 0 1 1 10 50 Z" fill="#FFEBF0" opacity="0.4" />
                {/* Left slice highlighted */}
                <path d="M 50 50 L 10 50 A 40 40 0 0 1 50 10 Z" fill="#FF8E9E" stroke="#DB5165"
strokeWidth="2" />
              </svg>
            </div>
          </div>
        )}
        {currentQuestion === 1 && (
          <div className="text-center font-bold text-3xl flex flex-col items-center">
            <span className="text-orange-500">2</span>
            <div className="w-16 h-1 bg-slate-400 my-1" />
            <span className="text-teal-500 text-4xl underline decoration-wavy
decoration-teal-200">5</span>
          </div>
        )}
        {currentQuestion === 3 && (
          <div className="flex gap-4 items-center">
            <div className="text-center">
              <p className="text-[10px] text-slate-400 font-bold mb-1">FAIR CUTS (Equal)</p>
              <div className="flex gap-1">
                <div className="w-10 h-10 bg-amber-200 border border-amber-400" />

```

```

        <div className="w-10 h-10 bg-amber-200 border border-amber-400" />
        <div className="w-10 h-10 bg-amber-200 border border-amber-400" />
    </div>
</div>
<div className="text-center">
    <p className="text-[10px] text-slate-400 font-bold mb-1">UNFAIR CUTS
(Unequal)</p>
    <div className="flex gap-1 items-end">
        <div className="w-6 h-10 bg-rose-100 border border-rose-300" />
        <div className="w-14 h-10 bg-rose-100 border border-rose-300" />
        <div className="w-4 h-10 bg-rose-100 border border-rose-300" />
    </div>
</div>
</div>
)}
{currentQuestion === 4 && (
    <div className="flex gap-8 items-center justify-center">
        <div className="text-center">
            <span className="text-xs text-slate-400 block mb-1">2 Slices (Large)</span>
            <div className="w-16 h-16 rounded-full border-2 border-slate-300 relative
overflow-hidden bg-orange-100">
                <div className="absolute top-0 bottom-0 left-1/2 w-0.5 bg-slate-400" />
            </div>
        </div>
        <div className="text-center">
            <span className="text-xs text-slate-400 block mb-1">8 Slices (Tiny!)</span>
            <div className="w-16 h-16 rounded-full border-2 border-slate-300 relative
overflow-hidden bg-orange-100">
                <div className="absolute top-0 bottom-0 left-1/2 w-0.5 bg-slate-400" />
                <div className="absolute left-0 right-0 top-1/2 h-0.5 bg-slate-400" />
                <div className="absolute inset-0 rotate-45 flex items-center justify-center">
                    <div className="w-full h-0.5 bg-slate-400" />
                </div>
                <div className="absolute inset-0 -rotate-45 flex items-center justify-center">
                    <div className="w-full h-0.5 bg-slate-400" />
                </div>
            </div>
        </div>
    </div>
)}
{ /* Default graphic for others */ }
{currentQuestion !== 0 && currentQuestion !== 1 && currentQuestion !== 3 &&
currentQuestion !== 4 && (
    <div className="text-slate-400 text-xs font-bold flex items-center gap-2">

```



Think Visually • Real Foods are Fractions in Action!

```

    </div>
  })
</div>

{/* Options Grid */}
<div className="grid grid-cols-1 gap-3">
  {questions[currentQuestion].options.map((opt, idx) => {
    const isSelected = selectedOption === idx;
    let btnClass = "bg-slate-50 hover:bg-slate-100 border-slate-200 text-slate-700";

    if (isSelected) {
      btnClass = "bg-orange-50 border-orange-400 text-orange-900 ring-2
ring-orange-200";
    }

    if (isSubmitted) {
      if (opt.isCorrect) {
        btnClass = "bg-emerald-50 border-emerald-400 text-emerald-800 ring-2
ring-emerald-200 font-semibold";
      } else if (isSelected) {
        btnClass = "bg-rose-50 border-rose-400 text-rose-800 line-through
decoration-slate-400";
      } else {
        btnClass = "bg-slate-50 border-slate-200 text-slate-400 opacity-60";
      }
    }

    return (
      <button
        key={idx}
        onClick={() => handleOptionSelect(idx)}
        className={`w-full text-left p-4 rounded-2xl border transition-all flex items-center
gap-3.5 ${btnClass}`}
        disabled={isSubmitted}
      >
        <span className={`w-7 h-7 rounded-full flex items-center justify-center text-xs
font-bold ${
          isSelected ? 'bg-orange-500 text-white' : 'bg-slate-200 text-slate-600'
        }`}>
          {opt.label}
        </span>
        <span className="text-sm font-medium">{opt.text}</span>
      </button>
    )
  })
}

```

```

    );
  }}}
</div>

{/* Action buttons */}
<div className="mt-6 flex justify-end gap-3">
  {!isSubmitted ? (
    <button
      onClick={handleSubmit}
      disabled={selectedOption === null}
      className={`px-6 py-3 rounded-2xl text-sm font-bold shadow-md transition-all ${
        selectedOption === null
          ? 'bg-slate-100 text-slate-400 border border-slate-200 cursor-not-allowed'
          : 'bg-slate-900 hover:bg-slate-800 text-white hover:shadow-lg'
        }`}
    >
      Submit Answer
    </button>
  ) : (
    <button
      onClick={handleNext}
      className="px-6 py-3 rounded-2xl text-sm font-bold bg-orange-500
hover:bg-orange-600 text-white shadow-md hover:shadow-lg transition-all"
    >
      {currentQuestion < questions.length - 1 ? "Next Question" : "See Final Score"}
    </button>
  )}
</div>

{/* Explanations banner */}
{isSubmitted && (
  <div className="mt-5 p-4 bg-blue-50 border border-blue-100 rounded-2xl">
    <h4 className="text-xs font-bold text-blue-700 uppercase tracking-widest mb-1">
Teacher's Tip Box:</h4>
    <p className="text-xs text-blue-800 leading-relaxed">
      {questions[currentQuestion].explanation}
    </p>
  </div>
)}
</div>
) : (
  /* Results View */
  <div className="text-center py-8">
    <span className="text-6xl block mb-4">🏆</span>

```

```

<h2 className="text-2xl font-extrabold text-slate-900">Quiz Completed!</h2>
<p className="text-sm text-slate-500 mt-1">Excellent practice. You did really well!</p>

  { /* Score display circle */ }
  <div className="w-36 h-36 rounded-full border-4 border-orange-500 bg-orange-50 flex
flex-col items-center justify-center mx-auto my-6 shadow-inner">
    <span className="text-3xl font-extrabold text-orange-600">{score} /
{questions.length}</span>
    <span className="text-[10px] font-bold text-orange-400 uppercase
tracking-wider">CORRECT ANSWERS</span>
  </div>

  <p className="text-sm text-slate-600 px-6 max-w-md mx-auto leading-relaxed">
    {score >= 8
      ? "🌟 Incredible! You are a qualified master fractions chef. Your sharing skills are
perfect!"
      : "🌟 Nice try! Review the visual hints in the Lesson tab to make your sharing skills
even better!"}
  </p>

  <div className="mt-8 flex justify-center gap-3">
    <button
      onClick={resetQuiz}
      className="px-6 py-3 rounded-2xl text-sm font-bold border border-slate-200
hover:bg-slate-50 text-slate-700 transition-all"
    >
      🔄 Try Again
    </button>
    <button
      onClick={() => { playSound('click'); resetQuiz(); }}
      className="px-6 py-3 rounded-2xl text-sm font-bold bg-slate-900 text-white
hover:bg-slate-800 transition-all"
    >
      Back to Start
    </button>
  </div>
</div>
  )}
}

// =====

```

```
// FUN LESSONS & DIY VIEW COMPONENT
// =====
function LessonView() {
  const [activeFaq, setActiveFaq] = useState(null);

  const toggleFaq = (idx) => {
    playSound('click');
    setActiveFaq(activeFaq === idx ? null : idx);
  };

  return (
    <div className="max-w-4xl mx-auto space-y-8 select-none">

      {/* Visual Header Card */}
      <div className="bg-gradient-to-r from-orange-400 to-amber-500 rounded-3xl p-6 text-white shadow-md">
        <h2 className="text-2xl font-extrabold tracking-tight">Let's Learn Fractions!</h2>
        <p className="text-sm text-orange-50 opacity-90 mt-1">
          Have you ever shared a fresh cake or a chocolate bar fairly? Fractions are exactly that!
        </p>
      </div>

      {/* CORE CONCEPT SECTION */}
      <div className="grid grid-cols-1 md:grid-cols-2 gap-6 items-start">

        <div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-sm">
          <h3 className="font-extrabold text-slate-900 flex items-center gap-2 mb-3">
            <span className="text-lg">🍷</span> What is a Fraction?
          </h3>
          <p className="text-xs text-slate-600 leading-relaxed">
            A fraction represents <strong>parts of a single, whole item</strong> (like a pizza or
            cake).
            When we cut something into several equal pieces, we use fractions to write exactly how
            many parts we have.
          </p>

          {/* Fraction illustration */}
          <div className="mt-5 p-4 bg-orange-50 rounded-2xl flex items-center justify-around">
            <div className="text-center font-bold text-xl flex flex-col items-center">
              <span className="text-orange-600 text-2xl" title="Numerator">3</span>
              <div className="w-12 h-0.5 bg-slate-400 my-1.5 />
              <span className="text-teal-600 text-2xl" title="Denominator">4</span>
            </div>
            <div className="text-xs text-slate-500 space-y-1.5 border-l border-orange-200 pl-4">

```


<p> 🍌 <strong className="text-orange-600">Numerator (Top): How many pieces we are choosing/eating.</p>

<p> 🍌 <strong className="text-teal-600">Denominator (Bottom): Total pieces we cut the whole item into.</p>

</div>

</div>

</div>

<div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-sm">

<h3 className="font-extrabold text-slate-900 flex items-center gap-2 mb-3">

 💡 The Denominator Memory Trick!

</h3>

<div className="bg-amber-50 border border-amber-200 p-4 rounded-2xl mb-3">

<p className="text-xs font-bold text-amber-800">

How do I remember which is which?

</p>

<p className="text-xs text-amber-700 mt-1 leading-relaxed">

Think of the letter <strong className="text-sm font-black underline decoration-amber-400 text-slate-900">D!

Denominator starts with D, which stands for Down! The number pointing Down at the bottom is always the Denominator!

</p>

</div>

<p className="text-[11px] text-slate-400 italic">

So if you see 1/2, the number 2 is Down, making it the Denominator!

</p>

</div>

</div>

{/* DIY KITCHEN ACTIVITY CORNER */}

<div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-sm">

<div className="flex items-center gap-3 mb-4">

 🏠

</div>

<h3 className="font-extrabold text-slate-950 text-base">DIY Home Lab: The Kitchen Paper Slicer!</h3>

<p className="text-xs text-slate-500">Perfect weekend activity with your family!</p>

</div>

</div>

```

<div className="grid grid-cols-1 md:grid-cols-3 gap-4">
  <div className="p-3 bg-slate-50 border border-slate-100 rounded-xl text-center">
    <div className="w-8 h-8 rounded-full bg-slate-200 flex items-center justify-center
font-bold text-xs mx-auto mb-2 text-slate-700">1</div>
    <h4 className="font-bold text-xs text-slate-800 mb-1">Get Paper Plates</h4>
    <p className="text-[11px] text-slate-500 leading-normal">Take 2 circular paper plates
or draw a large circle on plain sheets of white paper.</p>
  </div>
  <div className="p-3 bg-slate-50 border border-slate-100 rounded-xl text-center">
    <div className="w-8 h-8 rounded-full bg-slate-200 flex items-center justify-center
font-bold text-xs mx-auto mb-2 text-slate-700">2</div>
    <h4 className="font-bold text-xs text-slate-800 mb-1">Fold & Slice</h4>
    <p className="text-[11px] text-slate-500 leading-normal">Fold one paper perfectly in
half. Open it up, draw a line on the fold crease. You now have two halves (1/2)!</p>
  </div>
  <div className="p-3 bg-slate-50 border border-slate-100 rounded-xl text-center">
    <div className="w-8 h-8 rounded-full bg-slate-200 flex items-center justify-center
font-bold text-xs mx-auto mb-2 text-slate-700">3</div>
    <h4 className="font-bold text-xs text-slate-800 mb-1">Create Quarters</h4>
    <p className="text-[11px] text-slate-500 leading-normal">Fold the other plate in half,
then fold it in half once again. Open it up. You have 4 equal quarters (1/4)!</p>
  </div>
</div>
</div>

```

```

{/* STUDENT FAQs (INTERACTIVE TOY ACCORDIONS) */}

```

```

<div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-sm">
  <h3 className="font-extrabold text-slate-900 flex items-center gap-2 mb-4">
    ? Frequently Asked Questions (Fractions Edition)
  </h3>

```

```

<div className="space-y-2">
  {[
    {
      q: "Is 1/4 bigger than 1/2 because 4 is bigger than 2?",
      a: "Nope! Remember, the denominator (bottom) tells us how many cuts we made. More
cuts make pieces smaller! So, 1/2 of a pizza is much larger than a tiny 1/4 slice because the
pizza was only divided into 2 parts instead of 4."
    },
    {
      q: "Can a fraction ever have a 0 at the bottom (denominator)?",
      a: "No! You can't cut an object into zero pieces if it exists right in front of you. That
makes it mathematically undefined."
    },
  ]}

```

```

    {
      q: "What does it mean if the top number is equal to the bottom number (like 4/4)?",
      a: "It means you have taken all the pieces! 4/4 represents exactly 1 Whole object."
    },
    {
      q: "Why do we even need fractions in real life?",
      a: "Because life isn't always made of whole things! We need fractions to measure
      baking ingredients, look at clocks (like quarter-past-six), share pizzas with friends, and measure
      height or length."
    }
  ].map((faq, idx) => (
    <div key={idx} className="border border-slate-100 rounded-2xl overflow-hidden">
      <button
        onClick={() => toggleFaq(idx)}
        className="w-full text-left p-4 bg-slate-50 hover:bg-slate-100/75 font-bold text-xs
sm:text-sm text-slate-800 flex items-center justify-between"
      >
        <span>{faq.q}</span>
        <span className="text-slate-400 font-bold">{activeFaq === idx ? "▲" : "▼"}</span>
      </button>
      {activeFaq === idx && (
        <div className="p-4 bg-white border-t border-slate-100 text-xs text-slate-600
leading-relaxed">
          {faq.a}
        </div>
      )}
    </div>
  ))}
</div>
);
}

```

Perimeter and Area5–6

```

import React, { useState, useEffect, useRef } from 'react';

// Custom lightweight sound synthesizer for agricultural/gardening themes
const playSound = (type) => {
  try {
    const AudioContext = window.AudioContext || window.webkitAudioContext;
    if (!AudioContext) return;
    const ctx = new AudioContext();
    const now = ctx.currentTime;

    switch(type) {
      case 'hammer': // Hammering fence post sound
        const osc1 = ctx.createOscillator();
        const gain1 = ctx.createGain();
        osc1.type = 'triangle';
        osc1.frequency.setValueAtTime(120, now);
        osc1.frequency.exponentialRampToValueAtTime(10, now + 0.1);
        gain1.gain.setValueAtTime(0.4, now);
        gain1.gain.exponentialRampToValueAtTime(0.01, now + 0.1);
        osc1.connect(gain1);
        gain1.connect(ctx.destination);
        osc1.start();
        osc1.stop(now + 0.1);
        break;
      case 'plant': // Soft water bubble/pop sound of planting flowers
        const osc2 = ctx.createOscillator();
        const gain2 = ctx.createGain();
        osc2.type = 'sine';
        osc2.frequency.setValueAtTime(300, now);
        osc2.frequency.exponentialRampToValueAtTime(600, now + 0.12);
        gain2.gain.setValueAtTime(0.2, now);
        gain2.gain.exponentialRampToValueAtTime(0.01, now + 0.12);
        osc2.connect(gain2);
        gain2.connect(ctx.destination);
        osc2.start();
        osc2.stop(now + 0.12);
    }
  }
};

```

```

    break;
case 'win': // Beautiful celebration chord
    [261.63, 329.63, 392.00, 523.25].forEach((freq, index) => {
        const osc = ctx.createOscillator();
        const gain = ctx.createGain();
        osc.type = 'sine';
        osc.frequency.setValueAtTime(freq, now + index * 0.08);
        gain.gain.setValueAtTime(0.15, now + index * 0.08);
        gain.gain.exponentialRampToValueAtTime(0.01, now + index * 0.08 +
0.3);

        osc.connect(gain);
        gain.connect(ctx.destination);
        osc.start(now + index * 0.08);
        osc.stop(now + index * 0.08 + 0.3);
    });
    break;
case 'click': // Standard UI click
    const oscClick = ctx.createOscillator();
    const gainClick = ctx.createGain();
    oscClick.type = 'sine';
    oscClick.frequency.setValueAtTime(500, now);
    oscClick.frequency.exponentialRampToValueAtTime(250, now + 0.04);
    gainClick.gain.setValueAtTime(0.05, now);
    gainClick.gain.exponentialRampToValueAtTime(0.01, now + 0.04);
    oscClick.connect(gainClick);
    gainClick.connect(ctx.destination);
    oscClick.start();
    oscClick.stop(now + 0.04);
    break;
case 'fail': // Disappointment warning buzz
    const oscF = ctx.createOscillator();
    const gainF = ctx.createGain();
    oscF.type = 'sawtooth';
    oscF.frequency.setValueAtTime(150, now);
    oscF.frequency.linearRampToValueAtTime(90, now + 0.25);
    gainF.gain.setValueAtTime(0.12, now);

```

```

    gainF.gain.exponentialRampToValueAtTime(0.01, now + 0.25);
    oscF.connect(gainF);
    gainF.connect(ctx.destination);
    oscF.start();
    oscF.stop(now + 0.25);
    break;
case 'reset': // Gardening sweep
    const oscR = ctx.createOscillator();
    const gainR = ctx.createGain();
    oscR.type = 'sine';
    oscR.frequency.setValueAtTime(400, now);
    oscR.frequency.exponentialRampToValueAtTime(150, now + 0.4);
    gainR.gain.setValueAtTime(0.1, now);
    gainR.gain.exponentialRampToValueAtTime(0.01, now + 0.4);
    oscR.connect(gainR);
    gainR.connect(ctx.destination);
    oscR.start();
    oscR.stop(now + 0.4);
    break;
default:
    break;
}
} catch (e) {
    // Audio Context blocked or not supported
}
};

export default function App() {
    const [activeTab, setActiveTab] = useState('simulator'); // 'simulator' |
    'quiz' | 'lessons'
    const [xp, setXp] = useState(150);

    const addXp = (amount) => {
        setXp(prev => prev + amount);
    };
};

```

```

return (
  <div className="min-h-screen bg-emerald-50/30 text-slate-800 font-sans
antialiased selection:bg-emerald-200">
    {/* Brand Header */}
    <header className="bg-white border-b border-emerald-100 sticky top-0
z-30 shadow-sm">
      <div className="max-w-6xl mx-auto px-4 py-3 flex flex-col
sm:flex-row items-center justify-between gap-3">
        <div className="flex items-center gap-3">
          <span className="text-3xl" role="img" aria-label="Garden
Hat"><img alt="Garden Hat icon" data-bbox="168 315 192 332"/></span>
          <div>
            <h1 className="text-xl font-bold text-slate-900
tracking-tight flex items-center gap-2">
              The Magic Garden Resizer
              <span className="text-xs bg-emerald-100 text-emerald-800
font-semibold px-2.5 py-0.5 rounded-full border border-emerald-200">
                NCERT Classes 5-6
              </span>
            </h1>
            <p className="text-xs text-slate-500">Visual Math Simulator:
Discover Perimeter vs. Area Instantly</p>
          </div>
        </div>
      </div>

      <div className="flex items-center gap-4">
        {/* XP Tracker */}
        <div className="flex items-center gap-2 bg-amber-50 border
border-amber-200 px-3.5 py-1.5 rounded-full shadow-inner">
          <span className="text-lg">✨</span>
          <span className="text-sm font-bold text-amber-700">{xp} <span
className="text-xs font-semibold text-amber-600">XP Points</span></span>
        </div>

        {/* Navigation Tabs */}

```



```

        <nav className="flex bg-slate-100 p-1 rounded-xl border
border-slate-200">
            <button
                onClick={() => { setActiveTab('simulator');
playSound('click'); }}
                className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm
font-semibold transition-all ${
                    activeTab === 'simulator'
                        ? 'bg-white text-emerald-700 shadow-sm'
                        : 'text-slate-600 hover:text-slate-900'
                }}`
            >
                🎮 Play Garden
            </button>
            <button
                onClick={() => { setActiveTab('quiz'); playSound('click');
}}
                className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm
font-semibold transition-all ${
                    activeTab === 'quiz'
                        ? 'bg-white text-emerald-700 shadow-sm'
                        : 'text-slate-600 hover:text-slate-900'
                }}`
            >
                📝 Math Quiz
            </button>
            <button
                onClick={() => { setActiveTab('lessons');
playSound('click'); }}
                className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm
font-semibold transition-all ${
                    activeTab === 'lessons'
                        ? 'bg-white text-emerald-700 shadow-sm'
                        : 'text-slate-600 hover:text-slate-900'
                }}`
            >

```

```

         Fun Lesson
      </button>
    </nav>
  </div>
</div>
</header>

  { /* Main Wrapper */ }

  <main className="max-w-6xl mx-auto px-4 py-6">
    {activeTab === 'simulator' && <SimulatorView addXp={addXp} />}
    {activeTab === 'quiz' && <QuizView addXp={addXp} />}
    {activeTab === 'lessons' && <LessonView />}
  </main>

  <footer className="bg-slate-900 text-slate-400 py-6 mt-12 border-t
border-slate-800 text-center text-xs">
    <p>🌻 Hand-designed for Budding Mathematicians • Designed according
to CBSE/NCERT Math-Magic guidelines.</p>
    <p className="mt-1 text-slate-600">Tip: Change length & width with
sliders or complete challenges to unlock achievements!</p>
  </footer>
</div>
);
}

// =====
// SIMULATOR COMPONENT (THE GARDEN RESIZER)
// =====
function SimulatorView({ addXp }) {
  const [gardenWidth, setGardenWidth] = useState(6); // range 1-10
  const [gardenLength, setGardenLength] = useState(4); // range 1-10
  const [isChallengeMode, setIsChallengeMode] = useState(false);
  const [activeChallengeIdx, setActiveChallengeIdx] = useState(0);
  const [tipMessage, setTipMessage] = useState("Drag the sliders to watch
how the Fence (Perimeter) and Grass (Area) change!");
  const [confetti, setConfetti] = useState([]);

```

```

const [challengeSuccess, setChallengeSuccess] = useState(false);

const challenges = [
  {
    title: "Design a Small Square Garden ,
    description: "Build a square garden where both sides are equal, and has exactly an Area of 16 square meters.",
    targetWidth: 4,
    targetLength: 4,
    hint: "Remember, a square has equal sides. If Area is 16, what times itself is 16? Try 4 × 4!"
  },
  {
    title: "The Save-the-Fence Challenge ,
    description: "Design a garden with an Area of exactly 12 square meters, but make the Perimeter as SMALL as possible to save on fence ropes!",
    isCustomValidator: (w, l) => {
      const area = w * l;
      const perimeter = 2 * (w + l);
      // Possible factors of 12: 1x12 (P=26), 2x6 (P=16), 3x4 (P=14)
      // Smallest perimeter is when dimensions are closest: 3x4 or 4x3 (P=14)
      return area === 12 && perimeter === 14;
    },
    hint: "The factors of 12 are: 1×12, 2×6, and 3×4. Try building a 3×4 or 4×3 garden to see which needs less fence!"
  },
  {
    title: "The Max Grass Patch ,
    description: "You have exactly 20 meters of fence boundary (Perimeter). Build a rectangle that encloses the LARGEST possible Area!",
    isCustomValidator: (w, l) => {
      const perimeter = 2 * (w + l);
      const area = w * l;
      // P=20 means W+L = 10.

```

```

        // Possibilities: 1x9 (A=9), 2x8 (A=16), 3x7 (A=21), 4x6 (A=24),
        5x5 (A=25)
        // Largest area is 5x5 (Square)
        return perimeter === 20 && area === 25;
    },
    hint: "A square always encloses the largest area for any given
    perimeter! If Perimeter is 20, each of the 4 sides should be equal."
  }
];

// Auto sound feedback on adjustments
useEffect(() => {
  playSound('plant');
}, [gardenWidth, gardenLength]);

// Check Challenge Requirements
useEffect(() => {
  if (!isChallengeMode) return;
  const current = challenges[activeChallengeIdx];
  let passed = false;

  if (current.isCustomValidator) {
    passed = current.isCustomValidator(gardenWidth, gardenLength);
  } else {
    passed = gardenWidth === current.targetWidth && gardenLength ===
current.targetLength;
  }

  if (passed) {
    setChallengeSuccess(true);
    setTipMessage("🎉 Correct! You perfectly designed the target garden!
Click Next to continue.");
    triggerConfetti();
    playSound('win');
    addXp(30);
  } else {

```

```

    setChallengeSuccess(false);
    setTipMessage(`💡 Tip: ${current.hint}`);
  }
}, [gardenWidth, gardenLength, isChallengeMode, activeChallengeIdx]);

const triggerConfetti = () => {
  const items = Array.from({ length: 30 }).map((_, i) => ({
    id: i,
    x: Math.random() * 100,
    y: -20 - Math.random() * 50,
    size: Math.random() * 8 + 6,
    color: ['#10B981', '#34D399', '#FBBF24', '#F59E0B',
'#F87171'][Math.floor(Math.random() * 5)]
  }));
  setConfetti(items);
  setTimeout(() => setConfetti([], 3000));
};

const handleNextChallenge = () => {
  playSound('click');
  setChallengeSuccess(false);
  if (activeChallengeIdx < challenges.length - 1) {
    setActiveChallengeIdx(prev => prev + 1);
  } else {
    setIsChallengeMode(false);
    setActiveChallengeIdx(0);
    setTipMessage("Amazing! You finished all of the Master Gardener's
challenges!");
  }
};

const resetGarden = () => {
  playSound('reset');
  setGardenWidth(6);
  setGardenLength(4);
  setChallengeSuccess(false);

```

```

};

// Calculations
const calculatedPerimeter = 2 * (gardenWidth + gardenLength);
const calculatedArea = gardenWidth * gardenLength;

return (
  <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 items-start
relative">

    {/* Confetti celebration overlays */}
    {confetti.map((c) => (
      <div
        key={c.id}
        className="absolute pointer-events-none rounded-full
animate-bounce z-50 transition-all duration-1000 ease-out"
        style={{
          left: `${c.x}%`,
          top: `${c.y + 300}px`,
          width: `${c.size}px`,
          height: `${c.size}px`,
          backgroundColor: c.color,
          opacity: 0.8,
        }}
      />
    ))}

    {/* LEFT: VISUAL GARDEN CANVAS (8 Cols) */}
    <div className="lg:col-span-8 bg-white border border-slate-200
rounded-3xl p-6 shadow-sm flex flex-col justify-between min-h-[550px]">

      {/* Toggle Mode Headbar */}
      <div className="flex flex-wrap items-center justify-between gap-4
border-b border-slate-100 pb-4 mb-4">
        <div className="flex items-center gap-2">

```

```

        <span className="text-xs font-bold text-slate-500 uppercase
tracking-widest">Select Mode:</span>
        <div className="bg-slate-100 p-1 rounded-xl border
border-slate-200 flex">
            <button
                onClick={() => { playSound('click');
setIsChallengeMode(false); }}
                className={`px-3 py-1.5 rounded-lg text-xs font-bold
transition-all ${
                    !isChallengeMode ? 'bg-white text-emerald-700 shadow-sm'
: 'text-slate-600'
                }}`>
                >
                🌳 Sandbox Mode
            </button>
            <button
                onClick={() => { playSound('click');
setIsChallengeMode(true); setChallengeSuccess(false); }}
                className={`px-3 py-1.5 rounded-lg text-xs font-bold
transition-all ${
                    isChallengeMode ? 'bg-white text-emerald-700 shadow-sm' :
'text-slate-600'
                }}`>
                >
                🏆 Garden Challenges
            </button>
        </div>
    </div>

    {isChallengeMode && (
        <span className="text-xs font-extrabold px-3 py-1 bg-amber-50
text-amber-700 border border-amber-200 rounded-full">
            Level {activeChallengeIdx + 1} of {challenges.length}
        </span>
    )}
</div>

```

```

    { /* ACTIVE CHALLENGE CARD IN CHALLENGE MODE */
    {isChallengeMode && (
        <div className="mb-4 p-4 bg-emerald-50/50 border
border-emerald-100 rounded-2xl animate-fade-in">
            <h3 className="text-sm font-bold text-emerald-950 flex
items-center gap-2">
                <span>🎯 Goal:</span>
                {challenges[activeChallengeIdx].title}
            </h3>
            <p className="text-xs text-slate-600 mt-1 leading-relaxed">
                {challenges[activeChallengeIdx].description}
            </p>
        </div>
    ) }

    { /* THE VISUAL GARDEN GRIDS DISPLAY */
    <div className="relative flex-1 bg-slate-50 rounded-2xl border-2
border-dashed border-slate-200 p-6 flex flex-col items-center
justify-center min-h-[320px] select-none">

        <div className="w-full max-w-lg relative">
            { /* Visual SVG representing Garden & Enclosure */
            <svg
                className="w-full h-auto max-h-[320px] overflow-visible"
                viewBox="0 0 440 380"
                fill="none"
                xmlns="http://www.w3.org/2000/svg"
            >
                { /* Outer Boundary Grid representation (Max 10x10) */
                <defs>
                    <pattern id="grid" width="36" height="36"
patternUnits="userSpaceOnUse">
                        <path d="M 36 0 L 0 0 0 36" fill="none" stroke="#E2E8F0"
strokeWidth="1" />
                    </pattern>

```



```

</defs>

{/* Standard backing board */}
<rect x="20" y="20" width="360" height="360"
fill="url(#grid)" stroke="#CBD5E1" strokeWidth="2" rx="4" />

{/* Garden Active Filled Rect (Area) */}
<rect
  x="20"
  y="20"
  width={gardenWidth * 36}
  height={gardenLength * 36}
  fill="#A7F3D0" // Soft light green representing lawn
  className="transition-all duration-300 ease-out"
  opacity="0.85"
/>

{/* Render dynamic individual decorative flowers inside the
active area */}
{Array.from({ length: gardenWidth }).map((_, col) =>
  Array.from({ length: gardenLength }).map((_, row) => (
    <g
      key={`flower-${col}-${row}`}
      transform={`translate(${20 + col * 36 + 18}, ${20 + row
* 36 + 18})`}

      className="transition-all duration-300 ease-out"
    >
      {/* Flower Petals */}
      <circle cx="-5" cy="0" r="4.5" fill="#F472B6" />
      <circle cx="5" cy="0" r="4.5" fill="#F472B6" />
      <circle cx="0" cy="-5" r="4.5" fill="#F472B6" />
      <circle cx="0" cy="5" r="4.5" fill="#F472B6" />
      {/* Flower Core */}
      <circle cx="0" cy="0" r="3.5" fill="#FBBF24" />
    </g>
  ))
}

```

```

    })

    { /* Dynamic Outer Fence Border (Perimeter) */
    <rect
      x="20"
      y="20"
      width={gardenWidth * 36}
      height={gardenLength * 36}
      stroke="#B45309" // Thick brown fence border line
      strokeWidth="5"
      strokeLinecap="round"
      className="transition-all duration-300 ease-out"
    />

    { /* Small circles acting as fence posts at each intersection
    along the perimeter */
      {Array.from({ length: gardenWidth + 1 }).map((_, col) => (
        <g key={`post-top-${col}`} transform={`translate(${20 + col
        * 36}, 20)`} className="transition-all duration-300 ease-out">
          <circle cx="0" cy="0" r="5" fill="#451A03"
stroke="#FDE047" strokeWidth="1" />
        </g>
      ))}
      {Array.from({ length: gardenWidth + 1 }).map((_, col) => (
        <g key={`post-bottom-${col}`} transform={`translate(${20 +
        col * 36}, ${20 + gardenLength * 36})`} className="transition-all
duration-300 ease-out">
          <circle cx="0" cy="0" r="5" fill="#451A03"
stroke="#FDE047" strokeWidth="1" />
        </g>
      ))}
      {Array.from({ length: gardenLength + 1 }).map((_, row) => (
        <g key={`post-left-${row}`} transform={`translate(20, ${20
        + row * 36})`} className="transition-all duration-300 ease-out">
          <circle cx="0" cy="0" r="5" fill="#451A03"
stroke="#FDE047" strokeWidth="1" />

```

```

        </g>
    )))
    {Array.from({ length: gardenLength + 1 }).map((_, row) => (
        <g key={`post-right-${row}`} transform={`translate(${20 +
gardenWidth * 36}, ${20 + row * 36})`} className="transition-all
duration-300 ease-out">
            <circle cx="0" cy="0" r="5" fill="#451A03"
stroke="#FDE047" strokeWidth="1" />
        </g>
    )))

    {/* Dimension Labels overlaying side widths */}
    {/* Length Indicator (Top Width) */}
    <g transform={`translate(${20 + (gardenWidth * 18)}, 10)`}
className="transition-all duration-300 ease-out">
        <rect x="-35" y="-12" width="70" height="18" rx="4"
fill="#1E293B" />
        <text x="0" y="1" fill="FFFFFF" fontSize="10"
fontWeight="bold" textAnchor="middle">
            {gardenWidth} m (Length)
        </text>
    </g>

    {/* Width Indicator (Left Height) */}
    <g transform={`translate(5, ${20 + (gardenLength * 18)})`}
className="transition-all duration-300 ease-out">
        <rect x="-10" y="-22" width="18" height="44" rx="4"
fill="#1E293B" />
        <text x="-1" y="2" fill="FFFFFF" fontSize="10"
fontWeight="bold" textAnchor="middle" transform="rotate(-90 -1 2)">
            {gardenLength} m (Width)
        </text>
    </g>
</svg>
</div>

```

```

        { /* Quick interactive stats inside canvas */ }
        <div className="absolute top-3 right-3 bg-white/90 backdrop-blur
border border-slate-200 rounded-xl p-2.5 shadow-sm text-right text-xs">
            <p className="text-slate-400 font-bold">LEGEND</p>
            <p className="flex items-center gap-1.5 justify-end
text-slate-700 mt-1">
                <span className="w-3.5 h-3.5 bg-emerald-100 border
border-emerald-400 inline-block rounded" />
                    Area (Grass)
            </p>
            <p className="flex items-center gap-1.5 justify-end
text-slate-700 mt-1">
                <span className="w-3.5 h-0.5 bg-amber-700 inline-block" />
                    Perimeter (Fence)
            </p>
        </div>

    </div>

    { /* DYNAMIC FEEDBACK TEXT MESSAGE */ }
    <div className="mt-4 p-4 rounded-xl bg-emerald-50 border
border-emerald-100 flex items-center gap-3">
        <span className="text-2xl" role="img" aria-label="Bulb
Tip">💡</span>
        <p className="text-sm font-semibold text-slate-700
leading-normal">{tipMessage}</p>
    </div>

    { /* CONTROLS AREA WITH SLIDERS */ }
    <div className="mt-6 border-t border-slate-100 pt-5 flex flex-col
md:flex-row items-center gap-6 justify-between">

        <div className="flex-1 w-full space-y-4">
            { /* Length slider */ }
            <div>
                <div className="flex justify-between items-center mb-1">

```

```

        <label className="text-xs font-extrabold text-slate-500
uppercase tracking-widest flex items-center gap-1.5">
         Length (Horizontal):
        <span className="text-sm text-emerald-700
font-black">{gardenWidth} meters</span>
        </label>
    </div>
    <input
        type="range"
        min="1"
        max="10"
        value={gardenWidth}
        onChange={ (e) => {
            playSound('hammer');
            setGardenWidth(parseInt(e.target.value));
        }}
        className="w-full h-2 bg-slate-200 rounded-lg
appearance-none cursor-pointer accent-emerald-600"
    />
</div>

{/* Width slider */}
<div>
    <div className="flex justify-between items-center mb-1">
        <label className="text-xs font-extrabold text-slate-500
uppercase tracking-widest flex items-center gap-1.5">
         Width (Vertical):
        <span className="text-sm text-emerald-700
font-black">{gardenLength} meters</span>
        </label>
    </div>
    <input
        type="range"
        min="1"
        max="10"
        value={gardenLength}

```

```

        onChange={ (e) => {
          playSound('hammer');
          setGardenLength(parseInt(e.target.value));
        }}
        className="w-full h-2 bg-slate-200 rounded-lg
appearance-none cursor-pointer accent-emerald-600"
      />
    </div>
  </div>

  { /* RESET / CHALLENGE ACTIONS */ }
  <div className="flex items-center gap-2 w-full md:w-auto
justify-end">
    <button
      onClick={resetGarden}
      className="flex-1 md:flex-none flex items-center
justify-center gap-1.5 px-5 py-2.5 rounded-xl border border-slate-200
hover:bg-slate-50 text-slate-700 text-sm font-bold transition-all"
    >
       Reset dimensions
    </button>

    {isChallengeMode && challengeSuccess && (
      <button
        onClick={handleNextChallenge}
        className="flex-1 md:flex-none flex items-center
justify-center gap-1.5 px-6 py-2.5 rounded-xl bg-amber-500
hover:bg-amber-600 text-white text-sm font-bold shadow-md animate-bounce"
      >
        Next Challenge 
      </button>
    ) }
  </div>
</div>

```

```

</div>

{/* RIGHT: LIVE CALCULATION BOARD (4 Cols) */}
<div className="lg:col-span-4 flex flex-col gap-6">

  {/* MATH CALCULATION READOUT */}
  <div className="bg-gradient-to-br from-slate-900 to-slate-800
text-white rounded-3xl p-6 shadow-xl border border-slate-700">
    <h2 className="text-xs uppercase tracking-widest text-slate-400
font-extrabold mb-4"><img alt="Math icon" data-bbox="333 293 355 308"/> Math calculations</h2>

    <div className="space-y-4">
      {/* Area Box */}
      <div className="bg-slate-800/85 p-4 rounded-2xl border
border-slate-700 relative overflow-hidden">
        <span className="absolute top-2 right-2 text-2xl
opacity-15"><img alt="Clover icon" data-bbox="241 451 261 466"/></span>
        <p className="text-xs text-slate-400 font-extrabold
tracking-wider">AREA (Grass Enclosed)</p>

        <div className="flex items-baseline gap-1.5 mt-2">
          <span className="text-3xl font-black
text-emerald-400">{calculatedArea}</span>
          <span className="text-sm font-semibold text-slate-300">m2
(sq. meters)</span>
        </div>

        <div className="mt-2.5 pt-2.5 border-t border-slate-700/60
text-xs text-slate-400 font-mono">
          Formula: Length × Width
          <br />
          Calculation: {gardenWidth} × {gardenLength} =
{calculatedArea}
        </div>
      </div>
    </div>
  </div>

```

```

        { /* Perimeter Box */ }
        <div className="bg-slate-800/85 p-4 rounded-2xl border
border-slate-700 relative overflow-hidden">
            <span className="absolute top-2 right-2 text-2xl
opacity-15">🚧</span>
            <p className="text-xs text-slate-400 font-extrabold
tracking-wider">PERIMETER (Fence Rope)</p>

            <div className="flex items-baseline gap-1.5 mt-2">
                <span className="text-3xl font-black
text-amber-400">{calculatedPerimeter}</span>
                <span className="text-sm font-semibold
text-slate-300">meters</span>
            </div>

            <div className="mt-2.5 pt-2.5 border-t border-slate-700/60
text-xs text-slate-400 font-mono">
                Formula:  $2 \times (\text{Length} + \text{Width})$ 
                <br />
                Calculation:  $2 \times (\{\text{gardenWidth}\} + \{\text{gardenLength}\}) =$ 
                {calculatedPerimeter}
            </div>
        </div>
    </div>
</div>

{ /* Educational Concept Link */ }
<div className="mt-5 p-3 bg-emerald-950/40 border
border-emerald-900 rounded-xl text-xs flex flex-col gap-1.5">
    <h4 className="font-bold text-emerald-300 flex items-center
gap-1">
        💡 NCERT Key Takeaway:
    </h4>
    <p className="text-slate-300 leading-relaxed">
        <strong>Perimeter</strong> is the total length of the
        boundary line (outer fences).
    </p>

```



```

    <p className="text-slate-300 leading-relaxed">
      <strong>Area</strong> is the actual region of surface space
covered inside those boundaries (number of grass tiles).
    </p>
  </div>
</div>

```

```

{ /* COMPARISON AND CONTEXT CARD */ }

<div className="bg-white border border-slate-200 rounded-3xl p-6
shadow-sm">

```

```

  <h2 className="text-sm font-extrabold text-slate-900 mb-3 flex
items-center gap-1.5">

```

```

    🔍 Fun Comparisons

```

```

  </h2>

```

```

  <p className="text-xs text-slate-500 leading-relaxed mb-4">

```

```

    Notice how you can have different shapes with the exact same
Area, but totally different Perimeters!

```

```

  </p>

```

```

  <div className="space-y-3">

```

```

    <div className="p-3 bg-slate-50 rounded-xl border
border-slate-100 flex items-center justify-between text-xs">

```

```

      <span className="font-bold text-slate-700">6 × 2
Garden</span>

```

```

      <div className="text-right">

```

```

        <p className="text-slate-500">Area: <strong
className="text-slate-800">12 m²</strong></p>

```

```

        <p className="text-slate-500">Perimeter: <strong
className="text-slate-800">16 m</strong></p>

```

```

      </div>

```

```

    </div>

```

```

  <div className="p-3 bg-slate-50 rounded-xl border
border-slate-100 flex items-center justify-between text-xs">

```

```

    <span className="font-bold text-slate-700">4 × 3
Garden</span>

```

```

        <div className="text-right">
            <p className="text-slate-500">Area: <strong
className="text-slate-800">12 m²</strong></p>
            <p className="text-slate-500">Perimeter: <strong
className="text-slate-800">14 m</strong></p>
        </div>
    </div>

    <div className="p-3 bg-emerald-50 rounded-xl border
border-emerald-100 text-xs text-emerald-800 leading-relaxed">
        📌 <strong>Look closely!</strong> Both gardens have an Area
of 12 m², but the 4 × 3 garden uses 2 meters LESS fencing!
    </div>
</div>
</div>
);
}

// =====
// QUIZ VIEW COMPONENT (10 NCERT QUESTIONS)
// =====
function QuizView({ addXp }) {
    const [currentQuestion, setCurrentQuestion] = useState(0);
    const [selectedOption, setSelectedOption] = useState(null);
    const [isSubmitted, setIsSubmitted] = useState(false);
    const [score, setScore] = useState(0);
    const [showResults, setShowResults] = useState(false);

    const questions = [
        {
            question: "You want to put a dynamic fencing rope all around a
rectangular field. Which measurement do you need to calculate?",
            options: [

```

```

    { label: "A", text: "Area of the field" },
    { label: "B", text: "Perimeter of the field", isCorrect: true },
    { label: "C", text: "Weight of the field" },
    { label: "D", text: "Height of the plants" }
  ],
  explanation: "Fencing goes around the boundary of a shape. The
boundary's total length is called the Perimeter."
},
{
  question: "A square field has sides of length 5 meters. What is the
total Perimeter of this field?",
  options: [
    { label: "A", text: "25 meters" },
    { label: "B", text: "10 meters" },
    { label: "C", text: "20 meters", isCorrect: true },
    { label: "D", text: "15 meters" }
  ],
  explanation: "A square has 4 equal sides. So, the perimeter is $5 + 5
+ 5 + 5 = 20$ meters (or $4 \times 5 = 20$ meters).",
},
{
  question: "For the same square field with sides of 5 meters, what is
the total Area?",
  options: [
    { label: "A", text: "20 square meters" },
    { label: "B", text: "25 square meters", isCorrect: true },
    { label: "C", text: "10 square meters" },
    { label: "D", text: "50 square meters" }
  ],
  explanation: "The area of a square is calculated by multiplying side
by side: $\text{Side} \times \text{Side} = 5 \times 5 = 25$ square
meters."
},
{

```

```

    question: "Two different rectangular garden fields have the exact
same Area of 24 square meters. Do they have to have the exact same
Perimeter?",
    options: [
        { label: "A", text: "Yes, if the Area is the same, the boundary is
always equal." },
        { label: "B", text: "No, fields can have the same area but
different boundary lengths.", isCorrect: true }
    ],
    explanation: "No! For example, a 6x4 garden has Area = 24 and
Perimeter = 20. But a 12x2 garden also has Area = 24, yet its Perimeter is
28!"
},
{
    question: "A classroom board is 3 meters long and 2 meters wide. How
much surface area does the board cover?",
    options: [
        { label: "A", text: "10 square meters" },
        { label: "B", text: "5 square meters" },
        { label: "C", text: "6 square meters", isCorrect: true },
        { label: "D", text: "12 square meters" }
    ],
    explanation: "The surface space covered is the Area. For a rectangle,
 $\text{Length} \times \text{Width} = 3 \times 2 = 6$  square meters."
},
{
    question: "What is the correct standard unit for measuring the Area
of a surface?",
    options: [
        { label: "A", text: "Meters (m)" },
        { label: "B", text: "Square Meters (sq. m or m2)", isCorrect: true },
        { label: "C", text: "Centimeters (cm)" },
        { label: "D", text: "Liters (l)" }
    ],

```



```

        { label: "B", text: "The sum of the lengths of all its outer
sides.", isCorrect: true },
        { label: "C", text: "The product of its length and width." },
        { label: "D", text: "The height of its outer corners." }
    ],
    explanation: "Perimeter is always the total length of the outer
boundary line of a shape."
},
{
    question: "A triangle has side lengths of 3 cm, 4 cm, and 5 cm. What
is its Perimeter?",
    options: [
        { label: "A", text: "12 cm", isCorrect: true },
        { label: "B", text: "60 cm" },
        { label: "C", text: "7 cm" },
        { label: "D", text: "9 cm" }
    ],
    explanation: "The perimeter of a triangle is simply the sum of its
three side lengths:  $3 + 4 + 5 = 12$  cm."
}
];

```

```

const handleOptionSelect = (index) => {
    if (isSubmitted) return;
    playSound('click');
    setSelectedOption(index);
};

const handleSubmit = () => {
    if (selectedOption === null) return;
    setIsSubmitted(true);
    const isCorrect =
questions[currentQuestion].options[selectedOption].isCorrect;

    if (isCorrect) {
        playSound('win');
    }
}

```

```

        setScore(prev => prev + 1);
        addXp(10);
    } else {
        playSound('fail');
    }
};

const handleNext = () => {
    setSelectedOption(null);
    setIsSubmitted(false);
    if (currentQuestion < questions.length - 1) {
        setCurrentQuestion(prev => prev + 1);
    } else {
        setShowResults(true);
    }
};

const resetQuiz = () => {
    playSound('reset');
    setCurrentQuestion(0);
    setSelectedOption(null);
    setIsSubmitted(false);
    setScore(0);
    setShowResults(false);
};

return (
    <div className="max-w-3xl mx-auto bg-white border border-slate-200
rounded-3xl p-6 shadow-sm">

        {/* Quiz Progress header */}
        <div className="flex items-center justify-between mb-6 border-b
border-slate-100 pb-4">
            <span className="text-xs font-bold text-slate-400 uppercase
tracking-wider">

```

```

        Practice Deck • Question {currentQuestion + 1} of
{questions.length}
    </span>
    <div className="w-32 bg-slate-100 rounded-full h-2">
        <div
            className="bg-emerald-600 h-2 rounded-full transition-all"
            style={{ width: `${((currentQuestion + (isSubmitted ? 1 : 0)) /
questions.length) * 100}%` }}
        />
    </div>
</div>

{!showResults ? (
    <div>
        {/* Question text */}
        <h2 className="text-lg font-bold text-slate-800 leading-snug
mb-5">
            {questions[currentQuestion].question}
        </h2>

        {/* Graphics for Question Help */}
        <div className="my-6 p-4 bg-slate-50 border border-slate-200
rounded-2xl flex justify-center items-center">
            {currentQuestion === 1 && (
                <div className="text-center">
                    <p className="text-[10px] text-slate-400 font-bold
mb-1.5">SQUARE FIELD (5m Sides)</p>
                    <div className="w-24 h-24 bg-emerald-100 border-4
border-amber-800 rounded-md flex items-center justify-center font-mono
font-bold text-emerald-800">
                        Area = ?
                    </div>
                    <div className="text-[10px] text-slate-500 mt-1">Perimeter
= sum of 4 sides</div>
                </div>
            )}
        </div>
    )}

```



```

    {currentQuestion === 4 && (
      <div className="text-center">
        <p className="text-[10px] text-slate-400 font-bold
mb-1.5">BOARD SURFACE SPACE</p>
        <div className="w-28 h-20 bg-slate-800 border-4
stroke-slate-500 rounded flex flex-col items-center justify-center
text-white text-xs">
          <span>3 meters Long</span>
          <span className="text-[10px] text-slate-400">2 meters
Wide</span>
        </div>
      </div>
    )}
    {currentQuestion !== 1 && currentQuestion !== 4 && (
      <div className="text-slate-400 text-xs font-bold">
        🌱 Visualize the math! Perimeter is boundaries, Area is
inside surface space.
      </div>
    )}
  </div>

  { /* Options Grid */ }
  <div className="grid grid-cols-1 gap-3">
    {questions[currentQuestion].options.map((opt, idx) => {
      const isSelected = selectedOption === idx;
      let btnClass = "bg-slate-50 hover:bg-slate-100
border-slate-200 text-slate-700";

      if (isSelected) {
        btnClass = "bg-emerald-50 border-emerald-400
text-emerald-900 ring-2 ring-emerald-200";
      }

      if (isSubmitted) {
        if (opt.isCorrect) {

```

```

        btnClass = "bg-emerald-50 border-emerald-400
text-emerald-800 ring-2 ring-emerald-200 font-bold";
    } else if (isSelected) {
        btnClass = "bg-rose-50 border-rose-400 text-rose-800
line-through decoration-slate-400";
    } else {
        btnClass = "bg-slate-50 border-slate-200 text-slate-400
opacity-60";
    }
}

return (
    <button
        key={idx}
        onClick={() => handleOptionSelect(idx)}
        className={`w-full text-left p-4 rounded-2xl border
transition-all flex items-center gap-3.5 ${btnClass}`}
        disabled={isSubmitted}
    >
        <span className={`w-7 h-7 rounded-full flex items-center
justify-center text-xs font-bold ${
            isSelected ? 'bg-emerald-600 text-white' :
'bg-slate-200 text-slate-600'
        }`>
            {opt.label}
        </span>
        <span className="text-sm font-medium">{opt.text}</span>
    </button>
);
}}}
</div>

{/* Action buttons */}
<div className="mt-6 flex justify-end gap-3">
    {!isSubmitted ? (
        <button

```

```

        onClick={handleSubmit}
        disabled={selectedOption === null}
        className={`px-6 py-3 rounded-2xl text-sm font-bold
shadow-md transition-all ${
            selectedOption === null
                ? 'bg-slate-100 text-slate-400 border border-slate-200
cursor-not-allowed'
                : 'bg-slate-900 hover:bg-slate-800 text-white
hover:shadow-lg'
        }}
    >
        Submit Answer
    </button>
) : (
    <button
        onClick={handleNext}
        className="px-6 py-3 rounded-2xl text-sm font-bold
bg-emerald-600 hover:bg-emerald-700 text-white shadow-md hover:shadow-lg
transition-all"
    >
        {currentQuestion < questions.length - 1 ? "Next Question" :
"See Final Score"}
    </button>
)}
</div>

{/* Explanations Banner */}
{isSubmitted && (
    <div className="mt-5 p-4 bg-emerald-50 border
border-emerald-100 rounded-2xl">
        <h4 className="text-xs font-bold text-emerald-800 uppercase
tracking-widest mb-1"><img alt="Teacher icon" data-bbox="345 788 365 805"/> Teacher Explanation:</h4>
        <p className="text-xs text-slate-700 leading-relaxed">
            {questions[currentQuestion].explanation}
        </p>
    </div>
)}

```

```

    })
  </div>
) : (
  /* Results View */
  <div className="text-center py-8">
    <span className="text-6xl block mb-4">🏆</span>
    <h2 className="text-2xl font-extrabold text-slate-900">Quiz
Completed!</h2>
    <p className="text-sm text-slate-500 mt-1">Excellent job! You are
becoming an area and perimeter expert.</p>

    { /* Score display circle */
    <div className="w-36 h-36 rounded-full border-4
border-emerald-600 bg-emerald-50 flex flex-col items-center justify-center
mx-auto my-6 shadow-inner">
      <span className="text-3xl font-extrabold
text-emerald-700">{score} / {questions.length}</span>
      <span className="text-[10px] font-bold text-emerald-500
uppercase tracking-wider">CORRECT ANSWERS</span>
    </div>

    <p className="text-sm text-slate-600 px-6 max-w-md mx-auto
leading-relaxed">
      {score >= 8
      ? "🌟 Wonderful! You designed and calculated every park
correctly. You deserve a Master Gardener Badge!"
      : "🌟 Great effort! Head over to the Lessons tab or play more
in Sandbox Mode to practice calculations!"}
    </p>

    <div className="mt-8 flex justify-center gap-3">
      <button
        onClick={resetQuiz}
        className="px-6 py-3 rounded-2xl text-sm font-bold border
border-slate-200 hover:bg-slate-50 text-slate-700 transition-all"
      >

```

 Try Again

```
</button>
<button
  onClick={() => { playSound('click'); resetQuiz(); }}
  className="px-6 py-3 rounded-2xl text-sm font-bold
bg-slate-900 text-white hover:bg-slate-800 transition-all"
>
  Back to Start
</button>
</div>
</div>
)}

</div>
);
}

// =====
// FUN LESSONS & DIY VIEW COMPONENT
// =====
function LessonView() {
  const [activeFaq, setActiveFaq] = useState(null);

  const toggleFaq = (idx) => {
    playSound('click');
    setActiveFaq(activeFaq === idx ? null : idx);
  };

  return (
    <div className="max-w-4xl mx-auto space-y-8 select-none">

      {/* Visual Header Card */}
      <div className="bg-gradient-to-r from-emerald-500 to-teal-600
rounded-3xl p-6 text-white shadow-md">
        <h2 className="text-2xl font-extrabold
tracking-tight">Understanding Perimeter and Area!</h2>
```

```

    <p className="text-sm text-emerald-50 opacity-90 mt-1">
      Two simple guidelines that help us measure outer fences vs. inner
spaces!
    </p>
  </div>

  { /* CORE CONCEPT SECTION */ }

  <div className="grid grid-cols-1 md:grid-cols-2 gap-6 items-start">

    { /* Perimeter Explainer */ }

    <div className="bg-white border border-slate-200 rounded-3xl p-6
shadow-sm">
      <h3 className="font-extrabold text-slate-900 flex items-center
gap-2 mb-3">
        <span className="text-lg">🍷</span> What is Perimeter?
      </h3>
      <p className="text-xs text-slate-600 leading-relaxed mb-4">
        The <strong>Perimeter</strong> is the total length of the outer
boundary line that encloses any shape. Think of a fence surrounding a park
or a ribbon pasted along the edge of a greeting card.
      </p>

      <div className="p-4 bg-amber-50 border border-amber-100
rounded-2xl">
        <h4 className="text-xs font-bold text-amber-800 mb-1">📐
Formula for Rectangle:</h4>
        <p className="text-xs font-mono text-amber-900">Perimeter = 2 ×
(Length + Width)</p>
        <p className="text-[11px] text-slate-500 mt-2">Simply add all 4
sides together! ($L + W + L + W$)</p>
      </div>
    </div>

    { /* Area Explainer */ }

    <div className="bg-white border border-slate-200 rounded-3xl p-6
shadow-sm">

```

```

    <h3 className="font-extrabold text-slate-900 flex items-center
gap-2 mb-3">
    <span className="text-lg">🍀</span> What is Area?
  </h3>
  <p className="text-xs text-slate-600 leading-relaxed mb-4">
    The <strong>Area</strong> is the actual measure of the flat
    surface covered inside those boundary lines. Think of the grass tiles
    growing inside the garden, or the carpet laid inside a room.
  </p>

  <div className="p-4 bg-emerald-50 border border-emerald-100
rounded-2xl">
    <h4 className="text-xs font-bold text-emerald-800 mb-1">📐
    Formula for Rectangle:</h4>
    <p className="text-xs font-mono text-emerald-900">Area = Length
    × Width</p>
    <p className="text-[11px] text-slate-500 mt-2">Calculated by
    multiplying the horizontal length by the vertical width.</p>
  </div>
</div>

{ /* REAL LIFE DIY EXPERIENCE */ }
<div className="bg-white border border-slate-200 rounded-3xl p-6
shadow-sm">
  <div className="flex items-center gap-3 mb-4">
    <span className="text-3xl">🏠</span>
    <div>
      <h3 className="font-extrabold text-slate-950 text-base">DIY
      Home Activity: Thread & Tiles!</h3>
      <p className="text-xs text-slate-500">Fun way to understand
      measurements around your house!</p>
    </div>
  </div>
</div>

```

```

    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      <div className="p-3 bg-slate-50 border border-slate-100
rounded-xl text-center">
        <div className="w-8 h-8 rounded-full bg-emerald-100
text-emerald-700 flex items-center justify-center font-bold text-xs
mx-auto mb-2">1</div>
        <h4 className="font-bold text-xs text-slate-800 mb-1">Find some
Cards</h4>
        <p className="text-[11px] text-slate-500 leading-normal">Take a
few rectangular post cards, playing cards, or small books of different
shapes.</p>
      </div>
      <div className="p-3 bg-slate-50 border border-slate-100
rounded-xl text-center">
        <div className="w-8 h-8 rounded-full bg-emerald-100
text-emerald-700 flex items-center justify-center font-bold text-xs
mx-auto mb-2">2</div>
        <h4 className="font-bold text-xs text-slate-800 mb-1">The Rope
Method</h4>
        <p className="text-[11px] text-slate-500 leading-normal">Wrap a
sewing thread tightly once around the borders of the card. Cut the thread
and measure it on a scale. That's your <strong>Perimeter</strong>!!</p>
      </div>
      <div className="p-3 bg-slate-50 border border-slate-100
rounded-xl text-center">
        <div className="w-8 h-8 rounded-full bg-emerald-100
text-emerald-700 flex items-center justify-center font-bold text-xs
mx-auto mb-2">3</div>
        <h4 className="font-bold text-xs text-slate-800 mb-1">The Coin
Grid</h4>
        <p className="text-[11px] text-slate-500
leading-normal">Arrange equal coin/button circles next to each other to
cover the whole surface. The count of coins tells you its surface
<strong>Area</strong>!!</p>
      </div>
    </div>

```



```

</div>

{/* FREQUENT ACCORDIONS */}

<div className="bg-white border border-slate-200 rounded-3xl p-6
shadow-sm">
  <h3 className="font-extrabold text-slate-900 flex items-center
gap-2 mb-4">
    ? Common Misconceptions Solved!
  </h3>

  <div className="space-y-2">
    {[
      {
        q: "If two garden shapes have the same exact Area, do they
have to use the same length of fence rope (Perimeter)?",
        a: "Absolutely not! This is a very common mistake. For
example, a garden of 12m2 could be 12m × 1m (Perimeter = 26m) or 4m × 3m
(Perimeter = 14m). Both contain the exact same amount of ground area, but
the long narrow one needs way more fencing rope!"
      },
      {
        q: "Why is Area measured in 'Square' units (like cm2 or
m2)?",
        a: "Because Area counts how many perfect little 1m × 1m
squares can fit inside the surface of the shape. Since you are multiplying
length (m) by width (m), you get 'meters times meters', written as square
meters or m2."
      },
      {
        q: "What shape encloses the LARGEST possible area for any
given fencing length?",
        a: "If you have a fixed perimeter, a perfect Square (all
sides equal) will always enclose more Area than any other rectangular
combination. Go test this out on Challenge 3 of the simulator tab!"
      }
    ]}.map((faq, idx) => (

```

```

        <div key={idx} className="border border-slate-100 rounded-2xl
overflow-hidden">
            <button
                onClick={() => toggleFaq(idx)}
                className="w-full text-left p-4 bg-slate-50
hover:bg-slate-100/75 font-bold text-xs sm:text-sm text-slate-800 flex
items-center justify-between"
            >
                <span>{faq.q}</span>
                <span className="text-slate-400 font-bold">{activeFaq ===
idx ? "▲" : "▼"}</span>
            </button>
            {activeFaq === idx && (
                <div className="p-4 bg-white border-t border-slate-100
text-xs text-slate-600 leading-relaxed">
                    {faq.a}
                </div>
            )}
        </div>
    )}
</div>
</div>
</div>
);
}

```

Tab 4

```

import React, { useState, useEffect, useRef } from 'react';

// Custom high-quality SVG icons to ensure zero external dependency issues
const Icons = {
  Sparkles: () => (
    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none"
    stroke="currentColor" strokeWidth="2" strokeLinecap="round"
    strokeLinejoin="round" className="w-5 h-5 text-amber-500 animate-pulse">
      <path d="m12 3-1.912 5.813a2 2 0 0 1-1.275 1.275L3 12l15.813 1.912a2 2
    0 0 1 1.275 1.275L12 21l11.912-5.813a2 2 0 0 1 1.275-1.275L21
    12l-5.813-1.912a2 2 0 0 1-1.275-1.275L12 3Z" />
      <path d="m5 3 1 2.5L8.5 6 6 7 5 9.5 4 7 1.5 6 4 5.5z" opacity="0.8"
    />
      <path d="m19 17 1 2.5 2.5.5-2.5 1-1 2.5-1-2.5-2.5-1 2.5-1z"
    opacity="0.8" />
    </svg>
  ),
  CheckCircle: () => (
    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none"
    stroke="currentColor" strokeWidth="2.5" strokeLinecap="round"
    strokeLinejoin="round" className="w-5 h-5 text-emerald-500">
      <path d="M22 11.08V12a10 10 0 1 1-5.93-9.14" />
      <path d="m9 11 3 3L22 4" />
    </svg>
  ),
  Trophy: () => (
    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none"
    stroke="currentColor" strokeWidth="2" strokeLinecap="round"
    strokeLinejoin="round" className="w-6 h-6 text-yellow-500">
      <path d="M6 9H4.5a2.5 2.5 0 0 1 0-5H6" />
      <path d="M18 9h1.5a2.5 2.5 0 0 0 0-5H18" />
      <path d="M4 22h16" />
      <path d="M10 14.66V17c0 .55-.45 1-1 1H4v2h16v-2h-5c-.55
    0-1-.45-1-1v-2.34" />
      <path d="M12 2a6 6 0 0 1 6 6v4H6V8a6 6 0 0 1 6-6Z" />
    </svg>
  )
};

```

```

),
Refresh: () => (
  <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round" className="w-4 h-4 mr-1">
    <path d="M3 12a9 9 0 0 1 9-9 9.75 9.75 0 0 1 6.74 2.74L21 8" />
    <path d="M21 3v5h-5" />
    <path d="M21 12a9 9 0 0 1-9 9 9.75 9.75 0 0 1-6.74-2.74L3 16" />
    <path d="M3 21v-5h5" />
  </svg>
),
Target: () => (
  <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round" className="w-5 h-5 text-rose-500">
    <circle cx="12" cy="12" r="10" />
    <circle cx="12" cy="12" r="6" />
    <circle cx="12" cy="12" r="2" />
  </svg>
),
Eye: () => (
  <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round" className="w-4 h-4 mr-1">
    <path d="M2 12s3-7 10-7 10 7 7-10 7-10-7-10-7Z" />
    <circle cx="12" cy="12" r="3" />
  </svg>
)
};

```

```

export default function App() {
  // Simulator State
  const [angle, setAngle] = useState(45);
  const [showProtractor, setShowProtractor] = useState(true);
  const [showRuler, setShowRuler] = useState(false);
  const [xp, setXp] = useState(0);

```

```

const [discovered, setDiscovered] = useState([]);
const [activeTab, setActiveTab] = useState('explore'); // 'explore' or
'challenges'
const [showCelebration, setShowCelebration] = useState(false);

// Challenge System State
const [currentChallenge, setCurrentChallenge] = useState(0);
const [challengeCompleted, setChallengeCompleted] = useState(false);

const svgRef = useRef(null);
const isDragging = useRef(false);

// Define challenges matching Class 5 & 6 standards
const challenges = [
  {
    id: 0,
    title: "The Golden Right Angle",
    prompt: "Can you set the machine to a perfect Right Angle (90°)?",
    targetCheck: (deg) => deg === 90,
    tip: "Look for the vertical 'L' shape! It's how walls meet the
floor.",
    successMsg: "Splendid! You made a perfect 90° Right Angle!"
  },
  {
    id: 1,
    title: "Acute Angle Hunt",
    prompt: "Adjust the machine to show any Acute Angle (between 1° and
89°).",
    targetCheck: (deg) => deg > 0 && deg < 90,
    tip: "Keep it sharp and narrow! Smaller than a corner.",
    successMsg: "Superb! Acute angles are 'cute' and sharp!"
  },
  {
    id: 2,
    title: "The Straight Runway",
    prompt: "Can you stretch the rays to form a Straight Angle (180°)?",

```

```

        targetCheck: (deg) => deg === 180,
        tip: "Make a perfectly flat horizontal line.",
        successMsg: "Fantastic! A straight angle is exactly 180°."
    },
    {
        id: 3,
        title: "The Reclined Obtuse Angle",
        prompt: "Find an Obtuse Angle (between 91° and 179°).",
        targetCheck: (deg) => deg > 90 && deg < 180,
        tip: "Make it wider than a corner, but not quite a flat line.",
        successMsg: "Great work! That's a wide, lazy Obtuse angle."
    },
    {
        id: 4,
        title: "The Great Reflex Leap",
        prompt: "Explore past the straight line to find a Reflex Angle
(between 181° and 359°).",
        targetCheck: (deg) => deg > 180 && deg < 360,
        tip: "Keep rotating! This is the giant outer swing of the ray.",
        successMsg: "Excellent! Reflex angles represent massive outside
turns."
    }
];

// Core SVG geometry constants
const cx = 200;
const cy = 200;
const r = 130;

// Convert degrees to polar endpoints (handling standard mathematical
counter-clockwise angles)
const rad = (angle * Math.PI) / 180;
const xTarget = cx + r * Math.cos(-rad);
const yTarget = cy + r * Math.sin(-rad);

// Core Classification Engine

```

```

const classifyAngle = (deg) => {
  if (deg === 0) return { name: "Zero Angle", desc: "No turn at all",
    color: "text-slate-500", bg: "bg-slate-50", border: "border-slate-300",
    badge: "bg-slate-100 text-slate-800", emoji: "😬" };
  if (deg > 0 && deg < 90) return { name: "Acute Angle", desc: "Sharp,
    tiny, and small!", color: "text-teal-600", bg: "bg-teal-50", border:
    "border-teal-200", badge: "bg-teal-100 text-teal-800", emoji: "📐" };
  if (deg === 90) return { name: "Right Angle", desc: "A perfect square
    corner!", color: "text-amber-600", bg: "bg-amber-50/70", border:
    "border-amber-200", badge: "bg-amber-100 text-amber-800", emoji: "🏠" };
  if (deg > 90 && deg < 180) return { name: "Obtuse Angle", desc: "Wide
    open and stretched", color: "text-orange-600", bg: "bg-orange-50", border:
    "border-orange-200", badge: "bg-orange-100 text-orange-800", emoji: "🪑"
  };
  if (deg === 180) return { name: "Straight Angle", desc: "A completely
    flat path", color: "text-rose-600", bg: "bg-rose-50", border:
    "border-rose-200", badge: "bg-rose-100 text-rose-800", emoji: "🖋️" };
  if (deg > 180 && deg < 360) return { name: "Reflex Angle", desc: "The
    giant wrap-around turn", color: "text-purple-600", bg: "bg-purple-50",
    border: "border-purple-200", badge: "bg-purple-100 text-purple-800",
    emoji: "🌀" };
  return { name: "Complete Angle", desc: "One perfect full spin", color:
    "text-blue-600", bg: "bg-blue-50", border: "border-blue-200", badge:
    "bg-blue-100 text-blue-800", emoji: "🤖" };
};

const currentClassification = classifyAngle(angle);

// Dynamic illustrative matching icons/objects for student visual aid
const getIllustrationDetails = (deg) => {
  if (deg > 0 && deg < 90) return { label: "Pizza Slice Wedge", icon:
    "🍕" };
  if (deg === 90) return { label: "Perfect Window Corner", icon: "🪟" };
  if (deg > 90 && deg < 180) return { label: "Fully Open Laptop", icon:
    "💻" };
  if (deg === 180) return { label: "Flat Seesaw Board", icon: "⚖️" };
};

```



```

    if (deg > 180 && deg < 360) return { label: "Clock Hands (at 7:00)",
icon: "🕒" };
    return { label: "Perfect Circle Shield", icon: "🛡️" };
};

const illustration = getIllustrationDetails(angle);

// Handle live calculation when clicking and dragging the rotating ray
directly on the SVG canvas
const handleSvgMove = (clientX, clientY) => {
    if (!svgRef.current) return;
    const rect = svgRef.current.getBoundingClientRect();
    const x = clientX - rect.left - (rect.width / 2);
    const y = clientY - rect.top - (rect.height / 2);

    // Calculate counter-clockwise angle relative to the 3 o'clock positive
horizontal axis
    let angleRad = Math.atan2(-y, x);
    if (angleRad < 0) {
        angleRad += 2 * Math.PI;
    }
    let deg = Math.round((angleRad * 180) / Math.PI);

    // Apply smart locking/magnetic snappings near important reference
points (90°, 180°, 270°, 360°)
    if (Math.abs(deg - 90) < 3) deg = 90;
    else if (Math.abs(deg - 180) < 3) deg = 180;
    else if (Math.abs(deg - 270) < 3) deg = 270;
    else if (Math.abs(deg - 360) < 3 || deg < 2) deg = 0;

    setAngle(deg);
};

const handleMouseDown = () => { isDragging.current = true; };
const handleMouseMove = (e) => {
    if (!isDragging.current) return;

```

```

    handleSvgMove(e.clientX, e.clientY);
  };
  const handleTouchMove = (e) => {
    if (e.touches.length === 0) return;
    handleSvgMove(e.touches[0].clientX, e.touches[0].clientY);
  };
  const handleMouseUpOrLeave = () => { isDragging.current = false; };

  // Watch for milestone discoveries to trigger positive educational
  reinforcement
  useEffect(() => {
    let discoveredType = null;
    if (angle > 0 && angle < 90) discoveredType = "Acute";
    else if (angle === 90) discoveredType = "Right";
    else if (angle > 90 && angle < 180) discoveredType = "Obtuse";
    else if (angle === 180) discoveredType = "Straight";
    else if (angle > 180 && angle < 360) discoveredType = "Reflex";

    if (discoveredType && !discovered.includes(discoveredType)) {
      const updated = [...discovered, discoveredType];
      setDiscovered(updated);
      setXp((prev) => prev + 40);

      // If student discovers all five types, reward them with a grand
      celebration!
      if (updated.length === 5) {
        setShowCelebration(true);
        setXp((prev) => prev + 100);
      }
    }
  }, [angle, discovered]);

  // Handle challenge verification
  useEffect(() => {
    if (activeTab === 'challenges') {
      const activeChallenge = challenges[currentChallenge];

```

```

    if (activeChallenge.targetCheck(angle)) {
      if (!challengeCompleted) {
        setChallengeCompleted(true);
        setXp((prev) => prev + 60);
      }
    } else {
      setChallengeCompleted(false);
    }
  }
}, [angle, currentChallenge, activeTab]);

const handleNextChallenge = () => {
  if (currentChallenge < challenges.length - 1) {
    setCurrentChallenge(currentChallenge + 1);
    setChallengeCompleted(false);
  } else {
    // Completed all challenges!
    setShowCelebration(true);
  }
};

// SVG Large Arc Flag calculation for the filled visual sector
const largeArcFlag = angle > 180 ? 1 : 0;
const sweepFlag = 0; // counter-clockwise
const arcPathData = angle === 0 ? "" : `
  M ${cx} ${cy}
  L ${cx + r} ${cy}
  A ${r} ${r} 0 ${largeArcFlag} ${sweepFlag} ${xTarget} ${yTarget}
  Z
`;

return (
  <div className="min-h-screen bg-slate-50 flex flex-col justify-between
select-none font-sans text-slate-800">

    {/* Dynamic Celebration Modal Overlay */}

```

```

    {showCelebration && (
      <div className="fixed inset-0 z-50 bg-slate-900/80 flex
items-center justify-center p-4 backdrop-blur-sm animate-fade-in">
        <div className="bg-white rounded-3xl p-8 max-w-md w-full
text-center shadow-2xl border border-amber-200 animate-scale-up">
          <div className="text-6xl mb-4 animate-bounce">🏆</div>
          <h2 className="text-3xl font-extrabold text-slate-900
tracking-tight">Grand Angle Master!</h2>
          <p className="text-slate-600 mt-2 text-md leading-relaxed">
            Incredible exploration! You have discovered all the unique
shapes angles can make and proven your mathematical genius.
          </p>
          <div className="bg-amber-50 rounded-2xl p-4 my-6 border
border-amber-100">
            <div className="text-xs text-amber-600 font-bold uppercase
tracking-wider">Total Reward Earned</div>
            <div className="text-3xl font-black text-amber-500 mt-1">+350
XP Star Points</div>
          </div>
          <button
            onClick={() => {
              setShowCelebration(false);
              setDiscovered([]);
              setXp(50);
              setCurrentChallenge(0);
              setChallengeCompleted(false);
            }}
            className="w-full bg-indigo-600 text-white font-bold py-4
px-6 rounded-2xl shadow-lg hover:bg-indigo-700 transition active:scale-95
text-lg"
          >
            Start New Adventure
          </button>
        </div>
      </div>
    ) }
  ) }

```

```

    { /* Main Educational Dashboard Header */ }

    <header className="bg-white border-b border-slate-200 sticky top-0
z-40 px-4 py-3 sm:px-6 shadow-sm">

        <div className="max-w-6xl mx-auto flex flex-col sm:flex-row
justify-between items-center gap-3">

            <div className="flex items-center gap-3">
                <div className="bg-indigo-100 p-2.5 rounded-2xl border
border-indigo-200 shadow-sm">
                    <span className="text-2xl">📚</span>
                </div>
                <div>
                    <h1 className="text-lg sm:text-xl font-black text-slate-900
leading-tight">Secret World of Angles</h1>
                    <p className="text-xs text-slate-500 font-medium flex
items-center gap-1.5">
                        <span>Class 5-6 Mathematics Laboratory</span>
                        <span className="inline-block w-1 h-1 rounded-full
bg-slate-300"></span>
                        <span className="text-indigo-600 font-bold">NCERT
Curriculum</span>
                    </p>
                </div>
            </div>

            <div className="flex items-center gap-3 w-full sm:w-auto
justify-end">
                { /* XP Gamified Progress Counter */ }
                <div className="bg-amber-50 border border-amber-200
text-amber-800 rounded-2xl px-4 py-2 flex items-center gap-2 shadow-sm
font-semibold text-sm">
                    <span className="text-lg">★</span>
                    <div>
                        <span className="font-extrabold text-amber-600">{xp}</span>
                        <span className="text-xs text-amber-500 ml-1">XP</span>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        </div>
    </div>

    { /* Quick Reset Global Actions */ }
    <button
        onClick={() => {
            setAngle(45);
            setDiscovered([]);
            setXp(0);
            setChallengeCompleted(false);
            setCurrentChallenge(0);
        }}
        className="p-2.5 bg-slate-100 border border-slate-200
text-slate-600 rounded-2xl hover:bg-slate-200 transition active:scale-95
shadow-sm"
        title="Reset Simulator"
    >
        <Icons.Refresh />
    </button>
</div>

</div>
</header>

{ /* Primary Interactive Laboratory Grid Workspace */ }
<main className="flex-1 max-w-6xl w-full mx-auto px-4 py-6 grid
grid-cols-1 lg:grid-cols-12 gap-6 items-stretch">

    { /* Left Interactive Playground (7 Columns) */ }
    <section className="lg:col-span-7 flex flex-col gap-5">

        <div className="bg-white rounded-3xl p-5 border
border-slate-200/80 shadow-md flex flex-col items-center justify-center
relative overflow-hidden flex-1 min-h-[400px]">

            { /* Background Grid Accent Lines */ }

```

```

    <div className="absolute inset-0
bg-[radial-gradient(#e2e8f0_1px,transparent_1px)]
[background-size:16px_16px] opacity-40 pointer-events-none"></div>

    {/* Quick Mode Toggle Switch for Student exploration */}
    <div className="absolute top-4 left-4 z-10 flex gap-2">
      <button
        onClick={() => setActiveTab('explore')}
        className={`px-4 py-2 rounded-xl text-xs font-bold
transition-all shadow-sm flex items-center gap-1.5 border ${
          activeTab === 'explore'
            ? 'bg-indigo-600 text-white border-indigo-700'
            : 'bg-white text-slate-600 border-slate-200
hover:bg-slate-50'
        }}`
        >
        <span><img alt="rocket icon" data-bbox="335 450 355 465" style="vertical-align: middle; height: 1em;"/> Free Play</span>
      </button>
      <button
        onClick={() => setActiveTab('challenges')}
        className={`px-4 py-2 rounded-xl text-xs font-bold
transition-all shadow-sm flex items-center gap-1.5 border ${
          activeTab === 'challenges'
            ? 'bg-rose-600 text-white border-rose-700'
            : 'bg-white text-slate-600 border-slate-200
hover:bg-slate-50'
        }}`
        >
        <Icons.Target />
        <span>Play Quests</span>
      </button>
    </div>

    {/* Direct Ray Rotation Guide Pointer */}

```

```

<div className="absolute top-4 right-4 z-10 bg-indigo-50 border
border-indigo-100 rounded-xl px-3 py-1.5 text-[10px] text-indigo-700
font-extrabold uppercase tracking-wide shadow-sm animate-pulse">

```



Tip: Click & Drag the Pink Arrow!

```

</div>

```

```

{/* Dynamic Math SVG Canvas Board */}

<div className="relative w-full max-w-[340px] aspect-square
flex items-center justify-center select-none">
  <svg
    ref={svgRef}
    width="100%"
    height="100%"
    viewBox="0 0 400 400"
    onMouseMove={handleMouseMove}
    onTouchMove={handleTouchMove}
    onMouseUp={handleMouseUpOrLeave}
    onMouseLeave={handleMouseUpOrLeave}
    onTouchEnd={handleMouseUpOrLeave}
    className="cursor-crosshair overflow-visible touch-none"
  >
    {/* 1. Protractor Angular Scale Grid Overlay */}
    {showProtractor && (
      <g className="transition-opacity duration-300">
        <circle cx={cx} cy={cy} r={r} fill="none"
stroke="#cbd5e1" strokeWidth="2" strokeDasharray="3 4" />

        {/* Generates degree markers at 15° intervals */}
        {[...Array(24)].map((_, i) => {
          const markerDeg = i * 15;
          const radMarker = (markerDeg * Math.PI) / 180;

          const isMajor = markerDeg % 30 === 0;
          const tickLength = isMajor ? 12 : 6;
          const strokeColor = isMajor ? "#64748b" : "#94a3b8";
          const strokeWidth = isMajor ? 2.5 : 1.5;

```



```

        const x1 = cx + (r - tickLength) *
Math.cos(radMarker);
        const y1 = cy + (r - tickLength) *
Math.sin(radMarker);
        const x2 = cx + r * Math.cos(radMarker);
        const y2 = cy + r * Math.sin(radMarker);

        // Text labels for major angles
        const textDist = r - 25;
        const tx = cx + textDist * Math.cos(-radMarker);
        const ty = cy + textDist * Math.sin(-radMarker) + 4;
// micro offset for vertical alignment

        return (
            <g key={i}>
                <line x1={x1} y1={y1} x2={x2} y2={y2}
stroke={strokeColor} strokeWidth={strokeWidth} />
                {isMajor && markerDeg < 360 && (
                    <text
                        x={tx}
                        y={ty}
                        textAnchor="middle"
                        fontSize="10"
                        fontWeight="bold"
                        fill="#94a3b8"
                        className="font-mono pointer-events-none"
                    >
                        {markerDeg}°
                    </text>
                )}
            </g>
        );
    }
}
</g>
)}

```

```

    { /* 2. Soft Dynamic Transparent Arc Sector Fill */
    {angle > 0 && (
      <path
        d={arcPathData}
        fill="rgba(236, 72, 153, 0.16)"
        stroke="rgba(236, 72, 153, 0.4)"
        strokeWidth="2"
        className="transition-all duration-75"
      />
    )}

    { /* 3. Base Horizontal Ground Ray (Fixed at 0 degrees) */
    <g>
      <line x1={cx} y1={cy} x2={cx + r + 15} y2={cy}
stroke="#1e293b" strokeWidth="6" strokeLinecap="round" />
      <polygon points={` ${cx+r+25}, ${cy} ${cx+r+13}, ${cy-7}
${cx+r+13}, ${cy+7} `} fill="#1e293b" />
      <text x={cx + 50} y={cy + 18} fontSize="11"
fontWeight="bold" fill="#64748b" className="tracking-wide">BASE LINE
(0°)</text>
    </g>

    { /* 4. Right Angle Corner Square Box Marker (Only appears
if exactly 90°) */
    {angle === 90 && (
      <path
        d={`M ${cx+24} ${cy} L ${cx+24} ${cy-24} L ${cx}
${cy-24} `}

        fill="none"
        stroke="#f59e0b"
        strokeWidth="3.5"
        strokeLinecap="square"
        className="animate-pulse"
      />
    )}
  )}

```

```

    { /* 5. Moving Dynamic Rotating Ray Arrow Vector */
    <g
      onMouseDown={handleMouseDown}
      onTouchStart={handleMouseDown}
      className="group cursor-pointer"
    >
      { /* Invisible thick outer stroke detector for comfortable
touch interactions */}
      <line
        x1={cx}
        y1={cy}
        x2={xTarget}
        y2={yTarget}
        stroke="transparent"
        strokeWidth="24"
        strokeLinecap="round"
      />

      { /* Visible magenta line vector */}
      <line
        x1={cx}
        y1={cy}
        x2={xTarget}
        y2={yTarget}
        stroke="#ec4899"
        strokeWidth="6"
        strokeLinecap="round"
        className="transition-all duration-75
group-hover:stroke-pink-600"
      />

      { /* Arrowhead endpoint */}
      <g transform={`translate(${xTarget}, ${yTarget})
rotate(${ -angle})`} >

```

```

        <polygon points="12,0 0,-7 0,7" fill="#ec4899"
className="transition-all duration-75 group-hover:fill-pink-600" />
        <circle cx="0" cy="0" r="12" fill="transparent" />
    </g>
</g>

    { /* 6. Glowing Circular Pivot Point (Vertex) */ }
    <circle cx={cx} cy={cy} r="10" fill="#f97316"
stroke="#ffffff" strokeWidth="3" className="shadow-lg" />
    </svg>
</div>

    { /* Live Interactive Readout Bubble */ }
    <div className="mt-2 bg-slate-900 text-white rounded-2xl px-6
py-2.5 shadow-md flex items-center gap-3 border border-slate-700">
        <span
className="text-xl">{currentClassification.emoji}</span>
        <div className="font-mono font-black text-2xl tracking-tight
text-pink-400">
            {angle}°
        </div>
        <div className="h-4 w-px bg-slate-700"></div>
        <div className="text-xs font-bold text-slate-300 uppercase
tracking-widest">
            {currentClassification.name}
        </div>
    </div>

</div>

    { /* Quick Access Control Board Panel */ }
    <div className="bg-white rounded-2xl p-4 border
border-slate-200/80 shadow-sm flex flex-col sm:flex-row items-center
justify-between gap-4">

        { /* Massive Easy-to-use Range Slider Control */ }

```

```

    <div className="w-full flex-1 flex items-center gap-3">
      <span className="text-xs font-black text-slate-400 uppercase
tracking-widest">Adjust Turn</span>
      <input
        type="range"
        min="0"
        max="360"
        value={angle}
        onChange={ (e) => {
          const val = parseInt(e.target.value, 10);
          // Dynamic snapping at critical quadrants
          if (Math.abs(val - 90) <= 2) setAngle(90);
          else if (Math.abs(val - 180) <= 2) setAngle(180);
          else if (Math.abs(val - 270) <= 2) setAngle(270);
          else if (Math.abs(val - 360) <= 2 || val < 2)
setAngle(0);

          else setAngle(val);
        }}
        className="w-full h-3 bg-slate-100 rounded-lg
appearance-none cursor-pointer accent-pink-500 border border-slate-200"
      />
    </div>

```

```

    { /* Display Assistant Toggles */ }
    <div className="flex gap-2 w-full sm:w-auto justify-end">
      <button
        onClick={ () => setShowProtractor(!showProtractor) }
        className={`flex-1 sm:flex-none px-4 py-2.5 rounded-xl
text-xs font-bold transition-all border shadow-sm flex items-center
justify-center gap-1 ${
          showProtractor
            ? 'bg-slate-800 text-white border-slate-900
hover:bg-slate-700'
            : 'bg-white text-slate-700 border-slate-200
hover:bg-slate-50'
        }}
      />
    </div>

```

```

        >
        <Icons.Eye />
        <span>{showProtractor ? "Protractor On" : "Protractor
Off"}</span>
    </button>
</div>

</div>

</section>

{/* Right Information & gamified panels (5 Columns) */}
<section className="lg:col-span-5 flex flex-col gap-5">

    {/* Dynamic Tab Switch Board */}
    {activeTab === 'explore' ? (

        // EXPLORE PANEL
        <div className="flex-1 flex flex-col gap-5">

            {/* Dynamic Readout Card */}
            <div className={`p-6 rounded-3xl border transition-all
duration-300 ${currentClassification.bg} ${currentClassification.border}
shadow-md flex flex-col justify-between`}>
                <div>
                    <div className="flex justify-between items-start">
                        <span className={`text-xs font-black uppercase
tracking-widest px-2.5 py-1 rounded-md ${currentClassification.badge}`}>
                            Angle Category
                        </span>
                        <span
className="text-3xl">{currentClassification.emoji}</span>
                    </div>
                    <h2 className={`text-3xl font-extrabold
${currentClassification.color} mt-4 tracking-tight`}>
                        {currentClassification.name}

```

```

        </h2>
        <p className="text-slate-600 font-medium mt-1 text-sm
leading-relaxed">
            {currentClassification.desc}
        </p>
    </div>

    <div className="mt-6 pt-5 border-t border-slate-200/50 flex
items-center justify-between gap-3 text-slate-500 text-xs">
        <span className="font-bold uppercase
tracking-wider">Memory Rule:</span>
        <span className="font-semibold text-slate-700 text-right
bg-white px-3 py-1 rounded-lg border border-slate-200 shadow-sm">
            {angle > 0 && angle < 90 && "Cute little angle (<
90°)"}

            {angle === 90 && "Perfect corner (= 90°)"}
            {angle > 90 && angle < 180 && "Obtuse is heavy and wide
(> 90°)"}

            {angle === 180 && "Perfect straight horizontal line (=
180°)"}

            {angle > 180 && angle < 360 && "Reflex wraps around the
back (> 180°)"}

            {angle === 360 && "One full spin (= 360°)"}
            {angle === 0 && "Zero opening (= 0°)"}
        </span>
    </div>
</div>

    {/* Dynamic Real World Connection Card */}
    <div className="bg-white rounded-3xl p-5 border
border-slate-200 shadow-sm flex items-center gap-4">
        <div className="text-4xl p-3 bg-indigo-50 rounded-2xl
border border-indigo-100">
            {illustration.icon}
        </div>
    </div>

```

```

        <h4 className="text-xs font-black text-slate-400
uppercase tracking-widest">Real-World Match</h4>
        <p className="text-lg font-bold text-slate-800
mt-0.5">{illustration.label}</p>
        <p className="text-xs text-slate-500 leading-relaxed
mt-0.5">Found daily in objects turned at {angle}°.</p>
    </div>
</div>

{/* Discovery Tracker Milestone Achievements Progress Box */}
<div className="bg-white rounded-3xl p-5 border
border-slate-200 shadow-sm flex-1">
    <div className="flex justify-between items-center mb-4">
        <h4 className="text-xs font-bold text-slate-400 uppercase
tracking-wider">Milestone Checklist</h4>
        <span className="text-xs font-bold bg-indigo-50
text-indigo-700 px-2 py-1 rounded-md">
            {discovered.length}/5 Found
        </span>
    </div>

    <div className="grid grid-cols-1 gap-2.5">
        {[ 'Acute', 'Right', 'Obtuse', 'Straight',
'Reflex' ].map((m) => {
            const isFound = discovered.includes(m);
            return (
                <div
                    key={m}
                    className={`p-3 rounded-2xl border flex
items-center justify-between transition-all ${
                        isFound
                            ? 'bg-emerald-50/50 border-emerald-100
text-slate-800'
                            : 'bg-slate-50/50 border-slate-100
text-slate-400'
                    }`
                >
            )
        })
    </div>

```



```

>
<div className="flex items-center gap-3">
  <span className={`text-md ${isFound ?
'opacity-100' : 'opacity-30'}}`>
    {m === 'Acute' && '📐'}
    {m === 'Right' && '🧱'}
    {m === 'Obtuse' && '🪑'}
    {m === 'Straight' && '📏'}
    {m === 'Reflex' && '🌀'}
  </span>
  <span className="text-sm font-semibold">{m}
Angle</span>
</div>
{isFound ? (
  <div className="flex items-center gap-1
bg-emerald-100 text-emerald-800 text-xs px-2.5 py-0.5 rounded-full
font-bold">
    <span>+40 XP</span>
    <Icons.CheckCircle />
  </div>
) : (
  <span className="text-xs font-semibold
text-slate-400">Locked</span>
)}
</div>
);
}}
</div>
</div>

) : (

// QUESTS/CHALLENGES PANEL
<div className="flex-1 flex flex-col gap-4">

```

```

    { /* Quest HUD Card */
    <div className="bg-white rounded-3xl p-6 border
border-rose-200/80 shadow-md relative overflow-hidden">
        <div className="absolute top-0 right-0 w-32 h-32 bg-rose-50
rounded-full -mr-10 -mt-10 -z-0 opacity-50"></div>

        <div className="relative z-10">
            <span className="bg-rose-100 text-rose-800 text-xs
font-black uppercase tracking-widest px-3 py-1 rounded-md">
                Quest {currentChallenge + 1} of {challenges.length}
            </span>

            <h3 className="text-2xl font-black text-slate-900 mt-4
leading-tight">
                {challenges[currentChallenge].title}
            </h3>

            <p className="text-slate-600 font-medium text-sm mt-2
leading-relaxed bg-slate-50 p-3 rounded-2xl border border-slate-100">
                {challenges[currentChallenge].prompt}
            </p>

            <div className="mt-5 p-3 bg-amber-50 rounded-2xl border
border-amber-100 text-amber-900 text-xs flex gap-2">
                <span className="text-base">💡 </span>
                <div>
                    <span className="font-extrabold uppercase
tracking-wider block mb-0.5 text-amber-700">Professor's Tip</span>
                    <p className="leading-relaxed
font-medium">{challenges[currentChallenge].tip}</p>
                </div>
            </div>
        </div>
    </div>
}

```

```

    { /* Dynamic Validation Card */ }
    <div className={`p-6 rounded-3xl border text-center
transition-all ${
    challengeCompleted
      ? 'bg-emerald-50 border-emerald-200 text-emerald-900'
      : 'bg-slate-50 border-slate-200 text-slate-400'
    }`} >
      {challengeCompleted ? (
        <div className="animate-scale-up">
          <div className="text-5xl mb-3
animate-bounce">🌟🏆🌟</div>
          <h4 className="text-xl font-extrabold
text-emerald-800">Challenge Mastered!</h4>
          <p className="text-sm text-emerald-600 mt-1
font-medium">
            {challenges[currentChallenge].successMsg}
          </p>
          <div className="my-4 inline-block bg-emerald-100
text-emerald-800 text-xs px-4 py-1.5 rounded-full font-black
tracking-wide">
            +60 XP Reward Acquired!
          </div>

          <button
            onClick={handleNextChallenge}
            className="w-full mt-2 bg-emerald-600
hover:bg-emerald-700 text-white font-bold py-3.5 px-6 rounded-2xl
shadow-md active:scale-95 transition text-md"
          >
            {currentChallenge === challenges.length - 1 ?
"Complete Quest Journey" : "Next Quest Task →"}
          </button>
        </div>
      ) : (
        <div>
          <div className="text-3xl mb-2">⚙️</div>

```

```

        <h4 className="text-md font-bold
text-slate-600">Checking parameters...</h4>
        <p className="text-xs text-slate-400 mt-1
leading-relaxed">
            Rotate the pink ray to align with the prompt
requirements.
        </p>
    </div>
    })
</div>

</div>

    })

</section>

</main>

{/* Structured NCERT Textbook Summary/Help Card Footer */}
<footer className="bg-white border-t border-slate-200 py-6 px-4">
    <div className="max-w-6xl mx-auto flex flex-col sm:flex-row
justify-between items-center gap-4 text-xs text-slate-500 font-medium">

        <div className="flex items-center gap-2">
            <span className="text-lg">💡</span>
            <p className="text-center sm:text-left leading-relaxed">
                <strong>Memory Tip:</strong> Think of <strong>Acute</strong>
as "a-cute" (small, cute); <strong>Obtuse</strong> is heavy & wide;
<strong>Right</strong> is perfect; <strong>Straight</strong> is flat like
a road!
            </p>
        </div>

        <div className="text-right whitespace-nowrap">
            NCERT Mathematics Syllabus • CBSE/ICSE Approved Lab Unit
        </div>
    </div>
</footer>

```

</div>

</div>

</footer>

</div>

);

}

Reflection in Mirrors 6–7

```
import React, { useState, useEffect, useRef, useMemo } from 'react';
```

```
// Custom lightweight SVG Icons to avoid import dependency issues
```

```
const Icons = {  
  Sparkles: () => (  
    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"  
    fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"  
    className="inline-block mr-1"><path d="m12 3-1.912 5.813a2 2 0 0 1-1.275 1.275L3 12l5.813  
1.912a2 2 0 0 1 1.275 1.275L12 21l1.912-5.813a2 2 0 0 1 1.275-1.275L21 12l-5.813-1.912a2 2  
0 0 1-1.275-1.275L12 3Z"/><path d="m5 3 1 2.5L8.5 6 6 7 5 9.5 4 7 1.5 6 4 5.5z"/><path  
d="m19 17 1 2.5 2.5 5-2.5 1-1 2.5-1-2.5-2.5-1 2.5-1z"/></svg>  
  ),  
  Flame: () => (  
    <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24"  
    fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"  
    className="text-orange-500 animate-pulse"><path d="M8.5 14.5A2.5 2.5 0 0 0 11  
12c0-1.38-.5-2-1-3-1.072-2.143-.224-4.054 2-6 .5 2.5 2 4.9 4 6.5 2 1.6 3 3.5 3 5.5a7 7 0 1 1-14  
0c0-1.153.433-2.294 1-3a2.5 2.5 0 0 0 2.5 2.5z"/></svg>  
  ),  
  Ruler: () => (  
    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"  
    fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"  
    className="mr-2"><path d="M21.3 15.3a2.82 2.82 0 0 1 0 4c-1 1-2.5 1-3.5 0L2.3 3.8a2.82  
2.82 0 0 1 0-4c1-1 2.5-1 3.5 0Z"/><path d="m5.6 7.2 1.4-1.4"/><path d="m7.2 10.4  
1.4-1.4"/><path d="m10.4 12 1.4-1.4"/><path d="m12 15.2 1.4-1.4"/><path d="m15.2 16.8  
1.4-1.4"/></svg>  
  ),  
  BookOpen: () => (  
    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"  
    fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"  
    className="mr-2"><path d="M2 3h6a4 4 0 0 1 4 4v14a3 3 0 0 0-3-3H2z"/><path d="M22  
3h-6a4 4 0 0 0-4 4v14a3 3 0 0 1 3-3h7z"/></svg>  
  ),  
  Trophy: () => (  
    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"  
    fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"  
    className="text-yellow-400 mr-2"><path d="M6 9H4.5a2.5 2.5 0 0 1 0-5H6"/><path d="M18  
9h1.5a2.5 2.5 0 0 0 0-5H18"/><path d="M4 22h16"/><path d="M10 14.66V17c0 .55-.45 1-1  
1H4v2h16v-2h-5c-.55 0-1-.45-1-1v-2.34"/><path d="M12 2a6 6 0 0 1 6 6v1c0 2.2-1.8 4-4  
4h-4c-2.2 0-4-1.8-4-4V8a6 6 0 0 1 6-6z"/></svg>  
  ),  
  RotateCcw: () => (  
    <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"  
    fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
```

```

className="mr-1"><path d="M3 12a9 9 0 1 0 9-9 9.75 9.75 0 0 0-6.74 2.74L3 8"/><path d="M3
3v5h5"/></svg>
),
Eye: () => (
  <svg xmlns="http://www.w3.org/2000/svg" width="22" height="22" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" className="text-emerald-400"><path d="M2
12s3-7 10-7 10 7 10 7-3 7-10 7-10-7-10-7Z"/><circle cx="12" cy="12" r="3"/></svg>
),
Award: () => (
  <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
className="text-pink-400 mr-1"><circle cx="12" cy="8" r="7"/><path d="M8.21 13.89 7 23l5-3
5 3-1.21-9.12"/></svg>
),
HelpCircle: () => (
  <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
className="mr-2"><circle cx="12" cy="12" r="10"/><path d="M9.09 9a3 3 0 0 1 5.83 1c0 2-3
3-3 3"/><path d="M12 17h.01"/></svg>
),
Lightbulb: () => (
  <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
className="text-yellow-400 mr-2"><path d="M15 14c.2-1 .7-1.7 1.5-2.5 1-.9 1.5-2.2 1.5-3.5A6
6 0 0 0 6 8c0 1 .2 2.2 1.5 3.5.7 1.3 1.5 1.5 2.5"/><path d="M9 18h6"/><path d="M10
22h4"/></svg>
)
};

```

```
// Sound synthesizer using Web Audio API
```

```

const playSound = (type) => {
  try {
    const AudioContext = window.AudioContext || window.webkitAudioContext;
    if (!AudioContext) return;
    const ctx = new AudioContext();

    if (type === 'click') {
      const osc = ctx.createOscillator();
      const gain = ctx.createGain();
      osc.type = 'sine';
      osc.frequency.setValueAtTime(600, ctx.currentTime);
      osc.frequency.exponentialRampToValueAtTime(150, ctx.currentTime + 0.1);
      gain.gain.setValueAtTime(0.1, ctx.currentTime);
      gain.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.1);
    }
  }
};

```



```

    osc.connect(gain);
    gain.connect(ctx.destination);
    osc.start();
    osc.stop(ctx.currentTime + 0.1);
  } else if (type === 'success') {
    // Play a beautiful ascending major chord chime
    const freqs = [330, 440, 554, 660, 880];
    freqs.forEach((freq, idx) => {
      const osc = ctx.createOscillator();
      const gain = ctx.createGain();
      osc.type = 'triangle';
      osc.frequency.setValueAtTime(freq, ctx.currentTime + idx * 0.08);
      gain.gain.setValueAtTime(0.15, ctx.currentTime + idx * 0.08);
      gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + idx * 0.08 + 0.4);
      osc.connect(gain);
      gain.connect(ctx.destination);
      osc.start(ctx.currentTime + idx * 0.08);
      osc.stop(ctx.currentTime + idx * 0.08 + 0.4);
    });
  } else if (type === 'correct') {
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();
    osc.type = 'sine';
    osc.frequency.setValueAtTime(523.25, ctx.currentTime); // C5
    osc.frequency.setValueAtTime(659.25, ctx.currentTime + 0.1); // E5
    gain.gain.setValueAtTime(0.15, ctx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.25);
    osc.connect(gain);
    gain.connect(ctx.destination);
    osc.start();
    osc.stop(ctx.currentTime + 0.25);
  } else if (type === 'wrong') {
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();
    osc.type = 'sawtooth';
    osc.frequency.setValueAtTime(150, ctx.currentTime);
    osc.frequency.linearRampToValueAtTime(100, ctx.currentTime + 0.2);
    gain.gain.setValueAtTime(0.15, ctx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.2);
    osc.connect(gain);
    gain.connect(ctx.destination);
    osc.start();
    osc.stop(ctx.currentTime + 0.2);
  }
}

```

```

    } catch (e) {
      console.log("Audio contexts locked or unsupported", e);
    }
  };

export default function App() {
  // State variables
  const [activeTab, setActiveTab] = useState('sim'); // 'sim' | 'quiz' | 'diy' | 'concept'
  const [objectDistance, setObjectDistance] = useState(5.0); // 1.0 to 10.0 cm
  const [objectType, setObjectType] = useState('candle'); // 'candle' | 'letterF' | 'letterA'
  const [isRayTraceOn, setIsRayTraceOn] = useState(true);
  const [isHandRaised, setIsHandRaised] = useState(false);
  const [isTrueMirror, setIsTrueMirror] = useState(false); // True mirror doesn't invert laterally

  // Scoring & gamification state
  const [xp, setXp] = useState(0);
  const [stars, setStars] = useState(3);
  const [completedChallenges, setCompletedChallenges] = useState([]);
  const [currentChallengeId, setCurrentChallengeId] = useState(0);
  const [confetti, setConfetti] = useState([]);

  // Canvas / SVG dragging reference
  const svgRef = useRef(null);
  const [isDragging, setIsDragging] = useState(false);

  // Challenge definitions
  const challenges = [
    {
      id: 0,
      title: "Perfect Matcher",
      instruction: "Move the object so that the distance from the mirror is exactly 6.0 cm.",
      targetValue: 6.0,
      rewardXp: 50,
      hint: "Drag the object left or right until the status panel shows 6.0 cm."
    },
    {
      id: 1,
      title: "Dual Distance Matrix",
      instruction: "Set the total distance between the Object and its Mirror Image to exactly 16.0 cm.",
      targetValue: 8.0, // Object at 8, Image at 8 = 16
      rewardXp: 70,
      hint: "Remember: Total distance = Object Distance + Image Distance. That means Object must be at 8.0 cm!"
    }
  ]

```

```

    },
    {
      id: 2,
      title: "Close Reflection",
      instruction: "Get the object as close to the mirror surface as possible (exactly 2.0 cm).",
      targetValue: 2.0,
      rewardXp: 90,
      hint: "Slide it closer to the shiny center glass line."
    }
  ];

```

```

// Particle explosion logic
const triggerConfetti = () => {
  playSound('success');
  const newConfetti = Array.from({ length: 40 }).map((_, i) => ({
    id: Date.now() + i,
    x: 300,
    y: 180,
    vx: (Math.random() - 0.5) * 8,
    vy: (Math.random() - 0.7) * 8 - 4,
    color: ['#f59e0b', '#38bdf8', '#10b981', '#ec4899', '#a855f7'][Math.floor(Math.random() * 5)],
    size: Math.random() * 6 + 4,
    life: 1.0
  }));
  setConfetti(newConfetti);
};

```

```

// Particle system loop
useEffect(() => {
  if (confetti.length === 0) return;
  const interval = setInterval(() => {
    setConfetti((prev) =>
      prev
        .map((p) => ({
          ...p,
          x: p.x + p.vx,
          y: p.y + p.vy,
          vy: p.vy + 0.15, // gravity
          life: p.life - 0.03
        }))
        .filter((p) => p.life > 0)
    );
  }, 20);
  return () => clearInterval(interval);

```

```
}, [confetti]);
```

```
// Handle challenge completion check
```

```
useEffect(() => {  
  const currentChallenge = challenges[currentChallengeId];  
  if (currentChallenge && !completedChallenges.includes(currentChallenge.id)) {  
    if (Math.abs(objectDistance - currentChallenge.targetValue) < 0.05) {  
      setCompletedChallenges([...completedChallenges, currentChallenge.id]);  
      setXp(prev => prev + currentChallenge.rewardXp);  
      triggerConfetti();  
  
      // Auto-progress to next challenge after short delay  
      if (currentChallengeId < challenges.length - 1) {  
        setTimeout(() => {  
          setCurrentChallengeId(prev => prev + 1);  
        }, 3000);  
      }  
    }  
  }  
}, [objectDistance, currentChallengeId]);
```

```
// Quiz state
```

```
const [currentQuizIndex, setCurrentQuizIndex] = useState(0);  
const [selectedAnswer, setSelectedAnswer] = useState(null);  
const [quizScore, setQuizScore] = useState(0);  
const [showExplanation, setShowExplanation] = useState(false);  
const [quizFinished, setQuizFinished] = useState(false);
```

```
const quizQuestions = [  
  {  
    q: "If you stand 3 meters away from a plane mirror, how far away is your image from the mirror surface?",  
    options: ["1.5 meters", "3 meters", "6 meters", "0 meters"],  
    correct: 1,  
    exp: "Since Object Distance = Image Distance in a plane mirror, if you stand 3m away, your twin inside the mirror is also exactly 3m away!"  
  },  
  {  
    q: "You look into a plane mirror and wink your right eye. Which eye does your mirror image appear to wink?",  
    options: ["Right eye", "Left eye", "Both eyes", "Neither"],  
    correct: 1,  
    exp: "This is Lateral Inversion! The left side of the object appears as the right side of the image, so winking your right eye makes your mirror twin wink their left eye."
```

```

    },
    {
        q: "The image formed by a plane mirror is always:",
        options: ["Inverted & Magnified", "Erect & Same Size", "Upside-down & Small", "Erect & Double Size"],
        correct: 1,
        exp: "A plane mirror creates a perfectly erect (upright) image that is the exact same size as the real-world object."
    },
    {
        q: "An image that can NOT be captured on a real screen or sheet of paper is called a:",
        options: ["Real Image", "Erect Image", "Virtual Image", "Shadow"],
        correct: 2,
        exp: "Because the reflected light rays do not actually meet but only appear to meet behind the mirror, we call it a Virtual Image."
    },
    {
        q: "If the total distance between you and your mirror image is 10 cm, how far are you standing from the mirror?",
        options: ["10 cm", "20 cm", "5 cm", "2.5 cm"],
        correct: 2,
        exp: "The total distance is the sum of object distance and image distance ( $D_o + D_i$ ). Since  $D_o = D_i$ , your distance from the mirror is  $10 / 2 = 5$  cm."
    },
    {
        q: "Why does the front of an ambulance have the word 'AMBULANCE' written backwards (Ǝ᠒I᠕_ǝ᠘_᠒_᠕A)?",
        options: [
            "To look beautiful",
            "So drivers looking at their rearview mirrors can read it normally due to lateral inversion",
            "It is a foreign spelling",
            "To save space on the bumper"
        ],
        correct: 1,
        exp: "The driver in front sees the word in their rearview mirror, which laterally inverts it back to normal 'AMBULANCE', letting them give way immediately!"
    },
    {
        q: "Which of the following surfaces will produce a clear, sharp reflection?",
        options: ["A rough cardboard sheet", "A highly polished steel plate", "A clean piece of wood", "A dry concrete wall"],
        correct: 1,
        exp: "Highly polished steel plate has a perfectly flat surface, causing regular reflection of light rays in one direction to produce a clean image."
    }

```

```

    },
    {
      q: "In a plane mirror reflection, if a student runs closer to the mirror, what happens to the image?",
      options: ["It moves further back", "It moves closer to the mirror surface at the same speed", "It stays completely still", "It turns upside down"],
      correct: 1,
      exp: "The mirror image mimics your movement perfectly! If you step closer, it moves closer to the mirror boundary too."
    },
    {
      q: "What is the secret element hidden behind mirror glass that actually does the reflection work?",
      options: ["A layer of white paint", "A thin metallic coating of silver or aluminum", "Special dark paper", "Nothing, glass is naturally reflective"],
      correct: 1,
      exp: "Ordinary glass is transparent. The highly reflective layer is a thin backing of silver or aluminum metal painted over to prevent scratches."
    },
    {
      q: "State whether true or false: A plane mirror flips your reflection vertically (meaning your head goes to the bottom).",
      options: ["True", "False"],
      correct: 1,
      exp: "False! Plane mirrors only flip horizontally (left-right), keeping the vertical orientation erect (upright)."
    }
  ];

```

// Drag handlers

```

const handlePointerDown = (e) => {
  setIsDragging(true);
  playSound('click');
};

```

```

const handlePointerMove = (e) => {
  if (!isDragging || !svgRef.current) return;
  const rect = svgRef.current.getBoundingClientRect();
  const x = e.clientX - rect.left;

```

// We want to map the coordinates

// Center of SVG (Width 600px) is x = 300 (Mirror Line)

// The draggable area for object is from x = 50 to x = 270 (Left side)

```

const relativeX = Math.max(50, Math.min(270, x));

```

```

// Convert 50-270px to 1.0-10.0 cm range
// 270px = 1.0cm (Very close)
// 50px = 10.0cm (Far away)
const distanceVal = 1.0 + ((270 - relativeX) / 220) * 9.0;
setObjectDistance(parseFloat(distanceVal.toFixed(1)));
};

const handlePointerUp = () => {
  setIsDragging(false);
};

useEffect(() => {
  const handleGlobalMouseUp = () => setIsDragging(false);
  window.addEventListener('mouseup', handleGlobalMouseUp);
  return () => window.removeEventListener('mouseup', handleGlobalMouseUp);
}, []);

const resetAll = () => {
  playSound('click');
  setObjectDistance(5.0);
  setObjectType('candle');
  setIsRayTraceOn(true);
  setIsHandRaised(false);
  setIsTrueMirror(false);
  setCompletedChallenges([]);
  setCurrentChallengeId(0);
  setXp(0);
};

// Helper values for drawing
const scale = 23; // pixels per cm
const centerLineX = 300;
const objectX = centerLineX - (objectDistance * scale);
const imageX = centerLineX + (objectDistance * scale);
const objectY = 200;

return (
  <div className="flex flex-col min-h-screen bg-slate-950 text-slate-100 font-sans
selection:bg-sky-500 selection:text-white">
    {/* HEADER SECTION */}
    <header className="bg-slate-900 border-b border-slate-800 py-3 px-6 shadow-md sticky
top-0 z-40">

```

```
<div className="max-w-7xl mx-auto flex flex-col md:flex-row items-center justify-between gap-4">
```

```
  <div className="flex items-center gap-3">
    <div className="p-2 bg-gradient-to-tr from-sky-500 to-indigo-600 rounded-xl shadow-lg shadow-sky-900/30">
      <Icons.Sparkles />
    </div>
    <div>
      <div className="flex items-center gap-2">
        <span className="text-xs font-semibold uppercase tracking-widest bg-sky-500/10 text-sky-400 px-2.5 py-0.5 rounded-full border border-sky-500/20">NCERT Science</span>
        <span className="text-xs font-semibold uppercase tracking-widest bg-emerald-500/10 text-emerald-400 px-2.5 py-0.5 rounded-full border border-emerald-500/20">Class 6-7</span>
      </div>
      <h1 className="text-xl md:text-2xl font-bold bg-clip-text text-transparent bg-gradient-to-r from-white via-slate-100 to-slate-400">
        The Magic Mirror Matrix
      </h1>
    </div>
  </div>
```

```
{/* Gamification Dashboard */}
<div className="flex items-center gap-4 bg-slate-950/60 p-2 rounded-2xl border border-slate-800/80">
  <div className="flex items-center gap-2 px-3">
    <span className="text-amber-400 text-lg font-bold"> ✨ {xp}</span>
    <span className="text-slate-400 text-xs">XP</span>
  </div>

  <div className="h-6 w-px bg-slate-800" />

  <div className="flex items-center gap-1.5 px-3">
    <span className="text-yellow-400"> ★ </span>
    <span className="text-sm font-semibold">{completedChallenges.length}/{challenges.length} Done</span>
  </div>
```

```
  <div className="h-6 w-px bg-slate-800" />

  <div className="px-3">
    <span className="text-xs bg-indigo-500/10 text-indigo-300 border border-indigo-500/20 px-2 py-1 rounded-md font-semibold">
```



```

        {xp > 150 ? '🏆 Master Explorer' : xp > 50 ? '🏆 Novice Reflector' : '🐣 Explorer'}
      </span>
    </div>
  </div>

</div>
</header>

{ /* TABS NAVIGATION */ }
<nav className="bg-slate-900/40 border-b border-slate-800/50 sticky top-[69px] z-30
backdrop-blur-md">
  <div className="max-w-7xl mx-auto px-4 flex justify-start gap-1 p-2 overflow-x-auto">
    <button
      onClick={() => { playSound('click'); setActiveTab('sim'); }}
      className={`flex items-center px-4 py-2 text-sm font-medium rounded-lg transition-all
$ {
      activeTab === 'sim'
        ? 'bg-sky-600/20 text-sky-400 border border-sky-500/30'
        : 'text-slate-400 hover:text-slate-200 hover:bg-slate-800/40'
      }}
    >
      <Icons.Eye /> <span className="ml-2">Interactive Lab</span>
    </button>

    <button
      onClick={() => { playSound('click'); setActiveTab('quiz'); }}
      className={`flex items-center px-4 py-2 text-sm font-medium rounded-lg transition-all
$ {
      activeTab === 'quiz'
        ? 'bg-sky-600/20 text-sky-400 border border-sky-500/30'
        : 'text-slate-400 hover:text-slate-200 hover:bg-slate-800/40'
      }}
    >
      <Icons.Trophy /> <span className="ml-2">Practice Challenge</span>
    </button>

    <button
      onClick={() => { playSound('click'); setActiveTab('diy'); }}
      className={`flex items-center px-4 py-2 text-sm font-medium rounded-lg transition-all
$ {
      activeTab === 'diy'
        ? 'bg-sky-600/20 text-sky-400 border border-sky-500/30'
        : 'text-slate-400 hover:text-slate-200 hover:bg-slate-800/40'
      }}
    >

```

```

    >
    <Icons.Ruler /> <span className="ml-2">DIY Fun Activity</span>
  </button>

  <button
    onClick={() => { playSound('click'); setActiveTab('concept'); }}
    className={`flex items-center px-4 py-2 text-sm font-medium rounded-lg transition-all
$ {
      activeTab === 'concept'
        ? 'bg-sky-600/20 text-sky-400 border border-sky-500/30'
        : 'text-slate-400 hover:text-slate-200 hover:bg-slate-800/40'
    }`
  >
    <Icons.BookOpen /> <span className="ml-2">Concept Guide</span>
  </button>
</div>
</nav>

{/* MAIN CONTAINER */}
<main className="flex-grow max-w-7xl w-full mx-auto p-4 md:p-6">

  {/*
===== */}
  {/* TAB 1: INTERACTIVE SIMULATOR */}
  {/*
===== */}
  {activeTab === 'sim' && (
    <div className="grid grid-cols-1 lg:grid-cols-12 gap-6">

      {/* Left Content Column - Canvas & Controls (8/12 grid) */}
      <div className="lg:col-span-8 flex flex-col gap-5">

        {/* CURRENT ACTIVE QUEST HEADER */}
        {challenges[currentChallengeId] && (
          <div className="bg-gradient-to-r from-sky-950/40 to-indigo-950/40 border
border-sky-800/30 rounded-2xl p-4 flex items-center justify-between gap-4">
            <div className="flex items-start gap-3">
              <div className="bg-sky-500/20 text-sky-400 p-2.5 rounded-xl border
border-sky-500/30">
                <Icons.Sparkles />
              </div>
            </div>
            <div className="flex items-center gap-2">

```

```

        <span className="text-xs font-bold text-sky-400 uppercase
tracking-wider">Current Mission</span>
        {completedChallenges.includes(challenges[currentChallengeId].id) && (
            <span className="text-xs bg-emerald-500/10 text-emerald-400 px-2 py-0.5
rounded-md border border-emerald-500/20">Completed!
+{challenges[currentChallengeId].rewardXp} XP</span>
        )}
    </div>
    <h3 className="font-bold text-slate-100 text-sm
md:text-base">{challenges[currentChallengeId].title}</h3>
    <p className="text-slate-400 text-xs
mt-0.5">{challenges[currentChallengeId].instruction}</p>
</div>
</div>

    {/* Score circle */}
    <div className="flex flex-col items-center justify-center bg-slate-900 border
border-slate-800 w-16 h-16 rounded-2xl shrink-0">
        <span className="text-xs text-slate-400">Award</span>
        <span className="font-bold
text-amber-400">+{challenges[currentChallengeId].rewardXp}</span>
    </div>
</div>
)}

    {/* SIMULATOR SCREEN */}
    <div className="relative bg-slate-900 border border-slate-800 rounded-3xl
overflow-hidden shadow-2xl">

        {/* Labels/Legend overlay */}
        <div className="absolute top-4 left-4 right-4 flex justify-between items-center
pointer-events-none z-10">
            <div className="bg-amber-500/10 border border-amber-500/20 backdrop-blur-md
px-3 py-1.5 rounded-xl">
                <span className="text-xs font-bold text-amber-400 tracking-wide">🌟 REAL
WORLD (Left)</span>
            </div>
            <div className="bg-sky-500/10 border border-sky-500/20 backdrop-blur-md px-3
py-1.5 rounded-xl">
                <span className="text-xs font-bold text-sky-400 tracking-wide">❄️ MIRROR
WORLD (Right)</span>
            </div>
        </div>
    </div>

```

```

    { /* SVG Live Simulation */ }
    <svg
      ref={svgRef}
      viewBox="0 0 600 360"
      className="w-full h-auto select-none bg-gradient-to-b from-slate-950
via-slate-900/90 to-slate-950"
      onPointerDown={handlePointerDown}
      onPointerMove={handlePointerMove}
      onPointerUp={handlePointerUp}
      style={{ touchAction: 'none' }}
    >
      { /* Subtle Background Shading for both worlds */ }
      <rect x="0" y="0" width="300" height="360" fill="url(#real-grad)" />
      <rect x="300" y="0" width="300" height="360" fill="url(#mirror-grad)" />

      <defs>
        <linearGradient id="real-grad" x1="0%" y1="0%" x2="100%" y2="0%">
          <stop offset="0%" stopColor="#f59e0b" stopOpacity="0.01" />
          <stop offset="100%" stopColor="#f59e0b" stopOpacity="0.05" />
        </linearGradient>
        <linearGradient id="mirror-grad" x1="0%" y1="0%" x2="100%" y2="0%">
          <stop offset="0%" stopColor="#38bdf8" stopOpacity="0.06" />
          <stop offset="100%" stopColor="#38bdf8" stopOpacity="0.01" />
        </linearGradient>

        { /* Candle SVG Definitions */ }
        <g id="candle-svg">
          { /* Body */ }
          <rect x="-10" y="0" width="20" height="60" fill="#f43f5e" rx="4" />
          <rect x="-8" y="5" width="16" height="50" fill="#e11d48" rx="2" />
          <line x1="0" y1="0" x2="0" y2="-8" stroke="#f43f5e" strokeWidth="2" />
          { /* Wax drips */ }
          <rect x="-10" y="8" width="3" height="15" fill="#be123c" rx="1.5" />
          <rect x="7" y="14" width="3" height="8" fill="#be123c" rx="1.5" />
        </g>

        <g id="ghost-candle-svg">
          { /* Semi-transparent Image copy */ }
          <rect x="-10" y="0" width="20" height="60" fill="#0ea5e9" fillOpacity="0.6" rx="4"

/>
          <rect x="-8" y="5" width="16" height="50" fill="#0284c7" fillOpacity="0.6" rx="2" />
          <line x1="0" y1="0" x2="0" y2="-8" stroke="#0ea5e9" strokeWidth="2" />
          { /* Wax drips */ }

```

/>

```
<rect x="-10" y="8" width="3" height="15" fill="#0369a1" fillOpacity="0.6" rx="1.5"
```

```
<rect x="7" y="14" width="3" height="8" fill="#0369a1" fillOpacity="0.6" rx="1.5" />
```

```
</g>
```

```
</defs>
```

```
{/* Horizontal Guideline Grid */}
```

```
{Array.from({ length: 11 }).map((_, i) => {
```

```
  const yVal = 30 + i * 30;
```

```
  return (
```

```
    <line
```

```
      key={`grid-y-${i}`}
```

```
      x1="0"
```

```
      y1={yVal}
```

```
      x2="600"
```

```
      y2={yVal}
```

```
      stroke="#1e293b"
```

```
      strokeWidth="0.5"
```

```
      strokeDasharray="4 4"
```

```
    />
```

```
  );
```

```
}}}
```

```
{/* Vertical cm lines centered around 300 */}
```

```
{Array.from({ length: 21 }).map((_, i) => {
```

```
  const cmValue = i - 10;
```

```
  const xVal = centerLineX + cmValue * scale;
```

```
  return (
```

```
    <g key={`grid-x-${i}`}>
```

```
      <line
```

```
        x1={xVal}
```

```
        y1="0"
```

```
        x2={xVal}
```

```
        y2="340"
```

```
        stroke={cmValue === 0 ? '#38bdf8' : '#1e293b'}
```

```
        strokeWidth={cmValue === 0 ? '2' : '0.5'}
```

```
      />
```

```
    {/* Numbers along the bottom rule */}
```

```
    <text
```

```
      x={xVal}
```

```
      y="348"
```

```
      fill={cmValue === 0 ? '#38bdf8' : '#64748b'}
```

```
      fontSize="9.5"
```

```
      textAnchor="middle"
```

```

        fontWeight={cmValue === 0 ? 'bold' : 'normal'}
      >
        {cmValue === 0 ? 'Mirror' : `${Math.abs(cmValue)}`}
      </text>
    </g>
  );
}}

  { /* Mirror Body Visual Indicator (The actual shiny surface) */
    <rect x="298" y="15" width="4" height="310" fill="#38bdf8" rx="2" opacity="0.8"
className="animate-pulse" />
    <line x1="300" y1="15" x2="300" y2="325" stroke="#e2e8f0" strokeWidth="1.5" />

    { /* EYE OBJECT (representing the student observer) */
    <g transform={`translate(70, 70)}`>
      <Icons.Eye />
      <text x="-5" y="-12" fill="#34d399" fontSize="9"
fontWeight="semibold">OBSERVER</text>
    </g>

    { /* ===== */
    { /* LIGHT RAY SIMULATION (THE RAY TRACING!) */
    { /* ===== */
    {isRayTraceOn && (
    <g>
      { /* Top ray path from tip of candle to mirror, reflecting to the eye */
      { /* Object point: x=objectX, y=objectY - 20 (tip of candle or shape) */
      { /* Center Point of reflection on mirror: (300, 140) */
      { /* Observer Eye: (70, 75) */

      { /* Incident Ray */
      <path
        d={`M ${objectX} ${objectY - 15} L 300 130`}
        stroke="#f59e0b"
        strokeWidth="2.5"
        fill="none"
        className="animate-[dash_2s_linear_infinite]"
        strokeDasharray="4 2"
      />
      { /* Reflected Ray back to eye */
      <path
        d="M 300 130 L 70 75"
        stroke="#f59e0b"
        strokeWidth="2.5"

```

```

        fill="none"
    />

    { /* Virtual Ray behind the mirror (Dotted projection extending straight) */
    <path
      d={` M 300 130 L ${imageX} ${objectY - 15}`}
      stroke="#38bdf8"
      strokeWidth="2"
      strokeDasharray="5 5"
      fill="none"
    />

    { /* Second ray (Lower body reflection) */
    <path
      d={` M ${objectX} ${objectY + 40} L 300 240`}
      stroke="#ef4444"
      strokeWidth="1.5"
      fill="none"
      strokeDasharray="3 3"
    />
    <path
      d="M 300 240 L 70 80"
      stroke="#ef4444"
      strokeWidth="1.5"
      fill="none"
    />
    <path
      d={` M 300 240 L ${imageX} ${objectY + 40}`}
      stroke="#06b6d4"
      strokeWidth="1.5"
      strokeDasharray="4 4"
      fill="none"
    />

    { /* Normal line at the point of reflection */
    <line x1="240" y1="130" x2="360" y2="130" stroke="#64748b" strokeWidth="1"
strokeDasharray="2 2" />
    <text x="250" y="125" fill="#94a3b8" fontSize="8">Normal</text>

    { /* Angle Labels i and r */
    <text x="282" y="145" fill="#f59e0b" fontSize="10" fontWeight="bold">i</text>
    <text x="282" y="115" fill="#f59e0b" fontSize="10" fontWeight="bold">r</text>
    </g>
  )}

```

```

{ /* ===== */
{ /* REAL-WORLD OBJECT */
{ /* ===== */
<g
  transform={`translate(${objectX}, ${objectY})`}
  className="cursor-ew-resize group"
>
  { /* Draggable highlight ring */
    <circle cx="0" cy="20" r="45" fill="#f59e0b" fillOpacity={isDragging ? "0.15" : "0.05"}
stroke="#f59e0b" strokeWidth="1" strokeDasharray="3 3" />

    { /* Render Candle or Letter F */
      {objectType === 'candle' && (
        <g>
          { /* Animated flickering flame */
            <path
              d="M -5 -8 C -5 -25 5 -25 5 -8 C 5 2 -5 2 -5 -8 Z"
              fill="#f59e0b"
              className="origin-bottom animate-bounce"
              style={{ animationDuration: '0.8s' }}
            />
            <path
              d="M -2.5 -10 C -2.5 -20 2.5 -20 2.5 -10 C 2.5 -5 -2.5 -5 -2.5 -10 Z"
              fill="#facc15"
            />
            <use href="#candle-svg" />
          </g>
        )}

        {objectType === 'letterF' && (
          <g transform="translate(-15, -20)">
            { /* Perfect block Letter 'F' */
              <path d="M 0 0 L 30 0 L 30 10 L 10 10 L 10 25 L 25 25 L 25 35 L 10 35 L 10 60
L 0 60 Z" fill="#f59e0b" stroke="#d97706" strokeWidth="1" />
            </g>
          )}

            {objectType === 'letterA' && (
              <g transform="translate(-15, -20)">
                { /* Non-lateral inverted shape 'A' to compare */
                  <path d="M 15 0 L 30 50 L 22 50 L 19 35 L 11 35 L 8 50 L 0 50 Z M 15 12 L 17
25 L 13 25 Z" fill="#ec4899" stroke="#db2777" strokeWidth="1" />
                </g>
              )}
            )}
          )}
        )}
      )}
    }
  }
</g>

```



```

    })

    { /* Raised Hand visual helper for Lateral Inversion */}
    {isHandRaised && (
      <g transform="translate(-25, 10)">
        <path d="M -5 10 Q -15 0 -8 -10 Q -5 -15 0 -12 Q 2 -18 7 -14 Q 10 -15 12 -10 Q
15 -5 10 12 Z" fill="#fbbf24" stroke="#d97706" strokeWidth="1" />
        <text x="-25" y="-18" fill="#fbbf24" fontSize="9" fontWeight="bold">RAISED
HAND (Left)</text>
      </g>
    })

    { /* Interactive Hand indicator tooltips */}
    <rect x="-35" y="65" width="70" height="18" fill="#1e293b" rx="4" stroke="#475569"
strokeWidth="1" />
    <text x="0" y="77" fill="#f59e0b" fontSize="8" fontWeight="bold"
textAnchor="middle">◀ DRAG ME ▶</text>
  </g>

  { /* ===== */}
  { /* MIRROR-WORLD GHOST IMAGE */}
  { /* ===== */}
  { /* It tracks symmetrical coordinates: x_image = 300 + (300 - x_object) */}
  <g
    transform={`translate(${imageX}, ${objectY}) scale(${isTrueMirror ? 1 : -1}, 1)`}
    className="opacity-70"
  >
    { /* Reflection back aura */}
    <circle cx="0" cy="20" r="45" fill="#38bdf8" fillOpacity="0.08" stroke="#38bdf8"
strokeWidth="0.5" strokeDasharray="2 2" />

    {objectType === 'candle' && (
      <g>
        { /* Ghost flame */}
        <path
          d="M -5 -8 C -5 -25 5 -25 5 -8 C 5 2 -5 2 -5 -8 Z"
          fill="#0ea5e9"
          className="origin-bottom animate-pulse"
        />
        <path
          d="M -2.5 -10 C -2.5 -20 2.5 -20 2.5 -10 C 2.5 -5 -2.5 -5 -2.5 -10 Z"
          fill="#38bdf8"
        />
        <use href="#ghost-candle-svg" />
      </g>
    )}
  </g>

```

```

    </g>
  })

  {objectType === 'letterF' && (
    <g transform="translate(-15, -20)">
      {/* Flipped 'J' image (laterally inverted) */}
      <path d="M 0 0 L 30 0 L 30 10 L 10 10 L 10 25 L 25 25 L 25 35 L 10 35 L 10 60
L 0 60 Z" fill="#0ea5e9" stroke="#0284c7" strokeWidth="1" />
    </g>
  })

  {objectType === 'letterA' && (
    <g transform="translate(-15, -20)">
      <path d="M 15 0 L 30 50 L 22 50 L 19 35 L 11 35 L 8 50 L 0 50 Z M 15 12 L 17
25 L 13 25 Z" fill="#0ea5e9" stroke="#0284c7" strokeWidth="1" />
    </g>
  })

  {/* Raised Hand mirror response */}
  {isHandRaised && (
    // Notice how we don't change scales, but because of the overarching scale(-1, 1),
    // it naturally looks like the opposite side hand is raised! Symmetrical physics.
    <g transform="translate(-25, 10)">
      <path d="M -5 10 Q -15 0 -8 -10 Q -5 -15 0 -12 Q 2 -18 7 -14 Q 10 -15 12 -10 Q
15 -5 10 12 Z" fill="#38bdf8" stroke="#0284c7" strokeWidth="1" />
      <text x="-25" y="-18" fill="#38bdf8" fontSize="9" fontWeight="bold"
transform="scale(-1, 1)">RAISED HAND (Right)</text>
    </g>
  })

  <rect x="-35" y="65" width="70" height="18" fill="#0f172a" rx="4" stroke="#0369a1"
strokeWidth="1" />
  <text x="0" y="77" fill="#38bdf8" fontSize="8" fontWeight="bold"
textAnchor="middle" transform="scale(-1, 1)">IMAGE TWIN</text>
</g>

  {/* Confetti celebration render */}
  {confetti.map((p) => (
    <circle
      key={p.id}
      cx={p.x}
      cy={p.y}
      r={p.size}
      fill={p.color}

```

```

        opacity={p.life}
      />
    )))
  </svg>

  {/* Instant Metric Dashboard Overlay */}
  <div className="bg-slate-900/90 border-t border-slate-800 p-4 grid grid-cols-2
sm:grid-cols-4 gap-4 text-center">
    <div>
      <span className="text-xs text-slate-400 block mb-0.5">Object Distance
($D_o$)</span>
      <span className="text-lg font-bold text-amber-400">{objectDistance.toFixed(1)}
cm</span>
    </div>
    <div>
      <span className="text-xs text-slate-400 block mb-0.5">Image Distance
($D_i$)</span>
      <span className="text-lg font-bold text-sky-400">{objectDistance.toFixed(1)}
cm</span>
    </div>
    <div>
      <span className="text-xs text-slate-400 block mb-0.5">Distance
Symmetry</span>
      <span className="text-xs font-semibold px-2 py-1 rounded bg-emerald-500/10
text-emerald-400 inline-block border border-emerald-500/20">
        $D_o$ = $D_i$ (Perfect!)
      </span>
    </div>
    <div>
      <span className="text-xs text-slate-400 block mb-0.5">Total Twin Span</span>
      <span className="text-lg font-extrabold text-indigo-400">{(objectDistance *
2).toFixed(1)} cm</span>
    </div>
  </div>

  {/* LIVE ACTION CONTROLLER PANEL */}
  <div className="bg-slate-900 border border-slate-800 rounded-3xl p-6 flex flex-col
gap-5 shadow-lg">
    <h4 className="text-sm font-semibold uppercase tracking-wider
text-slate-300">Control Center</h4>

    {/* Distance Slider */}

```

```

    <div className="flex flex-col gap-2">
      <div className="flex justify-between items-center text-xs font-semibold
text-slate-400">
        <span>Move Object Closer / Farther</span>
        <span className="text-amber-400 bg-amber-500/10 px-2.5 py-0.5 rounded border
border-amber-500/20">Current: {objectDistance} cm</span>
      </div>
      <div className="flex items-center gap-3">
        <span className="text-xs text-slate-500">1.0 cm</span>
        <input
          type="range"
          min="1.0"
          max="10.0"
          step="0.1"
          value={objectDistance}
          onChange={(e) => {
            playSound('click');
            setObjectDistance(parseFloat(e.target.value));
          }}
          className="flex-grow accent-sky-500 h-2 bg-slate-800 rounded-lg cursor-pointer"
        />
        <span className="text-xs text-slate-500">10.0 cm</span>
      </div>
    </div>
  </div>

```

```

  { /* Grid of control buttons */ }
  <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-3">

    { /* Select Object Type */ }
    <div className="flex flex-col gap-1.5">
      <label className="text-xs font-bold text-slate-400">Select Object:</label>
      <div className="flex rounded-xl bg-slate-950 p-1 border border-slate-800">
        <button
          onClick={() => { playSound('click'); setObjectType('candle'); }}
          className={`flex-1 py-1.5 text-xs font-medium rounded-lg transition-all ${
            objectType === 'candle' ? 'bg-sky-500/20 text-sky-400 border
border-sky-500/30' : 'text-slate-400 hover:text-slate-200'
          }`
        >
          Candle
        </button>
        <button
          onClick={() => { playSound('click'); setObjectType('letterF'); }}
          className={`flex-1 py-1.5 text-xs font-medium rounded-lg transition-all ${

```

```

        objectType === 'letterF' ? 'bg-sky-500/20 text-sky-400 border
border-sky-500/30' : 'text-slate-400 hover:text-slate-200'
      }}
    >
      Letter 'F'
    </button>
    <button
      onClick={() => { playSound('click'); setObjectType('letterA'); }}
      className={`flex-1 py-1.5 text-xs font-medium rounded-lg transition-all ${
        objectType === 'letterA' ? 'bg-sky-500/20 text-sky-400 border
border-sky-500/30' : 'text-slate-400 hover:text-slate-200'
      }}
    >
      Letter 'A'
    </button>
  </div>
</div>

{/* Ray Tracing toggle */}
<div className="flex flex-col gap-1.5">
  <label className="text-xs font-bold text-slate-400">Ray Diagram Trace:</label>
  <button
    onClick={() => { playSound('click'); setIsRayTraceOn(!isRayTraceOn); }}
    className={`py-2 text-xs font-semibold rounded-xl border transition-all ${
      isRayTraceOn
        ? 'bg-emerald-500/10 text-emerald-400 border-emerald-500/30'
        : 'bg-slate-950 text-slate-400 border-slate-800'
      }}
  >
    {isRayTraceOn ? '🟢 Ray-Trace Enabled' : '🔴 Ray-Trace Disabled'}
  </button>
</div>

{/* Raised Hand toggle */}
<div className="flex flex-col gap-1.5">
  <label className="text-xs font-bold text-slate-400">Lateral Test (Hand):</label>
  <button
    onClick={() => { playSound('click'); setIsHandRaised(!isHandRaised); }}
    className={`py-2 text-xs font-semibold rounded-xl border transition-all ${
      isHandRaised
        ? 'bg-purple-500/10 text-purple-400 border-purple-500/30'
        : 'bg-slate-950 text-slate-400 border-slate-800'
      }}
  >

```

```

        {isHandRaised ? '👋 Left Hand Raised' : '👉 Natural State'}
      </button>
    </div>

</div>

<div className="border-t border-slate-800/80 pt-4 flex flex-wrap gap-3 items-center
justify-between">
  <div className="flex gap-2 items-center">
    {/* True non-reversing mirror mode */}
    <button
      onClick={() => { playSound('click'); setIsTrueMirror(!isTrueMirror); }}
      className={`px-3 py-1.5 text-xs rounded-lg border font-semibold transition-all ${
        isTrueMirror
          ? 'bg-rose-500/10 text-rose-400 border-rose-500/30'
          : 'bg-slate-950 text-slate-500 border-slate-800'
      }}
    >
      {isTrueMirror ? '⚡ Non-Reversing (Magic Mirror)' : '✨ Regular Plane Mirror'}
    </button>
  </div>

  <button
    onClick={resetAll}
    className="flex items-center text-xs font-bold text-slate-400 hover:text-white
bg-slate-950 px-3 py-2 rounded-xl border border-slate-800 transition-all hover:bg-slate-800"
  >
    <Icons.RotateCcw /> Reset Lab
  </button>
</div>

</div>

</div>

{/* Right Panel - Progress & Challenges Dashboard (4/12 grid) */}
<div className="lg:col-span-4 flex flex-col gap-6">

  {/* CHALLENGE CHECKPOINT COMPONENT */}
  <div className="bg-slate-900 border border-slate-800 rounded-3xl p-6 shadow-md">
    <div className="flex items-center justify-between mb-4 pb-2 border-b
border-slate-800">
      <h3 className="font-bold text-slate-200 text-sm tracking-wide">🏆 QUEST
BOARD</h3>

```

```

    <span className="text-xs bg-slate-950 px-2 py-1 rounded text-slate-400
font-mono">
      {completedChallenges.length}/{challenges.length}
    </span>
  </div>

  <div className="flex flex-col gap-3">
    {challenges.map((challenge, index) => {
      const isCompleted = completedChallenges.includes(challenge.id);
      const isActive = currentChallengeId === index;
      return (
        <div
          key={challenge.id}
          onClick={() => { playSound('click'); setCurrentChallengeId(index); }}
          className={`p-3.5 rounded-2xl cursor-pointer transition-all border ${
            isCompleted
              ? 'bg-emerald-950/20 border-emerald-500/30 text-emerald-300'
              : isActive
              ? 'bg-sky-950/20 border-sky-500/40 text-slate-100'
              : 'bg-slate-950/60 border-slate-800/80 text-slate-400 hover:border-slate-700'
            }`}
        >
          <div className="flex items-center justify-between">
            <span className="text-xs font-bold tracking-widest uppercase">Quest #{index
+ 1}</span>
            {isCompleted ? (
              <span className="text-xs bg-emerald-500/20 text-emerald-400 border
border-emerald-500/30 px-2 py-0.5 rounded-full font-bold">✓ SOLVED</span>
            ) : isActive ? (
              <span className="text-xs bg-sky-500/20 text-sky-400 border
border-sky-500/30 px-2 py-0.5 rounded-full font-bold animate-pulse">ACTIVE</span>
            ) : (
              <span className="text-xs bg-slate-900 text-slate-500 px-2 py-0.5
rounded-full">LOCKED</span>
            )}
          </div>
          <h4 className="font-bold text-sm mt-1">{challenge.title}</h4>
          <p className="text-xs opacity-85 mt-1">{challenge.instruction}</p>

          {isActive && !isCompleted && (
            <div className="mt-3 bg-slate-900/60 p-2.5 rounded-xl border
border-slate-800 text-[11px] text-sky-300 flex items-start">
              <Icons.Lightbulb />
              <span><strong>Hint:</strong> {challenge.hint}</span>
            </div>
          )}
        </div>
      )
    })}
  </div>

```

```

        </div>
    })
</div>
);
}}
</div>
</div>

{ /* REAL-TIME PHYSICS FACTS */
  <div className="bg-gradient-to-tr from-indigo-950/20 to-slate-900 border
border-slate-800 rounded-3xl p-6">
    <div className="flex items-center gap-2 text-indigo-400 mb-3 font-semibold
text-sm">
      <Icons.Lightbulb />
      <span>Interactive Observations</span>
    </div>
    <ul className="space-y-3.5 text-xs text-slate-300">
      <li className="flex items-start gap-2">
        <span className="text-indigo-400 font-bold">•</span>
        <span>Notice how the <strong>Image size</strong> is identical to the object size,
no matter how close or far you stand.</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-indigo-400 font-bold">•</span>
        <span>Observe the letter <strong>'F'</strong>. Note how its arms point to the right
on the left side, but point to the left in the reflection. That's <strong>Lateral
Inversion</strong>!</span>
      </li>
      <li className="flex items-start gap-2">
        <span className="text-indigo-400 font-bold">•</span>
        <span>With the <strong>Ray diagram</strong> enabled, notice how the Angle of
Incidence ($i$) is always exactly equal to the Angle of Reflection ($r$).</span>
      </li>
    </ul>
  </div>

</div>

</div>
})

{ /*
===== */
  { /* TAB 2: ACADEMIC QUIZ
  */

```



```

    {/*
===== */}
    {activeTab === 'quiz' && (
      <div className="max-w-3xl mx-auto bg-slate-900 border border-slate-800 rounded-3xl
p-6 md:p-8 shadow-2xl">

        {!quizFinished ? (
          <div>
            {/* Quiz Header */}
            <div className="flex items-center justify-between pb-4 border-b border-slate-800
mb-6">
              <div>
                <span className="text-xs font-bold text-sky-400 uppercase
tracking-widest">Question {currentQuizIndex + 1} of {quizQuestions.length}</span>
                <h2 className="text-lg font-bold text-slate-100 mt-1">Symmetrical Test</h2>
              </div>
              <div className="bg-slate-950 px-3.5 py-1.5 rounded-xl border border-slate-800">
                <span className="text-xs text-slate-400 mr-2">Quiz XP:</span>
                <span className="font-extrabold text-amber-400">{quizScore} XP</span>
              </div>
            </div>

            {/* Question */}
            <div className="bg-slate-950 p-5 rounded-2xl border border-slate-800/80 mb-6">
              <p className="text-slate-100 font-medium md:text-lg leading-relaxed">
                {quizQuestions[currentQuizIndex].q}
              </p>
            </div>

            {/* Options */}
            <div className="grid grid-cols-1 gap-3.5 mb-6">
              {quizQuestions[currentQuizIndex].options.map((opt, idx) => {
                const isSelected = selectedAnswer === idx;
                const isCorrect = quizQuestions[currentQuizIndex].correct === idx;
                let btnClass = "bg-slate-950 hover:bg-slate-800/60 text-slate-300 border-slate-800";

                if (selectedAnswer !== null) {
                  if (isCorrect) {
                    btnClass = "bg-emerald-500/10 text-emerald-400 border-emerald-500/40
font-semibold";
                  } else if (isSelected) {
                    btnClass = "bg-rose-500/10 text-rose-400 border-rose-500/40 font-semibold";
                  } else {

```

```

        btnClass = "bg-slate-950/40 text-slate-500 border-slate-900 opacity-50
pointer-events-none";
    }
}

return (
  <button
    key={idx}
    disabled={selectedAnswer !== null}
    onClick={() => {
      setSelectedAnswer(idx);
      setShowExplanation(true);
      if (idx === quizQuestions[currentQuizIndex].correct) {
        playSound('correct');
        setQuizScore(prev => prev + 20);
        setXp(prev => prev + 20);
      } else {
        playSound('wrong');
      }
    }}
    className={`w-full text-left p-4 rounded-xl border transition-all text-sm
md:text-base flex justify-between items-center ${btnClass}`}
  >
    <span>{opt}</span>
    {selectedAnswer !== null && isCorrect && (
      <span className="text-xs font-bold uppercase bg-emerald-500/20
text-emerald-400 px-2 py-0.5 rounded border border-emerald-500/30">✓ Correct</span>
    )}
    {selectedAnswer !== null && isSelected && !isCorrect && (
      <span className="text-xs font-bold uppercase bg-rose-500/20 text-rose-400
px-2 py-0.5 rounded border border-rose-500/30">✗ Incorrect</span>
    )}
  </button>
);
  )}
</div>

{/* Explanation block */}
{showExplanation && (
  <div className="bg-indigo-950/20 border border-indigo-500/30 p-5 rounded-2xl
mb-6 text-sm">
    <div className="flex items-center gap-2 text-indigo-400 mb-2 font-bold">
      <Icons.Lightbulb />
      <span>NCERT Explanation:</span>

```

```

    </div>
    <p className="text-slate-300 leading-relaxed">
      {quizQuestions[currentQuizIndex].exp}
    </p>
  </div>
)}

{/* Next button */}
{selectedAnswer !== null && (
  <div className="flex justify-end">
    <button
      onClick={() => {
        playSound('click');
        setSelectedAnswer(null);
        setShowExplanation(false);
        if (currentQuizIndex < quizQuestions.length - 1) {
          setCurrentQuizIndex(prev => prev + 1);
        } else {
          setQuizFinished(true);
          triggerConfetti();
        }
      }}
      className="bg-sky-600 hover:bg-sky-500 text-white font-bold px-6 py-2.5
rounded-xl text-sm transition-all"
    >
      {currentQuizIndex === quizQuestions.length - 1 ? "Finish Quiz 🏁" : "Next
Question →"}
    </button>
  </div>
)}

</div>
): (
  /* Quiz Finished Screen */
  <div className="text-center py-8">
    <div className="text-6xl mb-4">🏆</div>
    <h2 className="text-2xl font-extrabold text-slate-100">Symmetrical Champion!</h2>
    <p className="text-slate-400 text-sm mt-2 max-w-md mx-auto">
      You completed the Mirror Matrix NCERT Quiz. You have gained a deep scientific
      understanding of reflection!
    </p>

    <div className="my-8 bg-slate-950 p-6 rounded-2xl border border-slate-800
max-w-sm mx-auto">

```

```

    <div className="flex justify-between items-center mb-3">
      <span className="text-slate-400 text-sm">Correct Answers:</span>
      <span className="font-bold text-slate-100">{quizQuestions.length} /
{quizQuestions.length}</span>
    </div>
    <div className="flex justify-between items-center">
      <span className="text-slate-400 text-sm">XP Score Boost:</span>
      <span className="font-bold text-amber-400">+{quizScore} XP</span>
    </div>
  </div>

  <div className="flex justify-center gap-3">
    <button
      onClick={() => {
        playSound('click');
        setCurrentQuizIndex(0);
        setSelectedAnswer(null);
        setShowExplanation(false);
        setQuizFinished(false);
        setQuizScore(0);
      }}
      className="border border-slate-800 hover:bg-slate-800 text-slate-200 font-bold
px-6 py-2.5 rounded-xl text-sm transition-all"
    >
      Try Again ↻
    </button>
    <button
      onClick={() => {
        playSound('click');
        setActiveTab('sim');
      }}
      className="bg-sky-600 hover:bg-sky-500 text-white font-bold px-6 py-2.5
rounded-xl text-sm transition-all"
    >
      Back to Simulator
    </button>
  </div>
</div>
)}
</div>
)}

```

```

    {/*
===== */}
    {/* TAB 3: DIY ACTIVITY */}
    {/*
===== */}
    {activeTab === 'diy' && (
      <div className="max-w-4xl mx-auto grid grid-cols-1 md:grid-cols-2 gap-6">

        <div className="bg-slate-900 border border-slate-800 rounded-3xl p-6 md:p-8 flex
flex-col gap-4">
          <span className="text-xs font-bold text-amber-400 uppercase
tracking-wider">Kitchen Physics Experiment</span>
          <h2 className="text-xl md:text-2xl font-extrabold text-slate-100">The Bathroom Code
Breaker 🧑🔬</h2>
          <p className="text-sm text-slate-400 leading-relaxed">
            Unlock the real-world mystery of mirrors inside your own home! This simple, safe
experiment lets you test lateral inversion with your friends and family.
          </p>

          <div className="bg-slate-950 p-4 rounded-2xl border border-slate-800 text-xs">
            <span className="font-bold text-slate-300 block mb-2">📦 What You Need:</span>
            <ul className="space-y-1.5 text-slate-400 list-disc list-inside">
              <li>A small household plane mirror (or dressing mirror)</li>
              <li>A white sheet of paper</li>
              <li>A dark marker pen / pencil</li>
            </ul>
          </div>

          <div className="flex flex-col gap-3 text-xs text-slate-300">
            <div className="flex items-start gap-2.5">
              <span className="bg-sky-500/10 text-sky-400 w-5 h-5 rounded-full flex
items-center justify-center font-bold text-[10px] shrink-0 border border-sky-500/20">1</span>
              <p>Write your name in bold block letters on the paper (e.g.
<strong>"ROY"</strong>).</p>
            </div>
            <div className="flex items-start gap-2.5">
              <span className="bg-sky-500/10 text-sky-400 w-5 h-5 rounded-full flex
items-center justify-center font-bold text-[10px] shrink-0 border border-sky-500/20">2</span>
              <p>Hold the paper up directly to the mirror. Can you read it easily? Notice how some
letters appear backwards.</p>
            </div>
            <div className="flex items-start gap-2.5">
              <span className="bg-sky-500/10 text-sky-400 w-5 h-5 rounded-full flex
items-center justify-center font-bold text-[10px] shrink-0 border border-sky-500/20">3</span>

```

<p>The Code Challenge: Try to write the word
"SECRET" in such a way that it appears perfectly correct inside the
mirror!</p>

</div>

</div>

</div>

<div className="bg-slate-900 border border-slate-800 rounded-3xl p-6 md:p-8 flex
flex-col justify-between">

<div>

<h3 className="font-bold text-slate-100 mb-3 text-base">🔬 The Science
Explained</h3>

<p className="text-xs text-slate-400 leading-relaxed mb-4">

Why do we have to write backwards to make it look forwards inside the mirror?

A plane mirror reverses the coordinate axis perpendicular to the mirror surface.

Since the mirror face is vertical:

</p>

<div className="bg-slate-950 p-4 rounded-xl border border-slate-800 space-y-2
mb-4 text-xs">

<div className="flex justify-between">

Vertical Axis (Up/Down)

Unchanged (Erect)

</div>

<div className="flex justify-between">

Horizontal Axis (Left/Right)

Flipped (Lateral Inversion)

</div>

</div>

<div className="border-t border-slate-800 pt-4">

<h4 className="text-xs font-bold text-indigo-400 uppercase tracking-widest
mb-2">Ambulance Mystery Solver:</h4>

<p className="text-xs text-slate-400 leading-relaxed">

Have you ever looked closely at an ambulance? Because drivers in front look at
their mirrors, they see the laterally reversed word ƎCNA_JU_□_MA turn into
a normal, readable AMBULANCE. This is a life-saving application of plane
mirrors!

</p>

</div>

</div>

<div className="mt-6 flex justify-end">

<button

onClick={() => {

```

        playSound('success');
        setXp(prev => prev + 30);
    }}
    className="bg-indigo-600 hover:bg-indigo-500 text-white font-bold text-xs px-5
py-2.5 rounded-xl transition-all"
    >
        I Completed this Activity! (+30 XP)
    </button>
</div>

</div>

</div>
)}}

{ /*
===== */}
{ /* TAB 4: CONCEPT GUIDE */}
{ /*
===== */}
{activeTab === 'concept' && (
    <div className="max-w-4xl mx-auto bg-slate-900 border border-slate-800 rounded-3xl
p-6 md:p-8 shadow-md">
        <h2 className="text-xl md:text-2xl font-bold text-slate-100 mb-4 flex items-center">
            <Icons.BookOpen /> The 4 Golden Rules of Plane Mirrors (NCERT Class 7)
        </h2>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-6 my-6">

            <div className="bg-slate-950 p-5 rounded-2xl border border-slate-800">
                <h3 className="font-bold text-sky-400 text-sm md:text-base mb-2">1. Same Size &
Height</h3>
                <p className="text-xs text-slate-400 leading-relaxed">
                    The image formed is of the exact same dimensions as the object. A plane mirror
neither magnifies nor shrinks the image. If you are 150 cm tall, your mirror image is exactly 150
cm tall.
                </p>
            </div>

            <div className="bg-slate-950 p-5 rounded-2xl border border-slate-800">
                <h3 className="font-bold text-sky-400 text-sm md:text-base mb-2">2. Erect
Orientation</h3>
                <p className="text-xs text-slate-400 leading-relaxed">

```

The image formed is erect (upright). This means your head is on top, and your feet are on the bottom in the image, unlike some curved mirrors where you might see yourself upside down!

</p>

</div>

<div className="bg-slate-950 p-5 rounded-2xl border border-slate-800">

<h3 className="font-bold text-sky-400 text-sm md:text-base mb-2">3. Equal Distance (Symmetry)</h3>

<p className="text-xs text-slate-400 leading-relaxed">

The distance of the image behind the mirror is exactly equal to the distance of the object in front of the mirror.

$D_{\text{object}} = D_{\text{image}}$

If you step back 2 steps, your mirror image also retreats 2 steps back!

</p>

</div>

<div className="bg-slate-950 p-5 rounded-2xl border border-slate-800">

<h3 className="font-bold text-sky-400 text-sm md:text-base mb-2">4. Virtual Image</h3>

<p className="text-xs text-slate-400 leading-relaxed">

The image formed is virtual, meaning it cannot be caught or projected onto a physical screen. The light rays only appear to meet behind the mirror when traced backwards.

</p>

</div>

</div>

<div className="bg-indigo-950/20 border border-indigo-500/20 p-5 rounded-2xl">

<h4 className="font-bold text-indigo-400 text-xs uppercase tracking-wider mb-2">🧠 Scientific Memory Trick</h4>

<p className="text-xs text-slate-300 leading-relaxed">

Remember the word V.E.S.E. to quickly recall the characteristics of a plane mirror:

</p>

<div className="grid grid-cols-4 gap-2 text-center font-bold text-xs mt-3">

<div className="bg-slate-950 p-2.5 rounded-lg border border-slate-800 text-slate-100">

V Virtual

</div>

<div className="bg-slate-950 p-2.5 rounded-lg border border-slate-800 text-slate-100">

E Erect

</div>


```

<div className="bg-slate-950 p-2.5 rounded-lg border border-slate-800
text-slate-100">
  <span className="text-sky-400 text-sm block mb-0.5">S</span> Same Size
</div>
<div className="bg-slate-950 p-2.5 rounded-lg border border-slate-800
text-slate-100">
  <span className="text-sky-400 text-sm block mb-0.5">E</span> Equal Dist
</div>
</div>
)}

</main>

{/* FOOTER */}
<footer className="bg-slate-900 border-t border-slate-800 py-4 px-6 text-center text-xs
text-slate-500 mt-8">
  <div className="max-w-7xl mx-auto flex flex-col md:flex-row justify-between items-center
gap-2">
    <span>Designed with ❤️ according to NCERT Light Syllabus for Class 6 & 7
Science.</span>
    <span className="text-slate-400">Interactive Mirror Simulation Matrix &copy;
2026</span>
  </div>
</footer>
</div>
);
}

```

States of Matter 5–6

```

import React, { useState, useEffect, useRef } from 'react';
import {
  Flame,
  Snowflake,
  Award,
  BookOpen,
  CheckCircle,
  HelpCircle,
  RotateCcw,
  Lightbulb,
  FileText,
  Sparkles,
  ChevronRight,
  XCircle,
  HelpCircle as HelpIcon,
  Coffee,
  Info
} from 'lucide-react';

export default function App() {
  // State variables
  const [temperature, setTemperature] = useState(0); // Range: -20 to 140
  °C

  const [currentState, setCurrentState] = useState('solid'); // 'solid' |
  'liquid' | 'gas'
  const [xp, setXp] = useState(100);
  const [progress, setProgress] = useState(20); // starts at 20%
  const [activeTab, setActiveTab] = useState('explore'); // 'explore' |
  'quiz' | 'diy'
  const [selectedQuizAnswers, setSelectedQuizAnswers] = useState({});
  const [quizSubmitted, setQuizSubmitted] = useState(false);
  const [quizScore, setQuizScore] = useState(0);
  const [showQuizResults, setShowQuizResults] = useState(false);

  // Milestone triggers
  const [unlockedLiquid, setUnlockedLiquid] = useState(false);

```

```

const [unlockedGas, setUnlockedGas] = useState(false);
const [unlockedSubZero, setUnlockedSubZero] = useState(false);
// Interactive Popup State
const [activePopup, setActivePopup] = useState(null); //
'liquid-discovery' | 'gas-discovery' | null

// Canvas ref
const canvasRef = useRef(null);
const animationFrameRef = useRef(null);
const particlesRef = useRef([]);

// Create particles list on mount
useEffect(() => {
  const particles = [];
  const numParticles = 32; // Perfect grid structure (4x8) for solids

  // Grid offsets for solid layout
  const startX = 140;
  const startY = 170;
  const spacingX = 22;
  const spacingY = 22;

  for (let i = 0; i < numParticles; i++) {
    const row = Math.floor(i / 8);
    const col = i % 8;

    particles.push({
      id: i,
      // Solid grid base coordinates
      baseX: startX + col * spacingX,
      baseY: startY + row * spacingY,
      // Current coordinates
      x: startX + col * spacingX,
      y: startY + row * spacingY,
      // Velocities (for liquid/gas)
      vx: (Math.random() - 0.5) * 4,

```

```

        vy: (Math.random() - 0.5) * 4,
        radius: 9,
        color: '#60a5fa', // blue default
    });
}
particlesRef.current = particles;
}, []);

// Update State & Progress based on Temperature
useEffect(() => {
    let state = 'solid';
    if (temperature > 0 && temperature <= 99) {
        state = 'liquid';
        if (!unlockedLiquid) {
            setUnlockedLiquid(true);
            setXp(prev => prev + 50);
            setProgress(prev => Math.min(prev + 25, 100));
            setActivePopup('liquid-discovery');
        }
    } else if (temperature >= 100) {
        state = 'gas';
        if (!unlockedGas) {
            setUnlockedGas(true);
            setXp(prev => prev + 50);
            setProgress(prev => Math.min(prev + 25, 100));
            setActivePopup('gas-discovery');
        }
    } else if (temperature <= -15) {
        if (!unlockedSubZero) {
            setUnlockedSubZero(true);
            setXp(prev => prev + 30);
            setProgress(prev => Math.min(prev + 10, 100));
        }
    }
    setCurrentState(state);
}, [temperature, unlockedLiquid, unlockedGas, unlockedSubZero]);

```

```

// Canvas Particle Simulation Engine
useEffect(() => {
  const canvas = canvasRef.current;
  if (!canvas) return;
  const ctx = canvas.getContext('2d');
  let animationId;

  const width = canvas.width;
  const height = canvas.height;

  // Simulation Bounds inside container bottle/beaker
  const bLeft = 35;
  const bRight = width - 35;
  const bTop = 50;
  const bBottom = height - 30;

  const render = () => {
    ctx.clearRect(0, 0, width, height);

    // 1. Draw Glass Beaker
    ctx.strokeStyle = 'rgba(255, 255, 255, 0.4)';
    ctx.lineWidth = 4;
    ctx.lineCap = 'round';
    ctx.beginPath();
    // Left wall
    ctx.moveTo(bLeft - 5, bTop - 15);
    ctx.lineTo(bLeft - 5, bBottom + 5);
    // Bottom curves
    ctx.quadraticCurveTo(bLeft - 5, bBottom + 15, bLeft + 15, bBottom +
15);
    ctx.lineTo(bRight - 15, bBottom + 15);
    ctx.quadraticCurveTo(bRight + 5, bBottom + 15, bRight + 5, bBottom +
5);
    // Right wall
    ctx.lineTo(bRight + 5, bTop - 15);

```

```

ctx.stroke();

// Draw Beaker Measurements / Markings
ctx.strokeStyle = 'rgba(255, 255, 255, 0.15)';
ctx.lineWidth = 2;
for (let mark = 1; mark <= 4; mark++) {
  const markY = bBottom - (mark * 45);
  ctx.beginPath();
  ctx.moveTo(bLeft + 5, markY);
  ctx.lineTo(bLeft + 18, markY);
  ctx.stroke();
  ctx.fillStyle = 'rgba(255, 255, 255, 0.3)';
  ctx.font = '10px sans-serif';
  ctx.fillText(`${mark * 50} ml`, bLeft + 24, markY + 3);
}

// 2. Draw Heating / Cooling glow beneath beaker
if (temperature > 0) {
  const intensity = Math.min(temperature / 140, 1);
  const gradient = ctx.createLinearGradient(width / 2, bBottom + 15,
width / 2, bBottom + 60);
  gradient.addColorStop(0, `rgba(239, 68, 68, ${intensity * 0.4})`);
  gradient.addColorStop(1, 'rgba(239, 68, 68, 0)');
  ctx.fillStyle = gradient;
  ctx.beginPath();
  ctx.ellipse(width / 2, bBottom + 15, 120, 30, 0, 0, Math.PI * 2);
  ctx.fill();
} else if (temperature < 0) {
  const intensity = Math.min(Math.abs(temperature) / 20, 1);
  const gradient = ctx.createLinearGradient(width / 2, bBottom + 15,
width / 2, bBottom + 60);
  gradient.addColorStop(0, `rgba(59, 130, 246, ${intensity * 0.3})`);
  gradient.addColorStop(1, 'rgba(59, 130, 246, 0)');
  ctx.fillStyle = gradient;
  ctx.beginPath();
  ctx.ellipse(width / 2, bBottom + 15, 120, 30, 0, 0, Math.PI * 2);

```

```

    ctx.fill();
  }

  // 3. Update & Draw Particles
  const tempFactor = (temperature + 20) / 160; // range 0 to 1
normalized
  const particles = particlesRef.current;

  particles.forEach((p, idx) => {
    if (currentState === 'solid') {
      // SOLID STATE: Particles vibrate around a fixed grid lattice
      const vibrationAmplitude = 0.5 + (tempFactor * 3); // increases
slightly with mild heat
      const randomAngle = Math.random() * Math.PI * 2;

      // Smooth pull back to their original solid grid spots
      const targetX = p.baseX;
      const targetY = p.baseY;

      p.x += (targetX - p.x) * 0.15 + Math.cos(randomAngle) *
vibrationAmplitude * 0.3;
      p.y += (targetY - p.y) * 0.15 + Math.sin(randomAngle) *
vibrationAmplitude * 0.3;

      // Color shifts from ice blue to standard blue
      p.color = `hsl(${210 + (temperature * 0.8)}, 85%, ${65 -
(temperature * 0.2)}%)`;
    } else if (currentState === 'liquid') {
      // LIQUID STATE: Particles flow dynamically, tumble around, stay
closely packed at base
      const targetSpeed = 1.0 + (temperature * 0.03);

      // Random walk with gravity vector pushing down
      p.vx += (Math.random() - 0.5) * 0.4;

```



```

    p.vy += (Math.random() - 0.5) * 0.4 + 0.08; // small down force
    to aggregate at bottom

    // Speed regulator
    const speed = Math.sqrt(p.vx * p.vx + p.vy * p.vy);
    if (speed > targetSpeed) {
        p.vx = (p.vx / speed) * targetSpeed;
        p.vy = (p.vy / speed) * targetSpeed;
    }

    p.x += p.vx;
    p.y += p.vy;

    // Inter-particle weak attraction forces to stick them together
    at bottom
    particles.forEach(other => {
        if (other.id === p.id) return;
        const dx = other.x - p.x;
        const dy = other.y - p.y;
        const dist = Math.sqrt(dx * dx + dy * dy);
        const minDist = 18; // double the radius
        if (dist < minDist) {
            const overlap = minDist - dist;
            // Push away slightly to prevent overlap
            p.x -= (dx / dist) * overlap * 0.35;
            p.y -= (dy / dist) * overlap * 0.35;
        } else if (dist < 35) {
            // Gentle attraction pull to maintain liquid structure volume
            p.x += (dx / dist) * 0.03;
            p.y += (dy / dist) * 0.03;
        }
    });

    // Liquid boundary containment box at base
    const liquidBottom = bBottom + 10;
    const liquidTopHeight = bBottom - 95; // liquid stays low

```

```

    if (p.x < bLeft + p.radius) { p.x = bLeft + p.radius; p.vx *=
-0.5; }
    if (p.x > bRight - p.radius) { p.x = bRight - p.radius; p.vx *=
-0.5; }
    if (p.y > liquidBottom - p.radius) { p.y = liquidBottom -
p.radius; p.vy *= -0.3; }
    if (p.y < liquidTopHeight) {
      // Keep liquid constrained to bottom layer with splash dynamics
      p.vy *= -0.3;
      p.y = liquidTopHeight;
    }

    p.color = `hsl(${200 - ((temperature - 1) * 0.8)}, 90%, 60%)`; //
Blueish green hues

  } else if (currentState === 'gas') {
    // GAS STATE: Rapid collisions, filling whole container
    const targetSpeed = 3.5 + ((temperature - 100) * 0.08);

    p.x += p.vx;
    p.y += p.vy;

    // Elastic collision with other particles
    particles.forEach(other => {
      if (other.id === p.id) return;
      const dx = other.x - p.x;
      const dy = other.y - p.y;
      const dist = Math.sqrt(dx * dx + dy * dy);
      const minDist = 18;
      if (dist < minDist) {
        // Snap apart
        const overlap = minDist - dist;
        p.x -= (dx / dist) * overlap * 0.5;
        p.y -= (dy / dist) * overlap * 0.5;
      }
    });
  }
}

```

```

        // Bounce velocities
        p.vx = -p.vx * 0.9;
        p.vy = -p.vy * 0.9;
    }
});

// Enforce constant gas energy
const currentSpeed = Math.sqrt(p.vx * p.vx + p.vy * p.vy);
if (currentSpeed < targetSpeed) {
    // Accelerate slightly to match temperature kinetic level
    p.vx = (p.vx / currentSpeed) * targetSpeed;
    p.vy = (p.vy / currentSpeed) * targetSpeed;
}

// Full container boundaries
if (p.x < bLeft + p.radius) { p.x = bLeft + p.radius; p.vx *= -1;
}

if (p.x > bRight - p.radius) { p.x = bRight - p.radius; p.vx *=
-1; }

if (p.y < bTop + p.radius) { p.y = bTop + p.radius; p.vy *= -1; }
if (p.y > bBottom + p.radius) { p.y = bBottom + p.radius; p.vy *=
-1; }

// Orange and pinkish gas high temperature hues
p.color = `hsl(${15 + ((temperature - 100) * 1.5)}, 100%, 60%)`;
}

// Draw individual particle with smooth glow shadow
ctx.shadowColor = p.color;
ctx.shadowBlur = currentState === 'gas' ? 12 : 4;
ctx.fillStyle = p.color;
ctx.beginPath();
ctx.arc(p.x, p.y, p.radius, 0, Math.PI * 2);
ctx.fill();

// Little dynamic visual core

```

```

    ctx.fillStyle = 'rgba(255, 255, 255, 0.5)';
    ctx.beginPath();
    ctx.arc(p.x - 2, p.y - 2, 2.5, 0, Math.PI * 2);
    ctx.fill();
    ctx.shadowBlur = 0; // reset shadow
  });

  animationId = requestAnimationFrame(render);
};

render();

return () => {
  cancelAnimationFrame(animationId);
};
}, [currentState, temperature]);

// Handle Preset Snaps
const snapToPreset = (targetTemp) => {
  setTemperature(targetTemp);
  // Add audio-visual feedback feel
  setXp(prev => prev + 5);
};

// Reset Lab
const handleReset = () => {
  setTemperature(0);
  setCurrentState('solid');
  setUnlockedLiquid(false);
  setUnlockedGas(false);
  setUnlockedSubZero(false);
  setProgress(20);
  setXp(100);
  setSelectedQuizAnswers({});
  setQuizSubmitted(false);
  setShowQuizResults(false);

```

```

// Reset velocities of particles
particlesRef.current.forEach(p => {
  p.vx = (Math.random() - 0.5) * 4;
  p.vy = (Math.random() - 0.5) * 4;
});
};

// Live descriptions based on state
const getStateDescription = () => {
  switch (currentState) {
    case 'solid':
      return {
        title: "Solid (Ice Mode)",
        text: "Particles are packed very closely in a neat pattern. They cannot walk or swap places. They can only shake and vibrate right where they stand! This gives solids a fixed, rigid shape.",
        colorClass: "text-blue-400 bg-blue-950/40 border-blue-800",
        particleMovement: "Vibrating in place",
        shape: "Fixed (Stays identical)",
        volume: "Fixed (Does not shrink or expand)"
      };
    case 'liquid':
      return {
        title: "Liquid (Water Mode)",
        text: "Heat has given particles energy to break out of their grid! They now tumble, roll, and slide past each other at the bottom of the beaker. They have dynamic shape, but remain closely bound.",
        colorClass: "text-emerald-400 bg-emerald-950/40 border-emerald-800",
        particleMovement: "Sliding & tumbling over each other",
        shape: "Changes shape to fit container bottom",
        volume: "Fixed (Still same physical space)"
      };
    case 'gas':
      return {
        title: "Gas (Steam Mode)",

```

```

        text: "Maximum energy! The particles have completely broken free
from all attractions. They sprint, fly in straight lines, and bounce
elastically off walls and each other to fill the entire container.",
        colorClass: "text-amber-400 bg-amber-950/40 border-amber-800",
        particleMovement: "Sprinting wildly in all directions",
        shape: "Flexible (Takes shape of container)",
        volume: "Flexible (Expands or squeezes to fill space)"
    };
    default:
        return {};
}
};

const activeDesc = getStateDescription();

// Academic Quiz Data
const quizQuestions = [
    {
        id: 1,
        q: "In which state of matter are particles locked together in a
highly organized pattern?",
        options: ["Gas", "Liquid", "Solid", "Plasma"],
        ans: 2, // Solid
        explain: "Solids maintain a rigid, highly aligned lattice grid
structure where particles merely vibrate in static spots."
    },
    {
        id: 2,
        q: "What happens to the average speed of particles when you add heat
energy?",
        options: ["They slow down", "They stop moving entirely", "They speed
up", "They get heavier"],
        ans: 2, // They speed up
        explain: "Heat energy transfers kinetic energy directly into
particles, driving them to slide, bounce, or sprint faster."
    },

```

```
{
  id: 3,
  q: "True or False: A liquid has a fixed volume but changes shape to
match its container.",
  options: ["True", "False"],
  ans: 0, // True
  explain: "Liquids remain aggregated together due to general
attraction forces, keeping volume identical while shape reshapes
completely."
},
{
  id: 4,
  q: "When vapor gas cools down and converts back into water, the
transition is called:",
  options: ["Evaporation", "Melting", "Sublimation", "Condensation"],
  ans: 3, // Condensation
  explain: "Condensation is the process of cooling gas particles,
causing them to lose thermal speed and aggregate closely as liquid."
},
{
  id: 5,
  q: "Fill in the blank: A _____ has neither fixed shape nor fixed
volume.",
  options: ["Liquid", "Solid", "Gas", "Brick"],
  ans: 2, // Gas
  explain: "Gases scatter quickly to occupy whatever container
boundaries envelope them, completely filling the volume."
},
{
  id: 6,
  q: "Rohan pours 50 ml of water from a tall glass into a wide plate.
What changes?",
  options: ["The volume increases", "The shape changes", "Water turns
into solid", "No change at all"],
  ans: 1, // The shape changes
```

```
    explain: "Since water is liquid, its shape shifts to match the plate,  
but the total volume stays exactly 50 ml."
```

```
  },
```

```
  {
```

```
    id: 7,
```

```
    q: "Why can you squeeze (compress) air easily inside a plastic  
syringe, but not liquid water?",
```

```
    options: [
```

```
      "Gas particles have huge empty spaces between them",
```

```
      "Liquid particles are too slippery to catch",
```

```
      "Air is lighter than gravity forces",
```

```
      "Squeezing destroys gas molecules"
```

```
    ],
```

```
    ans: 0,
```

```
    explain: "Gases feature vast intermolecular distances, allowing  
pressure to easily pack them together. Liquid particles are already  
touching closely."
```

```
  },
```

```
  {
```

```
    id: 8,
```

```
    q: "Which state of matter matches the description: 'Particles vibrate  
dynamically inside structural grid lines'?",
```

```
    options: ["Solid", "Liquid", "Gas"],
```

```
    ans: 0, // Solid
```

```
    explain: "Solids vibrate in fixed positions, maintaining a set  
pattern."
```

```
  },
```

```
  {
```

```
    id: 9,
```

```
    q: "What will happen to a puddle of water left outside under the  
blazing hot sun?",
```

```
    options: [
```

```
      "It will freeze solid",
```

```
      "It will evaporate into invisible water vapor gas",
```

```
      "The particles will physically break and vanish",
```

```
      "It turns into plasma juice"
```



```

    ],
    ans: 1, // It will evaporate
    explain: "Sunlight heat drives water particles to speed up until they
snap liquid attractions and rise as vapor gas."
  },
  {
    id: 10,
    q: "Are individual particles of matter hot or cold?",
    options: [
      "Yes, they turn hot or cold individually",
      "No, temperature is just how fast the entire group of particles is
moving"
    ],
    ans: 1,
    explain: "Individual particles do not hold temperatures. Temperature
is a macro-measurement of collective kinetic speeds of particles."
  }
];

```

```

const handleSelectAnswer = (qId, optionIdx) => {
  if (quizSubmitted) return;
  setSelectedQuizAnswers(prev => ({
    ...prev,
    [qId]: optionIdx
  }));
};

```

```

const handleSubmitQuiz = () => {
  let score = 0;
  quizQuestions.forEach(q => {
    if (selectedQuizAnswers[q.id] === q.ans) {
      score += 10;
    }
  });
  setQuizScore(score);
  setQuizSubmitted(true);
}

```

```

    setShowQuizResults(true);
    setXp(prev => prev + (score * 2));
    setProgress(prev => Math.min(prev + 15, 100));
  };

  return (
    <div className="min-h-screen bg-slate-950 text-white font-sans flex
flex-col antialiased selection:bg-indigo-500/30
selection:text-indigo-200">

      {/* 1. HEADER HEADER */}
      <header className="sticky top-0 z-30 bg-slate-900/80 backdrop-blur-md
border-b border-slate-800 px-4 py-3 shadow-md">
        <div className="max-w-7xl mx-auto flex flex-col md:flex-row
items-center justify-between gap-3">
          <div className="flex items-center gap-3">
            <div className="p-2 bg-gradient-to-tr from-indigo-500
to-rose-500 rounded-xl shadow-lg shadow-indigo-500/20 animate-pulse">
              <Sparkles className="w-6 h-6 text-white" />
            </div>
            <div>
              <h1 className="text-xl font-extrabold tracking-tight
bg-gradient-to-r from-white via-slate-100 to-indigo-300 bg-clip-text
text-transparent">
                The Particle Lab: Dance of the Molecules
              </h1>
              <p className="text-xs text-slate-400 font-medium">NCERT
Science • Classes 5-6 • Interactive State Simulator</p>
            </div>
          </div>
        </div>

      <div className="flex items-center gap-4 w-full md:w-auto
justify-between md:justify-end">
        {/* Class Badge */}

```

```

        <span className="hidden sm:inline-flex items-center gap-1.5
px-3 py-1 rounded-full text-xs font-semibold bg-indigo-950 text-indigo-300
border border-indigo-800">
            <BookOpen className="w-3.5 h-3.5" /> Class 5-6
        </span>

        { /* XP Counter */ }
        <div className="flex items-center gap-2 px-3 py-1
bg-amber-950/60 border border-amber-800/80 rounded-full text-amber-300
shadow-sm">
            <span className="text-lg"> ✨ </span>
            <span className="text-sm font-black tracking-wide"> {xp}
XP </span>
        </div>

        { /* Micro Progress Bar */ }
        <div className="flex flex-col items-end w-32">
            <div className="flex justify-between w-full text-[10px]
font-bold text-slate-400 mb-0.5">
                <span> LAB PROGRESS </span>
                <span> {progress}% </span>
            </div>
            <div className="w-full bg-slate-800 h-2 rounded-full
overflow-hidden">
                <div
                    className="bg-gradient-to-r from-indigo-500
to-emerald-400 h-full transition-all duration-500 ease-out"
                    style={{ width: `${progress}%` }}
                ></div>
            </div>
        </div>
    </div>
</div>
</header>

{ /* 2. DYNAMIC ACHIEVEMENT HUD POPUPS */ }

```

```

{activePopup && (
  <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-sm
flex items-center justify-center p-4">
    <div className="bg-slate-900 border-2 border-indigo-500 max-w-md
w-full rounded-2xl p-6 shadow-2xl shadow-indigo-500/10 text-center
relative overflow-hidden animate-in zoom-in-95 duration-200">
      {/* Glow background */}
      <div className="absolute -top-12 -left-12 w-32 h-32
bg-indigo-500/10 rounded-full blur-2xl"></div>
      <div className="absolute -bottom-12 -right-12 w-32 h-32
bg-emerald-500/10 rounded-full blur-2xl"></div>

      <Award className="w-16 h-16 text-amber-400 mx-auto mb-4
animate-bounce" />

      {activePopup === 'liquid-discovery' ? (
        <>
          <h3 className="text-xl font-black text-white mb-2">🧪 Phase
Changer Unlocked!</h3>
          <p className="text-slate-300 text-sm mb-4 leading-relaxed">
            Excellent work! You heated the solid ice above 0°C.
            Notice how the particles broke out of their tight grid
and started sliding past each other?
          </p>
          <div className="bg-slate-950/60 p-4 rounded-xl border
border-slate-800 text-left text-xs text-indigo-300 space-y-2 mb-4">
            <span className="font-extrabold text-indigo-200 block">💡
Discovery Question:</span>
            "Did the particles get closer together, or did they start
sliding past each other?"
            <span className="text-slate-400 block mt-1">Answer: They
gained energy and started sliding, changing their shape!</span>
          </div>
        </>
      ) : (
        <>

```

```

        <h3 className="text-xl font-black text-white mb-2">🔥 Super
Charged Gas Unlocked!</h3>
        <p className="text-slate-300 text-sm mb-4 leading-relaxed">
            You boiled the liquid past 100°C!
            The particles are now moving super fast and bouncing
wildly all over the vessel. They have filled every single corner.
        </p>
        <div className="bg-slate-950/60 p-4 rounded-xl border
border-slate-800 text-left text-xs text-amber-300 space-y-2 mb-4">
            <span className="font-extrabold text-amber-200 block">💡
Observation Question:</span>
                "Is the space between gas particles larger or smaller
than in liquids?"
            <span className="text-slate-400 block mt-1">Answer: Far
larger! There is mostly empty space inside gases, which is why we can
squeeze them!</span>
        </div>
    </>
)}

<button
    onClick={() => {
        setActivePopup(null);
        setXp(prev => prev + 15);
    }}
    className="w-full py-2.5 bg-gradient-to-r from-indigo-600
to-indigo-500 hover:from-indigo-500 hover:to-indigo-400 text-white
font-bold rounded-xl transition shadow-lg shadow-indigo-600/30 text-sm"
    >
        Continue Exploring (+15 XP)
    </button>
</div>
</div>
)}

{/* 3. MAIN WORKSPACE */}

```

```

    <main className="flex-1 max-w-7xl mx-auto w-full p-4 grid grid-cols-1
lg:grid-cols-12 gap-6">

    { /* LEFT COLUMN - INTERACTIVE CANVAS LAB (7 COLS) */ }
    <section className="lg:col-span-7 flex flex-col gap-4">

        { /* Simulation Header with Active Info Badges */ }
        <div className="bg-slate-900 border border-slate-800 rounded-2xl
p-4 shadow-md flex flex-wrap items-center justify-between gap-3">
            <div className="flex items-center gap-3">
                <div className="flex flex-col">
                    <span className="text-[10px] tracking-widest
text-indigo-400 font-extrabold uppercase">Current State</span>
                    <span className={`text-lg font-black tracking-wide
capitalize ${currentState === 'solid' ? 'text-blue-400' : currentState ===
'liquid' ? 'text-emerald-400' : 'text-amber-400'}`}>
                        {currentState === 'solid' && '❄️ Solid (Ice)'}
                        {currentState === 'liquid' && '💧 Liquid (Water)'}
                        {currentState === 'gas' && '💨 Gas (Vapor)'}
                    </span>
                </div>
            </div>

            { /* Thermometer Display */ }
            <div className="flex items-center gap-2 bg-slate-950 px-4 py-2
rounded-xl border border-slate-800">
                <div className="relative h-10 w-3.5 bg-slate-800 rounded-full
overflow-hidden flex flex-col justify-end">
                    { /* Mercury column */ }
                    <div
                        className={`w-full transition-all duration-300
rounded-t-full ${temperature >= 100 ? 'bg-amber-500' : temperature > 0 ?
'bg-emerald-500' : 'bg-blue-500'}`}
                        style={{ height: `${((temperature + 20) / 160) * 100}%`
                    }}
                ></div>
            </div>

```

```

    </div>
    <div>
      <span className="text-[10px] text-slate-400 block font-bold
leading-none">THERMOMETER</span>
      <span className="text-lg font-mono
font-black">{temperature}°C</span>
    </div>
  </div>

  { /* Reset Button */ }
  <button
    onClick={handleReset}
    className="flex items-center gap-1.5 px-3 py-2
bg-slate-800/60 hover:bg-slate-800 text-slate-300 hover:text-white
rounded-xl transition border border-slate-700 text-xs font-bold"
    title="Reset Simulation"
  >
    <RotateCcw className="w-3.5 h-3.5" />
    <span>Reset</span>
  </button>
</div>

  { /* SIMULATION VISUALIZER CANVAS */ }
  <div className="relative bg-gradient-to-b from-slate-900
to-slate-950 border border-slate-800 rounded-3xl overflow-hidden shadow-xl
aspect-[4/3] flex flex-col items-center justify-center">

    { /* Visual labels on canvas area */ }
    <div className="absolute top-4 left-4 z-10 pointer-events-none
text-slate-500 text-[10px] font-mono tracking-wider">
      [SEALED CHAMBER]
    </div>

    { /* Micro instructions overlay */ }

```

```

    <div className="absolute top-4 right-4 z-10 pointer-events-none
bg-slate-900/60 backdrop-blur-md border border-slate-800 px-2 py-1
rounded-md text-[10px] text-slate-300">
      Active Particles: <span className="text-white font-bold
font-mono">32</span>
    </div>

    { /* HTML5 Canvas */ }
    <canvas
      ref={canvasRef}
      width={480}
      height={320}
      className="w-full h-full max-w-full"
    />

    { /* Frost/Heat Vignette Overlay edges */ }
    {temperature < 0 && (
      <div
        className="absolute inset-0 pointer-events-none border-2
border-sky-400/20 rounded-3xl transition-opacity duration-300"
        style={{ opacity: Math.abs(temperature) / 20 }}
      ></div>
    )}
    {temperature > 100 && (
      <div
        className="absolute inset-0 pointer-events-none border-2
border-amber-500/20 rounded-3xl transition-opacity duration-300"
        style={{ opacity: (temperature - 100) / 40 }}
      ></div>
    )}
  </div>

  { /* CONTROLS BAR CONTAINER */ }
  <div className="bg-slate-900 border border-slate-800 rounded-2xl
p-5 shadow-lg space-y-4">

```



```

    { /* Slider Control */ }
    <div className="space-y-2">
      <div className="flex justify-between items-center text-xs
font-bold text-slate-300">
        <div className="flex items-center gap-1.5 text-blue-400">
          <Snowflake className="w-4 h-4 animate-spin" style={{
animationDuration: '6s' }} />
          <span>Freezing Cold (-20°C)</span>
        </div>
        <span className="px-2.5 py-0.5 bg-slate-950 rounded-md
border border-slate-800 text-slate-300 font-mono text-sm">
          Slide to Heat or Cool 🌡️
        </span>
        <div className="flex items-center gap-1.5 text-rose-500">
          <Flame className="w-4 h-4 animate-bounce" />
          <span>Boiling Hot (140°C)</span>
        </div>
      </div>

      <div className="relative flex items-center group">
        { /* Cool Indicator Track Background */ }
        <div className="absolute left-0 right-0 h-2
bg-gradient-to-r from-blue-600 via-emerald-500 to-rose-600 rounded-lg
pointer-events-none"></div>
        <input
          type="range"
          min="-20"
          max="140"
          value={temperature}
          onChange={ (e) =>
setTemperature(parseInt(e.target.value)) }
          className="relative w-full h-8 opacity-0 cursor-pointer
z-10"
        />
        { /* Customized Virtual Handle */ }
        <div

```

```

        className="absolute w-6 h-6 rounded-full bg-white
border-2 border-indigo-600 shadow-md flex items-center justify-center
pointer-events-none transition-transform duration-75
group-hover:scale-110"
        style={{ left: `calc(${(temperature + 20) / 160} * 100)%
- 12px)` }}
      >
        <div className="w-2.5 h-2.5 rounded-full bg-indigo-600
animate-ping"></div>
      </div>
    </div>
  </div>

  { /* Quick Snap Preset State Buttons */ }
  <div className="pt-2 border-t border-slate-800">
    <div className="text-slate-400 text-xs font-bold mb-3
uppercase tracking-wider text-center">
      Quick Jump Presets ⚡
    </div>
    <div className="grid grid-cols-3 gap-2">
      <button
        onClick={() => snapToPreset(-10)}
        className={`py-3 px-3 rounded-xl flex flex-col
items-center justify-center transition border text-xs font-bold
${currentState === 'solid' ? 'bg-blue-950/80 border-blue-500 text-blue-300
shadow-lg shadow-blue-500/10' : 'bg-slate-950 border-slate-850
hover:bg-slate-850 text-slate-400 hover:text-slate-200'}`} `>
        >
        <span className="text-xl mb-1">🧊</span>
        <span>Solid State</span>
        <span className="text-[10px] opacity-70 mt-0.5">-10°C
(Ice)</span>
      </button>

      <button
        onClick={() => snapToPreset(45)}

```

```

        className={`py-3 px-3 rounded-xl flex flex-col
items-center justify-center transition border text-xs font-bold
${currentState === 'liquid' ? 'bg-emerald-950/80 border-emerald-500
text-emerald-300 shadow-lg shadow-emerald-500/10' : 'bg-slate-950
border-slate-850 hover:bg-slate-850 text-slate-400
hover:text-slate-200'}`}
      >
        <span className="text-xl mb-1">💧 </span>
        <span>Liquid State</span>
        <span className="text-[10px] opacity-70 mt-0.5">45°C
(Water)</span>
      </button>

      <button
        onClick={() => snapToPreset(120)}
        className={`py-3 px-3 rounded-xl flex flex-col
items-center justify-center transition border text-xs font-bold
${currentState === 'gas' ? 'bg-amber-950/80 border-amber-500
text-amber-300 shadow-lg shadow-amber-500/10' : 'bg-slate-950
border-slate-850 hover:bg-slate-850 text-slate-400
hover:text-slate-200'}`}
      >
        <span className="text-xl mb-1">💨 </span>
        <span>Gas State</span>
        <span className="text-[10px] opacity-70 mt-0.5">120°C
(Steam)</span>
      </button>
    </div>
  </div>

</div>

</section>

{/* RIGHT COLUMN - LESSON & ASSESSMENT TABS (5 COLS) */}
<section className="lg:col-span-5 flex flex-col gap-4">

```

```

    {/* Tabs Navigation */}
    <div className="bg-slate-900 p-1.5 rounded-2xl border
border-slate-800 grid grid-cols-3 gap-1 shadow-md">
      <button
        onClick={() => setActiveTab('explore')}
        className={`py-2.5 rounded-xl text-xs font-bold flex
items-center justify-center gap-1.5 transition ${activeTab === 'explore' ?
'bg-indigo-600 text-white shadow-md' : 'text-slate-400
hover:bg-slate-800'}}`>
        >
        <Lightbulb className="w-3.5 h-3.5" />
        <span>Observe</span>
      </button>
      <button
        onClick={() => setActiveTab('quiz')}
        className={`py-2.5 rounded-xl text-xs font-bold flex
items-center justify-center gap-1.5 transition ${activeTab === 'quiz' ?
'bg-indigo-600 text-white shadow-md' : 'text-slate-400
hover:bg-slate-800'}}`>
        >
        <CheckCircle className="w-3.5 h-3.5" />
        <span>Quiz ({quizSubmitted ? 'Done' : '10 Qs'})</span>
      </button>
      <button
        onClick={() => setActiveTab('diy')}
        className={`py-2.5 rounded-xl text-xs font-bold flex
items-center justify-center gap-1.5 transition ${activeTab === 'diy' ?
'bg-indigo-600 text-white shadow-md' : 'text-slate-400
hover:bg-slate-800'}}`>
        >
        <Coffee className="w-3.5 h-3.5" />
        <span>DIY Magic</span>
      </button>
    </div>

```

```

    { /* TAB CONTENT: OBSERVE */ }
    { activeTab === 'explore' && (
      <div className="flex flex-col gap-4 animate-fadeIn">

        { /* Dynamic State Observation Card */ }
        <div className={`border-2 rounded-2xl p-5 space-y-4 shadow-lg
        ${activeDesc.colorClass}`}>
          <div className="flex items-center justify-between">
            <h4 className="text-base font-black flex items-center
            gap-2">

              <Sparkles className="w-5 h-5" />
              {activeDesc.title}
            </h4>
            <span className="px-2.5 py-1 text-[10px] uppercase
            tracking-wider font-extrabold bg-slate-950/60 rounded-full border
            border-current">
              Active Phase
            </span>
          </div>

          <p className="text-slate-200 text-sm leading-relaxed">
            {activeDesc.text}
          </p>

          { /* Properties Matrix */ }
          <div className="grid grid-cols-2 gap-3 pt-2 text-xs">
            <div className="bg-slate-950/65 p-2.5 rounded-xl border
            border-slate-800/40">
              <span className="text-slate-400 font-bold block mb-0.5
              uppercase tracking-wide text-[9px]">Shape</span>
              <span className="text-white
              font-extrabold">{activeDesc.shape}</span>
            </div>
            <div className="bg-slate-950/65 p-2.5 rounded-xl border
            border-slate-800/40">

```

```

        <span className="text-slate-400 font-bold block mb-0.5
uppercase tracking-wide text-[9px]">Volume</span>
        <span className="text-white
font-extrabold">{activeDesc.volume}</span>
    </div>
    <div className="bg-slate-950/65 p-2.5 rounded-xl border
border-slate-800/40 col-span-2">
        <span className="text-slate-400 font-bold block mb-0.5
uppercase tracking-wide text-[9px]">Particle Action</span>
        <span className="text-white font-extrabold flex
items-center gap-1.5">
            <span className="w-2 h-2 rounded-full bg-current
animate-ping"></span>
            {activeDesc.particleMovement}
        </span>
    </div>
</div>
</div>

{/* Memory Trick Panel */}
<div className="bg-slate-900 border border-slate-800
rounded-2xl p-5 space-y-3">
    <h4 className="text-sm font-black flex items-center gap-1.5
text-indigo-300">
        <Info className="w-4 h-4" /> Memory Helper Tricks!
    </h4>
    <div className="space-y-2.5 text-xs text-slate-300">
        <div className="flex gap-2">
            <span className="text-blue-400 font-black">S</span>
            <p><strong className="text-white">Solid</strong> =
<strong>S</strong>tructural grid, <strong>S</strong>tiff, and
<strong>S</strong>tationary vibrate.</p>
        </div>
        <div className="flex gap-2">
            <span className="text-emerald-400 font-black">L</span>

```

```

        <p><strong className="text-white">Liquid</strong> =
<strong>L</strong>oose clusters, <strong>L</strong>ower boundary
flowing.</p>

    </div>
    <div className="flex gap-2">
        <span className="text-amber-400 font-black">G</span>
        <p><strong className="text-white">Gas</strong> =
<strong>G</strong>iant gaps, <strong>G</strong>o everywhere wildly!</p>
    </div>
</div>
</div>

    { /* Achievement Badge Status Indicators */ }
    <div className="bg-slate-900 border border-slate-800
rounded-2xl p-5 space-y-3">
        <h4 className="text-sm font-black text-slate-300 flex
items-center gap-1">
            <Award className="w-4 h-4 text-amber-500" /> Lab
Milestones & Badges
        </h4>
        <div className="grid grid-cols-2 gap-3 text-xs">
            <div className={`p-3 rounded-xl border flex flex-col
gap-1 transition ${unlockedSubZero ? 'bg-sky-950/20 border-sky-800
text-sky-200' : 'bg-slate-950 border-slate-850 opacity-40'}`}>
                <span className="text-lg">🧊</span>
                <span className="font-bold">Deep Freeze</span>
                <span className="text-[10px] text-slate-400">Reach
subzero (-15°C)</span>
            </div>

            <div className={`p-3 rounded-xl border flex flex-col
gap-1 transition ${unlockedLiquid ? 'bg-emerald-950/20 border-emerald-800
text-emerald-200' : 'bg-slate-950 border-slate-850 opacity-40'}`}>
                <span className="text-lg">🔥</span>
                <span className="font-bold">Ice Melter</span>

```

```

        <span className="text-[10px] text-slate-400">Unlock
Liquid water</span>
    </div>

    <div className={`p-3 rounded-xl border flex flex-col
gap-1 transition ${unlockedGas ? 'bg-amber-950/20 border-amber-800
text-amber-200' : 'bg-slate-950 border-slate-850 opacity-40'}`}>
        <span className="text-lg">🌀</span>
        <span className="font-bold">Gas Giant</span>
        <span className="text-[10px] text-slate-400">Transform
to hot steam</span>
    </div>

    <div className={`p-3 rounded-xl border flex flex-col
gap-1 transition ${quizSubmitted ? 'bg-indigo-950/20 border-indigo-800
text-indigo-200' : 'bg-slate-950 border-slate-850 opacity-40'}`}>
        <span className="text-lg">🎓</span>
        <span className="font-bold">Quiz Master</span>
        <span className="text-[10px] text-slate-400">Submit
class test</span>
    </div>
</div>
</div>
</div>
)}

{/* TAB CONTENT: QUIZ */}
{activeTab === 'quiz' && (
    <div className="bg-slate-900 border border-slate-800
rounded-2xl p-5 space-y-4 max-h-[500px] overflow-y-auto animate-fadeIn">

        <div className="flex items-center justify-between border-b
border-slate-800 pb-3">
            <div>

```



```

        <h4 className="text-base font-black
text-indigo-300">Quick Academic Quiz</h4>
        <p className="text-xs text-slate-400">Test your particle
knowledge! (10 Questions)</p>
    </div>
    {quizSubmitted && (
        <span className="bg-indigo-950 text-indigo-300 px-3 py-1
rounded-full text-xs font-black border border-indigo-800">
            Score: {quizScore}/100
        </span>
    )}
</div>

<div className="space-y-6">
    {quizQuestions.map((item, idx) => {
        const hasAnswered = selectedQuizAnswers[item.id] !==
undefined;

        const isCorrect = selectedQuizAnswers[item.id] ===
item.ans;

        return (
            <div key={item.id} className="space-y-2 border-b
border-slate-800/60 pb-4 last:border-b-0">
                <span className="text-[10px] uppercase font-extrabold
tracking-widest text-indigo-400">
                    Question {idx + 1}
                </span>
                <p className="text-sm font-semibold text-slate-200
leading-snug">
                    {item.q}
                </p>

                <div className="grid grid-cols-1 gap-2 pt-1.5">
                    {item.options.map((opt, optIdx) => {
                        const isSelected = selectedQuizAnswers[item.id]
=== optIdx;

```

```

        let btnStyle = "bg-slate-950 border-slate-800
text-slate-300 hover:bg-slate-850";

        if (isSelected) {
            btnStyle = "bg-indigo-950 border-indigo-500
text-indigo-300 font-bold";
        }

        if (quizSubmitted) {
            if (optIdx === item.ans) {
                // Correct answer glows green
                btnStyle = "bg-emerald-950 border-emerald-500
text-emerald-300 font-bold";
            } else if (isSelected) {
                // User's wrong answer glows red
                btnStyle = "bg-rose-950 border-rose-500
text-rose-300 line-through";
            } else {
                btnStyle = "bg-slate-950 border-slate-900
opacity-50 cursor-not-allowed";
            }
        }

        return (
            <button
                key={optIdx}
                disabled={quizSubmitted}
                onClick={() => handleSelectAnswer(item.id,
optIdx)}

                className={`py-2 px-3 text-xs rounded-xl
text-left border transition-all flex items-center justify-between
${btnStyle}`}
            >
            <span>{opt}</span>
            {quizSubmitted && optIdx === item.ans && (

```

```

        <CheckCircle className="w-4 h-4
text-emerald-400 shrink-0" />
      )}
      {quizSubmitted && isSelected && optIdx !==
item.ans && (
        <XCircle className="w-4 h-4 text-rose-400
shrink-0" />
      )}
    </button>
  );
  }}}
</div>

{/* Diagnostic Explanation block visible after
submission */}

{quizSubmitted && (
  <div className="bg-slate-950/80 p-3 rounded-lg
border border-slate-800 text-[11px] text-slate-400 mt-2 leading-relaxed">
    <strong className="text-slate-300">Why? </strong>
    {item.explain}
  </div>
  )}
</div>
);
}}}
</div>

{/* Submit Action */}
{!quizSubmitted ? (
  <button
    onClick={handleSubmitQuiz}
    disabled={Object.keys(selectedQuizAnswers).length < 10}
    className={`w-full py-3 rounded-xl font-bold transition
text-xs flex items-center justify-center gap-2
${Object.keys(selectedQuizAnswers).length < 10 ? 'bg-slate-800
text-slate-500 cursor-not-allowed' : 'bg-gradient-to-r from-emerald-600

```

```

to-emerald-500 hover:from-emerald-500 hover:to-emerald-400 text-white
shadow-lg shadow-emerald-500/20'}`}`
    >
    <CheckCircle className="w-4 h-4" />
    <span>
      {Object.keys(selectedQuizAnswers).length < 10
        ? `Answer all questions
        (${Object.keys(selectedQuizAnswers).length}/10)`
        : 'Submit Test & Claim XP!'}
    </span>
  </button>
) : (
  <div className="bg-indigo-950/20 border
border-indigo-800/60 rounded-xl p-4 text-center space-y-2">
    <span className="text-xl">🏆</span>
    <p className="text-xs font-bold text-indigo-300">Test
Completed! You earned {(quizScore * 2)} XP!</p>
    <button
      onClick={() => {
        setSelectedQuizAnswers({});
        setQuizSubmitted(false);
        setShowQuizResults(false);
      }}
      className="mt-1 px-4 py-1.5 bg-slate-800
hover:bg-slate-700 text-slate-300 text-[11px] font-bold rounded-lg
transition"
    >
      Try Quiz Again
    </button>
  </div>
)}

</div>
)}

{/* TAB CONTENT: DIY ACTIVITY */}

```

```

{activeTab === 'diy' && (
  <div className="bg-slate-900 border border-slate-800
rounded-2xl p-5 space-y-4 animate-fadeIn">

    <div className="border-b border-slate-800 pb-3 flex
items-center gap-2">
      <span className="text-lg">📌</span>
      <div>
        <h4 className="text-base font-black text-rose-300">DIY
Kitchen Lab</h4>
        <p className="text-xs text-slate-400">The Magic Expanding
Balloon Experiment</p>
      </div>
    </div>

    <div className="space-y-3.5 text-xs">

      <div className="bg-slate-950 p-3 rounded-xl border
border-slate-850">
        <span className="font-extrabold text-slate-200 block
mb-1">🍷 Materials Required:</span>
        <p className="text-slate-400 font-medium">An empty small
plastic bottle, a deflated rubber balloon, and a container of warm tap
water.</p>
      </div>

      <div className="space-y-3">
        <span className="font-extrabold text-slate-200 block">📖
Easy Steps:</span>

        <div className="flex gap-2.5">
          <span className="w-5 h-5 rounded-full bg-slate-800
text-slate-300 flex items-center justify-center text-[10px] font-bold
shrink-0 mt-0.5">1</span>

```

```
        <p className="text-slate-300">Stretch the lip of your
deflated balloon completely over the narrow mouth of the plastic
bottle.</p>
```

```
    </div>
```

```
    <div className="flex gap-2.5">
```

```
        <span className="w-5 h-5 rounded-full bg-slate-800
text-slate-300 flex items-center justify-center text-[10px] font-bold
shrink-0 mt-0.5">2</span>
```

```
        <p className="text-slate-300">Carefully sit the bottom
of the bottle flat inside a bowl filled with hot tap water.</p>
```

```
    </div>
```

```
    <div className="flex gap-2.5">
```

```
        <span className="w-5 h-5 rounded-full bg-slate-800
text-slate-300 flex items-center justify-center text-[10px] font-bold
shrink-0 mt-0.5">3</span>
```

```
        <p className="text-slate-300">Watch for 30-60 seconds!
The balloon magically expands and stands upright without anybody blowing
into it!</p>
```

```
    </div>
```

```
</div>
```

```
    <div className="bg-indigo-950/30 border
border-indigo-900/60 p-3.5 rounded-xl space-y-1.5">
```

```
        <span className="font-black text-indigo-300 flex
items-center gap-1">
```

```
            <Lightbulb className="w-4 h-4 text-yellow-400" />
```

```
            How does this work?
```

```
        </span>
```

```
        <p className="text-slate-300 leading-relaxed
text-[11px]">
```

```
            The hot water heats up the air particles inside the
bottle. As they gain temperature, they move much faster and spread apart,
rushing up into the balloon and ballooning it open!
```

```
        </p>
```

```

    </div>

    <div className="border-t border-slate-800 pt-3">
      <p className="text-slate-400 text-[11px] italic">
        ⚠ <strong>Safety Check:</strong> Only use warm/hot
water from the tap. Avoid boiling water to keep it perfectly safe.
      </p>
    </div>

  </div>

</div>
)}

</section>

</main>

{/* 4. PREMIUM FOOTER / TEACHER GUIDANCE NOTIFICATIONS */}
<footer className="bg-slate-900 border-t border-slate-800 px-4 py-4
mt-8">
  <div className="max-w-7xl mx-auto flex flex-col md:flex-row
items-center justify-between gap-4 text-xs text-slate-400">
    <div className="flex items-center gap-3">
      <div className="p-1.5 bg-slate-800 rounded-lg">
        🎓
      </div>
      <div>
        <p className="font-extrabold text-slate-300">NCERT Curriculum
Guideline</p>
        <p>Designed for Grade 5 & 6 CBSE/NCERT curriculum to
understand atomic arrangements in common substances.</p>
      </div>
    </div>
    <div className="flex flex-wrap gap-2">

```

```
        <span className="px-2 py-1 bg-slate-950 rounded border
border-slate-800 text-[10px]">Solid: 0°C Below</span>
        <span className="px-2 py-1 bg-slate-950 rounded border
border-slate-800 text-[10px]">Liquid: 1°C - 99°C</span>
        <span className="px-2 py-1 bg-slate-950 rounded border
border-slate-800 text-[10px]">Gas: 100°C Above</span>
    </div>
</div>
</footer>

</div>
);
}
```


Magnetic Attraction5–6

```

import React, { useState, useEffect, useRef } from 'react';
import {
  Sparkles,
  RotateCcw,
  Eye,
  EyeOff,
  Compass as CompassIcon,
  Trophy,
  BookOpen,
  AlertTriangle,
  Info,
  ChevronRight,
  HelpCircle,
  Lightbulb,
  CheckCircle2,
  XCircle,
  Award,
  Play,
  Volume2,
  VolumeX
} from 'lucide-react';

// --- Web Audio API Synthesizer for high-quality, zero-asset sound
effects ---
class SoundManager {
  constructor() {
    this.ctx = null;
    this.muted = false;
  }

  init() {
    if (!this.ctx) {
      this.ctx = new (window.AudioContext || window.webkitAudioContext)();
    }
    if (this.ctx.state === 'suspended') {
      this.ctx.resume();
    }
  }
}

```

```

    }
}

playClink() {
    if (this.muted) return;
    this.init();
    try {
        const osc1 = this.ctx.createOscillator();
        const osc2 = this.ctx.createOscillator();
        const gainNode = this.ctx.createGain();

        osc1.type = 'sine';
        osc1.frequency.setValueAtTime(800, this.ctx.currentTime);
        osc1.frequency.exponentialRampToValueAtTime(150, this.ctx.currentTime
+ 0.15);

        osc2.type = 'triangle';
        osc2.frequency.setValueAtTime(1200, this.ctx.currentTime);
        osc2.frequency.exponentialRampToValueAtTime(300, this.ctx.currentTime
+ 0.12);

        gainNode.gain.setValueAtTime(0.3, this.ctx.currentTime);
        gainNode.gain.exponentialRampToValueAtTime(0.01, this.ctx.currentTime
+ 0.18);

        osc1.connect(gainNode);
        osc2.connect(gainNode);
        gainNode.connect(this.ctx.destination);

        osc1.start();
        osc2.start();
        osc1.stop(this.ctx.currentTime + 0.2);
        osc2.stop(this.ctx.currentTime + 0.2);
    } catch (e) {
        console.warn("Audio Context failed", e);
    }
}

```

```

}

playRepulsionHum(intensity) {
  if (this.muted) return;
  this.init();
  try {
    const osc = this.ctx.createOscillator();
    const gainNode = this.ctx.createGain();

    osc.type = 'sawtooth';
    osc.frequency.setValueAtTime(60 + intensity * 40,
this.ctx.currentTime);

    gainNode.gain.setValueAtTime(intensity * 0.15, this.ctx.currentTime);
    gainNode.gain.exponentialRampToValueAtTime(0.001,
this.ctx.currentTime + 0.15);

    // Low pass filter to make it sound muffled/magnetic
    const filter = this.ctx.createBiquadFilter();
    filter.type = 'lowpass';
    filter.frequency.setValueAtTime(150, this.ctx.currentTime);

    osc.connect(filter);
    filter.connect(gainNode);
    gainNode.connect(this.ctx.destination);

    osc.start();
    osc.stop(this.ctx.currentTime + 0.15);
  } catch (e) {}
}

playSuccess() {
  if (this.muted) return;
  this.init();
  try {
    const now = this.ctx.currentTime;

```

```

    const freqs = [523.25, 659.25, 783.99, 1046.50]; // C5, E5, G5, C6
chords
    freqs.forEach((f, index) => {
      const osc = this.ctx.createOscillator();
      const gainNode = this.ctx.createGain();
      osc.type = 'sine';
      osc.frequency.setValueAtTime(f, now + index * 0.08);
      gainNode.gain.setValueAtTime(0, now + index * 0.08);
      gainNode.gain.linearRampToValueAtTime(0.15, now + index * 0.08 +
0.02);
      gainNode.gain.exponentialRampToValueAtTime(0.001, now + index *
0.08 + 0.35);
      osc.connect(gainNode);
      gainNode.connect(this.ctx.destination);
      osc.start(now + index * 0.08);
      osc.stop(now + index * 0.08 + 0.4);
    });
  } catch (e) {}
}
}

const sounds = new SoundManager();

export default function App() {
  const [activeTab, setActiveTab] = useState('lab'); // 'lab' | 'theory' |
'quiz'
  const [xp, setXp] = useState(0);
  const [isMuted, setIsMuted] = useState(false);

  // Gamification states
  const [unlockedBadges, setUnlockedBadges] = useState([]);
  const unlockBadge = (id, name, desc) => {
    if (unlockedBadges.find(b => b.id === id)) return;
    const newBadge = { id, name, desc, timestamp: new
Date().toLocaleTimeString() };
    setUnlockedBadges(prev => [...prev, newBadge]);
  };

```

```

    setXp(prev => prev + 50);
    sounds.playSuccess();
    triggerFloatingToast(`Badge Unlocked: ${name}! (+50 XP)`);
  };

  const [toastMessage, setToastMessage] = useState(null);
  const triggerFloatingToast = (msg) => {
    setToastMessage(msg);
    setTimeout(() => setToastMessage(null), 3000);
  };

  return (
    <div className="min-h-screen bg-slate-950 text-slate-100 flex flex-col font-sans selection:bg-indigo-500 selection:text-white">
      { /* HEADER BAR */ }
      <header className="bg-slate-900 border-b border-slate-800 sticky top-0 z-40 px-4 py-3 shadow-lg backdrop-blur-md bg-opacity-95">
        <div className="max-w-7xl mx-auto flex flex-col sm:flex-row items-center justify-between gap-3">
          <div className="flex items-center gap-3">
            <div className="p-2 bg-gradient-to-tr from-rose-500 to-indigo-600 rounded-xl shadow-inner animate-pulse">
              <Sparkles className="w-6 h-6 text-white" />
            </div>
            <div>
              <h1 className="text-xl font-bold tracking-tight bg-gradient-to-r from-rose-400 via-indigo-200 to-indigo-400 bg-clip-text text-transparent">
                Magnetic Playground
              </h1>
              <p className="text-xs text-slate-400 font-medium">Class 5-6 NCERT Science Module</p>
            </div>
          </div>
        </div>
      </header>
      { /* Nav Tabs */ }
    </div>
  );

```

```

<div className="flex bg-slate-950 p-1 rounded-xl border
border-slate-800">
  <button
    onClick={() => setActiveTab('lab')}
    className={`flex items-center gap-1.5 px-4 py-1.5 rounded-lg
text-sm font-semibold transition-all duration-200 ${activeTab === 'lab' ?
'bg-indigo-600 text-white shadow' : 'text-slate-400
hover:text-slate-200'}`}
  >
    <CompassIcon className="w-4 h-4" />
    Interactive Lab
  </button>
  <button
    onClick={() => setActiveTab('theory')}
    className={`flex items-center gap-1.5 px-4 py-1.5 rounded-lg
text-sm font-semibold transition-all duration-200 ${activeTab === 'theory'
? 'bg-indigo-600 text-white shadow' : 'text-slate-400
hover:text-slate-200'}`}
  >
    <BookOpen className="w-4 h-4" />
    Concept Hub
  </button>
  <button
    onClick={() => {
      setActiveTab('quiz');
      unlockBadge('quiz_seeker', 'Curious mind', 'Visited the
quiz center to test knowledge.');
```

```

    </div>

    { /* Gamification Indicator & Sound Control */ }
    <div className="flex items-center gap-4">
        <div className="flex items-center gap-2 bg-slate-800/80 px-3.5
py-1.5 rounded-full border border-indigo-500/20 shadow-md">
            <Award className="w-4 h-4 text-yellow-400" />
            <span className="text-sm font-bold text-yellow-300">{xp}
<span className="text-xs text-slate-400">XP</span></span>
        </div>

        <button
            onClick={() => {
                const muteState = !isMuted;
                setIsMuted(muteState);
                sounds.muted = muteState;
            }}
            className="p-2 bg-slate-800 hover:bg-slate-700 text-slate-300
rounded-lg transition-colors border border-slate-700"
            title={isMuted ? "Unmute sounds" : "Mute sounds"}
        >
            {isMuted ? <VolumeX className="w-4 h-4" /> : <Volume2
className="w-4 h-4" />}
        </button>
    </div>
</div>
</header>

{ /* FLOATING TOAST NOTIFICATION */ }
{toastMessage && (
    <div className="fixed bottom-6 right-6 z-50 bg-indigo-600
text-white px-5 py-3 rounded-xl shadow-2xl border border-indigo-400 flex
items-center gap-2 animate-bounce">
        <Sparkles className="w-5 h-5 text-yellow-300 animate-spin" />
        <span className="font-bold text-sm">{toastMessage}</span>
    </div>

```



```

    })

    { /* MAIN LAYOUT */ }

    <main className="flex-1 max-w-7xl w-full mx-auto p-4 grid grid-cols-1
lg:grid-cols-12 gap-6 items-start">
      {activeTab === 'lab' && (
        <LabTab
          unlockBadge={unlockBadge}
          triggerFloatingToast={triggerFloatingToast}
          unlockedBadges={unlockedBadges}
        />
      )}
      {activeTab === 'theory' && <TheoryTab />}
      {activeTab === 'quiz' && <QuizTab addXp={ (amt) => setXp(p => p +
amt)} unlockBadge={unlockBadge} />}
    </main>

    { /* FOOTER */ }

    <footer className="bg-slate-950 border-t border-slate-900 py-6 mt-12
text-center text-xs text-slate-500">
      <div className="max-w-7xl mx-auto px-4 flex flex-col md:flex-row
justify-between items-center gap-4">
        <p>© 2026 Magnetic Playground. Empowering Class 5-6 NCERT
Scientists across the globe.</p>
        <div className="flex gap-4">
          <span className="hover:text-slate-300 cursor-pointer">Privacy
Policy</span>
          <span className="hover:text-slate-300 cursor-pointer">Terms of
Service</span>
          <span className="hover:text-slate-300 cursor-pointer">Teacher
Support</span>
        </div>
      </div>
    </footer>
  </div>
);

```

```

}

// =====
// 1. THE INTERACTIVE LAB COMPONENT
// =====

function LabTab({ unlockBadge, triggerFloatingToast, unlockedBadges = []
}) {
  // Magnet Position & Orientation Logic
  const workspaceRef = useRef(null);
  // Magnet A is stationary in the center-left:
  const stationaryX = 180;
  const stationaryY = 180;

  // Magnet B is draggable:
  const [magBX, setMagBX] = useState(450);
  const [magBY, setMagBY] = useState(180);
  const [isFlipped, setIsFlipped] = useState(false); // false = North (Red)
on Right, true = South (Blue) on Right
  const [showFieldLines, setShowFieldLines] = useState(true);
  const [isDragging, setIsDragging] = useState(false);
  const [dragOffset, setDragOffset] = useState({ x: 0, y: 0 });

  // Floating compass feature
  const [isCompassActive, setIsCompassActive] = useState(true);
  const [compassPos, setCompassPos] = useState({ x: 320, y: 80 });
  const [isDraggingCompass, setIsDraggingCompass] = useState(false);
  const [compassDragOffset, setCompassDragOffset] = useState({ x: 0, y: 0
});

  // Status metrics
  const [distance, setDistance] = useState(270);
  const [forceType, setForceType] = useState('None'); // 'Attracting' |
'Repelling' | 'Locked' | 'None'
  const [proximityPercent, setProximityPercent] = useState(0);
  const [shakeIntensity, setShakeIntensity] = useState(0);

```

```

// Reset Lab
const handleReset = () => {
  setMagBX(460);
  setMagBY(180);
  setCompassPos({ x: 320, y: 80 });
  setIsFlipped(false);
  setForceType('None');
  setShakeIntensity(0);
  triggerFloatingToast("Lab successfully reset!");
};

// Dimensions of magnets
const magWidth = 140;
const magHeight = 50;

// Run physical forces loop when coordinates change
useEffect(() => {
  // Distance between right side of Magnet A and left side of Magnet B
  const targetAX = stationaryX + magWidth / 2;
  const targetBX = magBX - magWidth / 2;
  const dx = targetBX - targetAX;
  const dy = magBY - stationaryY;
  const realDist = Math.max(0, Math.round(Math.sqrt(dx * dx + dy * dy)));
  setDistance(realDist);

  // Calculate proximity percent for visual bars (0% at 250px or more,
  100% at 0px)
  const rawProximity = Math.max(0, Math.min(100, Math.round(((250 -
realDist) / 250) * 100)));
  setProximityPercent(rawProximity);

  const opposingIsFacing = !isFlipped;

  if (realDist <= 5) {
    setForceType('Locked');
    setShakeIntensity(0);
  }

```

```

    } else if (realDist < 120) {
      if (opposingIsFacing) {
        setForceType('Attracting');
        setShakeIntensity(0);

        // Snap!
        if (realDist < 45 && !isDragging) {
          setMagBX(stationaryX + magWidth);
          setMagBY(stationaryY);
          sounds.playClink();
          unlockBadge('first_snap', 'Opposites Attract!', 'Successfully
brought opposite magnetic poles together and snapped them close.');
```

triggerFloatingToast("Magnetic Snap! Perfect alignment.");

```
        }
      } else {
        setForceType('Repelling');
        // Repulsion vibration effect
        const intensity = Math.max(0, (120 - realDist) / 20);
        setShakeIntensity(intensity);
        sounds.playRepulsionHum(intensity);

        // Active push-away feedback logic if not dragging
        if (realDist < 75 && !isDragging) {
          // Slide/Push Magnet B back
          const pushX = magBX + 35;
          setMagBX(Math.min(520, pushX));
          unlockBadge('repulsion_power', 'Invisible Shields', 'Experienced
how like magnetic poles push each other away.');
```

triggerFloatingToast("Woah! Same poles pushed each other away.");

```
        }
      }
    } else {
      setForceType('None');
      setShakeIntensity(0);
    }
  }, [magBX, magBY, isFlipped, isDragging]);

```

```

// Handle Dragging Magnet B (custom pointer implementation for
cross-device support)
const handlePointerDown = (e) => {
  e.preventDefault();
  if (!workspaceRef.current) return;
  const rect = workspaceRef.current.getBoundingClientRect();
  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;

  // Check if clicked near magnet B
  const clickDX = x - magBX;
  const clickDY = y - magBY;
  if (Math.abs(clickDX) < magWidth/2 + 10 && Math.abs(clickDY) <
magHeight/2 + 10) {
    setIsDragging(true);
    setDragOffset({ x: clickDX, y: clickDY });
    sounds.init();
  }
};

const handlePointerMove = (e) => {
  if (!workspaceRef.current) return;
  const rect = workspaceRef.current.getBoundingClientRect();
  const rawX = e.clientX - rect.left;
  const rawY = e.clientY - rect.top;

  if (isDragging) {
    // Keep inside bounds
    let newX = rawX - dragOffset.x;
    let newY = rawY - dragOffset.y;

    // Constrain inside container
    newX = Math.max(stationaryX + 45, Math.min(rect.width - magWidth/2 -
10, newX));

```

```

    newY = Math.max(magHeight/2 + 20, Math.min(rect.height - magHeight/2
- 20, newY));

    // Visual helper: semi-constrain Y coordinate to make sliding
straight easier
    if (Math.abs(newY - stationaryY) < 30) {
        newY = stationaryY; // Snap strictly to horizontal alignment axis
    }

    setMagBX(newX);
    setMagBY(newY);
}

if (isDraggingCompass) {
    let compassX = rawX - compassDragOffset.x;
    let compassY = rawY - compassDragOffset.y;

    compassX = Math.max(25, Math.min(rect.width - 25, compassX));
    compassY = Math.max(25, Math.min(rect.height - 25, compassY));

    setCompassPos({ x: compassX, y: compassY });
}
};

const handlePointerUp = () => {
    setIsDragging(false);
    setIsDraggingCompass(false);
};

// Dragging Compass Trigger
const handleCompassPointerDown = (e) => {
    e.stopPropagation();
    if (!workspaceRef.current) return;
    const rect = workspaceRef.current.getBoundingClientRect();
    const x = e.clientX - rect.left;
    const y = e.clientY - rect.top;

```

```

    setIsDraggingCompass(true);
    setCompassDragOffset({ x: x - compassPos.x, y: y - compassPos.y });
    unlockBadge('compass_user', 'Navigator Badge', 'Used the interactive
floating compass to chart the magnet fields.');
```

```

};

// Dynamic calculation of the compass needle rotation angle
const calculateCompassAngle = () => {
    const poleAN_X = stationaryX + magWidth / 4;
    const poleAN_Y = stationaryY;
    const poleAS_X = stationaryX - magWidth / 4;
    const poleAS_Y = stationaryY;

    let poleBN_X = magBX + magWidth / 4;
    let poleBN_Y = magBY;
    let poleBS_X = magBX - magWidth / 4;
    let poleBS_Y = magBY;

    if (isFlipped) {
        poleBN_X = magBX - magWidth / 4;
        poleBS_X = magBX + magWidth / 4;
    }

    const p1 = getFieldContribution(compassPos, { x: poleAN_X, y: poleAN_Y
}, 1);
    const p2 = getFieldContribution(compassPos, { x: poleAS_X, y: poleAS_Y
}, -1);
    const p3 = getFieldContribution(compassPos, { x: poleBN_X, y: poleBN_Y
}, 1);
    const p4 = getFieldContribution(compassPos, { x: poleBS_X, y: poleBS_Y
}, -1);

    const bx = p1.x + p2.x + p3.x + p4.x;
    const by = p1.y + p2.y + p3.y + p4.y;

```

```

    return Math.atan2(by, bx) * (180 / Math.PI);
};

const getFieldContribution = (point, pole, polarity) => {
    const dx = point.x - pole.x;
    const dy = point.y - pole.y;
    const r2 = dx * dx + dy * dy + 1000;
    const r = Math.sqrt(r2);
    const strength = (10000 / (r2 * r)) * polarity;
    return {
        x: dx * strength,
        y: dy * strength
    };
};

const compassAngle = calculateCompassAngle();

// Field Lines SVG path calculations (Dynamically calculated curves
// matching high physics accuracy)
const drawFieldLines = () => {
    const paths = [];
    const count = 5;
    const poleAN_X = stationaryX + magWidth / 4;
    const poleAN_Y = stationaryY;
    const poleAS_X = stationaryX - magWidth / 4;
    const poleAS_Y = stationaryY;

    let poleBN_X = magBX + magWidth / 4;
    let poleBN_Y = magBY;
    let poleBS_X = magBX - magWidth / 4;
    let poleBS_Y = magBY;

    if (isFlipped) {
        poleBN_X = magBX - magWidth / 4;
        poleBS_X = magBX + magWidth / 4;
    }
};

```



```

const isAttracting = !isFlipped;

for (let i = 0; i < count; i++) {
  const offset = (i - (count - 1) / 2) * 20;

  if (isAttracting) {
    const cp1x = poleAN_X + 60;
    const cp1y = poleAN_Y + offset * 1.5;
    const cp2x = poleBS_X - 60;
    const cp2y = poleBS_Y + offset * 1.5;

    paths.push(
      <path
        key={`attr-${i}`}
        d={`M ${poleAN_X} ${poleAN_Y} C ${cp1x} ${cp1y}, ${cp2x}
${cp2y}, ${poleBS_X} ${poleBS_Y}`}
        className="stroke-cyan-400/45 fill-none stroke-2 animate-dash"
        strokeDasharray="6,6"
        style={{ strokeWidth: Math.max(0.5, 3 - Math.abs(offset)/20) }}
      />
    );
  } else {
    const controlA_X = poleAN_X + 80;
    const controlA_Y = poleAN_Y + (offset < 0 ? -120 : 120);

    const controlB_X = poleBN_X - 80;
    const controlB_Y = poleBN_Y + (offset < 0 ? -120 : 120);

    paths.push(
      <path
        key={`rep-a-${i}`}
        d={`M ${poleAN_X} ${poleAN_Y} Q ${controlA_X} ${controlA_Y},
${poleAS_X} ${poleAS_Y}`}
        className="stroke-rose-500/40 fill-none stroke-2
animate-dash-reverse"

```

```

        strokeDasharray="5,6"
      />
    );

    paths.push(
      <path
        key={`rep-b-${i}`}
        d={`M ${poleBN_X} ${poleBN_Y} Q ${controlB_X} ${controlB_Y},
${poleBS_X} ${poleBS_Y}`}
        className="stroke-rose-500/40 fill-none stroke-2 animate-dash"
        strokeDasharray="5,6"
      />
    );
  }
}
return paths;
};

return (
  <div className="lg:col-span-12 grid grid-cols-1 lg:grid-cols-12 gap-6">

    {/* 1A. WORKSPACE & CONTROLS CONTAINER (9-cols wide on large) */}
    <div className="lg:col-span-8 flex flex-col gap-4">

      {/* Lab Workspace Display Panel */}
      <div className="bg-slate-900 border border-slate-800 rounded-2xl
shadow-xl overflow-hidden relative flex flex-col">

        {/* Workspace Status Ribbon */}
        <div className="bg-slate-950 px-4 py-2 flex items-center
justify-between border-b border-slate-800 text-xs font-semibold
text-slate-400">
          <span className="flex items-center gap-1.5 text-indigo-400">
            <span className="w-2 h-2 rounded-full bg-emerald-500
animate-ping"></span>
            Live Sandbox Physics
          </span>
        </div>
      </div>
    </div>
  </div>
);

```

```

    </span>
    <span className="bg-slate-800 px-2.5 py-0.5 rounded-full">
      Scale: 1 Pixel = 1 Millimeter
    </span>
  </div>

  { /* Interactive Core Canvas Container */ }
  <div
    ref={workspaceRef}
    onPointerMove={handlePointerMove}
    onPointerUp={handlePointerUp}
    onPointerLeave={handlePointerUp}
    className="w-full h-[380px] bg-slate-950/40 relative
overflow-hidden select-none cursor-crosshair touch-none"
    style={{
      backgroundImage: 'radial-gradient(#1e293b 1.5px, transparent
1.5px)',
      backgroundSize: '24px 24px'
    }}
  >
    { /* SVG Force field line visualizer overlay */ }
    { showFieldLines && (
      <svg className="absolute inset-0 w-full h-full
pointer-events-none">
        { drawFieldLines() }
      </svg>
    ) }

    { /* --- STATIC MAGNET A (FIXED) --- */ }
    <div
      style={{
        left: `${stationaryX - magWidth/2}px`,
        top: `${stationaryY - magHeight/2}px`,
        width: `${magWidth}px`,
        height: `${magHeight}px`
      }}
    >

```

```

        className="absolute rounded-xl shadow-2xl flex border-2
border-slate-900 select-none overflow-hidden z-10 hover:shadow-cyan-500/10
transition-shadow duration-300"
    >
        { /* South Pole - Blue */ }
        <div className="w-1/2 h-full bg-gradient-to-br
from-indigo-700 via-indigo-600 to-indigo-950 flex flex-col items-center
justify-center relative">
            <span className="text-xl font-black text-white/90
select-none tracking-wider">S</span>
            <span className="text-[9px] font-bold text-indigo-300/60
uppercase tracking-widest absolute bottom-1">South</span>
        </div>
        { /* North Pole - Red */ }
        <div className="w-1/2 h-full bg-gradient-to-br from-rose-500
via-rose-600 to-rose-950 flex flex-col items-center justify-center
relative border-1 border-slate-800/20">
            <span className="text-xl font-black text-white/90
select-none tracking-wider">N</span>
            <span className="text-[9px] font-bold text-rose-300/60
uppercase tracking-widest absolute bottom-1">North</span>
        </div>
        { /* Locked pin visual */ }
        <div className="absolute top-1/2 left-1/2 -translate-x-1/2
-translate-y-1/2 bg-slate-950 text-slate-400 border border-slate-800
rounded-full p-1.5 shadow-inner">
            <span className="text-[8px] font-bold tracking-widest
uppercase block text-slate-500">LOCKED</span>
        </div>
    </div>

    { /* --- DRAGGABLE MAGNET B --- */ }
    <div
        style={{
            left: `${magBX - magWidth/2}px`,
            top: `${magBY - magHeight/2}px`,

```

```

        width: `${magWidth}px`,
        height: `${magHeight}px`,
        transform: `translate(${shakeIntensity ? (Math.random() -
0.5) * shakeIntensity * 4 : 0}px, ${shakeIntensity ? (Math.random() - 0.5)
* shakeIntensity * 4 : 0}px)`
    }}
    onPointerDown={handlePointerDown}
    className={`absolute rounded-xl shadow-2xl flex border-2
select-none overflow-hidden z-20 cursor-grab active:cursor-grabbing
transition-shadow ${isDragging ? 'shadow-indigo-500/30 border-indigo-400
ring-2 ring-indigo-500/20' : 'shadow-slate-950/80 border-slate-800'}`}
    >
    {!isFlipped ? (
        <>
            {/* South Pole - Blue */}
            <div className="w-1/2 h-full bg-gradient-to-br
from-indigo-700 via-indigo-600 to-indigo-950 flex flex-col items-center
justify-center relative">
                <span className="text-xl font-black text-white/90
select-none">S</span>
                <span className="text-[9px] font-bold
text-indigo-300/60 uppercase tracking-widest absolute
bottom-1">South</span>
            </div>
            {/* North Pole - Red */}
            <div className="w-1/2 h-full bg-gradient-to-br
from-rose-500 via-rose-600 to-rose-950 flex flex-col items-center
justify-center relative border-l border-slate-800/20">
                <span className="text-xl font-black text-white/90
select-none">N</span>
                <span className="text-[9px] font-bold text-rose-300/60
uppercase tracking-widest absolute bottom-1">North</span>
            </div>
        </>
    ) : (
        <>

```

```

        { /* North Pole - Red (Flipped state) */
        <div className="w-1/2 h-full bg-gradient-to-br
from-rose-500 via-rose-600 to-rose-950 flex flex-col items-center
justify-center relative">
            <span className="text-xl font-black text-white/90
select-none">N</span>
            <span className="text-[9px] font-bold text-rose-300/60
uppercase tracking-widest absolute bottom-1">North</span>
        </div>
        { /* South Pole - Blue (Flipped state) */
        <div className="w-1/2 h-full bg-gradient-to-br
from-indigo-700 via-indigo-600 to-indigo-950 flex flex-col items-center
justify-center relative border-1 border-slate-800/20">
            <span className="text-xl font-black text-white/90
select-none">S</span>
            <span className="text-[9px] font-bold
text-indigo-300/60 uppercase tracking-widest absolute
bottom-1">South</span>
        </div>
    </>
    })
    { /* Drag handle graphic */
    <div className="absolute top-1 right-2 w-1.5 h-1.5
rounded-full bg-white/20"></div>
    <div className="absolute top-1 left-2 w-1.5 h-1.5
rounded-full bg-white/20"></div>
    </div>

    { /* --- FLOATING COMPASS TOOL --- */
    {isCompassActive && (
        <div
            style={{
                left: `${compassPos.x - 24}px`,
                top: `${compassPos.y - 24}px`,
            }}
            onPointerDown={handleCompassPointerDown}

```

```

        className="absolute w-12 h-12 rounded-full bg-slate-900
border-2 border-indigo-500/50 flex items-center justify-center cursor-move
select-none shadow-xl z-30 group hover:border-indigo-400 active:scale-105
transition-transform"

        title="Drag compass near magnets to map vectors"
    >
        { /* Compass dial markings */ }
        <div className="absolute inset-0.5 rounded-full border
border-slate-800 border-dashed animate-spin-slow"></div>
        <div className="text-[7px] text-indigo-400/80 absolute
top-0.5 font-bold">N</div>
        <div className="text-[7px] text-indigo-400/80 absolute
bottom-0.5 font-bold">S</div>

        { /* Pivot needle element */ }
        <div
            style={{ transform: `rotate(${compassAngle}deg)` }}
            className="w-full h-1 flex items-center justify-between
px-1 relative transition-transform duration-100 ease-out"
        >
            { /* Red half of needle (Points along magnetic field
direction) */ }
            <div className="w-1/2 h-full bg-rose-500
rounded-l"></div>
            { /* Silver half of needle */ }
            <div className="w-1/2 h-full bg-slate-300
rounded-r"></div>
            { /* Center pivot point pin */ }
            <div className="absolute left-1/2 top-1/2
-translate-x-1/2 -translate-y-1/2 w-2 h-2 rounded-full bg-amber-500 border
border-slate-950"></div>
        </div>
    </div>
)}

{ /* Hint Box popups in workspace corner */ }

```

```

        <div className="absolute bottom-2.5 left-2.5 right-2.5
bg-slate-900/90 border border-indigo-500/10 backdrop-blur rounded-xl p-2.5
flex items-center justify-between text-xs gap-3">
    <span className="flex items-center gap-1.5 text-slate-300">
        <Lightbulb className="w-4 h-4 text-amber-400 shrink-0" />
        <span>
            {forceType === 'None' && "💡 Drag the mobile magnet
closer to the stationary magnet."}
            {forceType === 'Attracting' && "★ Opposite poles
attract! Observe how they lock together with full strength."}
            {forceType === 'Repelling' && "🔥 Same poles repel! Hear
the magnetic vibration as they push apart."}
            {forceType === 'Locked' && "🔒 Magnetic Snap! Try
clicking FLIP to see how same poles interact."}
        </span>
    </span>
    <button
        onClick={() => {
            unlockBadge('hint_seeker', 'Curious Explorer', 'Looked
into sandbox insights for tips.');
```

triggerFloatingToast("Hint activated: Try aligning them
directly center!");

```

        }}
        className="bg-indigo-600 hover:bg-indigo-500 text-white
px-2.5 py-1 rounded text-[11px] font-bold shrink-0 transition"
    >
        Explore Hint
    </button>
</div>
</div>
</div>

    { /* BOTTOM WORKSPACE CONTROLS */ }
    <div className="bg-slate-900 border border-slate-800 rounded-2xl
p-4 flex flex-wrap items-center justify-between gap-4 shadow-md">
        <div className="flex flex-wrap items-center gap-3">

```



```

<button
  onClick={() => {
    setIsFlipped(!isFlipped);
    unlockBadge('magnet_flipper', 'Pole Reverser', 'Flipped
Magnet B to alternate its North and South poles.');
```

```
  }}
```

```
  className="px-4 py-2 bg-gradient-to-r from-rose-600
to-indigo-600 hover:from-rose-500 hover:to-indigo-500 text-white
rounded-xl font-bold text-sm flex items-center gap-1.5 transition-all
shadow-md active:scale-95"
```

```
>
```

```
  <RotateCcw className="w-4 h-4 animate-spin-slow" />
```

```
  Flip Magnet B
```

```
</button>
```

```
<button
```

```
  onClick={() => setShowFieldLines(!showFieldLines)}
```

```
  className={`px-4 py-2 rounded-xl font-bold text-sm flex
items-center gap-1.5 transition border ${showFieldLines ?
'bg-emerald-950/60 border-emerald-500/30 text-emerald-300' : 'bg-slate-800
border-slate-700 text-slate-300'}`}
```

```
>
```

```
  {showFieldLines ? <Eye className="w-4 h-4" /> : <EyeOff
className="w-4 h-4" />}
```

```
  {showFieldLines ? "Hide Force Fields" : "Show Force Fields"}
```

```
</button>
```

```
<button
```

```
  onClick={() => setIsCompassActive(!isCompassActive)}
```

```
  className={`px-4 py-2 rounded-xl font-bold text-sm flex
items-center gap-1.5 transition border ${isCompassActive ?
'bg-indigo-950/60 border-indigo-500/30 text-indigo-300' : 'bg-slate-800
border-slate-700 text-slate-300'}`}
```

```
>
```

```
  <CompassIcon className="w-4 h-4" />
```

```
  {isCompassActive ? "Hide Compass" : "Show Compass"}
```

```

        </button>
    </div>

    <button
        onClick={handleReset}
        className="px-4 py-2 bg-slate-800 hover:bg-slate-700
text-slate-300 hover:text-white rounded-xl text-sm font-semibold
transition border border-slate-700 flex items-center gap-1"
    >
        <RotateCcw className="w-4 h-4" />
        Reset Space
    </button>
</div>

{/* CSS KEYFRAME ANIMATIONS FOR SVGS & ROTATION */}
<style dangerouslySetInnerHTML={{__html: `
    @keyframes dash {
        to {
            stroke-dashoffset: -40;
        }
    }
    @keyframes dash-reverse {
        to {
            stroke-dashoffset: 40;
        }
    }
    .animate-dash {
        animation: dash 1.5s linear infinite;
    }
    .animate-dash-reverse {
        animation: dash-reverse 1.5s linear infinite;
    }
    @keyframes spin-slow {
        from { transform: rotate(0deg); }
        to { transform: rotate(360deg); }
    }
`}}

```

```

        .animate-spin-slow {
            animation: spin-slow 16s linear infinite;
        }
    `}} />

</div>

{/* 1B. DYNAMIC OBSERVATION CENTER PANEL (4-cols wide on large) */}
<div className="lg:col-span-4 flex flex-col gap-6">

    {/* Force Measurement Meter */}
    <div className="bg-slate-900 border border-slate-800 rounded-2x1
p-5 shadow-lg flex flex-col gap-4">
        <h2 className="text-sm font-bold text-indigo-300 tracking-wide
uppercase flex items-center gap-1.5">
            <Info className="w-4 h-4 text-indigo-400" />
            Observation Center
        </h2>

        {/* Current Proximity Meter */}
        <div className="flex flex-col gap-1.5">
            <div className="flex justify-between text-xs">
                <span className="text-slate-400">Pole-to-Pole
Distance:</span>
                <span className="font-mono text-emerald-400
font-bold">{distance} mm</span>
            </div>
            <div className="w-full bg-slate-950 h-3 rounded-full
overflow-hidden border border-slate-800">
                <div
                    style={{ width: `${Math.min(100, proximityPercent)}%` }}
                    className={`h-full rounded-full transition-all duration-300
bg-gradient-to-r ${forceType === 'Repelling' ? 'from-rose-500 to-rose-600'
: 'from-indigo-500 to-emerald-500'} ``}
                ></div>
            </div>
        </div>
    </div>

```

```
</div>
```

```
    { /* Active Force Gauge readout */ }

    <div className="bg-slate-950 p-4 rounded-xl border
border-slate-800 flex flex-col gap-3">
      <div className="flex items-center justify-between">
        <span className="text-xs text-slate-400">Interaction
State:</span>
        <span className={`px-2.5 py-0.5 rounded-full text-[11px]
font-black uppercase tracking-wider ${
          forceType === 'Locked' ? 'bg-emerald-950 text-emerald-300
border border-emerald-500/20' :
          forceType === 'Attracting' ? 'bg-indigo-950 text-indigo-300
border border-indigo-500/20' :
          forceType === 'Repelling' ? 'bg-rose-950 text-rose-300
border border-rose-500/20 animate-pulse' :
            'bg-slate-800 text-slate-400'
        }`}>
          {forceType}
        </span>
      </div>

      { /* Display simple child-friendly physical explanations of
current states */ }

      <div className="text-xs text-slate-300 leading-relaxed
min-h-[48px]">
        {forceType === 'None' && "The magnets are too far apart to
feel each other. Bring Magnet B closer to begin!"}
        {forceType === 'Attracting' && "Opposites attract! The North
pole (Red) of Magnet A is pulling the South pole (Blue) of Magnet B toward
itself."}
        {forceType === 'Repelling' && "Like poles repel! Red is
trying to touch Red, forcing Magnet B to push outward. Feel the
vibration!"}
```

```

        {forceType === 'Locked' && "SNAP! The magnets have attracted
and are locked together. To separate them, drag Magnet B back to the
right."}

    </div>
  </div>
</div>

  { /* Discovery Goals list (Acts as dynamic lab checkmark checklist)
  */ }

  <div className="bg-slate-900 border border-slate-800 rounded-2xl
p-5 shadow-lg flex flex-col gap-4">
    <h2 className="text-sm font-bold text-slate-300 tracking-wide
uppercase">Your Science Missions</h2>
    <div className="flex flex-col gap-2.5 text-xs">

      { /* Goal 1 */ }
      <div className="flex items-start gap-2.5">
        <div className={`mt-0.5 w-4 h-4 rounded-full flex
items-center justify-center shrink-0 border ${unlockedBadges.find(b =>
b.id === 'first_snap') ? 'bg-emerald-950 text-emerald-400
border-emerald-500' : 'border-slate-700 bg-slate-950'}`}>
          {unlockedBadges.find(b => b.id === 'first_snap') &&
<CheckCircle2 className="w-3.5 h-3.5 fill-emerald-950" />}
        </div>
        <div className="flex-1">
          <p className={`font-semibold ${unlockedBadges.find(b =>
b.id === 'first_snap') ? 'text-emerald-400 line-through' :
'text-slate-300'}`}>Opposites Attract Mission</p>
          <p className="text-[11px] text-slate-500">Snap Magnet B
onto Magnet A by bringing opposite poles close.</p>
        </div>
      </div>
    </div>

    { /* Goal 2 */ }
    <div className="flex items-start gap-2.5">

```

```

        <div className={`mt-0.5 w-4 h-4 rounded-full flex
items-center justify-center shrink-0 border ${unlockedBadges.find(b =>
b.id === 'repulsion_power') ? 'bg-emerald-950 text-emerald-400
border-emerald-500' : 'border-slate-700 bg-slate-950'}`}>
            {unlockedBadges.find(b => b.id === 'repulsion_power') &&
<CheckCircle2 className="w-3.5 h-3.5 fill-emerald-950" />}
        </div>
        <div className="flex-1">
            <p className={`font-semibold ${unlockedBadges.find(b =>
b.id === 'repulsion_power') ? 'text-emerald-400 line-through' :
'text-slate-300'}`}>>Likes Repel Mission</p>
            <p className="text-[11px] text-slate-500">Flip the magnet
and push similar poles close to feel repulsion.</p>
        </div>
    </div>

    { /* Goal 3 */ }
    <div className="flex items-start gap-2.5">
        <div className={`mt-0.5 w-4 h-4 rounded-full flex
items-center justify-center shrink-0 border ${unlockedBadges.find(b =>
b.id === 'compass_user') ? 'bg-emerald-950 text-emerald-400
border-emerald-500' : 'border-slate-700 bg-slate-950'}`}>
            {unlockedBadges.find(b => b.id === 'compass_user') &&
<CheckCircle2 className="w-3.5 h-3.5 fill-emerald-950" />}
        </div>
        <div className="flex-1">
            <p className={`font-semibold ${unlockedBadges.find(b =>
b.id === 'compass_user') ? 'text-emerald-400 line-through' :
'text-slate-300'}`}>>Vector Navigator Mission</p>
            <p className="text-[11px] text-slate-500">Drag the Floating
Compass around the field lines to map magnetic flow.</p>
        </div>
    </div>

</div>
</div>

```

```

    { /* Fun Badge Room */ }

    <div className="bg-slate-900 border border-slate-800 rounded-2xl
p-5 shadow-lg flex flex-col gap-3">
      <h2 className="text-sm font-bold text-slate-300 tracking-wide
uppercase flex items-center gap-1.5">
        <Award className="w-4.5 h-4.5 text-yellow-500" />
        Earned Badges ({unlockedBadges.length})
      </h2>
      {unlockedBadges.length === 0 ? (
        <p className="text-xs text-slate-500 italic">No badges unlocked
yet. Start playing in the simulator space above!</p>
      ) : (
        <div className="grid grid-cols-1 gap-2">
          {unlockedBadges.map((badge) => (
            <div key={badge.id} className="flex items-center gap-2.5
p-2 bg-slate-950 rounded-xl border border-slate-800 hover:border-slate-700
transition">
              <div className="w-8 h-8 rounded-full bg-indigo-900/40
border border-indigo-500/30 flex items-center justify-center shrink-0">
                <Sparkles className="w-4 h-4 text-indigo-400" />
              </div>
              <div>
                <h3 className="text-xs font-extrabold
text-indigo-300">{badge.name}</h3>
                <p className="text-[10px] text-slate-500
leading-tight">{badge.desc}</p>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  </div>

```

```

    </div>
  );
}

// =====
// 2. THE CONCEPT HUB (THEORY / DO-IT-YOURSELF)
// =====
function TheoryTab() {
  return (
    <div className="lg:col-span-12 grid grid-cols-1 md:grid-cols-12 gap-8 text-slate-300">

      {/* Visual Hook & Core Explanation */}
      <div className="md:col-span-7 flex flex-col gap-6">

        {/* Core Lesson */}
        <div className="bg-slate-900 border border-slate-800 rounded-2xl p-6 shadow-md flex flex-col gap-4">
          <h2 className="text-2xl font-bold bg-gradient-to-r from-indigo-300 to-rose-300 bg-clip-text text-transparent">
            How do Magnets Talk to Each Other?
          </h2>
          <p className="text-sm leading-relaxed text-slate-400">
            Have you ever felt an invisible wall? A physical force pushing your hands away even though there is absolutely nothing standing there? That's the secret power of a magnetic field!
          </p>
          <p className="text-sm leading-relaxed text-slate-400">
            Every magnet has two opposite ends called <strong className="text-indigo-400">Poles</strong>: a <strong>North Pole (N)</strong> and a <strong>South Pole (S)</strong>. These poles communicate using a cosmic law of nature:
          </p>

          <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 my-2">

```



```

    <div className="bg-slate-950 p-4 rounded-xl border
border-rose-500/20 flex flex-col items-center text-center">
      <div className="w-10 h-10 rounded-full bg-rose-950
text-rose-400 flex items-center justify-center text-lg font-bold
mb-2">S</div>
      <h3 className="text-sm font-bold text-rose-300">Like Poles
Repel</h3>
      <p className="text-[11px] text-slate-500 mt-1">If you push
North to North (or South to South), they will fight back and push
apart!</p>
    </div>
    <div className="bg-slate-950 p-4 rounded-xl border
border-indigo-500/20 flex flex-col items-center text-center">
      <div className="w-10 h-10 rounded-full bg-indigo-950
text-indigo-400 flex items-center justify-center text-lg font-bold
mb-2">N</div>
      <h3 className="text-sm font-bold text-indigo-300">Opposites
Attract</h3>
      <p className="text-[11px] text-slate-500 mt-1">If you bring
North and South together, they will snap close and stick together
tightly!</p>
    </div>
  </div>

  <div className="bg-indigo-950/40 border border-indigo-500/30 p-4
rounded-xl flex items-start gap-3">
    <Lightbulb className="w-5 h-5 text-indigo-400 shrink-0 mt-0.5"
/>
    <div>
      <p className="text-xs font-bold text-indigo-200">The Magnetic
Field</p>
      <p className="text-xs text-indigo-300 leading-relaxed
mt-0.5">
        The space around a magnet where this pulling or pushing
force can be felt is called its <strong>Magnetic Field</strong>. This

```

field **is** completely invisible to human eyes, but flowing force lines are constantly streaming **from North** directly into **South**!

```
</p>
</div>
</div>
</div>

{/* Common Mistakes */}
<div className="bg-slate-900 border border-slate-800 rounded-2xl
p-6 shadow-md flex flex-col gap-4">
  <h2 className="text-base font-bold text-rose-400 uppercase
tracking-wider flex items-center gap-1.5">
    <AlertTriangle className="w-5 h-5 text-rose-500" />
    Common Student Misconceptions
  </h2>

  <div className="flex flex-col gap-3 text-xs">
    <div className="bg-slate-950 p-3.5 rounded-xl border
border-slate-800">
      <p className="font-bold text-slate-300">🔴 "Magnets stick to
all shiny metal objects like foil or soda cans."</p>
      <p className="text-slate-500 mt-1">
        <strong>Why it's wrong:</strong> Magnets only attract very
specific metals like <span className="text-slate-300">Iron, Nickel, and
Cobalt</span>. Aluminum foil, copper coins, and gold rings will never
interact with standard magnets!
      </p>
    </div>

    <div className="bg-slate-950 p-3.5 rounded-xl border
border-slate-800">
      <p className="font-bold text-slate-300">🔴 "Breaking a magnet
in half gives you one pure North bar and one pure South bar."</p>
      <p className="text-slate-500 mt-1">
        <strong>Why it's wrong:</strong> If you snap a magnet in
half, you simply get <span className="text-slate-300">two smaller
```

magnets, each with its own brand new North and South poles. You can never separate magnetic poles!

```
        </p>
      </div>
    </div>
  </div>

</div>

{ /* DIY Activity & FAQs */ }
<div className="md:col-span-5 flex flex-col gap-6">

  { /* DIY Floating Paperclip */ }
  <div className="bg-slate-900 border border-slate-800 rounded-2xl
p-6 shadow-md flex flex-col gap-4">
    <span className="bg-indigo-900/30 text-indigo-300 border
border-indigo-500/20 px-3 py-1 rounded-full text-[10px] font-bold
uppercase tracking-wider self-start">
      DIY Home Experiment
    </span>
    <h2 className="text-lg font-bold text-slate-100">
      The Floating Paperclip Magic
    </h2>
    <p className="text-xs text-slate-400 leading-relaxed">
      Want to demonstrate invisible magnetic fields to your family?
      Try this neat science trick at home using basic household supplies!
    </p>

    <div className="bg-slate-950 p-4 rounded-xl border
border-slate-800 flex flex-col gap-3 text-xs text-slate-300">
      <div>
        <p className="font-bold text-indigo-300">🔧 Materials
        Needed:</p>
        <p className="text-slate-400 mt-0.5">• A magnet, a steel
        paperclip, thread (20cm), tape.</p>
      </div>
```

```

<div>
  <p className="font-bold text-indigo-300">⚡ Action Steps:</p>
  <ol className="list-decimal pl-4 text-slate-400 mt-0.5
space-y-1">
    <li>Tie the thread securely to the metal paperclip.</li>
    <li>Tape the other end of the thread onto your table.</li>
    <li>Hold your magnet in the air right above the
paperclip.</li>
    <li>Pull the paperclip up towards the magnet until it is
almost touching, but <strong>do not let them touch</strong>.</li>
    <li>Gently let go! The clip floats in mid-air against
gravity!</li>
  </ol>
</div>

```

```

<div className="bg-indigo-950/20 p-2.5 rounded border
border-indigo-500/10 text-slate-400">
  <p className="text-[11px]"><strong>Why it works:</strong> The
invisible magnetic field lines stretch out into open space, holding the
metal clip up without touching it!</p>
</div>
</div>
</div>

```

```

{/* Quick FAQs */}
<div className="bg-slate-900 border border-slate-800 rounded-2xl
p-6 shadow-md flex flex-col gap-4">
  <h2 className="text-sm font-bold text-indigo-300 uppercase
tracking-wider">Frequently Asked Questions</h2>
  <div className="flex flex-col gap-3 text-xs leading-relaxed">
    <div>
      <p className="font-bold text-slate-200">Q: Why are magnet
poles painted Red and Blue?</p>
      <p className="text-slate-400 mt-0.5">A: It is an
international standard! Red always represents the North pole, and Blue

```

```

represents the South pole so scientists everywhere can communicate
instantly.</p>
    </div>
    <div className="border-t border-slate-800/80 pt-3">
      <p className="font-bold text-slate-200">Q: Can a magnet lose
its power?</p>
      <p className="text-slate-400 mt-0.5">A: Yes! Dropping
magnets, hitting them with a hammer, or heating them over fire will
scramble their atomic layout, destroying their magnetism.</p>
    </div>
  </div>
</div>
);
}

// =====
// 3. GAMIFIED ACADEMIC QUIZ COMPONENT
// =====
const QUIZ_QUESTIONS = [
  {
    question: "What are the names of the two poles found on every single
magnet?",
    options: [
      { text: "Plus and Minus", label: "A" },
      { text: "East and West", label: "B" },
      { text: "North and South", label: "C" },
      { text: "Up and Down", label: "D" }
    ],
    correctIndex: 2,
    explanation: "Every magnet, no matter its shape or size, possesses a
North pole and a South pole. Plus/minus are electrical charges."
  },

```

```
{
  question: "What happens if you break a bar magnet directly in half?",
  options: [
    { text: "You get one North magnet and one South magnet", label: "A"
  },
    { text: "Both halves lose all their magnetic power", label: "B" },
    { text: "Each half becomes a new magnet with both N and S poles",
label: "C" },
    { text: "The magnet explodes with bright sparks", label: "D" }
  ],
  correctIndex: 2,
  explanation: "Magnetic poles always exist in inseparable pairs.
Snapping a magnet simply creates two smaller complete magnets."
},
{
  question: "What will happen when you bring two North poles (N) close
together?",
  options: [
    { text: "They will attract each other", label: "A" },
    { text: "They will repel (push apart)", label: "B" },
    { text: "They will melt instantly", label: "C" },
    { text: "Nothing happens", label: "D" }
  ],
  correctIndex: 1,
  explanation: "Like poles always repel each other. This is the
fundamental rule of magnetism!"
},
{
  question: "Magnetic field lines outside a magnet are defined to flow
from:",
  options: [
    { text: "South pole to North pole", label: "A" },
    { text: "North pole to South pole", label: "B" },
    { text: "East side to West side", label: "C" },
    { text: "The center to the edges", label: "D" }
  ],
}
```

```
    correctIndex: 1,
    explanation: "By convention, magnetic field lines loop outward from the
North pole and curve back into the South pole."
},
{
    question: "Which of the following items will be strongly attracted by a
magnet?",
    options: [
        { text: "A shiny copper coin", label: "A" },
        { text: "A plastic ruler", label: "B" },
        { text: "An iron nail", label: "C" },
        { text: "A gold wedding ring", label: "D" }
    ],
    correctIndex: 2,
    explanation: "Iron is a ferromagnetic material. Copper, plastic, and
gold are non-magnetic materials."
},
{
    question: "As you pull a magnet further away from a metallic pin, what
happens to the force?",
    options: [
        { text: "It stays exactly the same", label: "A" },
        { text: "It gets weaker", label: "B" },
        { text: "It gets stronger", label: "C" },
        { text: "The poles swap sides", label: "D" }
    ],
    correctIndex: 1,
    explanation: "Magnetic force is heavily distance-dependent. The further
away the object is, the weaker the magnetic pull becomes."
},
{
    question: "Which of these is the ONLY foolproof test that an object is
actually a magnet?",
    options: [
        { text: "It is attracted to a magnet's North pole", label: "A" },
        { text: "It conducts electricity", label: "B" },
```

```
    { text: "It can repel another magnet", label: "C" },
    { text: "It sinks in water", label: "D" }
  ],
  correctIndex: 2,
  explanation: "Plain iron is attracted to both poles. Only a real magnet
will show repulsion when similar poles face each other!"
},
{
  question: "Why can a magnet pull a steel pin across a table without
even touching it?",
  options: [
    { text: "It blows air toward the pin", label: "A" },
    { text: "Its invisible magnetic field reaches across space", label:
"B" },
    { text: "It uses static electricity sparks", label: "C" },
    { text: "It is a magic trick", label: "D" }
  ],
  correctIndex: 1,
  explanation: "Magnets operate using non-contact force fields. The
invisible magnetic field lines pull materials across open space."
},
{
  question: "How do high-speed bullet Maglev trains float above their
tracks?",
  options: [
    { text: "They run on massive steam jet engines", label: "A" },
    { text: "They use wings to glide like planes", label: "B" },
    { text: "Powerful electromagnets repel each other to lift the train",
label: "C" },
    { text: "They ride on large pillows of air", label: "D" }
  ],
  correctIndex: 2,
  explanation: "Maglev (Magnetic Levitation) trains use powerful
electromagnets to repel the tracks, raising the train and eliminating
friction."
},
```



```

{
  question: "Where is the magnetic force strongest on a bar magnet?",
  options: [
    { text: "Right in the exact middle of the bar", label: "A" },
    { text: "At both of the terminal poles", label: "B" },
    { text: "Only on the red North pole", label: "C" },
    { text: "Only on the bottom surface", label: "D" }
  ],
  correctIndex: 1,
  explanation: "The concentration of magnetic field lines is densest and
strongest at the designated terminal poles (both North and South)."
```

```

};

function QuizTab({ addXp, unlockBadge }) {
  const [currentIdx, setCurrentIdx] = useState(0);
  const [selectedIdx, setSelectedIdx] = useState(null);
  const [isSubmitted, setIsSubmitted] = useState(false);
  const [score, setScore] = useState(0);
  const [quizCompleted, setQuizCompleted] = useState(false);

  const handleSelect = (idx) => {
    if (isSubmitted) return;
    setSelectedIdx(idx);
    sounds.init();
  };

  const handleSubmit = () => {
    if (selectedIdx === null || isSubmitted) return;
    setIsSubmitted(true);

    const correct = selectedIdx ===
QUIZ_QUESTIONS[currentIdx].correctIndex;
    if (correct) {
      setScore(s => s + 1);
      addXp(10);
    }
  };
}
```

```

        sounds.playClink();
    }
};

const handleNext = () => {
    if (currentIdx < QUIZ_QUESTIONS.length - 1) {
        setCurrentIdx(c => c + 1);
        setSelectedIdx(null);
        setIsSubmitted(false);
    } else {
        setQuizCompleted(true);
        if (score >= 8) {
            unlockBadge('quiz_master', 'Magnetic Genius', `Completed the
playground quiz with an outstanding score of ${score}/10.`);
        }
    }
};

const handleRestart = () => {
    setCurrentIdx(0);
    setSelectedIdx(null);
    setIsSubmitted(false);
    setScore(0);
    setQuizCompleted(false);
};

const progressPercent = Math.round(((currentIdx) / QUIZ_QUESTIONS.length)
* 100);

if (quizCompleted) {
    return (
        <div className="lg:col-span-12 max-w-xl mx-auto w-full bg-slate-900
border border-slate-800 rounded-3xl p-8 text-center shadow-2xl flex
flex-col items-center gap-6">

```

```

    <div className="w-20 h-20 bg-indigo-950 text-indigo-400
rounded-full flex items-center justify-center border-2
border-indigo-500/30 shadow-inner">
      <Trophy className="w-10 h-10 animate-bounce" />
    </div>

    <div>
      <h2 className="text-2xl font-black tracking-tight
text-white">Quiz Completed!</h2>
      <p className="text-sm text-slate-400 mt-1">Excellent effort
exploring the playground.</p>
    </div>

    <div className="bg-slate-950 px-8 py-5 rounded-2xl border
border-slate-800 w-full flex justify-around">
      <div>
        <p className="text-xs text-slate-500">Your Score</p>
        <p className="text-3xl font-black text-indigo-400
font-mono">{score} / 10</p>
      </div>
      <div className="border-l border-slate-800"></div>
      <div>
        <p className="text-xs text-slate-500">XP Earned</p>
        <p className="text-3xl font-black text-emerald-400
font-mono">+{score * 10} XP</p>
      </div>
    </div>

    {score >= 8 ? (
      <div className="bg-emerald-950/40 border border-emerald-500/30
p-4 rounded-xl flex items-center gap-3">
        <Sparkles className="w-5 h-5 text-emerald-400 shrink-0" />
        <p className="text-xs text-emerald-300 font-medium">Amazing
job! You really understand magnetic physics and force fields!</p>
      </div>
    ) : (

```

```
    <p className="text-xs text-slate-500 leading-relaxed px-4">
      Great try! Review the Concept Hub tab anytime to refresh your
memory on magnetic poles, then try again for a perfect score.
```

```
    </p>
  )}

  <button
    onClick={handleRestart}
    className="w-full py-3 bg-indigo-600 hover:bg-indigo-500
text-white rounded-xl font-bold transition shadow-lg"
    >
    Try Quiz Again
  </button>
</div>
);
}
```

```
const q = QUIZ_QUESTIONS[currentIdx];

return (
  <div className="lg:col-span-12 max-w-2xl mx-auto w-full bg-slate-900
border border-slate-800 rounded-3xl shadow-xl overflow-hidden flex
flex-col">

    {/* Quiz Progress header */}
    <div className="bg-slate-950 px-6 py-4 flex items-center
justify-between border-b border-slate-800 text-xs">
      <span className="font-semibold text-slate-400">Questions:
{currentIdx + 1} of {QUIZ_QUESTIONS.length}</span>
      <div className="flex items-center gap-2">
        <span className="text-slate-500">Score: {score}</span>
        <span className="bg-indigo-950 text-indigo-300 border
border-indigo-500/20 px-2.5 py-0.5 rounded-full font-bold">10 XP Per
Correct Answer</span>
      </div>
    </div>
  </div>
```

```

    {/* Progress Bar */}
    <div className="w-full bg-slate-800 h-1.5">
      <div
        style={{ width: `${progressPercent}%` }}
        className="bg-indigo-500 h-full transition-all duration-300"
      ></div>
    </div>

    {/* Question Content */}
    <div className="p-6 sm:p-8 flex flex-col gap-6">
      <h3 className="text-lg sm:text-xl font-bold text-slate-100
leading-snug">
        {q.question}
      </h3>

      {/* Options grid */}
      <div className="grid grid-cols-1 gap-3.5">
        {q.options.map((opt, index) => {
          let btnStyle = "border-slate-800 hover:border-slate-700
bg-slate-950 text-slate-300";

          if (selectedIdx === index) {
            btnStyle = "border-indigo-500 bg-indigo-950/40
text-indigo-100 ring-2 ring-indigo-500/20";
          }

          if (isSubmitted) {
            if (index === q.correctIndex) {
              btnStyle = "border-emerald-500 bg-emerald-950/40
text-emerald-200 font-bold";
            } else if (selectedIdx === index) {
              btnStyle = "border-rose-500 bg-rose-950/40 text-rose-200
line-through";
            } else {

```

```

        btnStyle = "border-slate-800 opacity-50 bg-slate-950
text-slate-500";
    }
}

return (
    <button
        key={index}
        disabled={isSubmitted}
        onClick={() => handleSelect(index)}
        className={`w-full p-4 rounded-xl border flex items-center
gap-4 text-left text-sm transition-all duration-150 ${btnStyle}`}
    >
        <span className={`w-6 h-6 rounded-lg flex items-center
justify-center font-bold text-xs ${selectedIdx === index ? 'bg-indigo-600
text-white' : 'bg-slate-800 text-slate-400'}`}>
            {opt.label}
        </span>
        <span className="flex-1 font-medium">{opt.text}</span>

        {isSubmitted && index === q.correctIndex && (
            <CheckCircle2 className="w-5 h-5 text-emerald-400
shrink-0" />
        )}
        {isSubmitted && selectedIdx === index && index !==
q.correctIndex && (
            <XCircle className="w-5 h-5 text-rose-500 shrink-0" />
        )}
    </button>
);
}}}
</div>

{/* Explanation Alert Box */}
{isSubmitted && (

```

```

        <div className="bg-slate-950 p-4 rounded-2xl border
border-slate-800/80 flex gap-3 text-xs leading-relaxed text-slate-400">
          <Info className="w-5 h-5 text-indigo-400 shrink-0 mt-0.5" />
          <div>
            <p className="font-bold text-slate-300">Science
Explanation:</p>
            <p className="mt-0.5">{q.explanation}</p>
          </div>
        </div>
      )}

    { /* Action button bar */}
    <div className="flex justify-end mt-2">
      {!isSubmitted ? (
        <button
          disabled={selectedIdx === null}
          onClick={handleSubmit}
          className={`px-8 py-3 rounded-xl font-bold text-sm transition
shadow ${selectedIdx !== null ? 'bg-indigo-600 hover:bg-indigo-500
text-white' : 'bg-slate-800 text-slate-500 cursor-not-allowed'}`} `}
        >
          Check Answer
        </button>
      ) : (
        <button
          onClick={handleNext}
          className="px-8 py-3 bg-indigo-600 hover:bg-indigo-500
text-white rounded-xl font-bold text-sm flex items-center gap-1.5
transition shadow"
        >
          {currentIdx < QUIZ_QUESTIONS.length - 1 ? "Next Question" :
"View Results"}
          <ChevronRight className="w-4 h-4" />
        </button>
      )}
    </div>

```

</div>

</div>

);

}

Human Digestive System5–7

```

import React, { useState, useEffect, useMemo, useRef } from 'react';
import {
  Sparkles,
  RotateCcw,
  Info,
  BookOpen,
  Award,
  ArrowRight,
  ArrowLeft,
  Activity,
  Flame,
  ShieldAlert,
  Droplet,
  CheckCircle2,
  HelpCircle,
  ChevronRight,
  Compass,
  Heart,
  Smile,
  Volume2,
  VolumeX,
  Plus
} from 'lucide-react';

// --- SOUND SYNTHESIZER UTILITY ---
// Creates rich retro game/educational sound effects without external audio assets
class SoundFX {
  constructor() {
    this.ctx = null;
    this.muted = false;
  }
  init() {
    if (!this.ctx) {
      this.ctx = new (window.AudioContext || window.webkitAudioContext)();
    }
  }
  playChew() {
    if (this.muted) return;
    this.init();
    const osc = this.ctx.createOscillator();
    const gain = this.ctx.createGain();
    osc.connect(gain);
    gain.connect(this.ctx.destination);
  }
}

```

```

osc.type = 'triangle';
osc.frequency.setValueAtTime(150, this.ctx.currentTime);
osc.frequency.exponentialRampToValueAtTime(40, this.ctx.currentTime + 0.15);

gain.gain.setValueAtTime(0.3, this.ctx.currentTime);
gain.gain.linearRampToValueAtTime(0.01, this.ctx.currentTime + 0.15);

osc.start();
osc.stop(this.ctx.currentTime + 0.15);
}
playSlurp() {
  if (this.muted) return;
  this.init();
  const osc = this.ctx.createOscillator();
  const gain = this.ctx.createGain();
  osc.connect(gain);
  gain.connect(this.ctx.destination);

  osc.type = 'sine';
  osc.frequency.setValueAtTime(200, this.ctx.currentTime);
  osc.frequency.exponentialRampToValueAtTime(600, this.ctx.currentTime + 0.3);

  gain.gain.setValueAtTime(0.2, this.ctx.currentTime);
  gain.gain.linearRampToValueAtTime(0.01, this.ctx.currentTime + 0.3);

  osc.start();
  osc.stop(this.ctx.currentTime + 0.3);
}
playChurn() {
  if (this.muted) return;
  this.init();
  const osc = this.ctx.createOscillator();
  const gain = this.ctx.createGain();
  osc.connect(gain);
  gain.connect(this.ctx.destination);

  osc.type = 'triangle';
  osc.frequency.setValueAtTime(80, this.ctx.currentTime);
  osc.frequency.linearRampToValueAtTime(120, this.ctx.currentTime + 0.2);

  gain.gain.setValueAtTime(0.4, this.ctx.currentTime);
  gain.gain.linearRampToValueAtTime(0.01, this.ctx.currentTime + 0.25);

  osc.start();

```

```

    osc.stop(this.ctx.currentTime + 0.25);
  }
  playAbsorb() {
    if (this.muted) return;
    this.init();
    const osc = this.ctx.createOscillator();
    const gain = this.ctx.createGain();
    osc.connect(gain);
    gain.connect(this.ctx.destination);

    osc.type = 'sine';
    osc.frequency.setValueAtTime(523.25, this.ctx.currentTime); // C5
    osc.frequency.exponentialRampToValueAtTime(1046.50, this.ctx.currentTime + 0.2); // C6

    gain.gain.setValueAtTime(0.15, this.ctx.currentTime);
    gain.gain.linearRampToValueAtTime(0.01, this.ctx.currentTime + 0.2);

    osc.start();
    osc.stop(this.ctx.currentTime + 0.2);
  }
  playSuccess() {
    if (this.muted) return;
    this.init();
    const now = this.ctx.currentTime;
    const playNote = (freq, delay, duration) => {
      const osc = this.ctx.createOscillator();
      const gain = this.ctx.createGain();
      osc.connect(gain);
      gain.connect(this.ctx.destination);
      osc.frequency.setValueAtTime(freq, now + delay);
      gain.gain.setValueAtTime(0.2, now + delay);
      gain.gain.exponentialRampToValueAtTime(0.01, now + delay + duration);
      osc.start(now + delay);
      osc.stop(now + delay + duration);
    };
    playNote(523.25, 0, 0.15); // C5
    playNote(659.25, 0.15, 0.15); // E5
    playNote(783.99, 0.3, 0.15); // G5
    playNote(1046.50, 0.45, 0.4); // C6
  }
}

const sound = new SoundFX();

```

// --- DATA STRUCTURES ---

```
const FOODS = [
  {
    id: 'apple',
    name: 'Apple Slice',
    type: 'Vitamins & Fiber',
    color: '#ef4444',
    bgColor: 'bg-red-500',
    borderColor: 'border-red-600',
    character: '🍏',
    nutrition: 'Vitamins boost immunity, while Fiber sweeps the digestive tract clean!',
    unchewedForm: 'Firm red block',
    chewedForm: 'Soft red pulp (Bolus)',
    stomachForm: 'Acid-melted red syrup',
    absorbedBenefit: '+30 Immunity Sparkles ✨'
  },
  {
    id: 'bread',
    name: 'Bread Roll',
    type: 'Carbohydrates',
    color: '#f59e0b',
    bgColor: 'bg-amber-500',
    borderColor: 'border-amber-600',
    character: '🍞',
    nutrition: 'Carbohydrates are your body\'s primary power fuel source!',
    unchewedForm: 'Dry golden square',
    chewedForm: 'Sticky starch mash (Bolus)',
    stomachForm: 'Semi-liquid starch soup',
    absorbedBenefit: '+30 Muscle Energy ⚡'
  },
  {
    id: 'cheese',
    name: 'Cheese Cube',
    type: 'Fats & Proteins',
    color: '#eab308',
    bgColor: 'bg-yellow-500',
    borderColor: 'border-yellow-600',
    character: '🧀',
    nutrition: 'Proteins build muscles, and Fats insulate nerves and store energy!',
    unchewedForm: 'Solid yellow cube',
    chewedForm: 'Creamy emulsified ball (Bolus)',
    stomachForm: 'Liquid fat emulsion',
    absorbedBenefit: '+30 Strength Points 💪'
  }
]
```

```
}  
];
```

```
const STAGES = {  
  PLATE: {  
    id: 'PLATE',  
    title: 'The Feast Begins',  
    organ: 'Plate',  
    instruction: 'Drag your favorite snack to the mouth to begin the journey!',  
    description: 'Food is rich in nutrients, but your body cannot use them in solid form. We must  
break it down!',  
    progress: 0,  
    xpReward: 10  
  },  
  MOUTH: {  
    id: 'MOUTH',  
    title: 'Mouth & Saliva',  
    organ: 'Mouth',  
    instruction: 'Tap "CHEW" or slide to grind the food with your teeth!',  
    description: 'Teeth perform mechanical digestion by cutting and grinding. Salivary glands  
release saliva (with amylase enzymes) to wet food and begin chemical breakdown.',  
    progress: 15,  
    xpReward: 15  
  },  
  OESOPHAGUS: {  
    id: 'OESOPHAGUS',  
    title: 'Oesophagus (Food Pipe)',  
    organ: 'Oesophagus',  
    instruction: 'Use the slider to trigger Peristalsis waves and push food down!',  
    description: 'The food pipe uses rhythmic, wave-like muscle contractions called peristalsis to  
squeeze the food ball (bolus) down to the stomach. Gravity isn\'t even required!',  
    progress: 30,  
    xpReward: 15  
  },  
  STOMACH: {  
    id: 'STOMACH',  
    title: 'The Stomach Acid Bath',  
    organ: 'Stomach',  
    instruction: 'Tap "CHURN" to squeeze muscle walls and mix hydrochloric acid!',  
    description: 'The stomach is a muscular bag. It churns food physically while mixing in strong  
hydrochloric acid (pH 1.5-2.0) and pepsin enzymes to melt proteins into a creamy liquid called  
chyme.',  
    progress: 50,  
    xpReward: 20  
  }  
}
```

```

},
SMALL_INT: {
  id: 'SMALL_INT',
  title: 'Small Intestine Absorption',
  organ: 'Small Intestine',
  instruction: 'Tap the absorbing villi to collect nutrients into the blood vessels!',
  description: 'At nearly 6 meters long, this narrow, folded tube is where chemical digestion finishes. Helpful secretions from the Liver (bile) and Pancreas (pancreatic juices) break down foods, while tiny finger-like villi absorb nutrients directly into your blood!',
  progress: 80,
  xpReward: 25
},
LARGE_INT: {
  id: 'LARGE_INT',
  title: 'Large Intestine Water Saver',
  organ: 'Large Intestine',
  instruction: 'Hold the "Absorb Water" pump to extract water from undigested waste!',
  description: 'The large intestine is wider but much shorter. It absorbs remaining water and crucial minerals from undigested fibers, turning the soup-like waste into solid stool.',
  progress: 95,
  xpReward: 15
},
COMPLETE: {
  id: 'COMPLETE',
  title: 'The Waste Exit',
  organ: 'Anus',
  instruction: 'Digestion complete! Expel the safe waste and celebrate!',
  description: 'The remaining solid waste is stored in the rectum and expelled through the anus. Your body has successfully extracted all the power, vitamins, and energy from your meal!',
  progress: 100,
  xpReward: 20
}
};

const QUIZ_QUESTIONS = [
  {
    question: "Where does the process of digestion actually begin?",
    options: ["Stomach", "Mouth", "Small Intestine", "Oesophagus"],
    correct: 1,
    explanation: "Digestion starts immediately in the mouth! Teeth grind the food, and saliva starts breaking down starch chemically."
  },
  {

```

question: "What is the name of the wave-like muscle movement that pushes food down the Oesophagus?",
options: ["Gravitational Drop", "Peristalsis", "Stomach Churning", "Capillary Action"],
correct: 1,
explanation: "Peristalsis is the rhythmic contraction and relaxation of muscles in the food pipe walls that pushes food downwards."

},

{

question: "Why doesn't strong stomach acid digest the stomach itself?",
options: [
"The stomach acid is not very strong",
"The stomach is made of bone",
"It has a thick protective lining of mucus that is constantly renewed",
"Food neutralizes all acid instantly"
],
correct: 2,
explanation: "The stomach has a thick layer of slimy mucus protecting its inner walls. This lining is replaced constantly to prevent self-digestion!"

},

{

question: "Which of the following is an accessory gland where food NEVER actually enters?",
options: ["Stomach", "Oesophagus", "Liver", "Small Intestine"],
correct: 2,
explanation: "Food never travels through the Liver or Pancreas. Instead, they squirt vital chemicals (like Bile) into the small intestine through small ducts."

},

{

question: "What are the tiny, finger-like structures lining the small intestine that absorb nutrients?",
options: ["Villi", "Capillaries", "Fibers", "Bile Ducts"],
correct: 0,
explanation: "Villi are microscopic finger-like projections that increase the surface area of the small intestine, boosting nutrient absorption immensely."

},

{

question: "What is the primary job of the Large Intestine?",
options: ["Digest large proteins", "Absorb water and salts from remaining waste", "Store oxygen for the blood", "Produce saliva"],
correct: 1,
explanation: "The large intestine absorbs water and essential salts from leftover undigested matter, converting fluid waste into solid stool."

},

{

question: "If you swallow a whole grape without chewing it, what happens?",


```

options: [
  "It gets stuck forever",
  "The oesophagus must perform much stronger peristalsis to squeeze it down",
  "It is digested easily in 5 seconds",
  "It goes to your lungs instead"
],
correct: 1,
explanation: "Unchewed food causes discomfort because the food pipe muscles have to work
extra hard (stronger peristalsis) to push the massive bolus down."
},
{
  question: "How long is the small intestine of an adult human roughly?",
  options: ["1 meter", "3 meters", "6 meters", "12 meters"],
  correct: 2,
  explanation: "The small intestine is about 6 meters (20 feet) long! It is packed tightly into
loops to fit inside your abdomen."
},
{
  question: "What fluid is created by the liver and stored in the gall bladder to help digest fats?",
  options: ["Saliva", "Gastric Acid", "Pancreatic Juice", "Bile"],
  correct: 3,
  explanation: "Bile is a yellow-green liquid made by the liver. It acts like dish soap, emulsifying
big fat droplets into tiny ones so enzymes can digest them."
},
{
  question: "How does fiber help our digestive system?",
  options: [
    "It dissolves into high sugar reserves",
    "It acts like a broom, keeping waste moving smoothly through the intestines",
    "It builds bones",
    "It does absolutely nothing"
  ],
  correct: 1,
  explanation: "Fiber cannot be digested, but it adds bulk to our stool, acting like a natural
broom that sweeps waste smoothly through our intestines to prevent constipation."
}
};

```

```

export default function App() {
  // Navigation & Core States
  const [activeTab, setActiveTab] = useState('simulator'); // 'simulator', 'quiz', 'diy', 'concepts'
  const [soundMuted, setSoundMuted] = useState(false);

  // Simulation States

```

```

const [currentStageKey, setCurrentStageKey] = useState('PLATE');
const [selectedFood, setSelectedFood] = useState(null);
const [xp, setXp] = useState(0);
const [unlockedLogs, setUnlockedLogs] = useState({
  PLATE: true,
  MOUTH: false,
  OESOPHAGUS: false,
  STOMACH: false,
  SMALL_INT: false,
  LARGE_INT: false,
  COMPLETE: false,
});

```

// Stage-specific Interactive states

```

const [chewsCount, setChewsCount] = useState(0);
const [chewScale, setChewScale] = useState(1);
const [peristalsisPos, setPeristalsisPos] = useState(0); // 0 to 100
const [stomachAcidCount, setStomachAcidCount] = useState(0);
const [intestinalAbsorption, setIntestinalAbsorption] = useState(0); // number of tapped villi
(max 5)
const [villiStates, setVilliStates] = useState([
  { id: 1, active: true, nutrient: 'Vitamins', style: { top: '35%', left: '42%' } },
  { id: 2, active: true, nutrient: 'Glucose', style: { top: '48%', left: '45%' } },
  { id: 3, active: true, nutrient: 'Proteins', style: { top: '60%', left: '40%' } },
  { id: 4, active: true, nutrient: 'Fats', style: { top: '55%', left: '55%' } },
  { id: 5, active: true, nutrient: 'Minerals', style: { top: '40%', left: '50%' } }
]);
const [waterPumpValue, setWaterPumpValue] = useState(0); // 0 to 100%
const [particles, setParticles] = useState([]);

```

// Quiz States

```

const [quizAnswers, setQuizAnswers] = useState(Array(QUIZ_QUESTIONS.length).fill(null));
const [quizSubmitted, setQuizSubmitted] = useState(false);
const [currentQuizQuestion, setCurrentQuizQuestion] = useState(0);

```

// Toggle Mute

```

const toggleMute = () => {
  sound.muted = !soundMuted;
  setSoundMuted(!soundMuted);
};

```

// Drag & Drop Simulation Helpers for Mouse/Touch

```

const handleFoodSelect = (food) => {
  setSelectedFood(food);

```

```
    sound.playSlurp();
    transitionStage('MOUTH');
};
```

```
const transitionStage = (nextStage) => {
    setCurrentStageKey(nextStage);
    setUnlockedLogs(prev => ({ ...prev, [nextStage]: true }));
    setXp(prev => prev + STAGES[nextStage].xpReward);
    sound.playSuccess();
}
```

```
// Add glowing particle on transition
triggerParticle(150, 200, '✨', 5);
};
```

```
const triggerParticle = (x, y, emoji, count = 3) => {
    const newParticles = [];
    for (let i = 0; i < count; i++) {
        newParticles.push({
            id: Math.random(),
            x: x + (Math.random() - 0.5) * 50,
            y: y + (Math.random() - 0.5) * 50,
            emoji: emoji,
            vx: (Math.random() - 0.5) * 6,
            vy: -Math.random() * 5 - 2,
            life: 1.0
        });
    }
    setParticles(prev => [...prev, ...newParticles]);
};
```

```
// Particle update animation frame emulation
useEffect(() => {
    if (particles.length === 0) return;
    const interval = setInterval(() => {
        setParticles(prev =>
            prev
                .map(p => ({
                    ...p,
                    x: p.x + p.vx,
                    y: p.y + p.vy,
                    vy: p.vy + 0.2, // gravity
                    life: p.life - 0.05
                }))
                .filter(p => p.life > 0)
    });
});
```

```

    );
  }, 30);
  return () => clearInterval(interval);
}, [particles]);

```

// Stage Handlers

```

const handleChew = () => {
  if (chewsCount < 10) {
    sound.playChew();
    setChewsCount(prev => prev + 1);
    setChewScale(1.3);
    setTimeout(() => setChewScale(1), 100);
    triggerParticle(150, 100, '🦷', 2);

    if (chewsCount + 1 === 10) {
      setTimeout(() => transitionStage('OESOPHAGUS'), 800);
    }
  }
};

```

```

const handlePeristalsisSlider = (e) => {
  const value = parseInt(e.target.value);
  setPeristalsisPos(value);
  if (value % 20 === 0) {
    sound.playChew();
  }
  if (value >= 100) {
    transitionStage('STOMACH');
  }
};

```

```

const handleChurnStomach = () => {
  if (stomachAcidCount < 8) {
    sound.playChurn();
    setStomachAcidCount(prev => prev + 1);
    triggerParticle(170, 220, '🥴', 2);
    if (stomachAcidCount + 1 === 8) {
      setTimeout(() => transitionStage('SMALL_INT'), 1000);
    }
  }
};

```

```

const handleVilliTap = (villiId, nutrient, e) => {
  sound.playAbsorb();

```

```

setVilliStates(prev => prev.map(v => v.id === villid ? { ...v, active: false } : v));
setIntestinalAbsorption(prev => {
  const newVal = prev + 1;
  if (newVal === 5) {
    setTimeout(() => transitionStage('LARGE_INT'), 1200);
  }
  return newVal;
});
// Particle feedback
const rect = e.target.getBoundingClientRect();
triggerParticle(rect.left - 50, rect.top - 50, '✨', 4);
triggerParticle(rect.left - 50, rect.top - 50, '🩸', 2);
};

```

```

const handleWaterPump = (e) => {
  const val = parseInt(e.target.value);
  setWaterPumpValue(val);
  if (val % 25 === 0) {
    sound.playSlurp();
  }
  if (val >= 100) {
    transitionStage('COMPLETE');
  }
};

```

// Reset Everything

```

const handleReset = () => {
  setCurrentStageKey('PLATE');
  setSelectedFood(null);
  setChewsCount(0);
  setPeristalsisPos(0);
  setStomachAcidCount(0);
  setIntestinalAbsorption(0);
  setWaterPumpValue(0);
  setVilliStates([
    { id: 1, active: true, nutrient: 'Vitamins', style: { top: '35%', left: '42%' } },
    { id: 2, active: true, nutrient: 'Glucose', style: { top: '48%', left: '45%' } },
    { id: 3, active: true, nutrient: 'Proteins', style: { top: '60%', left: '40%' } },
    { id: 4, active: true, nutrient: 'Fats', style: { top: '55%', left: '55%' } },
    { id: 5, active: true, nutrient: 'Minerals', style: { top: '40%', left: '50%' } }
  ]);
  sound.playSuccess();
};

```

```

// Quiz Score Calculator
const quizScore = useMemo(() => {
  return quizAnswers.reduce((score, ans, idx) => {
    return ans === QUIZ_QUESTIONS[idx].correct ? score + 10 : score;
  }, 0);
}, [quizAnswers]);

const handleAnswerSelect = (optionIdx) => {
  if (quizSubmitted) return;
  const updated = [...quizAnswers];
  updated[currentQuizQuestion] = optionIdx;
  setQuizAnswers(updated);
  sound.playChew();
};

// Coordinates for the moving food icon inside the SVG
const foodCoordinates = useMemo(() => {
  switch (currentStageKey) {
    case 'PLATE':
      return { cx: 75, cy: 380, visible: true };
    case 'MOUTH':
      return { cx: 150, cy: 65, visible: true };
    case 'OESOPHAGUS':
      // Map 0-100% slider to Y: 80 to 180
      return { cx: 150, cy: 80 + (peristalsisPos / 100) * 100, visible: true };
    case 'STOMACH':
      return { cx: 175, cy: 215, visible: true };
    case 'SMALL_INT':
      return { cx: 155, cy: 300, visible: true };
    case 'LARGE_INT':
      return { cx: 145, cy: 290, visible: true };
    case 'COMPLETE':
      return { cx: 150, cy: 400, visible: true };
    default:
      return { cx: 75, cy: 380, visible: false };
  }
}, [currentStageKey, peristalsisPos]);

return (
  <div className="min-h-screen bg-slate-950 text-slate-100 flex flex-col font-sans
selection:bg-amber-500 selection:text-slate-900 overflow-x-hidden">
    {/* Floating Particles Overlay */}
    {particles.map(p => (
      <span

```

```

      key={p.id}
      className="absolute pointer-events-none text-2xl transition-all duration-75 z-50
select-none"
      style={{
        left: p.x,
        top: p.y,
        opacity: p.life,
        transform: `scale(${p.life * 1.5})`
      }}
    >
      {p.emoji}
    </span>
  )}}

  { /* Header HUD */
    <header className="border-b border-slate-800 bg-slate-900/80 backdrop-blur-md sticky
top-0 z-30 px-4 py-3 shadow-lg">
      <div className="max-w-7xl mx-auto flex flex-col sm:flex-row justify-between items-center
gap-4">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-amber-500/10 rounded-xl border border-amber-500/30
text-amber-400 shadow-inner">
            <Activity className="w-6 h-6 animate-pulse" />
          </div>
          <div>
            <h1 className="text-xl font-bold tracking-tight bg-gradient-to-r from-amber-400
via-orange-300 to-amber-500 bg-clip-text text-transparent">
              The Great Food Journey
            </h1>
            <p className="text-xs text-slate-400">Class 5–7 Science • NCERT Interactive
Lab</p>
          </div>
        </div>
      </div>

    { /* Navigation Tabs */
      <div className="flex bg-slate-950 p-1 rounded-xl border border-slate-800">
        <button
          onClick={() => { setActiveTab('simulator'); sound.playChew(); }}
          className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm font-medium transition-all ${
            activeTab === 'simulator'
              ? 'bg-amber-500 text-slate-950 shadow-md font-bold'
              : 'text-slate-400 hover:text-slate-200'
            }`
        >

```

 Interactive Lab

</button>

<button

```
onClick={() => { setActiveTab('concepts'); sound.playChew(); }}
className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm font-medium transition-all ${
  activeTab === 'concepts'
    ? 'bg-amber-500 text-slate-950 shadow-md font-bold'
    : 'text-slate-400 hover:text-slate-200'
}`}
}}
>
```

 Science Book

</button>

<button

```
onClick={() => { setActiveTab('quiz'); sound.playChew(); }}
className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm font-medium transition-all ${
  activeTab === 'quiz'
    ? 'bg-amber-500 text-slate-950 shadow-md font-bold'
    : 'text-slate-400 hover:text-slate-200'
}`}
}}
>
```

 Knowledge Quiz

</button>

<button

```
onClick={() => { setActiveTab('diy'); sound.playChew(); }}
className={`px-4 py-1.5 rounded-lg text-xs sm:text-sm font-medium transition-all ${
  activeTab === 'diy'
    ? 'bg-amber-500 text-slate-950 shadow-md font-bold'
    : 'text-slate-400 hover:text-slate-200'
}`}
}}
>
```

 DIY Activity

</button>

</div>

{/* XP & Sound Panel */}

<div className="flex items-center gap-4">

<div className="flex items-center gap-2 bg-slate-950/60 border border-slate-800
px-3.5 py-1.5 rounded-xl">

<Award className="w-5 h-5 text-amber-400 animate-bounce" />

<div className="text-right">

<div className="text-[10px] text-slate-400 leading-none">ENERGY GAINED</div>

<div className="text-sm font-black text-amber-400">{xp} XP</div>

</div>

</div>


```

        <button
            onClick={toggleMute}
            className="p-2 rounded-xl bg-slate-800 hover:bg-slate-700 border
border-slate-700/50 text-slate-300 transition-all"
            title={soundMuted ? "Unmute" : "Mute"}
        >
            {soundMuted ? <VolumeX className="w-5 h-5" /> : <Volume2 className="w-5 h-5
text-amber-400" />}
        </button>
    </div>
</div>
</header>

{/* Main Container */}
<main className="flex-1 max-w-7xl w-full mx-auto p-4 flex flex-col gap-6">

    {/* TAB 1: THE INTERACTIVE SIMULATOR */}
    {activeTab === 'simulator' && (
        <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 items-stretch">

            {/* Visual Canvas Panel (Left 70%) */}
            <div className="lg:col-span-7 bg-slate-900 border border-slate-800 rounded-2xl
overflow-hidden shadow-2xl flex flex-col min-h-[580px] relative">

                {/* Simulator Header */}
                <div className="bg-slate-900/60 border-b border-slate-800 px-5 py-3.5 flex
justify-between items-center">
                    <div className="flex items-center gap-2">
                        <span className="inline-block w-2.5 h-2.5 bg-green-500 rounded-full animate-ping"
/>
                        <span className="text-xs uppercase tracking-wider font-bold text-slate-400">Live
Digestion Pipeline</span>
                    </div>
                    <div className="text-xs text-slate-400">
                        Selected Food: {selectedFood ? (
                            <span className="font-semibold text-amber-400">{selectedFood.character}
{selectedFood.name}</span>
                        ) : (
                            <span className="italic text-slate-500">None</span>
                        )}
                    </div>
                </div>
            </div>

```

```

    { /* Central Area: Interactive Human Canvas */ }
    <div className="flex-1 flex flex-col md:flex-row items-center justify-around p-6 gap-6
relative">

    { /* SVG Human Anatomic Map */ }
    <div className="w-full max-w-[320px] h-[450px] relative">
    <svg
      viewBox="0 0 300 450"
      className="w-full h-full drop-shadow-[0_10px_30px_rgba(245,158,11,0.05)]"
    >
    { /* Glowing Filter Definitions */ }
    <defs>
      <filter id="glow" x="-20%" y="-20%" width="140%" height="140%">
        <feGaussianBlur stdDeviation="8" result="blur" />
        <feComposite in="SourceGraphic" in2="blur" operator="over" />
      </filter>
      <filter id="organ-pulse" x="-10%" y="-10%" width="120%" height="120%">
        <feGaussianBlur stdDeviation="3" result="blur" />
        <feComposite in="SourceGraphic" in2="blur" operator="over" />
      </filter>
    </defs>

    { /* Semi-transparent Human Silhouette Outline */ }
    <path
      d="M150,15 C190,15 210,35 210,65 C210,85 195,100 195,115 C195,125 240,140
245,180 C250,220 230,290 230,360 C230,410 200,440 180,440 C170,440 165,420 150,420
C135,420 130,440 120,440 C100,440 70,410 70,360 C70,290 50,220 55,180 C60,140 105,125
105,115 C105,100 90,85 90,65 C90,35 110,15 150,15 Z"
      fill="#1e293b"
      fillOpacity="0.4"
      stroke="#475569"
      strokeWidth="2.5"
      strokeDasharray="4,4"
    />

    { /* MOUTH WAYPOINT */ }
    <g className="cursor-pointer">
    <circle
      cx="150" cy="65" r="14"
      fill={currentStageKey === 'MOUTH' ? '#fbbf24' : '#334155'}
      fillOpacity={currentStageKey === 'MOUTH' ? '0.2' : '0.4'}
      stroke={currentStageKey === 'MOUTH' ? '#fbbf24' : '#64748b'}
      strokeWidth={currentStageKey === 'MOUTH' ? '3' : '2'}
      filter={currentStageKey === 'MOUTH' ? 'url(#organ-pulse)' : ''}

```

```

        className={currentStageKey === 'MOUTH' ? 'animate-pulse' : ''}
      />
      <text x="150" y="69" fontSize="10" fontWeight="bold" textAnchor="middle"
fill="#f8fafc">M</text>
    </g>

    { /* SALIVARY GLANDS (Helper Accessory Gland) */}
    <g opacity={currentStageKey === 'MOUTH' ? 1 : 0.4}
className="transition-opacity">
      { /* Submandibular */}
      <path d="M128,73 Q132,78 135,76" stroke="#fb923c" strokeWidth="3" fill="none"
/>

      <circle cx="127" cy="72" r="4" fill="#fb923c" />
      <text x="110" y="75" fontSize="8" fill="#cbd5e1" textAnchor="end">Salivary
Gland</text>
    </g>

    { /* OESOPHAGUS PATHWAY */}
    <path
      d="M150,79 L150,175"
      stroke={currentStageKey === 'OESOPHAGUS' ? '#f59e0b' : '#475569'}
      strokeWidth={currentStageKey === 'OESOPHAGUS' ? '7' : '4'}
      strokeLinecap="round"
    />

    { /* STOMACH (J-shaped bag) */}
    <path
      d="M150,175 C165,175 190,180 195,200 C200,225 185,245 155,245 C140,245
135,230 135,215 C135,190 145,180 150,175 Z"
      fill={currentStageKey === 'STOMACH' ? '#ef4444' : '#334155'}
      fillOpacity={currentStageKey === 'STOMACH' ? '0.25' : '0.4'}
      stroke={currentStageKey === 'STOMACH' ? '#ef4444' : '#64748b'}
      strokeWidth={currentStageKey === 'STOMACH' ? '3.5' : '2'}
      filter={currentStageKey === 'STOMACH' ? 'url(#glow)' : ''}
    />

    { /* ACCESSORY GLANDS: LIVER & PANCREAS */}
    { /* Liver (Large wedge-like organ on left/anatomical right) */}
    <path
      d="M110,180 L145,185 L135,205 L105,200 Z"
      fill="#7f1d1d"
      stroke="#ef4444"
      strokeWidth="1.5"

```

```

        opacity={currentStageKey === 'STOMACH' || currentStageKey === 'SMALL_INT'
? '0.9' : '0.4'}
    />
    <text x="95" y="195" fontSize="8" fill="#fca5a5" textAnchor="end">Liver (Bile
Production)</text>

    {/* Pancreas (hidden behind/below stomach) */}
    <rect
      x="145" y="222" width="25" height="8" rx="3"
      fill="#eab308"
      stroke="#facc15"
      strokeWidth="1"
      opacity={currentStageKey === 'SMALL_INT' ? '1' : '0.3'}
    />
    <text x="195" y="235" fontSize="8" fill="#fef08a" textAnchor="start">Pancreas
(Enzymes)</text>

    {/* SMALL INTESTINE (Highly folded tube in center) */}
    <g filter={currentStageKey === 'SMALL_INT' ? 'url(#glow)' : ''}>
      <path
        d="M150,245 C145,260 170,255 170,270 C170,285 140,280 140,295 C140,310
170,305 170,320 C170,335 145,330 145,345 C145,350 155,350 155,345"
        fill="none"
        stroke={currentStageKey === 'SMALL_INT' ? '#22c55e' : '#475569'}
        strokeWidth={currentStageKey === 'SMALL_INT' ? '9' : '5'}
        strokeLinecap="round"
        strokeLinejoin="round"
      />
    </g>

    {/* LARGE INTESTINE (Frames the small intestine) */}
    <path
      d="M135,350 L130,265 L175,265 L175,345 L150,350 L150,385"
      fill="none"
      stroke={currentStageKey === 'LARGE_INT' ? '#3b82f6' : '#475569'}
      strokeWidth={currentStageKey === 'LARGE_INT' ? '14' : '8'}
      strokeLinecap="square"
      strokeLinejoin="miter"
      opacity="0.8"
      filter={currentStageKey === 'LARGE_INT' ? 'url(#glow)' : ''}
    />

    {/* RECTUM / ANUS EXITWAY */}
    <line

```

```

x1="150" y1="385" x2="150" y2="415"
stroke={currentStageKey === 'COMPLETE' ? '#f43f5e' : '#64748b'}
strokeWidth="6"
/>

{/* MOVING FOOD BALOON IN TRANSIT */}
{selectedFood && foodCoordinates.visible && (
  <g transform={`translate(${foodCoordinates.cx}, ${foodCoordinates.cy})`}>
    <circle
      r="16"
      fill={selectedFood.color}
      fillOpacity="0.15"
      stroke={selectedFood.color}
      strokeWidth="2"
      className="animate-ping"
    />
    <circle r="12" fill={selectedFood.color} className="shadow-lg" />
    <text y="4" textAnchor="middle" fontSize="12">{selectedFood.character}</text>
  </g>
)}
</svg>

```

```

{/* Organ Tooltip Badges on Map */}
<span className={`absolute top-[40px] left-[5%] bg-slate-950/90 text-amber-400
border border-amber-500/30 px-2 py-0.5 rounded text-[10px] uppercase font-black transition-all
${currentStageKey === 'MOUTH' ? 'scale-110 border-amber-400' : 'opacity-40'}`}>1.
Mouth</span>

<span className={`absolute top-[110px] left-[5%] bg-slate-950/90 text-amber-400
border border-amber-500/30 px-2 py-0.5 rounded text-[10px] uppercase font-black transition-all
${currentStageKey === 'OESOPHAGUS' ? 'scale-110 border-amber-400' : 'opacity-40'}`}>2.
Food Pipe</span>

<span className={`absolute top-[180px] right-[5%] bg-slate-950/90 text-amber-400
border border-amber-500/30 px-2 py-0.5 rounded text-[10px] uppercase font-black transition-all
${currentStageKey === 'STOMACH' ? 'scale-110 border-amber-400' : 'opacity-40'}`}>3.
Stomach</span>

<span className={`absolute top-[280px] right-[5%] bg-slate-950/90 text-amber-400
border border-amber-500/30 px-2 py-0.5 rounded text-[10px] uppercase font-black transition-all
${currentStageKey === 'SMALL_INT' ? 'scale-110 border-amber-400' : 'opacity-40'}`}>4. Small
Intestine</span>

<span className={`absolute top-[350px] left-[5%] bg-slate-950/90 text-amber-400
border border-amber-500/30 px-2 py-0.5 rounded text-[10px] uppercase font-black transition-all
${currentStageKey === 'LARGE_INT' ? 'scale-110 border-amber-400' : 'opacity-40'}`}>5. Large
Intestine</span>
</div>

```

```

    { /* Microscopic Close-up View (Dynamic per stage) */
    <div className="flex-1 w-full bg-slate-950/80 border border-slate-800 rounded-xl p-5
relative overflow-hidden flex flex-col justify-between min-h-[340px] max-w-[340px]">

    { /* Top Header of Microscope */
    <div className="flex justify-between items-center border-b border-slate-800 pb-2
mb-3">
        <span className="text-[11px] uppercase tracking-wider text-amber-500
font-extrabold flex items-center gap-1">
            <Compass className="w-3.5 h-3.5 animate-spin" /> Microscope Zoom
        </span>
        <span className="text-xs text-slate-400">Organ:
{STAGES[currentStageKey].organ}</span>
    </div>

    { /* Micro-Animation Canvas depending on State */
    <div className="flex-1 flex flex-col justify-center items-center relative py-4">

    { /* STAGE: PLATE (Pick food) */
    {currentStageKey === 'PLATE' && (
    <div className="text-center">
        <div className="flex justify-center gap-4 mb-4">
            {FOODS.map(food => (
                <button
                    key={food.id}
                    onClick={() => handleFoodSelect(food)}
                    className="w-16 h-16 rounded-2xl border-2 border-dashed
border-slate-700 hover:border-amber-400 bg-slate-900/60 flex flex-col items-center
justify-center transition-all transform hover:scale-110 active:scale-95 group relative"
                >
                    <span className="text-3xl
group-hover:animate-bounce">{food.character}</span>
                    <span className="text-[9px] font-bold text-slate-400
group-hover:text-amber-400 mt-1">{food.name}</span>
                </button>
            ))}
        </div>
        <p className="text-xs text-slate-400 italic">Select one of these delicious meals
to study its chemical decomposition journey!</p>
    </div>
    ))}

    { /* STAGE: MOUTH (Tooth movement & Chewing) */

```

```

{currentStageKey === 'MOUTH' && selectedFood && (
  <div className="w-full text-center flex flex-col items-center">
    <div className="relative w-28 h-28 bg-slate-900 rounded-full flex items-center
justify-center border-2 border-amber-500/20 overflow-hidden mb-3">
      {/* Top Teeth */}
      <div
        className="absolute top-1 left-0 right-0 flex justify-center gap-1
transition-transform"
        style={{ transform: `translateY(${chewsCount % 2 === 0 ? '0px' : '15px'})` }}
      >
        <span className="text-lg"> 🦷 </span><span
className="text-lg"> 🦷 </span><span className="text-lg"> 🦷 </span>
      </div>

      {/* Active Food Item */}
      <span
        className="text-4xl transition-all"
        style={{ transform: `scale(${chewScale}) rotate(${chewsCount * 15}deg)` }}
      >
        {chewsCount >= 10 ? '🍌' : selectedFood.character}
      </span>

      {/* Bottom Teeth */}
      <div
        className="absolute bottom-1 left-0 right-0 flex justify-center gap-1
transition-transform"
        style={{ transform: `translateY(${chewsCount % 2 === 0 ? '0px' : '-15px'})` }}
      >
        <span className="text-lg"> 🦷 </span><span
className="text-lg"> 🦷 </span><span className="text-lg"> 🦷 </span>
      </div>

      {/* Saliva Spray Animations */}
      {chewsCount > 0 && chewsCount < 10 && (
        <div className="absolute inset-0 flex justify-center items-center
pointer-events-none">
          <span className="animate-ping text-blue-400 text-xs">💧 </span>
          <span className="animate-pulse text-blue-300 text-sm absolute left-3
top-6">💧 </span>
          <span className="animate-pulse text-blue-300 text-sm absolute right-3
bottom-6">💧 </span>
        </div>
      )}
    </div>
  )}

```

```

<div className="text-xs font-semibold text-slate-200">
  {chewsCount >= 10 ? (
    <span className="text-green-400 flex items-center gap-1 justify-center">
      <CheckCircle2 className="w-3.5 h-3.5" /> Food is chewed (Bolus)! Ready
to swallow.
    </span>
  ) : (
    <span>Chews count: <b className="text-amber-400
text-sm">{chewsCount}/10</b></span>
  )}
</div>
</div>
)}

{/* STAGE: OESOPHAGUS (Squeeze pipe animation) */}
{currentStageKey === 'OESOPHAGUS' && selectedFood && (
  <div className="w-full flex items-center justify-center gap-4">
    {/* Interactive Squeezing visualization */}
    <div className="w-16 h-48 bg-slate-900 border-2 border-slate-800 rounded-full
relative overflow-hidden flex flex-col justify-between p-1">
      <div className={`w-full rounded-full transition-all duration-300 ${peristalsisPos
< 33 ? 'h-10 bg-amber-500/20 border-2 border-amber-500 scale-105' : 'h-14 bg-slate-800'}`} />
      <div className={`w-full rounded-full transition-all duration-300 ${peristalsisPos
>= 33 && peristalsisPos < 66 ? 'h-10 bg-amber-500/20 border-2 border-amber-500 scale-105' :
'h-14 bg-slate-800'}`} />
      <div className={`w-full rounded-full transition-all duration-300 ${peristalsisPos
>= 66 ? 'h-10 bg-amber-500/20 border-2 border-amber-500 scale-105' : 'h-14 bg-slate-800'}`} />

      {/* Moving Bolus ball */}
      <div
        className="absolute left-1/2 -translate-x-1/2 w-10 h-10 bg-orange-600
rounded-full flex items-center justify-center text-lg shadow-lg font-bold transition-all
duration-100"
        style={{ top: `${Math.min(90, Math.max(10, peristalsisPos - 10))}%` }}
      >

      </div>
    </div>
    <div>
      <div className="text-left max-w-[150px]">
        <p className="text-[11px] text-slate-400 uppercase font-black tracking-wider
text-amber-500">Peristalsis</p>
        <p className="text-xs text-slate-300 mt-1">Muscles squeeze behind the food
to slide it down like toothpaste!</p>

```



```

    </div>
  </div>
)}

{/* STAGE: STOMACH (Acid Swirl & Expansion) */}
{currentStageKey === 'STOMACH' && selectedFood && (
  <div className="w-full text-center flex flex-col items-center">
    <div className="w-32 h-32 rounded-full border-4 border-dashed
border-red-500/20 flex items-center justify-center relative overflow-hidden mb-3">
      {/* Stomach Liquid background fill */}
      <div
        className="absolute bottom-0 left-0 right-0 bg-red-600/20 transition-all
duration-500"
        style={{ height: `${20 + stomachAcidCount * 10}%` }}
      />

      {/* Churning particles */}
      {stomachAcidCount > 0 && (
        <div className="absolute inset-0 flex items-center justify-center animate-spin
duration-[3000ms] pointer-events-none">
          <span className="text-xs text-green-300 absolute top-2">🥄 (Acid)</span>
          <span className="text-xs text-amber-300 absolute bottom-2">🥄</span>
          <span className="text-[10px] text-slate-300 absolute left-2 font-bold">pH
1.5</span>
        </div>
      )}

      {/* Food Particle changing form */}
      <div className={`text-3xl transition-transform duration-300
${stomachAcidCount >= 4 ? 'scale-75 opacity-70 filter blur-[1px]' : ''}`>
        {stomachAcidCount >= 8 ? '🥣' : '🍌'}
      </div>
    </div>

    <div className="text-xs text-slate-300">
      {stomachAcidCount >= 8 ? (
        <span className="text-green-400 font-semibold flex items-center gap-1
justify-center">
          <CheckCircle2 className="w-3.5 h-3.5" /> Chyme created successfully!
          Safe to advance.
        </span>
      ) : (
        <span>Acid mixtures churned: <b className="text-red-400
text-sm">{stomachAcidCount}/8</b></span>

```

```

    })
  </div>
</div>
})

{/* STAGE: SMALL INTESTINE (Tap to absorb nutrient micro-game) */}
{currentStageKey === 'SMALL_INT' && selectedFood && (
  <div className="w-full h-full relative">
    <div className="text-center mb-3">
      <p className="text-[10px] uppercase font-black text-green-500
tracking-wider">Absorption Active</p>
      <p className="text-xs text-slate-300 font-medium">Tap the floating villi loops
to absorb nutrition into the red blood vessels!</p>
    </div>

    {/* Micro-canvas containing Villi and nutrients */}
    <div className="w-full h-40 bg-slate-900 rounded-xl relative border
border-slate-800 overflow-hidden">
      {/* Blood Capillary pipeline running at bottom */}
      <div className="absolute bottom-0 left-0 right-0 h-8 bg-red-600/30 border-t
border-red-500 flex items-center justify-around overflow-hidden">
        <span className="text-[10px] font-bold text-red-100 flex items-center
gap-1"> 🩸 Red Blood Flow <Activity className="w-3 h-3 text-red-400 animate-pulse"
/></span>

        <div className="flex gap-2 text-[10px] animate-pulse">
          {villiStates.filter(v => !v.active).map((v, i) => (
            <span key={i} className="bg-amber-400 text-slate-950 font-black px-1.5
py-0.5 rounded text-[8px] animate-bounce">{v.nutrient}</span>
          ))}
        </div>
      </div>

      {/* Villi loops projecting downwards */}
      {villiStates.map(v => (
        <button
          key={v.id}
          onClick={(e) => v.active && handleVilliTap(v.id, v.nutrient, e)}
          style={v.style}
          disabled={!v.active}
          className={`absolute p-1.5 rounded-full border flex flex-col items-center
justify-center transition-all transform hover:scale-115 active:scale-90 ${
            v.active
              ? 'bg-emerald-500/10 border-emerald-500 hover:border-amber-400
text-emerald-400 animate-bounce'

```

```

        : 'bg-slate-800 border-slate-700 text-slate-600 cursor-not-allowed scale-90'
      }}
    >
      <span className="text-xs font-bold">{v.active ? '🌿' : '✨'}</span>
      <span className="text-[8px] font-black uppercase
mt-0.5">{v.nutrient}</span>
    </button>
  )}}
</div>

<div className="text-center mt-3 text-xs">
  Absorbed: <span className="text-green-400 font-extrabold
text-sm">{intestinalAbsorption}/5</span> essentials
</div>
</div>
)}

{/* STAGE: LARGE INTESTINE (Water Pump Game) */}
{currentStageKey === 'LARGE_INT' && selectedFood && (
  <div className="w-full text-center flex flex-col items-center">
    <div className="relative w-28 h-28 bg-slate-900 rounded-full flex items-center
justify-center border-4 border-dashed border-blue-500/20 mb-3 overflow-hidden">
      {/* Squeezing pulp rendering */}
      <div className="text-4xl z-10 transition-all duration-300" style={{ transform:
`scale(${1 - (waterPumpValue / 200)})` }}>
        🍌
      </div>

      {/* Extracted Water droplets animation */}
      {waterPumpValue > 10 && waterPumpValue < 95 && (
        <div className="absolute inset-0 pointer-events-none">
          <span className="absolute text-blue-400 text-lg animate-ping left-4
top-10">💧 </span>
          <span className="absolute text-blue-300 text-md animate-ping right-4
bottom-10">💧 </span>
        </div>
      )}

      {/* Blue water background filling up elsewhere */}
      <div
        className="absolute bottom-0 left-0 right-0 bg-blue-600/30 transition-all"
        style={{ height: `${waterPumpValue}%` }}
      />
    </div>
  )}

```

```

    <div className="text-xs text-slate-300 w-full max-w-[200px]">
      <label className="block text-[10px] uppercase font-bold text-blue-400
mb-1">Water Extracted</label>
      <input
        type="range"
        min="0"
        max="100"
        value={waterPumpValue}
        onChange={handleWaterPump}
        className="w-full h-2 bg-slate-800 rounded-lg appearance-none
cursor-pointer accent-blue-500"
      />
      <div className="flex justify-between text-[10px] text-slate-500 mt-1">
        <span>Soggy</span>
        <span className="text-blue-400 font-bold">{waterPumpValue}% Dry</span>
        <span>Solid Stool</span>
      </div>
    </div>
  </div>
)}

```

```

{ /* STAGE: COMPLETE (Celebration) */
  {currentStageKey === 'COMPLETE' && (
    <div className="text-center flex flex-col items-center justify-center py-4">
      <div className="w-20 h-20 rounded-full bg-emerald-500/20 border-2
border-emerald-500 flex items-center justify-center mb-3 animate-bounce">
        <CheckCircle2 className="w-12 h-12 text-emerald-400" />
      </div>
      <h4 className="text-base font-bold text-emerald-400">Mission
Accomplished!</h4>
      <p className="text-xs text-slate-300 px-4 mt-1">The body successfully
unlocked nutrients & expelled safe solid waste!</p>

```

```

      {selectedFood && (
        <div className="mt-3 p-2 bg-slate-900 border border-slate-800 rounded-lg
text-[11px] text-left">
          <span className="font-bold text-amber-400">Absorption Report
({selectedFood.character} {selectedFood.name}):</span>
          <p className="text-slate-400 mt-0.5">{selectedFood.nutrition}</p>
          <p className="text-green-400 font-black mt-1">Benefit:
{selectedFood.absorbedBenefit}</p>
        </div>
      )}

```

```

    </div>
  })

</div>

  { /* Microscopic Description box */
    <div className="bg-slate-900/90 border border-slate-800 rounded-lg p-2.5
text-[11px] text-slate-300">
      <span className="font-bold text-amber-500 flex items-center gap-1">
        <Info className="w-3 h-3" /> Focus Description
      </span>
      <p className="mt-0.5 text-slate-400 leading-relaxed text-xs">
        {currentStageKey === 'PLATE' && "Food starts as rich solids. Our teeth and
digestive juices will reduce them into tiny cells."}
        {currentStageKey === 'MOUTH' && `Mouth turns ${selectedFood?.name || 'food'}
from a "${selectedFood?.unchewedForm || 'solid block'}" into "${selectedFood?.chewedForm ||
'bolus'}" using physical force and salivary enzymes.`}
        {currentStageKey === 'OESOPHAGUS' && "Peristalsis is muscle power!
Squeezing ensures food moves regardless of gravity."}
        {currentStageKey === 'STOMACH' && `Food transforms into a
"${selectedFood?.stomachForm || 'soup'}" by stomach acid.`}
        {currentStageKey === 'SMALL_INT' && "Tiny capillaries inside Villi soak up simple
chemical compounds straight into your blood stream!"}
        {currentStageKey === 'LARGE_INT' && "Water saving is vital! Solidification
prevents dangerous dehydration."}
        {currentStageKey === 'COMPLETE' && "A perfectly clean, efficient, fully
functional biological machine!"}
      </p>
    </div>

  </div>

</div>

  { /* Bottom Interactive Dashboard panel */
    <div className="bg-slate-900/90 border-t border-slate-800 p-5 flex flex-col
md:flex-row justify-between items-center gap-4">

      { /* Stage Indicator & Instructions */
        <div className="flex-1 text-center md:text-left">
          <div className="flex items-center justify-center md:justify-start gap-2">
            <span className="text-xs bg-amber-500 text-slate-950 font-black px-2 py-0.5
rounded-full uppercase">
              Stage: {STAGES[currentStageKey].organ}

```

```

    </span>
    <h3 className="text-sm font-bold text-slate-200">
      {STAGES[currentStageKey].title}
    </h3>
  </div>
  <p className="text-xs text-slate-400 mt-1 italic font-medium">
    🍴 {STAGES[currentStageKey].instruction}
  </p>
</div>

{/* Live Action Interaction Area */}
<div className="flex items-center gap-3">
  {currentStageKey === 'PLATE' && (
    <div className="text-xs text-amber-400 animate-pulse flex items-center gap-1
bg-amber-500/10 border border-amber-500/20 px-3 py-1.5 rounded-lg font-bold">
      Pick a food item above to start!
    </div>
  )}

  {currentStageKey === 'MOUTH' && (
    <button
      onClick={handleChew}
      disabled={chewsCount >= 10}
      className="px-5 py-2.5 bg-gradient-to-r from-amber-400 to-amber-500
hover:from-amber-500 hover:to-amber-600 disabled:from-slate-800 disabled:to-slate-800
text-slate-950 disabled:text-slate-500 font-extrabold rounded-xl shadow-lg transition-all
active:scale-95 flex items-center gap-2 text-sm"
    >
      🦷 CHEW! ({chewsCount}/10)
    </button>
  )}

  {currentStageKey === 'OESOPHAGUS' && (
    <div className="w-[180px]">
      <span className="text-[10px] text-slate-400 block text-center mb-1">Slide to
Push Food: {peristalsisPos}%</span>
      <input
        type="range"
        min="0"
        max="100"
        value={peristalsisPos}
        onChange={handlePeristalsisSlider}
        className="w-full h-2.5 bg-slate-800 rounded-lg appearance-none
cursor-pointer accent-amber-500"

```

```

    />
  </div>
)}

{currentStageKey === 'STOMACH' && (
  <button
    onClick={handleChurnStomach}
    disabled={stomachAcidCount >= 8}
    className="px-5 py-2.5 bg-gradient-to-r from-red-500 to-orange-500
    hover:from-red-600 hover:to-orange-600 disabled:from-slate-800 disabled:to-slate-800
    text-white disabled:text-slate-500 font-extrabold rounded-xl shadow-lg transition-all
    active:scale-95 flex items-center gap-2 text-sm animate-pulse"
    >
       CHURN ACID! ({stomachAcidCount}/8)
    </button>
  )}

{currentStageKey === 'SMALL_INT' && (
  <div className="text-xs text-green-400 font-bold bg-green-950/40 border
  border-green-800/30 px-3 py-2 rounded-lg animate-pulse">
    Tap {5 - intestinalAbsorption} more villi loop(s) above!
  </div>
)}

{currentStageKey === 'LARGE_INT' && (
  <div className="text-xs text-blue-400 font-bold bg-blue-950/40 border
  border-blue-800/30 px-3 py-2 rounded-lg animate-pulse">
    Adjust dry bar above to 100%!
  </div>
)}

{currentStageKey === 'COMPLETE' && (
  <button
    onClick={handleReset}
    className="px-5 py-2.5 bg-slate-800 hover:bg-slate-700 text-white font-extrabold
    rounded-xl border border-slate-700 shadow-md transition-all active:scale-95 flex items-center
    gap-2 text-sm"
    >
    <RotateCcw className="w-4 h-4 text-amber-400" /> Start Another Meal
  </button>
)}

{currentStageKey !== 'PLATE' && currentStageKey !== 'COMPLETE' && (
  <button

```

```

        onClick={handleReset}
        className="p-2.5 bg-slate-800/60 hover:bg-slate-800 text-slate-400
hover:text-slate-200 rounded-xl border border-slate-700/50 transition-all"
        title="Reset Meal"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    )}
  </div>

</div>

</div>

{ /* Sidebar Information / Discovery Log (Right 30%) */}
<div className="lg:col-span-5 flex flex-col gap-6">

  { /* Active Info Board (NCERT Based) */}
  <div className="bg-gradient-to-br from-slate-900 to-slate-950 border border-slate-800
rounded-2xl p-5 shadow-xl relative overflow-hidden">
    <div className="absolute top-0 right-0 w-24 h-24 bg-amber-500/5 rounded-full filter
blur-xl" />
    <h3 className="text-base font-bold text-amber-400 flex items-center gap-2 mb-3">
      <BookOpen className="w-5 h-5 text-amber-400" />
      Science Fact Finder
    </h3>

    <div className="space-y-4">
      <div>
        <h4 className="text-xs font-bold text-slate-300 uppercase tracking-wider">How
This Stage Works:</h4>
        <p className="text-xs text-slate-400 mt-1 leading-relaxed">
          {STAGES[currentStageKey].description}
        </p>
      </div>

      <div className="border-t border-slate-800 pt-3">
        <h4 className="text-xs font-bold text-slate-300 uppercase
tracking-wider">Curriculum Checkpoints:</h4>
        <ul className="text-xs text-slate-400 mt-1.5 space-y-2 list-none pl-0">
          {currentStageKey === 'PLATE' && (
            <>
              <li className="flex items-start gap-2">
                <span className="text-amber-500 mt-0.5">•</span>

```


The physical process of breaking down complex molecules into simpler, soluble elements is called **digestion**.

•

Nutrients enter through the mouth, but actual chemical absorption starts later.

{currentStageKey === 'MOUTH' && (

<>

•

Salivary Amylase breaks down complex starches into simple sugars.

•

Chewing increases the **surface area** of food, making it much easier for stomach juices to digest it.

{currentStageKey === 'OESOPHAGUS' && (

<>

•

The food pipe is about 25 cm long. No digestion occurs inside it!

•

Peristalsis is so powerful, you can swallow even while lying flat!

{currentStageKey === 'STOMACH' && (

<>

•

Hydrochloric Acid (HCl) kills unwanted bacteria on food and activates **Pepsin** enzymes.

```

    </li>
    <li className="flex items-start gap-2">
      <span className="text-amber-500 mt-0.5">•</span>
      <span>The J-shape acts as an accordion-style storage, holding food up to 4
hours.</span>
    </li>
  </>
)}
{currentStageKey === 'SMALL_INT' && (
  <>
    <li className="flex items-start gap-2">
      <span className="text-amber-500 mt-0.5">•</span>
      <span><b>Villi</b> drastically increase surface area, boosting absorption
speed.</span>
    </li>
    <li className="flex items-start gap-2">
      <span className="text-amber-500 mt-0.5">•</span>
      <span>This is the longest part of the alimentary canal (almost 6 meters long in
adults!).</span>
    </li>
  </>
)}
{currentStageKey === 'LARGE_INT' && (
  <>
    <li className="flex items-start gap-2">
      <span className="text-amber-500 mt-0.5">•</span>
      <span>The large intestine is wider but much shorter (only about 1.5
meters).</span>
    </li>
    <li className="flex items-start gap-2">
      <span className="text-amber-500 mt-0.5">•</span>
      <span>Losing too much water through the large intestine due to quick flow
causes diarrhea.</span>
    </li>
  </>
)}
{currentStageKey === 'COMPLETE' && (
  <>
    <li className="flex items-start gap-2">
      <span className="text-amber-500 mt-0.5">•</span>
      <span>Expelled waste consists of indigestible fibers, bacteria, and dead cell
matter.</span>
    </li>
    <li className="flex items-start gap-2">

```

```

        <span className="text-amber-500 mt-0.5">•</span>
        <span>Hydration is key! High fiber foods ensure regular waste
pathways.</span>
      </li>
    </>
  )}
</ul>
</div>
</div>
</div>

{ /* Progress & Discovery Log Tracker */}
<div className="bg-slate-900 border border-slate-800 rounded-2xl p-5 shadow-xl
flex-1 flex flex-col justify-between">
  <div>
    <h3 className="text-base font-bold text-slate-200 flex items-center gap-2 mb-3">
      <Compass className="w-5 h-5 text-amber-400" />
      Discovery Log Checklist
    </h3>
    <p className="text-xs text-slate-400 mb-4">Complete each phase of digestion to
unlock scientific field notes:</p>

    <div className="space-y-2.5">
      {Object.keys(STAGES).map((key) => {
        const st = STAGES[key];
        const isUnlocked = unlockedLogs[key];
        return (
          <div
            key={key}
            onClick={() => isUnlocked && setCurrentStageKey(key)}
            className={`flex items-center justify-between p-2.5 rounded-xl border text-left
transition-all ${
              isUnlocked
                ? 'bg-slate-950/60 border-slate-800 hover:border-amber-500/50
cursor-pointer'
                : 'bg-slate-900/20 border-slate-900 text-slate-600 cursor-not-allowed'
            }`}
          >
            <div className="flex items-center gap-2.5">
              <span className="text-xs font-black w-5 h-5 rounded-full flex items-center
justify-center bg-slate-800 text-slate-400">
                {isUnlocked ? '✓' : '?'}
              </span>
            </div>
          </div>
        )
      })}
    </div>
  </div>

```

```

        <span className={`text-xs font-bold block ${isUnlocked ? 'text-slate-200' :
'text-slate-500'}}`>
            {st.organ}
        </span>
        <span className="text-[10px] text-slate-400 block max-w-[150px]
truncate">{st.title}</span>
    </div>
</div>

```

```

        {isUnlocked ? (
            <span className="text-[10px] bg-green-500/10 text-green-400 px-2 py-0.5
rounded-full font-bold">Unlocked</span>
        ) : (
            <span className="text-[10px] bg-slate-800 text-slate-500 px-2 py-0.5
rounded-full font-bold">Locked</span>
        )}
    </div>
);
}}
</div>
</div>
</div>

```

```

</div>

```

```

</div>

```

```

)}

```

```

{/* TAB 2: SCIENCE BOOK (CONCEPTS) */}
{activeTab === 'concepts' && (
    <div className="bg-slate-900 border border-slate-800 rounded-2xl p-6 shadow-2xl
space-y-8 max-w-4xl mx-auto">

```

```

        {/* The Hook */}
        <div className="border-b border-slate-800 pb-5">
            <h2 className="text-2xl font-black text-amber-400">The Human Digestive System
Explained</h2>
            <p className="text-sm text-slate-400 mt-1">Have you ever wondered what happens
to an apple after you take a crunchy bite? It doesn't just sit in your tummy! It goes on a wild,
looping roller coaster ride through your body.</p>
        </div>

```

```

        {/* Structured Concept sections */}
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6">

```

```
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800 space-y-2">
  <span className="text-xl"> 🦷 </span>
```

```
  <h4 className="text-sm font-bold text-slate-200">1. Mechanical Digestion</h4>
```

```
  <p className="text-xs text-slate-400 leading-relaxed">
```

The physical breakdown of big chunks of food into smaller ones by your teeth and stomach. Chewing increases the **surface area** of food, allowing digestive enzymes to work their magic much faster.

```
  </p>
```

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800 space-y-2">
  <span className="text-xl"> 🍷 </span>
```

```
  <h4 className="text-sm font-bold text-slate-200">2. Chemical Digestion</h4>
```

```
  <p className="text-xs text-slate-400 leading-relaxed">
```

The chemical decomposition of complex organic compounds (like proteins, fats, and starches) into smaller, water-soluble molecules (like amino acids and simple glucose sugars) using specialized saliva and acid fluids.

```
  </p>
```

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800 space-y-2">
  <span className="text-xl"> 🌿 </span>
```

```
  <h4 className="text-sm font-bold text-slate-200">3. Villi: The Absorber loops</h4>
```

```
  <p className="text-xs text-slate-400 leading-relaxed">
```

Microscopic, finger-like structures lining the small intestine walls. They acts like a super-fine juice strainer, passing vital nutrients directly into your red blood capillaries while holding back undigested solid pulp.

```
  </p>
```

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800 space-y-2">
  <span className="text-xl"> 💧 </span>
```

```
  <h4 className="text-sm font-bold text-slate-200">4. Water Recovery</h4>
```

```
  <p className="text-xs text-slate-400 leading-relaxed">
```

Your body secretes liters of water inside stomach acids and intestinal juices to keep food flowing. The Large Intestine is a smart recycling plant that reabsorbs all that vital moisture back into your bloodstream.

```
  </p>
```

```
</div>
```

```
</div>
```

```
{/* Memory Trick block */}
```

```
<div className="bg-amber-500/10 border border-amber-500/20 rounded-xl p-4 flex gap-4 items-center">
```

```
<span className="text-3xl">🧠</span>
```

```
</div>
```

```
<h4 className="text-sm font-bold text-amber-400">Memory Trick: Digestion Sequence order</h4>
```

```
<p className="text-xs text-slate-300 mt-1">
```

```
Remember this sentence to master the organ order forever: <br />
```

```
<b>"My Owls Sing Sweet Loud Anthems"</b>
```

```
</p>
```

```
<div className="flex flex-wrap gap-2 mt-2">
```

```
<span className="bg-slate-950 border border-slate-800 px-2 py-0.5 rounded text-[10px] font-bold"><b className="text-amber-400">M</b>outh</span>
```

```
<span className="bg-slate-950 border border-slate-800 px-2 py-0.5 rounded text-[10px] font-bold"><b className="text-amber-400">O</b>esophagus</span>
```

```
<span className="bg-slate-950 border border-slate-800 px-2 py-0.5 rounded text-[10px] font-bold"><b className="text-amber-400">S</b>tomach</span>
```

```
<span className="bg-slate-950 border border-slate-800 px-2 py-0.5 rounded text-[10px] font-bold"><b className="text-amber-400">S</b>mall Intestine</span>
```

```
<span className="bg-slate-950 border border-slate-800 px-2 py-0.5 rounded text-[10px] font-bold"><b className="text-amber-400">L</b>arge Intestine</span>
```

```
<span className="bg-slate-950 border border-slate-800 px-2 py-0.5 rounded text-[10px] font-bold"><b className="text-amber-400">A</b>nus</span>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Quick Digestion Fun Facts */}
```

```
<div className="space-y-3">
```

```
<h4 className="text-sm font-bold text-slate-200">Crazy Biological Facts:</h4>
```

```
<div className="grid grid-cols-1 sm:grid-cols-3 gap-4 text-xs">
```

```
<div className="p-3 bg-slate-900 border border-slate-800 rounded-lg">
```

```
<span className="font-extrabold text-amber-400 block mb-1">Gravity Proof!</span>
```

Even if you eat while hanging upside down like a bat, your food pipe muscles will still push your lunch safely into your stomach.

```
</div>
```

```
<div className="p-3 bg-slate-900 border border-slate-800 rounded-lg">
```

Acid Power!
Stomach acid is so strong that it can dissolve metal razor blades, but your tummy protects itself with constantly renewed mucus sheets!

```
</div>
```

```
<div className="p-3 bg-slate-900 border border-slate-800 rounded-lg">
```

Super Length!

If uncoiled completely, an adult's digestive canal is almost 9 meters long—nearly the size of a standard school bus!

`{/* TAB 3: ACADEMIC QUIZ */}`

`{activeTab === 'quiz' && (`

`<div className="bg-slate-900 border border-slate-800 rounded-2xl p-6 shadow-2xl max-w-2xl mx-auto flex flex-col gap-6">`

`<div className="border-b border-slate-800 pb-4 flex justify-between items-center">`

`<div>`

`<h2 className="text-xl font-bold text-amber-400 flex items-center gap-2">`

`<Award className="w-5 h-5 animate-pulse" /> Class 5–7 Digestion Quiz`

`</h2>`

`<p className="text-xs text-slate-400">Challenge yourself with these 10 curriculum-based questions!</p>`

`</div>`

`<span className="text-xs bg-slate-950 px-3 py-1 border border-slate-800 rounded-xl text-slate-400">`

`Question <b className="text-amber-400">{currentQuizQuestion + 1}</b> of {QUIZ_QUESTIONS.length}`

`</span>`

`</div>`

`{/* Quiz Progress bar */}`

`<div className="w-full bg-slate-950 h-2 rounded-full overflow-hidden">`

`<div`

`className="bg-amber-400 h-full transition-all duration-300"`

`style={{ width: `${((currentQuizQuestion + 1) / QUIZ_QUESTIONS.length) * 100}%` }}`

`/>`

`</div>`

`{/* Active Question Box */}`

`<div className="space-y-4">`

`<h3 className="text-base font-extrabold text-slate-100">`

`{QUIZ_QUESTIONS[currentQuizQuestion].question}`

`</h3>`

```

<div className="grid grid-cols-1 gap-3">
  {QUIZ_QUESTIONS[currentQuizQuestion].options.map((option, idx) => {
    const isSelected = quizAnswers[currentQuizQuestion] === idx;
    let optionClass = "bg-slate-950 hover:bg-slate-800 text-slate-300 border-slate-800";

    if (isSelected) {
      optionClass = "bg-amber-500/20 border-amber-400 text-amber-300 font-bold";
    }
    if (quizSubmitted) {
      const isCorrect = idx === QUIZ_QUESTIONS[currentQuizQuestion].correct;
      if (isCorrect) {
        optionClass = "bg-green-500/20 border-green-500 text-green-300 font-black";
      } else if (isSelected) {
        optionClass = "bg-red-500/20 border-red-500 text-red-300";
      } else {
        optionClass = "opacity-40 border-slate-800 bg-slate-950";
      }
    }
  })

  return (
    <button
      key={idx}
      onClick={() => handleAnswerSelect(idx)}
      disabled={quizSubmitted}
      className={`w-full p-4 text-left text-xs sm:text-sm rounded-xl border transition-all
flex justify-between items-center ${optionClass}`}
    >
      <span>{option}</span>
      {isSelected && !quizSubmitted && <span className="text-amber-400">●
Selected</span>}
      {quizSubmitted && idx === QUIZ_QUESTIONS[currentQuizQuestion].correct && (
        <span className="text-green-400 font-bold text-[10px] uppercase">✓ Correct
Answer</span>
      )}
    </button>
  );
  })}
</div>
</div>

{/* Explanation box under active question if submitted */}
{quizSubmitted && (

```



```

    <div className="bg-slate-950/60 border border-slate-800 p-4 rounded-xl text-xs
text-slate-300 space-y-1">
      <span className="font-extrabold text-amber-500 flex items-center gap-1">  Deep
Fact Finder:</span>
      <p>{QUIZ_QUESTIONS[currentQuizQuestion].explanation}</p>
    </div>
  )}

  { /* Quiz Actions */ }
  <div className="flex justify-between items-center border-t border-slate-800 pt-4">
    <button
      onClick={() => currentQuizQuestion > 0 &&
setCurrentQuizQuestion(currentQuizQuestion - 1)}
      disabled={currentQuizQuestion === 0}
      className="px-4 py-2 bg-slate-950 hover:bg-slate-800 disabled:opacity-30 border
border-slate-800 text-xs text-slate-300 rounded-lg transition-all flex items-center gap-1"
    >
      <ArrowLeft className="w-4 h-4" /> Previous
    </button>

    {quizSubmitted ? (
      <button
        onClick={() => {
          setQuizAnswers(Array(QUIZ_QUESTIONS.length).fill(null));
          setQuizSubmitted(false);
          setCurrentQuizQuestion(0);
          sound.playSuccess();
        }}
        className="px-5 py-2 bg-amber-500 hover:bg-amber-600 text-slate-950 font-bold
text-xs rounded-lg transition-all"
      >
         Retake Quiz
      </button>
    ) : (
      currentQuizQuestion === QUIZ_QUESTIONS.length - 1 ? (
        <button
          onClick={() => {
            setQuizSubmitted(true);
            sound.playSuccess();
            setXp(prev => prev + (quizScore > 70 ? 50 : 20));
          }}
          className="px-5 py-2 bg-gradient-to-r from-emerald-500 to-green-500
hover:from-emerald-600 hover:to-green-600 text-white font-extrabold text-xs rounded-lg
transition-all shadow-md"

```

```

    >
     Submit Answers
  </button>
) : (
  <button
    onClick={() => currentQuizQuestion < QUIZ_QUESTIONS.length - 1 &&
setCurrentQuizQuestion(currentQuizQuestion + 1)}
    disabled={quizAnswers[currentQuizQuestion] === null}
    className="px-5 py-2 bg-amber-500 disabled:bg-slate-800 hover:bg-amber-600
text-slate-950 disabled:text-slate-500 font-bold text-xs rounded-lg transition-all flex items-center
gap-1"
  >
    Next <ArrowRight className="w-4 h-4" />
  </button>
)
)}
</div>

{/* Score Summary Modal style */}
{quizSubmitted && (
  <div className="border border-green-500/20 bg-green-500/5 rounded-2xl p-5
text-center space-y-2">
    <span className="text-3xl"> 🏆 </span>
    <h4 className="text-base font-extrabold text-green-400">Quiz Completed!</h4>
    <p className="text-xs text-slate-300">
      You scored <b className="text-lg text-amber-400">{quizScore} / 100</b> points!
    </p>
    <p className="text-[11px] text-slate-400">
      {quizScore >= 80 ? "Fantastic job! You're a certified human digestion expert! 🌟" :
"Great effort! Review the book facts and try again to hit 100!"}
    </p>
  </div>
)}

</div>
)}

{/* TAB 4: DIY ACTIVITIES */}
{activeTab === 'diy' && (
  <div className="bg-slate-900 border border-slate-800 rounded-2xl p-6 shadow-2xl
space-y-6 max-w-3xl mx-auto">
    <div className="border-b border-slate-800 pb-4">
      <h2 className="text-xl font-black text-amber-400 flex items-center gap-2">
 Home DIY Experiment: "The Kitchen Tube"
    </h2>
    </div>
  </div>
)}

```

```

    </h2>
    <p className="text-xs text-slate-400">Simulate how the food pipe and stomach
    physically crush food using common kitchen supplies!</p>
    </div>

    { /* Materials Checklist */ }
    <div className="bg-slate-950 border border-slate-800 p-4 rounded-xl space-y-3">
      <h4 className="text-xs font-bold uppercase tracking-wider text-slate-300">Materials
    Needed:</h4>
      <div className="grid grid-cols-1 sm:grid-cols-2 gap-2 text-xs text-slate-300">
        <div className="flex items-center gap-2">
          <span className="text-amber-500">✓</span> 1 clear long plastic zipper bag
        (Stomach)
        </div>
        <div className="flex items-center gap-2">
          <span className="text-amber-500">✓</span> 1 cardboard tube from paper towels
        (Food Pipe)
        </div>
        <div className="flex items-center gap-2">
          <span className="text-amber-500">✓</span> 1 soft piece of bread or biscuit
        (Snack)
        </div>
        <div className="flex items-center gap-2">
          <span className="text-amber-500">✓</span> 3 tablespoons of colored water/juice
        (Saliva/Acid)
        </div>
      </div>
    </div>

    { /* Steps Timeline */ }
    <div className="space-y-4">
      <h4 className="text-xs font-bold uppercase tracking-wider
    text-slate-300">Step-by-Step Method:</h4>

      <div className="relative border-l-2 border-slate-800 pl-6 space-y-6 text-xs
    text-slate-300">
        { /* Step 1 */ }
        <div className="relative">
          <span className="absolute -left-9 top-0.5 bg-amber-500 text-slate-950 text-[10px]
    font-black w-5 h-5 rounded-full flex items-center justify-center">1</span>
          <h5 className="font-extrabold text-slate-200">The Mechanical Chop (Mouth)</h5>
          <p className="text-slate-400 mt-1">Tear the piece of bread into tiny crumbs with
        your bare hands. This represents your incisors and molars grinding down food chunks!</p>
        </div>
      </div>
    </div>

```

```

    { /* Step 2 */
    <div className="relative">
      <span className="absolute -left-9 top-0.5 bg-amber-500 text-slate-950 text-[10px]
font-black w-5 h-5 rounded-full flex items-center justify-center">2</span>
      <h5 className="font-extrabold text-slate-200">The Squeeze Tube
(Oesophagus)</h5>
      <p className="text-slate-400 mt-1">Drop the bread pieces down the vertical
cardboard tube. Use your fingers to squeeze and nudge the crumbs down into the zipper bag at
the bottom.</p>
    </div>

```

```

    { /* Step 3 */
    <div className="relative">
      <span className="absolute -left-9 top-0.5 bg-amber-500 text-slate-950 text-[10px]
font-black w-5 h-5 rounded-full flex items-center justify-center">3</span>
      <h5 className="font-extrabold text-slate-200">Acid Splashes (Stomach
Fluids)</h5>
      <p className="text-slate-400 mt-1">Pour the colored water into the zipper bag with
the bread. Zip the lock extremely tight so nothing leaks out!</p>
    </div>

```

```

    { /* Step 4 */
    <div className="relative">
      <span className="absolute -left-9 top-0.5 bg-amber-500 text-slate-950 text-[10px]
font-black w-5 h-5 rounded-full flex items-center justify-center">4</span>
      <h5 className="font-extrabold text-slate-200">The Blender Churn (Digestion)</h5>
      <p className="text-slate-400 mt-1">Use both hands to continuously squeeze,
mash, and squish the zipper bag for 2 solid minutes. Watch the solid crumbs dissolve entirely
into a wet slurry paste!</p>
    </div>
  </div>
</div>

```

```

    { /* Science explanation box */
    <div className="bg-amber-500/15 border border-amber-500/20 rounded-xl p-4 text-xs
text-slate-200 space-y-1.5">
      <span className="font-bold text-amber-400 flex items-center gap-1">🧠
Parent/Teacher Observation Tip:</span>
      <p className="text-slate-300">
        Ask the student: <i>"If we didn't rip the bread into crumbs in step 1, would it take more
time or less time to mush it up inside the bag?"</i>
      </p>
      <p className="text-[11px] text-slate-400">

```

Explanation: Ripping/chewing first increases surface area, allowing saliva water to coat and soften the starches immediately. Larger unchewed parts are much harder to dissolve!

```
    </p>
  </div>

  </div>
})

</main>

{ /* Footer */
  <footer className="border-t border-slate-800/80 bg-slate-950/80 backdrop-blur-md px-4
py-3 mt-auto text-center text-xs text-slate-500">
    <div className="max-w-7xl mx-auto flex flex-col sm:flex-row justify-between items-center
gap-2">
      <span>Created in strict accordance with <b>NCERT Class 5–7 Science
Syllabus</b>.</span>
      <span>© 2026 The Great Food Journey. Safe, Discovery-based learning.</span>
    </div>
  </footer>
</div>
);
}
```

Photosynthesis6–7

```

import React, { useState, useEffect, useMemo } from 'react';
import {
  Sun,
  Droplet,
  Wind,
  RefreshCw,
  Award,
  BookOpen,
  HelpCircle,
  CheckCircle,
  Leaf,
  Beaker,
  Play,
  Check,
  ChevronRight,
  Star,
  Flame,
  Info,
  Users,
  Home,
  Code
} from 'lucide-react';

// --- GAMIFIED BADGES CONSTANTS ---
const BADGES = [
  { id: 'first_light', name: 'First Light', desc: 'Turned on the sun simulator', icon: Sun, color:
'text-amber-500 bg-amber-50' },
  { id: 'perfect_recipe', name: 'Master Chef', desc: 'Discovered the 100% optimal growth setup',
icon: Award, color: 'text-emerald-500 bg-emerald-50' },
  { id: 'quiz_champion', name: 'Super Botanist', desc: 'Scored 100% on the Science Quiz', icon:
Star, color: 'text-purple-500 bg-purple-50' },
  { id: 'diy_enthusiast', name: 'Eco Explorer', desc: 'Examined the DIY breathing experiment',
icon: Beaker, color: 'text-blue-500 bg-blue-50' }
];

export default function App() {
  // --- STATE ---
  const [activeTab, setActiveTab] = useState('simulator');
  const [xp, setXp] = useState(100);
  const [unlockedBadges, setUnlockedBadges] = useState([]);
  const [showCelebration, setShowCelebration] = useState(false);
  const [celebrationMessage, setCelebrationMessage] = useState("");

  // Simulator Inputs

```

```
const [light, setLight] = useState(50);
const [water, setWater] = useState(50);
const [co2, setCo2] = useState(true);
```

```
// Quiz State
```

```
const [currentQuestionIndex, setCurrentQuestionIndex] = useState(0);
const [selectedAnswer, setSelectedAnswer] = useState(null);
const [isAnswerSubmitted, setIsAnswerSubmitted] = useState(false);
const [quizScore, setQuizScore] = useState(0);
const [quizCompleted, setQuizCompleted] = useState(false);
```

```
// DIY Checklist
```

```
const [diySteps, setDiySteps] = useState([
  { id: 1, text: 'Collect 2-3 fresh green leaves (mint, spinach, or garden leaves)', completed: false },
  { id: 2, text: 'Submerge leaves completely in a clear glass bowl filled with warm water', completed: false },
  { id: 3, text: 'Place a small stone on the leaves to hold them underwater', completed: false },
  { id: 4, text: 'Place the glass on a bright, sunny windowsill for 1-2 hours', completed: false },
  { id: 5, text: 'Observe tiny oxygen bubbles forming on the leaf surface!', completed: false }
]);
```

```
// --- SCIENTIFIC ENGINE CALCULATIONS ---
```

```
const { efficiency, plantHealth, bubbleCount, starCount, observationText, activeHint } =
useMemo(() => {
  const rawEff = (light / 100) * (water / 100) * (co2 ? 1.0 : 0.15);
  const efficiencyPercentage = Math.round(rawEff * 100);
```

```
  let health = 'healthy';
  let observation = 'Your plant is doing well! It is absorbing sunlight, water, and carbon dioxide to make food.';
  let hint = 'Try adjusting the sliders to see what makes the plant happiest!';
```

```
  if (water < 25) {
    health = 'wilted';
    observation = 'Oh no! The soil is too dry. The plant is wilting because its roots cannot find water to perform photosynthesis.';
    hint = '💧 Drag the Water slider above 50% to quench the plant\'s thirst!';
  } else if (light < 25) {
    health = 'starved';
    observation = 'It is too dark! Without light energy, the chloroplasts inside the leaves cannot cook up any food (glucose).';
    hint = '☀️ Increase the Light slider to turn up the power of the sun!';
  } else if (!co2) {
```



```

    health = 'suffocating';
    observation = 'The plant has run out of Carbon Dioxide gas ($CO_2$). It cannot bind the
ingredients together to build food stars.';
    hint = '💧 Flip the CO2 gas switch back to ON to feed the leaves breathing gas!';
  } else if (light >= 80 && water >= 80 && co2) {
    health = 'optimal';
    observation = '✨ STUNNING! Perfect conditions achieved! The plant is in hyper-drive,
creating maximum Glucose (food stars) and pumping out Oxygen bubbles!';
    hint = '🍰 You found the golden recipe! Enjoy the bubble rush!';
  } else if (light > 70 || water > 70) {
    observation = 'Great job! Higher inputs are speeding up food production. Watch the oxygen
bubbles float up faster!';
    hint = 'What happens if you crank up BOTH Light and Water to maximum? Give it a try!';
  }

```

```

const bubbles = Math.floor(rawEff * 12);
const stars = Math.floor(rawEff * 8);

```

```

return {
  efficiency: efficiencyPercentage,
  plantHealth: health,
  bubbleCount: bubbles,
  starCount: stars,
  observationText: observation,
  activeHint: hint
};
}, [light, water, co2]);

```

// --- SIDE EFFECTS (ACHIEVEMENTS & BADGES) ---

```

useEffect(() => {
  if (light > 0 && !unlockedBadges.includes('first_light')) {
    triggerBadge('first_light', 'First Light Unlocked! You brought sunlight to the world! ☀️');
  }
}, [light]);

```

```

useEffect(() => {
  if (plantHealth === 'optimal' && !unlockedBadges.includes('perfect_recipe')) {
    triggerBadge('perfect_recipe', 'Master Chef Unlocked! You discovered the perfect
Photosynthesis recipe! 🌿🏆');
    setXp(prev => prev + 150);
  }
}, [plantHealth]);

```

```

const triggerBadge = (id, message) => {

```

```

setUnlockedBadges(prev => [...prev, id]);
setCelebrationMessage(message);
setShowCelebration(true);
setXp(prev => prev + 50);
setTimeout(() => {
  setShowCelebration(false);
}, 4000);
};

const handleReset = () => {
  setLight(50);
  setWater(50);
  setCo2(true);
};

const handleDiyCheck = (id) => {
  setDiySteps(prev => {
    const updated = prev.map(step => step.id === id ? { ...step, completed: !step.completed } :
step);
    const allCompleted = updated.every(s => s.completed);
    if (allCompleted && !unlockedBadges.includes('diy_enthusiast')) {
      triggerBadge('diy_enthusiast', 'Eco Explorer Unlocked! You successfully understood the
underwater leaf experiment! 🌿🌱');
    }
    return updated;
  });
};

// --- QUIZ QUESTIONS ---
const quizQuestions = [
  {
    type: 'mcq',
    question: 'What are the three main ingredients a plant needs to complete Photosynthesis?',
    options: [
      'A) Oxygen, soil minerals, and insects',
      'B) Sunlight, water, and Carbon Dioxide',
      'C) Sugar, fertilizer, and shadow',
      'D) Milk, water, and clean air'
    ],
    correct: 1,
    tip: 'Remember the acronym L.W.C. (Light, Water, CO2)!',
    explain: 'Plants trap sunlight energy, absorb water through roots, and breathe in Carbon
Dioxide from the air to make starch.',
  }
];

```

wrongExplain: 'Oxygen is actually what the plant makes and releases, not what it needs to start cooking!'

},

{

type: 'tf',

question: 'True or False: Plants perform photosynthesis during the pitch-black middle of the night.',

options: ['True', 'False'],

correct: 1,

tip: 'The clue is in the name: "Photo" means Light!',

explain: 'Without a light source, plants have no energy to power the chemical factory inside their leaves.',

wrongExplain: 'Even though they still breathe at night, they cannot synthesize food in the dark.'

},

{

type: 'fill',

question: 'The beautiful green color of plant leaves is caused by a special solar-trapping pigment called...',

options: ['Chlorophyll', 'Hemoglobin', 'Melanin', 'Carotene'],

correct: 0,

tip: 'Sounds like "Chloro-fill"!',

explain: 'Chlorophyll lives inside chloroplasts and acts like tiny solar panels to capture photons.',

wrongExplain: 'Hemoglobin makes human blood red, while carotene gives carrots their bright orange color!'

},

{

type: 'scenario',

question: 'Rohan put his potted mint plant in a completely dark cardboard box and watered it daily. After a week, why did the leaves turn yellow and weak?',

options: [

'A) It missed getting fresh fertilizer from Rohan.',

'B) It could not perform photosynthesis without light.',

'C) The water levels were way too high.',

'D) Box cardboard is toxic to plants.'

],

correct: 1,

tip: 'Water is only half the equation!',

explain: 'Sunlight is the power source. Without light energy, the plant starved even though it had plenty of water.',

wrongExplain: 'Fertilizers only help, but without the solar energy, the kitchen is completely closed!'

},

```

{
  type: 'mcq',
  question: 'Which of these is the actual FOOD that the plant makes and stores for itself?',
  options: [
    'A) Carbon Dioxide gas',
    'B) Oxygen gas bubbles',
    'C) Glucose (sugar/starch)',
    'D) Nitrogen compounds'
  ],
  correct: 2,
  tip: 'It is a sweet, energy-rich molecule!',
  explain: 'Glucose is a simple sugar made during photosynthesis that plants use to grow taller, build flowers, and store in fruits.',
  wrongExplain: 'Oxygen is released back into the air as a byproduct, and Carbon Dioxide is the fuel raw material.'
},
{
  type: 'mcq',
  question: 'Why are leaves typically flat, thin, and wide?',
  options: [
    'A) To catch falling raindrops easily',
    'B) To maximize their surface area to absorb sunlight',
    'C) To hide their seeds from hungry birds',
    'D) To float better if they fall into streams'
  ],
  correct: 1,
  tip: 'Imagine a bigger solar panel!',
  explain: 'A wider, flatter surface lets chloroplasts gather as many light photons as possible in the green kitchen.',
  wrongExplain: 'While catching rain is nice, capturing solar beams is the leaf's main architectural focus.'
},
{
  type: 'tf',
  question: 'True or False: More than half of the oxygen in Earth's atmosphere is produced by microscopic ocean plants!',
  options: ['True', 'False'],
  correct: 0,
  tip: 'The ocean is the lungs of our planet!',
  explain: 'Microscopic marine organisms called phytoplankton do photosynthesis on a massive scale, providing most of our breath.',
  wrongExplain: 'While rain forests are extremely important, the vast oceans produce the absolute majority of global oxygen.'
},

```

```

{
  type: 'scenario',
  question: 'An astronaut is attempting to grow spinach in a sealed glass capsule in space. She provides bright lights and water, but the plant refuses to grow. What is missing?',
  options: [
    'A) Gravity simulators',
    'B) Carbon Dioxide gas',
    'C) Soil worm particles',
    'D) Helium wind currents'
  ],
  correct: 1,
  tip: 'Plants need the air they breathe to be loaded with a specific carbon-heavy gas!',
  explain: 'Without carbon dioxide gas in the sealed container, the chemical reactions of photosynthesis cannot begin.',
  wrongExplain: 'Spinach can easily grow in zero gravity and without soil (hydroponics) if all gases are balanced.'
},
{
  type: 'mcq',
  question: 'Where inside a plant cell does photosynthesis take place?',
  options: [
    'A) The Nucleus',
    'B) The Chloroplast',
    'C) The Cell Wall',
    'D) The Vacuole'
  ],
  correct: 1,
  tip: 'These look like tiny green jelly beans under a microscope!',
  explain: 'Chloroplasts are the specialized kitchen hubs containing chlorophyll where solar energy reacts with water and gas.',
  wrongExplain: 'The nucleus is the brain of the cell, and the cell wall is just the external structural boundary.'
},
{
  type: 'mcq',
  question: 'What happens to the Carbon Dioxide we exhale when we breathe?',
  options: [
    'A) It gets trapped in space forever.',
    'B) Green plants absorb it to build sugar and make fresh oxygen.',
    'C) It turns into rainwater automatically.',
    'D) It is completely useless and gets destroyed.'
  ],
  correct: 1,
  tip: 'It\'s a beautiful circle of life!',

```

explain: 'We are partners with nature! The carbon waste we exhale is the exact raw material plants crave to make their fruits and leaves.'

wrongExplain: 'Nature never wastes! Gases are recycled dynamically between animal respiration and plant photosynthesis.'

}

];

const handleQuizAnswer = (index) => {

if (isAnswerSubmitted) return;

setSelectedAnswer(index);

setIsAnswerSubmitted(true);

if (index === quizQuestions[currentQuestionIndex].correct) {

setQuizScore(prev => prev + 1);

setXp(prev => prev + 25);

}

};

const handleNextQuiz = () => {

setSelectedAnswer(null);

setIsAnswerSubmitted(false);

if (currentQuestionIndex < quizQuestions.length - 1) {

setCurrentQuestionIndex(prev => prev + 1);

} else {

setQuizCompleted(true);

if (quizScore + (selectedAnswer === quizQuestions[currentQuestionIndex].correct ? 1 : 0) === quizQuestions.length) {

if (!unlockedBadges.includes('quiz_champion')) {

triggerBadge('quiz_champion', 'Super Botanist Unlocked! 10/10 Score on the Photosynthesis Quiz! 🎓 🧠');

}

}

}

};

const restartQuiz = () => {

setCurrentQuestionIndex(0);

setSelectedAnswer(null);

setIsAnswerSubmitted(false);

setQuizScore(0);

setQuizCompleted(false);

};

return (

```

<div className="min-h-screen bg-slate-50 text-slate-800 font-sans flex flex-col antialiased">
  {/ * --- FLOATING CELEBRATION CARD --- */}
  {showCelebration && (
    <div className="fixed top-6 left-1/2 transform -translate-x-1/2 z-50 animate-bounce">
      <div className="bg-gradient-to-r from-emerald-600 to-teal-500 text-white px-6 py-4
rounded-full shadow-2xl flex items-center gap-3 border-4 border-white">
        <Award className="w-8 h-8 text-yellow-300 animate-spin" />
        <div>
          <p className="font-extrabold text-lg">Badge Achieved!</p>
          <p className="text-sm opacity-90">{celebrationMessage}</p>
        </div>
        <div className="bg-white text-emerald-700 font-black px-3 py-1 rounded-full text-xs">
          +50 XP
        </div>
      </div>
    </div>
  )}

  {/ * --- HEADER --- */}
  <header className="bg-white border-b border-slate-100 sticky top-0 z-40 shadow-sm">
    <div className="max-w-7xl mx-auto px-4 py-3 sm:py-4 flex flex-col sm:flex-row
items-center justify-between gap-4">

      {/ * Logo */}
      <div className="flex items-center gap-3">
        <div className="bg-emerald-500 p-2.5 rounded-2xl shadow-md
shadow-emerald-200">
          <Leaf className="w-6 h-6 text-white animate-pulse" />
        </div>
        <div>
          <span className="text-xs font-bold text-emerald-600 tracking-wider
uppercase">Class 6-7 NCERT Lab</span>
          <h1 className="text-xl sm:text-2xl font-black text-slate-900 tracking-tight">The Green
Maker Lab</h1>
        </div>
      </div>

      {/ * Gamified Stat Badges */}
      <div className="flex items-center gap-3 w-full sm:w-auto justify-end">
        <div className="flex items-center gap-2 bg-amber-50 border border-amber-200 px-3
py-1.5 rounded-full shadow-inner">
          <Flame className="w-5 h-5 text-amber-500 fill-amber-500 animate-pulse" />
          <span className="font-black text-amber-700 text-sm">{xp}</span> <span className="text-xs
font-medium">XP</span></span>

```

```

<div className="flex items-center gap-1.5 bg-emerald-50 border border-emerald-100
px-3 py-1.5 rounded-full">
  <span className="text-xs font-bold text-emerald-800">Badges:</span>
  <div className="flex gap-1">
    {BADGES.map((b) => {
      const licon = b.icon;
      const isUnlocked = unlockedBadges.includes(b.id);
      return (
        <div
          key={b.id}
          title={` ${b.name}: ${b.desc}`}
          className={`p-1 rounded-full transition-all duration-300 ${isUnlocked ? b.color + '
scale-110 shadow-sm' : 'bg-slate-200 text-slate-400 opacity-40'}`}
        >
          <Icon className="w-3.5 h-3.5" />
        </div>
      );
    })}
  </div>
</div>
</div>
</div>
</div>
</header>

```

```

    {/ * --- MODULE CONTROLS / TABS --- */}
    <nav className="bg-emerald-900/5 py-3 border-b border-emerald-900/10">
      <div className="max-w-7xl mx-auto px-4 flex flex-wrap gap-2 justify-center
sm:justify-start">
        <button
          onClick={() => setActiveTab('simulator')}
          className={`flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-bold transition-all
${activeTab === 'simulator' ? 'bg-emerald-600 text-white shadow-md' : 'bg-white text-slate-600
hover:bg-slate-100'}`}
        >
          <Beaker className="w-4 h-4" />
          1. Lab Simulator
        </button>

        <button
          onClick={() => setActiveTab('explanation')}

```



```

        className={`flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-bold transition-all
${activeTab === 'explanation' ? 'bg-emerald-600 text-white shadow-md' : 'bg-white
text-slate-600 hover:bg-slate-100'}`}
      >
        <BookOpen className="w-4 h-4" />
        2. The Magic Kitchen
      </button>

      <button
        onClick={() => setActiveTab('diy')}
        className={`flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-bold transition-all
${activeTab === 'diy' ? 'bg-emerald-600 text-white shadow-md' : 'bg-white text-slate-600
hover:bg-slate-100'}`}
      >
        <Droplet className="w-4 h-4" />
        3. DIY Activity
      </button>

      <button
        onClick={() => setActiveTab('quiz')}
        className={`flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-bold transition-all
${activeTab === 'quiz' ? 'bg-emerald-600 text-white shadow-md' : 'bg-white text-slate-600
hover:bg-slate-100'}`}
      >
        <Award className="w-4 h-4" />
        4. Challenge Quiz
      </button>

      <button
        onClick={() => setActiveTab('resources')}
        className={`flex items-center gap-2 px-4 py-2 rounded-xl text-sm font-bold transition-all
${activeTab === 'resources' ? 'bg-emerald-600 text-white shadow-md' : 'bg-white text-slate-600
hover:bg-slate-100'}`}
      >
        <Info className="w-4 h-4" />
        5. Guides & Notes
      </button>
    </div>
  </nav>

  {/* --- CONTENT CONTAINER --- */}
  <main className="flex-1 max-w-7xl w-full mx-auto p-4 md:py-8">

    {/* =====

```

TAB 1: INTERACTIVE LABORATORY SIMULATOR

```

===== */}
{activeTab === 'simulator' && (
  <div className="grid grid-cols-1 lg:grid-cols-10 gap-6">

    {/* Interactive Canvas (70% Equivalent) */}
    <div className="lg:col-span-7 flex flex-col gap-6">

      {/* Premium Canvas Box */}
      <div className="bg-white border border-slate-200/80 rounded-3xl p-6 shadow-xl
relative overflow-hidden flex-1 min-h-[420px] flex flex-col justify-between">

        {/* Background Decor Gradients */}
        <div className="absolute top-0 right-0 w-64 h-64 bg-emerald-500/5 rounded-full
blur-3xl pointer-events-none" />
        <div className="absolute bottom-0 left-0 w-64 h-64 bg-amber-500/5 rounded-full
blur-3xl pointer-events-none" />

        {/* Sky Light Tint overlay depending on solar slider */}
        <div
          className="absolute inset-0 pointer-events-none transition-opacity duration-500"
          style={{
            background: `radial-gradient(circle at top, rgba(253, 224, 71, ${light / 400}) 0%,
transparent 70%)`,
            opacity: light > 10 ? 1 : 0
          }}
        />

        {/* Canvas Header info overlay */}
        <div className="flex justify-between items-center z-10">
          <div className="flex items-center gap-2 bg-slate-900/5 px-3 py-1 rounded-full
text-xs font-semibold">
            <span className="w-2.5 h-2.5 bg-emerald-500 rounded-full animate-ping" />
            Live Environment Feed
          </div>
          <div className="flex items-center gap-2">
            <span className="text-sm font-bold text-slate-600">Recipe Efficiency:</span>
            <span className={`text-lg font-black ${efficiency > 75 ? 'text-emerald-600' :
efficiency > 30 ? 'text-amber-500' : 'text-rose-500'}`}>{efficiency}%</span>
          </div>
        </div>

        {/* --- CENTRAL PLANT GRAPHIC (SVG ANIMATED SYSTEM) --- */}
        <div className="flex-1 flex justify-center items-center py-6 relative">

```

```

    { /* Floating Oxygen Bubbles (O2) */
    {Array.from({ length: bubbleCount }).map((_, i) => (
      <div
        key={`bubble-${i}`}
        className="absolute bottom-40 bg-cyan-100/70 border border-cyan-300
rounded-full flex items-center justify-center text-[8px] font-black text-cyan-600 select-none
shadow-sm shadow-cyan-200"
        style={{
          width: `${12 + (i % 3) * 6}px`,
          height: `${12 + (i % 3) * 6}px`,
          left: `${40 + (i * 14) % 25}%`,
          animation: `floatUp ${2 + (i % 3) * 1.5}s infinite ease-in-out`,
          animationDelay: `${i * 0.4}s`
        }}
      >
        O2
      </div>
    ))}

    { /* Floating Glucose Stars (C6H12O6) */
    {Array.from({ length: starCount }).map((_, i) => (
      <div
        key={`star-${i}`}
        className="absolute bottom-28 text-amber-500 drop-shadow-md"
        style={{
          left: `${35 + (i * 22) % 35}%`,
          animation: `sparkleFloat ${2.5 + (i % 2) * 1.5}s infinite ease-in-out`,
          animationDelay: `${i * 0.6}s`
        }}
      >
        <Star className="w-4 h-4 fill-amber-300 text-amber-500" />
      </div>
    ))}

    { /* Sun Radiation Rays */
    {light > 15 && (
      <div className="absolute top-2 left-1/2 -translate-x-1/2 flex justify-between w-72
pointer-events-none opacity-80">
        {Array.from({ length: 6 }).map((_, i) => (
          <div
            key={i}
            className="h-28 w-[2px] bg-gradient-to-b from-amber-300/40 to-transparent
transform origin-top"

```

```

        style={{
          transform: `rotate(${ -30 + i * 12 }deg)`,
          animation: 'pulseRays 2s infinite ease-in-out',
          animationDelay: `${i * 0.2}s`
        }}
      />
    ))}
  </div>
)}

{/* Carbon Dioxide Floating Input Molecules */}
{co2 && Array.from({ length: 4 }).map((_, i) => (
  <div
    key={`co2-${i}`}
    className="absolute top-12 bg-slate-100 border border-slate-300 rounded-full
px-2 py-0.5 text-[9px] font-bold text-slate-500 shadow-sm"
    style={{
      left: `${15 + i * 22}%`,
      animation: `driftIn ${4 + i * 2}s infinite linear`,
      animationDelay: `${i * 1.2}s`
    }}
  >
    CO2
  </div>
)}

{/* --- SVG Plant Render --- */}
<svg
  viewBox="0 0 400 400"
  className="w-full max-w-[280px] h-auto drop-shadow-2xl transition-all
duration-700"
  style={{ transform: `scale(${plantHealth === 'optimal' ? 1.12 : plantHealth ===
'wilted' ? 0.95 : 1.0})` }}
  >
  {/* Terrarium Dome (Premium lab layout) */}
  <path d="M 50,320 A 150,150 0 0,1 350,320 Z" fill="none" stroke="#cbd5e1"
strokeWidth="2" strokeDasharray="6,4" />
  <rect x="40" y="320" width="320" height="15" rx="5" fill="#94a3b8" />

  {/* Soil base (Dynamic based on water) */}
  <rect x="90" y="270" width="220" height="60" rx="20" fill={water > 20 ? '#543d2b' :
'#a1a1aa'} className="transition-all duration-700" />

  {/* Water Moisture particles inside Soil */}

```

```

{water > 10 && (
  <g className="opacity-80 transition-opacity">
    <circle cx="120" cy="290" r="3" fill="#38bdf8" className="animate-pulse" />
    <circle cx="280" cy="295" r="3.5" fill="#38bdf8" className="animate-pulse" />
    <circle cx="200" cy="305" r="2.5" fill="#38bdf8" className="animate-pulse" />
  </g>
)}

{ /* Roots System (Dynamic opacity depending on plant quality) */}
<path
  d="M 200,270 Q 180,290 170,310 M 200,270 Q 220,285 230,310 M 200,270 V
315"
  fill="none"
  stroke={water > 15 ? '#854d0e' : '#71717a'}
  strokeWidth="3.5"
  strokeLinecap="round"
  className="transition-all duration-700"
/>

{ /* Terrarium Glass Pot container */}
<path d="M 90,260 L 120,330 L 280,330 L 310,260 Z" fill="none" stroke="#94a3b8"
strokeWidth="4" />

{ /* MAIN STEM */}
<path
  d={plantHealth === 'wilted' ? "M 200,260 Q 190,190 170,140" : "M 200,260 Q
205,180 200,110"}
  fill="none"
  stroke={plantHealth === 'wilted' ? '#84cc16' : '#22c55e'}
  strokeWidth="8"
  strokeLinecap="round"
  className="transition-all duration-700"
/>

{ /* Leaf Left Bottom */}
<g
  transform={plantHealth === 'wilted' ? "translate(195, 210) rotate(-40)" :
"translate(200, 200) rotate(-20)"}
  className="transition-all duration-700"
>
  <path d="M 0,0 C -35,-20 -60,0 -80,-5 C -60,25 -25,25 0,0" fill={plantHealth ===
'wilted' ? '#ca8a04' : '#15803d'} className="transition-colors duration-700" />
  <path d="M 0,0 Q -40,5 -80,-5" fill="none" stroke="#22c55e" strokeWidth="2" />
</g>

```

```

    { /* Leaf Right Middle */ }
    <g
      transform={plantHealth === 'wilted' ? "translate(205, 185) rotate(45)" :
"translate(200, 160) rotate(15)"}
      className="transition-all duration-700"
    >
      <path d="M 0,0 C 35,-20 60,0 80,-5 C 60,25 25,25 0,0" fill={plantHealth ===
'wilted' ? '#eab308' : '#16a34a'} className="transition-colors duration-700" />
      <path d="M 0,0 Q 40,5 80,-5" fill="none" stroke="#4ade80" strokeWidth="2" />
    </g>

    { /* Leaf Left Top */ }
    <g
      transform={plantHealth === 'wilted' ? "translate(185, 140) rotate(-55)" :
"translate(198, 120) rotate(-10)"}
      className="transition-all duration-700"
    >
      <path d="M 0,0 C -30,-15 -55,0 -70,-3 C -55,20 -20,20 0,0" fill={plantHealth ===
'wilted' ? '#a3e635' : '#15803d'} className="transition-colors duration-700" />
      <path d="M 0,0 Q -35,5 -70,-3" fill="none" stroke="#22c55e" strokeWidth="2" />
    </g>

    { /* Sprouting Top Bud */ }
    <path
      d={plantHealth === 'wilted' ? "M 170,140 Q 165,115 155,100 Q 170,115 170,140" :
"M 200,110 Q 200,80 190,65 Q 210,85 200,110"}
      fill={plantHealth === 'wilted' ? '#ca8a04' : '#22c55e'}
      className="transition-colors duration-700"
    />
  </svg>
</div>

{ /* --- REAL-TIME STATUS BAR --- */ }
<div className="bg-slate-900 text-white rounded-2xl p-4 flex flex-wrap items-center
justify-between gap-4 z-10 border border-slate-800">
  <div className="flex items-center gap-3">
    <div className={`p-2 rounded-xl ${plantHealth === 'optimal' ? 'bg-emerald-500' :
plantHealth === 'wilted' ? 'bg-amber-500' : 'bg-slate-700'}`}>
      <Leaf className="w-5 h-5 text-white" />
    </div>
  </div>
  <p className="text-xs uppercase text-slate-400 font-bold tracking-wider">Plant
Status</p>

```

```

        <p className="text-sm font-extrabold capitalize">{plantHealth === 'optimal' ? '🌱
Hyper-active Growth!' : plantHealth === 'wilted' ? '🍃 Wilting (Thirsty)' : plantHealth === 'starved'
? '❄️ Starving (No Light)' : plantHealth === 'suffocating' ? '💨 Suffocating (No Gas)' : '🟢
Growing Normally'}</p>
    </div>
</div>

```

```

    <div className="flex gap-4">
      <div className="text-center">
        <span className="block text-[10px] text-slate-400 uppercase
font-black">Oxygen Bubble Rate</span>
        <span className="text-base font-black text-cyan-400">{bubbleCount * 10}
bubbles/min</span>
      </div>
      <div className="w-[1px] bg-slate-800 self-stretch" />
      <div className="text-center">
        <span className="block text-[10px] text-slate-400 uppercase
font-black">Glucose Sugar Energy</span>
        <span className="text-base font-black text-amber-400">{starCount * 5}
units</span>
      </div>
    </div>
  </div>

```

```

    { /* Dynamic Bottom Sliders Card */ }
    <div className="bg-white border border-slate-200/80 rounded-3xl p-6 shadow-md">
      <div className="flex justify-between items-center mb-6">
        <h3 className="font-extrabold text-slate-900 text-base flex items-center gap-2">
          <Beaker className="w-5 h-5 text-emerald-600" />
          Environment Controllers
        </h3>
        <button
          onClick={handleReset}
          className="flex items-center gap-1.5 text-xs text-emerald-600
hover:text-emerald-700 font-extrabold border border-emerald-200 px-3 py-1.5 rounded-full
hover:bg-emerald-50/50 transition-all"
          >
          <RefreshCw className="w-3.5 h-3.5" />
          Reset Lab
        </button>
      </div>
    </div>

```

```

<div className="grid grid-cols-1 md:grid-cols-3 gap-6">

  {/* SLIDER 1: LIGHT */}
  <div className="space-y-3">
    <div className="flex justify-between items-center">
      <span className="text-sm font-bold text-slate-700 flex items-center gap-1.5">
        <Sun className="w-4 h-4 text-amber-500" />
        Sunlight Intensity
      </span>
      <span className="text-xs bg-amber-100 text-amber-800 px-2 py-0.5 rounded-full
font-black">{light}%</span>
    </div>
    <input
      type="range"
      min="0"
      max="100"
      value={light}
      onChange={(e) => setLight(Number(e.target.value))}
      className="w-full accent-amber-500 h-2 bg-slate-200 rounded-lg
appearance-none cursor-pointer"
    />
    <p className="text-[10px] text-slate-500">Determines total energy inputs captured
by leaves.</p>
  </div>

  {/* SLIDER 2: WATER */}
  <div className="space-y-3">
    <div className="flex justify-between items-center">
      <span className="text-sm font-bold text-slate-700 flex items-center gap-1.5">
        <Droplet className="w-4 h-4 text-blue-500" />
        Soil Moisture
      </span>
      <span className="text-xs bg-blue-100 text-blue-800 px-2 py-0.5 rounded-full
font-black">{water}%</span>
    </div>
    <input
      type="range"
      min="0"
      max="100"
      value={water}
      onChange={(e) => setWater(Number(e.target.value))}
      className="w-full accent-blue-500 h-2 bg-slate-200 rounded-lg appearance-none
cursor-pointer"
    />
  </div>

```



```

    <p className="text-[10px] text-slate-500">Provides water molecules through the
root system.</p>
</div>

```

```

    {/* SLIDER 3: CO2 TOGGLE */}
    <div className="space-y-3">
      <div className="flex justify-between items-center">
        <span className="text-sm font-bold text-slate-700 flex items-center gap-1.5">
          <Wind className="w-4 h-4 text-slate-400" />
          Carbon Dioxide (CO2)
        </span>
        <span className={`text-xs px-2.5 py-0.5 rounded-full font-black ${co2 ?
'bg-emerald-100 text-emerald-800' : 'bg-rose-100 text-rose-800'}`}>
          {co2 ? 'ACTIVE' : 'NONE'}
        </span>
      </div>
    </div>

```

```

    <div className="flex items-center gap-3 h-10">
      <button
        onClick={() => setCo2(!co2)}
        className={`w-full py-2 px-4 rounded-xl text-xs font-black tracking-wider
uppercase transition-all border ${co2 ? 'bg-slate-900 text-white border-slate-900 shadow-md' :
'bg-slate-100 text-slate-400 border-slate-200'}`}
      >
        {co2 ? 'Injecting CO2 Gas' : 'CO2 Source Cutoff'}
      </button>
    </div>

```

```

    <p className="text-[10px] text-slate-500">The essential carbon source from
surrounding air.</p>
</div>

```

```

</div>
</div>

```

```

</div>

```

```

    {/* Right Panel (30% Equivalent) */}
    <div className="lg:col-span-3 flex flex-col gap-6">

      {/* Live Observation Panel */}
      <div className="bg-gradient-to-br from-emerald-900 to-emerald-950 text-white
rounded-3xl p-6 shadow-lg relative overflow-hidden flex-1">
        <div className="absolute top-0 right-0 w-32 h-32 bg-emerald-500/10 rounded-full
blur-2xl" />

```

```

<h3 className="font-extrabold text-lg flex items-center gap-2 mb-4
text-emerald-200">
  <Leaf className="w-5 h-5 text-emerald-300" />
  Live Lab Notebook
</h3>

<div className="bg-emerald-800/40 border border-emerald-700 rounded-2xl p-4
min-h-[140px] text-sm leading-relaxed mb-6">
  <p className="italic text-slate-200">"{observationText}"</p>
</div>

<div className="border-t border-emerald-800/80 pt-4">
  <span className="text-xs uppercase font-extrabold text-emerald-300 tracking-wider
block mb-2">Equation Tracker:</span>
  <div className="bg-emerald-950/80 border border-emerald-900 p-3 rounded-xl
text-[11px] font-mono leading-relaxed text-emerald-100 space-y-1">
    <div>
      <span className={`font-black ${light > 15 ? 'text-amber-300' :
'text-slate-500'}}>☀️ Light Energy</span> + <span className={`font-black ${water > 15 ?
'text-blue-300' : 'text-slate-500'}}>💧 {water}% H2O</span> + <span className={`font-black
${co2 ? 'text-slate-300' : 'text-slate-500'}}>🌬️ CO2 Gas</span>
    </div>
    <div className="text-emerald-400 font-bold">⬇️ Produces via Chlorophyll:</div>
    <div>
      <span className={`font-black ${efficiency > 20 ? 'text-yellow-300' :
'text-slate-500'}}>⚡ Glucose Starch</span> + <span className={`font-black ${efficiency > 20
? 'text-cyan-300' : 'text-slate-500'}}>💧 Oxygen Gas</span>
    </div>
  </div>
</div>

{ /* Scientist Hint Panel */ }
<div className="bg-amber-50/80 border border-amber-200 rounded-3xl p-6
shadow-sm">
  <h4 className="font-bold text-amber-900 text-sm flex items-center gap-2 mb-3">
    <HelpCircle className="w-5 h-5 text-amber-500" />
    Scientist Hint
  </h4>
  <p className="text-xs text-amber-800 leading-relaxed font-medium">
    {activeHint}
  </p>
</div>

```

```

    { /* Gamification Level up Progress */
    <div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-sm
space-y-4">
    <div className="flex items-center justify-between">
    <div>
    <h4 className="font-extrabold text-slate-800 text-sm">Level 2 Botanist</h4>
    <p className="text-[10px] text-slate-500">Next Badge unlocks at 300 XP</p>
    </div>
    <Award className="w-8 h-8 text-amber-500 fill-amber-100 animate-bounce" />
    </div>

    { /* Visual Progress Bar */
    <div className="w-full bg-slate-100 rounded-full h-3 overflow-hidden shadow-inner">
    <div
    className="bg-emerald-500 h-full rounded-full transition-all duration-500"
    style={{ width: `${Math.min(100, (xp / 400) * 100)}%` }}
    />
    </div>
    <div className="flex justify-between items-center text-xs font-bold text-slate-500">
    <span>{xp} XP</span>
    <span>400 XP</span>
    </div>
    </div>

    </div>

    </div>
  )}

  { /* =====
  TAB 2: THE MAGIC KITCHEN (CONCEPT EXPLANATION)
  ===== */
  {activeTab === 'explanation' && (
    <div className="max-w-4xl mx-auto bg-white border border-slate-200 rounded-3xl p-6
md:p-8 shadow-xl space-y-8">

    { /* Main Title Hook */
    <div className="text-center space-y-2 border-b border-slate-100 pb-6">
    <span className="text-xs font-bold text-emerald-600 uppercase tracking-widest
bg-emerald-50 px-3 py-1 rounded-full">Educational Adventure</span>
    <h2 className="text-2xl sm:text-3xl font-black text-slate-900">How do Trees Cook
without a Stove? 🌳🔍</h2>

```

```
<p className="text-sm text-slate-500">Let's peek inside nature's most magical
chemical kitchen.</p>
</div>
```

```
{/* Hook-Imagine-Observe Structure */}
```

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-6 items-start">
```

```
<div className="space-y-4">
```

```
<div className="bg-emerald-50/50 p-4 rounded-2xl border-l-4 border-emerald-500">
```

```
<h4 className="font-extrabold text-slate-900 text-sm mb-1">🤔 Imagine
```

```
This...</h4>
```

```
<p className="text-xs text-slate-600 leading-relaxed">
```

Imagine you are a master chef. But instead of turning on a gas stove or microwave, your oven is the mighty **Sun**! Your kitchen counters are flat **green leaves**, and your raw ingredients are just invisible **Carbon Dioxide gas** in the air and **moist water** from under the soil.

```
</p>
```

```
</div>
```

```
<div className="bg-blue-50/50 p-4 rounded-2xl border-l-4 border-blue-500">
```

```
<h4 className="font-extrabold text-slate-900 text-sm mb-1">🧐 Observe
```

```
Carefully...</h4>
```

```
<p className="text-xs text-slate-600 leading-relaxed">
```

When you look at a mango tree outside, it looks completely still. But inside its green leaves, millions of tiny solar-powered factories are working at speed of light! Roots suck up water from dark dirt, leaves breathe in gas via microscopic mouth pores called **stomata**, and **chlorophyll** traps the golden solar energy.

```
</p>
```

```
</div>
```

```
</div>
```

```
<div className="space-y-4">
```

```
<div className="bg-amber-50/50 p-4 rounded-2xl border-l-4 border-amber-500">
```

```
<h4 className="font-extrabold text-slate-900 text-sm mb-1">🔬 What Happened &
```

```
Why?</h4>
```

```
<p className="text-xs text-slate-600 leading-relaxed">
```

Using trapped solar energy, the green chloroplast breaks down water (H_2O) and carbon dioxide (CO_2) molecules, and fits them back together like Lego blocks to build delicious plant food called **Glucose**! The leftover byproduct is fresh **Oxygen gas**, which is released into the atmosphere for humans to breathe.

```
</p>
```

```
</div>
```

```
{/* The Formula Card */}
```

```

    <div className="bg-slate-900 text-white p-4 rounded-2xl border border-slate-800
text-center relative overflow-hidden">
      <span className="text-[9px] uppercase font-bold text-slate-400
tracking-wider">The Sacred Chemical Equation</span>
      <div className="text-xs font-mono font-black mt-2 text-emerald-400
leading-relaxed">
        Carbon Dioxide ($CO_2$) + Water ($H_2O$) <br />
        <span className="text-white text-[10px]">☀️ Sunlight + Chlorophyll ➡️</span>
      <br />
        Glucose ($C_6H_{12}O_6$) + Oxygen ($O_2$)
      </div>
    </div>
  </div>

```

```

  {/* Memory Trick Panel */}
  <div className="bg-gradient-to-r from-emerald-600 to-teal-500 text-white rounded-2xl
p-5 text-center shadow-lg">
    <h4 className="font-bold text-sm uppercase tracking-widest text-emerald-100
mb-1">🧠 Quick Memory Trick!</h4>
    <p className="text-base font-black">Just remember: <span className="underline
decoration-yellow-300 decoration-2">S.W.A.G.</span></p>
    <p className="text-xs opacity-90 mt-2">
      **S**unlight + **W**ater + **A**ir (CO2) ➡️ **G**lucose + Oxygen!
    </p>
  </div>

```

```

  {/* 10 Real Life Examples Slider / List */}
  <div className="space-y-4 pt-4 border-t border-slate-100">
    <h3 className="font-extrabold text-slate-900 text-lg flex items-center gap-2">
      <CheckCircle className="w-5 h-5 text-emerald-600" />
      10 Amazing Real-Life Examples
    </h3>

    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex
items-start gap-3">
        <span className="bg-emerald-100 text-emerald-800 text-xs font-bold px-2 py-0.5
rounded-lg">1</span>
        <p className="text-xs text-slate-600"><strong>Under a shady tree:</strong> It
feels incredibly cool and fresh because trees are actively exhaling fresh Oxygen and pure water
moisture.</p>
      </div>

```

```

    <div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex
items-start gap-3">
      <span className="bg-emerald-100 text-emerald-800 text-xs font-bold px-2 py-0.5
rounded-lg">2</span>
      <p className="text-xs text-slate-600"><strong>Cactus Needles:</strong> In hot
deserts, a cactus shrinks its leaves into sharp needles to save water from evaporating during
food cooking.</p>
    </div>
    <div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex
items-start gap-3">
      <span className="bg-emerald-100 text-emerald-800 text-xs font-bold px-2 py-0.5
rounded-lg">3</span>
      <p className="text-xs text-slate-600"><strong>Aquarium plants:</strong> If you
watch aquarium plants closely under direct tank lighting, you will notice tiny bubbles rising up -
that's pure Oxygen!</p>
    </div>
    <div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex
items-start gap-3">
      <span className="bg-emerald-100 text-emerald-800 text-xs font-bold px-2 py-0.5
rounded-lg">4</span>
      <p className="text-xs text-slate-600"><strong>Window bending:</strong> Indoor
house plants turn and bend their stems toward windows because they are seeking light
energy.</p>
    </div>
    <div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex
items-start gap-3">
      <span className="bg-emerald-100 text-emerald-800 text-xs font-bold px-2 py-0.5
rounded-lg">5</span>
      <p className="text-xs text-slate-600"><strong>Autumn leaves:</strong> As winter
approaches, trees break down green chlorophyll, turning leaves yellow as they pause
photosynthesis.</p>
    </div>
    <div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex
items-start gap-3">
      <span className="bg-emerald-100 text-emerald-800 text-xs font-bold px-2 py-0.5
rounded-lg">6</span>
      <p className="text-xs text-slate-600"><strong>Farmers' Greenhouses:</strong>
Crops are grown inside bright glass houses to guarantee constant sunlight, boosting food
yield.</p>
    </div>
    <div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex
items-start gap-3">
      <span className="bg-emerald-100 text-emerald-800 text-xs font-bold px-2 py-0.5
rounded-lg">7</span>

```

<p className="text-xs text-slate-600">Money Plant Patches: The yellow/white spots on money plant leaves lack chlorophyll, meaning those specific patches cannot perform photosynthesis.</p>

</div>

<div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex items-start gap-3">

8

<p className="text-xs text-slate-600">Super grass growth: Your garden grass becomes super green after monsoon rains combined with sunny afternoons.</p>

</div>

<div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex items-start gap-3">

9

<p className="text-xs text-slate-600">Green pond scum: Algae spreads ultra-fast over pond surfaces because it captures light particles perfectly to breed.</p>

</div>

<div className="bg-slate-50 p-3.5 rounded-xl border border-slate-100 flex items-start gap-3">

10

<p className="text-xs text-slate-600">Your breath: The carbon dioxide gas you exhale right now is captured by outdoor plants tomorrow to grow sweet fruit!</p>

</div>

</div>

</div>

{/* Fun Facts Accordion style */}

<div className="space-y-4 pt-4 border-t border-slate-100">

<h3 className="font-extrabold text-slate-900 text-lg flex items-center gap-2">

<Star className="w-5 h-5 text-amber-500 fill-amber-100" />

5 Mind-Blowing Fun Facts

</h3>

<div className="space-y-2 text-xs">

<div className="bg-amber-50 p-3 rounded-xl border border-amber-100">

✨ Ocean Plants: Over 50% of the Oxygen we breathe doesn't come from land forests! It is made by microscopic drifting plants in the ocean.

</div>

<div className="bg-amber-50 p-3 rounded-xl border border-amber-100">

✨ Earth Cleaners: Before plants evolved photosynthesis billions of years ago, Earth's atmosphere was toxic with zero breathing oxygen!

```

    </div>
    <div className="bg-amber-50 p-3 rounded-xl border border-amber-100">
      ✨ <strong>Purple Power:</strong> Plants can perform photosynthesis under
artificial purple and blue LEDs just as happily as under real sunlight.
    </div>
    <div className="bg-amber-50 p-3 rounded-xl border border-amber-100">
      ✨ <strong>Bamboo Sprint:</strong> Some species of bamboo grow over 3 feet in a
single day thanks to super-speedy photosynthesis.
    </div>
    <div className="bg-amber-50 p-3 rounded-xl border border-amber-100">
      ✨ <strong>Shadow Senses:</strong> Plants have chemical sensors that warn
them if neighboring weeds are stealing their solar shadows.
    </div>
  </div>
</div>

```

```

  </div>
)}

```

```

{ /* =====
  TAB 3: DIY ACTIVITY (UNDERWATER BUBBLE BREATH)
  ===== */
  {activeTab === 'diy' && (
    <div className="max-w-3xl mx-auto bg-white border border-slate-200 rounded-3xl p-6
md:p-8 shadow-xl space-y-6">

      <div className="text-center space-y-2 border-b border-slate-100 pb-6">
        <span className="text-xs font-bold text-blue-600 uppercase tracking-widest
bg-blue-50 px-3 py-1 rounded-full">Home Experiment</span>
        <h2 className="text-2xl sm:text-3xl font-black text-slate-900">Project: Underwater
Bubble Breath 🫧🔬</h2>
        <p className="text-sm text-slate-500">Witness a live leaf breathe out oxygen bubbles
using common kitchen tools.</p>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-6 items-stretch">

        <div className="bg-slate-50 border border-slate-100 rounded-2xl p-5 space-y-4">
          <h3 className="font-extrabold text-slate-800 text-sm">🔬 Materials Needed:</h3>
          <ul className="space-y-2 text-xs font-semibold text-slate-600">
            <li className="flex items-center gap-2">
              <span className="w-2 h-2 bg-blue-500 rounded-full" />
              Clear glass bowl or glass tumbler
            </li>

```



```

<li className="flex items-center gap-2">
  <span className="w-2 h-2 bg-blue-500 rounded-full" />
  2-3 freshly picked green garden leaves
</li>
<li className="flex items-center gap-2">
  <span className="w-2 h-2 bg-blue-500 rounded-full" />
  Lukewarm water from tap
</li>
<li className="flex items-center gap-2">
  <span className="w-2 h-2 bg-blue-500 rounded-full" />
  A small stone weight
</li>
</ul>

```

```

<div className="bg-amber-50 border border-amber-200 rounded-xl p-4 text-xs">
  <span className="font-black text-amber-950 block mb-1"> ⚠ Safety First:</span>
  No sharp knives, heat, or chemicals are required. Always wash your hands after
  touching garden plants!
</div>
</div>

```

```

{/* Progress checklist card */}
<div className="bg-emerald-50/50 border border-emerald-100 rounded-2xl p-5 flex
flex-col justify-between">
  <div>
    <h3 className="font-extrabold text-emerald-900 text-sm mb-3"> 📋 Interactive Step
    Checklist:</h3>
    <div className="space-y-2">
      {diySteps.map((step) => (
        <button
          key={step.id}
          onClick={() => handleDiyCheck(step.id)}
          className="w-full text-left flex items-start gap-2.5 p-2 rounded-lg
          hover:bg-white/60 transition-colors"
        >
          <div className={`mt-0.5 border-2 rounded w-4.5 h-4.5 flex items-center
          justify-center transition-all ${step.completed ? 'bg-emerald-600 border-emerald-600 text-white' :
          'border-slate-300 bg-white'}`}>
            {step.completed && <Check className="w-3 h-3 stroke-[3]" />}
          </div>
          <span className={`text-xs ${step.completed ? 'line-through
          text-emerald-800/60' : 'text-slate-700 font-bold'}`}>
            {step.text}
          </span>
        </button>
      )
    }
  </div>

```

```

        </button>
      )}}
    </div>
  </div>

  <div className="border-t border-emerald-200/50 pt-3 mt-4 text-center">
    <span className="text-[10px] uppercase font-bold text-emerald-800">Challenge
Question:</span>
    <p className="text-xs text-slate-700 italic mt-1 font-semibold">What will happen if
we place the glass in a dark drawer? Give it a try!</p>
  </div>
</div>

</div>

  </div>
  )}

  {/* =====
  TAB 4: CHALLENGE QUIZ (10 QUESTIONS)
  ===== */}
  {activeTab === 'quiz' && (
    <div className="max-w-2xl mx-auto bg-white border border-slate-200 rounded-3xl p-6
md:p-8 shadow-xl space-y-6">

      {/* Quiz Header info */}
      <div className="flex justify-between items-center border-b border-slate-100 pb-4">
        <div>
          <span className="text-[10px] uppercase font-bold text-emerald-600
tracking-wider">Quiz Progress</span>
          <h3 className="font-extrabold text-slate-900 text-base">Question
{currentQuestionIndex + 1} of {quizQuestions.length}</h3>
        </div>
        <div className="bg-emerald-50 text-emerald-800 px-3 py-1.5 rounded-full font-black
text-xs flex items-center gap-1.5 shadow-sm">
          <Star className="w-4 h-4 fill-emerald-500 text-emerald-600" />
          Score: {quizScore} / {quizQuestions.length}
        </div>
      </div>

      {/* Main Quiz Body */}
      {!quizCompleted ? (
        <div className="space-y-6">

```

```

    { /* Question Text */
    <div className="bg-slate-950 text-white p-5 rounded-2xl border border-slate-800
relative overflow-hidden">
      <div className="absolute top-0 right-0 w-32 h-32 bg-emerald-500/10 rounded-full
blur-2xl" />
      <span className="bg-emerald-500 text-white text-[9px] uppercase px-2 py-0.5
rounded font-black tracking-wider block w-max mb-2">
        {quizQuestions[currentQuestionIndex].type.toUpperCase()}
      </span>
      <p className="text-sm font-bold
leading-relaxed">{quizQuestions[currentQuestionIndex].question}</p>
    </div>

    { /* Option buttons */
    <div className="space-y-3">
      {quizQuestions[currentQuestionIndex].options.map((opt, optIndex) => {
        const isSelected = selectedAnswer === optIndex;
        const isCorrectOption = optIndex === quizQuestions[currentQuestionIndex].correct;

        let btnStyle = 'border-slate-200 hover:bg-slate-50 text-slate-700 bg-white';
        if (isAnswerSubmitted) {
          if (isCorrectOption) {
            btnStyle = 'border-emerald-500 bg-emerald-50 text-emerald-900 font-extrabold
shadow-sm';
          } else if (isSelected) {
            btnStyle = 'border-rose-300 bg-rose-50 text-rose-800 shadow-sm';
          } else {
            btnStyle = 'opacity-50 border-slate-100 bg-slate-50 text-slate-400';
          }
        }
      })

    return (
      <button
        key={optIndex}
        onClick={() => handleQuizAnswer(optIndex)}
        disabled={isAnswerSubmitted}
        className={`w-full text-left p-4 rounded-2xl border-2 transition-all flex
items-center justify-between text-xs font-semibold ${btnStyle}`}
      >
        <span>{opt}</span>
        {isAnswerSubmitted && isCorrectOption && (
          <CheckCircle className="w-5 h-5 text-emerald-600 flex-shrink-0 ml-2" />
        )}
      </button>

```

```

    );
  }}
</div>

  { /* Answer Explanation Panel */
  {isAnswerSubmitted && (
    <div className="bg-emerald-50/50 border border-emerald-100 rounded-2xl p-5
space-y-3 animate-fadeIn">
      <div className="flex items-center gap-2">
        <div className="p-1.5 bg-emerald-100 rounded-lg text-emerald-800">
          <Info className="w-4 h-4" />
        </div>
        <span className="text-xs font-black text-emerald-900">Scientist
Explains:</span>
      </div>
      <p className="text-xs text-slate-600 leading-relaxed
font-semibold">{quizQuestions[currentQuestionIndex].explain}</p>
      <p className="text-xs text-rose-600 italic leading-relaxed border-t
border-emerald-100 pt-2">{quizQuestions[currentQuestionIndex].wrongExplain}</p>

      { /* Scientist tips block */
      <div className="bg-amber-50 border border-amber-200 p-3 rounded-xl text-[11px]
text-amber-900 font-bold flex items-center gap-2">
        <Star className="w-4 h-4 text-amber-500 fill-amber-300 flex-shrink-0" />
        <span>Tip: {quizQuestions[currentQuestionIndex].tip}</span>
      </div>

      <button
        onClick={handleNextQuiz}
        className="w-full mt-2 bg-slate-900 hover:bg-slate-800 text-white py-3
rounded-xl text-xs font-black uppercase tracking-widest flex items-center justify-center gap-2"
      >
        <span>{currentQuestionIndex === quizQuestions.length - 1 ? 'Finish Quiz' : 'Next
Question'}</span>
        <ChevronRight className="w-4 h-4" />
      </button>
    </div>
  )}

  </div>
) : (
  /* Quiz completed celebratory panel */
  <div className="text-center space-y-6 py-6 animate-fadeIn">

```

```

    <div className="w-20 h-20 bg-emerald-100 rounded-full flex items-center
justify-center mx-auto shadow-inner">
      <Award className="w-10 h-10 text-emerald-600" />
    </div>

    <div className="space-y-2">
      <h3 className="text-2xl font-black text-slate-900">Quiz Completed! 🌿🎓</h3>
      <p className="text-sm text-slate-500">You scored a robust <span
className="font-black text-emerald-600 text-lg">{quizScore} /
{quizQuestions.length}</span></p>
    </div>

    <div className="bg-slate-50 border border-slate-100 rounded-2xl p-5 max-w-sm
mx-auto text-xs font-bold text-slate-600">
      {quizScore === quizQuestions.length ? (
        <span className="text-emerald-700 block">🏆 Phenomenal job! You are officially
an autotroph specialist. Score 10/10 awarded perfect badges!</span>
      ) : quizScore >= 7 ? (
        <span className="text-emerald-700 block">✨ Fantastic job! You have a highly
optimized understanding of plant biology.</span>
      ) : (
        <span className="text-amber-700 block">🌿 Good effort! Go back to Tab 1 & Tab
2 to experiment with more environment parameters!</span>
      )}
    </div>

    <div className="flex gap-3 justify-center">
      <button
        onClick={restartQuiz}
        className="bg-slate-900 text-white font-extrabold text-xs px-6 py-3 rounded-xl
hover:bg-slate-800"
      >
        Try Again
      </button>
      <button
        onClick={() => setActiveTab('simulator')}
        className="border border-slate-300 text-slate-700 font-extrabold text-xs px-6 py-3
rounded-xl hover:bg-slate-100"
      >
        Back to Simulator
      </button>
    </div>
  </div>
)}

```

```

    </div>
  })

  {/* =====
    TAB 5: GUIDES & DEVELOPER RESOURCES
    ===== */}
  {activeTab === 'resources' && (
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6 max-w-5xl mx-auto">

      {/* Teacher Notes */}
      <div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-md
space-y-4">
        <h3 className="font-extrabold text-slate-900 text-base flex items-center gap-2
text-emerald-700">
          <Users className="w-5 h-5" />
          Teacher Resources & Classroom Activities
        </h3>

        <div className="space-y-3 text-xs leading-relaxed text-slate-600">
          <div>
            <strong className="block text-slate-900">🏠 Classroom Activity:</strong>
            Bring two potted plants into the school science room. Place one near a window and
            cover the other with dark cardboard. After 5 days, let the students observe and document leaf
            color shifts and stem directions.
          </div>
          <div>
            <strong className="block text-slate-900">★ Assessment Idea:</strong>
            Ask students to write down a short narrative essay as if they were a "Chlorophyll
            Solar Cell" inside a leaf. They must describe their role in catching photons and fitting gases
            together.
          </div>
          <div>
            <strong className="block text-slate-900">📖 NCERT Syllabus
            Connection:</strong>
            Links directly with Chapter 1: "Nutrition in Plants" for Classes 6-7, helping students
            understand concepts of autotrophic nutrition and leaf starch tests.
          </div>
        </div>

      {/* Parent Notes */}
      <div className="bg-white border border-slate-200 rounded-3xl p-6 shadow-md
space-y-4">

```

```
<h3 className="font-extrabold text-slate-900 text-base flex items-center gap-2 text-blue-700">
```

```
<Home className="w-5 h-5" />
```

```
Parent Connection & Conversational starters
```

```
</h3>
```

```
<div className="space-y-3 text-xs leading-relaxed text-slate-600">
```

```
<div>
```

```
<strong className="block text-slate-900">🍎 Dinner Table Conversation:</strong>  
Next time you eat fresh fruits or vegetables at home, remind your child: *"We are  
eating stored sunshine energy! How did this carrot trap the light?"*
```

```
</div>
```

```
<div>
```

```
<strong className="block text-slate-900">🌿 Backyard Exploration:</strong>  
Go into the garden with a simple magnifying glass. Look at the back side of various  
leaves to talk about invisible stomata pores they use to breathe carbon dioxide.
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Developer Guide Card */}
```

```
<div className="bg-slate-900 text-slate-100 border border-slate-800 rounded-3xl p-6 shadow-md space-y-4 md:col-span-2">
```

```
<h3 className="font-extrabold text-white text-base flex items-center gap-2">
```

```
<Code className="w-5 h-5 text-emerald-400" />
```

```
Developer Technical Implementation Guide
```

```
</h3>
```

```
<div className="grid grid-cols-1 sm:grid-cols-3 gap-4 text-xs">
```

```
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
```

```
<span className="text-emerald-400 font-extrabold block mb-1">State Flow &  
Logic:</span>
```

The game uses reactive mathematical modeling (`efficiency`) calculated immediately upon sliding. No heavy continuous game loops required, resulting in a buttery-smooth 60fps responsive UI.

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
```

```
<span className="text-emerald-400 font-extrabold block mb-1">CSS  
Animations:</span>
```

Uses infinite translation and keyframes (`floatUp` and `sparkleFloat`) with dynamic delay configurations to simulate floating bubbles and starch stars at low resource overhead.

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
```

Architecture:

Fully written inside a single React component using Tailwind CSS layout utilities.
Easily extensible to store quiz milestones in standard student databases.

</div>

</div>

</div>

</div>

)}

</main>

{/* --- FOOTER --- */}

<footer className="bg-white border-t border-slate-100 py-6 text-center text-xs
text-slate-400 mt-12">

<div className="max-w-7xl mx-auto px-4 flex flex-col sm:flex-row justify-between
items-center gap-4">

<p>© 2026 Interactive EdTech Inc. Designed for premium school subscription
modules.</p>

<p className="font-semibold text-slate-500">Topic: Photosynthesis | Target Grade: 6 -
8</p>

</div>

</footer>

{/* --- CUSTOM CSS ANIMATIONS INLINE EMBED --- */}

<style>{`

@keyframes floatUp {

0% {

transform: translateY(0) scale(0.6) rotate(0deg);

opacity: 0;

}

10% {

opacity: 0.9;

}

90% {

opacity: 0.7;

}

100% {

transform: translateY(-240px) scale(1.1) rotate(15deg);

opacity: 0;

}

}


```
@keyframes sparkleFloat {
  0% {
    transform: translate(0, 0) scale(0.8) rotate(0deg);
    opacity: 0;
  }
  20% {
    opacity: 1;
  }
  80% {
    opacity: 0.9;
  }
  100% {
    transform: translate(30px, -180px) scale(1.2) rotate(180deg);
    opacity: 0;
  }
}
```

```
@keyframes driftIn {
  0% {
    transform: translateX(-40px) translateY(0px) rotate(0deg);
    opacity: 0;
  }
  15% {
    opacity: 0.6;
  }
  85% {
    opacity: 0.6;
  }
  100% {
    transform: translateX(200px) translateY(30px) rotate(45deg);
    opacity: 0;
  }
}
```

```
@keyframes pulseRays {
  0%, 100% {
    opacity: 0.7;
    transform: scaleY(1);
  }
  50% {
    opacity: 1;
    transform: scaleY(1.08);
  }
}
```

```
.animate-fadeIn {
  animation: fadeIn 0.4s ease-out forwards;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
`}</style>
</div>
);
}
```

photosynthesis

```

import React, { useState, useEffect } from 'react';

// =====
// 🎨 SVGs & VECTOR GRAPHICS (PREMIUM DESIGN)
// =====

// Custom Inline SVG Organism Icons for High Fidelity
const OrganismIcon = ({ name, className = "w-16 h-16" }) => {
  switch (name) {
    case 'Sun':
      return (
        <svg className={` ${className} text-amber-400 animate-pulse`} viewBox="0 0 24 24"
fill="currentColor">
          <circle cx="12" cy="12" r="5" />
          <path d="M12 1v2M12 21v2M4.22 4.22l1.42 1.42M18.36 18.36l1.42 1.42M1 12h2M21 12h2M4.22 19.78l1.42-1.42M18.36 5.64l1.42-1.42" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" />
        </svg>
      );
    case 'Grass':
      return (
        <svg className={` ${className} text-emerald-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2">
          <path d="M3 22c1-5 3-9 7-11M10 22c2-7 5-11 11-13M7 22c.5-4 1.5-7 4-8.5"
strokeLinecap="round" />
          <path d="M14 22c1-6 2.5-9.5 5.5-11.5M17 22c.5-4 1-6 3-7.5" strokeLinecap="round" />
        </svg>
      );
    case 'Grasshopper':
      return (
        <svg className={` ${className} text-lime-600`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5" strokeLinecap="round" strokeLinejoin="round">
          <path d="M4 14c2-2 5-3 8-1s4 4 6 2l3-2" />
          <path d="M12 13l2-6 3 1" />
          <path d="M7 14l-3 5h2l2-4" />
          <path d="M17 13l3 6h-2l-2-5" />
          <circle cx="19" cy="11" r="1" fill="currentColor" />
        </svg>
      );
    case 'Frog':
      return (
        <svg className={` ${className} text-emerald-600`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
          <path d="M12 21c-4.5 0-7-2.5-7-5.5s1.5-4.5 3-4.5h8c1.5 0 3 1.5 3 4.5s-2.5 5.5-7 5.5z" />

```

```

    <ellipse cx="9" cy="9" rx="2" ry="3" fill="none" />
    <circle cx="9" cy="9" r="0.75" fill="currentColor" />
    <ellipse cx="15" cy="9" rx="2" ry="3" fill="none" />
    <circle cx="15" cy="9" r="0.75" fill="currentColor" />
    <path d="M7 16c2 1.5 8 1.5 10 0" />
    <path d="M5 19c-1 0-2-1-2-2.5S4.5 14 5.5 14" />
    <path d="M19 19c1 0 2-1 2-2.5S19.5 14 18.5 14" />
  </svg>
);
case 'Snake':
  return (
    <svg className={` ${className} text-emerald-800`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M8.5 4.5A2.5 2.5 0 0111 7c0 1.5-1 3-3 4.5S5 14 5 16s2 3 5 3 5-1.5
5-3.5-1.5-3-3.5-3-3-2-3-3.5a2.5 2.5 0 014.5-1.5" strokeLinecap="round" strokeLinejoin="round"
/>
      <path d="M11 6h.01M9 7h.01" strokeWidth="2" strokeLinecap="round" />
      <path d="M11.5 3.5l1-1M11.5 4.5l1.5-.5" />
    </svg>
);
case 'Algae (Pond Plant)':
  return (
    <svg className={` ${className} text-teal-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2">
      <path d="M4 18c2-4 1-8 4-11s5-1 6 3-1 7-4 9-5 1-6-1z" opacity="0.6" />
      <path d="M11 20c2-5 1-9 4-11s4 0 5 3-2 7-5 9-3-1-4-1z" opacity="0.8" />
      <path d="M8 22c1.5-6 .5-10 3.5-12s3.5-.5 4.5 2.5-2 7.5-5 9.5-2.5-1-3-2z" />
    </svg>
);
case 'Small Fish':
  return (
    <svg className={` ${className} text-sky-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M2 12c3-4 10-5 15-2 2.5 1.5 4 3 5 4-1 1-2.5 2.5-5 4-5 3-12 2-15-2z" />
      <path d="M17 10l4-4v12l-4-4" />
      <circle cx="6" cy="11" r="1" fill="currentColor" />
      <path d="M10 11.5c1.5 0 2 .5 2 1" />
    </svg>
);
case 'Seal':
  return (
    <svg className={` ${className} text-slate-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M4 14c2-4 6-6 10-5s6 3 8 6-3 4-7 4-7-1-11-5z" />

```

```

    <path d="M22 15l-3 3M21 13l-4 4" />
    <circle cx="7" cy="11" r="1" fill="currentColor" />
    <path d="M2 13c1 0 2 1.5 2 3s-1 3-2 3" />
  </svg>
);
case 'Orca Whale':
  return (
    <svg className={` ${className} text-slate-800`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M2 13c2-5 8-8 14-6s5 4 6 6-4 4-9 4-9-1-11-4z" />
      <path d="M14 7c0-2-1-4-3-5-.5 2 0 4 1 5" fill="currentColor" />
      <path d="M18 10c1-1 3-1 3 0s-2 2-3 0" />
      <path d="M8 12.5C10 13 13 13 14 12c-1-.5-3-1-6 .5z" fill="currentColor" />
    </svg>
  );
case 'Oak Tree':
  return (
    <svg className={` ${className} text-emerald-700`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2">
      <path d="M12 20v-5M10 20h4" strokeWidth="3" strokeLinecap="round" />
      <path d="M12 15c3.5 0 6-2 6-5.5S15.5 4 12 4s-6 2-6 5.5S8.5 15 12 15z"
fill="currentColor" fillOpacity="0.15" />
      <path d="M12 13a4.5 4.5 0 100-9 4.5 4.5 0 000 9z" />
    </svg>
  );
case 'Caterpillar':
  return (
    <svg className={` ${className} text-lime-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round">
      <circle cx="5" cy="14" r="2" fill="currentColor" />
      <circle cx="9" cy="12" r="2" fill="currentColor" />
      <circle cx="13" cy="13" r="2" fill="currentColor" />
      <circle cx="17" cy="11" r="2" fill="currentColor" />
      <circle cx="20" cy="9" r="1.5" />
      <path d="M19 7l-1-1.5M20.5 7.5L22 7M4 16l-1 2M8 14l-1 2M12 15l-1 2M16 13l-1 2" />
    </svg>
  );
case 'Bluebird':
  return (
    <svg className={` ${className} text-blue-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M17 6c-1.5.5-3 2-4 3.5-.5-1-1.5-2.5-3-3-2-.7-4 .5-4.5 2S6 13.5 9 14.5c4 1.3 7-.5
9-2.5s2.5-4 1-5l-2-1z" />
      <path d="M7 11.5s-2 1-3 3c2 0 3-1 3-3z" fill="currentColor" />

```

```

        <path d="M18 7.5l2-1.5v2" />
        <circle cx="16" cy="9" r="1" fill="currentColor" />
      </svg>
    );
    case 'Owl':
    return (
      <svg className={` ${className} text-amber-800`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
        <rect x="6" y="4" width="12" height="16" rx="4" />
        <circle cx="9.5" cy="9.5" r="2" />
        <circle cx="9.5" cy="9.5" r="0.5" fill="currentColor" />
        <circle cx="14.5" cy="9.5" r="2" />
        <circle cx="14.5" cy="9.5" r="0.5" fill="currentColor" />
        <path d="M12 11l-1 2h2l-1-2z" fill="currentColor" />
        <path d="M8 18c1-1 2-1 4-1s3 0 4 1" />
        <path d="M6 5L4 7M18 5l2 7" />
      </svg>
    );
    default:
    return (
      <svg className={` ${className} text-slate-400`} viewBox="0 0 24 24" fill="none"
stroke="currentColor">
        <circle cx="12" cy="12" r="10" strokeWidth="2" />
        <path d="M8 12h8M12 8v8" strokeWidth="2" />
      </svg>
    );
  }
};

```

```

export default function App() {
  // =====
  // 📁 STATE & CONSTANTS SETUP
  // =====
  const [activeTab, setActiveTab] = useState('lab');
  const [selectedChain, setSelectedChain] = useState('meadow');

  // Game state
  const [xp, setXp] = useState(0);
  const [level, setLevel] = useState(1);
  const [completedChains, setCompletedChains] = useState({
    meadow: false,
    ocean: false,
    forest: false
  });
};

```

```

// Active chain states
const [slots, setSlots] = useState([null, null, null, null]);
const [availableCards, setAvailableCards] = useState([]);
const [feedback, setFeedback] = useState({ text: "Place the energy builder (the source of all life!) in the first slot.", type: "hint" });
const [isChainValid, setIsChainValid] = useState(false);
const [showConfetti, setShowConfetti] = useState(false);

// Biome-specific configuration
const biomes = {
  meadow: {
    title: "🌞 Sunny Meadow",
    color: "from-emerald-50 to-green-100 border-emerald-200",
    accent: "emerald",
    organisms: [
      { id: 'sun', name: 'Sun', role: 'Energy Source', desc: 'The starting energy for all life.' },
      { id: 'grass', name: 'Grass', role: 'Producer', desc: 'Makes food using sunlight.' },
      { id: 'hopper', name: 'Grasshopper', role: 'Herbivore', desc: 'Eats grass for energy.' },
      { id: 'frog', name: 'Frog', role: 'Carnivore', desc: 'Snaps up insects for energy.' }
    ],
    correctOrder: ['sun', 'grass', 'hopper', 'frog'],
    hints: [
      "Every food chain starts with a beaming source of heat and light!",
      "Next, what green organism drinks sunlight to make its own sweet sugars?",
      "Now look for a small jumping insect that loves to munch on green grass leaves.",
      "Lastly, who is sitting quietly in the weeds waiting to snap up that grasshopper?"
    ]
  },
  ocean: {
    title: "🌊 Crystal Ocean",
    color: "from-sky-50 to-blue-100 border-sky-200",
    accent: "sky",
    organisms: [
      { id: 'sun', name: 'Sun', role: 'Energy Source', desc: 'Heats the planet and feeds the sea.' },
      { id: 'algae', name: 'Algae (Pond Plant)', role: 'Producer', desc: 'Microscopic plants floating on the surface.' },
      { id: 'fish', name: 'Small Fish', role: 'Herbivore/Consumer', desc: 'Gathers energy by eating green ocean algae.' },
      { id: 'seal', name: 'Seal', role: 'Carnivore', desc: 'Dives deep to chase down small tasty fish.' }
    ],
    correctOrder: ['sun', 'algae', 'fish', 'seal'],
    hints: [

```



```

    "Even ocean food chains require solar rays to kickstart their energy cycles!",
    "What tiny, leafy green organisms drift in the ocean, creating oxygen and food?",
    "Who is swimming by, looking for a school of algae to munch on?",
    "What sleek creature with flippers loves catching fish in cold waters?"
  ]
},
forest: {
  title: "🌲 Deep Oak Forest",
  color: "from-amber-50 to-orange-100 border-amber-200",
  accent: "amber",
  organisms: [
    { id: 'sun', name: 'Sun', role: 'Energy Source', desc: 'Shines through the thick forest canopy.' },
    { id: 'oak', name: 'Oak Tree', role: 'Producer', desc: 'Produces massive green leaves and acorns.' },
    { id: 'caterpillar', name: 'Caterpillar', role: 'Herbivore', desc: 'Crawls slowly, chewing on oak leaves.' },
    { id: 'owl', name: 'Owl', role: 'Carnivore', desc: 'A nocturnal hunter with super sharp vision.' }
  ],
  correctOrder: ['sun', 'oak', 'caterpillar', 'owl'],
  hints: [
    "Sunlight is always the master charger of the food web!",
    "What giant woodland producer extends its branches high to catch solar rays?",
    "Which wiggly crawler feeds directly on fresh green oak leaves?",
    "What wise nocturnal predator watches the tree branches for scurrying bugs?"
  ]
}
};

```

```
// Set up cards when biome changes or reset occurs
```

```

useEffect(() => {
  resetLevel();
}, [selectedChain]);

```

```

const resetLevel = () => {
  const biome = biomes[selectedChain];
  // Shuffle organisms for available card dock
  const shuffled = [...biome.organisms].sort(() => Math.random() - 0.5);
  setAvailableCards(shuffled);
  setSlots([null, null, null, null]);
  setFeedback({ text: biome.hints[0], type: "hint" });
  setIsChainValid(false);
  setShowConfetti(false);
};

```

```

// Drag-and-drop & Click interactions
const handleClick = (card) => {
  // Find first empty slot
  const emptyIndex = slots.findIndex(slot => slot === null);
  if (emptyIndex !== -1) {
    placeCard(card, emptyIndex);
  }
};

const placeCard = (card, index) => {
  const newSlots = [...slots];
  // If there is already a card in that slot, return it to the dock first
  if (newSlots[index]) {
    returnCardToDock(newSlots[index]);
  }
  newSlots[index] = card;
  setSlots(newSlots);
  setAvailableCards(prev => prev.filter(c => c.id !== card.id));
  checkIncrementalPlacement(newSlots, index);
};

const returnCardToDock = (card) => {
  setAvailableCards(prev => [...prev, card]);
};

const removeCardFromSlot = (index) => {
  const cardToRemove = slots[index];
  if (cardToRemove) {
    const newSlots = [...slots];
    newSlots[index] = null;
    setSlots(newSlots);
    returnCardToDock(cardToRemove);

    // Recalculate guidance feedback
    const nextEmpty = newSlots.findIndex(s => s === null);
    if (nextEmpty !== -1) {
      setFeedback({ text: biomes[selectedChain].hints[nextEmpty], type: "hint" });
    }
    setIsChainValid(false);
  }
};

// Continuous micro-feedback

```

```

const checkIncrementalPlacement = (currentSlots, placedIndex) => {
  const correctSeq = biomes[selectedChain].correctOrder;

  // Check if what is placed so far is correct
  let isCorrectSoFar = true;
  for (let i = 0; i <= placedIndex; i++) {
    if (currentSlots[i] && currentSlots[i].id !== correctSeq[i]) {
      isCorrectSoFar = false;
      break;
    }
  }

  if (!isCorrectSoFar) {
    // Trigger dynamic hints for misplaced blocks
    if (placedIndex === 0 && currentSlots[0].id !== 'sun') {
      setFeedback({
        text: `Oops! "${currentSlots[0].name}" is awesome, but does it create raw energy? Hint:
Look for our solar superstar`,
        type: "warning"
      });
    } else {
      setFeedback({
        text: "Hmm, the flow of energy looks blocked. Check the order of who gives energy to
whom!",
        type: "warning"
      });
    }
  } else {
    // Correct placement so far
    const nextEmpty = currentSlots.findIndex(s => s === null);
    if (nextEmpty !== -1) {
      setFeedback({
        text: `Fantastic! ${currentSlots[placedIndex].name} is in the right spot! ` +
biomes[selectedChain].hints[nextEmpty],
        type: "success"
      });
    } else {
      // All filled and looks perfect!
      setFeedback({
        text: "Excellent work! All organisms are linked. Press 'Verify Energy Flow' to send the
sparks!",
        type: "success"
      });
    }
  }
}

```

```

    }
  };

  // Submit and verify final food chain
  const verifyChain = () => {
    const correctSeq = biomes[selectedChain].correctOrder;
    const finalMatches = slots.every((slot, idx) => slot && slot.id === correctSeq[idx]);

    if (finalMatches) {
      setIsChainValid(true);
      setShowConfetti(true);
      setFeedback({
        text: "🎉 SUCCESS! The energy flows seamlessly from left to right. You earned +50 XP  
and a Biome Explorer Badge!",
        type: "success"
      });
      // Award XP
      if (!completedChains[selectedChain]) {
        setXp(prev => {
          const nextXp = prev + 50;
          if (nextXp >= 100) {
            setLevel(l => l + 1);
            return nextXp - 100;
          }
          return nextXp;
        });
        setCompletedChains(prev => ({ ...prev, [selectedChain]: true }));
      }
    } else {
      setFeedback({
        text: "The energy line broke! Remember: Sun goes first, then the plant (producer), then  
herbivores, and carnivores last.",
        type: "error"
      });
    }
  };

  // =====
  // 🛠 INTERACTIVE QUIZ CONFIGURATION
  // =====
  const quizQuestions = [
    {
      id: 1,
      type: 'mcq',

```

```

q: "Where does the energy in nearly all food chains start?",
options: [
  { label: "A) High nutrients in the soil", val: "soil" },
  { label: "B) Light and heat from the Sun", val: "sun" },
  { label: "C) Fresh spring rain water", val: "water" },
  { label: "D) Oxygen in the atmosphere", val: "oxygen" }
],
correct: "sun",
explanation: "The Sun is the source of all life! Plants utilize solar energy during
photosynthesis to generate food that animals eat."
},
{
  id: 2,
  type: 'tf',
  q: "A fierce lion is a carnivore, so it does not depend on grass or plants to survive at all.",
  options: [
    { label: "True", val: "true" },
    { label: "False", val: "false" }
  ],
  correct: "false",
  explanation: "False! If there were no plants, herbivores like deer would starve. If deer starve,
predators like lions would have no food either!"
},
{
  id: 3,
  type: 'mcq',
  q: "What does the arrow in a food chain (e.g., Grass → Deer) represent?",
  options: [
    { label: "A) The direction the animals like to run", val: "run" },
    { label: "B) 'Is eaten by' (Energy is transferred to)", val: "energy" },
    { label: "C) The animal that is bigger and stronger", val: "size" },
    { label: "D) Which organism is faster at chasing", val: "speed" }
  ],
  correct: "energy",
  explanation: "The arrow indicates the direction of energy transfer! Energy from the Grass
goes INTO the stomach of the Deer."
},
{
  id: 4,
  type: 'scenario',
  q: "Scenario: A gardener sprays eco-friendly soap that accidentally chases away all
Grasshoppers in a meadow. Who suffers first?",
  options: [
    { label: "A) The Green Grass (it grows wild without grazers)", val: "grass" },

```

```

    { label: "B) The Frogs (they lose their primary food source)", val: "frogs" },
    { label: "C) The Hawks flying high in the sky", val: "hawks" },
    { label: "D) The earthworms living in the damp dirt", val: "worms" }
  ],
  correct: "frogs",
  explanation: "Frogs are the immediate consumers that eat grasshoppers. If their food
disappears, they suffer first!"
},
{
  id: 5,
  type: 'fill',
  q: "Living organisms that can synthesize their own food using sunlight are called:
_____.",
  options: [
    { label: "A) Consumers", val: "consumers" },
    { label: "B) Decomposers", val: "decomposers" },
    { label: "C) Producers", val: "producers" },
    { label: "D) Predators", val: "predators" }
  ],
  correct: "producers",
  explanation: "Plants are 'Producers' because they produce energy out of simple solar light
and water!"
},
{
  id: 6,
  type: 'mcq',
  q: "If a plant stores 100 units of energy from the Sun, how much of that is generally passed
up to the herbivore that eats it?",
  options: [
    { label: "A) Exactly 100% of the energy", val: "100" },
    { label: "B) About 50% of the energy", val: "50" },
    { label: "C) Only about 10% of the energy", val: "10" },
    { label: "D) 0% (energy disappears completely)", val: "0" }
  ],
  correct: "10",
  explanation: "This is known as the '10% Rule'. Organisms use up 90% of their energy to
grow, run around, breathe, and stay warm. Only 10% transfers to the next consumer!"
},
{
  id: 7,
  type: 'mcq',
  q: "Which pond-based chain is in the correct order?",
  options: [
    { label: "A) Small Fish → Sun → Pond Algae → Crane", val: "a" },

```

```

    { label: "B) Sun → Pond Algae → Small Fish → Crane", val: "b" },
    { label: "C) Crane → Small Fish → Pond Algae → Sun", val: "c" },
    { label: "D) Pond Algae → Sun → Crane → Small Fish", val: "d" }
  ],
  correct: "b",
  explanation: "The correct biological sequence starts with Sun (Energy) → Algae (Producer)
→ Small Fish (Herbivore) → Crane (Carnivore).",
},
{
  id: 8,
  type: 'scenario',
  q: "If an animal eats both berries (plants) and salmon (meat/fish), what is its scientific
ecological classification?",
  options: [
    { label: "A) Carnivore", val: "carnivore" },
    { label: "B) Herbivore", val: "herbivore" },
    { label: "C) Omnivore", val: "omnivore" },
    { label: "D) Decomposer", val: "decomposer" }
  ],
  correct: "omnivore",
  explanation: "Omni means 'all/everything'. Omnivores (like brown bears and humans)
happily feed on both producers and other consumers.",
},
{
  id: 9,
  type: 'tf',
  q: "A food chain can easily support 25 different levels of animals in a line.",
  options: [
    { label: "True", val: "true" },
    { label: "False", val: "false" }
  ],
  correct: "false",
  explanation: "False! Because energy drops so sharply at every single step, food chains
rarely exceed 4 or 5 steps before running completely out of fuel.",
},
{
  id: 10,
  type: 'mcq',
  q: "What helpful creatures clean up the forest floor by breaking down old logs and dead
leaves?",
  options: [
    { label: "A) Top Predators", val: "predators" },
    { label: "B) Decomposers (like Mushrooms & Bacteria)", val: "decomposers" },
    { label: "C) Herbivore Insects", val: "herbivores" },

```

```

      { label: "D) Sun Rays", val: "sun" }
    ],
    correct: "decomposers",
    explanation: "Decomposers act as nature's recycling squad! They recycle dead materials
back into soil nutrients, which help brand-new green plants grow."
  }
];

```

```

const [currentQuestionIdx, setCurrentQuestionIdx] = useState(0);
const [selectedAnswer, setSelectedAnswer] = useState(null);
const [quizScore, setQuizScore] = useState(0);
const [showExplanation, setShowExplanation] = useState(false);
const [quizFinished, setQuizFinished] = useState(false);

```

```

const handleAnswerSelect = (optionValue) => {
  if (showExplanation) return; // Prevent clicking after submit
  setSelectedAnswer(optionValue);
};

```

```

const submitAnswer = () => {
  if (!selectedAnswer) return;
  if (selectedAnswer === quizQuestions[currentQuestionIdx].correct) {
    setQuizScore(prev => prev + 1);
  }
  setShowExplanation(true);
};

```

```

const nextQuestion = () => {
  setSelectedAnswer(null);
  setShowExplanation(false);
  if (currentQuestionIdx < quizQuestions.length - 1) {
    setCurrentQuestionIdx(prev => prev + 1);
  } else {
    setQuizFinished(true);
    // Give bonus XP for finishing quiz
    setXp(prev => {
      const nextXp = prev + 30;
      if (nextXp >= 100) {
        setLevel(l => l + 1);
        return nextXp - 100;
      }
      return nextXp;
    });
  }
};

```



```

};

const restartQuiz = () => {
  setCurrentQuestionIdx(0);
  setSelectedAnswer(null);
  setQuizScore(0);
  setShowExplanation(false);
  setQuizFinished(false);
};

// =====
// 🔄 CONCEPTS: ARROW CORRECTOR MINI-GAME
// =====
const [arrowCorrect, setArrowCorrect] = useState(null);

return (
  <div className="min-h-screen bg-slate-50 text-slate-800 font-sans flex flex-col antialiased">
    {/* =====
      Header component
      ===== */}
    <header className="sticky top-0 z-30 bg-white border-b border-slate-200 shadow-sm py-4
px-6 flex flex-wrap justify-between items-center gap-4">
      <div className="flex items-center space-x-3">
        <div className="bg-emerald-500 text-white p-2.5 rounded-2xl shadow-md
shadow-emerald-200 flex items-center justify-center">
          <svg className="w-6 h-6 animate-spin-slow" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2.5" d="M12 3v1m0
16v1m9-9h-1M4 12H3m15.364-6.364l-.707.707M6.343
17.657l-.707.707m0-12.728l.707.707m12.728 12.728l.707.707M12 8a4 4 0 100 8 4 4 0 000-8z"
/>
          </svg>
        </div>
        <div>
          <h1 className="text-xl font-bold tracking-tight text-slate-800">The Energy Linker</h1>
          <p className="text-xs font-semibold text-emerald-600 tracking-wide uppercase">Class
3–8 • Interactive Biology Lab</p>
        </div>
      </div>
    </header>

    {/* Gamification Dashboard */}
    <div className="flex items-center space-x-4">
      {/* Level badge */}

```

```

    <div className="bg-gradient-to-r from-emerald-500 to-teal-600 text-white font-bold
py-1.5 px-4 rounded-xl flex items-center gap-1.5 shadow-md">
      <span className="text-xs text-emerald-100">LEVEL</span>
      <span className="text-lg">{level}</span>
    </div>
    { /* XP Progress bar */ }
    <div className="w-36 sm:w-48 bg-slate-100 h-3 rounded-full overflow-hidden relative
border border-slate-200">
      <div
        className="bg-gradient-to-r from-emerald-400 to-emerald-600 h-full transition-all
duration-500 ease-out"
        style={{ width: `${xp}%` }}
      />
      <span className="absolute inset-0 flex items-center justify-center text-[9px]
font-extrabold text-slate-700">
        {xp}/100 XP
      </span>
    </div>
    { /* Active Badge list */ }
    <div className="hidden md:flex items-center space-x-2 text-xs">
      {completedChains.meadow && <span className="bg-emerald-100 text-emerald-800
py-1 px-2.5 rounded-full font-bold shadow-sm"> 🌿 Meadow Pro</span>}
      {completedChains.ocean && <span className="bg-sky-100 text-sky-800 py-1 px-2.5
rounded-full font-bold shadow-sm"> 🌊 Ocean Pro</span>}
      {completedChains.forest && <span className="bg-amber-100 text-amber-800 py-1
px-2.5 rounded-full font-bold shadow-sm"> 🌲 Forest Pro</span>}
    </div>
  </div>
</header>

{ /* =====
Navigation Tabs
===== */ }
<nav className="bg-slate-100 border-b border-slate-200 py-2 px-6 flex overflow-x-auto
space-x-2 scrollbar-none">
  <button
    onClick={() => setActiveTab('lab')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
transition-all whitespace-nowrap ${
      activeTab === 'lab'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
      }` }
  />

```

```

    <span>🔬 Lab Simulator</span>
  </button>
  <button
    onClick={() => setActiveTab('learn')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'learn'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span>📖 Learn Concept</span>
  </button>
  <button
    onClick={() => setActiveTab('diy')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'diy'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span>🔧 Backyard DIY</span>
  </button>
  <button
    onClick={() => setActiveTab('quiz')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'quiz'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span>👉 Interactive Quiz</span>
  </button>
  <button
    onClick={() => setActiveTab('info')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'info'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >

```

```

    <span>💡 Facts & FAQs</span>
  </button>
  <button
    onClick={() => setActiveTab('teacher_parent')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'teacher_parent'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span>🍎 Educator & Parent Zone</span>
  </button>
</nav>

{
  /* =====
  MAIN WORKSPACE & LAYOUT CONTAINER
  ===== */
  <main className="flex-1 p-4 md:p-6 max-w-7xl mx-auto w-full">

    {
      /* =====
      TAB: 🧪 LAB SIMULATOR
      ===== */
      {activeTab === 'lab' && (
        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">

          {
            /* LEFT AREA: Simulator Canvas (approx 70% equivalent) */
            <div className="lg:col-span-2 flex flex-col space-y-4">

              {
                /* Biome Picker */
                <div className="bg-white rounded-3xl p-4 shadow-sm border border-slate-100 flex
  flex-wrap justify-between items-center gap-3">
                  <span className="text-sm font-black uppercase text-slate-400
  tracking-wider">Choose Ecosystem Biome:</span>
                  <div className="flex space-x-2">
                    {Object.keys(biomes).map((biomeKey) => (
                      <button
                        key={biomeKey}
                        onClick={() => setSelectedChain(biomeKey)}
                        className={`px-4 py-2 rounded-2xl text-xs font-extrabold border-2 transition-all ${
                          selectedChain === biomeKey
                            ? 'border-emerald-500 bg-emerald-50 text-emerald-800'
                            : 'border-slate-100 bg-slate-50 text-slate-600 hover:border-slate-300'
                        }}
                      >
                    </button>
                  </div>
                </div>
              )}
            </div>
          )}
        </div>
      )}
    )}
  </main>
}

```

```

    >
      {biomes[biomeKey].title}
    </button>
  )))
</div>
</div>

{ /* Main Interactive Canvas */}
<div className={`relative flex-1 min-h-[380px] bg-gradient-to-b
${biomes[selectedChain].color} rounded-3xl border-3 p-6 flex flex-col justify-between
transition-all duration-500 overflow-hidden`}>

  { /* Visual Background Accent / Floating Sun */}
  <div className="absolute top-4 right-4 pointer-events-none opacity-20">
    <OrganismIcon name="Sun" className="w-32 h-32 text-amber-300" />
  </div>

  { /* Sub-Header inside Canvas */}
  <div className="z-10 flex justify-between items-start">
    <div>
      <h2 className="text-2xl font-black text-slate-800 flex items-center gap-2">
        {biomes[selectedChain].title} Lab
      </h2>
      <p className="text-sm font-semibold text-slate-600 mt-0.5">
        Drag or click the organism cards from the dock to build the flow of life!
      </p>
    </div>
    <button
      onClick={resetLevel}
      className="bg-white/80 hover:bg-white text-slate-700 py-1.5 px-3 rounded-2xl
font-bold text-xs flex items-center gap-1 shadow-sm border border-slate-200 transition-all
active:scale-95"
    >
       Clear Slots
    </button>
  </div>

  { /* SLOTS CONTAINER (THE CHAIN PATHWAY) */}
  <div className="my-10 flex flex-col sm:flex-row items-center justify-center gap-2
sm:gap-4 z-10">
    {slots.map((card, idx) => (
      <React.Fragment key={idx}>
        { /* Slot Card */}
        <div

```

```

        onClick={() => removeCardFromSlot(idx)}
        className={`group relative w-28 h-40 rounded-3xl flex flex-col items-center
justify-center border-4 border-dashed cursor-pointer transition-all ${
        card
        ? 'bg-white border-emerald-400 shadow-md hover:scale-105 active:scale-95
hover:border-red-400'
        : 'bg-slate-50/50 hover:bg-slate-50/80 border-slate-300 hover:border-slate-400
shadow-inner'
        }}
    >
    {card ? (
    <>
    <div className="absolute top-1.5 right-1.5 bg-slate-100
group-hover:bg-red-100 text-slate-400 group-hover:text-red-500 rounded-full w-5 h-5 flex
items-center justify-center text-[10px] font-bold transition-colors">
    X
    </div>
    <OrganismIcon name={card.name} className="w-14 h-14" />
    <span className="text-center text-xs font-black text-slate-700 mt-2 px-1
line-clamp-1">{card.name}</span>
    <span className="text-[9px] font-extrabold text-emerald-600 bg-emerald-50
px-2 py-0.5 rounded-full mt-1 uppercase">
    {card.role}
    </span>
    </>
    ) : (
    <div className="flex flex-col items-center justify-center text-center p-2">
    <span className="text-xl text-slate-300">📦</span>
    <span className="text-[10px] font-bold text-slate-400 mt-1 uppercase">Slot
{idx + 1}</span>
    </div>
    )}
    </div>

    {/* Sparking Connector Arrow */}
    {idx < 3 && (
    <div className="flex items-center justify-center py-2 sm:py-0">
    {isChainValid ? (
    <div className="flex flex-col items-center animate-bounce">
    <span className="text-lg font-extrabold text-amber-500
animate-pulse">⚡</span>
    <svg className="w-8 h-8 text-amber-400" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="3">

```

```

        <path strokeLinecap="round" strokeLinejoin="round" d="M14 5l7 7m0 0l-7
7m7-7H3" />
      </svg>
    </div>
  ) : (
    <svg className="w-8 h-8 text-slate-400 transform rotate-90 sm:rotate-0"
fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth="2.5">
      <path strokeLinecap="round" strokeLinejoin="round" d="M14 5l7 7m0 0l-7
7m7-7H3" />
    </svg>
  )}
</div>
)}
</React.Fragment>
)})
</div>

```

```

{ /* BOTTOM COMPONENT: Card Dock */
  <div className="bg-white/80 backdrop-blur-md rounded-2xl p-4 shadow-md border
border-slate-100 z-10">
    <p className="text-xs font-bold text-slate-400 uppercase tracking-wider
mb-2">Available Organisms Dock (Click to place):</p>
    <div className="flex flex-wrap justify-center gap-3">
      {availableCards.length === 0 && slots.every(s => s !== null) ? (
        <p className="text-xs font-bold text-emerald-600 animate-pulse">All cards
placed! Tap 'Verify Energy Flow' below! 🖱️</p>
      ) : (
        availableCards.map((card) => (
          <button
            key={card.id}
            onClick={() => handleCardClick(card)}
            className="bg-white border-2 border-slate-100 hover:border-emerald-300
rounded-2xl p-3 flex flex-col items-center w-24 shadow-sm hover:shadow-md transform
hover:-translate-y-1 transition-all active:scale-95"
          >
            <OrganismIcon name={card.name} className="w-10 h-10 mb-1" />
            <span className="text-[10px] font-black text-slate-700 text-center
line-clamp-1">{card.name}</span>
          </button>
        ))
      )}
    </div>
  </div>
}

```

```

</div>

{/* ACTION FOOTER */}
<div className="flex justify-between items-center bg-white p-4 rounded-3xl
shadow-sm border border-slate-100">
  <button
    onClick={resetLevel}
    className="bg-slate-100 hover:bg-slate-200 text-slate-700 font-extrabold py-3 px-6
rounded-2xl text-sm transition-all"
  >
    Reset Level
  </button>
  <button
    onClick={verifyChain}
    disabled={slots.some(s => s === null)}
    className={`font-black py-3 px-8 rounded-2xl text-sm transition-all shadow-md ${
      slots.some(s => s === null)
        ? 'bg-slate-200 text-slate-400 cursor-not-allowed shadow-none'
        : 'bg-gradient-to-r from-emerald-500 to-teal-600 text-white hover:brightness-110
active:scale-95'
      }`}
  >
    ⚡ Verify Energy Flow
  </button>
</div>

```

```

</div>

{/* RIGHT PANEL: Live Observation & Guidelines (approx 30% equivalent) */}
<div className="flex flex-col space-y-4">

  {/* Guidance & Assistant Character Box */}
  <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-100
relative overflow-hidden">
    <div className="flex items-center space-x-3 mb-4">
      <div className="bg-amber-100 text-amber-600 w-12 h-12 rounded-2xl flex
items-center justify-center text-2xl font-bold">
        🍀
      </div>
      <div>
        <h3 className="font-black text-slate-800">Leafy the Assistant</h3>
        <p className="text-xs font-semibold text-slate-400">Your Ecosystem
Companion</p>
      </div>
    </div>
  </div>

```


</div>

{/* Assistant Chat Bubble */}

```
<div className={`p-4 rounded-2xl text-sm font-semibold border-l-4 leading-relaxed ${
  feedback.type === 'success'
    ? 'bg-emerald-50 border-emerald-500 text-emerald-800'
    : feedback.type === 'warning'
    ? 'bg-amber-50 border-amber-500 text-amber-800 animate-shake'
    : feedback.type === 'error'
    ? 'bg-red-50 border-red-500 text-red-800'
    : 'bg-sky-50 border-sky-400 text-sky-800'
}`}>
  {feedback.text}
</div>
```

{/* Success celebration box */}

```
{showConfetti && (
  <div className="mt-4 bg-emerald-500 text-white p-4 rounded-2xl text-center
shadow-lg animate-bounce">
    <p className="font-extrabold text-base">★ EXCELLENT ECOLOGIST! ★</p>
    <p className="text-xs font-semibold opacity-90 mt-1">
      You mapped the flow of organic energy perfectly. Select another Biome above to
unlock more badges!
    </p>
  </div>
)}
```

{/* Dynamic Organism Card Explainer */}

```
<div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-100
flex-1">
  <h3 className="font-black text-slate-800 mb-3 text-sm uppercase tracking-wider
text-slate-400">Organism Guidebook</h3>
  <div className="space-y-4 max-h-[300px] overflow-y-auto pr-1">
    {biomes[selectedChain].organisms.map((org) => {
      const isPlaced = slots.some(slot => slot && slot.id === org.id);
      return (
        <div
          key={org.id}
          className={`p-3 rounded-2xl border-2 transition-colors ${
            isPlaced ? 'bg-emerald-50/50 border-emerald-100' : 'bg-slate-50
border-slate-100'
          }`}
        >
```

```

    <div className="flex items-center space-x-2">
      <OrganismIcon name={org.name} className="w-8 h-8" />
      <div>
        <span className="font-black text-xs text-slate-700">{org.name}</span>
        <span className="ml-2 text-[9px] font-extrabold px-1.5 py-0.5 bg-slate-200
text-slate-600 rounded">
          {org.role}
        </span>
      </div>
    </div>
    <div>
      <p className="text-xs font-semibold text-slate-500 mt-1.5
leading-relaxed">{org.desc}</p>
    </div>
  );
  }}}
</div>
</div>

</div>

</div>
)}}

{ /* =====
  TAB: 📖 CONCEPT EXPLANATION
  ===== */ }
{activeTab === 'learn' && (
  <div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100
max-w-4xl mx-auto space-y-8">
    <div className="text-center max-w-2xl mx-auto">
      <span className="bg-emerald-100 text-emerald-800 font-extrabold px-3 py-1
rounded-full text-xs uppercase">Interactive Textbook</span>
      <h2 className="text-3xl font-black text-slate-800 mt-2">Who Eats Whom In
Nature?</h2>
      <p className="text-slate-500 mt-2 font-semibold">
        Discover the elegant transfer of organic power that keeps the animal kingdom alive.
      </p>
    </div>

    { /* Step-by-Step Learning Grid */ }
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6">

      <div className="bg-emerald-50/50 p-6 rounded-2xl border border-emerald-100
space-y-3">

```

```
<div className="bg-emerald-100 text-emerald-800 w-10 h-10 rounded-xl flex
items-center justify-center text-lg font-bold">🌱 </div>
```

```
<h3 className="font-black text-slate-800">1. The Producers</h3>
```

```
<p className="text-sm font-semibold text-slate-600 leading-relaxed">
```

Plants are the superstars of our food webs. Using green chlorophyll in their leaves, they capture raw solar beams and turn water and air into edible sugars! They are called **Producers** because they produce food from scratch.

```
</p>
```

```
</div>
```

```
<div className="bg-sky-50/50 p-6 rounded-2xl border border-sky-100 space-y-3">
```

```
<div className="bg-sky-100 text-sky-800 w-10 h-10 rounded-xl flex items-center
justify-center text-lg font-bold">🦌 </div>
```

```
<h3 className="font-black text-slate-800">2. The Consumers</h3>
```

```
<p className="text-sm font-semibold text-slate-600 leading-relaxed">
```

Animals can't make food inside their bodies. To charge up their muscles and breathe, they must consume plants or other animals. Thus, they are called **Consumers**! They are split into:

```
</p>
```

```
<ul className="text-xs font-bold text-slate-500 space-y-1 pl-4 list-disc">
```

```
<li><span className="text-emerald-600">Herbivores</span> (Plant-eaters like
deer, caterpillars)</li>
```

```
<li><span className="text-amber-700">Carnivores</span> (Meat-eaters like tigers,
hawks)</li>
```

```
<li><span className="text-blue-700">Omnivores</span> (Eaters of both like bears,
humans)</li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
{/* INTERACTIVE COMPONENT: Arrow Direction Corrector Widget */}
```

```
<div className="bg-slate-50 rounded-2xl p-6 border border-slate-200">
```

```
<h3 className="font-black text-slate-800 flex items-center gap-1.5 text-base">
```

```
⚡ Interactive Challenge: The Arrow Confuser!
```

```
</h3>
```

```
<p className="text-xs font-semibold text-slate-500 mt-1 leading-relaxed">
```

Many students construct arrows backward. Try clicking the toggles below to see the biological truth of arrows!

```
</p>
```

```
<div className="mt-4 bg-white p-4 rounded-xl border border-slate-200 flex flex-col
md:flex-row items-center justify-around gap-4 text-center">
```

```

    { /* Visual Representation of Arrows */ }
    <div className="flex items-center space-x-3 font-bold">
      <span className="p-2 bg-emerald-50 rounded text-sm">🌿 Grass</span>
      <div className="flex flex-col items-center">
        <span className="text-xs text-slate-400 font-extrabold">Arrow points:</span>
        <span className={`text-xl font-black ${arrowCorrect === 'correct' ?
'text-emerald-600' : arrowCorrect === 'incorrect' ? 'text-red-500' : 'text-slate-300'}`}>
          {arrowCorrect === 'correct' ? '→ (To Deer)' : arrowCorrect === 'incorrect' ? '□ (To
Grass)' : '→ or □?'}
        </span>
      </div>
      <span className="p-2 bg-amber-50 rounded text-sm">🦌 Deer</span>
    </div>

    { /* Toggles */ }
    <div className="flex space-x-2">
      <button
        onClick={() => setArrowCorrect('incorrect')}
        className={`px-3 py-1.5 rounded-lg text-xs font-bold ${arrowCorrect ===
'incorrect' ? 'bg-red-500 text-white' : 'bg-slate-100 hover:bg-slate-200 text-slate-700'}`}
      >
        Deer eats Grass (□)
      </button>
      <button
        onClick={() => setArrowCorrect('correct')}
        className={`px-3 py-1.5 rounded-lg text-xs font-bold ${arrowCorrect === 'correct'
? 'bg-emerald-500 text-white' : 'bg-slate-100 hover:bg-slate-200 text-slate-700'}`}
      >
        Energy flows into Deer (→)
      </button>
    </div>

  </div>

  { /* Dynamic feedback on arrow corrector */ }
  {arrowCorrect && (
    <p className={`mt-3 text-xs font-bold p-3 rounded-lg ${
      arrowCorrect === 'correct'
        ? 'bg-emerald-50 text-emerald-800'
        : 'bg-red-50 text-red-800'
    }`}>
      {arrowCorrect === 'correct'
        ? "🌟 Correct! The arrow ALWAYS points to where the energy is going. The energy
flows FROM the grass INTO the stomach of the deer!"

```

: " ⚠️ Common Mistake! While the deer eats the grass, arrows in science show energy flow. If the arrow pointed left, it would mean energy goes into the grass!"

</p>

)}

</div>

{/* Summary Block */}

<div className="bg-amber-50/50 p-6 rounded-2xl border border-amber-100">

<h3 className="font-black text-amber-800 mb-2">💡 Quick Memory Rule:</h3>

<p className="text-sm font-semibold text-amber-900 leading-relaxed">

"Plants Produce (Producers). Animals Consume (Consumers). Decomposers Clean up!"

</p>

</div>

</div>

)}

{/* =====

TAB: 🛠️ BACKYARD DIY ACTIVITY

===== */}

{activeTab === 'diy' && (

<div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100 max-w-3xl mx-auto space-y-6">

<div className="flex items-center space-x-3 mb-4">

🔍

<div>

<h2 className="text-2xl font-black text-slate-800">My Backyard Food Chain Tracker</h2>

<p className="text-xs font-bold text-emerald-600 uppercase mt-0.5">Hands-On Home Science Experiment</p>

</div>

</div>

<p className="text-sm font-semibold text-slate-600 leading-relaxed">

Step away from the screen and search for real, living food chains happening outside your windows! Grab a notebook and your pencil.

</p>

<div className="space-y-4">

<h3 className="font-black text-slate-800 border-b pb-2 text-base">Steps to Accomplish:</h3>

<div className="flex items-start space-x-3">

```
<span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">1</span>
<div>
  <h4 className="font-bold text-slate-800 text-sm">Find Your Producer</h4>
  <p className="text-xs text-slate-500">Go into your garden or look out a window.
Find a green bush, tree, flower, or even simple weed grass. Draw it on the far-left side of your
page.</p>
</div>
</div>
```

```
<div className="flex items-start space-x-3">
  <span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">2</span>
  <div>
    <h4 className="font-bold text-slate-800 text-sm">Seek Out a Tiny Herbivore</h4>
    <p className="text-xs text-slate-500">Watch the leaves closely. Do you see any
ants, ladybugs, butterflies, caterpillars, or beetles? Draw the insect next to your plant.</p>
  </div>
</div>
```

```
<div className="flex items-start space-x-3">
  <span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">3</span>
  <div>
    <h4 className="font-bold text-slate-800 text-sm">Add the Energy Arrow</h4>
    <p className="text-xs text-slate-500">Draw a bold arrow from your plant pointing
directly to your insect. You just logged energy transfer!</p>
  </div>
</div>
```

```
<div className="flex items-start space-x-3">
  <span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">4</span>
  <div>
    <h4 className="font-bold text-slate-800 text-sm">Predict the Carnivore</h4>
    <p className="text-xs text-slate-500">What backyard creature might swoop down
to grab that insect? (A sparrow, a squirrel, a gecko, or a cat?) Sketch it on the right to complete
the chain!</p>
  </div>
</div>
</div>
```

```
<div className="bg-amber-50 p-4 rounded-2xl border border-amber-200 mt-6">
  <h4 className="font-extrabold text-amber-800 text-sm">⚠ Safety Rule:</h4>
```

```

    <p className="text-xs font-semibold text-amber-950 mt-1">
      Keep hands off wild bugs, spikes, and thorns! Keep your eyes sharp and observe
safely from a short distance.
    </p>
  </div>
</div>
})

{/* =====
TAB: 🛠 INTERACTIVE QUIZ
===== */}
{activeTab === 'quiz' && (
  <div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100
max-w-2xl mx-auto">

    {/* If Quiz is in progress */}
    {!quizFinished ? (
      <div className="space-y-6">

        {/* Progress bar */}
        <div className="flex justify-between items-center text-xs font-extrabold
text-slate-400 uppercase tracking-wider">
          <span>Question {currentQuestionIdx + 1} of {quizQuestions.length}</span>
          <span className="text-emerald-600">Correct:
{quizScore}/{currentQuestionIdx}</span>
        </div>
        <div className="w-full bg-slate-100 h-2 rounded-full overflow-hidden">
          <div
            className="bg-emerald-500 h-full transition-all duration-300"
            style={{ width: `${((currentQuestionIdx + 1) / quizQuestions.length) * 100}%` }}
          />
        </div>

        {/* Question */}
        <div className="space-y-2">
          <span className="bg-emerald-100 text-emerald-800 font-extrabold text-[10px]
px-2.5 py-1 rounded-full uppercase">
            {quizQuestions[currentQuestionIdx].type === 'mcq' ? 'Multiple Choice' :
quizQuestions[currentQuestionIdx].type === 'tf' ? 'True / False' : 'Scenario Challenge'}
          </span>
          <h3 className="text-lg font-black text-slate-800 leading-snug">
            {quizQuestions[currentQuestionIdx].q}
          </h3>
        </div>

```

```

    { /* Options List */ }
    <div className="space-y-3">
      {quizQuestions[currentQuestionIdx].options.map((opt) => {
        const isSelected = selectedAnswer === opt.val;
        const isCorrectOption = opt.val === quizQuestions[currentQuestionIdx].correct;

        let cardStyle = "border-slate-200 hover:border-slate-300 hover:bg-slate-50";
        if (isSelected) cardStyle = "border-emerald-500 bg-emerald-50 text-emerald-900";

        if (showExplanation) {
          if (isCorrectOption) {
            cardStyle = "border-emerald-500 bg-emerald-100 text-emerald-900 font-bold";
          } else if (isSelected && !isCorrectOption) {
            cardStyle = "border-red-500 bg-red-50 text-red-900";
          } else {
            cardStyle = "border-slate-200 opacity-60";
          }
        }

        return (
          <button
            key={opt.val}
            disabled={showExplanation}
            onClick={() => handleAnswerSelect(opt.val)}
            className={`w-full p-4 rounded-2xl border-2 text-left text-sm font-semibold
transition-all ${cardStyle}`}
          >
            {opt.label}
          </button>
        );
      })}
    </div>

    { /* Explanations block */ }
    {showExplanation && (
      <div className={`p-4 rounded-2xl text-xs leading-relaxed ${
        selectedAnswer === quizQuestions[currentQuestionIdx].correct
        ? 'bg-emerald-50 text-emerald-800 border border-emerald-200'
        : 'bg-red-50 text-red-800 border border-red-200'
      }`} >
        <p className="font-extrabold text-sm mb-1">
          {selectedAnswer === quizQuestions[currentQuestionIdx].correct ? '🎉 Correct!' :
'❌ Incorrect'}
        </p>
      </div>
    )}

```



```

    </p>
    <p>{quizQuestions[currentQuestionIdx].explanation}</p>
  </div>
)}

{/* Quiz controls */}
<div className="flex justify-end pt-4 border-t border-slate-100">
  {!showExplanation ? (
    <button
      onClick={submitAnswer}
      disabled={!selectedAnswer}
      className={`font-extrabold py-3 px-6 rounded-xl text-sm transition-all shadow ${
        !selectedAnswer
          ? 'bg-slate-200 text-slate-400 cursor-not-allowed shadow-none'
          : 'bg-emerald-500 text-white hover:brightness-105 active:scale-95'
      }`}
    >
      Submit Answer
    </button>
  ) : (
    <button
      onClick={nextQuestion}
      className="bg-emerald-600 hover:bg-emerald-700 text-white font-extrabold py-3
px-6 rounded-xl text-sm transition-all shadow active:scale-95"
    >
      {currentQuestionIdx < quizQuestions.length - 1 ? 'Next Question →' : 'View
Results'}
    </button>
  )}
</div>

): (
  // Quiz completion state
  <div className="text-center space-y-6 py-8">
    <span className="text-5xl">🏆</span>
    <h3 className="text-2xl font-black text-slate-800">Quiz Completed!</h3>
    <p className="text-slate-500 font-semibold max-w-md mx-auto">
      You scored <span className="text-emerald-600 font-bold">{quizScore}</span> out of
{quizQuestions.length}</span> questions correctly! You earned <strong
className="text-amber-500">+30 Energy XP</strong>!
    </p>

```

```

    <div className="inline-block bg-slate-50 p-4 rounded-2xl border border-slate-100
text-left max-w-sm w-full space-y-2">
      <div className="flex justify-between font-bold text-xs">
        <span>Ecosystem Rating:</span>
        <span className="text-emerald-600">
          {quizScore >= 8 ? '🌟 Master Ecologist' : quizScore >= 5 ? '🌿 Nature Ranger' :
'🌱 Budding Scientist'}
        </span>
      </div>
      <div className="w-full bg-slate-200 h-2 rounded-full overflow-hidden">
        <div
          className="bg-emerald-500 h-full"
          style={{ width: `${(quizScore / quizQuestions.length) * 100}%` }}
        />
      </div>
    </div>

    <div className="flex justify-center space-x-3 pt-4">
      <button
        onClick={restartQuiz}
        className="bg-slate-100 hover:bg-slate-200 text-slate-700 font-extrabold py-3
px-6 rounded-xl text-sm transition-all"
      >
        Try Again
      </button>
      <button
        onClick={() => setActiveTab('lab')}
        className="bg-emerald-500 hover:bg-emerald-600 text-white font-extrabold py-3
px-6 rounded-xl text-sm transition-all shadow"
      >
        Back to Simulator
      </button>
    </div>
  </div>
)}

</div>
)}

{ /* =====
  TAB: 💡 FACTS & FAQs
  ===== */ }
{activeTab === 'info' && (
  <div className="max-w-4xl mx-auto space-y-8">

```

```

    { /* Mindblowing Facts Section */ }
    <div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border
border-slate-100">
      <h2 className="text-2xl font-black text-slate-800 flex items-center gap-2 mb-6">
        <span>🤯</span> Mind-Blowing Food Chain Records
      </h2>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">

        <div className="bg-amber-50/50 p-5 rounded-2xl border border-amber-100
space-y-2">
          <h3 className="font-extrabold text-amber-800 text-sm">🚀 Super-Fast Light
Travel</h3>
          <p className="text-xs text-slate-600 leading-relaxed">
            It takes only about <strong>8 minutes and 20 seconds</strong> for light energy
leaving the Sun to slide through cold outer space, hit a green leaf on Earth, and begin the food
chain!
          </p>
        </div>

        <div className="bg-sky-50/50 p-5 rounded-2xl border border-sky-100 space-y-2">
          <h3 className="font-extrabold text-sky-800 text-sm">🐋 The Blue Whale
Paradox</h3>
          <p className="text-xs text-slate-600 leading-relaxed">
            The largest creature to ever live on Earth, the Blue Whale, has an incredibly short
food chain. It skips intermediate steps and eats small shrimp-like Krill directly!
          </p>
        </div>

        <div className="bg-emerald-50/50 p-5 rounded-2xl border border-emerald-100
space-y-2">
          <h3 className="font-extrabold text-emerald-800 text-sm">🕸 Food Web Safety
Nets</h3>
          <p className="text-xs text-slate-600 leading-relaxed">
            In real life, linear food chains overlap to form a giant safety network called a
<strong>Food Web</strong>. If one insect species declines, predators can adapt by eating
other insects.
          </p>
        </div>

        <div className="bg-purple-50/50 p-5 rounded-2xl border border-purple-100
space-y-2">
          <h3 className="font-extrabold text-purple-800 text-sm">🍄 Decomposers: The
Garbage Crew</h3>

```

```
<p className="text-xs text-slate-600 leading-relaxed">
  When high predators die, they are digested by mushrooms, worms, and bacteria.
  This converts their complex biology back into pure nitrogen-rich dirt, feeding new green grass!
</p>
</div>
```

```
</div>
</div>
```

```
{/* Student FAQs Section */}
<div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100
space-y-4">
  <h2 className="text-xl font-black text-slate-800"> ? Frequently Asked Questions
  (FAQs)</h2>
```

```
    <div className="space-y-3">
      <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200">
        <h4 className="font-black text-slate-800 text-sm">Q: Can a food chain go on
        forever?</h4>
```

```
        <p className="text-xs text-slate-600 mt-1 leading-relaxed">
          A: No! Because organisms use up nearly 90% of their energy just staying alive
          (breathing and moving), only 10% transfers to the next step. After 4 or 5 steps, there's not
          enough leftover fuel to support a bigger animal.
        </p>
      </div>
```

```
    <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200">
      <h4 className="font-black text-slate-800 text-sm">Q: What happens if we are
      omnivores? Where do we belong?</h4>
```

```
      <p className="text-xs text-slate-600 mt-1 leading-relaxed">
        A: Omnivores (like humans) occupy multiple positions at once! When you eat a
        juicy strawberry, you are a primary consumer. When you eat chicken or salmon, you are a
        secondary or tertiary consumer.
      </p>
    </div>
```

```
    <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200">
      <h4 className="font-black text-slate-800 text-sm">Q: Is dirt or water a
      producer?</h4>
```

```
      <p className="text-xs text-slate-600 mt-1 leading-relaxed">
        A: No. Water and soil are essential raw ingredients, but they are not alive. Only
        organisms with cells (like plants and green algae) are biological "producers".
      </p>
    </div>
```



```

    </div>
  </div>

  { /* Parent Kitchen Table Activities */ }
  <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-100
space-y-4">
    <div className="flex items-center space-x-2">
      <span className="text-2xl">🏠</span>
      <h2 className="text-lg font-black text-slate-800">Parent Table Conversations</h2>
    </div>
    <p className="text-xs text-slate-500 font-semibold leading-relaxed">
      Bring ecological awareness to daily conversations over meals or family walks.
    </p>

    <div className="space-y-3">
      <div className="border-l-4 border-amber-500 pl-3">
        <h4 className="font-extrabold text-xs text-slate-700 uppercase">The Dinner Plate
Trace</h4>
        <p className="text-xs text-slate-500 mt-1">
          Pick one item from tonight's dinner plate (e.g., a slice of potato or chicken). Trace it
back in steps to the field, the insects, and the shining Sun.
        </p>
      </div>

      <div className="border-l-4 border-amber-500 pl-3">
        <h4 className="font-extrabold text-xs text-slate-700 uppercase">Observation
Walk</h4>
        <p className="text-xs text-slate-500 mt-1">
          During weekend park outings, search for birds pulling worms out of lawns, or bees
collecting sweet pollen from wild flowers.
        </p>
      </div>
    </div>
  </div>
)}

</main>

{ /* =====
Footer with Copyright / Metadata
===== */ }

```

```
<footer className="bg-slate-900 text-slate-400 py-6 px-6 text-center text-xs mt-12 border-t border-slate-800">
```

```
<p className="font-semibold">The Energy Linker • Premium Interactive EdTech Simulator</p>
```

```
<p className="opacity-60 mt-1">Built in compliance with CBSE/NCERT curriculum and EdTech child-psychology guidelines.</p>
```

```
</footer>
```

```
</div>
```

```
);
```

```
}
```

Food-Chain


```

import React, { useState, useEffect } from 'react';

// =====
// 🎨 SVGs & VECTOR GRAPHICS (PREMIUM DESIGN)
// =====

// Custom Inline SVG Organism Icons for High Fidelity
const OrganismIcon = ({ name, className = "w-16 h-16" }) => {
  switch (name) {
    case 'Sun':
      return (
        <svg className={` ${className} text-amber-400 animate-pulse`} viewBox="0 0 24 24"
fill="currentColor">
          <circle cx="12" cy="12" r="5" />
          <path d="M12 1v2M12 21v2M4.22 4.22l1.42 1.42M18.36 18.36l1.42 1.42M1 12h2M21 12h2M4.22 19.78l1.42-1.42M18.36 5.64l1.42-1.42" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" />
        </svg>
      );
    case 'Grass':
      return (
        <svg className={` ${className} text-emerald-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2">
          <path d="M3 22c1-5 3-9 7-11M10 22c2-7 5-11 11-13M7 22c.5-4 1.5-7 4-8.5"
strokeLinecap="round" />
          <path d="M14 22c1-6 2.5-9.5 5.5-11.5M17 22c.5-4 1-6 3-7.5" strokeLinecap="round" />
        </svg>
      );
    case 'Grasshopper':
      return (
        <svg className={` ${className} text-lime-600`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5" strokeLinecap="round" strokeLinejoin="round">
          <path d="M4 14c2-2 5-3 8-1s4 4 6 2l3-2" />
          <path d="M12 13l2-6 3 1" />
          <path d="M7 14l-3 5h2l2-4" />
          <path d="M17 13l3 6h-2l-2-5" />
          <circle cx="19" cy="11" r="1" fill="currentColor" />
        </svg>
      );
    case 'Frog':
      return (
        <svg className={` ${className} text-emerald-600`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
          <path d="M12 21c-4.5 0-7-2.5-7-5.5s1.5-4.5 3-4.5h8c1.5 0 3 1.5 3 4.5s-2.5 5.5-7 5.5z" />

```

```

    <ellipse cx="9" cy="9" rx="2" ry="3" fill="none" />
    <circle cx="9" cy="9" r="0.75" fill="currentColor" />
    <ellipse cx="15" cy="9" rx="2" ry="3" fill="none" />
    <circle cx="15" cy="9" r="0.75" fill="currentColor" />
    <path d="M7 16c2 1.5 8 1.5 10 0" />
    <path d="M5 19c-1 0-2-1-2-2.5S4.5 14 5.5 14" />
    <path d="M19 19c1 0 2-1 2-2.5S19.5 14 18.5 14" />
  </svg>
);
case 'Snake':
  return (
    <svg className={` ${className} text-emerald-800`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M8.5 4.5A2.5 2.5 0 0111 7c0 1.5-1 3-3 4.5S5 14 5 16s2 3 5 3 5-1.5
5-3.5-1.5-3-3.5-3-3-2-3-3.5a2.5 2.5 0 014.5-1.5" strokeLinecap="round" strokeLinejoin="round"
/>
      <path d="M11 6h.01M9 7h.01" strokeWidth="2" strokeLinecap="round" />
      <path d="M11.5 3.5l1-1M11.5 4.5l1.5-.5" />
    </svg>
);
case 'Algae (Pond Plant)':
  return (
    <svg className={` ${className} text-teal-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2">
      <path d="M4 18c2-4 1-8 4-11s5-1 6 3-1 7-4 9-5 1-6-1z" opacity="0.6" />
      <path d="M11 20c2-5 1-9 4-11s4 0 5 3-2 7-5 9-3-1-4-1z" opacity="0.8" />
      <path d="M8 22c1.5-6 .5-10 3.5-12s3.5-.5 4.5 2.5-2 7.5-5 9.5-2.5-1-3-2z" />
    </svg>
);
case 'Small Fish':
  return (
    <svg className={` ${className} text-sky-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M2 12c3-4 10-5 15-2 2.5 1.5 4 3 5 4-1 1-2.5 2.5-5 4-5 3-12 2-15-2z" />
      <path d="M17 10l4-4v12l-4-4" />
      <circle cx="6" cy="11" r="1" fill="currentColor" />
      <path d="M10 11.5c1.5 0 2 .5 2 1" />
    </svg>
);
case 'Seal':
  return (
    <svg className={` ${className} text-slate-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M4 14c2-4 6-6 10-5s6 3 8 6-3 4-7 4-7-1-11-5z" />

```

```

    <path d="M22 15l-3 3M21 13l-4 4" />
    <circle cx="7" cy="11" r="1" fill="currentColor" />
    <path d="M2 13c1 0 2 1.5 2 3s-1 3-2 3" />
  </svg>
);
case 'Orca Whale':
  return (
    <svg className={` ${className} text-slate-800`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M2 13c2-5 8-8 14-6s5 4 6 6-4 4-9 4-9-1-11-4z" />
      <path d="M14 7c0-2-1-4-3-5-.5 2 0 4 1 5" fill="currentColor" />
      <path d="M18 10c1-1 3-1 3 0s-2 2-3 0" />
      <path d="M8 12.5C10 13 13 13 14 12c-1-.5-3-1-6 .5z" fill="currentColor" />
    </svg>
  );
case 'Oak Tree':
  return (
    <svg className={` ${className} text-emerald-700`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2">
      <path d="M12 20v-5M10 20h4" strokeWidth="3" strokeLinecap="round" />
      <path d="M12 15c3.5 0 6-2 6-5.5S15.5 4 12 4s-6 2-6 5.5S8.5 15 12 15z"
fill="currentColor" fillOpacity="0.15" />
      <path d="M12 13a4.5 4.5 0 100-9 4.5 4.5 0 000 9z" />
    </svg>
  );
case 'Caterpillar':
  return (
    <svg className={` ${className} text-lime-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round">
      <circle cx="5" cy="14" r="2" fill="currentColor" />
      <circle cx="9" cy="12" r="2" fill="currentColor" />
      <circle cx="13" cy="13" r="2" fill="currentColor" />
      <circle cx="17" cy="11" r="2" fill="currentColor" />
      <circle cx="20" cy="9" r="1.5" />
      <path d="M19 7l-1-1.5M20.5 7.5L22 7M4 16l-1 2M8 14l-1 2M12 15l-1 2M16 13l-1 2" />
    </svg>
  );
case 'Bluebird':
  return (
    <svg className={` ${className} text-blue-500`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <path d="M17 6c-1.5.5-3 2-4 3.5-.5-1-1.5-2.5-3-3-2-.7-4 .5-4.5 2S6 13.5 9 14.5c4 1.3 7-.5
9-2.5s2.5-4 1-5l-2-1z" />
      <path d="M7 11.5s-2 1-3 3c2 0 3-1 3-3z" fill="currentColor" />

```

```

    <path d="M18 7.5l2-1.5v2" />
    <circle cx="16" cy="9" r="1" fill="currentColor" />
  </svg>
);
case 'Owl':
  return (
    <svg className={` ${className} text-amber-800`} viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5">
      <rect x="6" y="4" width="12" height="16" rx="4" />
      <circle cx="9.5" cy="9.5" r="2" />
      <circle cx="9.5" cy="9.5" r="0.5" fill="currentColor" />
      <circle cx="14.5" cy="9.5" r="2" />
      <circle cx="14.5" cy="9.5" r="0.5" fill="currentColor" />
      <path d="M12 11l-1 2h2l-1-2z" fill="currentColor" />
      <path d="M8 18c1-1 2-1 4-1s3 0 4 1" />
      <path d="M6 5L4 7M18 5l2 7" />
    </svg>
  );
default:
  return (
    <svg className={` ${className} text-slate-400`} viewBox="0 0 24 24" fill="none"
stroke="currentColor">
      <circle cx="12" cy="12" r="10" strokeWidth="2" />
      <path d="M8 12h8M12 8v8" strokeWidth="2" />
    </svg>
  );
}
};

```

```

export default function App() {
  // =====
  // 📁 STATE & CONSTANTS SETUP
  // =====
  const [activeTab, setActiveTab] = useState('lab');
  const [selectedChain, setSelectedChain] = useState('meadow');

  // Game state
  const [xp, setXp] = useState(0);
  const [level, setLevel] = useState(1);
  const [completedChains, setCompletedChains] = useState({
    meadow: false,
    ocean: false,
    forest: false
  });
};

```

```

// Active chain states
const [slots, setSlots] = useState([null, null, null, null]);
const [availableCards, setAvailableCards] = useState([]);
const [feedback, setFeedback] = useState({ text: "Place the energy builder (the source of all life!) in the first slot.", type: "hint" });
const [isChainValid, setIsChainValid] = useState(false);
const [showConfetti, setShowConfetti] = useState(false);

// Biome-specific configuration
const biomes = {
  meadow: {
    title: "🌞 Sunny Meadow",
    color: "from-emerald-50 to-green-100 border-emerald-200",
    accent: "emerald",
    organisms: [
      { id: 'sun', name: 'Sun', role: 'Energy Source', desc: 'The starting energy for all life.' },
      { id: 'grass', name: 'Grass', role: 'Producer', desc: 'Makes food using sunlight.' },
      { id: 'hopper', name: 'Grasshopper', role: 'Herbivore', desc: 'Eats grass for energy.' },
      { id: 'frog', name: 'Frog', role: 'Carnivore', desc: 'Snaps up insects for energy.' }
    ],
    correctOrder: ['sun', 'grass', 'hopper', 'frog'],
    hints: [
      "Every food chain starts with a beaming source of heat and light!",
      "Next, what green organism drinks sunlight to make its own sweet sugars?",
      "Now look for a small jumping insect that loves to munch on green grass leaves.",
      "Lastly, who is sitting quietly in the weeds waiting to snap up that grasshopper?"
    ]
  },
  ocean: {
    title: "🌊 Crystal Ocean",
    color: "from-sky-50 to-blue-100 border-sky-200",
    accent: "sky",
    organisms: [
      { id: 'sun', name: 'Sun', role: 'Energy Source', desc: 'Heats the planet and feeds the sea.' },
      { id: 'algae', name: 'Algae (Pond Plant)', role: 'Producer', desc: 'Microscopic plants floating on the surface.' },
      { id: 'fish', name: 'Small Fish', role: 'Herbivore/Consumer', desc: 'Gathers energy by eating green ocean algae.' },
      { id: 'seal', name: 'Seal', role: 'Carnivore', desc: 'Dives deep to chase down small tasty fish.' }
    ],
    correctOrder: ['sun', 'algae', 'fish', 'seal'],
    hints: [

```

```

    "Even ocean food chains require solar rays to kickstart their energy cycles!",
    "What tiny, leafy green organisms drift in the ocean, creating oxygen and food?",
    "Who is swimming by, looking for a school of algae to munch on?",
    "What sleek creature with flippers loves catching fish in cold waters?"
  ]
},
forest: {
  title: "🌲 Deep Oak Forest",
  color: "from-amber-50 to-orange-100 border-amber-200",
  accent: "amber",
  organisms: [
    { id: 'sun', name: 'Sun', role: 'Energy Source', desc: 'Shines through the thick forest canopy.' },
    { id: 'oak', name: 'Oak Tree', role: 'Producer', desc: 'Produces massive green leaves and acorns.' },
    { id: 'caterpillar', name: 'Caterpillar', role: 'Herbivore', desc: 'Crawls slowly, chewing on oak leaves.' },
    { id: 'owl', name: 'Owl', role: 'Carnivore', desc: 'A nocturnal hunter with super sharp vision.' }
  ],
  correctOrder: ['sun', 'oak', 'caterpillar', 'owl'],
  hints: [
    "Sunlight is always the master charger of the food web!",
    "What giant woodland producer extends its branches high to catch solar rays?",
    "Which wiggly crawler feeds directly on fresh green oak leaves?",
    "What wise nocturnal predator watches the tree branches for scurrying bugs?"
  ]
}
};

```

```

// Set up cards when biome changes or reset occurs

```

```

useEffect(() => {
  resetLevel();
}, [selectedChain]);

```

```

const resetLevel = () => {
  const biome = biomes[selectedChain];
  // Shuffle organisms for available card dock
  const shuffled = [...biome.organisms].sort(() => Math.random() - 0.5);
  setAvailableCards(shuffled);
  setSlots([null, null, null, null]);
  setFeedback({ text: biome.hints[0], type: "hint" });
  setIsChainValid(false);
  setShowConfetti(false);
};

```

```

// Drag-and-drop & Click interactions
const handleClick = (card) => {
  // Find first empty slot
  const emptyIndex = slots.findIndex(slot => slot === null);
  if (emptyIndex !== -1) {
    placeCard(card, emptyIndex);
  }
};

const placeCard = (card, index) => {
  const newSlots = [...slots];
  // If there is already a card in that slot, return it to the dock first
  if (newSlots[index]) {
    returnCardToDock(newSlots[index]);
  }
  newSlots[index] = card;
  setSlots(newSlots);
  setAvailableCards(prev => prev.filter(c => c.id !== card.id));
  checkIncrementalPlacement(newSlots, index);
};

const returnCardToDock = (card) => {
  setAvailableCards(prev => [...prev, card]);
};

const removeCardFromSlot = (index) => {
  const cardToRemove = slots[index];
  if (cardToRemove) {
    const newSlots = [...slots];
    newSlots[index] = null;
    setSlots(newSlots);
    returnCardToDock(cardToRemove);

    // Recalculate guidance feedback
    const nextEmpty = newSlots.findIndex(s => s === null);
    if (nextEmpty !== -1) {
      setFeedback({ text: biomes[selectedChain].hints[nextEmpty], type: "hint" });
    }
    setIsChainValid(false);
  }
};

// Continuous micro-feedback

```

```

const checkIncrementalPlacement = (currentSlots, placedIndex) => {
  const correctSeq = biomes[selectedChain].correctOrder;

  // Check if what is placed so far is correct
  let isCorrectSoFar = true;
  for (let i = 0; i <= placedIndex; i++) {
    if (currentSlots[i] && currentSlots[i].id !== correctSeq[i]) {
      isCorrectSoFar = false;
      break;
    }
  }

  if (!isCorrectSoFar) {
    // Trigger dynamic hints for misplaced blocks
    if (placedIndex === 0 && currentSlots[0].id !== 'sun') {
      setFeedback({
        text: `Oops! "${currentSlots[0].name}" is awesome, but does it create raw energy? Hint:
Look for our solar superstar`,
        type: "warning"
      });
    } else {
      setFeedback({
        text: "Hmm, the flow of energy looks blocked. Check the order of who gives energy to
whom!",
        type: "warning"
      });
    }
  } else {
    // Correct placement so far
    const nextEmpty = currentSlots.findIndex(s => s === null);
    if (nextEmpty !== -1) {
      setFeedback({
        text: `Fantastic! ${currentSlots[placedIndex].name} is in the right spot! ` +
biomes[selectedChain].hints[nextEmpty],
        type: "success"
      });
    } else {
      // All filled and looks perfect!
      setFeedback({
        text: "Excellent work! All organisms are linked. Press 'Verify Energy Flow' to send the
sparks!",
        type: "success"
      });
    }
  }
}

```



```

    }
  };

  // Submit and verify final food chain
  const verifyChain = () => {
    const correctSeq = biomes[selectedChain].correctOrder;
    const finalMatches = slots.every((slot, idx) => slot && slot.id === correctSeq[idx]);

    if (finalMatches) {
      setIsChainValid(true);
      setShowConfetti(true);
      setFeedback({
        text: "🎉 SUCCESS! The energy flows seamlessly from left to right. You earned +50 XP  
and a Biome Explorer Badge!",
        type: "success"
      });
      // Award XP
      if (!completedChains[selectedChain]) {
        setXp(prev => {
          const nextXp = prev + 50;
          if (nextXp >= 100) {
            setLevel(l => l + 1);
            return nextXp - 100;
          }
          return nextXp;
        });
        setCompletedChains(prev => ({ ...prev, [selectedChain]: true }));
      }
    } else {
      setFeedback({
        text: "The energy line broke! Remember: Sun goes first, then the plant (producer), then  
herbivores, and carnivores last.",
        type: "error"
      });
    }
  };

  // =====
  // 🛠 INTERACTIVE QUIZ CONFIGURATION
  // =====
  const quizQuestions = [
    {
      id: 1,
      type: 'mcq',

```

q: "Where does the energy in nearly all food chains start?",
options: [
{ label: "A) High nutrients in the soil", val: "soil" },
{ label: "B) Light and heat from the Sun", val: "sun" },
{ label: "C) Fresh spring rain water", val: "water" },
{ label: "D) Oxygen in the atmosphere", val: "oxygen" }
],
correct: "sun",
explanation: "The Sun is the source of all life! Plants utilize solar energy during photosynthesis to generate food that animals eat."

},
{
id: 2,
type: 'tf',
q: "A fierce lion is a carnivore, so it does not depend on grass or plants to survive at all.",
options: [
{ label: "True", val: "true" },
{ label: "False", val: "false" }
],
correct: "false",
explanation: "False! If there were no plants, herbivores like deer would starve. If deer starve, predators like lions would have no food either!"

},
{
id: 3,
type: 'mcq',
q: "What does the arrow in a food chain (e.g., Grass → Deer) represent?",
options: [
{ label: "A) The direction the animals like to run", val: "run" },
{ label: "B) 'Is eaten by' (Energy is transferred to)", val: "energy" },
{ label: "C) The animal that is bigger and stronger", val: "size" },
{ label: "D) Which organism is faster at chasing", val: "speed" }
],
correct: "energy",
explanation: "The arrow indicates the direction of energy transfer! Energy from the Grass goes INTO the stomach of the Deer."

},
{
id: 4,
type: 'scenario',
q: "Scenario: A gardener sprays eco-friendly soap that accidentally chases away all Grasshoppers in a meadow. Who suffers first?",
options: [
{ label: "A) The Green Grass (it grows wild without grazers)", val: "grass" },

```

    { label: "B) The Frogs (they lose their primary food source)", val: "frogs" },
    { label: "C) The Hawks flying high in the sky", val: "hawks" },
    { label: "D) The earthworms living in the damp dirt", val: "worms" }
  ],
  correct: "frogs",
  explanation: "Frogs are the immediate consumers that eat grasshoppers. If their food
disappears, they suffer first!"
},
{
  id: 5,
  type: 'fill',
  q: "Living organisms that can synthesize their own food using sunlight are called:
_____.",
  options: [
    { label: "A) Consumers", val: "consumers" },
    { label: "B) Decomposers", val: "decomposers" },
    { label: "C) Producers", val: "producers" },
    { label: "D) Predators", val: "predators" }
  ],
  correct: "producers",
  explanation: "Plants are 'Producers' because they produce energy out of simple solar light
and water!"
},
{
  id: 6,
  type: 'mcq',
  q: "If a plant stores 100 units of energy from the Sun, how much of that is generally passed
up to the herbivore that eats it?",
  options: [
    { label: "A) Exactly 100% of the energy", val: "100" },
    { label: "B) About 50% of the energy", val: "50" },
    { label: "C) Only about 10% of the energy", val: "10" },
    { label: "D) 0% (energy disappears completely)", val: "0" }
  ],
  correct: "10",
  explanation: "This is known as the '10% Rule'. Organisms use up 90% of their energy to
grow, run around, breathe, and stay warm. Only 10% transfers to the next consumer!"
},
{
  id: 7,
  type: 'mcq',
  q: "Which pond-based chain is in the correct order?",
  options: [
    { label: "A) Small Fish → Sun → Pond Algae → Crane", val: "a" },

```

```

    { label: "B) Sun → Pond Algae → Small Fish → Crane", val: "b" },
    { label: "C) Crane → Small Fish → Pond Algae → Sun", val: "c" },
    { label: "D) Pond Algae → Sun → Crane → Small Fish", val: "d" }
  ],
  correct: "b",
  explanation: "The correct biological sequence starts with Sun (Energy) → Algae (Producer)
→ Small Fish (Herbivore) → Crane (Carnivore).",
},
{
  id: 8,
  type: 'scenario',
  q: "If an animal eats both berries (plants) and salmon (meat/fish), what is its scientific
ecological classification?",
  options: [
    { label: "A) Carnivore", val: "carnivore" },
    { label: "B) Herbivore", val: "herbivore" },
    { label: "C) Omnivore", val: "omnivore" },
    { label: "D) Decomposer", val: "decomposer" }
  ],
  correct: "omnivore",
  explanation: "Omni means 'all/everything'. Omnivores (like brown bears and humans)
happily feed on both producers and other consumers.",
},
{
  id: 9,
  type: 'tf',
  q: "A food chain can easily support 25 different levels of animals in a line.",
  options: [
    { label: "True", val: "true" },
    { label: "False", val: "false" }
  ],
  correct: "false",
  explanation: "False! Because energy drops so sharply at every single step, food chains
rarely exceed 4 or 5 steps before running completely out of fuel.",
},
{
  id: 10,
  type: 'mcq',
  q: "What helpful creatures clean up the forest floor by breaking down old logs and dead
leaves?",
  options: [
    { label: "A) Top Predators", val: "predators" },
    { label: "B) Decomposers (like Mushrooms & Bacteria)", val: "decomposers" },
    { label: "C) Herbivore Insects", val: "herbivores" },

```

```

      { label: "D) Sun Rays", val: "sun" }
    ],
    correct: "decomposers",
    explanation: "Decomposers act as nature's recycling squad! They recycle dead materials
back into soil nutrients, which help brand-new green plants grow."
  }
];

```

```

const [currentQuestionIdx, setCurrentQuestionIdx] = useState(0);
const [selectedAnswer, setSelectedAnswer] = useState(null);
const [quizScore, setQuizScore] = useState(0);
const [showExplanation, setShowExplanation] = useState(false);
const [quizFinished, setQuizFinished] = useState(false);

```

```

const handleAnswerSelect = (optionValue) => {
  if (showExplanation) return; // Prevent clicking after submit
  setSelectedAnswer(optionValue);
};

```

```

const submitAnswer = () => {
  if (!selectedAnswer) return;
  if (selectedAnswer === quizQuestions[currentQuestionIdx].correct) {
    setQuizScore(prev => prev + 1);
  }
  setShowExplanation(true);
};

```

```

const nextQuestion = () => {
  setSelectedAnswer(null);
  setShowExplanation(false);
  if (currentQuestionIdx < quizQuestions.length - 1) {
    setCurrentQuestionIdx(prev => prev + 1);
  } else {
    setQuizFinished(true);
    // Give bonus XP for finishing quiz
    setXp(prev => {
      const nextXp = prev + 30;
      if (nextXp >= 100) {
        setLevel(l => l + 1);
        return nextXp - 100;
      }
      return nextXp;
    });
  }
};

```

```

};

const restartQuiz = () => {
  setCurrentQuestionIdx(0);
  setSelectedAnswer(null);
  setQuizScore(0);
  setShowExplanation(false);
  setQuizFinished(false);
};

// =====
// 🔄 CONCEPTS: ARROW CORRECTOR MINI-GAME
// =====
const [arrowCorrect, setArrowCorrect] = useState(null);

return (
  <div className="min-h-screen bg-slate-50 text-slate-800 font-sans flex flex-col antialiased">
    {/ * =====
      Header component
      ===== */}
    <header className="sticky top-0 z-30 bg-white border-b border-slate-200 shadow-sm py-4
px-6 flex flex-wrap justify-between items-center gap-4">
      <div className="flex items-center space-x-3">
        <div className="bg-emerald-500 text-white p-2.5 rounded-2xl shadow-md
shadow-emerald-200 flex items-center justify-center">
          <svg className="w-6 h-6 animate-spin-slow" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2.5" d="M12 3v1m0
16v1m9-9h-1M4 12H3m15.364-6.364l-.707.707M6.343
17.657l-.707.707m0-12.728l.707.707m12.728 12.728l.707.707M12 8a4 4 0 100 8 4 4 0 000-8z"
/>
          </svg>
        </div>
        <div>
          <h1 className="text-xl font-bold tracking-tight text-slate-800">The Energy Linker</h1>
          <p className="text-xs font-semibold text-emerald-600 tracking-wide uppercase">Class
3–8 • Interactive Biology Lab</p>
        </div>
      </div>

      {/ * Gamification Dashboard */}
      <div className="flex items-center space-x-4">
        {/ * Level badge */}

```

```

    <div className="bg-gradient-to-r from-emerald-500 to-teal-600 text-white font-bold
py-1.5 px-4 rounded-xl flex items-center gap-1.5 shadow-md">
      <span className="text-xs text-emerald-100">LEVEL</span>
      <span className="text-lg">{level}</span>
    </div>
    { /* XP Progress bar */ }
    <div className="w-36 sm:w-48 bg-slate-100 h-3 rounded-full overflow-hidden relative
border border-slate-200">
      <div
        className="bg-gradient-to-r from-emerald-400 to-emerald-600 h-full transition-all
duration-500 ease-out"
        style={{ width: `${xp}%` }}
      />
      <span className="absolute inset-0 flex items-center justify-center text-[9px]
font-extrabold text-slate-700">
        {xp}/100 XP
      </span>
    </div>
    { /* Active Badge list */ }
    <div className="hidden md:flex items-center space-x-2 text-xs">
      {completedChains.meadow && <span className="bg-emerald-100 text-emerald-800
py-1 px-2.5 rounded-full font-bold shadow-sm"> 🌿 Meadow Pro</span>}
      {completedChains.ocean && <span className="bg-sky-100 text-sky-800 py-1 px-2.5
rounded-full font-bold shadow-sm"> 🌊 Ocean Pro</span>}
      {completedChains.forest && <span className="bg-amber-100 text-amber-800 py-1
px-2.5 rounded-full font-bold shadow-sm"> 🌲 Forest Pro</span>}
    </div>
  </div>
</header>

{ /* =====
Navigation Tabs
===== */ }
<nav className="bg-slate-100 border-b border-slate-200 py-2 px-6 flex overflow-x-auto
space-x-2 scrollbar-none">
  <button
    onClick={() => setActiveTab('lab')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
transition-all whitespace-nowrap ${
      activeTab === 'lab'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }`
  />
  >

```

```

    <span> Lab Simulator</span>
  </button>
  <button
    onClick={() => setActiveTab('learn')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'learn'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span> Learn Concept</span>
  </button>
  <button
    onClick={() => setActiveTab('diy')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'diy'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span> Backyard DIY</span>
  </button>
  <button
    onClick={() => setActiveTab('quiz')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'quiz'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span> Interactive Quiz</span>
  </button>
  <button
    onClick={() => setActiveTab('info')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'info'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >

```



```

    <span>💡 Facts & FAQs</span>
  </button>
  <button
    onClick={() => setActiveTab('teacher_parent')}
    className={`flex items-center space-x-2 px-4 py-2 rounded-xl text-sm font-bold
  transition-all whitespace-nowrap ${
      activeTab === 'teacher_parent'
        ? 'bg-white text-emerald-700 shadow-md border-b-2 border-emerald-500'
        : 'text-slate-600 hover:bg-slate-200'
    }}
  >
    <span>🍎 Educator & Parent Zone</span>
  </button>
</nav>

{
  /* =====
  MAIN WORKSPACE & LAYOUT CONTAINER
  ===== */
  <main className="flex-1 p-4 md:p-6 max-w-7xl mx-auto w-full">

    {
      /* =====
      TAB: 🧪 LAB SIMULATOR
      ===== */
      {activeTab === 'lab' && (
        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">

          {
            /* LEFT AREA: Simulator Canvas (approx 70% equivalent) */
            <div className="lg:col-span-2 flex flex-col space-y-4">

              {
                /* Biome Picker */
                <div className="bg-white rounded-3xl p-4 shadow-sm border border-slate-100 flex
  flex-wrap justify-between items-center gap-3">
                  <span className="text-sm font-black uppercase text-slate-400
  tracking-wider">Choose Ecosystem Biome:</span>
                  <div className="flex space-x-2">
                    {Object.keys(biomes).map((biomeKey) => (
                      <button
                        key={biomeKey}
                        onClick={() => setSelectedChain(biomeKey)}
                        className={`px-4 py-2 rounded-2xl text-xs font-extrabold border-2 transition-all ${
                          selectedChain === biomeKey
                            ? 'border-emerald-500 bg-emerald-50 text-emerald-800'
                            : 'border-slate-100 bg-slate-50 text-slate-600 hover:border-slate-300'
                        }}
                      >
                    </button>
                    </div>
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      )
    }
  </main>
}

```

```

      >
        {biomes[biomeKey].title}
      </button>
    )))
  </div>
</div>

{ /* Main Interactive Canvas */}
<div className={`relative flex-1 min-h-[380px] bg-gradient-to-b
${biomes[selectedChain].color} rounded-3xl border-3 p-6 flex flex-col justify-between
transition-all duration-500 overflow-hidden`}>

  { /* Visual Background Accent / Floating Sun */}
  <div className="absolute top-4 right-4 pointer-events-none opacity-20">
    <OrganismIcon name="Sun" className="w-32 h-32 text-amber-300" />
  </div>

  { /* Sub-Header inside Canvas */}
  <div className="z-10 flex justify-between items-start">
    <div>
      <h2 className="text-2xl font-black text-slate-800 flex items-center gap-2">
        {biomes[selectedChain].title} Lab
      </h2>
      <p className="text-sm font-semibold text-slate-600 mt-0.5">
        Drag or click the organism cards from the dock to build the flow of life!
      </p>
    </div>
    <button
      onClick={resetLevel}
      className="bg-white/80 hover:bg-white text-slate-700 py-1.5 px-3 rounded-2xl
font-bold text-xs flex items-center gap-1 shadow-sm border border-slate-200 transition-all
active:scale-95"
    >
       Clear Slots
    </button>
  </div>

  { /* SLOTS CONTAINER (THE CHAIN PATHWAY) */}
  <div className="my-10 flex flex-col sm:flex-row items-center justify-center gap-2
sm:gap-4 z-10">
    {slots.map((card, idx) => (
      <React.Fragment key={idx}>
        { /* Slot Card */}
        <div

```

```

        onClick={() => removeCardFromSlot(idx)}
        className={`group relative w-28 h-40 rounded-3xl flex flex-col items-center
justify-center border-4 border-dashed cursor-pointer transition-all ${
        card
        ? 'bg-white border-emerald-400 shadow-md hover:scale-105 active:scale-95
hover:border-red-400'
        : 'bg-slate-50/50 hover:bg-slate-50/80 border-slate-300 hover:border-slate-400
shadow-inner'
        }}
    >
    {card ? (
    <>
    <div className="absolute top-1.5 right-1.5 bg-slate-100
group-hover:bg-red-100 text-slate-400 group-hover:text-red-500 rounded-full w-5 h-5 flex
items-center justify-center text-[10px] font-bold transition-colors">
    X
    </div>
    <OrganismIcon name={card.name} className="w-14 h-14" />
    <span className="text-center text-xs font-black text-slate-700 mt-2 px-1
line-clamp-1">{card.name}</span>
    <span className="text-[9px] font-extrabold text-emerald-600 bg-emerald-50
px-2 py-0.5 rounded-full mt-1 uppercase">
    {card.role}
    </span>
    </>
    ) : (
    <div className="flex flex-col items-center justify-center text-center p-2">
    <span className="text-xl text-slate-300">📦</span>
    <span className="text-[10px] font-bold text-slate-400 mt-1 uppercase">Slot
{idx + 1}</span>
    </div>
    )}
    </div>

    {/* Sparking Connector Arrow */}
    {idx < 3 && (
    <div className="flex items-center justify-center py-2 sm:py-0">
    {isChainValid ? (
    <div className="flex flex-col items-center animate-bounce">
    <span className="text-lg font-extrabold text-amber-500
animate-pulse">⚡</span>
    <svg className="w-8 h-8 text-amber-400" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="3">

```

```

        <path strokeLinecap="round" strokeLinejoin="round" d="M14 5l7 7m0 0l-7
7m7-7H3" />
      </svg>
    </div>
  ) : (
    <svg className="w-8 h-8 text-slate-400 transform rotate-90 sm:rotate-0"
fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth="2.5">
      <path strokeLinecap="round" strokeLinejoin="round" d="M14 5l7 7m0 0l-7
7m7-7H3" />
    </svg>
  )}
</div>
)}
</React.Fragment>
)})
</div>

```

```

{ /* BOTTOM COMPONENT: Card Dock */
  <div className="bg-white/80 backdrop-blur-md rounded-2xl p-4 shadow-md border
border-slate-100 z-10">
    <p className="text-xs font-bold text-slate-400 uppercase tracking-wider
mb-2">Available Organisms Dock (Click to place):</p>
    <div className="flex flex-wrap justify-center gap-3">
      {availableCards.length === 0 && slots.every(s => s !== null) ? (
        <p className="text-xs font-bold text-emerald-600 animate-pulse">All cards
placed! Tap 'Verify Energy Flow' below! 🖱️</p>
      ) : (
        availableCards.map((card) => (
          <button
            key={card.id}
            onClick={() => handleCardClick(card)}
            className="bg-white border-2 border-slate-100 hover:border-emerald-300
rounded-2xl p-3 flex flex-col items-center w-24 shadow-sm hover:shadow-md transform
hover:-translate-y-1 transition-all active:scale-95"
          >
            <OrganismIcon name={card.name} className="w-10 h-10 mb-1" />
            <span className="text-[10px] font-black text-slate-700 text-center
line-clamp-1">{card.name}</span>
          </button>
        ))
      )}
    </div>
  </div>
}

```

```

</div>

{/* ACTION FOOTER */}
<div className="flex justify-between items-center bg-white p-4 rounded-3xl
shadow-sm border border-slate-100">
  <button
    onClick={resetLevel}
    className="bg-slate-100 hover:bg-slate-200 text-slate-700 font-extrabold py-3 px-6
rounded-2xl text-sm transition-all"
  >
    Reset Level
  </button>
  <button
    onClick={verifyChain}
    disabled={slots.some(s => s === null)}
    className={`font-black py-3 px-8 rounded-2xl text-sm transition-all shadow-md ${
      slots.some(s => s === null)
        ? 'bg-slate-200 text-slate-400 cursor-not-allowed shadow-none'
        : 'bg-gradient-to-r from-emerald-500 to-teal-600 text-white hover:brightness-110
active:scale-95'
      }`}
  >
    ⚡ Verify Energy Flow
  </button>
</div>

```

```

</div>

{/* RIGHT PANEL: Live Observation & Guidelines (approx 30% equivalent) */}
<div className="flex flex-col space-y-4">

  {/* Guidance & Assistant Character Box */}
  <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-100
relative overflow-hidden">
    <div className="flex items-center space-x-3 mb-4">
      <div className="bg-amber-100 text-amber-600 w-12 h-12 rounded-2xl flex
items-center justify-center text-2xl font-bold">
        🍀
      </div>
    </div>
    <div>
      <h3 className="font-black text-slate-800">Leafy the Assistant</h3>
      <p className="text-xs font-semibold text-slate-400">Your Ecosystem
Companion</p>
    </div>
  </div>

```

</div>

{/* Assistant Chat Bubble */}

```
<div className={`p-4 rounded-2xl text-sm font-semibold border-l-4 leading-relaxed ${
  feedback.type === 'success'
    ? 'bg-emerald-50 border-emerald-500 text-emerald-800'
    : feedback.type === 'warning'
    ? 'bg-amber-50 border-amber-500 text-amber-800 animate-shake'
    : feedback.type === 'error'
    ? 'bg-red-50 border-red-500 text-red-800'
    : 'bg-sky-50 border-sky-400 text-sky-800'
}`}>
  {feedback.text}
</div>
```

{/* Success celebration box */}

```
{showConfetti && (
  <div className="mt-4 bg-emerald-500 text-white p-4 rounded-2xl text-center
shadow-lg animate-bounce">
    <p className="font-extrabold text-base">★ EXCELLENT ECOLOGIST! ★</p>
    <p className="text-xs font-semibold opacity-90 mt-1">
      You mapped the flow of organic energy perfectly. Select another Biome above to
unlock more badges!
    </p>
  </div>
)}
```

{/* Dynamic Organism Card Explainer */}

```
<div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-100
flex-1">
  <h3 className="font-black text-slate-800 mb-3 text-sm uppercase tracking-wider
text-slate-400">Organism Guidebook</h3>
  <div className="space-y-4 max-h-[300px] overflow-y-auto pr-1">
    {biomes[selectedChain].organisms.map((org) => {
      const isPlaced = slots.some(slot => slot && slot.id === org.id);
      return (
        <div
          key={org.id}
          className={`p-3 rounded-2xl border-2 transition-colors ${
            isPlaced ? 'bg-emerald-50/50 border-emerald-100' : 'bg-slate-50
border-slate-100'
          }`}
        >
```

```

    <div className="flex items-center space-x-2">
      <OrganismIcon name={org.name} className="w-8 h-8" />
      <div>
        <span className="font-black text-xs text-slate-700">{org.name}</span>
        <span className="ml-2 text-[9px] font-extrabold px-1.5 py-0.5 bg-slate-200
text-slate-600 rounded">
          {org.role}
        </span>
      </div>
    </div>
    <div>
      <p className="text-xs font-semibold text-slate-500 mt-1.5
leading-relaxed">{org.desc}</p>
    </div>
  );
  }}}
</div>
</div>

</div>

</div>
)}

{ /* =====
  TAB: 📖 CONCEPT EXPLANATION
  ===== */ }
{activeTab === 'learn' && (
  <div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100
max-w-4xl mx-auto space-y-8">
    <div className="text-center max-w-2xl mx-auto">
      <span className="bg-emerald-100 text-emerald-800 font-extrabold px-3 py-1
rounded-full text-xs uppercase">Interactive Textbook</span>
      <h2 className="text-3xl font-black text-slate-800 mt-2">Who Eats Whom In
Nature?</h2>
      <p className="text-slate-500 mt-2 font-semibold">
        Discover the elegant transfer of organic power that keeps the animal kingdom alive.
      </p>
    </div>

    { /* Step-by-Step Learning Grid */ }
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6">

      <div className="bg-emerald-50/50 p-6 rounded-2xl border border-emerald-100
space-y-3">

```

```
<div className="bg-emerald-100 text-emerald-800 w-10 h-10 rounded-xl flex
items-center justify-center text-lg font-bold">🌱 </div>
```

```
<h3 className="font-black text-slate-800">1. The Producers</h3>
```

```
<p className="text-sm font-semibold text-slate-600 leading-relaxed">
```

Plants are the superstars of our food webs. Using green chlorophyll in their leaves, they capture raw solar beams and turn water and air into edible sugars! They are called **Producers** because they produce food from scratch.

```
</p>
```

```
</div>
```

```
<div className="bg-sky-50/50 p-6 rounded-2xl border border-sky-100 space-y-3">
```

```
<div className="bg-sky-100 text-sky-800 w-10 h-10 rounded-xl flex items-center
justify-center text-lg font-bold">🦌 </div>
```

```
<h3 className="font-black text-slate-800">2. The Consumers</h3>
```

```
<p className="text-sm font-semibold text-slate-600 leading-relaxed">
```

Animals can't make food inside their bodies. To charge up their muscles and breathe, they must consume plants or other animals. Thus, they are called **Consumers**! They are split into:

```
</p>
```

```
<ul className="text-xs font-bold text-slate-500 space-y-1 pl-4 list-disc">
```

```
<li><span className="text-emerald-600">Herbivores</span> (Plant-eaters like
deer, caterpillars)</li>
```

```
<li><span className="text-amber-700">Carnivores</span> (Meat-eaters like tigers,
hawks)</li>
```

```
<li><span className="text-blue-700">Omnivores</span> (Eaters of both like bears,
humans)</li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
{/* INTERACTIVE COMPONENT: Arrow Direction Corrector Widget */}
```

```
<div className="bg-slate-50 rounded-2xl p-6 border border-slate-200">
```

```
<h3 className="font-black text-slate-800 flex items-center gap-1.5 text-base">
```

```
⚡ Interactive Challenge: The Arrow Confuser!
```

```
</h3>
```

```
<p className="text-xs font-semibold text-slate-500 mt-1 leading-relaxed">
```

Many students construct arrows backward. Try clicking the toggles below to see the biological truth of arrows!

```
</p>
```

```
<div className="mt-4 bg-white p-4 rounded-xl border border-slate-200 flex flex-col
md:flex-row items-center justify-around gap-4 text-center">
```



```

    { /* Visual Representation of Arrows */ }
    <div className="flex items-center space-x-3 font-bold">
      <span className="p-2 bg-emerald-50 rounded text-sm">🌿 Grass</span>
      <div className="flex flex-col items-center">
        <span className="text-xs text-slate-400 font-extrabold">Arrow points:</span>
        <span className={`text-xl font-black ${arrowCorrect === 'correct' ?
'text-emerald-600' : arrowCorrect === 'incorrect' ? 'text-red-500' : 'text-slate-300'}`}>
          {arrowCorrect === 'correct' ? '➔ (To Deer)' : arrowCorrect === 'incorrect' ? '❑ (To
Grass)' : '➔ or ❑?'}
        </span>
      </div>
      <span className="p-2 bg-amber-50 rounded text-sm">🦌 Deer</span>
    </div>

    { /* Toggles */ }
    <div className="flex space-x-2">
      <button
        onClick={() => setArrowCorrect('incorrect')}
        className={`px-3 py-1.5 rounded-lg text-xs font-bold ${arrowCorrect ===
'incorrect' ? 'bg-red-500 text-white' : 'bg-slate-100 hover:bg-slate-200 text-slate-700'}`}
      >
        Deer eats Grass (❑)
      </button>
      <button
        onClick={() => setArrowCorrect('correct')}
        className={`px-3 py-1.5 rounded-lg text-xs font-bold ${arrowCorrect === 'correct'
? 'bg-emerald-500 text-white' : 'bg-slate-100 hover:bg-slate-200 text-slate-700'}`}
      >
        Energy flows into Deer (➔)
      </button>
    </div>

  </div>

  { /* Dynamic feedback on arrow corrector */ }
  {arrowCorrect && (
    <p className={`mt-3 text-xs font-bold p-3 rounded-lg ${
      arrowCorrect === 'correct'
        ? 'bg-emerald-50 text-emerald-800'
        : 'bg-red-50 text-red-800'
    }`}>
      {arrowCorrect === 'correct'
        ? "🌟 Correct! The arrow ALWAYS points to where the energy is going. The energy
flows FROM the grass INTO the stomach of the deer!"

```

: " ⚠️ Common Mistake! While the deer eats the grass, arrows in science show energy flow. If the arrow pointed left, it would mean energy goes into the grass!"

</p>

)}

</div>

{/* Summary Block */}

<div className="bg-amber-50/50 p-6 rounded-2xl border border-amber-100">

<h3 className="font-black text-amber-800 mb-2">💡 Quick Memory Rule:</h3>

<p className="text-sm font-semibold text-amber-900 leading-relaxed">

"Plants Produce (Producers). Animals Consume (Consumers). Decomposers Clean up!"

</p>

</div>

</div>

)}

{/* =====

TAB: 🛠️ BACKYARD DIY ACTIVITY

===== */}

{activeTab === 'diy' && (

<div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100 max-w-3xl mx-auto space-y-6">

<div className="flex items-center space-x-3 mb-4">

🔍

<div>

<h2 className="text-2xl font-black text-slate-800">My Backyard Food Chain Tracker</h2>

<p className="text-xs font-bold text-emerald-600 uppercase mt-0.5">Hands-On Home Science Experiment</p>

</div>

</div>

<p className="text-sm font-semibold text-slate-600 leading-relaxed">

Step away from the screen and search for real, living food chains happening outside your windows! Grab a notebook and your pencil.

</p>

<div className="space-y-4">

<h3 className="font-black text-slate-800 border-b pb-2 text-base">Steps to Accomplish:</h3>

<div className="flex items-start space-x-3">

```
<span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">1</span>
<div>
  <h4 className="font-bold text-slate-800 text-sm">Find Your Producer</h4>
  <p className="text-xs text-slate-500">Go into your garden or look out a window.
Find a green bush, tree, flower, or even simple weed grass. Draw it on the far-left side of your
page.</p>
</div>
</div>
```

```
<div className="flex items-start space-x-3">
  <span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">2</span>
  <div>
    <h4 className="font-bold text-slate-800 text-sm">Seek Out a Tiny Herbivore</h4>
    <p className="text-xs text-slate-500">Watch the leaves closely. Do you see any
ants, ladybugs, butterflies, caterpillars, or beetles? Draw the insect next to your plant.</p>
  </div>
</div>
```

```
<div className="flex items-start space-x-3">
  <span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">3</span>
  <div>
    <h4 className="font-bold text-slate-800 text-sm">Add the Energy Arrow</h4>
    <p className="text-xs text-slate-500">Draw a bold arrow from your plant pointing
directly to your insect. You just logged energy transfer!</p>
  </div>
</div>
```

```
<div className="flex items-start space-x-3">
  <span className="bg-emerald-100 text-emerald-800 w-6 h-6 rounded-full flex
items-center justify-center font-bold text-xs mt-0.5">4</span>
  <div>
    <h4 className="font-bold text-slate-800 text-sm">Predict the Carnivore</h4>
    <p className="text-xs text-slate-500">What backyard creature might swoop down
to grab that insect? (A sparrow, a squirrel, a gecko, or a cat?) Sketch it on the right to complete
the chain!</p>
  </div>
</div>
</div>
```

```
<div className="bg-amber-50 p-4 rounded-2xl border border-amber-200 mt-6">
  <h4 className="font-extrabold text-amber-800 text-sm">⚠ Safety Rule:</h4>
```

```

    <p className="text-xs font-semibold text-amber-950 mt-1">
      Keep hands off wild bugs, spikes, and thorns! Keep your eyes sharp and observe
safely from a short distance.
    </p>
  </div>
</div>
)}

{/* =====
TAB: 🛠 INTERACTIVE QUIZ
===== */}
{activeTab === 'quiz' && (
  <div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100
max-w-2xl mx-auto">

    {/* If Quiz is in progress */}
    {!quizFinished ? (
      <div className="space-y-6">

        {/* Progress bar */}
        <div className="flex justify-between items-center text-xs font-extrabold
text-slate-400 uppercase tracking-wider">
          <span>Question {currentQuestionIdx + 1} of {quizQuestions.length}</span>
          <span className="text-emerald-600">Correct:
{quizScore}/{currentQuestionIdx}</span>
        </div>
        <div className="w-full bg-slate-100 h-2 rounded-full overflow-hidden">
          <div
            className="bg-emerald-500 h-full transition-all duration-300"
            style={{ width: `${((currentQuestionIdx + 1) / quizQuestions.length) * 100}%` }}
          />
        </div>

        {/* Question */}
        <div className="space-y-2">
          <span className="bg-emerald-100 text-emerald-800 font-extrabold text-[10px]
px-2.5 py-1 rounded-full uppercase">
            {quizQuestions[currentQuestionIdx].type === 'mcq' ? 'Multiple Choice' :
quizQuestions[currentQuestionIdx].type === 'tf' ? 'True / False' : 'Scenario Challenge'}
          </span>
          <h3 className="text-lg font-black text-slate-800 leading-snug">
            {quizQuestions[currentQuestionIdx].q}
          </h3>
        </div>

```

```

    { /* Options List */
    <div className="space-y-3">
      {quizQuestions[currentQuestionIdx].options.map((opt) => {
        const isSelected = selectedAnswer === opt.val;
        const isCorrectOption = opt.val === quizQuestions[currentQuestionIdx].correct;

        let cardStyle = "border-slate-200 hover:border-slate-300 hover:bg-slate-50";
        if (isSelected) cardStyle = "border-emerald-500 bg-emerald-50 text-emerald-900";

        if (showExplanation) {
          if (isCorrectOption) {
            cardStyle = "border-emerald-500 bg-emerald-100 text-emerald-900 font-bold";
          } else if (isSelected && !isCorrectOption) {
            cardStyle = "border-red-500 bg-red-50 text-red-900";
          } else {
            cardStyle = "border-slate-200 opacity-60";
          }
        }

        return (
          <button
            key={opt.val}
            disabled={showExplanation}
            onClick={() => handleAnswerSelect(opt.val)}
            className={`w-full p-4 rounded-2xl border-2 text-left text-sm font-semibold
transition-all ${cardStyle}`}
          >
            {opt.label}
          </button>
        );
      })}
    </div>

    { /* Explanations block */
    {showExplanation && (
      <div className={`p-4 rounded-2xl text-xs leading-relaxed ${
        selectedAnswer === quizQuestions[currentQuestionIdx].correct
        ? 'bg-emerald-50 text-emerald-800 border border-emerald-200'
        : 'bg-red-50 text-red-800 border border-red-200'
      }`} >
        <p className="font-extrabold text-sm mb-1">
          {selectedAnswer === quizQuestions[currentQuestionIdx].correct ? '🎉 Correct!' :
'❌ Incorrect'}
        </p>
      </div>
    )}
  )}

```

```

    </p>
    <p>{quizQuestions[currentQuestionIdx].explanation}</p>
  </div>
)}

{/* Quiz controls */}
<div className="flex justify-end pt-4 border-t border-slate-100">
  {!showExplanation ? (
    <button
      onClick={submitAnswer}
      disabled={!selectedAnswer}
      className={`font-extrabold py-3 px-6 rounded-xl text-sm transition-all shadow ${
        !selectedAnswer
          ? 'bg-slate-200 text-slate-400 cursor-not-allowed shadow-none'
          : 'bg-emerald-500 text-white hover:brightness-105 active:scale-95'
      }}
    >
      Submit Answer
    </button>
  ) : (
    <button
      onClick={nextQuestion}
      className="bg-emerald-600 hover:bg-emerald-700 text-white font-extrabold py-3
px-6 rounded-xl text-sm transition-all shadow active:scale-95"
    >
      {currentQuestionIdx < quizQuestions.length - 1 ? 'Next Question →' : 'View
Results'}
    </button>
  )}
</div>

</div>
): (
  // Quiz completion state
  <div className="text-center space-y-6 py-8">
    <span className="text-5xl">🏆</span>
    <h3 className="text-2xl font-black text-slate-800">Quiz Completed!</h3>
    <p className="text-slate-500 font-semibold max-w-md mx-auto">
      You scored <span className="text-emerald-600 font-bold">{quizScore}</span> out of
{quizQuestions.length}</span> questions correctly! You earned <strong
className="text-amber-500">+30 Energy XP</strong>!
    </p>

```

```

    <div className="inline-block bg-slate-50 p-4 rounded-2xl border border-slate-100
text-left max-w-sm w-full space-y-2">
      <div className="flex justify-between font-bold text-xs">
        <span>Ecosystem Rating:</span>
        <span className="text-emerald-600">
          {quizScore >= 8 ? '🌟 Master Ecologist' : quizScore >= 5 ? '🌿 Nature Ranger' :
'🌱 Budding Scientist'}
        </span>
      </div>
      <div className="w-full bg-slate-200 h-2 rounded-full overflow-hidden">
        <div
          className="bg-emerald-500 h-full"
          style={{ width: `${(quizScore / quizQuestions.length) * 100}%` }}
        />
      </div>
    </div>

    <div className="flex justify-center space-x-3 pt-4">
      <button
        onClick={restartQuiz}
        className="bg-slate-100 hover:bg-slate-200 text-slate-700 font-extrabold py-3
px-6 rounded-xl text-sm transition-all"
      >
        Try Again
      </button>
      <button
        onClick={() => setActiveTab('lab')}
        className="bg-emerald-500 hover:bg-emerald-600 text-white font-extrabold py-3
px-6 rounded-xl text-sm transition-all shadow"
      >
        Back to Simulator
      </button>
    </div>
  </div>
)}

</div>
)}

{ /* =====
  TAB: 💡 FACTS & FAQs
  ===== */ }
{activeTab === 'info' && (
  <div className="max-w-4xl mx-auto space-y-8">

```

```

    { /* Mindblowing Facts Section */ }
    <div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border
border-slate-100">
      <h2 className="text-2xl font-black text-slate-800 flex items-center gap-2 mb-6">
        <span>🤯</span> Mind-Blowing Food Chain Records
      </h2>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">

        <div className="bg-amber-50/50 p-5 rounded-2xl border border-amber-100
space-y-2">
          <h3 className="font-extrabold text-amber-800 text-sm">🚀 Super-Fast Light
Travel</h3>
          <p className="text-xs text-slate-600 leading-relaxed">
            It takes only about <strong>8 minutes and 20 seconds</strong> for light energy
leaving the Sun to slide through cold outer space, hit a green leaf on Earth, and begin the food
chain!
          </p>
        </div>

        <div className="bg-sky-50/50 p-5 rounded-2xl border border-sky-100 space-y-2">
          <h3 className="font-extrabold text-sky-800 text-sm">🐋 The Blue Whale
Paradox</h3>
          <p className="text-xs text-slate-600 leading-relaxed">
            The largest creature to ever live on Earth, the Blue Whale, has an incredibly short
food chain. It skips intermediate steps and eats small shrimp-like Krill directly!
          </p>
        </div>

        <div className="bg-emerald-50/50 p-5 rounded-2xl border border-emerald-100
space-y-2">
          <h3 className="font-extrabold text-emerald-800 text-sm">🕸 Food Web Safety
Nets</h3>
          <p className="text-xs text-slate-600 leading-relaxed">
            In real life, linear food chains overlap to form a giant safety network called a
<strong>Food Web</strong>. If one insect species declines, predators can adapt by eating
other insects.
          </p>
        </div>

        <div className="bg-purple-50/50 p-5 rounded-2xl border border-purple-100
space-y-2">
          <h3 className="font-extrabold text-purple-800 text-sm">🍄 Decomposers: The
Garbage Crew</h3>

```



```
<p className="text-xs text-slate-600 leading-relaxed">
  When high predators die, they are digested by mushrooms, worms, and bacteria.
  This converts their complex biology back into pure nitrogen-rich dirt, feeding new green grass!
</p>
</div>
```

```
</div>
</div>
```

```
{/* Student FAQs Section */}
<div className="bg-white rounded-3xl p-6 md:p-8 shadow-sm border border-slate-100
space-y-4">
  <h2 className="text-xl font-black text-slate-800"> ? Frequently Asked Questions
  (FAQs)</h2>
```

```
    <div className="space-y-3">
      <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200">
        <h4 className="font-black text-slate-800 text-sm">Q: Can a food chain go on
forever?</h4>
```

```
        <p className="text-xs text-slate-600 mt-1 leading-relaxed">
          A: No! Because organisms use up nearly 90% of their energy just staying alive
          (breathing and moving), only 10% transfers to the next step. After 4 or 5 steps, there's not
          enough leftover fuel to support a bigger animal.
        </p>
      </div>
```

```
    <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200">
      <h4 className="font-black text-slate-800 text-sm">Q: What happens if we are
omnivores? Where do we belong?</h4>
```

```
      <p className="text-xs text-slate-600 mt-1 leading-relaxed">
        A: Omnivores (like humans) occupy multiple positions at once! When you eat a
        juicy strawberry, you are a primary consumer. When you eat chicken or salmon, you are a
        secondary or tertiary consumer.
      </p>
    </div>
```

```
    <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200">
      <h4 className="font-black text-slate-800 text-sm">Q: Is dirt or water a
producer?</h4>
```

```
      <p className="text-xs text-slate-600 mt-1 leading-relaxed">
        A: No. Water and soil are essential raw ingredients, but they are not alive. Only
        organisms with cells (like plants and green algae) are biological "producers".
      </p>
    </div>
```

```

    </div>

</div>

</div>
)}}

{ /* =====
  TAB: 🍎 EDUCATOR & PARENT ZONE
  ===== */
{activeTab === 'teacher_parent' && (
  <div className="grid grid-cols-1 md:grid-cols-2 gap-6 max-w-4xl mx-auto">

    { /* Teacher Classroom Guide */
    <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-100
space-y-4">
      <div className="flex items-center space-x-2">
        <span className="text-2xl">🍎</span>
        <h2 className="text-lg font-black text-slate-800">Educator Classroom Guide</h2>
      </div>
      <p className="text-xs text-slate-500 font-semibold leading-relaxed">
        This digital module aligns closely with NCERT Class 4 & 5 environmental studies
(EVS) curricula focusing on ecosystems and interdependent habitats.
      </p>

      <div className="space-y-3">
        <div className="border-l-4 border-emerald-500 pl-3">
          <h4 className="font-extrabold text-xs text-slate-700 uppercase">Class Activity
Recommendation</h4>
          <p className="text-xs text-slate-500 mt-1">
            Pass out name cards to students containing labels: "Sun", "Grass", "Caterpillar",
"Bluebird", "Worm". Connect students with colored wool strings. Pull the "Grass" student out to
simulate how ecological damage collapses the thread!
          </p>
        </div>

        <div className="border-l-4 border-emerald-500 pl-3">
          <h4 className="font-extrabold text-xs text-slate-700 uppercase">Key Learning
Check</h4>
          <p className="text-xs text-slate-500 mt-1">
            Check if students realize the arrow points to the receiver of energy, not the mouth
of the eater.
          </p>
        </div>

```

```

    </div>
  </div>

  { /* Parent Kitchen Table Activities */ }
  <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-100
space-y-4">
    <div className="flex items-center space-x-2">
      <span className="text-2xl">🏠</span>
      <h2 className="text-lg font-black text-slate-800">Parent Table Conversations</h2>
    </div>
    <p className="text-xs text-slate-500 font-semibold leading-relaxed">
      Bring ecological awareness to daily conversations over meals or family walks.
    </p>

    <div className="space-y-3">
      <div className="border-l-4 border-amber-500 pl-3">
        <h4 className="font-extrabold text-xs text-slate-700 uppercase">The Dinner Plate
Trace</h4>
        <p className="text-xs text-slate-500 mt-1">
          Pick one item from tonight's dinner plate (e.g., a slice of potato or chicken). Trace it
back in steps to the field, the insects, and the shining Sun.
        </p>
      </div>

      <div className="border-l-4 border-amber-500 pl-3">
        <h4 className="font-extrabold text-xs text-slate-700 uppercase">Observation
Walk</h4>
        <p className="text-xs text-slate-500 mt-1">
          During weekend park outings, search for birds pulling worms out of lawns, or bees
collecting sweet pollen from wild flowers.
        </p>
      </div>
    </div>
  </div>
)}

</main>

{ /* =====
Footer with Copyright / Metadata
===== */ }

```

```
<footer className="bg-slate-900 text-slate-400 py-6 px-6 text-center text-xs mt-12 border-t border-slate-800">
```

```
<p className="font-semibold">The Energy Linker • Premium Interactive EdTech Simulator</p>
```

```
<p className="opacity-60 mt-1">Built in compliance with CBSE/NCERT curriculum and EdTech child-psychology guidelines.</p>
```

```
</footer>
```

```
</div>
```

```
);
```

```
}
```

Earthquake

```

import React, { useState, useEffect, useRef } from 'react';

// Custom SVG Icons for consistent look without external image files
const IconSparkles = () => (
  <svg className="w-5 h-5 text-yellow-400 animate-pulse" fill="none" viewBox="0 0 24 24"
  stroke="currentColor" strokeWidth="2">
    <path strokeLinecap="round" strokeLinejoin="round" d="M5 3v4M3 5h4M6
  17v4m-2-2h4m5-16l2.286 6.857L21 12l-5.714 2.143L13 21l-2.286-6.857L5 12l5.714-2.143L13
  3z" />
  </svg>
);

const IconInfo = () => (
  <svg className="w-5 h-5 text-blue-400" fill="none" viewBox="0 0 24 24"
  stroke="currentColor" strokeWidth="2">
    <path strokeLinecap="round" strokeLinejoin="round" d="M13 16h-1v-4h-1m1-4h.01M21 12a9
  9 0 11-18 0 9 0 0 118 0z" />
  </svg>
);

const IconAlert = () => (
  <svg className="w-6 h-6 text-red-500 animate-bounce" fill="none" viewBox="0 0 24 24"
  stroke="currentColor" strokeWidth="2">
    <path strokeLinecap="round" strokeLinejoin="round" d="M12 9v2m0 4h.01m-6.938
  4h13.856c1.54 0 2.502-1.667 1.732-3L13.732 4c-.77-1.333-2.694-1.333-3.464 0L3.34 16c-.77
  1.333.192 3 1.732 3z" />
  </svg>
);

const IconCheck = () => (
  <svg className="w-6 h-6 text-green-500" fill="none" viewBox="0 0 24 24"
  stroke="currentColor" strokeWidth="3">
    <path strokeLinecap="round" strokeLinejoin="round" d="M5 13l4 4L19 7" />
  </svg>
);

const IconCross = () => (
  <svg className="w-6 h-6 text-red-500" fill="none" viewBox="0 0 24 24" stroke="currentColor"
  strokeWidth="3">
    <path strokeLinecap="round" strokeLinejoin="round" d="M6 18L18 6M6 6l12 12" />
  </svg>
);

export default function App() {

```

```

// State Management
const [activeTab, setActiveTab] = useState('lab'); // lab, explanation, diy, quiz
const [intensity, setIntensity] = useState(0.0); // Magnitude (0.0 to 10.0)

// Custom structural modifiers
const [modifiers, setModifiers] = useState({
  crossBracing: false,
  baseIsolators: false,
  massDamper: false
});

// Simulator tracking states
const [isSimulationRunning, setIsSimulationRunning] = useState(false);
const [buildingACollapseTime, setBuildingACollapseTime] = useState(0); // in frames / time
units
const [buildingAStatus, setBuildingAStatus] = useState('Stable'); // Stable, Stressed, Collapsed
const [buildingBStatus, setBuildingBStatus] = useState('Stable'); // Stable, Stressed, Safe
const [showCollapseModal, setShowCollapseModal] = useState(false);
const [showSuccessModal, setShowSuccessModal] = useState(false);
const [activePrediction, setActivePrediction] = useState(null); // 'A' or 'B'
const [points, setPoints] = useState(100);
const [achievement, setAchievement] = useState('Junior Apprentice');
const [collapsesWitnessed, setCollapsesWitnessed] = useState(0);

// Quiz State
const [currentQuizIndex, setCurrentQuizIndex] = useState(0);
const [selectedAnswer, setSelectedAnswer] = useState(null);
const [quizScore, setQuizScore] = useState(0);
const [quizFinished, setQuizFinished] = useState(false);
const [answeredQuestions, setAnsweredQuestions] = useState({}); // { index: { selected,
correct } }

// Canvas ref
const canvasRef = useRef(null);
const animationFrameId = useRef(null);
const timeRef = useRef(0);

// Ground vibration helper variables
const b1State = useRef({ y1: 0, y2: 0, y3: 0, rot1: 0, rot2: 0, rot3: 0, collapsed: false, fallY: 0,
jointsStress: 0 });
const b2State = useRef({ y1: 0, y2: 0, y3: 0, rot1: 0, rot2: 0, rot3: 0, collapsed: false, fallY: 0,
jointsStress: 0 });

// Quiz Questions Data

```

```
const quizQuestions = [
  {
    q: "Which geometric shape is the strongest and most commonly used to make building frames rigid?",
    options: ["Square", "Triangle", "Circle", "Rectangle"],
    correct: 1,
    explanation: "Triangles cannot be easily deformed without breaking their sides, making them perfect for distributing structural loads under horizontal and vertical stress."
  },
  {
    q: "A heavy weight placed at the top of a skyscraper that swings in the opposite direction of an earthquake is called a:",
    options: ["Base Isolator", "Seismic Wave", "Tuned Mass Damper", "Foundation Anchor"],
    correct: 2,
    explanation: "Tuned mass dampers act like massive pendulums that pull the top of the building backward when wind or earthquakes push it forward, canceling out the dangerous sway."
  },
  {
    q: "Earthquakes cause primary structural damage to buildings mostly through which type of motion?",
    options: ["Horizontal (side-to-side) shaking", "Vertical (up-and-down) jumps", "Rotational spinning", "No movement at all"],
    correct: 0,
    explanation: "Side-to-side (lateral) forces are highly destructive because traditional buildings are built primarily to withstand gravity (downward forces) rather than lateral sheer stresses."
  },
  {
    q: "Base isolators protect buildings by:",
    options: ["Making the building completely stiff and rigid", "Absorbing and separating (decoupling) the building from the shaking ground", "Creating an electromagnetic forcefield around the structure", "Alerting local emergency services before an earthquake starts"],
    correct: 1,
    explanation: "Base isolators act like heavy-duty car shock absorbers, letting the earth slide back and forth underneath while the main structure floats smoothly above it."
  },
  {
    q: "What is the instrument used by scientists to measure and record earthquake waves?",
    options: ["Thermometer", "Barometer", "Seismograph", "Speedometer"],
    correct: 2,
    explanation: "A seismograph records the seismic waves caused by earthquakes, drawing zigzag patterns on paper or digital sensors based on the intensity of vibrations."
  },
  {

```


q: "Why do tall skyscrapers sway more than short houses during long, rolling, low-frequency seismic waves?",

options: ["Tall buildings are lighter", "Tall buildings have a high center of mass and match the low resonance frequency of long rolling waves", "Wind pushes them down automatically during earthquakes", "Short houses do not experience earthquake waves at all"],

correct: 1,

explanation: "Skyscrapers are highly flexible. When slow, long waves move through soft soil, they can match the natural swing rate (resonance) of tall buildings, causing them to sway dramatically."

},

{

q: "What is the primary material component of a base isolation pad?",

options: ["Plaster of Paris", "Bricks and concrete", "Alternating layers of rubber, steel, and lead", "Fragile glass blocks"],

correct: 2,

explanation: "Alternating layers of flexible rubber (to absorb horizontal movement) and rigid steel plates (to hold structural weight) with a lead plug to dissipate kinetic energy are used."

},

{

q: "If an engineer designs a school building in a seismic zone using unreinforced brick walls, is it safe?",

options: ["Yes, bricks are incredibly heavy and resist all shaking", "Yes, mortar can never break", "No, unreinforced brick walls are highly brittle and shatter under horizontal shear forces", "No, bricks dissolve in water"],

correct: 2,

explanation: "Brick walls hold gravity loads well, but they lack tensile flexibility. When subjected to lateral shaking, they quickly crack and collapse catastrophically."

},

{

q: "What is 'soil liquefaction' during an earthquake?",

options: ["Soil turns into hard solid bedrock", "Loose, water-saturated sandy soil temporarily acts like a fluid, making buildings sink", "The ground turns into real volcanic lava", "Plants grow twice as fast due to shaking"],

correct: 1,

explanation: "Severe shaking squeezes water trapped between loose sand particles, turning solid soil into a soupy mixture that cannot support building weights."

},

{

q: "What is the main role of structural flexibility in earthquake design?",

options: ["To make buildings look cool", "To let the building fold down into a compact box", "To safely absorb and dissipate seismic energy by bending without breaking", "To generate green electricity from building vibrations"],

correct: 2,

explanation: "A building that is too rigid will crack instantly. Controlled flexibility allows the frame to bend slightly and return safely to its original shape."

```
}  
];
```

// Point trigger system

```
const triggerPointsAward = (value, message) => {  
  setPoints(prev => {  
    const next = prev + value;  
    // Upgrade title based on points  
    if (next >= 300) setAchievement('Seismic Master Elite 🌟');  
    else if (next >= 200) setAchievement('Senior Structural Engineer 🛠️');  
    else if (next >= 150) setAchievement('Assistant Architect 📐');  
    return next;  
  });  
};
```

// Reset simulator state

```
const resetSimulator = () => {  
  setIntensity(0);  
  setIsSimulationRunning(false);  
  b1State.current = { y1: 0, y2: 0, y3: 0, rot1: 0, rot2: 0, rot3: 0, collapsed: false, fallY: 0,  
jointsStress: 0 };  
  b2State.current = { y1: 0, y2: 0, y3: 0, rot1: 0, rot2: 0, rot3: 0, collapsed: false, fallY: 0,  
jointsStress: 0 };  
  setBuildingAStatus('Stable');  
  setBuildingBStatus('Stable');  
  setShowCollapseModal(false);  
  setShowSuccessModal(false);  
};
```

// Run/Stop Simulator

```
const handleToggleSimulation = () => {  
  if (isSimulationRunning) {  
    resetSimulator();  
  } else {  
    setIsSimulationRunning(true);  
    if (intensity === 0) setIntensity(5.0); // automatically set mid shake  
  }  
};
```

// Trigger modification toggle

```
const toggleModifier = (key) => {  
  setModifiers(prev => {
```

```

    const updated = { ...prev, [key]: !prev[key] };
    // Friendly feedback
    if (updated[key]) {
      triggerPointsAward(15, `Added ${key === 'crossBracing' ? 'Diagonal Cross-Braces' : key
=== 'baseIsolators' ? 'Base Isolation Bearings' : 'Tuned Mass Damper'}`);
    }
    return updated;
  });
};

```

```

// Draw simulation loop
useEffect(() => {
  const canvas = canvasRef.current;
  if (!canvas) return;
  const ctx = canvas.getContext('2d');

  // Make canvas responsive
  const resizeCanvas = () => {
    canvas.width = canvas.parentElement.clientWidth;
    canvas.height = 420;
  };
  resizeCanvas();
  window.addEventListener('resize', resizeCanvas);

```

```

const render = () => {
  const w = canvas.width;
  const h = canvas.height;
  ctx.clearRect(0, 0, w, h);

  // Simple grid background
  ctx.strokeStyle = '#334155';
  ctx.lineWidth = 1;
  const gridSize = 30;
  for (let x = 0; x < w; x += gridSize) {
    ctx.beginPath();
    ctx.moveTo(x, 0);
    ctx.lineTo(x, h);
    ctx.stroke();
  }
  for (let y = 0; y < h; y += gridSize) {
    ctx.beginPath();
    ctx.moveTo(0, y);
    ctx.lineTo(w, y);
    ctx.stroke();
  }

```

```

}

// Time accumulator
if (isSimulationRunning && intensity > 0) {
  timeRef.current += 0.15 + (intensity * 0.05);
} else {
  timeRef.current = 0;
}

const t = timeRef.current;

// Calculate dynamic shake ground displacement
// Frequency and amplitude grow with intensity
const shakeFreq = 1.2 + (intensity * 0.1);
const shakeAmp = intensity * 4.2;
const groundDisplacement = intensity > 0 ? Math.sin(t * shakeFreq) * shakeAmp : 0;

// Base Y coordinates
const groundLineY = h - 70;

// Draw Rock Strata Background (Visual depth)
ctx.fillStyle = '#1e293b';
ctx.fillRect(0, groundLineY, w, 100);
ctx.fillStyle = '#0f172a';
ctx.fillRect(0, groundLineY + 25, w, 75);

// Draw Seismic waves visualizer lines on ground
if (intensity > 0) {
  ctx.strokeStyle = `rgba(239, 68, 68, ${0.1 + (intensity / 10) * 0.6})`;
  ctx.lineWidth = 2 + (intensity / 3);
  ctx.beginPath();
  for (let ix = 0; ix < w; ix += 10) {
    const iy = groundLineY + 30 + Math.sin((ix + t * 40) * 0.05) * (intensity * 2);
    if (ix === 0) ctx.moveTo(ix, iy);
    else ctx.lineTo(ix, iy);
  }
  ctx.stroke();
}

// Draw Shake Table Rails
ctx.strokeStyle = '#475569';
ctx.lineWidth = 4;
ctx.beginPath();
ctx.moveTo(30, groundLineY);

```

```
ctx.lineTo(w - 30, groundLineY);  
ctx.stroke();
```

```
// Platform coordinates (Two separate shake plates)  
const plateWidth = w * 0.38;  
const plateA_X = (w * 0.25) - (plateWidth / 2) + groundDisplacement;  
const plateB_X = (w * 0.75) - (plateWidth / 2) + (modifiers.basisolators ?  
groundDisplacement * 0.2 : groundDisplacement);  
  
// Draw Shock absorbers / Rollers under Plate B if Base Isolators are active  
if (modifiers.basisolators) {  
  ctx.strokeStyle = '#a855f7';  
  ctx.fillStyle = '#3b0764';  
  ctx.lineWidth = 3;  
  const rollerCount = 6;  
  const step = plateWidth / (rollerCount - 1);  
  for (let i = 0; i < rollerCount; i++) {  
    const rx = (w * 0.75) - (plateWidth / 2) + (i * step) + (groundDisplacement * 0.6);  
    const ry = groundLineY - 12;  
    ctx.beginPath();  
    ctx.arc(rx, ry, 10, 0, Math.PI * 2);  
    ctx.fill();  
    ctx.stroke();  
    // Inner rubber lines  
    ctx.beginPath();  
    ctx.arc(rx, ry, 5, 0, Math.PI * 2);  
    ctx.stroke();  
  }  
}
```

```
// Draw Shake Plates
```

```
// Plate A
```

```
ctx.fillStyle = '#4b5563';  
ctx.fillRect(plateA_X, groundLineY - 10, plateWidth, 12);  
ctx.strokeStyle = '#9ca3af';  
ctx.strokeRect(plateA_X, groundLineY - 10, plateWidth, 12);
```

```
// Plate B
```

```
ctx.fillStyle = modifiers.basisolators ? '#6b21a8' : '#4b5563';  
ctx.fillRect(plateB_X, groundLineY - 10, plateWidth, 12);  
ctx.strokeStyle = modifiers.basisolators ? '#d8b4fe' : '#9ca3af';  
ctx.strokeRect(plateB_X, groundLineY - 10, plateWidth, 12);
```

```
// Calculate Physics for Building A (Control Group)
```

// Building A sways naturally depending on height. It is rigid but will collapse at intensity >= 6.5

```
let bA_CollapseThreshold = 6.5;
let bA_SwayAmpFactor = intensity * 1.8;

if (intensity >= bA_CollapseThreshold && isSimulationRunning) {
  b1State.current.jointsStress += 0.01 * intensity;
  if (b1State.current.jointsStress >= 1.0) {
    b1State.current.collapsed = true;
    setBuildingAStatus('Collapsed');
  } else {
    setBuildingAStatus('Stressed');
  }
} else if (intensity > 0) {
  setBuildingAStatus('Stressed');
  b1State.current.jointsStress = Math.min(b1State.current.jointsStress + 0.005, 0.6);
} else {
  setBuildingAStatus('Stable');
  b1State.current.jointsStress = Math.max(b1State.current.jointsStress - 0.02, 0);
}
```

```
// Calculate positions of Building A
const bHeight = 180;
const bWidth = 80;
const bCenterX_A = plateA_X + (plateWidth / 2);
const bBaseY = groundLineY - 10;
```

```
// Base sway lag
const swayPhase1 = Math.sin(t * 1.0) * bA_SwayAmpFactor;
const swayPhase2 = Math.sin(t * 0.9 - 0.4) * bA_SwayAmpFactor * 1.8;
const swayPhase3 = Math.sin(t * 0.8 - 0.8) * bA_SwayAmpFactor * 2.8;
```

```
if (!b1State.current.collapsed) {
  b1State.current.y1 = bBaseY - 60;
  b1State.current.y2 = bBaseY - 120;
  b1State.current.y3 = bBaseY - 180;
  b1State.current.rot1 = swayPhase1 * 0.15;
  b1State.current.rot2 = swayPhase2 * 0.22;
  b1State.current.rot3 = swayPhase3 * 0.30;
} else {
  // Falling down physics
  b1State.current.fallY = Math.min(b1State.current.fallY + 4.5, 120);
  b1State.current.rot1 += (Math.random() - 0.5) * 0.05;
  b1State.current.rot2 += (Math.random() - 0.5) * 0.08;
```

```

    b1State.current.rot3 += (Math.random() - 0.5) * 0.15;
}

// Physics for Building B (Test Group with toggled modules)
let bB_SwayAmpFactor = intensity * 1.8;

// Apply cross bracing rigidity
if (modifiers.crossBracing) {
    bB_SwayAmpFactor *= 0.35; // Cuts dynamic lateral flexing significantly
}
// Apply base isolation buffer reduction
if (modifiers.baseIsolators) {
    bB_SwayAmpFactor *= 0.2; // Drastically drops sway amplitude
}
// Tuned Mass Damper action (creates counteracting sway)
const damperEffect = modifiers.massDamper ? Math.sin(t * 0.8 + Math.PI) * (intensity * 1.8)
: 0;

const bCenterX_B = plateB_X + (plateWidth / 2);

const swayPhase1_B = Math.sin(t * 1.0) * bB_SwayAmpFactor;
const swayPhase2_B = Math.sin(t * 0.9 - 0.4) * bB_SwayAmpFactor * 1.8;
const swayPhase3_B = (Math.sin(t * 0.8 - 0.8) * bB_SwayAmpFactor * 2.8) + damperEffect;

// Determine collapse logic for building B
// B only collapses if no survival upgrades are active and intensity is super high
let bB_CollapseThreshold = 6.5;
if (modifiers.crossBracing && modifiers.baseIsolators && modifiers.massDamper) {
    bB_CollapseThreshold = 11.0; // Invincible!
} else if (modifiers.crossBracing && modifiers.baseIsolators) {
    bB_CollapseThreshold = 9.8;
} else if (modifiers.crossBracing || modifiers.baseIsolators) {
    bB_CollapseThreshold = 8.5;
} else if (modifiers.massDamper) {
    bB_CollapseThreshold = 7.5;
}

if (intensity >= bB_CollapseThreshold && isSimulationRunning) {
    b2State.current.jointsStress += 0.01 * intensity;
    if (b2State.current.jointsStress >= 1.0) {
        b2State.current.collapsed = true;
        setBuildingBStatus('Collapsed');
    } else {
        setBuildingBStatus('Stressed');
    }
}

```

```

    }
  } else if (intensity > 0) {
    setBuildingBStatus(bB_SwayAmpFactor < 2.0 ? 'Safe' : 'Stressed');
    b2State.current.jointsStress = Math.max(0, Math.min(b2State.current.jointsStress + 0.003,
0.4));
  } else {
    setBuildingBStatus('Stable');
    b2State.current.jointsStress = Math.max(b2State.current.jointsStress - 0.02, 0);
  }

  if (!b2State.current.collapsed) {
    b2State.current.y1 = bBaseY - 60;
    b2State.current.y2 = bBaseY - 120;
    b2State.current.y3 = bBaseY - 180;
    b2State.current.rot1 = swayPhase1_B * 0.15;
    b2State.current.rot2 = swayPhase2_B * 0.22;
    b2State.current.rot3 = swayPhase3_B * 0.30;
  } else {
    b2State.current.fallY = Math.min(b2State.current.fallY + 4.5, 120);
    b2State.current.rot1 += (Math.random() - 0.5) * 0.05;
    b2State.current.rot2 += (Math.random() - 0.5) * 0.08;
    b2State.current.rot3 += (Math.random() - 0.5) * 0.15;
  }

  // --- DRAW BUILDING A ---
  ctx.save();
  const drawBuilding = (state, baseCenterX, colorTheme, title, isControl) => {
    const stressVal = state.jointsStress;
    let strokeColor = colorTheme.normal;
    if (stressVal > 0.7) {
      strokeColor = `rgb(239, 68, 68)`; // Bright warning red
    } else if (stressVal > 0.4) {
      strokeColor = `rgb(249, 115, 22)`; // Warning orange
    }

    // Foundation point
    const x0 = baseCenterX;
    const y0 = bBaseY;

    // Calculate floor joint coordinates
    let x1 = baseCenterX + state.rot1;
    let y1 = state.y1 + (state.collapsed ? state.fallY * 0.3 : 0);

```



```
let x2 = baseCenterX + state.rot2;  
let y2 = state.y2 + (state.collapsed ? state.fallY * 0.6 : 0);
```

```
let x3 = baseCenterX + state.rot3;  
let y3 = state.y3 + (state.collapsed ? state.fallY : 0);
```

```
// Adjust for collapse rubble look  
if (state.collapsed) {  
  x1 += Math.sin(state.fallY) * 10;  
  x2 -= Math.sin(state.fallY) * 15;  
  x3 += Math.cos(state.fallY) * 25;  
}
```

```
// Draw structural columns (vertical pillars)  
ctx.strokeStyle = strokeColor;  
ctx.lineWidth = 6;  
ctx.lineCap = 'round';  
ctx.lineJoin = 'round';
```

```
// Ground columns to Floor 1  
ctx.beginPath();  
ctx.moveTo(x0 - bWidth / 2, y0);  
ctx.lineTo(x1 - bWidth / 2, y1);  
ctx.moveTo(x0 + bWidth / 2, y0);  
ctx.lineTo(x1 + bWidth / 2, y1);  
// Floor 1 to Floor 2  
ctx.moveTo(x1 - bWidth / 2, y1);  
ctx.lineTo(x2 - bWidth / 2, y2);  
ctx.moveTo(x1 + bWidth / 2, y1);  
ctx.lineTo(x2 + bWidth / 2, y2);  
// Floor 2 to Floor 3 (Roof)  
ctx.moveTo(x2 - bWidth / 2, y2);  
ctx.lineTo(x3 - bWidth / 2, y3);  
ctx.moveTo(x2 + bWidth / 2, y2);  
ctx.lineTo(x3 + bWidth / 2, y3);  
ctx.stroke();
```

```
// Draw structural horizontal floors (slabs)  
ctx.fillStyle = colorTheme.floorFill;  
ctx.strokeStyle = strokeColor;  
ctx.lineWidth = 4;
```

```
// Floor 1 Slab  
ctx.beginPath();
```

```

ctx.rect(x1 - (bWidth + 10) / 2, y1 - 4, bWidth + 10, 8);
ctx.fill();
ctx.stroke();

// Floor 2 Slab
ctx.beginPath();
ctx.rect(x2 - (bWidth + 10) / 2, y2 - 4, bWidth + 10, 8);
ctx.fill();
ctx.stroke();

// Roof Slab
ctx.beginPath();
ctx.rect(x3 - (bWidth + 14) / 2, y3 - 5, bWidth + 14, 10);
ctx.fill();
ctx.stroke();

// Add Cross Bracing lines on B if active
if (!isControl && modifiers.crossBracing && !state.collapsed) {
  ctx.strokeStyle = '#22c55e'; // green bracing lines
  ctx.lineWidth = 3;
  ctx.beginPath();
  // X-brace on floor 1
  ctx.moveTo(x0 - bWidth/2, y0);
  ctx.lineTo(x1 + bWidth/2, y1);
  ctx.moveTo(x0 + bWidth/2, y0);
  ctx.lineTo(x1 - bWidth/2, y1);
  // X-brace on floor 2
  ctx.moveTo(x1 - bWidth/2, y1);
  ctx.lineTo(x2 + bWidth/2, y2);
  ctx.moveTo(x1 + bWidth/2, y1);
  ctx.lineTo(x2 - bWidth/2, y2);
  // X-brace on floor 3
  ctx.moveTo(x2 - bWidth/2, y2);
  ctx.lineTo(x3 + bWidth/2, y3);
  ctx.moveTo(x2 + bWidth/2, y2);
  ctx.lineTo(x3 - bWidth/2, y3);
  ctx.stroke();
}

// Draw Tuned Mass Damper on top floor of B if active
if (!isControl && modifiers.massDamper && !state.collapsed) {
  // Pendulum origin point
  const damperOriginX = x3;
  const damperOriginY = y3 + 10;

```

```

// Pendulum swings opposite to building sway
const dAngle = Math.sin(t * 0.8) * 0.45;
const len = 30;
const ballX = damperOriginX + Math.sin(dAngle) * len;
const ballY = damperOriginY + Math.cos(dAngle) * len;

// Wire/string
ctx.strokeStyle = '#38bdf8';
ctx.lineWidth = 3;
ctx.beginPath();
ctx.moveTo(damperOriginX, damperOriginY);
ctx.lineTo(ballX, ballY);
ctx.stroke();

// Heavy golden pendulum mass
ctx.fillStyle = '#f59e0b';
ctx.strokeStyle = '#fbbf24';
ctx.lineWidth = 2;
ctx.beginPath();
ctx.arc(ballX, ballY, 12, 0, Math.PI * 2);
ctx.fill();
ctx.stroke();

// Action counterweight wave indicator
ctx.fillStyle = '#38bdf8';
ctx.font = '10px sans-serif';
ctx.fillText("TMD Damping Active", x3 - 45, y3 - 15);
}

// Draw Windows (Adds scaling realism)
ctx.fillStyle = 'rgba(14, 165, 233, 0.4)';
const winSize = 10;
const winDraw = (fx, fy) => {
  if (state.collapsed) return;
  ctx.fillRect(fx - 25, fy - 35, winSize, winSize);
  ctx.fillRect(fx + 15, fy - 35, winSize, winSize);
};
winDraw(x1, y1);
winDraw(x2, y2);
winDraw(x3, y3);

// Draw Joint Connector nodes (glowing indicator dots)
const drawJointNode = (jx, jy) => {
  ctx.fillStyle = stressVal > 0.7 ? '#ef4444' : stressVal > 0.4 ? '#f97316' : '#22c55e';

```

```

ctx.beginPath();
ctx.arc(jx, jy, 5, 0, Math.PI * 2);
ctx.fill();
// Simple glowing pulse ring
ctx.strokeStyle = ctx.fillStyle;
ctx.lineWidth = 1;
ctx.beginPath();
ctx.arc(jx, jy, 8 + Math.sin(t * 3) * 2, 0, Math.PI * 2);
ctx.stroke();
};

if (!state.collapsed) {
  drawJointNode(x1 - bWidth/2, y1);
  drawJointNode(x1 + bWidth/2, y1);
  drawJointNode(x2 - bWidth/2, y2);
  drawJointNode(x2 + bWidth/2, y2);
  drawJointNode(x3 - bWidth/2, y3);
  drawJointNode(x3 + bWidth/2, y3);
} else {
  // Fallen structural rubble debris indicators
  ctx.fillStyle = '#6b7280';
  ctx.font = 'bold 12px sans-serif';
  ctx.fillText("COLLAPSED!", baseCenterX - 35, y0 - 30);
}

// Title text
ctx.fillStyle = '#f8f9fc';
ctx.font = 'bold 14px sans-serif';
ctx.textAlign = 'center';
ctx.fillText(title, baseCenterX, y0 + 35);

// Subtitle status
let statText = "Status: Stable";
let statColor = '#22c55e';
if (state.collapsed) {
  statText = "CRITICAL FAIL";
  statColor = '#ef4444';
} else if (stressVal > 0.7) {
  statText = "HIGH DANGER";
  statColor = '#f97316';
} else if (stressVal > 0.2) {
  statText = "VIBRATING";
  statColor = '#eab308';
} else if (intensity > 0) {

```

```

    statText = "SAFE (Mitigated)";
    statColor = '#10b981';
  }

  ctx.fillStyle = statColor;
  ctx.font = '12px sans-serif';
  ctx.fillText(statText, baseCenterX, y0 + 52);
};

// Draw Control Building A
drawBuilding(b1State.current, w * 0.25, {
  normal: '#ef4444',
  floorFill: '#334155'
}, "Building A (Unreinforced)", true);

// Draw Test Building B
drawBuilding(b2State.current, w * 0.75, {
  normal: modifiers.baselsolators || modifiers.crossBracing || modifiers.massDamper ?
'#a855f7' : '#94a3b8',
  floorFill: '#1e293b'
}, "Building B (Test Group)", false);

ctx.restore();

// Trigger educational feedback overlay states
if (b1State.current.collapsed && !showCollapseModal && activePrediction === 'B') {
  setShowCollapseModal(true);
  setCollapsesWitnessed(c => c + 1);
  triggerPointsAward(20, "Discovered brittle joint structural limits!");
}

if (intensity >= 7.0 && !b2State.current.collapsed && (modifiers.crossBracing ||
modifiers.baselsolators || modifiers.massDamper) && !showSuccessModal) {
  setShowSuccessModal(true);
  triggerPointsAward(50, "Successfully saved the skyscraper!");
}

animationFrameId.current = requestAnimationFrame(render);
};

render();

return () => {
  cancelAnimationFrame(animationFrameId.current);

```

```

    window.removeEventListener('resize', resizeCanvas);
  };
}, [isSimulationRunning, intensity, modifiers, activePrediction]);

// Handle Quiz selection
const handleQuizAnswer = (optionIdx) => {
  if (selectedAnswer !== null) return; // Prevent double clicking
  setSelectedAnswer(optionIdx);

  const isCorrect = optionIdx === quizQuestions[currentQuizIndex].correct;
  if (isCorrect) {
    setQuizScore(s => s + 1);
    triggerPointsAward(10, "Correct Quiz Answer!");
  }

  setAnsweredQuestions(prev => ({
    ...prev,
    [currentQuizIndex]: {
      selected: optionIdx,
      correct: quizQuestions[currentQuizIndex].correct
    }
  }));
};

const handleNextQuiz = () => {
  setSelectedAnswer(null);
  if (currentQuizIndex < quizQuestions.length - 1) {
    setCurrentQuizIndex(currentQuizIndex + 1);
  } else {
    setQuizFinished(true);
    triggerPointsAward(30, "Completed the whole assessment!");
  }
};

const restartQuiz = () => {
  setCurrentQuizIndex(0);
  setSelectedAnswer(null);
  setQuizScore(0);
  setQuizFinished(false);
  setAnsweredQuestions({});
};

return (

```

```

<div className="min-h-screen bg-slate-950 text-slate-100 flex flex-col font-sans
selection:bg-indigo-500 selection:text-white">

  {/* Premium Gamified Header Banner */}
  <header className="bg-slate-900/90 border-b border-slate-800 backdrop-blur-md sticky
top-0 z-40 px-4 py-3 md:px-8 shadow-lg">
    <div className="max-w-7xl mx-auto flex flex-col md:flex-row justify-between items-center
gap-4">

      <div className="flex items-center gap-3">
        <div className="bg-gradient-to-tr from-rose-500 to-indigo-600 p-2.5 rounded-xl
shadow-inner animate-pulse">
          <svg className="w-6 h-6 text-white" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="2.5">
            <path strokeLinecap="round" strokeLinejoin="round" d="M19 11H5m14 0a2 2 0 0 12
2v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2 2v6a2 2 0 0 12-2m14 0V9a2 2 0 0 0-2-2M5 11V9a2 2 0
0 12-2m0 0V5a2 2 0 0 12-2h6a2 2 0 0 12 2v2M7 7h10" />
          </svg>
        </div>
        <div>
          <h1 className="text-xl md:text-2xl font-black bg-gradient-to-r from-rose-400
via-indigo-200 to-purple-400 bg-clip-text text-transparent">
            THE SHAKE LAB
          </h1>
          <p className="text-xs text-slate-400 font-medium tracking-wide">
            NCERT Science • Class 8 • Natural Hazards
          </p>
        </div>
      </div>

      <div>
        <h1 className="text-xl md:text-2xl font-black bg-gradient-to-r from-rose-400
via-indigo-200 to-purple-400 bg-clip-text text-transparent">
          THE SHAKE LAB
        </h1>
        <p className="text-xs text-slate-400 font-medium tracking-wide">
          NCERT Science • Class 8 • Natural Hazards
        </p>
      </div>
    </div>

    <div>
      <h1 className="text-xl md:text-2xl font-black bg-gradient-to-r from-rose-400
via-indigo-200 to-purple-400 bg-clip-text text-transparent">
        THE SHAKE LAB
      </h1>
      <p className="text-xs text-slate-400 font-medium tracking-wide">
        NCERT Science • Class 8 • Natural Hazards
      </p>
    </div>
  </div>

  <div>
    <h1 className="text-xl md:text-2xl font-black bg-gradient-to-r from-rose-400
via-indigo-200 to-purple-400 bg-clip-text text-transparent">
      THE SHAKE LAB
    </h1>
    <p className="text-xs text-slate-400 font-medium tracking-wide">
      NCERT Science • Class 8 • Natural Hazards
    </p>
  </div>

  <div>
    <h1 className="text-xl md:text-2xl font-black bg-gradient-to-r from-rose-400
via-indigo-200 to-purple-400 bg-clip-text text-transparent">
      THE SHAKE LAB
    </h1>
    <p className="text-xs text-slate-400 font-medium tracking-wide">
      NCERT Science • Class 8 • Natural Hazards
    </p>
  </div>

```

```

        ? 'bg-gradient-to-r from-indigo-600 to-violet-600 text-white shadow-lg
shadow-indigo-900/30'
        : 'text-slate-400 hover:text-slate-200 hover:bg-slate-900'
    }}
  >
    {tab.label}
  </button>
)}}
</nav>

{/* Gamified Achievements HUD */}
<div className="flex items-center gap-4 bg-slate-950 px-4 py-2 rounded-xl border
border-slate-800">
  <div className="text-right">
    <span className="block text-[10px] text-slate-500 font-extrabold uppercase
tracking-widest">Architect Rank</span>
    <span className="text-xs md:text-sm font-bold
text-indigo-400">{achievement}</span>
  </div>
  <div className="h-8 w-[1px] bg-slate-800" />
  <div className="flex items-center gap-1.5">
    <span className="text-yellow-500 font-bold">💎</span>
    <span className="font-extrabold text-sm md:text-base text-yellow-400">{points}</span>
  <span className="text-xs text-slate-500 font-normal">XP</span></div>
</div>

</div>
</header>

{/* Main Container */}
<main className="flex-1 max-w-7xl w-full mx-auto p-4 md:p-8 flex flex-col justify-center">

  {activeTab === 'lab' && (
    <div className="grid grid-cols-1 lg:grid-cols-12 gap-8 items-start">

      {/* Left side panel: Simulator View */}
      <div className="lg:col-span-8 bg-slate-900 rounded-3xl overflow-hidden border
border-slate-800 shadow-2xl relative">

        {/* Header inside canvas screen */}
        <div className="p-4 bg-slate-950/80 border-b border-slate-800 flex justify-between
items-center">
          <div className="flex items-center gap-2">

```



```

    <span className="relative flex h-3 w-3">
      <span className={`animate-ping absolute inline-flex h-full w-full rounded-full
opacity-75 ${intensity > 0 ? 'bg-red-500' : 'bg-green-400'}`}></span>
      <span className={`relative inline-flex rounded-full h-3 w-3 ${intensity > 0 ?
'bg-red-600' : 'bg-green-500'}`}></span>
    </span>
    <span className="text-xs font-bold text-slate-300">
      {intensity > 0 ? 'EARTHQUAKE WAVE SIMULATION ACTIVE' : 'SYSTEM CALM -
SHAKE TABLE STANDBY'}
    </span>
  </div>
  <div className="text-xs text-slate-400 font-mono">
    Ground Accel: <span className="text-indigo-400 font-bold">{(intensity *
0.98).toFixed(2)}g</span>
  </div>
</div>

{/* Prediction overlay if not set */}
{activePrediction === null && (
  <div className="absolute inset-0 bg-slate-950/95 backdrop-blur-md z-20 flex flex-col
items-center justify-center p-8 text-center">
    <div className="max-w-md">
      <span className="text-4xl">🤔</span>
      <h3 className="text-2xl font-black mt-3 text-slate-100">Make Your Seismic
Hypothesis!</h3>
      <p className="text-slate-400 mt-2 text-sm leading-relaxed">
        We have two structures. Building A has a basic unreinforced concrete frame.
        Building B has experimental customizable design points.
        Who will withstand tremors better when ground shaking starts?
      </p>
      <div className="flex gap-4 mt-6 justify-center">
        <button
          onClick={() => {
            setActivePrediction('A');
            triggerPointsAward(10, 'Formed a prediction');
          }}
          className="px-6 py-3 bg-red-950/60 border border-red-700/50 rounded-xl
text-red-200 font-bold hover:bg-red-900/60 transition-all text-xs md:text-sm"
        >
          Building A wins
        </button>
        <button
          onClick={() => {
            setActivePrediction('B');

```

```

        triggerPointsAward(10, 'Formed a prediction');
    }}
    className="px-6 py-3 bg-indigo-950/60 border border-indigo-700/50 rounded-xl
text-indigo-200 font-bold hover:bg-indigo-900/60 transition-all text-xs md:text-sm"
    >
        Building B is safer
    </button>
</div>
</div>
</div>
)}}

```

```

{/* Dynamic canvas drawing space */}
<div className="relative">
    <canvas ref={canvasRef} className="block w-full h-[420px]" />

    {/* Micro tooltip status icons */}
    <div className="absolute top-4 left-4 bg-slate-950/80 backdrop-blur-sm p-3
rounded-xl border border-slate-800 text-[11px] space-y-1 font-mono">
        <div className="flex items-center gap-1.5">
            <span className="text-red-400 font-bold">A Status:</span>
            <span className={buildingAStatus === 'Collapsed' ? 'text-red-500 font-black' :
buildingAStatus === 'Stressed' ? 'text-orange-400' : 'text-green-400'}>
                {buildingAStatus.toUpperCase()}
            </span>
        </div>
        <div className="flex items-center gap-1.5">
            <span className="text-purple-400 font-bold">B Status:</span>
            <span className={buildingBStatus === 'Collapsed' ? 'text-red-500 font-black' :
buildingBStatus === 'Safe' ? 'text-green-400 font-bold' : 'text-orange-400'}>
                {buildingBStatus.toUpperCase()}
            </span>
        </div>
    </div>
</div>

```

```

{/* Control Deck Interface */}
<div className="p-6 bg-slate-950 border-t border-slate-800/80 space-y-6">

    {/* Slider bar */}
    <div>
        <div className="flex justify-between items-center mb-2">
            <label className="text-sm font-extrabold text-slate-300 flex items-center
gap-1.5">

```

```

    <span>🔥</span>
    <span>Adjust Earthquake Magnitude (Richter Scale Value)</span>
  </label>
  <span className={`px-2.5 py-1 rounded-lg text-xs font-bold font-mono ${
    intensity >= 7.5 ? 'bg-red-500/20 text-red-400 animate-pulse border
border-red-500/30' :
    intensity >= 4.5 ? 'bg-amber-500/20 text-amber-400' :
    'bg-emerald-500/20 text-emerald-400'
  }`}>
    Magnitude {intensity.toFixed(1)}
  </span>
</div>

<div className="flex items-center gap-4">
  <span className="text-xs font-bold text-emerald-500">M 0.0</span>
  <input
    type="range"
    min="0"
    max="10"
    step="0.1"
    value={intensity}
    onChange={(e) => {
      setIntensity(parseFloat(e.target.value));
      if (parseFloat(e.target.value) > 0) {
        setIsSimulationRunning(true);
      }
    }}
    className="flex-1 h-2 rounded-lg bg-slate-800 appearance-none cursor-pointer
accent-indigo-500"
  />
  <span className="text-xs font-bold text-red-500">M 10.0</span>
</div>

<p className="text-[11px] text-slate-500 mt-1 font-medium">
  {intensity === 0 && "Ground is completely still. Safe to build."}
  {intensity > 0 && intensity < 4.0 && "Minor Tremor: Felt only by birds and sensitive
instruments."}
  {intensity >= 4.0 && intensity < 6.5 && "Moderate Shaking: Plaster cracks, windows
rattle!"}
  {intensity >= 6.5 && intensity < 8.5 && "Major Hazard: High threat of collapse for
normal rigid structures!"}
  {intensity >= 8.5 && "Extreme Tectonic Rupture: Devastating horizontal forces!"}
</p>
</div>

```

```

    {/* Bottom Control Buttons */}
    <div className="flex flex-wrap justify-between items-center gap-4 pt-2">
      <div className="flex items-center gap-3">
        <button
          onClick={handleToggleSimulation}
          className={`px-5 py-2.5 rounded-xl font-extrabold text-xs tracking-wider
uppercase transition-all duration-300 flex items-center gap-2 ${
            isSimulationRunning
              ? 'bg-red-600 hover:bg-red-700 text-white shadow-lg shadow-red-900/30'
              : 'bg-indigo-600 hover:bg-indigo-700 text-white shadow-lg
shadow-indigo-900/30'
          }}
        >
          <span>{isSimulationRunning ? '🛑 STOP SHAKING' : '🔧 START
SHAKING'}</span>
        </button>

        <button
          onClick={resetSimulator}
          className="px-4 py-2.5 bg-slate-900 hover:bg-slate-800 border border-slate-700
rounded-xl font-bold text-xs text-slate-300 transition-all duration-300"
        >
          🔄 Reset Lab
        </button>
      </div>

      <button
        onClick={() => setActivePrediction(null)}
        className="text-xs text-indigo-400 hover:text-indigo-300 font-bold underline"
      >
        Change Hypothesis
      </button>
    </div>

  </div>

  {/* Right side panel: Upgrades & Educational HUD */}
  <div className="lg:col-span-4 space-y-6">

    {/* Upgrade Building B Card */}
    <div className="bg-slate-900 border border-slate-800 rounded-3xl p-6 shadow-xl
relative overflow-hidden">

```

```

<div className="absolute top-0 right-0 p-3 opacity-20 pointer-events-none">
  <span className="text-6xl text-purple-500 font-black">⚙️</span>
</div>

```

```

<h3 className="text-lg font-black text-slate-100 flex items-center gap-2">
  <span>🔧</span>
  <span>Mitigation Upgrades (Building B)</span>
</h3>

```

```

<p className="text-xs text-slate-400 mt-1 leading-relaxed">
  Toggle different architectural solutions to see how they modify Building B's resilience
  parameters.
</p>

```

```

<div className="mt-5 space-y-3">

  { /* Cross-Bracing Upgrade */ }
  <button
    onClick={() => toggleModifier('crossBracing')}
    className={`w-full p-3.5 rounded-2xl border text-left flex items-start gap-3
  transition-all duration-300 ${
    modifiers.crossBracing
      ? 'bg-emerald-950/60 border-emerald-500/50 shadow-inner'
      : 'bg-slate-950/80 border-slate-800/80 hover:bg-slate-900 hover:border-slate-700'
    }`}
  >
    <div className={`p-2 rounded-xl text-sm ${modifiers.crossBracing ?
  'bg-emerald-500 text-slate-950' : 'bg-slate-800 text-slate-400'}`}>
      
    </div>
    <div>
      <div className="flex items-center gap-2">
        <span className="text-xs font-bold text-slate-100">Cross-Bracing
        (Triangles)</span>
        {modifiers.crossBracing && <span className="bg-emerald-500/10
  text-emerald-400 px-1.5 py-0.5 rounded text-[9px] font-black uppercase">Active</span>}
      </div>
      <p className="text-[11px] text-slate-400 mt-0.5">Adds rigid diagonal beams to
  form stable triangles. Reduces horizontal flexing.</p>
    </div>
  </button>

  { /* Base Isolation Upgrade */ }
  <button
    onClick={() => toggleModifier('baseIsolators')}

```

```

        className={`w-full p-3.5 rounded-2xl border text-left flex items-start gap-3
transition-all duration-300 ${
    modifiers.baseIsolators
      ? 'bg-purple-950/60 border-purple-500/50 shadow-inner'
      : 'bg-slate-950/80 border-slate-800/80 hover:bg-slate-900 hover:border-slate-700'
}}
    >
    <div className={`p-2 rounded-xl text-sm ${modifiers.baseIsolators ?
'bg-purple-500 text-slate-950' : 'bg-slate-800 text-slate-400'}`>

    </div>
    <div>
      <div className="flex items-center gap-2">
        <span className="text-xs font-bold text-slate-100">Base Isolators</span>
        {modifiers.baseIsolators && <span className="bg-purple-500/10
text-purple-400 px-1.5 py-0.5 rounded text-[9px] font-black uppercase">Active</span>}
      </div>
      <p className="text-[11px] text-slate-400 mt-0.5">Puts rubber roll-pads below.
Decouples the tower structure from fast ground wave forces.</p>
    </div>
  </button>

  { /* Tuned Mass Damper Upgrade */ }
  <button
    onClick={() => toggleModifier('massDamper')}
    className={`w-full p-3.5 rounded-2xl border text-left flex items-start gap-3
transition-all duration-300 ${
    modifiers.massDamper
      ? 'bg-amber-950/60 border-amber-500/50 shadow-inner'
      : 'bg-slate-950/80 border-slate-800/80 hover:bg-slate-900 hover:border-slate-700'
}}
  >
    <div className={`p-2 rounded-xl text-sm ${modifiers.massDamper ?
'bg-amber-500 text-slate-950' : 'bg-slate-800 text-slate-400'}`>

    </div>
    <div>
      <div className="flex items-center gap-2">
        <span className="text-xs font-bold text-slate-100">Tuned Mass
Damper</span>
        {modifiers.massDamper && <span className="bg-amber-500/10
text-amber-400 px-1.5 py-0.5 rounded text-[9px] font-black uppercase">Active</span>}
      </div>

```

<p className="text-[11px] text-slate-400 mt-0.5">Hangs a massive inertia pendulum weight on the roof. Swings backwards to cancel swaying.</p>

</div>

</button>

</div>

</div>

{/* Dynamic Hints & Real-time Info */}

<div className="bg-slate-900 border border-slate-800 rounded-3xl p-6 shadow-xl flex flex-col justify-between">

<div>

<h4 className="text-xs font-extrabold text-indigo-400 uppercase tracking-widest flex items-center gap-1.5 mb-2">

<IconSparkles />

Seismic Engineer Guidance

</h4>

<div className="space-y-3 text-xs leading-relaxed text-slate-300">

<p>

 **The Challenge:** Set your Ground Shake Slider to **7.0 or higher**. Watch what happens to the standard concrete frame of Building A.

</p>

<p>

 **Hint:** Notice the blinking joint nodes! They transition from **green** (safe) to **orange** (stressed), and finally flash **red** just before catastrophic fracture occurs.

</p>

<p>

 **Your Goal:** Combine all 3 upgrades on **Building B** to achieve complete seismic immunity. See if you can protect the structure even at **Magnitude 10.0**!

</p>

</div>

</div>

<div className="mt-4 pt-4 border-t border-slate-800 flex items-center gap-2 text-xs text-slate-400">

<IconInfo />

NCERT science concept: Structural stability & resonant shear stress forces.

</div>

</div>

</div>

```

    </div>
  )}

  {/* Modal overlays for simulator */}
  {showCollapseModal && (
    <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-sm flex items-center
justify-center p-4">
      <div className="bg-slate-900 border-2 border-red-500/30 rounded-3xl p-6 max-w-md
text-center shadow-2xl relative">
        <span className="text-5xl">💥</span>
        <h3 className="text-xl font-extrabold text-red-400 mt-3">Building A Collapsed!</h3>
        <p className="text-xs text-slate-400 mt-2 leading-relaxed">
          As predicted, unreinforced masonry & stiff framing frames have low lateral strength.
          Sideways ground acceleration shears joints apart, causing a complete floor collapse.
        </p>
        <div className="mt-5 flex justify-center gap-3">
          <button
            onClick={() => setShowCollapseModal(false)}
            className="px-5 py-2 bg-slate-800 hover:bg-slate-700 text-slate-200 text-xs
font-bold rounded-xl transition"
          >
            Close & Continue
          </button>
          <button
            onClick={() => {
              setActiveTab('explanation');
              setShowCollapseModal(false);
            }}
            className="px-5 py-2 bg-gradient-to-r from-indigo-600 to-purple-600 text-white
text-xs font-bold rounded-xl shadow-lg transition"
          >
            Read Science Behind It
          </button>
        </div>
      </div>
    </div>
  )}

  {showSuccessModal && (
    <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-sm flex items-center
justify-center p-4">
      <div className="bg-slate-900 border-2 border-green-500/30 rounded-3xl p-6
max-w-md text-center shadow-2xl relative">
        <span className="text-5xl">🎉</span>

```



```

<h3 className="text-xl font-extrabold text-emerald-400 mt-3">Survival
Achieved!</h3>
  <p className="text-xs text-slate-400 mt-2 leading-relaxed">
    Excellent engineering! Thanks to your active mitigations, Building B withstood severe
ground vibrations that flattened standard buildings.
  </p>
  <div className="mt-5 flex justify-center gap-3">
    <button
      onClick={() => setShowSuccessModal(false)}
      className="px-5 py-2 bg-slate-800 hover:bg-slate-700 text-slate-200 text-xs
font-bold rounded-xl transition"
    >
      Keep Testing
    </button>
    <button
      onClick={() => {
        setActiveTab('quiz');
        setShowSuccessModal(false);
      }}
      className="px-5 py-2 bg-gradient-to-r from-emerald-600 to-green-600 text-white
text-xs font-bold rounded-xl shadow-lg transition"
    >
      Take Quiz Challenge
    </button>
  </div>
</div>
</div>
))

{/* Tab 2: Interactive Explanatory Guide */}
{activeTab === 'explanation' && (
  <div className="bg-slate-900 rounded-3xl p-6 md:p-8 border border-slate-800
shadow-xl max-w-4xl mx-auto space-y-8">

    <div className="text-center">
      <span className="text-xs font-extrabold text-indigo-400 uppercase
tracking-widest">Core Science Concept</span>
      <h2 className="text-2xl md:text-3xl font-black text-slate-100 mt-1">Understanding
Structural Stability</h2>
      <p className="text-slate-400 mt-2 text-sm max-w-2xl mx-auto leading-relaxed">
        Why do buildings break when the ground shakes? Let's unpack the structural forces
and mathematical principles that engineers master.
      </p>
    </div>

```

```

<div className="grid grid-cols-1 md:grid-cols-3 gap-6">

  <div className="bg-slate-950 p-5 rounded-2xl border border-slate-800/80
text-center">
    <span className="text-3xl">🔺 </span>
    <h4 className="font-bold text-slate-200 mt-2 text-sm">Rigidity via Triangles</h4>
    <p className="text-slate-400 text-xs mt-1 leading-relaxed">
      Square frames bend easily under lateral forces into slanted parallelograms.
      However, a triangle cannot deform without snapping its members, distributing forces evenly.
    </p>
  </div>

  <div className="bg-slate-950 p-5 rounded-2xl border border-slate-800/80
text-center">
    <span className="text-3xl">🔧 </span>
    <h4 className="font-bold text-slate-200 mt-2 text-sm">Decoupled Base
Bearings</h4>
    <p className="text-slate-400 text-xs mt-1 leading-relaxed">
      Instead of screwing bases straight into bedrock, floating foundations act like massive
      rubber and spring systems to isolate structural acceleration.
    </p>
  </div>

  <div className="bg-slate-950 p-5 rounded-2xl border border-slate-800/80
text-center">
    <span className="text-3xl">⚖️ </span>
    <h4 className="font-bold text-slate-200 mt-2 text-sm">Tuned Mass
Pendulums</h4>
    <p className="text-slate-400 text-xs mt-1 leading-relaxed">
      Skyscrapers place heavy pendulums at upper levels. Inertia delays their movement,
      pulling structural tops in opposing directions to counter violent sways.
    </p>
  </div>

  </div>

  {/* Scientific Formula block */}
  <div className="bg-slate-950 p-6 rounded-2xl border border-slate-800 flex flex-col
md:flex-row gap-6 items-center">
    <div className="flex-1">
      <h4 className="text-xs font-extrabold text-rose-400 uppercase tracking-widest
mb-1">The Physics Formula</h4>

```

```
<h3 className="text-xl font-bold text-slate-100">Newton's Second Law of Motion</h3>
```

```
<p className="text-xs text-slate-400 mt-2 leading-relaxed">
```

During an earthquake, the destructive force (\$F\$) experienced by a building depends directly on its total structural mass (\$m\$) and ground acceleration (\$a\$):

```
</p>
```

```
<div className="bg-slate-900 border border-slate-800 px-4 py-2.5 rounded-xl mt-3 inline-block font-mono text-indigo-400 font-extrabold text-sm md:text-base">
```

$F = m \times a$

```
</div>
```

```
<p className="text-[11px] text-slate-500 mt-2">
```

Reducing mass with lightweight composite frames and cutting acceleration using isolators drops the total force!

```
</p>
```

```
</div>
```

```
<div className="bg-slate-900 p-4 rounded-xl border border-slate-800 text-xs space-y-1 text-slate-300 w-full md:w-64 font-mono">
```

```
<div className="font-bold text-slate-400 mb-1 border-b border-slate-800 pb-1">
```

Memory Helper Rule:

```
<div>★ <span className="text-indigo-400 font-bold">B</span>race the frame (Triangles)</div>
```

```
<div>★ <span className="text-purple-400 font-bold">B</span>uffer the base (Rollers)</div>
```

```
<div>★ <span className="text-amber-400 font-bold">B</span>alance the top (Dampers)</div>
```

```
</div>
```

```
</div>
```

```
{/* Real World Applications */}
```

```
<div className="space-y-3">
```

```
<h4 className="font-bold text-slate-200 text-sm">☀ Famous Real-World Examples:</h4>
```

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-3 text-xs leading-relaxed">
```

```
<div className="bg-slate-950 p-3 rounded-xl border border-slate-800">
```

 **Taipei 101 (Taiwan):** Features a massive 660-metric-ton golden spherical tuned mass damper hanging inside its top floor levels.

```
</div>
```

```
<div className="bg-slate-950 p-3 rounded-xl border border-slate-800">
```

 **Tokyo Skytree (Japan):** Uses a heavy decoupled center pillar core system to safely absorb wave shifts.

```
</div>
```

```
<div className="bg-slate-950 p-3 rounded-xl border border-slate-800">
```

 **Apple Park (USA):** Sit on top of 692 huge steel saucer base-isolation plates to keep systems running after massive faults.

```
</div>
<div className="bg-slate-950 p-3 rounded-xl border border-slate-800">
   **Machu Picchu (Peru):** Ancient dry-stone walls danced in place during earthquakes and resettled without mortar cracking!
</div>
```

```
</div>
</div>

{/* Common Mistakes */}
<div className="bg-red-950/20 border border-red-900/30 p-5 rounded-2xl">
  <h4 className="text-xs font-extrabold text-red-400 uppercase tracking-widest flex items-center gap-1.5 mb-2">
```

```
    <IconAlert />
    <span>Common Misunderstandings</span>
  </h4>
```

```
  <div className="space-y-2 text-xs leading-relaxed text-slate-300">
    <p>
       **Mistake:** Believing that making a building completely rigid, heavy, and concrete-thick is the best way to guard against earthquakes.
    </p>
```

```
    <p>
       **Correct Physics:** Stiffness increases brittle fractures under seismic shear loads. Controlled flexibility and dampeners absorb forces, helping structures bend safely without breaking.
    </p>
  </div>
```

```
</div>
  )}
```

```
{/* Tab 3: Interactive DIY guide */}
{activeTab === 'diy' && (
  <div className="bg-slate-900 rounded-3xl p-6 md:p-8 border border-slate-800 shadow-xl max-w-4xl mx-auto space-y-6">

    <div className="text-center">
      <span className="text-xs font-extrabold text-emerald-400 uppercase tracking-widest">At-Home Science Activity</span>
      <h2 className="text-2xl md:text-3xl font-black text-slate-100 mt-1">Jelly-Marshmallow Earthquake Test</h2>
      <p className="text-slate-400 mt-2 text-sm max-w-2xl mx-auto leading-relaxed">
        Build your own miniature shaking test lab at home using common household kitchen materials!
      </p>
    </div>
  </div>
)
```

</p>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-6">

<div className="bg-slate-950 p-5 rounded-2xl border border-slate-800">
<h4 className="font-bold text-indigo-400 mb-3 text-sm">🛒 Materials

Checklist:</h4>

<ul className="space-y-2 text-xs text-slate-300">
<li className="flex items-center gap-2">
✓
1 baking dish or tray filled with gelatin jelly (soft ground soil)

<li className="flex items-center gap-2">
✓
20-30 toothpicks or straw sticks (beams & pillars)

<li className="flex items-center gap-2">
✓
20 mini-marshmallows or clay playdough balls (joints)

<li className="flex items-center gap-2">
✓
A steady plastic ruler or clipboard tray (the shake plate)

</div>

<div className="bg-slate-950 p-5 rounded-2xl border border-slate-800">
<h4 className="font-bold text-indigo-400 mb-3 text-sm">💡 Quick Instructions:</h4>
<ol className="space-y-2 text-xs text-slate-300 list-decimal pl-4">

Build a simple square cube frame using toothpicks and marshmallows at corners.

Build a second identical frame, but add diagonal cross bracing toothpicks to form
triangles.

Place both structures on the smooth jelly surface.

Gently shake the tray horizontally back and forth for 10 seconds. Watch which
structure falls!

```

        </li>
      </ol>
    </div>

  </div>

  <div className="bg-indigo-950/20 border border-indigo-900/40 p-5 rounded-2xl flex
flex-col md:flex-row items-center gap-4">
    <span className="text-4xl md:text-5xl">🤖</span>
    <div className="text-xs leading-relaxed text-slate-300">
      <h4 className="font-bold text-slate-200 text-sm mb-1">What to Observe during
Shaking:</h4>
      <p>
        The unreinforced square structures skew rapidly, collapsing because squares have
flexible joints.
        The triangulated tower distributes energy throughout the rigid diagonal segments,
maintaining geometric shape under lateral shear stresses.
      </p>
    </div>
  </div>

</div>
)}

{/* Tab 4: Interactive Quiz Assessment */}
{activeTab === 'quiz' && (
  <div className="bg-slate-900 rounded-3xl p-6 md:p-8 border border-slate-800
shadow-xl max-w-2xl mx-auto">

    {/* Header progress bar */}
    {!quizFinished && (
      <div className="space-y-2 mb-6">
        <div className="flex justify-between text-xs font-bold text-slate-400">
          <span>QUIZ PROGRESS</span>
          <span>Question {currentQuizIndex + 1} of {quizQuestions.length}</span>
        </div>
        <div className="h-2 w-full bg-slate-950 rounded-full overflow-hidden">
          <div
            className="h-full bg-gradient-to-r from-indigo-500 to-violet-500 transition-all
duration-300"
            style={{ width: `${((currentQuizIndex + 1) / quizQuestions.length) * 100}%` }}
          />
        </div>
      </div>
    )}

    </div>
  )}

```

```

    })

    {!quizFinished ? (
      <div className="space-y-6">

        {/* Question */}
        <h3 className="text-lg font-black text-slate-100">
          {quizQuestions[currentQuizIndex].q}
        </h3>

        {/* Options List */}
        <div className="space-y-3">
          {quizQuestions[currentQuizIndex].options.map((option, idx) => {
            const hasBeenAnswered = selectedAnswer !== null;
            const isCorrectOption = idx === quizQuestions[currentQuizIndex].correct;
            const isSelectedOption = idx === selectedAnswer;

            let buttonStyles = "bg-slate-950/80 border-slate-800/80 hover:bg-slate-900
            hover:border-slate-700 text-slate-300";
            if (hasBeenAnswered) {
              if (isCorrectOption) {
                buttonStyles = "bg-green-950/60 border-green-500/80 text-green-200 font-bold";
              } else if (isSelectedOption) {
                buttonStyles = "bg-red-950/60 border-red-500/80 text-red-200 font-bold";
              } else {
                buttonStyles = "bg-slate-950/30 border-slate-800/20 text-slate-500";
              }
            }

            return (
              <button
                key={idx}
                disabled={hasBeenAnswered}
                onClick={() => handleQuizAnswer(idx)}
                className={`w-full p-4 rounded-2xl border text-left flex items-center
                justify-between text-xs md:text-sm transition-all duration-300 ${buttonStyles}`}
              >
                <span>{option}</span>
                {hasBeenAnswered && isCorrectOption && <IconCheck />}
                {hasBeenAnswered && isSelectedOption && !isCorrectOption && <IconCross />}
              </button>
            );
          })}
        </div>
      )
    )}
  </div>

```

```

    { /* Explanation Block */
    {selectedAnswer !== null && (
      <div className="bg-slate-950 p-4 rounded-2xl border border-slate-800
animate-fadeIn">
        <span className="text-[10px] font-extrabold text-indigo-400 uppercase
tracking-widest block mb-1">Concept Insight</span>
        <p className="text-xs text-slate-300 leading-relaxed">
          {quizQuestions[currentQuizIndex].explanation}
        </p>
        <button
          onClick={handleNextQuiz}
          className="mt-4 w-full md:w-auto px-5 py-2.5 bg-gradient-to-r from-indigo-600
to-purple-600 text-white rounded-xl text-xs font-bold shadow-md hover:shadow-lg transition"
        >
          {currentQuizIndex < quizQuestions.length - 1 ? 'Next Question ➡' : 'Submit Score
🎉'}
        </button>
      </div>
    )}

  </div>
) : (
  // Quiz Finished Screen
  <div className="text-center py-6 space-y-6">
    <span className="text-6xl">🏆</span>
    <h3 className="text-2xl font-black text-slate-100">Assessment Complete!</h3>
    <p className="text-slate-400 text-xs">
      Let's see how your structural engineering theory knowledge held up under our quiz
stress test.
    </p>

    <div className="max-w-xs mx-auto bg-slate-950 border border-slate-800
rounded-3xl p-5 space-y-3">
      <div className="flex justify-between text-xs font-bold text-slate-400 border-b
border-slate-900 pb-2.5">
        <span>Final Score:</span>
        <span className="text-indigo-400 text-sm">{quizScore} /
{quizQuestions.length}</span>
      </div>
      <div className="flex justify-between text-xs font-bold text-slate-400">
        <span>Performance Rating:</span>
        <span className="text-emerald-400">

```



```

        {quizScore === quizQuestions.length ? 'Perfect Score 🌟' : quizScore >= 7 ?
'Excellent Analyst 📊' : 'Assistant Builder 🏗️'}
      </span>
    </div>
  </div>

  <button
    onClick={restartQuiz}
    className="px-6 py-3 bg-gradient-to-r from-indigo-600 to-violet-600 text-white
font-bold text-xs rounded-xl shadow-lg transition"
  >
    🔄 Retake Assessment
  </button>
</div>
)}

</div>
)}

</main>

{/* Modern responsive Footer credits */}
<footer className="mt-auto py-6 border-t border-slate-800 bg-slate-950/80 text-center
text-[11px] text-slate-500 font-medium">
  <p>© 2026 The Shake Lab • Designed according to NCERT Class 8 natural disaster
curriculum specifications.</p>
  <p className="mt-1">Developed for immersive real-time HTML5 physics exploration.</p>
</footer>

</div>
);
}

```

floating and sinking

```

import React, { useState, useEffect, useRef } from 'react';

// --- Vector Graphics for Enhanced Assets ---
const SparklesIcon = () => (
  <svg className="w-5 h-5 text-yellow-400 animate-pulse" fill="currentColor" viewBox="0 0 20 20">
    <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07 3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />
  </svg>
);

const TrophyIcon = () => (
  <svg className="w-6 h-6 text-yellow-500" fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
    <path strokeLinecap="round" strokeLinejoin="round" d="M12 8v13m0-13V6a2 2 0 12 2h-2zm0 0V5.5A2.5 2.5 0 109.5 8H12zm-7 4h14M5 12a2 2 0 110-4h14a2 2 0 110 4M5 12v7a2 2 0 02 2h10a2 2 0 02-2v-7" />
  </svg>
);

const InfoIcon = () => (
  <svg className="w-4 h-4 text-blue-500" fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
    <path strokeLinecap="round" strokeLinejoin="round" d="M13 16h-1v-4h-1m1-4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />
  </svg>
);

// --- Object Specifications with Physics Profiles ---
const INITIAL_OBJECTS = [
  {
    id: 'nail',
    name: 'Iron Nail',
    weightLabel: 'Dense & Tiny',
    sizeLabel: 'Small',
    density: 7.8, // g/cm³
    volume: 18, // cm³
    color: '#94a3b8',
    borderColor: '#475569',
    iconText: '📌',
    description: 'An iron nail with metal atoms tightly packed together.',
  },

```

fact: 'Because metal atoms are so crowded, an iron nail weighs almost 8 times more than water of the exact same size!'

},

{

id: 'block',

name: 'Wooden Block',

weightLabel: 'Hollow & Light',

sizeLabel: 'Medium',

density: 0.6, // g/cm³

volume: 75, // cm³

color: '#d97706',

borderColor: '#78350f',

iconText: '🪵',

description: 'A solid block of dry oak with tiny built-in air pathways.',

fact: 'Wood fibers trap small air pockets inside, lowering its average density so it floats comfortably.'

},

{

id: 'stone',

name: 'River Pebble',

weightLabel: 'Packed & Heavy',

sizeLabel: 'Medium',

density: 2.7, // g/cm³

volume: 55, // cm³

color: '#64748b',

borderColor: '#334155',

iconText: '🪨',

description: 'A river rock formed of compact crystalline minerals.',

fact: 'Rock is extremely solid and has no hidden air chambers, meaning gravity easily pulls it down.'

},

{

id: 'boat',

name: 'Toy Sailboat',

weightLabel: 'Very Hollow',

sizeLabel: 'Large',

density: 0.3, // g/cm³

volume: 110, // cm³

color: '#ef4444',

borderColor: '#991b1b',

iconText: '🚤',

description: 'A hollow plastic boat structure enclosing a pocket of air.',

fact: 'Even though it is the largest object, its wide hollow shape captures so much light air that it floats easily!'

```

},
{
  id: 'foil',
  name: 'Aluminium Ball',
  weightLabel: 'Borderline Density',
  sizeLabel: 'Medium',
  density: 1.1, // g/cm³ (Floats in saltwater, sinks in fresh!)
  volume: 45, // cm³
  color: '#cbd5e1',
  borderColor: '#64748b',
  iconText: '🏠',
  description: 'Crumpled kitchen foil enclosing tiny residual air chambers.',
  fact: 'This foil is on the borderline! In fresh water it sinks, but salt water becomes dense enough to push it back up!'
}
];

```

```

export default function App() {
  const [activeTab, setActiveTab] = useState('simulator');

  // --- Simulator Physical & Gamified States ---
  const [waterDensity, setWaterDensity] = useState(1.0); // g/cm³
  const [objects, setObjects] = useState(
    INITIAL_OBJECTS.map((obj, idx) => ({
      ...obj,
      status: 'shelf', // 'shelf' | 'tank'
      x: 70 + idx * 115,
      y: 75,
      isDragging: false,
      prediction: null, // 'float' | 'sink' | null
      predictionCorrect: null, // boolean | null
      submergedRatio: 0,
    })))
  );

  const [activeObject, setActiveObject] = useState(null);
  const [dragOffset, setDragOffset] = useState({ x: 0, y: 0 });
  const [splashParticles, setSplashParticles] = useState([]);
  const [waveOffset, setWaveOffset] = useState(0);
  const [xp, setXp] = useState(0);
  const [unlockedMedals, setUnlockedMedals] = useState([]);
  const [activeFeedback, setActiveFeedback] = useState(null); // feedback popup messages

  // --- Archimedes Water Level Calculation ---

```

```

// Constant base water parameters within SVG viewBox (800 x 550)
const TANK_X_MIN = 180;
const TANK_X_MAX = 620;
const BASE_WATER_Y_TOP = 250; // default water level
const TANK_Y_BOTTOM = 460; // tank bottom

// Compute how much water level rises due to displacement of dropped objects
const getTotalDisplacedVolume = () => {
  return objects
    .filter((obj) => obj.status === 'tank' && !obj.isDragging)
    .reduce((acc, obj) => {
      // If it floats, it only displaces water equal to its own weight (submerged fraction)
      // If it sinks, it displaces its entire volume
      if (obj.density < waterDensity) {
        const submergedVolume = obj.volume * (obj.density / waterDensity);
        return acc + submergedVolume;
      }
      return acc + obj.volume;
    }, 0);
};

// 1 cm³ displaced volume raises the water surface by 0.3 pixels on screen
const displacedRisePixels = getTotalDisplacedVolume() * 0.22;
const DYNAMIC_WATER_Y_TOP = BASE_WATER_Y_TOP - displacedRisePixels;

// --- Dynamic Particle & Fluid Wave Engines ---
useEffect(() => {
  let animationFrameId;
  const animateWaves = () => {
    setWaveOffset((prev) => (prev + 0.04) % (Math.PI * 2));
    animationFrameId = requestAnimationFrame(animateWaves);
  };
  animateWaves();
  return () => cancelAnimationFrame(animationFrameId);
}, []);

// Frame simulation tick: smoothly animate objects towards their buoyant resting coordinates
useEffect(() => {
  const updated = objects.map((obj) => {
    if (obj.status !== 'tank' || obj.isDragging) return obj;

    let targetY;
    let ratio = 1.0; // 1.0 means fully submerged
  });

```

```

if (obj.density < waterDensity) {
  // FLOATS
  ratio = obj.density / waterDensity;
  const objectHeight = obj.volume * 0.4;
  // Float line is at the dynamic water surface, submerged proportional to density
  targetY = DYNAMIC_WATER_Y_TOP + (ratio * objectHeight) - (objectHeight * 0.85);
} else {
  // SINKS
  const objectHeight = obj.volume * 0.4;
  targetY = TANK_Y_BOTTOM - (objectHeight / 1.8);
}

// Smooth easing step
const currentY = obj.y;
const diff = targetY - currentY;
let nextY = currentY;

if (Math.abs(diff) > 0.4) {
  nextY += diff * 0.12;
} else {
  nextY = targetY;
}

// Check and update prediction achievements
let predCorrect = obj.predictionCorrect;
if (obj.prediction && predCorrect === null && Math.abs(diff) < 2) {
  const actuallyFloats = obj.density < waterDensity;
  const predictedFloats = obj.prediction === 'float';
  predCorrect = actuallyFloats === predictedFloats;

  if (predCorrect) {
    setXp(prev => prev + 25);
    triggerToast(`💡 Brilliant! Your prediction was correct: The ${obj.name} indeed
    ${actuallyFloats ? 'floats' : 'sinks'}! (+25 XP)`);
  } else {
    triggerToast(`🔍 Ah! The ${obj.name} behaved differently. Let's see why in the
    Telemetry!`);
  }
}

return {
  ...obj,
  y: nextY,
  submergedRatio: ratio,

```

```

    predictionCorrect: predCorrect
  };
});

const hasChanged = updated.some((obj, i) => Math.abs(obj.y - objects[i].y) > 0.1 ||
obj.predictionCorrect !== objects[i].predictionCorrect);
if (hasChanged) {
  setObjects(updated);
}
}, [objects, waterDensity, DYNAMIC_WATER_Y_TOP]);

// Handle game achievement criteria
useEffect(() => {
  const testedCount = objects.filter(o => o.status === 'tank').length;
  if (testedCount === 5 && !unlockedMedals.includes('grandmaster')) {
    setUnlockedMedals([...unlockedMedals, 'grandmaster']);
    setXp(prev => prev + 100);
    triggerToast("🏆 Grandmaster Medal Unlocked! You successfully experimented with all
materials!");
  } else if (testedCount >= 1 && !unlockedMedals.includes('first_step')) {
    setUnlockedMedals([...unlockedMedals, 'first_step']);
    setXp(prev => prev + 20);
    triggerToast("🔍 First Step! You launched your first fluid experiment. (+20 XP)");
  }

  if (waterDensity > 1.18 && !unlockedMedals.includes('salinity_max')) {
    setUnlockedMedals([...unlockedMedals, 'salinity_max']);
    setXp(prev => prev + 50);
    triggerToast("🧂 Salinity Max! You super-salted the water. Notice the boost in Buoyancy!
(+50 XP)");
  }
}, [objects, waterDensity]);

const triggerToast = (msg) => {
  setActiveFeedback(msg);
  setTimeout(() => {
    setActiveFeedback(null);
  }, 4500);
};

// Particle splash logic
const createSplashParticles = (x, y, count = 14) => {
  const newParticles = [];
  for (let i = 0; i < count; i++) {

```



```

newParticles.push({
  id: Math.random(),
  x,
  y,
  vx: (Math.random() - 0.5) * 7,
  vy: -Math.random() * 6 - 3,
  size: Math.random() * 5 + 3,
  opacity: 1
});
}
setSplashParticles((prev) => [...prev, ...newParticles]);

const interval = setInterval(() => {
  setSplashParticles((prev) => {
    const active = prev
    .map((p) => ({
      ...p,
      x: p.x + p.vx,
      y: p.y + p.vy,
      vy: p.vy + 0.28, // mock gravity pull
      opacity: p.opacity - 0.06
    }))
    .filter((p) => p.opacity > 0);

    if (active.length === 0) {
      clearInterval(interval);
    }
    return active;
  });
}, 30);
};

// --- Interaction & Drag and Drop Handlers ---
const handleStartDrag = (e, obj, clientX, clientY) => {
  e.preventDefault();
  const svgElement = e.currentTarget.ownerDocument.getElementById('sim-svg');
  if (!svgElement) return;
  const svgRect = svgElement.getBoundingClientRect();

  // Scale factor to translate screen pixels to internal 800x550 coordinate space
  const scaleX = 800 / svgRect.width;
  const scaleY = 550 / svgRect.height;

  const cursorX = (clientX - svgRect.left) * scaleX;

```

```

const cursorY = (clientY - svgRect.top) * scaleY;

setActiveObject(obj.id);
setDragOffset({
  x: cursorX - obj.x,
  y: cursorY - obj.y
});

setObjects(prev => prev.map(o => o.id === obj.id ? { ...o, isDragging: true } : o));
};

const handleDrag = (e, clientX, clientY) => {
  if (!activeObject) return;
  const svgElement = e.currentTarget.ownerDocument.getElementById('sim-svg');
  if (!svgElement) return;
  const svgRect = svgElement.getBoundingClientRect();

  const scaleX = 800 / svgRect.width;
  const scaleY = 550 / svgRect.height;

  const cursorX = (clientX - svgRect.left) * scaleX;
  const cursorY = (clientY - svgRect.top) * scaleY;

  const newX = Math.max(30, Math.min(770, cursorX - dragOffset.x));
  const newY = Math.max(30, Math.min(520, cursorY - dragOffset.y));

  setObjects(prev => prev.map(o => o.id === activeObject ? { ...o, x: newX, y: newY } : o));
};

const handleEndDrag = () => {
  if (!activeObject) return;

  setObjects(prev => prev.map(o => {
    if (o.id !== activeObject) return o;

    const inTankX = o.x >= TANK_X_MIN && o.x <= TANK_X_MAX;
    const inTankY = o.y >= DYNAMIC_WATER_Y_TOP - 90 && o.y <= TANK_Y_BOTTOM +
50;

    if (inTankX && inTankY) {
      // Verify if a prediction exists. If not, default auto-set to "float" to avoid soft-locking
      const pred = o.prediction || 'float';
      createSplashParticles(o.x, DYNAMIC_WATER_Y_TOP, 16);
      return {

```

```

    ...o,
    status: 'tank',
    isDragging: false,
    prediction: pred,
    y: DYNAMIC_WATER_Y_TOP + 12
  };
} else {
  // Snap back home to shelf
  const idx = INITIAL_OBJECTS.findIndex(io => io.id === o.id);
  return {
    ...o,
    status: 'shelf',
    isDragging: false,
    prediction: null,
    predictionCorrect: null,
    x: 70 + idx * 115,
    y: 75
  };
}
}));

setActiveObject(null);
};

const handlePredict = (id, direction) => {
  setObjects(prev => prev.map(o => o.id === id ? { ...o, prediction: direction } : o));
  // Trigger splash drop automatically
  setTimeout(() => {
    instantDrop(id);
  }, 400);
};

const instantDrop = (id) => {
  setObjects(prev => prev.map(o => {
    if (o.id === id) {
      // Set fallback prediction if they didn't explicitly tap one
      const fallbackPred = o.prediction || 'float';
      const randomX = 250 + (Math.random() * 300);
      createSplashParticles(randomX, DYNAMIC_WATER_Y_TOP, 14);
      return {
        ...o,
        status: 'tank',
        prediction: fallbackPred,
        x: randomX,

```

```

        y: DYNAMIC_WATER_Y_TOP + 10
    };
}
return o;
}));
};

```

```

const recallObject = (id) => {
    const idx = INITIAL_OBJECTS.findIndex(io => io.id === id);
    setObjects(prev => prev.map(o => {
        if (o.id === id) {
            return {
                ...o,
                status: 'shelf',
                prediction: null,
                predictionCorrect: null,
                x: 70 + idx * 115,
                y: 75
            };
        }
        return o;
    }));
};

```

```

const resetAllObjects = () => {
    setObjects(
        INITIAL_OBJECTS.map((obj, idx) => ({
            ...obj,
            status: 'shelf',
            x: 70 + idx * 115,
            y: 75,
            isDragging: false,
            prediction: null,
            predictionCorrect: null,
            submergedRatio: 0
        })))
    );
    setWaterDensity(1.0);
    setSplashParticles([]);
};

```

```

// --- Quiz Database (Interactive 10 Questions) ---
const quizQuestions = [
    {

```

question: "A massive steel cruise ship floats perfectly, but a tiny iron nail sinks to the bottom. Why?",

options: [

{ label: "The ship has giant bubble machines beneath it.", isCorrect: false },

{ label: "The ship is shaped hollow to trap air inside, making its average density lighter than water.", isCorrect: true },

{ label: "Large oceans are naturally more buoyant than small buckets of water.", isCorrect: false },

{ label: "The metal on the cruise ship is painted with a hydrophobic wax.", isCorrect: false }

],

explanation: "Amazing! The ship contains giant rooms of trapped hollow air. This averages out the total mass over a huge volume, making the ship's density lower than water!"

},

{

question: "What decides whether an object sinks or floats when dropped into a liquid?",

options: [

{ label: "The color of the object.", isCorrect: false },

{ label: "Its total weight only.", isCorrect: false },

{ label: "Its density (how heavy it is for its size) compared to the liquid's density.", isCorrect: true },

{ label: "The depth of the water container.", isCorrect: false }

],

explanation: "Spot on! An item will sink if its density is higher than the liquid's density, regardless of its size!"

},

{

question: "If you add loads of salt to fresh water, what happens to the water's buoyancy?",

options: [

{ label: "It decreases because salt weighs down the water.", isCorrect: false },

{ label: "It increases because dissolving salt packs more matter into the water, making it denser.", isCorrect: true },

{ label: "It remains identical since salt doesn't float.", isCorrect: false }

],

explanation: "Precisely! Salty water is packed tighter with dissolved mineral particles, which exerts a stronger upward buoyant force."

},

{

question: "What is the direction of the Buoyant Force?",

options: [

{ label: "Always downwards, pulling items to Earth.", isCorrect: false },

{ label: "Sideways, pushing items to the edge.", isCorrect: false },

{ label: "Always upwards, opposing gravity.", isCorrect: true }

],

explanation: "Correct! Gravity pulls objects downwards, while fluid buoyancy works to push objects upwards."

},

{

question: "Why does solid ice float on liquid water?",

options: [

{ label: "Solid ice expands when freezing, trapping microscopic air spaces and making it less dense.", isCorrect: true },

{ label: "Ice is naturally made of different molecules than water.", isCorrect: false },

{ label: "Ice is colder, which forces it to float.", isCorrect: false }

],

explanation: "Excellent! When water freezes into ice, its volume expands, making it lighter for its size (less dense) than liquid water."

},

{

question: "An object is floating on fresh water. What can you say about its density?",

options: [

{ label: "Its density is exactly equal to 0.", isCorrect: false },

{ label: "Its density is greater than 1.0 g/cm³.", isCorrect: false },

{ label: "Its density is less than 1.0 g/cm³.", isCorrect: true }

],

explanation: "Spot on! Fresh water has a density of 1.0 g/cm³. Any item floating on it must have a density less than 1.0 g/cm³."

},

{

question: "How does a submarine manage to sink deep down and then float back to the surface?",

options: [

{ label: "It uses giant propellers to drag itself down.", isCorrect: false },

{ label: "It changes its weight by pumping water inside its ballast tanks to sink, or pumping air to float.", isCorrect: true },

{ label: "It shrinks in size to become denser.", isCorrect: false }

],

explanation: "Perfect! By changing the amount of water vs. air inside its tanks, the submarine adjusts its average density at will!"

},

{

question: "What will happen to the water level when you drop a heavy stone inside a water tank?",

options: [

{ label: "The water level drops because the stone absorbs fluid.", isCorrect: false },

{ label: "The water level rises because the stone displaces (pushes aside) water equal to its volume.", isCorrect: true },

{ label: "The water level remains perfectly unchanged.", isCorrect: false }

```

    ],
    explanation: "Excellent! Dropping any object pushes aside water, forcing the water level to
rise. This is called water displacement."
  },
  {
    question: "Which of these volcanic rocks is famous for floating on water?",
    options: [
      { label: "Obsidian", isCorrect: false },
      { label: "Basalt", isCorrect: false },
      { label: "Pumice Stone", isCorrect: true },
      { label: "Granite", isCorrect: false }
    ],
    explanation: "Correct! Pumice stones are cooled volcanic foam, packed with thousands of
tiny gas bubbles, allowing them to float!"
  },
  {
    question: "If you squeeze a flat piece of aluminum foil into a super-tight ball, it sinks. Why?",
    options: [
      { label: "You packed all the aluminum closer and squeezed out the trapped air, making it
denser.", isCorrect: true },
      { label: "Aluminum gets heavier when we squeeze it.", isCorrect: false },
      { label: "Squeezing changes the chemical structure of the foil.", isCorrect: false }
    ],
    explanation: "Incredible job! Squeezing the foil removes the empty air pockets, compressing
the material into a higher density state."
  }
];

```

```

const [quizScore, setQuizScore] = useState(0);
const [currentQuizIndex, setCurrentQuizIndex] = useState(0);
const [selectedAnswer, setSelectedAnswer] = useState(null);
const [quizSubmitted, setQuizSubmitted] = useState(false);
const [quizFinished, setQuizFinished] = useState(false);

const handleAnswerSelection = (idx) => {
  if (quizSubmitted) return;
  setSelectedAnswer(idx);
};

const handleQuizSubmit = () => {
  if (selectedAnswer === null) return;
  const isCorrect = quizQuestions[currentQuizIndex].options[selectedAnswer].isCorrect;
  if (isCorrect) {
    setQuizScore(prev => prev + 1);
  }
};

```

```

    setXp(prev => prev + 20);
  }
  setQuizSubmitted(true);
};

const handleNextQuiz = () => {
  setSelectedAnswer(null);
  setQuizSubmitted(false);
  if (currentQuizIndex + 1 < quizQuestions.length) {
    setCurrentQuizIndex(currentQuizIndex + 1);
  } else {
    setQuizFinished(true);
    if (quizScore >= 8 && !unlockedMedals.includes('quiz_wizard')) {
      setUnlockedMedals([...unlockedMedals, 'quiz_wizard']);
      setXp(prev => prev + 120);
    }
  }
};

// --- DIY Virtual Egg Experiment State ---
const [diySaltSpoons, setDiySaltSpoons] = useState(0);
const isEggFloating = diySaltSpoons >= 3;

return (
  <div className="min-h-screen bg-slate-100 text-slate-800 flex flex-col font-sans
antialiased">

    {/* Dynamic Toast Popup Notification */}
    {activeFeedback && (
      <div className="fixed top-20 right-4 z-50 bg-slate-900 text-white px-5 py-3 rounded-xl
shadow-2xl flex items-center gap-3 border border-slate-700 animate-bounce max-w-sm">
        <span className="text-xl"> ✨ </span>
        <p className="text-xs font-semibold"> {activeFeedback} </p>
      </div>
    )}

    {/* --- DUOLINGO / APPLE EDUCATION STYLE HEADER --- */}
    <header className="bg-white border-b border-slate-200 sticky top-0 z-40 shadow-sm px-4
py-3">
      <div className="max-w-7xl mx-auto flex flex-col md:flex-row justify-between items-center
gap-4">

        <div className="flex items-center gap-3.5">

```



```
<div className="w-12 h-12 rounded-2xl bg-blue-600 flex items-center justify-center text-2xl shadow-md shadow-blue-200">
```



```
</div>
```

```
<div>
```

```
<h1 className="text-xl font-bold text-slate-900 tracking-tight">The Splash Lab</h1>
```

```
<div className="flex items-center gap-2 mt-0.5">
```

```
<span className="bg-blue-100 text-blue-700 text-[10px] font-bold px-2 py-0.5 rounded-full uppercase">Class 5 Science</span>
```

```
<span className="text-slate-400 text-xs">• NCERT Materials Curriculum</span>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* XP & Level Progress */}
```

```
<div className="flex items-center gap-4 bg-slate-50 border border-slate-200 px-5 py-2.5 rounded-2xl shadow-inner">
```

```
<div className="flex items-center gap-2">
```

```
<span className="text-xl animate-pulse">⚡</span>
```

```
</div>
```

```
<div className="text-[10px] text-slate-400 uppercase font-extrabold tracking-wider">Total Score</div>
```

```
<div className="text-sm font-black text-slate-800">{xp} XP</div>
```

```
</div>
```

```
</div>
```

```
<div className="h-8 w-px bg-slate-200" />
```

```
<div className="flex items-center gap-2">
```

```
<span className="text-xl text-yellow-500">🏆</span>
```

```
</div>
```

```
<div className="text-[10px] text-slate-400 uppercase font-extrabold tracking-wider">Milestones</div>
```

```
<div className="flex gap-1 items-center mt-0.5">
```

```
{unlockedMedals.length === 0 ? (
```

```
<span className="text-xs text-slate-400 font-bold">0 unlocked</span>
```

```
) : (
```

```
unlockedMedals.map((m) => (
```

```
<span
```

```
key={m}
```

```
title={m === 'first_step' ? 'First Drop' : m === 'salinity_max' ? 'Salinity Master' :
```

```
'Fluid Grandmaster'}
```

```
className="text-lg cursor-help transition-transform hover:scale-125"
```

```
>
```

```

        {m === 'first_step' ? '🔍' : m === 'salinity_max' ? '🧪' : '🏆'}
      </span>
    ))
  })
</div>
</div>
</div>
</div>

</div>

{/* Immersive Tabs Bar */}
<div className="max-w-7xl mx-auto mt-4 flex border-t border-slate-100 pt-3 gap-2
overflow-x-auto scrollbar-none">
  <button
    onClick={() => setActiveTab('simulator')}
    className={`px-5 py-2 rounded-xl text-xs font-black tracking-wide transition-all
uppercase ${
      activeTab === 'simulator'
        ? 'bg-blue-600 text-white shadow-lg shadow-blue-100'
        : 'text-slate-500 hover:bg-slate-100'
    }}
  >
    🧪 Dynamic Simulator
  </button>
  <button
    onClick={() => setActiveTab('quiz')}
    className={`px-5 py-2 rounded-xl text-xs font-black tracking-wide transition-all
uppercase ${
      activeTab === 'quiz'
        ? 'bg-blue-600 text-white shadow-lg shadow-blue-100'
        : 'text-slate-500 hover:bg-slate-100'
    }}
  >
    🧠 Concept Quiz ({currentQuizIndex + 1}/{quizQuestions.length})
  </button>
  <button
    onClick={() => setActiveTab('learn')}
    className={`px-5 py-2 rounded-xl text-xs font-black tracking-wide transition-all
uppercase ${
      activeTab === 'learn'
        ? 'bg-blue-600 text-white shadow-lg shadow-blue-100'
        : 'text-slate-500 hover:bg-slate-100'
    }}
  >
    📖 Learn
  </button>
</div>

```

```

>
 Micro-Lessons
</button>
<button
  onClick={() => setActiveTab('diy')}
  className={`px-5 py-2 rounded-xl text-xs font-black tracking-wide transition-all
uppercase ${
  activeTab === 'diy'
    ? 'bg-blue-600 text-white shadow-lg shadow-blue-100'
    : 'text-slate-500 hover:bg-slate-100'
  }}
>
 Virtual Egg Magic
</button>
</div>
</header>

{/* --- MAIN PAGE LAYOUT --- */}
<main className="flex-1 max-w-7xl w-full mx-auto p-4 md:p-6 flex flex-col gap-6">

  {/* ===== SIMULATOR TAB ===== */}
  {activeTab === 'simulator' && (
    <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 items-stretch">

      {/* Left Column: SVG Tank Sandbox */}
      <div className="lg:col-span-8 bg-white rounded-3xl shadow-sm border
border-slate-200/80 p-5 flex flex-col justify-between">

        <div>
          {/* Sandbox Title Area */}
          <div className="flex justify-between items-center mb-4">
            <div>
              <h2 className="text-lg font-bold text-slate-800">The Hydrodynamic Fluid
Tank</h2>
              <p className="text-xs text-slate-400">Predict the behaviour of common materials
and drop them to see physics in motion.</p>
            </div>
            <button
              onClick={resetAllObjects}
              className="bg-slate-100 hover:bg-slate-200 text-slate-700 px-3.5 py-2 rounded-xl
text-xs font-extrabold transition-all border border-slate-200 flex items-center gap-1.5"
              >
 Clear Tank
            </button>

```

</div>

{/* --- Interactive SVG Viewport --- */}

<div className="relative bg-gradient-to-b from-sky-50 to-blue-50 rounded-2xl
overflow-hidden border border-sky-100 shadow-inner h-[490px]">

{/* Floating Telemetry Stats HUD overlay */}

<div className="absolute top-3 left-3 bg-white/95 backdrop-blur px-4 py-2.5
rounded-xl text-[11px] font-bold shadow-md border border-slate-100 text-slate-700">

<p className="flex items-center gap-1.5 text-blue-600 font-black">

⚡ FLUID DENSITY

</p>

<p className="mt-1 text-slate-800 text-xs">

Water: {waterDensity.toFixed(2)}
g/cm³

</p>

<p className="mt-0.5 text-[9px] text-slate-400">

{waterDensity === 1.0 ? "💧 Normal Tap Water" : "🧂 Salty Brine (High density!)"}</p>

</p>

</div>

{/* Archimedes Displacement HUD overlay */}

<div className="absolute top-3 right-3 bg-white/95 backdrop-blur px-4 py-2.5
rounded-xl text-[11px] font-bold shadow-md border border-slate-100 text-slate-700">

<p className="flex items-center gap-1.5 text-indigo-600 font-black">

🌊 DISPLACEMENT

</p>

<p className="mt-1 text-slate-800 text-xs">

Displaced Fluid: <span className="font-black
text-indigo-600">{getTotalDisplacedVolume().toFixed(1)} cm³

</p>

<p className="mt-0.5 text-[9px] text-slate-400">

Displacement lifts the water surface!

</p>

</div>

{/* Canvas Graphics */}

<svg

id="sim-svg"

viewBox="0 0 800 550"

className="w-full h-full select-none"

onMouseMove={(e) => {

if (activeObject) {

handleDrag(e, e.clientX, e.clientY);

```

    }
  }}
  onMouseUp={handleEndDrag}
  onMouseLeave={handleEndDrag}
  onTouchMove={(e) => {
    if (activeObject && e.touches.length > 0) {
      handleDrag(e, e.touches[0].clientX, e.touches[0].clientY);
    }
  }}
  onTouchEnd={handleEndDrag}
>
  { /* Shelf Background Illustration */}
  <rect x="30" y="115" width="740" height="15" rx="5" fill="#a16207" />
  <rect x="50" y="130" width="12" height="15" fill="#854d0e" />
  <rect x="738" y="130" width="12" height="15" fill="#854d0e" />
  <text x="40" y="105" fill="#713f12" className="text-[10px] font-black uppercase
tracking-wider">Storage Shelf</text>

  { /* Liquid Tank Framework */}
  { /* Inner Fluid Backwall shadow */}
  <rect x="180" y="180" width="440" height="300" fill="#f8f8f8" fillOpacity="0.5"
  rx="16" />

  { /* Dynamic Water Volume with Waves */}
  <path
    d={`
      M 182,${DYNAMIC_WATER_Y_TOP}
      C 240,${DYNAMIC_WATER_Y_TOP - 4 + Math.sin(waveOffset) * 6}
      320,${DYNAMIC_WATER_Y_TOP + 4 + Math.cos(waveOffset) * 6}
      400,${DYNAMIC_WATER_Y_TOP + Math.sin(waveOffset + 1) * 4}
      C 480,${DYNAMIC_WATER_Y_TOP - 4 + Math.cos(waveOffset + 1) * 6}
      560,${DYNAMIC_WATER_Y_TOP + 4 + Math.sin(waveOffset) * 6}
      618,${DYNAMIC_WATER_Y_TOP}
      L 618,478
      L 182,478
      Z
    `}
    fill={waterDensity > 1.15 ? '#1d4ed8' : waterDensity > 1.05 ? '#2563eb' : '#38bdf8'}
    fillOpacity="0.65"
    className="transition-colors duration-500"
  />

  { /* Highlighted Water Crest Line */}
  <path

```

```

d={
  M 182,${DYNAMIC_WATER_Y_TOP}
  C 240,${DYNAMIC_WATER_Y_TOP + 2 + Math.cos(waveOffset) * 7}
320,${DYNAMIC_WATER_Y_TOP - 3 + Math.sin(waveOffset) * 7}
400,${DYNAMIC_WATER_Y_TOP + Math.cos(waveOffset + 2) * 5}
  C 480,${DYNAMIC_WATER_Y_TOP + 3 + Math.sin(waveOffset + 2) * 7}
560,${DYNAMIC_WATER_Y_TOP - 2 + Math.cos(waveOffset) * 7}
618,${DYNAMIC_WATER_Y_TOP}
`
  fill="none"
  stroke="#0284c7"
  strokeWidth="3.5"
  strokeOpacity="0.9"
/>

{/* Glass tank front outline to show realistic depth */}
<rect x="178" y="178" width="444" height="304" fill="none" stroke="#94a3b8"
strokeWidth="6" rx="18" />

{/* Base Solid Plate */}
<rect x="160" y="482" width="480" height="18" fill="#475569" rx="6" />

{/* Dynamic Tank Bed Rocks illustration */}
<circle cx="210" cy="470" r="12" fill="#94a3b8" />
<circle cx="230" cy="474" r="8" fill="#64748b" />
<circle cx="580" cy="472" r="10" fill="#94a3b8" />
<circle cx="600" cy="474" r="7" fill="#64748b" />

{/* --- RENDER SIMULATION OBJECTS --- */}
{objects.map((obj) => {
  const isDragging = activeObject === obj.id;

  // Gravity Vector:  $F_g = \text{mass} * g = \text{volume} * \text{density} * \text{scale}$ 
  const gravityForceValue = obj.volume * obj.density * 0.16;
  // Buoyant Vector:  $F_b = \text{displaced volume} * \text{fluid density} * \text{scale}$ 
  const buoyantForceValue = obj.volume * Math.min(obj.density, waterDensity) *
0.16;

  return (
    <g
      key={obj.id}
      className="cursor-pointer"
      transform={`translate(${obj.x}, ${obj.y})`}
      onMouseDown={(e) => handleStartDrag(e, obj, e.clientX, e.clientY)}

```

```

onTouchStart={e) => {
  if (e.touches.length > 0) {
    handleStartDrag(e, obj, e.touches[0].clientX, e.touches[0].clientY);
  }
}}
>
  { /* Pulsing Drag Ring Indicator */
  {isDragging && (
    <circle cx="0" cy="0" r={obj.volume / 1.5 + 14} fill="#2563eb" fillOpacity="0.25"
className="animate-ping" />
  )}

  { /* Render Realistic Object Graphic Assets */
  {obj.id === 'nail' && (
    <path d="M-6,-32 L6,-32 L3,-22 L3,24 L-3,24 L-3,-22 Z" fill={obj.color}
stroke={obj.borderColor} strokeWidth="2" />
  )}
  {obj.id === 'block' && (
    <rect x={-obj.volume/2} y={-obj.volume/3.2} width={obj.volume}
height={obj.volume/1.6} fill={obj.color} stroke={obj.borderColor} strokeWidth="3" rx="6" />
  )}
  {obj.id === 'stone' && (
    <ellipse cx="0" cy="0" rx={obj.volume/1.5} ry={obj.volume/2} fill={obj.color}
stroke={obj.borderColor} strokeWidth="3" />
  )}
  {obj.id === 'boat' && (
    <g>
      { /* Triangular Sail */
      <path d="M-18,-28 L15,-28 L-18,6 Z" fill="#f8fafc" stroke="#94a3b8"
strokeWidth="1.5" />
      { /* Wood Mast */
      <line x1="-18" y1="6" x2="-18" y2="-34" stroke="#78350f" strokeWidth="3.5"
/>

      { /* Boat Hull */
      <path d="M-40,6 L40,6 L25,28 L-25,28 Z" fill={obj.color}
stroke={obj.borderColor} strokeWidth="3" />
      </g>
    )}
    {obj.id === 'foil' && (
      <circle cx="0" cy="0" r={obj.volume/2} fill="url(#silver-foil-shimmer)"
stroke={obj.borderColor} strokeWidth="2" />
    )}

    { /* Object Emoji label */

```

```

        <text x="0" y="5" textAnchor="middle" className="text-xl select-none"
pointerEvents="none">
        {obj.iconText}
    </text>

    {/* Object Tag overlay */}
    <text x="0" y={obj.id === 'boat' ? '44' : obj.volume/2 + 20} textAnchor="middle"
className="text-[10px] font-black fill-slate-800 bg-white/90 px-1.5 py-0.5 rounded border
border-slate-200">
        {obj.name}
    </text>

    {/* --- FORCE VECTOR VECTORS --- */}
    {obj.status === 'tank' && !obj.isDragging && (
    <g>
        {/* Gravity Pull Vector (Red) */}
        <line
            x1="0"
            y1="0"
            x2="0"
            y2={Math.min(65, gravityForceValue)}
            stroke="#ef4444"
            strokeWidth="4"
            markerEnd="url(#vector-arrow-red)"
        />
        <text x="14" y={Math.min(65, gravityForceValue) - 2} className="fill-red-600
text-[10px] font-extrabold">
            Fg (Gravity: {gravityForceValue.toFixed(0)})
        </text>

        {/* Buoyant Upward Vector (Blue) */}
        <line
            x1="-10"
            y1="0"
            x2="-10"
            y2={-Math.min(65, buoyantForceValue)}
            stroke="#3b82f6"
            strokeWidth="4"
            markerEnd="url(#vector-arrow-blue)"
        />
        <text x="-84" y={-Math.min(65, buoyantForceValue) + 12}
className="fill-blue-600 text-[10px] font-extrabold">
            Fb (Buoyancy: {buoyantForceValue.toFixed(0)})
        </text>
    )}

```



```

        </g>
      })
    </g>
  );
}}

{ /* SVG Definitions for Glimmer Gradients and Arrows */
<defs>
  <marker id="vector-arrow-red" viewBox="0 0 10 10" refX="5" refY="5"
markerWidth="6" markerHeight="6" orient="auto-start-reverse">
    <path d="M 0 0 L 10 5 L 0 10 z" fill="#ef4444" />
  </marker>
  <marker id="vector-arrow-blue" viewBox="0 0 10 10" refX="5" refY="5"
markerWidth="6" markerHeight="6" orient="auto-start-reverse">
    <path d="M 0 0 L 10 5 L 0 10 z" fill="#3b82f6" />
  </marker>
  <radialGradient id="silver-foil-shimmer" cx="50%" cy="50%" r="50%" fx="30%"
fy="30%">
    <stop offset="0%" stopColor="#ffffff" />
    <stop offset="65%" stopColor="#cbd5e1" />
    <stop offset="100%" stopColor="#64748b" />
  </radialGradient>
</defs>

{ /* Splash Particle Layer */
{splashParticles.map((p) => (
  <circle
    key={p.id}
    cx={p.x}
    cy={p.y}
    r={p.size}
    fill="#f0f9ff"
    stroke="#0ea5e9"
    strokeWidth="1.5"
    opacity={p.opacity}
  />
)}}
</svg>

{ /* Empty Drag Target Box Hint Overlay */
{activeObject && (
  <div className="absolute inset-x-52 bottom-12 top-52 border-2 border-dashed
border-blue-400 rounded-3xl bg-blue-500/5 flex flex-col items-center justify-center
pointer-events-none animate-pulse gap-2">

```

```

        <span className="text-3xl">🌈</span>
        <p className="text-xs font-black text-blue-600 uppercase
tracking-widest">Release into the water!</p>
    </div>
  )}
</div>
</div>

{/* Salt Shaker Slider Controls */}
<div className="mt-4 p-4 bg-slate-50 border border-slate-200/80 rounded-2xl">
  <div className="flex flex-col md:flex-row justify-between items-start md:items-center
gap-4">
    <div className="flex-1">
      <div className="flex items-center gap-2">
        <span className="text-xl">🧂</span>
        <h4 className="font-extrabold text-slate-800 text-sm">Dissolve Pure Salt in
Tank</h4>
      </div>
      <p className="text-xs text-slate-500 mt-1">
        Dissolving salt compresses free water molecules. Squeeze more salt into the tank
to watch how borderline dense materials float!
      </p>
    </div>
    <div className="w-full md:w-72 flex items-center gap-3">
      <span className="text-xs text-slate-400 font-extrabold uppercase">Pure</span>
      <input
        type="range"
        min="1.0"
        max="1.2"
        step="0.01"
        value={waterDensity}
        onChange={(e) => setWaterDensity(parseFloat(e.target.value))}
        className="flex-1 h-2 bg-slate-200 rounded-lg appearance-none cursor-pointer
accent-blue-600"
      />
      <span className="text-xs text-blue-700 font-black uppercase">Super
Salty</span>
    </div>
  </div>
</div>

{/* Right Column: Prediction Game panel & live telemetry */}

```

```

<div className="lg:col-span-4 flex flex-col gap-6">

  {/* Prediction & Exploration Deck */}
  <div className="bg-white rounded-3xl shadow-sm border border-slate-200/80 p-5">
    <h3 className="font-black text-slate-800 text-sm flex items-center gap-2">
      <span>🎯</span> Predict & Drop Deck
    </h3>
    <p className="text-xs text-slate-400 mt-1 leading-relaxed">
      Before dropping, select a prediction. Can you make a perfect score sheet?
    </p>

    <div className="mt-4 space-y-3.5">
      {objects.map((obj) => {
        const status = obj.status === 'tank';
        const floats = obj.density < waterDensity;

        return (
          <div key={obj.id} className="p-3 bg-slate-50 border border-slate-100
rounded-2xl flex flex-col gap-2">
            <div className="flex justify-between items-center">
              <span className="font-black text-xs text-slate-700">
                {obj.iconText} {obj.name}
              </span>

              {/* Live Position Badge */}
              {status ? (
                <span className={`px-2 py-0.5 rounded-full text-[9px] font-black
tracking-wider uppercase ${
                  floats ? 'bg-blue-100 text-blue-700' : 'bg-amber-100 text-amber-700'
                }`}>
                  {floats ? 'Floats 🟡' : 'Sank 🟠'}
                </span>
              ) : (
                <span className="text-[10px] text-slate-400 uppercase font-bold">On
Shelf</span>
              )}
            </div>

            {/* Interactive Prediction buttons if on shelf */}
            {!status ? (
              <div className="grid grid-cols-2 gap-2 mt-1">
                <button
                  onClick={() => handlePredict(obj.id, 'float')}

```

```

border ${
    className={`py-1.5 rounded-lg text-[10px] font-black uppercase transition-all
    obj.prediction === 'float'
      ? 'bg-blue-600 border-blue-600 text-white'
      : 'bg-white border-slate-200 text-slate-600 hover:bg-slate-50'
    }}
  >
    Predict Float 🎈
  </button>
  <button
    onClick={() => handlePredict(obj.id, 'sink')}
    className={`py-1.5 rounded-lg text-[10px] font-black uppercase transition-all
    obj.prediction === 'sink'
      ? 'bg-amber-600 border-amber-600 text-white'
      : 'bg-white border-slate-200 text-slate-600 hover:bg-slate-50'
    }}
  >
    Predict Sink 🎯
  </button>
</div>
): (
  /* Show feedback on prediction */
  <div className="flex items-center justify-between text-[10px] bg-white px-2.5
py-1.5 rounded-lg border border-slate-100">
    <span className="text-slate-500 font-medium">Your prediction:</span>
    <span className={`font-black uppercase ${
      obj.predictionCorrect ? 'text-emerald-600' : 'text-slate-600'
    }`}>
      {obj.predictionCorrect === true ? '✅ CORRECT!' : obj.predictionCorrect ===
false ? '❌ INCORRECT' : 'Analysing...'}
    </span>
    <button
      onClick={() => recallObject(obj.id)}
      className="text-[9px] font-extrabold text-blue-600 hover:underline"
    >
      Recall
    </button>
  </div>
  )}
</div>
);
}}
</div>

```

```

</div>

{ /* Live Telemetry Database */
<div className="bg-white rounded-3xl shadow-sm border border-slate-200/80 p-5">
  <h3 className="font-black text-slate-800 text-sm flex items-center gap-2">
    <span><img alt="Telemetry Science Board logo" data-bbox="268 183 288 200"/></span> Telemetry Science Board
  </h3>
  <div className="mt-3 divide-y divide-slate-100 text-xs">
    <div className="py-2 flex justify-between">
      <span className="text-slate-400">Pure Water Density</span>
      <span className="font-bold text-slate-800">1.00 g/cm³</span>
    </div>
    <div className="py-2 flex justify-between">
      <span className="text-slate-400">Current Fluid Density</span>
      <span className="font-bold text-blue-600">{waterDensity.toFixed(2)}
g/cm³</span>
    </div>
    <div className="py-2 flex justify-between">
      <span className="text-slate-400">Borderline Density Target</span>
      <span className="font-bold text-amber-600">Aluminium (1.10 g/cm³)</span>
    </div>
  </div>

  <div className="bg-amber-50 rounded-2xl p-3 border border-amber-100/50 mt-4 flex
gap-2.5 items-start">
    <span className="text-base"><img alt="Lightbulb icon" data-bbox="461 550 478 565"/></span>
    <p className="text-[10.5px] text-amber-800 leading-relaxed font-medium">
      Try launching the **Aluminium Ball**. Squeeze salt into the water using the slider
below. Notice how it floats as the fluid density climbs past **1.10 g/cm³**!
    </p>
  </div>

</div>

</div>
)}}

{ /* ===== QUIZ TAB ===== */
{activeTab === 'quiz' && (
  <div className="bg-white rounded-3xl shadow-sm border border-slate-200/80 p-6
max-w-2xl mx-auto">
    {!quizFinished ? (
      <div>

```

```

    { /* Header indicators */ }
    <div className="flex justify-between items-center border-b border-slate-100 pb-4
mb-4">
      <span className="text-xs font-black text-blue-600 uppercase tracking-widest">
        Question {currentQuizIndex + 1} of {quizQuestions.length}
      </span>
      <div className="flex items-center gap-1 bg-yellow-100 px-3 py-1 rounded-full
text-xs font-black text-yellow-800">
        🏆 Score: {quizScore}
      </div>
    </div>

    { /* Progressive Bar */ }
    <div className="w-full bg-slate-100 h-2 rounded-full mb-6">
      <div
        className="bg-blue-600 h-2 rounded-full transition-all duration-300"
        style={{ width: `${((currentQuizIndex + 1) / quizQuestions.length) * 100}%` }}
      />
    </div>

    { /* Question */ }
    <h3 className="text-base font-black text-slate-800 mb-6 leading-snug">
      {quizQuestions[currentQuizIndex].question}
    </h3>

    { /* Options list */ }
    <div className="space-y-3">
      {quizQuestions[currentQuizIndex].options.map((option, idx) => {
        let btnStyle = "border-slate-200 hover:border-slate-300 hover:bg-slate-50";

        if (selectedAnswer === idx) {
          btnStyle = "border-blue-600 bg-blue-50/50 ring-2 ring-blue-600";
        }

        if (quizSubmitted) {
          if (option.isCorrect) {
            btnStyle = "border-emerald-500 bg-emerald-50 text-emerald-900 font-bold";
          } else if (selectedAnswer === idx) {
            btnStyle = "border-rose-300 bg-rose-50 text-rose-900 line-through";
          } else {
            btnStyle = "border-slate-100 opacity-50";
          }
        }
      })
    </div>

```

```

return (
  <button
    key={idx}
    disabled={quizSubmitted}
    onClick={() => handleAnswerSelection(idx)}
    className={`w-full text-left p-4 rounded-2xl border text-xs font-bold transition-all
flex items-start gap-3.5 ${btnStyle}`}
  >
    <span className="bg-slate-200 text-slate-700 font-extrabold w-6 h-6
rounded-full inline-flex items-center justify-center text-[10px] shrink-0">
      {String.fromCharCode(65 + idx)}
    </span>
    <span>{option.label}</span>
  </button>
);
}}
</div>

```

```

{/* Explanation text box */}
{quizSubmitted && (
  <div className="mt-6 p-4 bg-blue-50 border border-blue-100 rounded-2xl text-xs
leading-relaxed">
    <p className="font-black text-blue-900 flex items-center gap-1.5">
      {quizQuestions[currentQuizIndex].options[selectedAnswer].isCorrect ? '🎉 Correct
Answer!' : '💡 Lesson Card:'}
    </p>
    <p className="text-blue-800 mt-1 font-medium">
      {quizQuestions[currentQuizIndex].explanation}
    </p>
  </div>
)}

```

```

{/* Navigation Buttons */}
<div className="mt-8 flex justify-end">
  {!quizSubmitted ? (
    <button
      disabled={selectedAnswer === null}
      onClick={handleQuizSubmit}
      className="bg-blue-600 hover:bg-blue-700 disabled:opacity-50 text-white
font-black px-6 py-3 rounded-2xl text-xs uppercase tracking-wider shadow transition-all"
    >
      Verify Answer
    </button>
  ) : (

```

```

        <button
            onClick={handleNextQuiz}
            className="bg-blue-600 hover:bg-blue-700 text-white font-black px-6 py-3
rounded-2xl text-xs uppercase tracking-wider shadow transition-all flex items-center gap-1.5"
        >
            {currentQuizIndex + 1 < quizQuestions.length ? 'Continue' : 'Finish Evaluation'}
        </button>
    )}
</div>
</div>
): (
    /* Quiz Finished Summary Page */
    <div className="text-center py-8">
        <span className="text-6xl">🎉</span>
        <h3 className="text-xl font-black text-slate-800 mt-4">Evaluation Complete!</h3>
        <p className="text-slate-400 text-xs mt-1">Let's look at your dynamic
scorecard:</p>

        <div className="my-6 max-w-xs mx-auto bg-slate-50 p-6 rounded-3xl border
border-slate-100">
            <div className="text-4xl font-black text-blue-600">
                {quizScore} / {quizQuestions.length}
            </div>
            <p className="text-[10px] text-slate-400 font-extrabold uppercase mt-1">Total
Correct Answers</p>

            <div className="mt-4 text-xs font-black text-slate-700">
                {quizScore >= 8 ? (
                    <span className="text-emerald-600">🏆 Fluid Dynamics Wizard!</span>
                ) : quizScore >= 5 ? (
                    <span className="text-indigo-600">🏅 Elite Science Explorer!</span>
                ) : (
                    <span className="text-slate-500">👉 Great effort! Keep exploring.</span>
                )}
            </div>
        </div>

        <div className="flex justify-center gap-3">
            <button
                onClick={resetQuiz}
                className="bg-blue-600 hover:bg-blue-700 text-white font-black px-6 py-3
rounded-2xl text-xs uppercase tracking-wider transition-all"
            >
                Retake Quiz

```



```

    </button>
    <button
      onClick={() => setActiveTab('simulator')}
      className="bg-slate-100 hover:bg-slate-200 text-slate-700 font-black px-6 py-3
rounded-2xl text-xs uppercase tracking-wider border border-slate-200 transition-all"
    >
      Go to Simulator
    </button>
  </div>
</div>
)}
</div>
)}

{/* ===== LEARN TAB ===== */}
{activeTab === 'learn' && (
  <div className="grid grid-cols-1 md:grid-cols-12 gap-6 items-start">

    {/* Visual Quick Lessons */}
    <div className="md:col-span-8 bg-white rounded-3xl shadow-sm border
border-slate-200/80 p-6 space-y-6">

      <div>
        <h2 className="text-lg font-black text-slate-800 flex items-center gap-2">
          <span>📖</span> Density & Buoyancy Made Simple
        </h2>
        <p className="text-xs text-slate-400 mt-1">The science behind why things float and
sink (Class 5 curriculum)</p>
      </div>

      <hr className="border-slate-100" />

      <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
        <div className="p-4 bg-sky-50 rounded-2xl border border-sky-100">
          <h4 className="font-black text-sky-900 text-xs uppercase flex items-center
gap-1.5">
            <span>📦</span> Visualizing Density
          </h4>
          <p className="text-xs text-sky-800 mt-2 leading-relaxed font-medium">
            Think of density as how crowded a classroom is. If many students are packed
inside a tiny room, it is very crowded (**high density**). If there are only 2 students in a massive
playground, it has **low density**. Sinking materials like iron or rock are tightly crowded inside!
          </p>
        </div>
      </div>
    </div>
  )
}

```

```

<div className="p-4 bg-indigo-50 rounded-2xl border border-indigo-100">
  <h4 className="font-black text-indigo-900 text-xs uppercase flex items-center
gap-1.5">
    <span>📌</span> The Upward Force
  </h4>
  <p className="text-xs text-indigo-800 mt-2 leading-relaxed font-medium">
    When you force a balloon underwater, you can feel water pushing back up against
    your palm with an invisible force. This upward water pressure is called the Buoyant Force.
  </p>
</div>
</div>

```

```

{ /* Memory Trick Panel */ }
<div className="bg-amber-50/50 border border-amber-200/60 rounded-2xl p-4">
  <h4 className="font-bold text-amber-900 text-xs uppercase tracking-wider">💡 The
Ultimate Floating Gold Rule</h4>
  <ul className="text-xs text-amber-800 mt-2 space-y-2 list-disc pl-5 font-medium">
    <li><strong>Floater:</strong> Average density is less than water density. Water
easily wins and pushes it to the top!</li>
    <li><strong>Sinkers:</strong> Average density is greater than water density.
Gravity wins and pulls it down!</li>
    <li><strong>Volume Matters:</strong> Wide, hollow shapes trap lots of air, lowering
average density (e.g., ships, boats).</li>
  </ul>
</div>

```

```

</div>

{ /* Quick Real Life Cards */ }
<div className="md:col-span-4 bg-white rounded-3xl shadow-sm border
border-slate-200/80 p-5">
  <h3 className="font-black text-slate-800 text-xs uppercase tracking-widest flex
items-center gap-2 mb-4">
    <span>🌟</span> Real-world wonders
  </h3>

```

```

<div className="space-y-4">
  <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-100">
    <span className="text-xl">🌈</span>
    <h4 className="font-black text-slate-800 text-xs mt-1">Submarines</h4>
    <p className="text-[11px] text-slate-400 mt-1 leading-relaxed">
      By sucking ocean water into ballast tanks, submarines become heavy for their size
      and sink. By pumping air in, they become light for their size and float!
    </p>
  </div>

```

```
</p>
</div>
```

```
<div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-100">
  <span className="text-xl">❄️</span>
  <h4 className="font-black text-slate-800 text-xs mt-1">The Floating Iceberg</h4>
  <p className="text-[11px] text-slate-400 mt-1 leading-relaxed">
    As water turns into solid ice, it expands. This volume expansion lowers its density,
    letting colossal icebergs float safely on ocean waters.
```

```
</p>
</div>
</div>
</div>
```

```
</div>
)}}

```

```
{/* ===== DIY TAB ===== */}
{activeTab === 'diy' && (
  <div className="bg-white rounded-3xl shadow-sm border border-slate-200/80 p-6
    max-w-4xl mx-auto">
```

```
<div className="grid grid-cols-1 md:grid-cols-12 gap-8 items-stretch">
```

```
  {/* Virtual Glass with Egg Animation */}
  <div className="md:col-span-6 bg-slate-50 rounded-2xl p-5 border
    border-slate-200/50 flex flex-col justify-between items-center min-h-[380px]">
```

```
    <h4 className="text-xs font-black text-slate-500 uppercase tracking-widest
      text-center">
      Virtual Lab Companion: Floating Egg
    </h4>
```

```
    {/* Simulated Glass with Dynamic Egg Displacement */}
    <div className="relative my-6 w-36 h-60 border-4 border-slate-300 bg-white
      rounded-b-3xl shadow-lg flex flex-col justify-end overflow-hidden">
```

```
      {/* Fluid Surface overlay */}
      <div
        className="w-full bg-blue-300 transition-all duration-500 relative"
        style={{
          height: '80%',
          backgroundColor: isEggFloating ? '#1d4ed8' : diySaltSpoons > 0 ? '#3b82f6' :
            '#93c5fd',
```

```

        opacity: 0.8
      }}
    >
    { /* Raw Floating Egg */
    <div
      className="absolute left-1/2 -translate-x-1/2 w-14 h-18 bg-amber-50
rounded-[50%_50%_50%_50%/60%_60%_40%_40%] shadow-md border
border-amber-200/60 transition-all duration-1000 flex items-center justify-center text-[10px]
font-black text-amber-700 uppercase"
      style={{
        bottom: isEggFloating ? '110px' : '6px',
        transform: 'translate-x-1/2 rotate(4deg)'
      }}
    >
       Egg
    </div>

    { /* Salt grains swirl */
    {diySaltSpoons > 0 && (
      <div className="absolute inset-x-0 bottom-1.5 flex flex-wrap justify-around
opacity-30 select-none pointer-events-none">
        <span className="text-[8px] text-white animate-pulse">●</span>
        <span className="text-[8px] text-white animate-pulse">●</span>
        <span className="text-[8px] text-white animate-pulse">●</span>
      </div>
    )}
  </div>
</div>

{ /* Spatula Controls */
<div className="w-full space-y-3">
  <div className="flex justify-center gap-2">
    <button
      disabled={diySaltSpoons >= 4}
      onClick={() => {
        setDiySaltSpoons(prev => prev + 1);
        setXp(prev => prev + 10);
      }}
      className="bg-blue-600 hover:bg-blue-700 disabled:opacity-50 text-white text-xs
font-black px-4 py-2 rounded-xl transition-all uppercase tracking-wide shadow"
    >
       Dissolve 1 Spoon
    </button>
    <button

```

```

      onClick={() => setDiySaltSpoons(0)}
      className="bg-slate-200 hover:bg-slate-300 text-slate-700 text-xs font-black px-4
py-2 rounded-xl transition-all uppercase tracking-wide"
    >
       Reset Glass
    </button>
  </div>

```

```

    <p className="text-xs text-center font-bold text-slate-500">
      {diySaltSpoons === 0 && "Fresh Water: Egg density is high, so it sinks down."}
      {diySaltSpoons > 0 && diySaltSpoons < 3 && `Dissolved ${diySaltSpoons}
spoons... Fluid getting denser!`}
      {diySaltSpoons >= 3 && "Saturated Salty Brine! The water is denser than the egg.
FLOAT! 🥚💧"}
    </p>
  </div>

```

```

</div>

```

```

  {/* Physical Activity guide cards */}
  <div className="md:col-span-6 space-y-5">
    <div>
      <span className="bg-amber-100 text-amber-800 text-[10px] font-black uppercase
px-2.5 py-1 rounded-full">
        🏠 Kitchen Lab Guide
      </span>
      <h3 className="text-xl font-black text-slate-800 mt-2">The Floating Egg
Magic</h3>
      <p className="text-xs text-slate-400">Perform this simple, risk-free test at home to
amaze your friends!</p>
    </div>

```

```

    <div className="space-y-3.5 text-xs text-slate-600 font-medium">
      <div className="flex gap-3">
        <span className="bg-blue-100 text-blue-700 font-black w-5 h-5 rounded-full
inline-flex items-center justify-center shrink-0">1</span>
        <p>Pour fresh tap water into a clear transparent drinking glass.</p>
      </div>
      <div className="flex gap-3">
        <span className="bg-blue-100 text-blue-700 font-black w-5 h-5 rounded-full
inline-flex items-center justify-center shrink-0">2</span>
        <p>Carefully lower a raw egg into the glass. Watch it sink directly to the
bottom!</p>
      </div>

```

```
    <div className="flex gap-3">
      <span className="bg-blue-100 text-blue-700 font-black w-5 h-5 rounded-full
inline-flex items-center justify-center shrink-0">3</span>
```

```
      <p>Take the egg out. Stir 3-4 tablespoons of regular cooking salt until fully
dissolved.</p>
```

```
    </div>
```

```
    <div className="flex gap-3">
```

```
      <span className="bg-blue-100 text-blue-700 font-black w-5 h-5 rounded-full
inline-flex items-center justify-center shrink-0">4</span>
```

```
      <p>Gently put the egg back. You will see it floating gracefully like magic!</p>
```

```
    </div>
```

```
</div>
```

```
    <div className="p-4 bg-indigo-50 border border-indigo-100 rounded-2xl text-xs
font-medium text-indigo-900 leading-relaxed">
```

```
      <span className="font-black block">💡 The Physical Explanation:</span>
```

```
      Adding salt increases the number of water molecules per cubic centimeter. The salt
water becomes much heavier for its size than the egg, resulting in a giant upward Buoyant
Force!
```

```
    </div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
  )}
```

```
</main>
```

```
{/* --- Footer Safety Bar --- */}
```

```
<footer className="bg-slate-900 text-slate-400 text-[11px] py-6 border-t border-slate-800
mt-12">
```

```
  <div className="max-w-7xl mx-auto px-4 flex flex-col md:flex-row justify-between
items-center gap-4">
```

```
    <div className="text-center md:text-left space-y-1">
```

```
      <p className="font-black text-slate-300">The Splash Lab • NCERT Materials
Exploration</p>
```

```
      <p>Built with high-fidelity water displacement algorithms and vectors.</p>
```

```
    </div>
```

```
    <div className="flex gap-4">
```

```
      <span className="text-xs text-emerald-400">🛡️ 100% Kid-Safe Web Sandbox</span>
```

```
      <span className="text-xs text-blue-400">📱 Mobile & Touch Optimized</span>
```

```
    </div>
```

```
</div>
```

```
    </footer>
  </div>
);
}
```

Rain water harvesting


```
import React, { useState, useEffect, useRef, useMemo } from 'react';
import { initializeApp } from 'firebase/app';
import {
  getAuth,
  signInWithCustomToken,
  signInAnonymously,
  onAuthStateChanged
} from 'firebase/auth';
import {
  getFirestore,
  doc,
  setDoc,
  getDoc,
  collection,
  query,
  onSnapshot
} from 'firebase/firestore';
import {
  Droplets,
  Award,
  RotateCcw,
  Play,
  Pause,
  BookOpen,
  Compass,
  HelpCircle,
  Volume2,
  VolumeX,
  Info,
  Users,
  CheckCircle2,
  XCircle,
  Lightbulb,
  ArrowRight,
  Sparkles
} from 'lucide-react';
```

```

// --- FIREBASE CONFIGURATION & INITIALIZATION ---
// Safe fallback structures for our environment
const firebaseConfig = typeof __firebase_config !== 'undefined'
  ? JSON.parse(__firebase_config)
  : { apiKey: "", authDomain: "", projectId: "", storageBucket: "",
    messagingSenderId: "", appId: "" };

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getFirestore(app);
const appId = typeof __app_id !== 'undefined' ? __app_id :
  'rainwater-harvesting-6';

// Generates a playful animal-themed scientific username for the
// leaderboard
const generateScientificUsername = () => {
  const adjectives = ['Eco', 'Hydro', 'Aqua', 'Green', 'Solar', 'Cloud',
    'Terra', 'Bio'];
  const nouns = ['Beaver', 'Otter', 'Dolphin', 'Frog', 'Seedling',
    'Sprout', 'Panda', 'Gecko'];
  const randAdj = adjectives[Math.floor(Math.random() *
    adjectives.length)];
  const randNoun = nouns[Math.floor(Math.random() * nouns.length)];
  const randNum = Math.floor(100 + Math.random() * 900);
  return `${randAdj}${randNoun}${randNum}`;
};

export default function App() {
  // --- CORE STATE ---
  const [user, setUser] = useState(null);
  const [username, setUsername] = useState('');
  const [xp, setXp] = useState(0);
  const [unlockedAchievements, setUnlockedAchievements] = useState([]);
  const [activeTab, setActiveTab] = useState('missions'); // 'missions',
  'quiz', 'diy', 'leaderboard'

```

```

const [soundEnabled, setSoundEnabled] = useState(false);

// --- SIMULATION STATES ---
const [roofSize, setRoofSize] = useState(2); // 1: Small (15m²), 2:
Medium (30m²), 3: Large (50m²)
const [rainIntensity, setRainIntensity] = useState(2); // 1: Drizzle, 2:
Moderate, 3: Heavy Storm
const [isRaining, setIsRaining] = useState(false);
const [waterVolume, setWaterVolume] = useState(0); // in Liters (max 500)
const [evaporationActive, setEvaporationActive] = useState(false);
const [clogLevel, setClogLevel] = useState(0); // 0 to 100% clog
(simulates filter cleanliness)

// --- MISSION / CHALLENGE TRACKING ---
const [activeChallenge, setActiveChallenge] = useState(0);
const [challengeTimer, setChallengeTimer] = useState(0);
const [challengeState, setChallengeState] = useState('idle'); // 'idle',
'running', 'success', 'fail'
// --- QUIZ STATE ---
const [currentQuizIndex, setCurrentQuizIndex] = useState(0);
const [selectedAnswer, setSelectedAnswer] = useState(null);
const [quizScore, setQuizScore] = useState(0);
const [quizFinished, setQuizFinished] = useState(false);

// --- GLOBAL USERS / LEADERBOARD STATE ---
const [leaderboard, setLeaderboard] = useState([]);

// --- SIMULATION REFS ---
const canvasRef = useRef(null);
const animationRef = useRef(null); // FIX: Added animationRef definition
to resolve ReferenceError
const audioCtxRef = useRef(null);
const rainSoundRef = useRef(null);
const rainDrops = useRef([]);
const pipeWaterParticles = useRef([]);

```

```

// --- THE CONCEPTUAL CHALLENGES ---
const challenges = useMemo(() => [
  {
    title: "Save Your Garden's Supply",
    description: "Collect at least 150 Liters of fresh water under 15
seconds to water the organic garden beds.",
    targetLiters: 150,
    timeLimit: 15,
    requiredRoof: null,
    hint: "Try raising the precipitation rate or expanding the roof size
to capture maximum storm run-off!"
  },
  {
    title: "Maximize Drought Reserve",
    description: "Collect exactly 400 Liters. But hurry! You only have 12
seconds to harness this intense cloud burst.",
    targetLiters: 400,
    timeLimit: 12,
    requiredRoof: 3, // Must discover that Large roof is required
    hint: "A tiny catchment area won't trap enough storm volume in time.
Go as big as possible!"
  },
  {
    title: "Clean Filtration Challenge",
    description: "Collect 250 Liters in 20 seconds. Watch out! The filter
gets clogged slowly. Shake the filter to keep it flowing!",
    targetLiters: 250,
    timeLimit: 20,
    dynamicClogging: true,
    hint: "Click 'Clear Clogs' continuously in the control drawer when
the filter gets blocked!"
  }
], []);

// --- AUDIO SYNTHESIS ENGINE ---
const initAudio = () => {

```

```

    if (!audioCtxRef.current) {
      audioCtxRef.current = new (window.AudioContext ||
window.webkitAudioContext)();
    }
  };

const playSynthesizedChime = (success = true) => {
  if (!soundEnabled) return;
  initAudio();
  const ctx = audioCtxRef.current;
  if (ctx.state === 'suspended') ctx.resume();

  const now = ctx.currentTime;
  const osc = ctx.createOscillator();
  const gain = ctx.createGain();

  if (success) {
    osc.type = 'triangle';
    osc.frequency.setValueAtTime(523.25, now); // C5
    osc.frequency.exponentialRampToValueAtTime(783.99, now + 0.15); // G5
    osc.frequency.exponentialRampToValueAtTime(1046.50, now + 0.3); // C6
    gain.gain.setValueAtTime(0.15, now);
    gain.gain.exponentialRampToValueAtTime(0.01, now + 0.4);
  } else {
    osc.type = 'sawtooth';
    osc.frequency.setValueAtTime(220, now); // A3
    osc.frequency.linearRampToValueAtTime(110, now + 0.3);
    gain.gain.setValueAtTime(0.2, now);
    gain.gain.exponentialRampToValueAtTime(0.01, now + 0.35);
  }

  osc.connect(gain);
  gain.connect(ctx.destination);
  osc.start(now);
  osc.stop(now + 0.4);
};

```

```

// --- CONTROLS & HANDLERS ---
const handleStartStop = () => {
  setIsRaining(prev => !prev);
};

const handleReset = () => {
  setIsRaining(false);
  setWaterVolume(0);
  setClogLevel(0);
  setChallengeState('idle');
};

// --- AUTHENTICATION & INITIALIZATION (RULE 3) ---
useEffect(() => {
  const initAuth = async () => {
    try {
      if (typeof __initial_auth_token !== 'undefined' &&
__initial_auth_token) {
        await signInWithCustomToken(auth, __initial_auth_token);
      } else {
        await signInAnonymously(auth);
      }
    } catch (err) {
      console.error("Firebase auth failed:", err);
    }
  };
  initAuth();

  const unsubscribe = onAuthStateChanged(auth, async (currentUser) => {
    if (currentUser) {
      setUser(currentUser);
      // Load or create User profile document (RULE 1)
      const userDocRef = doc(db, 'artifacts', appId, 'users',
currentUser.uid, 'profile', 'data');
      const userSnap = await getDoc(userDocRef);
    }
  });
});

```

```

    if (userSnap.exists()) {
      const data = userSnap.data();
      setUsername(data.username || generateScientificUsername());
      setXp(data.xp || 100);
      setUnlockedAchievements(data.achievements || []);
    } else {
      const generatedName = generateScientificUsername();
      setUsername(generatedName);
      setXp(100);
      await setDoc(userDocRef, {
        username: generatedName,
        xp: 100,
        achievements: [],
        updatedAt: Date.now()
      });
    }
  }
});

return () => unsubscribe();
}, []);

// --- SYNC LOCAL PROGRESS WITH CLOUD FIRESTORE ---
const saveProgress = async (newXp, achievementsList) => {
  if (!user) return;
  try {
    // 1. Save Private Profile data
    const userDocRef = doc(db, 'artifacts', appId, 'users', user.uid,
'profile', 'data');
    await setDoc(userDocRef, {
      username,
      xp: newXp,
      achievements: achievementsList,
      updatedAt: Date.now()
    }, { merge: true });
  }

```

```

    // 2. Save/Update Public Leaderboard placement (RULE 1)
    const leaderboardRef = doc(db, 'artifacts', appId, 'public', 'data',
'leaderboard', user.uid);
    await setDoc(leaderboardRef, {
      userId: user.uid,
      username,
      xp: newXp,
      updatedAt: Date.now()
    });
  } catch (e) {
    console.error("Error saving progress to Firestore: ", e);
  }
};

// --- LEADERBOARD LIVE SUBSCRIPTION (RULE 2 - NO COMPLEX QUERIES) ---
useEffect(() => {
  if (!user) return;
  const collRef = collection(db, 'artifacts', appId, 'public', 'data',
'leaderboard');

  // Using a simple query as requested by Rule 2 (No compound indexes or
filters, sort in-memory)
  const unsubscribe = onSnapshot(collRef, (snapshot) => {
    const records = [];
    snapshot.forEach((doc) => {
      records.push(doc.data());
    });
    // Perform strict sort in JS memory to prevent index failures
    records.sort((a, b) => b.xp - a.xp);
    setLeaderboard(records.slice(0, 10)); // Display Top 10
  }, (error) => {
    console.warn("Firestore leaderboard snapshot error: ", error);
  });

  return () => unsubscribe();

```



```

}, [user]);

// --- AUDIO RAIN SENSITIVITY CONTROLLER ---
useEffect(() => {
  if (isRaining && soundEnabled) {
    initAudio();
    const ctx = audioCtxRef.current;
    if (ctx.state === 'suspended') ctx.resume();

    // Create white noise rain simulation
    const bufferSize = 2 * ctx.sampleRate;
    const noiseBuffer = ctx.createBuffer(1, bufferSize, ctx.sampleRate);
    const output = noiseBuffer.getChannelData(0);
    for (let i = 0; i < bufferSize; i++) {
      output[i] = Math.random() * 2 - 1;
    }

    const whiteNoise = ctx.createBufferSource();
    whiteNoise.buffer = noiseBuffer;
    whiteNoise.loop = true;

    const filter = ctx.createBiquadFilter();
    filter.type = 'lowpass';
    // Map rain rate to frequencies
    filter.frequency.value = rainIntensity === 1 ? 800 : rainIntensity
    === 2 ? 1600 : 3000;

    const gainNode = ctx.createGain();
    gainNode.gain.value = rainIntensity === 1 ? 0.04 : rainIntensity ===
    2 ? 0.08 : 0.15;

    whiteNoise.connect(filter);
    filter.connect(gainNode);
    gainNode.connect(ctx.destination);

    whiteNoise.start();
  }
}, [rainIntensity]);

```

```

    rainSoundRef.current = { source: whiteNoise, gain: gainNode, filter
};
  } else {
    if (rainSoundRef.current) {
      try {
        rainSoundRef.current.source.stop();
      } catch(e) {}
      rainSoundRef.current = null;
    }
  }

  return () => {
    if (rainSoundRef.current) {
      try {
        rainSoundRef.current.source.stop();
      } catch(e) {}
    }
  };
}, [isRaining, rainIntensity, soundEnabled]);

// --- MISSION TIMERS & WIN CHECKER ---
useEffect(() => {
  let interval = null;
  if (challengeState === 'running') {
    interval = setInterval(() => {
      setChallengeTimer(t => {
        if (t <= 1) {
          // Out of time check
          setChallengeState('fail');
          setIsRaining(false);
          playSynthesizedChime(false);
          return 0;
        }
        return t - 1;
      });
    }, 1000);
  }

```

```

    }
    return () => clearInterval(interval);
}, [challengeState]);

// Handle Dynamic Clogging Mechanism
useEffect(() => {
    let interval = null;
    if (isRaining && challenges[activeChallenge].dynamicClogging) {
        interval = setInterval(() => {
            setClogLevel(prev => Math.min(prev + 4, 100));
        }, 300);
    }
    return () => clearInterval(interval);
}, [isRaining, activeChallenge]);

// --- PHYSICS & ANIMATION ENGINE FOR HTML5 CANVAS ---
useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext('2d');

    // Make crisp on High-DPI screens
    const dpr = window.devicePixelRatio || 1;
    const logicalWidth = 600;
    const logicalHeight = 380;
    canvas.width = logicalWidth * dpr;
    canvas.height = logicalHeight * dpr;
    ctx.scale(dpr, dpr);

    // Populate local rain patterns
    const drops = [];
    for (let i = 0; i < 180; i++) {
        drops.push({
            x: Math.random() * logicalWidth,
            y: Math.random() * -logicalHeight,
            speed: Math.random() * 4 + 6,

```

```

        len: Math.random() * 8 + 4,
        weight: Math.random() * 1.5 + 0.5
    });
}

let pipeParticles = [];

const drawSimulation = () => {
    // 1. Draw dynamic background atmosphere
    const bgGrad = ctx.createLinearGradient(0, 0, 0, logicalHeight);
    if (isRaining) {
        bgGrad.addColorStop(0, '#334155'); // Overcast stormy grey
        bgGrad.addColorStop(1, '#475569');
    } else {
        bgGrad.addColorStop(0, '#bae6fd'); // Peaceful morning skies
        bgGrad.addColorStop(1, '#f0f9ff');
    }
    ctx.fillStyle = bgGrad;
    ctx.fillRect(0, 0, logicalWidth, logicalHeight);

    // Sun or Storm Clouds
    if (!isRaining) {
        ctx.beginPath();
        ctx.arc(500, 70, 30, 0, Math.PI * 2);
        ctx.fillStyle = '#f59e0b';
        ctx.shadowBlur = 30;
        ctx.shadowColor = '#fbbf24';
        ctx.fill();
        ctx.shadowBlur = 0; // reset
    } else {
        // Dark clouds
        ctx.fillStyle = '#1e293b';
        ctx.beginPath();
        ctx.arc(80, 40, 35, 0, Math.PI * 2);
        ctx.arc(140, 30, 50, 0, Math.PI * 2);
        ctx.arc(200, 40, 35, 0, Math.PI * 2);
    }

```

```

    ctx.arc(320, 35, 40, 0, Math.PI * 2);
    ctx.arc(380, 25, 55, 0, Math.PI * 2);
    ctx.arc(440, 40, 40, 0, Math.PI * 2);
    ctx.fill();
}

// Ground layers (Grass + Dirt section)
ctx.fillStyle = '#22c55e'; // Green grass
ctx.fillRect(0, 290, logicalWidth, 20);
ctx.fillStyle = '#78350f'; // Under-soil
ctx.fillRect(0, 310, logicalWidth, 70);

// --- STRUCTURAL MATHEMATICS ---
const houseX = 140;
const houseY = 160;
const houseWidth = 180;
const houseHeight = 130;

// Map dynamic roof width mapping (Small, Medium, Large)
const mappedRoofWidth = roofSize === 1 ? 140 : roofSize === 2 ? 220 :
310;

const roofLeftX = houseX + (houseWidth / 2) - (mappedRoofWidth / 2);
const roofRightX = houseX + (houseWidth / 2) + (mappedRoofWidth / 2);
const roofTopY = houseY - 25;

// 2. Draw House structure
ctx.fillStyle = '#fef3c7'; // Cream-painted bricks
ctx.fillRect(houseX, houseY, houseWidth, houseHeight);

// Windows & Doors
ctx.fillStyle = '#1e293b'; // Door frame
ctx.fillRect(houseX + 30, houseY + houseHeight - 75, 45, 75);
ctx.fillStyle = '#fbbf24'; // Door glowing window
ctx.fillRect(houseX + 35, houseY + houseHeight - 70, 12, 12);

ctx.fillStyle = '#bae6fd'; // Window

```

```

ctx.fillRect(houseX + 110, houseY + 30, 40, 40);
ctx.strokeStyle = '#475569';
ctx.lineWidth = 3;
ctx.strokeRect(houseX + 110, houseY + 30, 40, 40);

// 3. Draw Catchment Slanted Roof
ctx.beginPath();
ctx.moveTo(roofLeftX, houseY);
ctx.lineTo(houseX + (houseWidth / 2), roofTopY);
ctx.lineTo(roofRightX, houseY);
ctx.strokeStyle = '#dc2626'; // Terracotta red clay roof look
ctx.lineWidth = 10;
ctx.lineCap = 'round';
ctx.stroke();

// Rooftop solar/eco decoration
ctx.fillStyle = '#0f172a';
ctx.fillRect(houseX + 30, houseY - 12, 40, 6);

// 4. Gutter Catchment Collectors (Placed at edge limits)
ctx.fillStyle = '#475569';
ctx.beginPath();
ctx.arc(roofLeftX, houseY, 7, 0, Math.PI, false);
ctx.fill();

ctx.beginPath();
ctx.arc(roofRightX, houseY, 7, 0, Math.PI, false);
ctx.fill();

// 5. Down-take Pipeline Network routing to filtering unit
ctx.strokeStyle = '#64748b';
ctx.lineWidth = 6;
ctx.lineCap = 'round';
ctx.lineJoin = 'round';

```

```

    // Left gutter drains directly down to connect with central
collection pipeline
    ctx.beginPath();
    ctx.moveTo(roofLeftX, houseY + 4);
    ctx.lineTo(roofLeftX, houseY + 140);
    ctx.lineTo(390, houseY + 140); // Route right towards filter
    ctx.stroke();

    // Right gutter drains down towards filter directly
    ctx.beginPath();
    ctx.moveTo(roofRightX, houseY + 4);
    ctx.lineTo(roofRightX, houseY + 140);
    ctx.lineTo(410, houseY + 140);
    ctx.stroke();

    // Central intake down to storage tank
    ctx.beginPath();
    ctx.moveTo(400, houseY + 140);
    ctx.lineTo(400, 310);
    ctx.stroke();

    // 6. Filtration Unit Tank (Dual-media filter box)
    const filterX = 380;
    const filterY = houseY + 110;
    const filterW = 40;
    const filterH = 50;

    ctx.fillStyle = '#e2e8f0'; // Outer casing
    ctx.fillRect(filterX, filterY, filterW, filterH);
    ctx.strokeStyle = '#475569';
    ctx.lineWidth = 2;
    ctx.strokeRect(filterX, filterY, filterW, filterH);

    // Draw inside layers (Gravel, Charcoal, Sand)
    ctx.fillStyle = '#64748b'; // Gravel top
    ctx.fillRect(filterX + 2, filterY + 5, filterW - 4, 10);

```

```

ctx.fillStyle = '#1e293b'; // Charcoal core
ctx.fillRect(filterX + 2, filterY + 15, filterW - 4, 12);
ctx.fillStyle = '#f59e0b'; // Clean Sand layer
ctx.fillRect(filterX + 2, filterY + 27, filterW - 4, 12);

// Clogging visual warning indicator
if (clogLevel > 0) {
    ctx.fillStyle = `rgba(120, 53, 15, ${clogLevel / 100})`; // Brown
sludge opacity mapping
    ctx.fillRect(filterX + 2, filterY + 2, filterW - 4, 15);
}

// Filter text label
ctx.fillStyle = '#1e293b';
ctx.font = 'bold 8px sans-serif';
ctx.fillText("FILTER", filterX + 5, filterY - 4);

// 7. Underground Storage Cistern
const tankX = 460;
const tankY = 280;
const tankW = 110;
const tankH = 85;

// Draw soil concrete container backing
ctx.fillStyle = '#1e293b';
ctx.fillRect(tankX - 4, tankY - 4, tankW + 8, tankH + 8);

// Draw crystal transparent water chamber
ctx.fillStyle = 'rgba(186, 230, 253, 0.2)';
ctx.fillRect(tankX, tankY, tankW, tankH);

// Draw actual collected water height
const maximumWaterYHeight = tankH - 4;
const filledRatio = waterVolume / 500;
const currentWaterHeight = filledRatio * maximumWaterYHeight;

```



```

    if (currentWaterHeight > 0) {
        const waterColorGrad = ctx.createLinearGradient(tankX, tankY +
tankH - currentWaterHeight, tankX, tankY + tankH);
        waterColorGrad.addColorStop(0, '#38bdf8'); // Clear turquoise blue
        waterColorGrad.addColorStop(1, '#0284c7');
        ctx.fillStyle = waterColorGrad;
        ctx.fillRect(tankX, tankY + tankH - currentWaterHeight, tankW,
currentWaterHeight);

        // Gentle surface ripples inside water
        ctx.strokeStyle = '#ffffff';
        ctx.lineWidth = 1.5;
        ctx.beginPath();
        const waveOffset = (Date.now() / 150) % (Math.PI * 2);
        for (let wx = 0; wx <= tankW; wx += 5) {
            const wy = Math.sin((wx / 10) + waveOffset) * 2;
            if (wx === 0) ctx.moveTo(tankX + wx, tankY + tankH -
currentWaterHeight + wy);
            else ctx.lineTo(tankX + wx, tankY + tankH - currentWaterHeight +
wy);
        }
        ctx.stroke();
    }

    // Glass front grid scale ticks
    ctx.strokeStyle = 'rgba(255, 255, 255, 0.4)';
    ctx.lineWidth = 1;
    for (let j = 1; j <= 4; j++) {
        const stepY = tankY + tankH - (j * (tankH / 5));
        ctx.beginPath();
        ctx.moveTo(tankX, stepY);
        ctx.lineTo(tankX + 15, stepY);
        ctx.stroke();

        ctx.fillStyle = 'rgba(255, 255, 255, 0.8)';
        ctx.font = '8px monospace';
    }

```

```

    ctx.fillText(`${j * 100}L`, tankX + 18, stepY + 3);
  }

  // Tank Outline Border glow
  ctx.strokeStyle = '#94a3b8';
  ctx.lineWidth = 4;
  ctx.strokeRect(tankX, tankY, tankW, tankH);

  // Tube connection route to Cistern
  ctx.strokeStyle = '#64748b';
  ctx.lineWidth = 6;
  ctx.beginPath();
  ctx.moveTo(400, 310);
  ctx.lineTo(460, 310);
  ctx.stroke();

  // 8. Render Moving Water Particles inside pipes if Raining
  if (isRaining && Math.random() < 0.4 && clogLevel < 95) {
    // Spawn particle at left or right gutter depending on roof
    dimensions
    const side = Math.random() < 0.5 ? 'left' : 'right';
    const startX = side === 'left' ? roofLeftX : roofRightX;
    pipeParticles.push({
      x: startX,
      y: houseY + 6,
      stage: 0, // 0: vertical drop, 1: horizontal flow, 2: drop
      through filter, 3: feed to tank
      side: side
    });
  }

  // Update and draw pipe flow particles
  ctx.fillStyle = '#60a5fa';
  pipeParticles.forEach((p, idx) => {
    ctx.beginPath();
    ctx.arc(p.x, p.y, 3, 0, Math.PI * 2);
  });

```

```

ctx.fill();

// Particle routing state machine
const flowVelocity = 3.5 * ((100 - clogLevel) / 100);
if (p.stage === 0) {
  p.y += flowVelocity;
  if (p.y >= houseY + 140) {
    p.stage = 1;
  }
} else if (p.stage === 1) {
  if (p.side === 'left') {
    p.x += flowVelocity;
    if (p.x >= 390) p.stage = 2;
  } else {
    p.x -= flowVelocity;
    if (p.x <= 410) p.stage = 2;
  }
} else if (p.stage === 2) {
  p.x = 400; // Snap in center channel
  p.y += flowVelocity;
  if (p.y >= 310) {
    p.stage = 3;
  }
} else if (p.stage === 3) {
  p.x += flowVelocity;
  if (p.x >= tankX) {
    // Reached destination! Delete particle from pipeline array
    pipeParticles.splice(idx, 1);
  }
}
});

// Keep pipeline particles array constrained
if (pipeParticles.length > 50) pipeParticles.shift();

// 9. Draw Atmospheric Falling Raindrops

```

```

    if (isRaining) {
        ctx.strokeStyle = 'rgba(186, 230, 253, 0.6)';
        ctx.lineWidth = rainIntensity === 1 ? 1 : rainIntensity === 2 ? 2 :
3.5;

        drops.forEach(drop => {
            ctx.beginPath();
            ctx.moveTo(drop.x, drop.y);
            ctx.lineTo(drop.x - 1, drop.y + drop.len);
            ctx.stroke();

            // Particle downward fall logic
            drop.y += drop.speed * (rainIntensity * 0.7);
            drop.x -= 0.3; // subtle cross wind

            // Recycle drop once hitting soil or roof line
            if (drop.y > logicalHeight) {
                drop.y = Math.random() * -30;
                drop.x = Math.random() * logicalWidth;
            }

            // Collisions with active Roof catchment area to trigger flow
            if (drop.y >= houseY - 20 && drop.y <= houseY && drop.x >=
roofLeftX && drop.x <= roofRightX) {
                // Recycle immediately to act as catch absorption
                drop.y = Math.random() * -30;
                drop.x = Math.random() * logicalWidth;
            }
        });
    }

    animationRef.current = requestAnimationFrame(drawSimulation);
};

drawSimulation();

```

```

    return () => {
        cancelAnimationFrame(animationRef.current);
    };
}, [isRaining, roofSize, rainIntensity, waterVolume, clogLevel]);

// --- HARVESTING CALCULATOR CLOCK/TICK RUNNER ---
useEffect(() => {
    let tick = null;
    if (isRaining) {
        tick = setInterval(() => {
            // Harvesting yield mathematical formula:
            // Area (RoofSize) * RainIntensity * Constant Efficiency * Filter
            cleanliness (clog factor)

            const areaModifier = roofSize === 1 ? 1.5 : roofSize === 2 ? 3.2 :
5.8;

            const rainModifier = rainIntensity === 1 ? 1.2 : rainIntensity ===
2 ? 3.5 : 8.0;

            const cleanEfficiency = (100 - clogLevel) / 100;

            const addedLiters = areaModifier * rainModifier * cleanEfficiency *
0.25;

            setWaterVolume(prev => {
                const nextVolume = prev + addedLiters;
                const target = challenges[activeChallenge].targetLiters;

                // Check if Challenge criteria met inside run state
                if (challengeState === 'running') {
                    if (nextVolume >= target) {
                        setChallengeState('success');
                        setIsRaining(false);
                        setXp(curr => {
                            const updatedXp = curr + 100;
                            // Save updated parameters online! (RULE 3)
                            saveProgress(updatedXp, [...unlockedAchievements,
challenges[activeChallenge].title]);

```

```

        return updatedXp;
    });
    setUnlockedAchievements(prevAchievements => {
        if
(!prevAchievements.includes(challenges[activeChallenge].title)) {
            return [...prevAchievements,
challenges[activeChallenge].title];
        }
        return prevAchievements;
    });
    playSynthesizedChime(true);
    return target;
}
}

    if (nextVolume >= 500) {
        // Tank Overflow!
        return 500;
    }
    return nextVolume;
});
}, 100);
}

    return () => clearInterval(tick);
}, [isRaining, roofSize, rainIntensity, clogLevel, challengeState,
activeChallenge]);

// --- CHALLENGE TRIGGER SYSTEM ---
const launchChallenge = (index) => {
    handleReset();
    setActiveChallenge(index);
    setChallengeTimer(challenges[index].timeLimit);
    setChallengeState('running');
    setIsRaining(true);
    setClogLevel(0);

```

```

};

// --- QUIZ DATA BASE ---
const quizQuestions = [
  {
    question: "Which of the following describes the key purpose of
rainwater harvesting?",
    options: [
      "To increase total global cloud formation and rain days",
      "To gather and store rainfall run-off to conserve water reserves",
      "To cool down building roofs during intense summer heatwaves",
      "To construct decorative backyard outdoor fountains"
    ],
    correctIndex: 1,
    explanation: "Rainwater harvesting catches fresh rainwater on
collection surfaces (like roofs) and stores it for future dry spells,
stopping clean water from running down dirty drains!"
  },
  {
    question: "If we double the surface area (size) of our collection
roof, what happens to the water gathered?",
    options: [
      "The water volume decreases due to solar winds",
      "The water collected remains exactly identical",
      "The water volume increases substantially because of a larger
catchment area",
      "The water immediately turns into ice"
    ],
    correctIndex: 2,
    explanation: "A larger roof surface area (catchment area) acts like a
bigger net, intercepting more falling raindrops and channeling a higher
overall volume down the pipe system."
  },
  {
    question: "What is the primary role of the gravel, sand, and charcoal
filter in a harvesting system?",

```

```

    options: [
      "To heat the stored water for bathing",
      "To color the water and make it look clean",
      "To trap dirt, dried leaves, and dust before the water reaches the
storage cistern",
      "To speed up the wind flow under the roof structure"
    ],
    correctIndex: 3,
    explanation: "Roofs accumulate dirt, bird droppings, and leaves. The
filter screens out these solid contaminants, ensuring relatively clean
water enters the main storage reservoir."
  },
  {
    question: "Is rooftop-harvested rainwater immediately safe to drink
directly from the tank?",
    options: [
      "Yes, rainwater is always 100% pure under any circumstance",
      "No, it must be filtered and boiled or treated first to destroy
microscopic pathogens",
      "Only if collected during heavy lightning storms",
      "No, because rainwater has zero oxygen content"
    ],
    correctIndex: 1,
    explanation: "Even with physical sand/charcoal filters, fine dust or
bacteria from the air and roof surfaces may persist. Boiling or
purification is required for consumption."
  }
];

```

```

const handleQuizAnswer = (idx) => {
  setSelectedAnswer(idx);
  if (idx === quizQuestions[currentQuizIndex].correctIndex) {
    setQuizScore(s => s + 1);
    setXp(x => {
      const nextXp = x + 25;
      saveProgress(nextXp, unlockedAchievements);
    });
  }
};

```



```

        return nextXp;
    });
    playSynthesizedChime(true);
} else {
    playSynthesizedChime(false);
}
};

const nextQuizQuestion = () => {
    setSelectedAnswer(null);
    if (currentQuizIndex < quizQuestions.length - 1) {
        setCurrentQuizIndex(i => i + 1);
    } else {
        setQuizFinished(true);
    }
};

const restartQuiz = () => {
    setCurrentQuizIndex(0);
    setSelectedAnswer(null);
    setQuizScore(0);
    setQuizFinished(false);
};

return (
    <div className="w-full max-w-6xl mx-auto p-3 sm:p-6 bg-[#f8fafc]
text-slate-800 font-sans min-h-screen flex flex-col justify-between">

        {/* 1. TOP NAV BAR CONTROLLER */}
        <nav className="flex flex-col sm:flex-row items-center
justify-between bg-white border border-slate-100 rounded-2xl p-4 mb-5
shadow-sm gap-4">
            <div className="flex items-center gap-3">
                <div className="bg-indigo-50 p-2.5 rounded-xl border
border-indigo-100">
                    <Droplets className="w-7 h-7 text-indigo-600 animate-pulse" />

```

```

    </div>
    <div>
      <div className="flex items-center gap-2">
        <span className="bg-indigo-100 text-indigo-800 text-[10px]
font-bold px-2 py-0.5 rounded-full uppercase tracking-wider">NCERT Class
6</span>
        <span className="text-slate-400 text-xs">Chapter 14:
Water</span>
      </div>
      <h1 className="text-xl font-bold tracking-tight
text-slate-800">Rooftop Catchment Lab</h1>
    </div>
  </div>

  { /* Dynamic Telemetry stats */ }
  <div className="flex items-center gap-3 flex-wrap justify-center">
    <div className="bg-slate-50 border border-slate-200/60 rounded-xl
px-4 py-1.5 flex items-center gap-2 text-xs">
      <span className="w-2.5 h-2.5 rounded-full bg-emerald-500
animate-ping"></span>
      <span className="text-slate-500 font-medium">User:</span>
      <span className="font-bold text-slate-800 text-sm">{username ||
"Connecting..."</span>
    </div>

    <div className="bg-amber-50 border border-amber-200
text-amber-800 font-bold px-4 py-1.5 rounded-xl flex items-center gap-1.5
text-sm shadow-xs">
      <Award className="w-4 h-4 text-amber-500" />
      <span>{xp} XP</span>
    </div>

    <button
      onClick={() => setSoundEnabled(!soundEnabled)}

```

```

        className={`p-2 rounded-xl border transition-all ${soundEnabled
? 'bg-indigo-50 border-indigo-200 text-indigo-600' : 'bg-slate-50
border-slate-200 text-slate-400'}}`
        title="Toggle Synthesized Audio Feedback"
    >
        {soundEnabled ? <Volume2 className="w-4 h-4" /> : <VolumeX
className="w-4 h-4" />}
    </button>
</div>
</nav>

{/* 2. CORE WORKSPACE GRID */}
<div className="grid grid-cols-1 lg:grid-cols-12 gap-6 items-stretch
flex-1">

    {/* INTERACTIVE CANVAS PANEL (LHS 7 Columns) */}
    <div className="lg:col-span-7 flex flex-col bg-white rounded-2xl
border border-slate-200/80 overflow-hidden shadow-sm">

        {/* Header Controls Inside Simulator viewport */}
        <div className="bg-slate-900 px-4 py-3 text-white flex
items-center justify-between border-b border-slate-800">
            <div className="flex items-center gap-2">
                <span className="w-2 h-2 rounded-full bg-red-500
animate-pulse"></span>
                <span className="text-xs font-mono tracking-widest
text-slate-400 uppercase">Live Canvas Telemetry</span>
            </div>
            <div className="text-xs font-semibold text-sky-400
bg-sky-950/80 px-2.5 py-1 rounded-md border border-sky-900/60 flex
items-center gap-1.5">
                <Info className="w-3.5 h-3.5" />
                <span>Catchment efficiency: {Math.max(100 - clogLevel,
0)} %</span>
            </div>
        </div>
    </div>

```

```

    {/* Interactive Screen container */}
    <div className="relative bg-slate-800 flex justify-center
items-center w-full p-2">
      <canvas
        ref={canvasRef}
        className="w-full max-w-[600px] h-auto aspect-[600/380]
rounded-lg shadow-inner block"
      />

      {/* Float values on top of simulation for young minds */}
      {waterVolume >= 500 && (
        <div className="absolute top-4 left-4 bg-red-600/90
text-white font-bold text-[10px] uppercase tracking-wider px-2 py-1
rounded-md animate-bounce shadow-md">
          Tank Overflowing! 🌊
        </div>
      )}
    </div>

    {/* SIMULATOR KNOBS DRAWER */}
    <div className="p-4 bg-slate-50 border-t border-slate-200/80 grid
grid-cols-1 md:grid-cols-12 gap-4 items-center">

      {/* Control 1: Roof Size Selector */}
      <div className="md:col-span-4 flex flex-col gap-1.5">
        <div className="flex justify-between items-center">
          <span className="text-xs font-bold text-slate-500 uppercase
tracking-wide">1. Roof Area (Catchment)</span>
          <span className="text-xs font-bold text-indigo-600
bg-indigo-50 px-2 py-0.5 rounded">
            {roofSize === 1 ? '15m² (Small)' : roofSize === 2 ? '30m²
(Medium)' : '50m² (Large)'}
          </span>
        </div>
        <input

```

```

        type="range"
        min="1"
        max="3"
        step="1"
        value={roofSize}
        onChange={ (e) => setRoofSize(Number(e.target.value)) }
        disabled={isRaining}
        className="w-full accent-indigo-600 cursor-pointer
disabled:opacity-50"
    />
</div>

{/* Control 2: Precipitation Rate */}
<div className="md:col-span-4 flex flex-col gap-1.5">
    <div className="flex justify-between items-center">
        <span className="text-xs font-bold text-slate-500 uppercase
tracking-wide">2. Rain Intensity</span>
        <span className="text-xs font-bold text-blue-600 bg-blue-50
px-2 py-0.5 rounded">
            {rainIntensity === 1 ? 'Drizzle' : rainIntensity === 2 ?
'Moderate' : 'Heavy Storm'}
        </span>
    </div>
    <input
        type="range"
        min="1"
        max="3"
        step="1"
        value={rainIntensity}
        onChange={ (e) => setRainIntensity(Number(e.target.value)) }
        className="w-full accent-blue-600 cursor-pointer"
    />
</div>

{/* Control 3: Trigger actions */}
<div className="md:col-span-4 flex gap-2 pt-2 md:pt-0">

```

```

<button
  onClick={handleStartStop}
  disabled={waterVolume >= 500 && !isRaining}
  className={`flex-1 flex items-center justify-center gap-1.5
text-xs font-bold text-white px-3 py-2.5 rounded-xl shadow transition-all
active:scale-95 ${
  isRaining
    ? 'bg-amber-500 hover:bg-amber-600'
    : 'bg-indigo-600 hover:bg-indigo-700
disabled:bg-slate-300'
  }}`
>
  {isRaining ? (
    <>
      <Pause className="w-4 h-4 fill-current" /> Pause
Weather
    </>
  ) : (
    <>
      <Play className="w-4 h-4 fill-current" /> Trigger Rain
    </>
  )}
</button>

<button
  onClick={handleReset}
  className="bg-slate-200 hover:bg-slate-300 text-slate-700
p-2.5 rounded-xl transition-all active:scale-95"
  title="Reset simulation parameters"
>
  <RotateCcw className="w-4 h-4" />
</button>
</div>

{/* Special dynamic filter cleaning slider triggers for
Challenge 3 */}

```

```

        {challenges[activeChallenge].dynamicClogging && (
            <div className="col-span-12 mt-2 bg-amber-50 border
border-amber-200 rounded-xl p-3 flex flex-col sm:flex-row items-center
justify-between gap-3 animate-pulse">
                <div className="flex items-center gap-2">
                    <span className="font-bold text-xs text-amber-800">Filter
Status:</span>
                    <div className="w-28 bg-slate-200 h-2.5 rounded-full
overflow-hidden">
                        <div
                            className="bg-red-500 h-full transition-all
duration-300"
                            style={{ width: `${clogLevel}%` }}
                        />
                    </div>
                    <span className="text-xs font-mono font-bold
text-red-600">{clogLevel}% Clogged</span>
                </div>
                <button
                    onClick={() => setClogLevel(prev => Math.max(prev - 25,
0))}
                    className="bg-amber-600 hover:bg-amber-700 text-white
font-bold text-xs px-3 py-1.5 rounded-lg shadow-sm transition
active:scale-95"
                >
                    🧹 Clear Filter Clogs
                </button>
            </div>
        )}

        </div>

        { /* TELEMETRY READOUTS */ }
        <div className="bg-slate-900 text-slate-300 px-5 py-4 grid
grid-cols-2 sm:grid-cols-4 gap-4 border-t border-slate-800 text-center">
            <div className="border-r border-slate-800">

```

```

        <span className="text-[10px] font-bold tracking-wider
text-slate-500 block uppercase">Water Harvested</span>
        <span className="text-xl font-bold font-mono
text-white">{Math.round(waterVolume)} Liters</span>
    </div>

    <div className="sm:border-r border-slate-800">
        <span className="text-[10px] font-bold tracking-wider
text-slate-500 block uppercase">Cistern Space</span>
        <span className="text-xl font-bold font-mono
text-indigo-400">{500 - Math.round(waterVolume)} L Left</span>
    </div>

    <div className="border-r border-slate-800 pt-2 sm:pt-0">
        <span className="text-[10px] font-bold tracking-wider
text-slate-500 block uppercase">Catchment Size</span>
        <span className="text-xl font-bold font-mono
text-emerald-400">
            {roofSize === 1 ? '15 m²' : roofSize === 2 ? '30 m²' : '50
m²'}
        </span>
    </div>

    <div className="pt-2 sm:pt-0">
        <span className="text-[10px] font-bold tracking-wider
text-slate-500 block uppercase">Yield Multiplier</span>
        <span className="text-xl font-bold font-mono text-sky-400">
            {(roofSize * rainIntensity * 1.5).toFixed(1)}x
        </span>
    </div>
</div>

    { /* MODULAR CONTROL SIDEBAR DASHBOARD (RHS 5 Columns) */ }

    <div className="lg:col-span-5 flex flex-col bg-white rounded-2xl
border border-slate-200/80 overflow-hidden shadow-sm">

        { /* TAB SYSTEM BUTTONS */ }

```



```

    <div className="flex border-b border-slate-200 bg-slate-50/80
p-1.5 gap-1">
      <button
        onClick={() => setActiveTab('missions')}
        className={`flex-1 py-2 rounded-xl text-xs font-bold
transition-all flex items-center justify-center gap-1.5 ${
          activeTab === 'missions'
            ? 'bg-white text-indigo-600 shadow-xs border
border-slate-200'
            : 'text-slate-600 hover:bg-slate-100'
          }`}
      >
        <Compass className="w-4 h-4" /> Missions
      </button>
      <button
        onClick={() => setActiveTab('quiz')}
        className={`flex-1 py-2 rounded-xl text-xs font-bold
transition-all flex items-center justify-center gap-1.5 ${
          activeTab === 'quiz'
            ? 'bg-white text-indigo-600 shadow-xs border
border-slate-200'
            : 'text-slate-600 hover:bg-slate-100'
          }`}
      >
        <HelpCircle className="w-4 h-4" /> Lab Quiz
      </button>
      <button
        onClick={() => setActiveTab('diy')}
        className={`flex-1 py-2 rounded-xl text-xs font-bold
transition-all flex items-center justify-center gap-1.5 ${
          activeTab === 'diy'
            ? 'bg-white text-indigo-600 shadow-xs border
border-slate-200'
            : 'text-slate-600 hover:bg-slate-100'
          }`}
      >

```

```

        <BookOpen className="w-4 h-4" /> DIY Steps
    </button>
    <button
        onClick={() => setActiveTab('leaderboard')}
        className={`flex-1 py-2 rounded-xl text-xs font-bold
transition-all flex items-center justify-center gap-1.5 ${
            activeTab === 'leaderboard'
                ? 'bg-white text-indigo-600 shadow-xs border
border-slate-200'
                : 'text-slate-600 hover:bg-slate-100'
            }`}
    >
        <Users className="w-4 h-4" /> Leaders
    </button>
</div>

{/* ACTIVE TAB VIEWS */}
<div className="p-5 flex-1 flex flex-col justify-between
overflow-y-auto max-h-[500px]">

    {/* TAB A: THE MISSION TARGETS */}
    {activeTab === 'missions' && (
        <div className="space-y-4 h-full flex flex-col
justify-between">
            <div>
                <div className="flex items-center justify-between mb-2">
                    <h3 className="text-sm font-extrabold uppercase
tracking-wider text-slate-400">Rain Catching Missions</h3>
                    <span className="text-xs bg-indigo-50 text-indigo-600
font-semibold px-2 py-0.5 rounded-full">
                        Level 1
                    </span>
                </div>
                <p className="text-xs text-slate-500 mb-4
leading-relaxed">

```

Test your hypothesis using the interactive lab!
Complete each mission below to lock in XP credits.

</p>

```
<div className="space-y-3">
  {challenges.map((chal, index) => {
    const isUnlocked =
unlockedAchievements.includes(chal.title);
    const isActive = activeChallenge === index &&
challengeState === 'running';

    return (
      <div
        key={index}
        className={`p-3.5 rounded-xl border
transition-all ${
          isActive
            ? 'bg-indigo-50/60 border-indigo-300
shadow-xs ring-1 ring-indigo-300'
            : isUnlocked
            ? 'bg-emerald-50/40 border-emerald-100'
            : 'bg-slate-50 border-slate-200/60'
          }`}
      >
        <div className="flex items-start justify-between
gap-2">
          <div>
            <div className="flex items-center gap-1.5
mb-1">
              <span className="text-xs font-bold
text-slate-800">{chal.title}</span>
              {isUnlocked && <CheckCircle2 className="w-4
h-4 text-emerald-500 shrink-0" />}
            </div>
            <p className="text-[11px] text-slate-500
leading-relaxed">{chal.description}</p>
```

```

        </div>

        <button
            onClick={() => launchChallenge(index)}
            className={`shrink-0 text-xs font-bold px-3
py-1.5 rounded-lg shadow-xs transition-all active:scale-95 ${
                isUnlocked
                    ? 'bg-emerald-100 hover:bg-emerald-200
text-emerald-800'
                    : 'bg-indigo-600 hover:bg-indigo-700
text-white'
            }}
        >
            {isActive ? 'Retrying...' : isUnlocked ?
'Completed' : 'Launch'}
        </button>
    </div>

    { /* Render Active telemetry indicators inside
card */ }

    {isActive && (
        <div className="mt-3 pt-2.5 border-t
border-indigo-200/60 flex items-center justify-between text-xs
text-indigo-900 font-medium">
            <span className="flex items-center gap-1">
                <span className="inline-block w-2 h-2
rounded-full bg-red-500 animate-ping"></span>
                Time Left: <strong className="font-bold
text-indigo-700 text-sm">{challengeTimer}s</strong>
            </span>
            <span>Target: <strong className="font-bold
text-indigo-700">{chal.targetLiters}L</strong></span>
        </div>
    ) }
</div>
);

```

```

    }}}
  </div>
</div>

{/* Challenge Result Modals inside tab */}
{challengeState === 'success' && (
  <div className="mt-4 p-4 bg-emerald-50 border
border-emerald-200 rounded-xl text-center animate-bounce-short">
    <Sparkles className="w-8 h-8 text-amber-500 mx-auto
mb-1" />

    <h4 className="font-bold text-emerald-900
text-sm">Mission Success! 🎉</h4>
    <p className="text-[11px] text-emerald-700 mt-1">
      You successfully optimized your rainwater harvesting
catchment grid! +100 XP gained!
    </p>
  </div>
)}

{challengeState === 'fail' && (
  <div className="mt-4 p-4 bg-rose-50 border
border-rose-200 rounded-xl text-center animate-pulse">
    <XCircle className="w-8 h-8 text-rose-500 mx-auto mb-1"
/>

    <h4 className="font-bold text-rose-900 text-sm">Timer
Expired! ⌚</h4>
    <p className="text-[11px] text-rose-700 mt-1">
      Hint: {challenges[activeChallenge].hint} Try again by
hitting the launch button!
    </p>
  </div>
)}
</div>
)}

{/* TAB B: CONCEPT DIAGNOSTIC QUIZ */}

```

```

{activeTab === 'quiz' && (
  <div className="h-full flex flex-col justify-between">
    {!quizFinished ? (
      <div className="space-y-4">
        <div className="flex justify-between items-center
text-xs font-bold text-slate-400 uppercase tracking-wider">
          <span>Rooftop Concept Assessment</span>
          <span>{currentQuizIndex + 1} of
{quizQuestions.length}</span>
        </div>

        <div className="p-4 bg-slate-50 border
border-slate-200/60 rounded-xl">
          <p className="text-xs font-bold text-slate-700
leading-relaxed">
            {quizQuestions[currentQuizIndex].question}
          </p>
        </div>

        <div className="space-y-2">
          {quizQuestions[currentQuizIndex].options.map((opt,
oIdx) => {
            const isSelected = selectedAnswer === oIdx;
            const isCorrect = oIdx ===
quizQuestions[currentQuizIndex].correctIndex;
            let optionBg = 'bg-white hover:bg-slate-50
border-slate-200';

            if (selectedAnswer !== null) {
              if (isSelected) {
                optionBg = isCorrect ? 'bg-emerald-50
border-emerald-300 text-emerald-900' : 'bg-rose-50 border-rose-300
text-rose-900';
              } else if (isCorrect) {
                optionBg = 'bg-emerald-50 border-emerald-200
text-emerald-800';

```

```

        } else {
            optionBg = 'bg-slate-50/50 border-slate-100
text-slate-400 opacity-60';
        }
    }

    return (
        <button
            key={oIdx}
            onClick={() => selectedAnswer === null &&
handleQuizAnswer(oIdx)}

            disabled={selectedAnswer !== null}
            className={`w-full text-left p-3 rounded-xl
border text-xs font-medium transition-all flex items-start gap-2.5
${optionBg}`}

            >
                <span className="bg-slate-100 text-slate-500
px-1.5 py-0.5 rounded text-[10px] uppercase font-bold shrink-0">
                    {String.fromCharCode(65 + oIdx)}
                </span>
                <span>{opt}</span>
            </button>
        );
    ))}
</div>

{selectedAnswer !== null && (
    <div className="p-3.5 bg-indigo-50/60 border
border-indigo-100 rounded-xl text-xs leading-relaxed text-indigo-900
animate-fade-in">
        <div className="flex items-center gap-1.5 mb-1
text-indigo-950 font-bold">
            <Lightbulb className="w-4 h-4 text-amber-500" />
            <span>Scientific Explanation:</span>
        </div>
    )}

```

```

<p>{quizQuestions[currentQuizIndex].explanation}</p>

        <button
            onClick={nextQuizQuestion}
            className="mt-3 w-full bg-indigo-600
hover:bg-indigo-700 text-white font-bold py-2 rounded-lg flex items-center
justify-center gap-1 transition shadow-xs"
        >
            Continue <ArrowRight className="w-4 h-4" />
        </button>
    </div>
    ) : (
        <div className="text-center py-6 space-y-4">
            <Sparkles className="w-12 h-12 text-indigo-500 mx-auto
animate-spin" />
            <h3 className="text-lg font-bold
text-slate-800">Assessment Complete!</h3>
            <p className="text-xs text-slate-500">
                You scored <strong className="font-extrabold
text-indigo-600 text-sm">{quizScore} / {quizQuestions.length}</strong> on
the Rainwater Harvesting module assessment!
            </p>
            <div className="p-3 bg-emerald-50 border
border-emerald-100 rounded-xl text-xs text-emerald-800">
                Congratulations! You unlocked +{quizScore * 25} XP
credits towards your global badge tier.
            </div>
            <button
                onClick={restartQuiz}
                className="bg-slate-800 hover:bg-slate-900 text-white
font-bold text-xs px-5 py-2.5 rounded-xl transition"
            >
                Retry Assessment

```



```

        </button>
      </div>
    )}
  </div>
)}

{/* TAB C: DIY HOME ACTIVITY */}
{activeTab === 'diy' && (
  <div className="space-y-4 text-xs leading-relaxed
text-slate-600">
    <div>
      <h3 className="text-sm font-extrabold text-slate-800
mb-1">Make a Backyard Water Catcher</h3>
      <p className="text-slate-500">Test how catchment area
influences collection volumes at home!</p>
    </div>

    <div className="border border-slate-100 p-3 rounded-xl
bg-slate-50">
      <span className="font-bold text-slate-700 block mb-1">🔧
Required Supplies:</span>
      <ul className="list-disc pl-4 space-y-1 text-slate-500">
        <li>1 large shallow baking tray (representing your
Large Roof)</li>
        <li>1 small ceramic cup or drinking mug (representing a
Small Roof)</li>
        <li>A watering container or handheld garden hose</li>
        <li>2 identical plastic bottles to measure outcome
levels</li>
      </ul>
    </div>

    <div className="space-y-2">
      <div className="flex gap-2.5">
        <span className="bg-indigo-100 text-indigo-700 w-5 h-5
rounded-full flex items-center justify-center font-bold shrink-0">1</span>

```

```

        <p>Tilt the large tray slightly over the lip of your
first collection bottle to serve as a run-off gutter.</p>
    </div>
    <div className="flex gap-2.5">
        <span className="bg-indigo-100 text-indigo-700 w-5 h-5
rounded-full flex items-center justify-center font-bold shrink-0">2</span>
        <p>Position the second bottle directly beneath the
small cup opening.</p>
    </div>
    <div className="flex gap-2.5">
        <span className="bg-indigo-100 text-indigo-700 w-5 h-5
rounded-full flex items-center justify-center font-bold shrink-0">3</span>
        <p>Spray water evenly from above over both setups for
exactly 10 seconds.</p>
    </div>
</div>

<div className="bg-amber-50 border border-amber-200
rounded-xl p-3 text-amber-900 font-medium text-[11px]">
    <span className="font-bold block mb-0.5
text-amber-950">💡 Scientific Discovery:</span>
    The bottle under the large tray collects massive amounts
of water! Why? Because its larger surface area catches falling rain that
would otherwise fall uselessly on the floor!
</div>
</div>
)}

{/* TAB D: LIVE LEADERBOARD COMPONENT (COLLABORATIVE
PERSISTENCE) */}
{activeTab === 'leaderboard' && (
    <div className="space-y-4">
        <div>
            <h3 className="text-sm font-extrabold uppercase
tracking-wider text-slate-400 mb-1">Global Class Leaderboard</h3>
            <p className="text-xs text-slate-500 leading-relaxed">

```

See how your friends are doing! Collect XP by completing interactive simulation tasks and scoring in quizzes.

</p>

</div>

<div className="space-y-2">

{leaderboard.length === 0 ? (

<div className="p-6 text-center text-slate-400

text-xs">

Connecting with global classmates...

</div>

) : (

leaderboard.map((player, pIdx) => {

const isMe = player.userId === user?.uid;

return (

<div

key={player.userId || pIdx}

className={`flex items-center justify-between

p-2.5 rounded-xl border text-xs transition-all \${

isMe

? 'bg-amber-50 border-amber-200 font-bold'

: 'bg-white border-slate-100'

}`}

>

<div className="flex items-center gap-2">

<span className={`w-5 h-5 rounded-full flex items-center justify-center font-mono text-[10px] font-bold \${

pIdx === 0 ? 'bg-yellow-400 text-yellow-950'

:

pIdx === 1 ? 'bg-slate-300 text-slate-900' :

pIdx === 2 ? 'bg-amber-600 text-white' :

'bg-slate-100 text-slate-600'

}`}>

{pIdx + 1}


```

        <span className={isMe ? 'text-slate-800' :
'text-slate-600'}>
            {player.username} {isMe && '(You)'}
        </span>
    </div>
    <span className="font-bold
text-indigo-600">{player.xp} XP</span>
    </div>
    );
    })
    </div>
</div>
    })

</div>

</div>

</div>

    { /* 3. SCIENTIFIC GLOSSARY MEMORY TRICK OVERLAY */ }
    <footer className="mt-5 bg-white border border-slate-100 rounded-2xl
p-4 flex flex-col md:flex-row justify-between items-center gap-4 text-xs">
        <div className="flex items-start gap-3">
            <span className="bg-sky-100 text-sky-700 p-2 rounded-lg font-bold
uppercase shrink-0">Memory Trick 🧠</span>
            <div>
                <strong className="block text-slate-800 text-xs mb-0.5">The
C.P.F.S. Pathway:</strong>
                <span className="text-slate-500">
                    <strong>C</strong>atchment (The roof collects) $\rightarrow$
<strong>P</strong>ipelines (Channels run-off) $\rightarrow$
<strong>F</strong>iltration (Cleans debris) $\rightarrow$
<strong>S</strong>torage (Holds reservoir).
                </span>
            </div>
        </div>
    </div>

```

```
        </div>
    </div>

    <div className="flex gap-2 flex-wrap justify-center text-[11px]
font-semibold text-slate-500">
        <span className="bg-slate-100 px-2.5 py-1
rounded-md">#WaterConservation</span>
        <span className="bg-slate-100 px-2.5 py-1
rounded-md">#RooftopHarvesting</span>
        <span className="bg-slate-100 px-2.5 py-1
rounded-md">#HydrologicCycle</span>
    </div>
</footer>

</div>
);
}
```

Moon

```

import React, { useState, useEffect, useRef } from 'react';

export default function App() {
  // --- STATE MANAGEMENT ---

  const [angle, setAngle] = useState(270); // Default to First Quarter for
  a beautiful initial look

  const [isPlaying, setIsPlaying] = useState(false);
  const [showRays, setShowRays] = useState(true);
  const [showOrbitPath, setShowOrbitPath] = useState(true);
  const [xp, setXp] = useState(0);
  const [activeTab, setActiveTab] = useState('simulator'); // 'simulator' |
  'quiz' | 'diy' | 'facts'
  const [currentQuest, setCurrentQuest] = useState(0);
  const [unlockedQuests, setUnlockedQuests] = useState([false, false,
  false, false]);
  const [showCelebration, setShowCelebration] = useState(false);
  const [celebrationMessage, setCelebrationMessage] = useState('');

  // Quiz State
  const [currentQuestion, setCurrentQuestion] = useState(0);
  const [selectedAnswer, setSelectedAnswer] = useState(null);
  const [isSubmitted, setIsSubmitted] = useState(false);
  const [quizScore, setQuizScore] = useState(0);
  const [showQuizResults, setShowQuizResults] = useState(false);

  // DIY Checklist State
  const [diySteps, setDiySteps] = useState([
    { id: 1, text: "Make your bedroom completely dark (turn off lights,
  close curtains).", checked: false },
    { id: 2, text: "Place a bright flashlight or desk lamp on a table to
  represent the Sun.", checked: false },
    { id: 3, text: "Hold a tennis ball or small orange straight out at
  arm's length (this is the Moon!).", checked: false },
    { id: 4, text: "Stand in place (your head is Planet Earth) and slowly
  spin in a circle.", checked: false },
    { id: 5, text: "Observe the changing shadow patterns on the ball as you
  spin!", checked: false }
  ])

```

```

]);

const svgRef = useRef(null);
const animationRef = useRef(null);

// --- QUESTS DEFINITIONS ---
const quests = [
  {
    id: 0,
    title: "Find the New Moon!",
    description: "Position the Moon directly between the Sun and the
Earth. (Hint: Around 180°)",
    targetRange: [175, 185],
    successMsg: "Fantastic! You found the New Moon. The dark side is
facing us, making it invisible! 🌑 +50 XP"
  },
  {
    id: 1,
    title: "Find the First Quarter Moon!",
    description: "Move the Moon until the right half is fully lit up from
Earth's view. (Hint: Around 270°)",
    targetRange: [265, 275],
    successMsg: "Amazing! First Quarter discovered. It looks like a
perfect half-slice! 🌔 +50 XP"
  },
  {
    id: 2,
    title: "Find the Full Moon!",
    description: "Position the Earth directly between the Sun and the
Moon. (Hint: Around 0° or 360°)",
    targetRange: [-5, 5], // or [355, 365] handled in check
    successMsg: "Brilliant! You created a Full Moon. The entire face
looking at us is glowing bright! 🌕 +50 XP"
  },
  {
    id: 3,

```



```

    title: "Find the Third Quarter Moon!",
    description: "Move the Moon until the left half is fully lit up from
Earth's view. (Hint: Around 90°)",
    targetRange: [85, 95],
    successMsg: "Superb! Third Quarter found. The Moon is now
three-quarters of the way around its orbit! 🌖 +50 XP"
  }
];

// --- QUIZ QUESTIONS ---
const quizQuestions = [
  {
    question: "Where does the Moon get its bright, glowing light from?",
    options: [
      { label: "A", text: "It glows by itself like a giant space
lightbulb." },
      { label: "B", text: "It reflects intense rays of light from the
Sun." },
      { label: "C", text: "It collects stardust that shines in the dark."
},
      { label: "D", text: "It reflects light glowing from planet Earth."
}
    ],
    correctAnswer: 1,
    explanation: "The Moon has no light of its own! It acts like a giant
mirror reflecting the Sun's light down to Earth."
  },
  {
    question: "How long does it take for the Moon to orbit the Earth
once?",
    options: [
      { label: "A", text: "Exactly 24 hours (1 day)" },
      { label: "B", text: "About 365 days (1 year)" },
      { label: "C", text: "About 29 days (1 month)" },
      { label: "D", text: "Exactly 7 days (1 week)" }
    ],
  },
];

```

```
    correctAnswer: 2,
    explanation: "The Moon takes roughly 29.5 days to complete its orbit
around Earth. This is where our concept of a 'month' comes from!"
  },
  {
    question: "What percentage of the Moon is ALWAYS lit up by the Sun in
space?",
    options: [
      { label: "A", text: "Exactly 50% (one half)" },
      { label: "B", text: "100% (the whole Moon)" },
      { label: "C", text: "It changes from 0% to 100% every day" },
      { label: "D", text: "Only 10% (the outer crescent)" }
    ],
    correctAnswer: 0,
    explanation: "No matter where the Moon is in its orbit, the Sun
always lights up exactly half of it! We only see different shapes because
of our changing viewpoint from Earth."
  },
  {
    question: "If you see a Full Moon tonight, what phase will you see in
about two weeks?",
    options: [
      { label: "A", text: "Another Full Moon" },
      { label: "B", text: "A New Moon (completely dark)" },
      { label: "C", text: "A Half Moon" },
      { label: "D", text: "The Moon will disappear forever" }
    ],
    correctAnswer: 1,
    explanation: "A full cycle takes 4 weeks. Halfway through the cycle
(2 weeks), the Moon will be on the opposite side of its orbit, showing us
a New Moon."
  },
  {
    question: "What is the primary cause of the regular Moon phases we
see?",
    options: [
```

```

        { label: "A", text: "The shadow of Earth blocking the Sun's light."
    },
    { label: "B", text: "Clouds passing over the Moon in our
atmosphere." },
    { label: "C", text: "The Moon rotating faster than the Earth." },
    { label: "D", text: "The Moon revolving around Earth, changing our
view of its lit side." }
    ],
    correctAnswer: 3,
    explanation: "Earth's shadow only blocks the Moon during rare lunar
eclipses. The normal nightly phases happen purely because of the Moon's
revolution around us!"
}
];

// --- AUTOMATIC ORBIT ANIMATION ---
useEffect(() => {
    if (isPlaying) {
        const updateFrame = () => {
            setAngle((prev) => (prev + 0.5) % 360);
            animationRef.current = requestAnimationFrame(updateFrame);
        };
        animationRef.current = requestAnimationFrame(updateFrame);
    } else {
        if (animationRef.current) {
            cancelAnimationFrame(animationRef.current);
        }
    }
    return () => {
        if (animationRef.current) cancelAnimationFrame(animationRef.current);
    };
}, [isPlaying]);

// --- CHECK QUEST RESOLUTION ---
useEffect(() => {
    const currentQuestData = quests[currentQuest];

```

```

    if (!currentQuestData) return;

    const [min, max] = currentQuestData.targetRange;
    let isMatch = false;

    if (currentQuest === 2) {
        // Full Moon handles cross-boundary at 0/360
        isMatch = angle <= 5 || angle >= 355;
    } else {
        isMatch = angle >= min && angle <= max;
    }

    if (isMatch && !unlockedQuests[currentQuest]) {
        const updated = [...unlockedQuests];
        updated[currentQuest] = true;
        setUnlockedQuests(updated);
        setXp((prev) => prev + 50);
        setCelebrationMessage(currentQuestData.successMsg);
        setShowCelebration(true);
        setIsPlaying(false); // Pause so they can look
    }
}, [angle, currentQuest]);

// --- DRAG GESTURE LOGIC FOR THE MOON ---
const handleDrag = (e) => {
    if (!svgRef.current) return;
    const rect = svgRef.current.getBoundingClientRect();

    // Support touch and mouse pointer
    const clientX = e.clientX || (e.touches && e.touches[0].clientX);
    const clientY = e.clientY || (e.touches && e.touches[0].clientY);

    if (!clientX || !clientY) return;

    const centerX = rect.left + rect.width / 2;
    const centerY = rect.top + rect.height / 2;

```

```

const dx = clientX - centerX;
const dy = clientY - centerY;

let targetAngle = Math.atan2(dy, dx) * (180 / Math.PI);
if (targetAngle < 0) {
  targetAngle += 360;
}
setAngle(Math.round(targetAngle));
};

const handleTouchMove = (e) => {
  handleDrag(e);
};

// --- COMPUTE THE DETAILS FOR DISPLAY ---
const getPhaseDetails = (deg) => {
  // 0 deg: Opposite to Sun (Full Moon)
  // 90 deg: Bottom/Top depending on direction.
  // Let's standardise view:
  // Sun is Left (180 deg)
  // Earth is Center (0, 0)
  // Moon revolves counter-clockwise.
  if (deg >= 355 || deg <= 5) {
    return { name: "Full Moon", desc: "100% Illuminated. Earth is in the
middle of the cosmic sandwich!", icon: "🌕", progress: "100%" };
  } else if (deg > 5 && deg < 85) {
    return { name: "Waning Gibbous", desc: "Shrinking light! More than
half of the Moon is still lit.", icon: "🌔", progress: "75%" };
  } else if (deg >= 85 && deg <= 95) {
    return { name: "Third Quarter", desc: "Exactly half lit. The left
side is reflecting the Sun's light.", icon: "🌑", progress: "50%" };
  } else if (deg > 95 && deg < 175) {
    return { name: "Waning Crescent", desc: "A fading banana smile.
Almost completely dark.", icon: "🌒", progress: "25%" };
  } else if (deg >= 175 && deg <= 185) {

```

```

    return { name: "New Moon", desc: "No Moon visible! The bright side is
facing away from Earth.", icon: "🌑", progress: "0%" };
  } else if (deg > 185 && deg < 265) {
    return { name: "Waxing Crescent", desc: "The light is growing! Look
at that tiny sliver on the right.", icon: "🌒", progress: "25%" };
  } else if (deg >= 265 && deg <= 275) {
    return { name: "First Quarter", desc: "Exactly half lit. The right
side is catching the light.", icon: "🌓", progress: "50%" };
  } else {
    return { name: "Waxing Gibbous", desc: "Almost full! The light is
swelling up night after night.", icon: "🌔", progress: "75%" };
  }
};

```

```

const currentPhase = getPhaseDetails(angle);

```

```

// --- DYNAMIC MOON PHASE SVG PATH GENERATOR ---

```

```

// Renders Earth's view dynamically using an overlay technique

```

```

const renderEarthViewMoon = () => {

```

```

  const r = 65; // Radius

```

```

  const rad = (angle * Math.PI) / 180;

```

```

  // We determine the illuminated shape inside a 130x130 box centered at
  (65, 65)

```

```

  // Let's compute x-radius of the transition ellipse

```

```

  const cosVal = Math.cos(rad);

```

```

  const absCos = Math.abs(cosVal);

```

```

  const rx = r * absCos;

```

```

  // Define colors

```

```

  const darkColor = "#1e293b"; // slate-800

```

```

  const lightColor = "#fef08a"; // yellow-200 (glowing)

```

```

  // Quadrant logic for overlaying shapes:

```

```

  if (angle >= 0 && angle < 90) {

```

```

    // Waning Gibbous to Third Quarter

```

```

    // Left half lit (lightColor), Right half dark (darkColor), Middle
    ellipse lit (lightColor)
    return (
        <g>
            <path d={`M 65,${65 - r} A ${r},${r} 0 0,0 65,${65 + r} Z`}
            fill={lightColor} />
            <path d={`M 65,${65 - r} A ${r},${r} 0 0,1 65,${65 + r} Z`}
            fill={darkColor} />
            <ellipse cx="65" cy="65" rx={rx} ry={r} fill={lightColor} />
        </g>
    );
} else if (angle >= 90 && angle < 180) {
    // Third Quarter to New Moon (Waning Crescent)
    // Left half lit, Right half dark, Middle ellipse is dark
    return (
        <g>
            <path d={`M 65,${65 - r} A ${r},${r} 0 0,0 65,${65 + r} Z`}
            fill={lightColor} />
            <path d={`M 65,${65 - r} A ${r},${r} 0 0,1 65,${65 + r} Z`}
            fill={darkColor} />
            <ellipse cx="65" cy="65" rx={rx} ry={r} fill={darkColor} />
        </g>
    );
} else if (angle >= 180 && angle < 270) {
    // New Moon to First Quarter (Waxing Crescent)
    // Left half dark, Right half lit, Middle ellipse is dark
    return (
        <g>
            <path d={`M 65,${65 - r} A ${r},${r} 0 0,0 65,${65 + r} Z`}
            fill={darkColor} />
            <path d={`M 65,${65 - r} A ${r},${r} 0 0,1 65,${65 + r} Z`}
            fill={lightColor} />
            <ellipse cx="65" cy="65" rx={rx} ry={r} fill={darkColor} />
        </g>
    );
} else {

```

```

    // First Quarter to Full Moon (Waxing Gibbous)
    // Left half dark, Right half lit, Middle ellipse is lit
    return (
      <g>
        <path d={`M 65,${65 - r} A ${r},${r} 0 0,0 65,${65 + r} Z`}
        fill={darkColor} />
        <path d={`M 65,${65 - r} A ${r},${r} 0 0,1 65,${65 + r} Z`}
        fill={lightColor} />
        <ellipse cx="65" cy="65" rx={rx} ry={r} fill={lightColor} />
      </g>
    );
  }
};

// --- TRIGNOMETRIC POSITIONS FOR ORBIT MAP ---
const orbitRadius = 100;
const mapCenterX = 180;
const mapCenterY = 160;
const radAngle = (angle * Math.PI) / 180;
const moonX = mapCenterX + orbitRadius * Math.cos(radAngle);
const moonY = mapCenterY + orbitRadius * Math.sin(radAngle);

// Helper to trigger specific phase angles
const jumpToPhase = (targetAngle) => {
  setAngle(targetAngle);
  setIsPlaying(false);
};

// Reset Everything
const handleReset = () => {
  setAngle(270);
  setIsPlaying(false);
  setXp(0);
  setCurrentQuest(0);
  setUnlockedQuests([false, false, false, false]);
  setShowCelebration(false);
};

```



```

// Reset Quiz
setCurrentQuestion(0);
setSelectedAnswer(null);
setIsSubmitted(false);
setQuizScore(0);
setShowQuizResults(false);
// Reset DIY steps
setDiySteps(diySteps.map(s => ({ ...s, checked: false })));
};

// Toggle DIY step
const handleDiyCheck = (id) => {
  const updated = diySteps.map((step) => {
    if (step.id === id) {
      const nextVal = !step.checked;
      if (nextVal) setXp((prev) => prev + 10);
      return { ...step, checked: nextVal };
    }
    return step;
  });
  setDiySteps(updated);
};

// Handle Quiz Submission
const handleQuizAnswer = (index) => {
  if (isSubmitted) return;
  setSelectedAnswer(index);
};

const submitAnswer = () => {
  if (selectedAnswer === null || isSubmitted) return;
  setIsSubmitted(true);
  if (selectedAnswer === quizQuestions[currentQuestion].correctAnswer) {
    setQuizScore((prev) => prev + 1);
    setXp((prev) => prev + 30);
  }
}

```

```
};
```

```
const nextQuestion = () => {  
  setSelectedAnswer(null);  
  setIsSubmitted(false);  
  if (currentQuestion < quizQuestions.length - 1) {  
    setCurrentQuestion((prev) => prev + 1);  
  } else {  
    setShowQuizResults(true);  
  }  
};
```

```
const retakeQuiz = () => {  
  setCurrentQuestion(0);  
  setSelectedAnswer(null);  
  setIsSubmitted(false);  
  setQuizScore(0);  
  setShowQuizResults(false);  
};
```

```
return (  
  <div className="min-h-screen bg-slate-950 text-white font-sans  
antialiased selection:bg-indigo-500 selection:text-white">  
    { /* --- PREMIUM TOP HEADER --- */ }  
    <header className="sticky top-0 z-50 bg-slate-900/90 backdrop-blur  
border-b border-slate-800 px-4 py-3">  
      <div className="max-w-7xl mx-auto flex flex-col md:flex-row  
items-center justify-between gap-3">  
        <div className="flex items-center gap-3">  
          <span className="text-3xl">🌙</span>  
          <div>  
            <h1 className="text-xl md:text-2xl font-black  
bg-gradient-to-r from-amber-200 via-yellow-200 to-indigo-300 bg-clip-text  
text-transparent">  
              Cosmic Moon Lab  
            </h1>
```

```

        <p className="text-xs text-slate-400">Class 5 Science • NCERT
Curriculum Guide</p>
    </div>
</div>

```

```

    { /* Core Gamification Stats */ }
    <div className="flex items-center gap-4">
        <div className="bg-slate-950 px-4 py-1.5 rounded-full border
border-slate-800 flex items-center gap-2 shadow-inner">
            <span className="text-yellow-400 font-bold">★ {xp}</span>
            <span className="text-xs uppercase text-slate-500
font-semibold tracking-wider">XP</span>
        </div>

```

```

        <button
            onClick={handleReset}
            className="px-3.5 py-1.5 rounded-lg bg-slate-800
hover:bg-slate-700 text-xs font-semibold tracking-wide transition border
border-slate-700 hover:border-slate-600 flex items-center gap-1.5
active:scale-95"
        >
             Reset Lab
        </button>
    </div>
</div>
</header>

```

```

    { /* --- MAIN LAYOUT --- */ }
    <main className="max-w-7xl mx-auto px-4 py-6">

        { /* Navigation Tabs */ }
        <div className="flex overflow-x-auto gap-2 p-1 bg-slate-900
rounded-xl mb-6 border border-slate-800 max-w-2xl">
            <button
                onClick={() => setActiveTab('simulator')}

```

```

        className={`flex-1 min-w-[120px] py-2 px-3 rounded-lg text-sm
font-semibold transition duration-200 flex items-center justify-center
gap-2 ${
    activeTab === 'simulator'
      ? 'bg-gradient-to-r from-indigo-600 to-violet-600
text-white shadow-lg'
      : 'text-slate-400 hover:text-white hover:bg-slate-800'
    }`}
      >
        🚀 Space Simulator
    </button>
    <button
      onClick={() => setActiveTab('diy')}
      className={`flex-1 min-w-[120px] py-2 px-3 rounded-lg text-sm
font-semibold transition duration-200 flex items-center justify-center
gap-2 ${
    activeTab === 'diy'
      ? 'bg-gradient-to-r from-indigo-600 to-violet-600
text-white shadow-lg'
      : 'text-slate-400 hover:text-white hover:bg-slate-800'
    }`}
      >
        🏠 Bedroom Lab
    </button>
    <button
      onClick={() => setActiveTab('quiz')}
      className={`flex-1 min-w-[120px] py-2 px-3 rounded-lg text-sm
font-semibold transition duration-200 flex items-center justify-center
gap-2 ${
    activeTab === 'quiz'
      ? 'bg-gradient-to-r from-indigo-600 to-violet-600
text-white shadow-lg'
      : 'text-slate-400 hover:text-white hover:bg-slate-800'
    }`}
      >
        🧠 Space Quiz

```

```

</button>
<button
  onClick={() => setActiveTab('facts')}
  className={`flex-1 min-w-[120px] py-2 px-3 rounded-lg text-sm
font-semibold transition duration-200 flex items-center justify-center
gap-2 ${
    activeTab === 'facts'
      ? 'bg-gradient-to-r from-indigo-600 to-violet-600
text-white shadow-lg'
      : 'text-slate-400 hover:text-white hover:bg-slate-800'
  }}
>
  💡 Cosmic Facts
</button>
</div>

```

```

{/* --- QUEST SYSTEM ALERT CARD --- */}
{activeTab === 'simulator' && (
  <div className="bg-gradient-to-r from-indigo-950 via-slate-900
to-indigo-950 p-4 rounded-xl border border-indigo-500/40 shadow-xl mb-6
relative overflow-hidden">
    <div className="absolute right-0 top-0 text-7xl opacity-10
pointer-events-none">🚀</div>
    <div className="flex flex-col md:flex-row items-start
md:items-center justify-between gap-4">
      <div>
        <span className="text-xs uppercase font-extrabold
text-indigo-400 tracking-widest px-2 py-0.5 bg-indigo-900/60 rounded
border border-indigo-700/50">
          Active Space Quest
        </span>
        <h3 className="text-lg font-bold text-slate-100 mt-2">
          {quests[currentQuest]?.title}
        </h3>
        <p className="text-sm text-indigo-200 mt-1">
          {quests[currentQuest]?.description}

```

```

    </p>
  </div>
  <div className="flex items-center gap-2">
    {quests.map((q, idx) => (
      <button
        key={q.id}
        onClick={() => setCurrentQuest(idx)}
        className={`w-9 h-9 rounded-full flex items-center
justify-center font-bold text-sm transition-all border ${
          currentQuest === idx
            ? 'bg-indigo-600 border-indigo-400 text-white
scale-110 shadow-lg ring-2 ring-indigo-500/50'
            : unlockedQuests[idx]
              ? 'bg-emerald-950/80 border-emerald-500
text-emerald-400'
              : 'bg-slate-800 border-slate-700 text-slate-400
hover:bg-slate-700'
            }`}
      >
        {unlockedQuests[idx] ? '✓' : idx + 1}
      </button>
    ))}
  </div>
</div>
)}

{/* --- SIMULATOR VIEW TAB --- */}
{activeTab === 'simulator' && (
  <div className="grid grid-cols-1 lg:grid-cols-12 gap-6">

    {/* Left Side: Top-Down Orbit Map (Canvas Grid Col 7) */}
    <div className="lg:col-span-7 bg-slate-900 p-4 md:p-6
rounded-2xl border border-slate-800 shadow-xl flex flex-col
justify-between">
      <div>

```

```

<div className="flex justify-between items-center mb-3">
  <h2 className="text-lg font-bold text-slate-200 flex
items-center gap-2">
    🌌 Cosmic Orbit Tracker <span className="text-xs
text-slate-400">(Top-down view)</span>
  </h2>
  <div className="flex gap-2">
    <button
      onClick={() => setShowRays(!showRays)}
      className={`text-xs px-2.5 py-1 rounded transition
border ${
        showRays
          ? 'bg-amber-950/40 border-amber-600
text-amber-300'
          : 'bg-slate-800 border-slate-700 text-slate-400'
        }`}
    >
      💡 Rays {showRays ? 'ON' : 'OFF'}
    </button>
    <button
      onClick={() => setShowOrbitPath(!showOrbitPath)}
      className={`text-xs px-2.5 py-1 rounded transition
border ${
        showOrbitPath
          ? 'bg-indigo-950/40 border-indigo-600
text-indigo-300'
          : 'bg-slate-800 border-slate-700 text-slate-400'
        }`}
    >
      🌑 Path {showOrbitPath ? 'ON' : 'OFF'}
    </button>
  </div>
</div>
<p className="text-xs text-slate-400 mb-4">
  Drag the Moon along its path, or adjust the slide
controls to shift its position relative to Earth and Sun.

```

```
</p>
</div>
```

```
    { /* Main SVG Coordinate Canvas */ }
    <div className="relative aspect-video max-h-[360px]
md:max-h-none flex justify-center items-center bg-slate-950 rounded-xl
overflow-hidden border border-slate-800/80 my-4 select-none">

        { /* Sun Glow Gradient */ }
        <div className="absolute left-0 top-0 bottom-0 w-24
bg-gradient-to-r from-yellow-500/10 to-transparent pointer-events-none" />

        { /* Drag Guidance Prompt Overlay */ }
        <span className="absolute bottom-3 right-3 text-[10px]
bg-slate-900/80 px-2 py-1 rounded border border-slate-800 text-slate-500">
            🕒 Drag the Moon directly in orbit!
        </span>

        <svg
            ref={svgRef}
            viewBox="0 0 360 300"
            className="w-full h-full"
            onMouseMove={(e) => e.buttons === 1 && handleDrag(e)}
            onTouchMove={handleTouchMove}
        >
            { /* Sun Rays Rendering */ }
            { showRays && (
                <g opacity="0.35">
                    <line x1="0" y1="60" x2="160" y2="140"
stroke="#fef08a" strokeWidth="1" strokeDasharray="4,4" />
                    <line x1="0" y1="120" x2="160" y2="150"
stroke="#fef08a" strokeWidth="1" strokeDasharray="4,4" />
                    <line x1="0" y1="180" x2="160" y2="170"
stroke="#fef08a" strokeWidth="1" strokeDasharray="4,4" />
                    <line x1="0" y1="240" x2="160" y2="180"
stroke="#fef08a" strokeWidth="1" strokeDasharray="4,4" />
```



```

        { /* Rays striking moon */ }
        <line x1="0" y1={moonY} x2={moonX} y2={moonY}
stroke="#fef08a" strokeWidth="1.5" strokeDasharray="2,2" />
    </g>
    ) }

    { /* Left Side: Sun Illustration */ }
    <g transform="translate(-10, 160)">
        <circle cx="0" cy="0" r="45" fill="url(#sunGlow)" />
        <circle cx="0" cy="0" r="30" fill="#fef08a" />
        <text x="32" y="5" fill="#fef08a" fontSize="10"
fontWeight="bold">SUN</text>
    </g>

    { /* Circular Orbit Path */ }
    { showOrbitPath && (
        <circle
            cx={mapCenterX}
            cy={mapCenterY}
            r={orbitRadius}
            fill="none"
            stroke="#475569"
            strokeWidth="1.5"
            strokeDasharray="6,4"
            opacity="0.6"
        />
    ) }

    { /* Center: Planet Earth */ }
    <g transform={`translate(${mapCenterX}, ${mapCenterY})`} >
        <circle cx="0" cy="0" r="28" fill="#1e3a8a" />
        { /* Simplified Earth continent curves */ }
        <path d="M -15,-10 Q -5,-15 5,-5 T -10,15"
fill="#15803d" opacity="0.85" />
        <path d="M 5,5 Q 15,2 20,-10 T 10,10" fill="#15803d"
opacity="0.85" />
    </g>

```

```

        <circle cx="0" cy="0" r="28" fill="url(#earthShadow)"
/>

        <circle cx="0" cy="0" r="29" fill="none"
stroke="#3b82f6" strokeWidth="1" opacity="0.5" />
        <text x="-16" y="4" fill="ffffff" fontSize="9"
fontWeight="bold">EARTH</text>
    </g>

    { /* The Interactive Moon */ }
    <g
        transform={`translate(${moonX}, ${moonY})`}
        className="cursor-grab active:cursor-grabbing"
        onMouseDown={ (e) => handleDrag(e) }
        onTouchStart={ (e) => handleDrag(e) }
    >
        <circle cx="0" cy="0" r="16" fill="#475569" />
        { /* Lit Side facing the Sun (always facing Left toward
X=0) */ }

        <path d="M 0,-16 A 16,16 0 0,0 0,16 Z" fill="#fef08a"
/>

        { /* Small visual Moon craters */ }
        <circle cx="-6" cy="-4" r="2" fill="#e2e8f0"
opacity="0.6" />
        <circle cx="6" cy="4" r="3" fill="#334155"
opacity="0.7" />
        <circle cx="2" cy="-7" r="1.5" fill="#334155"
opacity="0.7" />

        { /* Glowing ring */ }
        <circle cx="0" cy="0" r="17" fill="none"
stroke="ffffff" strokeWidth="1" opacity="0.4" />

        { /* Dynamic helper labels */ }
        <g transform="translate(0, -22)">

```

```

        <rect x="-24" y="-8" width="48" height="13" rx="3"
fill="#0f172a" opacity="0.85" />
        <text x="0" y="1" fill="#e2e8f0" fontSize="7"
textAnchor="middle" fontWeight="semibold">
            MOON
        </text>
    </g>
</g>

{ /* SVG Definitions */}
<defs>
    <radialGradient id="sunGlow" cx="0%" cy="50%" r="100%">
        <stop offset="0%" stopColor="#fef08a" stopOpacity="1"
/>
        <stop offset="50%" stopColor="#f59e0b"
stopOpacity="0.5" />
        <stop offset="100%" stopColor="#f59e0b"
stopOpacity="0" />
    </radialGradient>
    <linearGradient id="earthShadow" x1="0%" y1="0%"
x2="100%" y2="0%">
        { /* Sun is on Left, so Left side of Earth is lit,
Right side is in shadow */}
        <stop offset="0%" stopColor="#ffffff" stopOpacity="0"
/>
        <stop offset="50%" stopColor="#000000"
stopOpacity="0.4" />
        <stop offset="100%" stopColor="#000000"
stopOpacity="0.95" />
    </linearGradient>
</defs>
</svg>
</div>

{ /* Slider & Primary Play Controls */}

```

```

<div className="bg-slate-950 p-4 rounded-xl border
border-slate-800/80">
  <div className="flex flex-col sm:flex-row justify-between
items-center gap-3 mb-3">
    <div className="flex items-center gap-2">
      <span className="text-xs font-bold
text-slate-400">Position Angle:</span>
      <span className="text-sm font-mono font-bold
text-yellow-400">{angle}°</span>
    </div>

    {/* Preset Jumper Buttons for Easy Phase Navigation */}
    <div className="flex flex-wrap gap-1.5 justify-center">
      <button onClick={() => jumpToPhase(180)}
className="px-2 py-0.5 rounded bg-slate-900 text-[10px] text-slate-300
hover:bg-slate-800 border border-slate-850">New ●</button>
      <button onClick={() => jumpToPhase(270)}
className="px-2 py-0.5 rounded bg-slate-900 text-[10px] text-slate-300
hover:bg-slate-800 border border-slate-850">1st Qtr ●</button>
      <button onClick={() => jumpToPhase(0)} className="px-2
py-0.5 rounded bg-slate-900 text-[10px] text-slate-300 hover:bg-slate-800
border border-slate-850">Full 🕒</button>
      <button onClick={() => jumpToPhase(90)} className="px-2
py-0.5 rounded bg-slate-900 text-[10px] text-slate-300 hover:bg-slate-800
border border-slate-850">3rd Qtr ●</button>
    </div>
  </div>

  <div className="flex items-center gap-4">
    {/* Play Button */}
    <button
      onClick={() => setIsPlaying(!isPlaying)}
      className={`h-11 px-5 rounded-lg font-bold text-sm
transition-all duration-200 shadow flex items-center gap-2 active:scale-95
shrink-0 ${
        isPlaying

```

```

        ? 'bg-amber-500 hover:bg-amber-600 text-slate-950'
        : 'bg-indigo-600 hover:bg-indigo-500 text-white'
      }`}
    >
    {isPlaying ? '⏸ Pause Orbit' : '▶ Play Orbit'}
  </button>

  {/* Manual Angle Slider */}
  <input
    type="range"
    min="0"
    max="359"
    value={angle}
    onChange={(e) => setAngle(parseInt(e.target.value))}
    className="w-full h-2 rounded-lg bg-slate-800
appearance-none cursor-pointer accent-indigo-500"
  />
</div>
</div>
</div>

{/* Right Side: The Perspective View From Earth (Canvas Grid
Col 5) */}
<div className="lg:col-span-5 flex flex-col gap-6">

  {/* Earth View Simulator Graphic */}
  <div className="bg-slate-900 p-5 rounded-2xl border
border-slate-800 shadow-xl relative overflow-hidden flex-1 flex flex-col
justify-between">

    {/* Twinkly Star Background Decoration */}
    <div className="absolute inset-0
bg-[radial-gradient(ellipse_at_center,_var(--tw-gradient-stops))]
from-indigo-950/20 via-transparent to-transparent opacity-80
pointer-events-none" />

```

```

        <div className="absolute top-12 left-10 w-1 h-1 bg-white
rounded-full opacity-60 animate-ping" />
        <div className="absolute top-24 right-16 w-0.5 h-0.5
bg-white rounded-full opacity-40" />
        <div className="absolute bottom-16 left-1/3 w-1.5 h-1.5
bg-white/80 rounded-full opacity-50" />

        <div>
            <h2 className="text-lg font-bold text-slate-200 flex
items-center gap-2 mb-1">
                🌌 View From Earth <span className="text-xs
text-indigo-400">Night Sky Scanner</span>
            </h2>
            <p className="text-xs text-slate-400">
                This is exactly how the Moon appears to us looking up
at the sky tonight!
            </p>
        </div>

        {/* Main Dynamic Phase Moon Component */}
        <div className="my-6 py-4 flex flex-col items-center
justify-center">
            <div className="relative w-44 h-44 rounded-full
bg-slate-950 flex items-center justify-center border-4 border-slate-800
shadow-2xl p-4">
                <svg viewBox="0 0 130 130" className="w-full h-full
drop-shadow-[0_0_20px_rgba(254,240,138,0.25)]">
                    {renderEarthViewMoon()}
                </svg>
            </div>

            <div className="mt-5 text-center">
                <span className="text-3xl animate-bounce duration-1000
inline-block">
                    {currentPhase.icon}
                </span>
            </div>
        </div>

```

```

        <h3 className="text-2xl font-black text-yellow-200
tracking-wide mt-1">
            {currentPhase.name}
        </h3>
        <p className="text-xs font-semibold text-indigo-300
tracking-widest uppercase mt-0.5">
            {currentPhase.progress} Light Fraction
        </p>
    </div>
</div>

    { /* Explanation Card */ }
    <div className="bg-slate-950 p-4 rounded-xl border
border-slate-800">
        <h4 className="text-xs uppercase font-extrabold
text-slate-400 tracking-widest mb-1.5">
            Cosmic Analyst Insights
        </h4>
        <p className="text-xs text-slate-300 leading-relaxed">
            {currentPhase.desc}
        </p>
        <div className="mt-3 pt-3 border-t border-slate-900 flex
justify-between text-[11px] text-slate-400">
            <span>Revolution: <b
className="text-slate-200">Active</b></span>
            <span>Tidal Gravity: <b
className="text-slate-200">1/6th</b></span>
        </div>
    </div>

</div>

    { /* Instant Fact / Myth Buster Bubble */ }
    <div className="bg-indigo-950/40 p-4 rounded-xl border
border-indigo-900/50">

```

```

        <span className="text-xs font-black text-indigo-300
uppercase tracking-widest block mb-1">
            🚫 Myth Buster
        </span>
        <p className="text-xs text-indigo-200 leading-relaxed">
            <b>Misconception:</b> Does Earth's shadow cause regular
Moon phases? <br/>
            <b className="text-emerald-400">No!</b> It only does so
during rare Lunar Eclipses. Normal phases are simply our angle of view of
the Moon's lit side as it orbits us.
        </p>
    </div>

</div>

</div>
)}

{/* --- BEDROOM LAB TAB --- */}
{activeTab === 'diy' && (
    <div className="bg-slate-900 p-6 rounded-2xl border
border-slate-800 shadow-xl max-w-4xl mx-auto">
        <div className="flex items-center gap-3 mb-4">
            <span className="text-4xl">🏠</span>
        </div>
        <h2 className="text-xl font-bold text-slate-100">
            DIY Project: The Bedroom Universe
        </h2>
        <p className="text-sm text-slate-400">
            Bring the Moon's phases inside your room! Complete this
real-life experiment and check steps off to earn XP.
        </p>
    </div>

</div>

<div className="grid grid-cols-1 md:grid-cols-12 gap-6 mt-6">

```



```

    { /* Steps Checklist */ }
    <div className="md:col-span-7 space-y-3.5">
      {diySteps.map((step) => (
        <div
          key={step.id}
          onClick={() => handleDiyCheck(step.id)}
          className={`flex items-start gap-3 p-3.5 rounded-xl
border cursor-pointer transition ${
            step.checked
              ? 'bg-indigo-950/30 border-indigo-500/50
text-indigo-200'
              : 'bg-slate-950 border-slate-850
hover:border-slate-800 text-slate-300'
            }`}
        >
          <input
            type="checkbox"
            checked={step.checked}
            onChange={() => {}} // handled by click container
            className="w-5 h-5 rounded border-slate-800
text-indigo-600 focus:ring-indigo-500 pointer-events-none mt-0.5 shrink-0"
          />
          <div>
            <p className="text-sm font-semibold">{step.text}</p>
            {step.checked && (
              <span className="text-[10px] text-emerald-400
font-extrabold uppercase mt-1 inline-block tracking-wider">
                ✓ Finished (+10 XP)
              </span>
            )}
          </div>
        </div>
      )})}
    </div>

```

```

    { /* Instructions and Warnings */ }
    <div className="md:col-span-5 flex flex-col justify-between
gap-4">
        <div className="bg-slate-950 p-4 rounded-xl border
border-slate-850">
            <h3 className="text-sm font-bold text-yellow-300 mb-2">📦
Materials Required:</h3>
            <ul className="text-xs text-slate-400 space-y-2 list-disc
list-inside">
                <li>A small ball (Tennis ball, Styrofoam, or
Orange)</li>
                <li>A bright single desk lamp or flashlight</li>
                <li>A dark, window-covered room</li>
                <li>Your head (acting as Earth!)</li>
            </ul>
        </div>

        <div className="bg-amber-950/20 p-4 rounded-xl border
border-amber-900/40 text-amber-200">
            <h4 className="text-xs font-black uppercase
tracking-wider mb-1">⚠️ Safety Guardrail</h4>
            <p className="text-[11px] leading-relaxed">
                Make sure to clear toys and cords from your floor
before turning off lights to prevent tripping. Do not stare directly into
bright flashlight beams for extended periods.
            </p>
        </div>

        <div className="bg-indigo-950/40 p-4 rounded-xl border
border-indigo-900/40 text-indigo-200">
            <h4 className="text-xs font-black uppercase
tracking-wider mb-1">💡 Science Discovery</h4>
            <p className="text-[11px] leading-relaxed">
                The flashlight shines in one direction, lighting up
exactly one-half of the ball at all times. When you turn, you see

```

different portions of that lit side. This is precisely how moon phases work in space!

```
        </p>
      </div>
    </div>

  </div>
</div>
)}

{/* --- ASSESSMENT SPACE QUIZ TAB --- */}
{activeTab === 'quiz' && (
  <div className="bg-slate-900 p-6 rounded-2xl border
border-slate-800 shadow-xl max-w-2xl mx-auto">

    {!showQuizResults ? (
      <div>
        {/* Progress bar */}
        <div className="flex justify-between items-center mb-4">
          <span className="text-xs text-slate-400 font-bold">
            Question {currentQuestion + 1} of
{quizQuestions.length}
          </span>
          <span className="text-xs text-indigo-400 font-bold">
            Score: {quizScore}/{quizQuestions.length}
          </span>
        </div>
        <div className="w-full bg-slate-950 h-2 rounded-full mb-6
overflow-hidden">
          <div
            className="bg-indigo-500 h-full transition-all
duration-300"
            style={{ width: `${((currentQuestion + 1) /
quizQuestions.length) * 100}%` }}
          />
        </div>
      </div>
    )}
```

```

    { /* Question */ }
    <h3 className="text-lg font-black text-slate-100 mb-6">
      {quizQuestions[currentQuestion].question}
    </h3>

    { /* Options Grid */ }
    <div className="space-y-3">
      {quizQuestions[currentQuestion].options.map((option, idx)
=> {
        let btnStyle = "border-slate-800 bg-slate-950
text-slate-300 hover:border-slate-700";
        if (selectedAnswer === idx) {
          btnStyle = "border-indigo-500 bg-indigo-950/40
text-indigo-200";
        }
        if (isSubmitted) {
          if (idx ===
quizQuestions[currentQuestion].correctAnswer) {
            btnStyle = "border-emerald-500 bg-emerald-950/30
text-emerald-300";
          } else if (selectedAnswer === idx) {
            btnStyle = "border-rose-500 bg-rose-950/30
text-rose-300";
          } else {
            btnStyle = "border-slate-800 bg-slate-950
opacity-50";
          }
        }
      })}

    return (
      <button
        key={idx}
        disabled={isSubmitted}
        onClick={() => handleQuizAnswer(idx)}

```

```

        className={`w-full text-left p-4 rounded-xl border
text-sm font-semibold transition flex items-center justify-between
${btnStyle}`}
      >
        <span className="flex items-center gap-3">
          <span className="w-6 h-6 rounded bg-slate-800
text-slate-400 flex items-center justify-center text-xs font-bold
font-mono">

            {option.label}
          </span>
          {option.text}
        </span>
        {isSubmitted && idx ===
quizQuestions[currentQuestion].correctAnswer && (
          <span className="text-emerald-400
font-bold">Correct! ✓</span>
        )}
      </button>
    );
  }}}
</div>

{/* Explanation Block */}
{isSubmitted && (
  <div className="mt-5 p-4 bg-slate-950 rounded-xl border
border-slate-850">
    <p className="text-xs text-yellow-300 font-bold
mb-1">👩 Teacher's Explanation:</p>
    <p className="text-xs text-slate-300 leading-relaxed">
      {quizQuestions[currentQuestion].explanation}
    </p>
  </div>
)}

{/* Footer Controls */}
<div className="flex justify-end gap-3 mt-6">

```

```

        {!isSubmitted ? (
          <button
            onClick={submitAnswer}
            disabled={selectedAnswer === null}
            className={`px-5 py-2.5 rounded-lg font-bold text-sm
transition ${
              selectedAnswer === null
                ? 'bg-slate-800 text-slate-500
cursor-not-allowed'
                : 'bg-indigo-600 hover:bg-indigo-500 text-white'
            }`}
          >
            Submit Answer
          </button>
        ) : (
          <button
            onClick={nextQuestion}
            className="px-5 py-2.5 rounded-lg bg-indigo-600
hover:bg-indigo-500 text-white font-bold text-sm transition"
          >
            {currentQuestion < quizQuestions.length - 1 ? "Next
Question →" : "See Results 🏆"}
          </button>
        )}
      </div>
    </div>
  ) : (
    // Quiz Results View
    <div className="text-center py-6">
      <span className="text-6xl mb-4 inline-block">🏆</span>
      <h3 className="text-2xl font-black text-slate-100">Space
Quiz Complete!</h3>
      <p className="text-slate-400 mt-2 text-sm">
        You scored <b className="text-indigo-400">{quizScore}</b>
out of <b>{quizQuestions.length}</b>.
      </p>

```

```

        <div className="my-6 max-w-xs mx-auto bg-slate-950 p-4
rounded-xl border border-slate-850">
            <span className="text-xs text-slate-400 block mb-1">Total
XP Gained</span>
            <span className="text-3xl font-extrabold
text-yellow-400">+{ (quizScore * 30)} XP</span>
        </div>
        <div className="flex justify-center gap-3">
            <button
                onClick={retakeQuiz}
                className="px-5 py-2.5 rounded-lg bg-slate-800
hover:bg-slate-700 text-white font-bold text-sm transition"
            >
                 Retake Quiz
            </button>
            <button
                onClick={() => setActiveTab('simulator')}
                className="px-5 py-2.5 rounded-lg bg-indigo-600
hover:bg-indigo-500 text-white font-bold text-sm transition"
            >
                 Back to Simulator
            </button>
        </div>
    </div>
)}

</div>
)}

{/* --- COSMIC FACTS TAB --- */}
{activeTab === 'facts' && (
    <div className="max-w-4xl mx-auto">
        <h2 className="text-xl font-bold text-slate-100 mb-4 flex
items-center gap-2">
             Intriguing Moon Mysteries
        </h2>
    </div>
)}

```

```

<div className="grid grid-cols-1 md:grid-cols-2 gap-4">

  <div className="bg-slate-900 p-5 rounded-xl border
border-slate-800 shadow-sm">
    <h3 className="text-base font-bold text-yellow-300 mb-2">🌑
Why does the Moon cheat?</h3>
    <p className="text-xs text-slate-300 leading-relaxed">
      The Moon looks like it's glowing beautifully, but its
surface is actually dark-grey and rocky, similar to concrete! It is
terrible at reflecting light, but it looks incredibly bright to us because
everything else in space is pitch black.
    </p>
  </div>

  <div className="bg-slate-900 p-5 rounded-xl border
border-slate-800 shadow-sm">
    <h3 className="text-base font-bold text-yellow-300 mb-2">🐼
The Hidden Face: Tidally Locked</h3>
    <p className="text-xs text-slate-300 leading-relaxed">
      The Moon spins on its axis at the exact same speed that
it orbits Earth. Because of this perfect match, we only ever see one side
of the Moon from our planet. The back side remains a permanent mystery to
ground-based stargazers!
    </p>
  </div>

  <div className="bg-slate-900 p-5 rounded-xl border
border-slate-800 shadow-sm">
    <h3 className="text-base font-bold text-yellow-300 mb-2">🌊
Pulling Earth's Water</h3>
    <p className="text-xs text-slate-300 leading-relaxed">
      The Moon's gravity is strong enough to pull entire oceans
on Earth! As the Moon orbits us, its gravity creates high tides and low
tides on our beaches twice every day.
    </p>
  </div>

```



```

        <div className="bg-slate-900 p-5 rounded-xl border
border-slate-800 shadow-sm">
            <h3 className="text-base font-bold text-yellow-300 mb-2">
Meaning of the Word</h3>
            <p className="text-xs text-slate-300 leading-relaxed">
                Did you know that the word "month" originally came from
the ancient word "moonth"? Early humans used the 29-day cycle of the
Moon's phases as their very first calendars to track crop planting.
            </p>
        </div>

    </div>
</div>
)}

</main>

{/* --- IN-APP DIALOGUE / CELEBRATION MODAL --- */}
{showCelebration && (
    <div className="fixed inset-0 z-[100] flex items-center
justify-center p-4 bg-slate-950/80 backdrop-blur-sm animate-fade-in">
        <div className="bg-slate-900 border border-indigo-500/50 p-6
rounded-2xl max-w-md w-full shadow-2xl text-center transform scale-100
transition-all">
            <span className="text-5xl inline-block mb-3
animate-bounce">★</span>
            <h3 className="text-xl font-black text-slate-100">Quest
Unlocked!</h3>
            <p className="text-sm text-slate-300 mt-2 leading-relaxed">
                {celebrationMessage}
            </p>
            <div className="mt-5 flex justify-center gap-3">
                <button
                    onClick={() => {
                        setShowCelebration(false);

```

```

        // Auto-advance quest if possible
        if (currentQuest < quests.length - 1) {
            setCurrentQuest((prev) => prev + 1);
        }
    }}

    className="px-5 py-2 rounded-lg bg-indigo-600
hover:bg-indigo-500 text-white font-bold text-sm tracking-wide transition
active:scale-95"
    >
        Onward, Cadet! 🚀
    </button>
</div>
</div>
</div>
)}

{/* Footer Branding */}
<footer className="mt-12 py-6 border-t border-slate-900 text-center
text-xs text-slate-500">
    <div className="max-w-7xl mx-auto px-4 flex flex-col sm:flex-row
items-center justify-between gap-3">
        <p>© 2026 Cosmic EdTech. Interactive Moon Module designed for
Class 5 Science.</p>
        <div className="flex gap-4">
            <span className="hover:text-slate-400 cursor-pointer">Terms of
Use</span>
            <span className="hover:text-slate-400 cursor-pointer">Teacher
Support</span>
        </div>
    </div>
</footer>
</div>
);
}

```